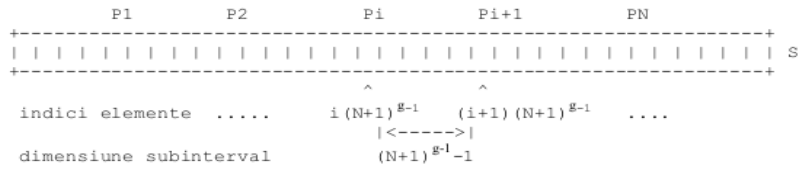
1 Cautare paralela (curs 5)

În cazul unui sistem CREW, difuzarea valorii lui **x** se poate face într-un singur pas. Mai mult, se poate adopta o politică de căutare diferită, obţinându-se o performanţă mai bună pentru timpul de execuţie.

Algoritmul se bazează pe împărţirea, în fiecare etapă, a secvenţei în care se face căutarea în N+1 subsecvenţe de aceeaşi lungime. Fiecare din cele N procesoare inspectează o valoare **s** aflată în poziţia dintre două secvenţe adiacente. Dacă x<s atunci toate elementele aflate la dreapta lui **s** sunt ignorate în pasul următor. Similar, dacă x>s sunt ignorate elementele mai mici ca **s**. Următorul subinterval de căutare se obţine făcand intersecţia subintervalelor păstrate de diferitele procesoare. Deoarece fiecare etapă se aplică unei secvenţe de dimensiune 1/(N+1) din secvenţa precedentă, sunt necesare $O(\log_{N+1}(n+1))$ etape.




```
/* Initializarea indicilor secventei de cautat */
int q = 1;                          /* limita stg interval crt */
int r = n;                          /* limita dr interval crt */
/* Initializarea rezultatului k si a numarului de pasi g */
int x, k = 0;              /* x = numar cautat  k = pozitia lui in sir */
int g = sup(log(n+1)/log(N+1))
enum sens {stg,dr};
sens c[0:N+1];             /*c[i] – x este la stg / dr pozitiei procesului i*/
c[0] = dr; c[N+1] = stg;
int j[1:N];               /* j[i] = pozitia din sir inspectata de proces i*/
process P[i = 1 to N]{
/* gaseste pozitia k unde se afla valoarea x */
    while (q<=r and k=0){          /* subsir exista si valoare negasita */
      j[i] = (q-1) + i*(N+1)^(g-1);
/* P[i] compara x cu s[j[i]] si determina partea de secv acceptata */
      if (j[i] <= r)               /* pozitie in interior sir */
         if (s[j[i]] == x) k = j[i];       /* pozitie gasita */
         else if (s[j[i]] > x) c[i] = stg;
         else c[i] = dr;
      else {j[i] = r+1;           /*j[i] indica o pozitie afara din sir*/
            c[i] = stg;}
/* calculeaza indicii subsecventei urmatoare */
/* doar un proces poate face modificarea intervalului de cautare*/

         if (c[i] <> c[i-1]){
              r = j[i] – 1;
              if (i>1) q = j[i-1] + 1;}
   /* doar procesul N executa operatia urmatoare */
         if (i==N and c[i]<>c[i+1]) q = j[i] +1;
         g := g-1;
      }        /*end while*/
   }           /* end process */
```

2. Inmultirea a 2 matrici in paralel. (note curs 2a, pag 6)

Aici sunt n^2 procese, fiecare având constantele locale *i* şi *j*, precum şi variabila locală *sum*. *k* este contorul unui cuantificator şi se modifică doar prin regulile proprii cuantificatorilor. În declararea proceselor se poate folosi şi următoarea formă, în care se asociază **nume** proceselor paralele conţinute de un program.

Exemplu

```
real a[1:n,1:n], b[1:n,1:n], c[1:n,1:n];

process Prod (i = 1 to n, j= 1 to n) {
      real sum = 0;
      for [k = 1 to n]
          sum=sum+a[i,k]*b[k,j];
      c[i,j] = sum;
}
```

3. Prioritate cititori (curs 4)

Procesul cititor

```
sem rw = 1;
sem mutexR = 1;    /*accesul la nr este exclusiv ptr. cititori*/
int nr = 0;
process Cititor [i = 1 to m]{
while (true){
   P(mutexR);     /*necesara pentru accesul exclusiv la nr*/
      nr = nr+1;
      if (nr == 1) P(rw);   /*daca prim cititor*/
   V(mutexR);
   citeste din resursa comuna;
   P(mutexR)
      nr = nr-1;
      if (nr == 0) V(rw);     /*daca ultim cititor*/
   V(mutexR);}
}
```

Procesul scriitor

```
sem rw = 1, mutexR = 1;
int nr = 0;

process Scriitor [j = 1 to n]{
while (true){
   P(rw);
   scrie in resursa comuna;
   V(rw);}
}
```

4. Cititori si scriitori cu split binary semaphore - curs 4

Pentru specificarea conditiilor se definesc

```
int nr = 0;     /*nr cititori care folosesc resursa */
int nw = 0;     /*nr scriitori care folosesc resursa */
int dr = 0;     /*nr cititori intarziati */
int dw = 0;     /*nr scriitori intarziati */
```

conditia 1	cititor intarziat daca	nw>0 OR dw>0
conditia 2	cititor trezit daca	nw=0 AND dw=0
conditia 3	scriitor intarziat daca	nr>0 OR nw>0

Pentru controlul accesului definim semafoarele

```
sem r = 0;      /*intarzie cititori daca nw>0 OR dw>0*/
sem w = 0;      /*intarzie scriitori pana cand nw=0 AND nr=0*/
```

```
process Cititor [i = 1 to m]{
 while (true){
   P(e);
   if (nw>0 or dw>0){
     dr = dr+1; V(e); P(r);        /*intarzie cititorul*/
   }
   nr = nr+1;
   if (dr>0) {dr = dr-1; V(r);}      /*trezeste in alt cititor*/
   else if (dr==0) V(e);
   citeste din resursa comuna;
   P(e);
   nr = nr-1;
   if (nr == 0 and dw>0){
         dw = dw-1; V(w);}          /*trezeste un scriitor*/
   else if (nr>0 or dw==0) V(e);
   sectiune necritica;
 }
}
 process Scriitor [j = 1 to n]{
  while (true){
    P(e);
    if (nr>0 or nw>0){
       dw = dw+1; V(e); P(w);}     /*intarzie scriitorul*/
    nw = nw+1;
    V(e);
    scrie in resursa comuna;
    P(e);
    nw = nw-1;
    if (dr>0 and dw==0){
          dr = dr-1; V(r);}              /*trezeste un cititor*/
    else if (dw>0){dw = dw-1; V(w);}     /*trezeste un scriitor*/
    else if (dr==0 and dw==0) V(e);
    sectiune necritica;
  }
 }
```

Semafoarele **s** si **w** sunt folosite in sectiunile decide din procesele Cititor si Scriitor

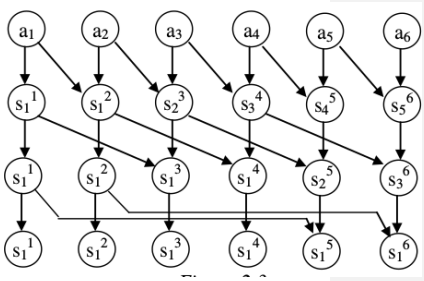
```
process Cititor [i = 1 to m]{
   while (true){
       decide intrare in sectiunea critica sau intarzierea;
       citeste din resursa comuna;
       decide daca si cine intra in sectiunea critica;
       sectiune necritica;
   }
}

process Scriitor [j = 1 to n]{
   while (true){
       decide intrare in sectiunea critica sau intarzierea;
       scrie in resursa comuna;
       decide daca si cine intra in sectiunea critica;
       sectiune necritica;
   }
}
```

I

5. Sume prefix - notite 2.c

```
int a[1:n], temp[1:n];
process suma [k=1 to n]{
     for [j = 1 to sup(log2 n)]{
          temp[k] = a[k];
          barrier;
          if (k-2^(j-1)>=1) a[k] = temp[k-2^(j-1)] + a[k];
          barrier;
     }
}
```



```
int a[1:n], temp[1:n];
process suma[k:1..n]{
   int d = 1;
   while (d<n){
     temp[k] = a[k];
     barrier;
     if (k-d>=1) a[k] = temp[k-d] + a[k];
     barrier;
     d = 2*d;
   }
}
```

Asta e fara logaritmi si ridicari la putere ^, dar e acelasi algoritm.
Simd e fara bariere

Descriere algoritmi SIMD

Având în vedere că în algoritmii SIMD toate acţiunile sunt sincronizate (deci orice acţiune ar trebui urmată de o barieră), în descrierea lor eliminăm sincronizările prin bariere. Calculul sumelor prefix poate fi reprezentat astfel (varianta SIMD):

```
int a[1:n];
process suma [k=1 to n]{
   int temp;            /*temp este locala procesului k*/
   for [j = 1 to sup(log2 n)]{
      if (k-2^(j-1)>=1){ temp = a[k-2^(j-1)];
                         a[k] = temp + a[k];
      }
   }
```

Commented [1]: Pentru SIMD

Commented [2]: yes

Commented [3]: ms. nu stiu cum sa scriu in doc intre imagini de aia am scris aici. stii tu?

Commented [4]: nush. am dat enter. dar imi place si asa ca e discutie si putem sa mai vorbim de chestii.

Commented [5]: asa e cmf?

6. Sfarsitul unei liste inlantuite - notite 2.c

```
int leg[1:n], end[1:n];
process Afla [i=1 to n){
   int nou, d=1;
   end[i] = leg[i];
   barrier;
   while (d<n){
     nou = 0;        //tampon pentru legatura urmatoare
     if (end[i]<>0 and end[end[i]]<>0) nou=end[end[i]];
     barrier;
     if (nou<>0) end[i]=nou;
     barrier;
     d = 2*d
   }
}
```

Proprietati fundamentale ale algoritmilor unda

- **terminare** – fiecare **calcul** este **finit**
- **decizie** – fiecare calcul contine cel putin un **eveniment de decizie** (decide)
- **dependenta** – in fiecare calcul, fiecare ***decide*** **este precedat** cauzal de **un eveniment in fiecare proces**

Alte caractreristici

Centralizare

- algoritmi centralizati – un initiator
- algoritmi descentralizati – set arbitrar de initiatori

Topologie

- regulata: inel, arbore, clica etc.
- fixa (nu se produc modificari topologice)
- nedirectionata (canale bi-directionale); exceptiile mentionate explicit
- conectata (exista o cale intre oricare doua procese)

8. Algoritmi de unda - curs 9

Algoritmul inel

fiecare proces are un vecin dedicat, Urm
transmiterea foloseste **adresarea prin directie** (Urm, Prec)
toate canalele selectate prin Urm formeaza un ciclu Hamiltonian
algoritmul este **centralizat**:

- initiatorul trimite un token (jeton) care este pasat de fiecare proces de a lungul inelului pana ajunge inapoi la initiator;
- initiatorul ia apoi decizia

```
chan token[1:n] (tok_type tok);
```

```
/*initiator*/
process P[I]{
tok_type tok;
send token[Urm](tok);
receive token[I](tok);
decide
```

```
/*non-initiatori*/
process P[k=1 to n,
k<>I]{
tok_type tok;
receive token[k](tok);
send token[Urm](tok);
```

Număr de mesaje = n
Timp = n

*** Arbore** - descentralizat - mai multe procese initiaza(adica frunzele)

```
  chan ch[1:n] (int id, tok_type tok);
      /* fiecare proces are un canal propriu */

  process Proc[p = 1 to n]{
    int r = numar_vecini_p;
    bool Vecini[1:n] = vecinii_lui_p;
        /* Vecini[i]=true daca i este vecin lui p */
    bool rec[1:n] = ([n] false);
        /* rec[i]=true daca p a primit mesaj de la i */
    tok_type tok; int id, q0;

    while (r>1){receive ch[p](id,tok);
           rec[id] = true; r = r-1};}
  /* de la un singur vecin, q0, nu s-a primit mesaj */
    afla q0 pentru care: Vecini[q0] and NOT rec[q0];
    send ch[q0](p, tok);
    receive ch[p](q0, tok); rec[q0] := true;
    decide;
    /* informeaza celelalte procese despre decizie */
    /* for [q = 1 to n st Vecini[q] and q<>q0]
                                send ch[q](p,tok); */
  }
```

Număr de mesaje = n Timp = O(D)
Fiecare proces trimite un mesaj și sunt n procese.
Mesajul trimis de o frunză atinge orice alt nod într-un număr de pași mai mic decât diametrul D al arborelui, fiecare pas fiind făcut într-o unitate de timp.
Algoritmul de bază se termină când unul din procese decide.

- **Ecou**

```
chan ch[1:n] (int id, tok_type tok);
const int I = id_initiator;
process Proc(I){
  int r = numar_vecini_I;
  bool Vecini[1:n] = vecinii_lui_I;
  tok_type tok;
  int id;
  for [i=1 to n st Vecin[i]]
    send ch[i](I, tok);
  while (r>0){receive ch[I](id, tok);
              r = r-1; }
  decide;
}

process Proc[p=1 to n, p<>I){
  int r = numar_vecini_p;
  bool Vecini[1:n] = vecinii_lui_p;
  tok_type tok;


 int id, parinte;
 receive ch[p](parinte, tok); r = r-1;
 for [q=1 to n st Vecini[q] and q<>parinte]
   send ch[q](p, tok);
 while (r>0){receive ch[p](id, tok);
             r = r-1};
 send ch[parinte](p, tok);
}
```

Algoritmul fazelor

Doua categorii de procese:

- Initiatori – pornesc cu trimiterea unui mesaj fiecarui out-vecin
- Ne-initiatori – asteapta primirea unui prim mesaj si apoi trimit un mesaj fiecarui out-vecin

```
chan ch[1:n] (int id, tok_type tok);
const int D = diametrul_retelei;
process Proc [p = 1 to n]{
  bool in[1:n] = in-vecinii_lui_p;
  /* in[i]==true daca i invecin */
  bool out[1:n] = out-vecinii_lui_p;
  /* out[i]==true daca i outvecin */
  int rec[1:n] = ([n] 0);
  /* rec[q] = numar mesaje primite de la q */
  int sent = 0;
  /* nr. mesaje transmise out-vecinilor */
  tok_type tok;
  int id;

  if (p este initiator){
    for [q = 1 to n st out[q]]
      send ch[q](p, tok);
    sent = sent+1;}
  while (min(rec) < D){
      /*min(rec) = min(rec[q]),q=1..n and in[q])*/
    receive ch[p](id, tok);
    rec[id] = rec[id]+1;
    if (min (rec) >= sent and sent < D){
      for [q=1 to n st out[q]]
        send ch[q](p,tok);
      sent = sent+1;}
  }
  decide;
}
```

Mesaje **= 2D|E|**
fiecare proces trimite D mesaje si sunt 2|E| canale uni-directionale

Timp **= 2D** sunt D faze, in fiecare avem **receive** si **send** (ops. send se deruleaza in paralel)

9. Problema producător-consumator. - notite 2.a

```
int buf, p=0, c=0;
     /*p = numarul de valori transmise in buf*/
     /*c = numarul de valori preluate din buf*/
process producer{
   int a [1:n];
   while (p<n){
      <await (p==c);>
      buf = a[p+1];
      p = p+1;
   }
```

```
process consumer{
   int b [1:n];
   while (c<n){
      <await (p>c);>
      b[c+1] = buf;
      c = c+1;
   }
}
```

Se consideră un producător și un consumator care comunică printr-un singur tampon partajat. Producătorul generează și pune câte o valoare în tampon, iar consumatorul preia câte o valoare din tampon. Presupunem că tamponul poate păstra o singură valoare.

În soluția prezentată aici, considerăm că producătorul păstrează într-un tablou **a[1:n]** valorile furnizate; consumatorul pune în tabloul **b[1:n]** valorile preluate din tampon. Soluția trebuie să asigure că valorile ajung în tabloul **b** în același număr și în aceeași ordine ca în **a**.

Multiple producers multiple consumers - notite 4

```
typeT buf[1:k];
sem gol = k, plin = 0;        //sem. generalizate
int prim = 1, ultim = 1;
sem mutexP = 1, mutexC = 1;

process Producator[i = 1 to M]{
     typeT v;
     while (true){
          v = produce ();
          P(gol);
               P(mutexP);
               buf[ultim]=v; ultim = ultim mod k + 1;
               V(mutexP);
          V(plin);
     }
}

process Consumator[i = 1 to N]{
    typeT w;
    while (true){
         P(plin);
              P(mutexC);
              w = buf[prim]; prim = prim mod k + 1;
              V(mutexC);
         V(gol);
         consuma (w);
     }
}
```

10. Secventa critica pt calcul distribuit(requesturi si replyuri) - ceva similar cred? - curs 8
(excluderea mutuala distribuita)

```
enum fel {V, P, ack};
chan opsem[1:n](int transm, fel op, int timp);
chan start[1:n](int timp);

process Utiliz [i = 1 to n]{
int cl = 0;                    /*ceas logic*/
int ts;                        /*timestamp mesaj*/
   ...
   cl = cl+1;
   broadcast opsem(i, P, cl);     /*operatia P*/
   receive start[i](ts);          /*asteapta permisiune*/
   cl = max(cl, ts) + 1;
       sectiune critica;
   cl = cl+1;
   broadcast opsem(i, V, cl);     /*operatia V*/
       sectiune ne-critica;
   …
   }

   process Ajutor [i = 1 to n){
   typedef struct {int transm, fel k, int ts} queue;
   queue qm [lmax];     /*coada ordonata dupa timestamp tm*/
   int cl = 0;
   int sem = valoare_initiala;
   int transm;
   fel k;
   int ts;

   while (true) {
     receive opsem[i](transm, k, ts);
     cl = max(cl, ts) + 1;
     if (k==P or k==V) {
       insereaza (transm,k,ts) in loc corespunzator in qm;
       cl = cl+1; broadcast opsem(i, ack, cl);}
     else if (k==ack) {
       inregistreaza primirea unui ack;
       for [mesajele V complet confirmate ] {
             scoate mesaj din qm;
             sem = sem+1;}
       for [mesajele P complet confirmate st sem>0] {
             scoate mesaj (transm, k, ts) din qm;
             sem = sem-1;
             if (transm=i) {cl := cl+1; send start[i](cl); }
     }
   }    /*end process*/
```

Commented [6]: if (transm=i) cred ca ar trebui sa fie if (transm nu e Ajutor) ca sa aiba sens.

Commented [7]: In schema din curs un proces Ajutor[i] primeste pe canalul opsem de la orice Utilizator sau Ajutor, dar trimite pe canalul send doar lui Utilizator[i]. Mai mult nu stiu

18. Alegere lider - curs 10

Arbore

ws = wakeup send - boolean, fiecare proces trimite o singura data wakeup
wr = wakeup receive;

```
chan ch[1:n](int id_transm, int id_mic);
chan wakeup[1:n]();
process Proc(p = 1 to n){
 bool ws = false;
 int wr = 0;
 int r = numar_vecini_p;
 bool Vecini[1:n] = vecinii_lui_p;
 bool rec [1:n] = ([n]*false);
 int V = p;
 enum state {sleep, leader, lost}
 state stare = sleep;
 int id;


/* aici incepe faza wake-up */
 if (p este initiator){
     ws = true;
     for [q = 1 to r st Vecini[q]]
        send wakeup[q]();
 }
 while (wr < numar_vecini_p){
     receive wakeup[p](); wr = wr+1;
     if (NOT ws) ->
     ws = true;
     for [q = 1 to n st Vecini[q]]
      send wakeup[q]();
 }

/* aici incepe algoritmul tree */
 do (r>1){
  receive ch[p](q,id);
    rec[q] = true; r = r-1;
    V = min(V, id);
 }

/* de la un singur vecin, q0 nu s-a primit mesaj */
  afla q0: Vecini[q0] and NOT rec[q0];
  send ch[q0](p, V);
  receive ch[p](q0, id);
  V = min (V, id);
  if (V=p) stare = leader;
  else stare = lost;
/*transmite vecinilor decizia*/
  for [q = 1 to n st Vecini[q] and q<>q0]
     send ch[q](p,V);
}
```

Numar mesaje = O(N)

Pe fiecare canal se trimit două mesaje *wakeup* și două *token*-uri ceeace înseamnă un total de 4N-4 mesaje.

Timp = O(D)

În D pași după ce primul proces pornește algoritmul, fiecare proces a trimis mesajele *wakeup*. Ca urmare, în D+1 pași fiecare proces a pornit unda. Prima decizie se ia la D pași după pornire, iar ultima după alți D pași. Rezultă totalul de 3D+1 pași.

Inel - Algoritmul LeLann

Algoritmul realizează următoarele acțiuni:

1. Fiecare proces transmite în inel un mesaj care conține identificatorul său
2. Fiecare proces colectează identificatorii celorlalte procese
3. Fiecare proces calculează maximul.
4. Procesul al cărui număr este egal cu maximul devine lider.

Complexitate $O(n^2)$ mesaje comunicate - fiecare proces transmite un mesaj care trece pe la toate celelalte.

```
chan ch[Ids](token tok, int id_mare);
process Proc[p = 1 to n]{
typedef SOP {multime identificatori};
SOP List = {p};
enum state (candidate,leader,lost);
state stare = candidate;
send ch[Urm](tok, p);
receive ch[p](tok, q);
while (q<>p){ List = List U {q};
          send ch[Urm](tok, q);
          receive ch[p](tok, q);
          }
if (p == max(List)) stare = leader;
else stare = lost;
}
```

Commented [8]: de la ce vine SOP?

Commented [9]: _Marked as resolved_

Commented [10]: _Re-opened_

Inel - LeLann-Chang-Robert

```
chan ch[1:n](token tok, int id_mare);
process Proc[p = 1 to n]{
enum state (candidate, leader, lost);
state stare = candidate;
send ch[Urm](tok, p);
while (state <> leader){
  receive ch[p](tok, q);
  if (q==p) stare = leader;
  else if (q>p){
    if (stare == candidate)stare = lost;
    send ch[Urm](tok, q);
    }
 }
}
```

Timp O(n)
Numar mesaje: worst case = n(n + 1) / 2; medium = n log n

Algoritmul Hirschberg-Sinclair

- Prea complicat

19 Stabilirea topologiei - curs 11

Algoritmul pulsatiilor

```
chan topo[1:N] (tip_top);
process Nod[p=1 to N]{
  tip_leg  leg = (vecinii_lui_p);
  tip_leg activ = leg;        {vecinii activi}
  tip_top top = ([N*N]FALSE);
  bool gata = FALSE;
  int transm; bool qgata; tip-top top_nou;
  top[p,1:N] = leg;      {actualizeaza vecini}

 while (NOT gata){
  /*transmite topologia curenta tuturor vecinilor*/
   for [q =1 to N st leg[q]]
     send topo[q](p,FALSE,top);
   /*receptioneaza topologiile de la vecini*/
   for [q =1 to N st leg[q]]{
     receive topo[p](transm,qgata,top_nou);
     top = top OR top_nou;
     if (qgata) activ[transm]=FALSE;
   }


    if (toate liniile din top au un element TRUE)
        gata = TRUE;
}

  /*transmite top vecinilor activi in ultimul rund*/
  for [q =1 to N st activ[q]]
      send topo[q](p,TRUE,top);
  for [q =1 to N st activ[q]]
      receive topo[p](transm,qgata,top_nou);
```

Complexitate: 2m*N*(D+1)

Mesaje de sondaj cu ecou

```
const int sursa=i;      /*index initiator*/
enum fel {sonda, ecou};
typedef tip_leg bool[1:n];
typedef tip_top bool[1:n, 1:n];
chan sonda-ecou[1:n](fel, int transm,   tip_top top);
chan ecou_final(tip_top top);

process Initiator{
 tip_top top;
 send sonda-ecou[sursa](sonda, sursa, Φ);
 receive ecou_final (top);
}

process Nod[p=1 to n){
  tip_leg leg = vecinii_lui_p;
  tip_top top = ([n*n] false);
  top[p, 1:n] = leg;
  tip_top top_nou;

 int prim;
    /*nodul de la care s-a primit prima sonda*/
 fel k; int transm;
 int nr_ecouri = numar_vecini - 1;
 receive sonda-ecou[p](k, prim, top_nou);
 /*transmite sondaje celorlalte noduri vecine, copiii
   lui p*/
 for [q := 1 to n st leg[q] and q<>prim]
    send sonda-ecou[q](sonda, p, Φ);
                            /*Φ este topo. nula*/


/*primeste ecourile si fa reuniunea lor, sau sonde si
   ignora*/
 while (nr_ecouri>0){
   receive sonda-ecou[p](k, transm, top_nou);
   if (k==sonda)
   send sonda-ecou[transm](ecou,p,Φ);
 else if (k=ecou){
    top = top or top_nou;
    nr_ecouri = nr_ecouri-1;}
 }
 if (p=sursa) send ecou_final(top);
       else send sonda-ecou[prim](ecou, p, top);
```

20. Terminare - mai mult in curs 12

Cu Jeton - inel (reguli)

acţiunile lui P(1) cînd devine liber prima dată:

```
culoare[1] = albastru;
jeton = 0;
send ch[2](jeton);
```

acţiunile lui P(i:1..n) la recepţia unui mesaj obişnuit:

```
culoare[i] = rosu;
```

acţiunile lui P(i:2..n) la primirea unui jeton:

```
culoare[i] = albastru;
jeton = jeton +1;
send ch[(i mod n) + 1](jeton);
```

acţiunile lui P(1) la recepţia jetonului:

```
if(culoare[1]== albastru){anunţă terminarea; stop}
culoare[1] = albastru;
jeton = 0;
send ch[2](jeton);
```

Cu jeton - general

acţiunile lui P[i=1 to n] la primirea unui mesaj uzual:

```
culoare[i] = rosu;
```

acţiunile lui P[i=1 to n] la recepţia jetonului:

```
if (jeton == nc) {anunţă terminarea; stop;}
if (culoare[i] == rosu){culoare[i] = albastru;
                        jeton = 0}
else jeton = jeton +1;    /*culoare[i] == albastru*/
setează j la canalul următor din ciclul c;
send ch[j](jeton);
```

Detecţia terminării folosind confirmările mesajelor

- dacă pentru fiecare mesaj transmis pe un canal, procesul primeşte de la receptor un semnal de confirmare, atunci canalul respectiv este gol
- existenta unui proces sursa, care nu are legături de intrare şi de la care orice alt proces este accesibil pe o cale oarecare din graf

Dacă procesele ar fi organizate într-o reţea cu topologie arborescentă, algoritmul de semnalare ar fi simplu: cînd un proces frunză se termină, el anunţă părintele său. Cînd un nod oarecare al arborelui se termină, el aşteaptă notificarea terminării de la toţi fiii săi şi apoi îşi anunţă părintele. Notificările se propagă astfel pînă la procesul sursa, aflat în rădăcina arborelui.

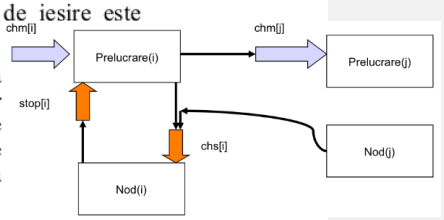
Acest algoritm poate fi extins la un **graf dirijat aciclic** de procese. Fiecărei legături i se asociază un număr, reprezentînd diferenţa dintre numărul de mesaje de date transmise şi numărul de semnale de confirmare primite pe acea legătură. Numim **deficit** această diferenţă. Cînd un nod doreşte să termine, el aşteaptă primirea pe legăturile de ieşire a datelor, a unor semnale care reduc deficitele la zero, după care trimite pe fiecare legătură de intrare a datelor un număr de semnale egal cu deficitul.

Algoritmul **Dijkstra-Scholten** rezolvă problema în **cazul general**, în care sunt permise cicluri în graful proceselor. Rezolvarea se face prin generarea unui **arbore de acoperire**, pe parcursul transferului mesajelor de date: fiecare proces păstrează o variabilă **prim** a cărei valoare este egală cu identificatorul procesului de la care s-a recepţionat primul mesaj de date.

Rezolvarea implică trei faze:
- trimiterea semnalelor pe toate legăturile de intrare, cu excepţia celei de la **prim**;
- recepţia semnalelor pe toate legăturile de ieşire;
- transmiterea semnalelor spre **prim**.

Pentru recepţia semnalelor pe legăturile de ieşire, se poate ţine o evidenţă globală, deoarece este important ca deficitele legăturilor de ieşire să se anuleze, neavînd importanţă identitatea legăturilor pe care se primesc semnale. Ca urmare, o singură variabilă **nsemnale** care să ţină minte suma deficitelor legăturilor de iesire este suficientă.

Algoritmul fiecărui nod este repartizat în două procese: Prelucrare[i] realizează transformarea datelor; Nod[i] realizează gestiunea mesajelor şi a semnalelor schimbate cu celelalte noduri. Prelucrare[i] are un canal chm[i] prin care primeşte mesajele de date. Nod[i] are un canal chs[i] prin care primeşte semnale de la celelalte procese, anunţuri de transmitere/recepţie de mesaje de date şi cereri de terminare de la Prelucrare[i]. Răspunsul la cererile de terminare este dat pe canalele stop[i].



Topologia (virtuală a) reţelei de procese este păstrată în tabloul **in**. Fiecare element al acestuia înregistrează existenţa legăturii (cîmpul **exista**) şi deficitul (cîmpul **deficit**).

Procesul **Prelucrare [i]** are o funcţionare ciclică. După recepţia unui mesaj de date, el realizează prelucrările cerute şi, eventual, transmite mesaje de date altor procese. El însuşi poate primi mesaje de date de la celelalte procese, pe canalele de intrare. Recepţiile şi transmiterile de mesaje sunt anunţate procesului Nod[i]. La sfîrşitul prelucrării, procesul trimite lui Nod[i] o cerere de terminare şi aşteaptă recepţia unei confirmări pe un canal special stop[i].

Procesul **Nod[i]** are o funcţionare ciclică. La fiecare iteraţie, el tratează unul din mesajele primite de la procesul de prelucrare local sau de la celelalte procese Nod. Pentru un anunţ de recepţie de date, procesul actualizează deficitul legăturii de intrare şi, eventual, **prim**. Un anunţ de transmitere determină incrementarea variabilei **nsemnale**. Un semnal determină decrementarea variabilei **nsemnale**. În fine, o cerere de terminare provoacă transmiterea de semnale pe legăturile de intrare. Dacă **nsemnale** este nul, atunci se transmit semnale către nodul părinte din arborele de acoperire (prim) şi se confirmă terminarea. Altfel, se transmite o infirmare a terminării.

```
enum fel{semnal, transmitere, receptie, terminare};
typedef struct {bool exista;
                int deficit} arc;
chan chs[1:n](fel op, int id);
chan chm[1:n](int id, tip_date date);
chan stop[1:n](bool);

process Nod[i=1 to n]{
 arc in[1:n];
 int nsemnale =0;
 int prim =0;
 int id; fel k;
 while (true){
   receive chs[i](k, id);
   if (k == receptie){
       if (prim == 0) prim = id;
       in[id].deficit = in[id].deficit+1;
   else if (k == transmitere)
       nsemnale = nsemnale+1;
   else if (k == semnal)
       nsemnale = nsemnale-1;
   else if (k = terminare)
/* trimite semnale pentru toate legăturile de intrare,
         mai puţin prim */
    for [j=1 to n st in[j].exista and j<>prim]
        while (in[j].deficit>0){
           send chs[j](semnal, i);
           in[j].deficit = in[j].deficit-1;
        }
          /* daca mai sunt semnale de primit */
    if (nsemnale<>0) send stop[i](false);

    else{     /*nsemnale==0*/
          // trimite semnale procesului prim
        if (i<>sursa and prim<>0)
          while (in[prim].deficit>0){
           send chs[prim](semnal, i);
           in[prim].deficit=in[prim].deficit-1;
          }
          /* procesul prelucrare[i] se poate opri */
         send stop[i](true);
    }   /*end if*/
  }     /*end while*/
}       /*end process*/

process Prelucrare(i=1 to n){
  tip_date data; int id, bool term;
  initializari;
  term = false;
  while (not term){
     ....
     receive chm[i](id, data);
     send chs[i](receptie, id);
     realizeaza prelucrare
     ....
     send chm[j](i, data);
     send chs[i](transmitere, j);
     ...
     if (decizie terminare) term := true;
  }
  term =false;
  while (not term){
     send chs[i](terminare, i);
     receive stop[i](term);
  }
  stop;
```

Detectia cu marcaje

În cea de a doua variantă, procesele transmit mesaje de marcaj al sfîrşitului comunicaţiei, urmînd să primească o confirmare doar pentru aceste mesaje, nu şi pentru cele de date. Cînd un proces decide terminarea, el aşteaptă marcaje pe toate legăturile de intrare şi transmite un marcaj pe fiecare legătură de ieşire. Îndată ce procesul a primit marcaje, el trimite confirmări pe legăturile de intrare corespunzătoare, mai puţin pe prima legătură. Apoi, el aşteaptă semnale pe legăturile de ieşire. Cînd le primeşte, transmite un semnal pe prima legătură.

Structurile utilizate pentru cele două categorii de arce au aceeaşi sintaxă. Ele conţin câmpurile **exista**, **activ** şi **marcaj**. De data aceasta, fiecare **arc de intrare** înregistrează dacă:
- s-a primit un mesaj (se pune **activ** pe true);
- s-a primit un marcaj (se pune **marcaj** pe true);
- s-a transmis un semnal de confirmare (se pun ambele pe false).

Similar, pentru fiecare **arc de ieşire**, se înregistrează dacă:
- s-a transmis un mesaj (se pune **activ** pe true);
- s-a transmis un marcaj (se pune **marcaj** pe true);
- s-a primit un semnal de confirmare (se pun ambele pe false).

Huang

Modelul se bazează pe următoarele:
– procesele pot fi active sau libere (idle)
– doar procesele active transmit mesaje
– procesele libere pot deveni active la recepţia unui mesaj de "calcul"
– procesele active pot deveni libere în price moment
– **terminarea**: toate procesele sunt libere şi nu există mesaje de "calcul" în tranzit

Ideea este următoarea:
• Un agent de control are iniţial ponderea 1
• Toate celelalte procese sunt libere şi au iniţial ponderea 0
• Calculul porneşte când agentul de control transmite un mesaj de "calcul" unui proces
• Un proces liber devine activ la recepţia unui mesaj de "calcul"
Fie:
• B(DW) – mesajul de "calcul" cu ponderea DW. Mesajul poate fi trimis doar de agentul de control sau de de un proces activ
• C(DW) – mesaj de control cu ponderea DW, transmis de un proces activ agentului de control când devine liber
• Ponderea curentă a unui proces este W

Algoritmul are următoarele reguli:

1. **Send B(DW)**:
• Găseşte W1, W2 astfel încât W1 > 0, W2 > 0, W1 + W2 = W
• Pune W = W1 şi transmite B(W2)

2. **Receive B(DW)**:
• W += DW;
• if liber -> devine activ

3. **Send C(DW)**:
• send C(W) agentului de control
• devine liber

4. **Receive C(DW)**:
• W += DW
• if W = 1, declară “terminarea”

1. **Cautare paralela**
2. **Inmultirea paralela a matricilor**
3. **Prioritate pt cititori**
4. **Cititori si scriitori cu split binary semaphore**
5. **Sume/Produs prefix cu MIMD**
6. **Sfarsitul listei**
7.
8. **Algoritmi de unda**
9. **Producatori Consumatori**
10. **Secventa critica pt calcul distribuit(requesturi si replyuri)**
11. **Terminare cu marcaje**
12. Bizantini - pb generalilor bizantini - descriere, sol cu msg, teoreme, pseudocod, complexitate, exemple cu 3 generali
13. **Stabilirea topologiei cu sondaje de ecou**
14. Huang
15. Complexitate paralela :)
16. **Max dintr-un graf - adica lider**
17. **N procese-graf, sa se afle topologia(care sunt parintii sau copiii) - la algoritmii unda; in cursul stabilire topologie se face matricea de adiacenta top, nu parinti copii**
18. **Alegere lider topologiei**
19. **Stabilirea topologiei**
20. **Terminare**

Terminare cu marcaje - cod

```
const tip_date marcaj = mesaj-special-sfirsit-date;
enum fel(semnal, transmitere, receptie,
                 receptie-marcaj, terminare);
typedef struct (exista: bool;
```

```
                activ: bool; /*initial false*/
                marcaj: bool;)arc; /*initial false*/
chan chs[1:n](fel op, int id);
chan chm[1:n](int id, tip_date date);
chan stop[1:n](bool);

process Prelucrare[i=1 to n){
        tip_date data, int id, bool term;
        initializari;
        term = false;
        while (not term){
                receive chm[i](id, data);
                if (data == marcaj) /* marcajul este un */
                        /* mesaj de date cu tipar special */
                        send chs[i](receptie-marcaj,id);
                else { /* informeaza nod despre op facuta */
                        send chs[i](receptie, id);
                        realizeaza prelucrare;
                        ....
                        send chm[j](i, data);
                        send chs[i](transmitere, j);
                        ...
                        if (decizie terminare) term = true;
                }
        }
        term = false;
        while (NOT term){
                send chs[i](terminare, i);
                receive stop[i](term);
        }
        stop;
}

process Nod[i=1 to n]{
        bool term = false;
        arc in[1:n], out[1:n];
        int nsemnale =0; /*numar semnale*/
        /*asteptate pe out ptr. marcaje trimise*/
        int prim =0;
        int nmarcaje =0; /*numar marcaje asteptate*/
        /*pe in după date primite*/
        int id, fel k;
        while (true){
                receive chs[i](k, id);
                if (k == receptie){
                        if (prim == 0) prim = id;
                        if (not in[id].activ){
                                in[id].activ = true;
                                nmarcaje += 1;
                        }
                else if (k == receptie-marcaj){
                        in[id].marcaj =true;
                        nmarcaje = nmarcaje-1;
                        if (id==prim) term = true;}
                        /*daca marcaj de la prim, incepe faza terminare*/
                else if (k == transmitere)
                        if (NOT out[id].activ){
                                out[id].activ = true;
                                nsemnale = nsemnale+1;
                        }
                else if (k = semnal){
                        out[id].activ = false;
```

```
                        out[id].marcaj = false;
                        nsemnale = nsemnale-1;}
                else if (k == terminare)
                        if (NOT term) send stop[i](false);
                                /* nu s-a primit marcaj de la prim */
                        else    /* s-a primit */
                                /* trimite marcaje pe iesirile active*/
                        for [j = 1 to n st j<>prim and
                        out[j].activ and not out[j].marcaj]{

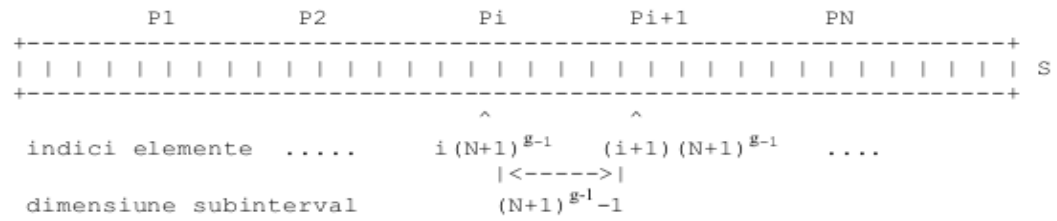
                                send chm[j](i, marcaj);
                                out[j].marcaj = true;
                        }
                if (nmarcaje <> 0) send stop[i](false);
                        /* nu s-au primit toate marcajele pe intrarile active*/
                else{   /*nmarcaje = 0 */
                        /* trimite semnale pe intrarile active <> prim*/
                for [j = 1 to n st in[j].exista and (j<>prim)
                        and in[j].activ]{

                        send chs[j](semnal, i);
                        /* marcheaza legatura inactiva */
                        in[j].activ = false;
                        in[j].marcaj = false;
                }
                if (nsemnale<>0)
                /* mai sunt de primit semnale ptr marcaje trimise*/
                        send stop[i](false);
                else{ /* nsemnale=0 */
                        if (i<>sursa and prim<>0){
                        /* sursa nu are legaturi de intrare */
                        /* trimite semnale la prim */
                        send chs[prim](semnal, i);
                        in[prim].activ = false;
                        in[prim].marcaj = false;
                }
                /* permite terminarea*/
                send stop[i](true);
                }
                }
        }
        }
}
}
```

1 Cautare paralela (curs 5)

În cazul unui sistem CREW, difuzarea valorii lui **x** se poate face într-un singur pas. Mai mult, se poate adopta o politică de căutare diferită, obţinându-se o performanţă mai bună pentru timpul de execuţie.

Algoritmul se bazează pe împărţirea, în fiecare etapă, a secvenţei în care se face căutarea în N+1 subsecvenţe de aceeaşi lungime. Fiecare din cele N procesoare inspectează o valoare **s** aflată în poziţia dintre două secvenţe adiacente. Dacă x<s atunci toate elementele aflate la dreapta lui **s** sunt ignorate în pasul următor. Similar, dacă x>s sunt ignorate elementele mai mici ca **s**. Următorul subinterval de căutare se obţine făcand intersecţia subintervalelor păstrate de diferitele procesoare. Deoarece fiecare etapă se aplică unei secvenţe de dimensiune 1/(N+1) din secvenţa precedentă, sunt necesare $O(\log_{N+1}(n+1))$ etape.




```
/* Initializarea indicilor secventei de cautat */
int q = 1;                          /* limita stg interval crt */
int r = n;                          /* limita dr interval crt */
/* Initializarea rezultatului k si a numarului de pasi g */
int x, k = 0;              /* x = numar cautat  k = pozitia lui in sir */
int g = sup(log(n+1)/log(N+1))
enum sens {stg,dr};
sens c[0:N+1];             /*c[i] – x este la stg / dr pozitiei procesului i*/
c[0] = dr; c[N+1] = stg;
int j[1:N];               /* j[i] = pozitia din sir inspectata de proces i*/
process P[i = 1 to N]{
/* gaseste pozitia k unde se afla valoarea x */
    while (q<=r and k=0){          /* subsir exista si valoare negasita */
      j[i] = (q-1) + i*(N+1)^(g-1);
/* P[i] compara x cu s[j[i]] si determina partea de secv acceptata */
      if (j[i] <= r)               /* pozitie in interior sir */
         if (s[j[i]] == x) k = j[i];       /* pozitie gasita */
        else if (s[j[i]] > x) c[i] = stg;
         else c[i] = dr;
      else {j[i] = r+1;           /*j[i] indica o pozitie afara din sir*/
            c[i] = stg;}
/* calculeaza indicii subsecventei urmatoare */
/* doar un proces poate face modificarea intervalului de cautare*/

         if (c[i] <> c[i-1]){
              r = j[i] – 1;
              if (i>1) q = j[i-1] + 1;}
  /* doar procesul N executa operatia urmatoare */
         if (i==N and c[i]<>c[i+1]) q = j[i] +1;
         g := g-1;
      }          /*end while*/
  }              /* end process */
```

2. Inmultirea a 2 matrici in paralel. (note curs 2a, pag 6)

Aici sunt n^2 procese, fiecare având constantele locale *i* şi *j*, precum şi variabila locală *sum*. *k* este contorul unui cuantificator şi se modifică doar prin regulile proprii cuantificatorilor. În declararea proceselor se poate folosi şi următoarea formă, în care se asociază **nume** proceselor paralele conţinute de un program.

Exemplu

```
real a[1:n,1:n], b[1:n,1:n], c[1:n,1:n];

process Prod (i = 1 to n, j= 1 to n) {
     real sum = 0;
     for [k = 1 to n]
         sum=sum+a[i,k]*b[k,j];
     c[i,j] = sum;
}
```

3. Prioritate cititori (curs 4)

Procesul cititor

```
sem rw = 1;
sem mutexR = 1;    /*accesul la nr este exclusiv ptr. cititori*/
int nr = 0;
process Cititor [i = 1 to m]{
while (true){
   P(mutexR);     /*necesara pentru accesul exclusiv la nr*/
      nr = nr+1;
      if (nr == 1) P(rw);   /*daca prim cititor*/
   V(mutexR);
   citeste din resursa comuna;
   P(mutexR)
      nr = nr-1;
      if (nr == 0) V(rw);      /*daca ultim cititor*/
   V(mutexR);}
}
```

Procesul scriitor

```
sem rw = 1, mutexR = 1;
int nr = 0;

process Scriitor [j = 1 to n]{
while (true){
   P(rw);
   scrie in resursa comuna;
   V(rw);}
}
```

4. Cititori si scriitori cu split binary semaphore - curs 4

Pentru specificarea conditiilor se definesc

```
int nr = 0;     /*nr cititori care folosesc resursa */
int nw = 0;     /*nr scriitori care folosesc resursa */
int dr = 0;     /*nr cititori intarziati */
int dw = 0;     /*nr scriitori intarziati */
```

conditia 1	cititor intarziat daca	nw>0 OR dw>0
conditia 2	cititor trezit daca	nw=0 AND dw=0
conditia 3	scriitor intarziat daca	nr>0 OR nw>0

Pentru controlul accesului definim semafoarele

```
sem r = 0;      /*intarzie cititori daca nw>0 OR dw>0*/
sem w = 0;      /*intarzie scriitori pana cand nw=0 AND nr=0*/
```

```
process Cititor [i = 1 to m]{
 while (true){
   P(e);
   if (nw>0 or dw>0){
     dr = dr+1; V(e); P(r);        /*intarzie cititorul*/
   }
   nr = nr+1;
   if (dr>0) {dr = dr-1; V(r);}      /*trezeste in alt cititor*/
   else if (dr==0) V(e);
   citeste din resursa comuna;
   P(e);
   nr = nr-1;
   if (nr == 0 and dw>0){
         dw = dw-1; V(w);}          /*trezeste un scriitor*/
   else if (nr>0 or dw==0) V(e);
   sectiune necritica;
 }
}
 process Scriitor [j = 1 to n]{
  while (true){
    P(e);
    if (nr>0 or nw>0){
       dw = dw+1; V(e); P(w);}      /*intarzie scriitorul*/
    nw = nw+1;
    V(e);
    scrie in resursa comuna;
    P(e);
    nw = nw-1;
    if (dr>0 and dw==0){
          dr = dr-1; V(r);}              /*trezeste un cititor*/
    else if (dw>0){dw = dw-1; V(w);}     /*trezeste un scriitor*/
    else if (dr==0 and dw==0) V(e);
    sectiune necritica;
  }
 }
```

Semafoarele **s** si **w** sunt folosite in sectiunile decide din procesele Cititor si Scriitor

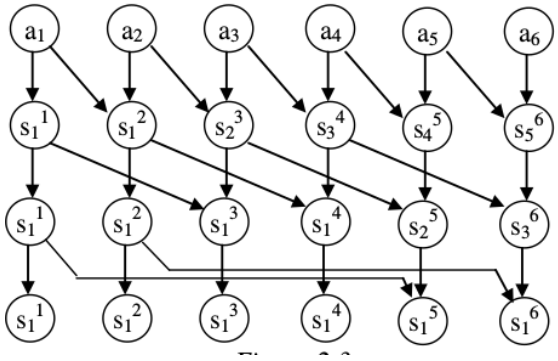
```
process Cititor [i = 1 to m]{
   while (true){
      decide intrare in sectiunea critica sau intarzierea;
      citeste din resursa comuna;
      decide daca si cine intra in sectiunea critica;
      sectiune necritica;
   }
}

process Scriitor [j = 1 to n]{
   while (true){
      decide intrare in sectiunea critica sau intarzierea;
      scrie in resursa comuna;
      decide daca si cine intra in sectiunea critica;
      sectiune necritica;
   }
}
```

I

5. Sume prefix - notite 2.c

```
int a[1:n], temp[1:n];
process suma [k=1 to n]{
     for [j = 1 to sup(log2 n)]{
          temp[k] = a[k];
          barrier;
          if (k-2^(j-1)>=1) a[k] = temp[k-2^(j-1)] + a[k];
          barrier;
     }
}
```




```
int a[1:n], temp[1:n];
process suma[k:1..n]{
   int d = 1;
   while (d<n){
     temp[k] = a[k];
     barrier;
     if (k-d>=1) a[k] = temp[k-d] + a[k];
     barrier;
     d = 2*d;
   }
}
```

Asta e fara logaritmi si ridicari la putere ^, dar e acelasi algoritm.
Simd e fara bariere

Descriere algoritmi SIMD

Având în vedere că in algoritmii SIMD toate acţiunile sunt sincronizate (deci orice acţiune ar trebui urmată de o barieră), în descrierea lor eliminăm sincronizările prin bariere. Calculul sumelor prefix poate fi reprezentat astfel (varianta SIMD):

```
int a[1:n];
process suma [k=1 to n]{
   int temp;            /*temp este locala procesului k*/
   for [j = 1 to sup(log2 n)]{
      if (k-2^(j-1)>=1){ temp = a[k-2^(j-1)];
                         a[k] = temp + a[k];
      }
   }
```

6. Sfarsitul unei liste inlantuite - notite 2.c

```
int leg[1:n], end[1:n];
process Afla [i=1 to n){
   int nou, d=1;
   end[i] = leg[i];
   barrier;
   while (d<n){
     nou = 0;        //tampon pentru legatura urmatoare
     if (end[i]<>0 and end[end[i]]<>0) nou=end[end[i]];
     barrier;
     if (nou<>0) end[i]=nou;
     barrier;
     d = 2*d
   }
}
```

Proprietati fundamentale ale algoritmilor unda

- **terminare** – fiecare **calcul** este **finit**
- **decizie** – fiecare calcul contine cel putin un **eveniment de decizie** (decide)
- **dependenta** – in fiecare calcul, fiecare ***decide*** **este precedat** cauzal de **un eveniment in fiecare proces**

Alte caractreristici

Centralizare

- algoritmi centralizati – un initiator
- algoritmi descentralizati – set arbitrar de initiatori

Topologie

- regulata: inel, arbore, clica etc.
- fixa (nu se produc modificari topologice)
- nedirectionata (canale bi-directionale); exceptiile mentionate explicit
- conectata (exista o cale intre oricare doua procese)

8. Algoritmi de unda - curs 9

Algoritmul inel

fiecare proces are un vecin dedicat, Urm
transmiterea foloseste **adresarea prin directie** (Urm, Prec)
toate canalele selectate prin Urm formeaza un ciclu Hamiltonian
algoritmul este **centralizat**:

- initiatorul trimite un token (jeton) care este pasat de fiecare proces de a lungul inelului pana ajunge inapoi la initiator;
- initiatorul ia apoi decizia

```
chan token[1:n] (tok_type tok);
```

`/*initiator*/`
`process P[I]{`
`tok_type tok;`
`send token[Urm](tok);`
`receive token[I](tok);`
`decide`
	`/*non-initiatori*/`
	`process P[k=1 to n,`
	`k<>I]{`
	`tok_type tok;`
	`receive token[k](tok);`
	`send token[Urm](tok);`

Număr de mesaje = n
Timp = n

*** Arbore** - descentralizat - mai multe procese initiaza(adica frunzele)

```
  chan ch[1:n] (int id, tok_type tok);
      /* fiecare proces are un canal propriu */

  process Proc[p = 1 to n]{
    int r = numar_vecini_p;
    bool Vecini[1:n] = vecinii_lui_p;
        /* Vecini[i]=true daca i este vecin lui p */
    bool rec[1:n] = ([n] false);
        /* rec[i]=true daca p a primit mesaj de la i */
    tok_type tok; int id, q0;

    while (r>1){receive ch[p](id,tok);
           rec[id] = true; r = r-1};}
  /* de la un singur vecin, q0, nu s-a primit mesaj */
    afla q0 pentru care: Vecini[q0] and NOT rec[q0];
    send ch[q0](p, tok);
    receive ch[p](q0, tok); rec[q0] := true;
    decide;
    /* informeaza celelalte procese despre decizie */
    /* for [q = 1 to n st Vecini[q] and q<>q0]
                                send ch[q](p,tok); */
  }
```

Număr de mesaje = n Timp = O(D)
Fiecare proces trimite un mesaj și sunt n procese.
Mesajul trimis de o frunză atinge orice alt nod într-un număr de pași mai mic decât diametrul D al arborelui, fiecare pas fiind făcut într-o unitate de timp.
Algoritmul de bază se termină când unul din procese decide.

- **Ecou**

```
chan ch[1:n] (int id, tok_type tok);
const int I = id_initiator;
process Proc(I){
  int r = numar_vecini_I;
  bool Vecini[1:n] = vecinii_lui_I;
  tok_type tok;
  int id;
  for [i=1 to n st Vecin[i]]
    send ch[i](I, tok);
  while (r>0){receive ch[I](id, tok);
              r = r-1; }
  decide;
}

process Proc[p=1 to n, p<>I){
  int r = numar_vecini_p;
  bool Vecini[1:n] = vecinii_lui_p;
  tok_type tok;


  int id, parinte;
  receive ch[p](parinte, tok); r = r-1;
  for [q=1 to n st Vecini[q] and q<>parinte]
    send ch[q](p, tok);
  while (r>0){receive ch[p](id, tok);
              r = r-1};
  send ch[parinte](p, tok);
}
```

Algoritmul fazelor

Doua categorii de procese:

- Initiatori – pornesc cu trimiterea unui mesaj fiecarui out-vecin
- Ne-initiatori – asteapta primirea unui prim mesaj si apoi trimit un mesaj fiecarui out-vecin

```
chan ch[1:n] (int id, tok_type tok);
const int D = diametrul_retelei;
process Proc [p = 1 to n]{
  bool in[1:n] = in-vecinii_lui_p;
  /* in[i]==true daca i invecin */
  bool out[1:n] = out-vecinii_lui_p;
  /* out[i]==true daca i outvecin */
  int rec[1:n] = ([n] 0);
  /* rec[q] = numar mesaje primite de la q */
  int sent = 0;
  /* nr. mesaje transmise out-vecinilor */
  tok_type tok;
  int id;

   if (p este initiator){
     for [q = 1 to n st out[q]]
       send ch[q](p, tok);
      sent = sent+1;}
   while (min(rec) < D){
       /*min(rec) = min(rec[q]),q=1..n and in[q])*/
     receive ch[p](id, tok);
     rec[id] = rec[id]+1;
     if (min (rec) >= sent and sent < D){
       for [q=1 to n st out[q]]
         send ch[q](p,tok);
       sent = sent+1;}
   }
   decide;
}
```

Mesaje **= 2D|E|**
fiecare proces trimite D mesaje si sunt 2|E| canale uni-directionale

Timp **= 2D** sunt D faze, in fiecare avem **receive** si **send** (ops. send se deruleaza in paralel)

9. Problema producător-consumator. - notite 2.a

```
int buf, p=0, c=0;
     /*p = numarul de valori transmise in buf*/
     /*c = numarul de valori preluate din buf*/
process producer{
   int a [1:n];
   while (p<n){
      <await (p==c);>
      buf = a[p+1];
      p = p+1;
   }
```

```
process consumer{
   int b [1:n];
   while (c<n){
      <await (p>c);>
      b[c+1] = buf;
      c = c+1;
   }
}
```

Se consideră un producător și un consumator care comunică printr-un singur tampon partajat. Producătorul generează și pune câte o valoare în tampon, iar consumatorul preia câte o valoare din tampon. Presupunem că tamponul poate păstra o singură valoare.

În soluția prezentată aici, considerăm că producătorul păstrează într-un tablou **a[1:n]** valorile furnizate; consumatorul pune în tabloul **b[1:n]** valorile preluate din tampon. Soluția trebuie să asigure că valorile ajung în tabloul **b** în același număr și în aceeași ordine ca în **a**.

Multiple producers multiple consumers - notite 4

```
typeT buf[1:k];
sem gol = k, plin = 0;        //sem. generalizate
int prim = 1, ultim = 1;
sem mutexP = 1, mutexC = 1;

process Producator[i = 1 to M]{
     typeT v;
     while (true){
          v = produce ();
          P(gol);
               P(mutexP);
               buf[ultim]=v; ultim = ultim mod k + 1;
               V(mutexP);
          V(plin);
     }
}

process Consumator[i = 1 to N]{
    typeT w;
    while (true){
         P(plin);
              P(mutexC);
              w = buf[prim]; prim = prim mod k + 1;
              V(mutexC);
         V(gol);
         consuma (w);
     }
}
```

10. Secventa critica pt calcul distribuit(requesturi si replyuri) - ceva similar cred? - curs 8
(excluderea mutuala distribuita)

```
enum fel {V, P, ack};
chan opsem[1:n](int transm, fel op, int timp);
chan start[1:n](int timp);

process Utiliz [i = 1 to n]{
int cl = 0;                    /*ceas logic*/
int ts;                        /*timestamp mesaj*/
...
cl = cl+1;
broadcast opsem(i, P, cl);     /*operatia P*/
receive start[i](ts);          /*asteapta permisiune*/
cl = max(cl, ts) + 1;
    sectiune critica;
cl = cl+1;
broadcast opsem(i, V, cl);     /*operatia V*/
    sectiune ne-critica;
…
}

process Ajutor [i = 1 to n){
typedef struct {int transm, fel k, int ts} queue;
queue qm [lmax];     /*coada ordonata dupa timestamp tm*/
int cl = 0;
int sem = valoare_initiala;
int transm;
fel k;
int ts;

while (true) {
  receive opsem[i](transm, k, ts);
  cl = max(cl, ts) + 1;
  if (k==P or k==V) {
    insereaza (transm,k,ts) in loc corespunzator in qm;
    cl = cl+1; broadcast opsem(i, ack, cl);}
  else if (k==ack) {
    inregistreaza primirea unui ack;
    for [mesajele V complet confirmate ] {
          scoate mesaj din qm;
          sem = sem+1;}
    for [mesajele P complet confirmate st sem>0] {
          scoate mesaj (transm, k, ts) din qm;
          sem = sem-1;
          if (transm=i) {cl := cl+1; send start[i](cl); }
  }
}    /*end process*/
```

18. Alegere lider - curs 10

Arbore

ws = wakeup send - boolean, fiecare proces trimite o singura data wakeup
wr = wakeup receive;

```
chan ch[1:n](int id_transm, int id_mic);
chan wakeup[1:n]();
process Proc(p = 1 to n){
 bool ws = false;
 int wr = 0;
 int r = numar_vecini_p;
 bool Vecini[1:n] = vecinii_lui_p;
 bool rec [1:n] = ([n]*false);
 int V = p;
 enum state {sleep, leader, lost}
 state stare = sleep;
 int id;


/* aici incepe faza wake-up */
 if (p este initiator){
     ws = true;
     for [q = 1 to r st Vecini[q]]
        send wakeup[q]();
 }
 while (wr < numar_vecini_p){
     receive wakeup[p](); wr = wr+1;
     if (NOT ws) ->
     ws = true;
     for [q = 1 to n st Vecini[q]]
      send wakeup[q]();
 }

/* aici incepe algoritmul tree */
 do (r>1){
  receive ch[p](q,id);
    rec[q] = true; r = r-1;
    V = min(V, id);
 }

/* de la un singur vecin, q0 nu s-a primit mesaj */
  afla q0: Vecini[q0] and NOT rec[q0];
  send ch[q0](p, V);
  receive ch[p](q0, id);
  V = min (V, id);
  if (V=p) stare = leader;
  else stare = lost;
/*transmite vecinilor decizia*/
  for [q = 1 to n st Vecini[q] and q<>q0]
     send ch[q](p,V);
}
```

Numar mesaje = O(N)

Pe fiecare canal se trimit două mesaje *wakeup* și două *token*-uri ceeace înseamnă un total de 4N-4 mesaje.

Timp = O(D)

În D pași după ce primul proces pornește algoritmul, fiecare proces a trimis mesajele *wakeup*. Ca urmare, în D+1 pași fiecare proces a pornit unda. Prima decizie se ia la D pași după pornire, iar ultima după alți D pași. Rezultă totalul de 3D+1 pași.

Inel - Algoritmul LeLann

Algoritmul realizează următoarele acțiuni:

1. Fiecare proces transmite în inel un mesaj care conține identificatorul său
2. Fiecare proces colectează identificatorii celorlalte procese
3. Fiecare proces calculează maximul.
4. Procesul al cărui număr este egal cu maximul devine lider.

Complexitate $O(n^2)$ mesaje comunicate - fiecare proces transmite un mesaj care trece pe la toate celelalte.

```
chan ch[Ids](token tok, int id_mare);
process Proc[p = 1 to n]{
typedef SOP {multime identificatori};
SOP List = {p};
enum state (candidate,leader,lost);
state stare = candidate;
send ch[Urm](tok, p);
receive ch[p](tok, q);
while (q<>p){ List = List U {q};
          send ch[Urm](tok, q);
          receive ch[p](tok, q);
          }
if (p == max(List)) stare = leader;
else stare = lost;
}
```

Inel - LeLann-Chang-Robert

```
chan ch[1:n](token tok, int id_mare);
process Proc[p = 1 to n]{
enum state (candidate, leader, lost);
state stare = candidate;
send ch[Urm](tok, p);
while (state <> leader){
  receive ch[p](tok, q);
  if (q==p) stare = leader;
  else if (q>p){
    if (stare == candidate)stare = lost;
    send ch[Urm](tok, q);
    }
 }
}
```

Timp O(n)
Numar mesaje: worst case = n(n + 1) / 2; medium = n log n

Algoritmul Hirschberg-Sinclair

- Prea complicat fmmmmmm

19 Stabilirea topologiei - curs 11

Algoritmul pulsatiilor

```
chan topo[1:N] (tip_top);
process Nod[p=1 to N]{
  tip_leg  leg = (vecinii_lui_p);
  tip_leg activ = leg;        {vecinii activi}
  tip_top top = ([N*N]FALSE);
  bool gata = FALSE;
  int transm; bool qgata; tip-top top_nou;
  top[p,1:N] = leg;      {actualizeaza vecini}

 while (NOT gata){
  /*transmite topologia curenta tuturor vecinilor*/
   for [q =1 to N st leg[q]]
     send topo[q](p,FALSE,top);
   /*receptioneaza topologiile de la vecini*/
   for [q =1 to N st leg[q]]{
     receive topo[p](transm,qgata,top_nou);
     top = top OR top_nou;
     if (qgata) activ[transm]=FALSE;
   }


    if (toate liniile din top au un element TRUE)
        gata = TRUE;
}

  /*transmite top vecinilor activi in ultimul rund*/
  for [q =1 to N st activ[q]]
      send topo[q](p,TRUE,top);
  for [q =1 to N st activ[q]]
      receive topo[p](transm,qgata,top_nou);
```

Complexitate: 2m*N*(D+1)

Mesaje de sondaj cu ecou

```
const int sursa=i;      /*index initiator*/
enum fel {sonda, ecou};
typedef tip_leg bool[1:n];
typedef tip_top bool[1:n, 1:n];
chan sonda-ecou[1:n](fel, int transm,   tip_top top);
chan ecou_final(tip_top top);

process Initiator{
 tip_top top;
 send sonda-ecou[sursa](sonda, sursa, Φ);
 receive ecou_final (top);
}

process Nod[p=1 to n){
  tip_leg leg = vecinii_lui_p;
  tip_top top = ([n*n] false);
  top[p, 1:n] = leg;
  tip_top top_nou;

  int prim;
     /*nodul de la care s-a primit prima sonda*/
  fel k; int transm;
  int nr_ecouri = numar_vecini - 1;
  receive sonda-ecou[p](k, prim, top_nou);
  /*transmite sondaje celorlalte noduri vecine, copiii
    lui p*/
  for [q := 1 to n st leg[q] and q<>prim]
     send sonda-ecou[q](sonda, p, Φ);
                              /*Φ este topo. nula*/


/*primeste ecourile si fa reuniunea lor, sau sonde si
    ignora*/
  while (nr_ecouri>0){
    receive sonda-ecou[p](k, transm, top_nou);
    if (k==sonda)
    send sonda-ecou[transm](ecou,p,Φ);
  else if (k=ecou){
     top = top or top_nou;
     nr_ecouri = nr_ecouri-1;}
  }
  if (p=sursa) send ecou_final(top);
       else send sonda-ecou[prim](ecou, p, top);
```

20. Terminare - mai mult in curs 12

Cu Jeton - inel (reguli)

acţiunile lui P(1) cînd devine liber prima dată:

```
culoare[1] = albastru;
jeton = 0;
send ch[2](jeton);
```

acţiunile lui P(i:1..n) la recepţia unui mesaj obişnuit:

```
culoare[i] = rosu;
```

acţiunile lui P(i:2..n) la primirea unui jeton:

```
culoare[i] = albastru;
jeton = jeton +1;
send ch[(i mod n) + 1](jeton);
```

acţiunile lui P(1) la recepţia jetonului:

```
if(culoare[1]== albastru){anunţă terminarea; stop}
culoare[1] = albastru;
jeton = 0;
send ch[2](jeton);
```

Cu jeton - general

acţiunile lui P[i=1 to n] la primirea unui mesaj uzual:

```
culoare[i] = rosu;
```

acţiunile lui P[i=1 to n] la recepţia jetonului:

```
if (jeton == nc) {anunţă terminarea; stop;}
if (culoare[i] == rosu){culoare[i] = albastru;
                        jeton = 0}
else jeton = jeton +1;    /*culoare[i] == albastru*/
setează j la canalul următor din ciclul c;
send ch[j](jeton);
```

Detecţia terminării folosind confirmările mesajelor

- dacă pentru fiecare mesaj transmis pe un canal, procesul primeşte de la receptor un semnal de confirmare, atunci canalul respectiv este gol
- existenta unui proces sursa, care nu are legături de intrare şi de la care orice alt proces este accesibil pe o cale oarecare din graf

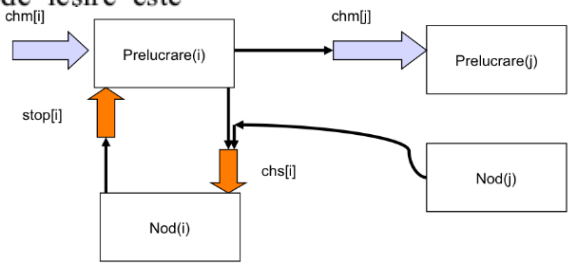
Dacă procesele ar fi organizate într-o reţea cu topologie arborescentă, algoritmul de semnalare ar fi simplu: cînd un proces frunză se termină, el anunţă părintele său. Cînd un nod oarecare al arborelui se termină, el aşteaptă notificarea terminării de la toţi fiii săi şi apoi îşi anunţă părintele. Notificările se propagă astfel pînă la procesul sursa, aflat în rădăcina arborelui.

Acest algoritm poate fi extins la un **graf dirijat aciclic** de procese. Fiecărei legături i se asociază un număr, reprezentînd diferenţa dintre numărul de mesaje de date transmise şi numărul de semnale de confirmare primite pe acea legătură. Numim **deficit** această diferenţă. Cînd un nod doreşte să termine, el aşteaptă primirea pe legăturile de ieşire a datelor, a unor semnale care reduc deficitele la zero, după care trimite pe fiecare legătură de intrare a datelor un număr de semnale egal cu deficitul.

Algoritmul **Dijkstra-Scholten** rezolvă problema în **cazul general**, în care sunt permise cicluri în graful proceselor. Rezolvarea se face prin generarea unui **arbore de acoperire**, pe parcursul transferului mesajelor de date: fiecare proces păstrează o variabilă **prim** a cărei valoare este egală cu identificatorul procesului de la care s-a recepţionat primul mesaj de date.

Rezolvarea implică trei faze:
- trimiterea semnalelor pe toate legăturile de intrare, cu excepţia celei de la **prim**;
- recepţia semnalelor pe toate legăturile de ieşire;
- transmiterea semnalelor spre **prim**.

Pentru recepţia semnalelor pe legăturile de ieşire, se poate ţine o evidenţă globală, deoarece este important ca deficitele legăturilor de ieşire să se anuleze, neavînd importanţă identitatea legăturilor pe care se primesc semnale. Ca urmare, o singură variabilă **nsemnale** care să ţină minte suma deficitelor legăturilor de ieşire este suficientă.



Algoritmul fiecărui nod este repartizat în două procese: Prelucrare[i] realizează transformarea datelor; Nod[i] realizează gestiunea mesajelor şi a semnalelor schimbate cu celelalte noduri. Prelucrare[i] are un canal chm[i] prin care primeşte mesajele de date. Nod[i] are un canal chs[i] prin care primeşte semnale de la celelalte procese, anunţuri de transmitere/recepţie de mesaje de date şi cereri de terminare de la Prelucrare[i]. Răspunsul la cererile de terminare este dat pe canalele stop[i].

Topologia (virtuală a) reţelei de procese este păstrată în tabloul **in**. Fiecare element al acestuia înregistrează existenţa legăturii (cîmpul **exista**) şi deficitul (cîmpul **deficit**).

Procesul **Prelucrare [i]** are o funcţionare ciclică. După recepţia unui mesaj de date, el realizează prelucrările cerute şi, eventual, transmite mesaje de date altor procese. El însuşi poate primi mesaje de date de la celelalte procese, pe canalele de intrare. Recepţiile şi transmiterile de mesaje sunt anunţate procesului Nod[i]. La sfîrşitul prelucrării, procesul trimite lui Nod[i] o cerere de terminare şi aşteaptă recepţia unei confirmări pe un canal special stop[i].

Procesul **Nod[i]** are o funcţionare ciclică. La fiecare iteraţie, el tratează unul din mesajele primite de la procesul de prelucrare local sau de la celelalte procese Nod. Pentru un anunţ de recepţie de date, procesul actualizează deficitul legăturii de intrare şi, eventual, **prim**. Un anunţ de transmitere determină incrementarea variabilei **nsemnale**. Un semnal determină decrementarea variabilei **nsemnale**. În fine, o cerere de terminare provoacă transmiterea de semnale pe legăturile de intrare. Dacă **nsemnale** este nul, atunci se transmit semnale către nodul părinte din arborele de acoperire (prim) şi se confirmă terminarea. Altfel, se transmite o infirmare a terminării.

```
enum fel{semnal, transmitere, receptie, terminare};
typedef struct {bool exista;
                int deficit} arc;
chan chs[1:n](fel op, int id);
chan chm[1:n](int id, tip_date date);
chan stop[1:n](bool);

process Nod[i=1 to n]{
 arc in[1:n];
 int nsemnale =0;
 int prim =0;
 int id; fel k;
 while (true){
   receive chs[i](k, id);
   if (k == receptie){
       if (prim == 0) prim = id;
       in[id].deficit = in[id].deficit+1;
   else if (k == transmitere)
       nsemnale = nsemnale+1;
   else if (k == semnal)
       nsemnale = nsemnale-1;
   else if (k = terminare)
/* trimite semnale pentru toate legăturile de intrare,
          mai puţin prim */
    for [j=1 to n st in[j].exista and j<>prim]
        while (in[j].deficit>0){
           send chs[j](semnal, i);
           in[j].deficit = in[j].deficit-1;
        }
          /* daca mai sunt semnale de primit */
    if (nsemnale<>0) send stop[i](false);

    else{      /*nsemnale==0*/
          // trimite semnale procesului prim
        if (i<>sursa and prim<>0)
          while (in[prim].deficit>0){
           send chs[prim](semnal, i);
           in[prim].deficit=in[prim].deficit-1;
          }
          /* procesul prelucrare[i] se poate opri */
         send stop[i](true);
    }    /*end if*/
 }       /*end while*/
}        /*end process*/

process Prelucrare(i=1 to n){
  tip_date data; int id, bool term;
  initializari;
  term = false;
  while (not term){
     ....
     receive chm[i](id, data);
     send chs[i](receptie, id);
     realizeaza prelucrare
     ....
     send chm[j](i, data);
     send chs[i](transmitere, j);
     ...
     if (decizie terminare) term := true;
  }
  term =false;
  while (not term){
     send chs[i](terminare, i);
     receive stop[i](term);
  }
  stop;
```

Detectia cu marcaje

În cea de a doua variantă, procesele transmit mesaje de marcaj al sfîrşitului comunicaţiei, urmînd să primească o confirmare doar pentru aceste mesaje, nu şi pentru cele de date. Cînd un proces decide terminarea, el aşteaptă marcaje pe toate legăturile de intrare şi transmite un marcaj pe fiecare legătură de ieşire. Îndată ce procesul a primit marcaje, el trimite confirmări pe legăturile de intrare corespunzătoare, mai puţin pe prima legătură. Apoi, el aşteaptă semnale pe legăturile de ieşire. Cînd le primeşte, transmite un semnal pe prima legătură.

Structurile utilizate pentru cele două categorii de arce au aceeaşi sintaxă. Ele conţin câmpurile **exista**, **activ** şi **marcaj**. De data aceasta, fiecare **arc de intrare** înregistrează dacă:
- s-a primit un mesaj (se pune **activ** pe true);
- s-a primit un marcaj (se pune **marcaj** pe true);
- s-a transmis un semnal de confirmare (se pun ambele pe false).

Similar, pentru fiecare **arc de ieşire**, se înregistrează dacă:
- s-a transmis un mesaj (se pune **activ** pe true);
- s-a transmis un marcaj (se pune **marcaj** pe true);
- s-a primit un semnal de confirmare (se pun ambele pe false).

Huang

Modelul se bazează pe următoarele:
– procesele pot fi active sau libere (idle)
– doar procesele active transmit mesaje
– procesele libere pot deveni active la recepţia unui mesaj de "calcul"
– procesele active pot deveni libere în price moment
– **terminarea**: toate procesele sunt libere şi nu există mesaje de "calcul" în tranzit

Ideea este următoarea:
• Un agent de control are iniţial ponderea 1
• Toate celelalte procese sunt libere şi au iniţial ponderea 0
• Calculul porneşte când agentul de control transmite un mesaj de "calcul" unui proces
• Un proces liber devine activ la recepţia unui mesaj de "calcul"
Fie:
• B(DW) – mesajul de "calcul" cu ponderea DW. Mesajul poate fi trimis doar de agentul de control sau de de un proces activ
• C(DW) – mesaj de control cu ponderea DW, transmis de un proces activ agentului de control când devine liber
• Ponderea curentă a unui proces este W

Algoritmul are următoarele reguli:

1. **Send B(DW)**:
• Găseşte W1, W2 astfel încât W1 > 0, W2 > 0, W1 + W2 = W
• Pune W = W1 şi transmite B(W2)

2. **Receive B(DW)**:
• W += DW;
• if liber -> devine activ

3. **Send C(DW)**:
• send C(W) agentului de control
• devine liber

4. **Receive C(DW)**:
• W += DW
• if W = 1, declară “terminarea”

1. **Cautare paralela**
2. **Inmultirea paralela a matricilor**
3. **Prioritate pt cititori**
4. **Cititori si scriitori cu split binary semaphore**
5. **Sume/Produs prefix cu MIMD**
6. **Sfarsitul listei**
7.
8. **Algoritmi de unda**
9. **Producatori Consumatori**
10. **Secventa critica pt calcul distribuit(requesturi si replyuri)**
11. **Terminare cu marcaje**
12. Bizantini - pb generalilor bizantini - descriere, sol cu msg, teoreme, pseudocod, complexitate, exemple cu 3 generali
13. **Stabilirea topologiei cu sondaje de ecou**
14. Huang
15. Complexitate paralela :)
16. **Max dintr-un graf - adica lider**
17. **N procese-graf, sa se afle topologia(care sunt parintii sau copiii) - la algoritmii unda; in cursul stabilire topologie se face matricea de adiacenta top, nu parinti copii**
18. **Alegere lider topologiei**
19. **Stabilirea topologiei**
20. **Terminare**

Terminare cu marcaje - cod

```
const tip_date marcaj = mesaj-special-sfirsit-date;
enum fel(semnal, transmitere, receptie,
                    receptie-marcaj, terminare);
typedef struct (exista: bool;
```

```
                activ: bool; /*initial false*/
                marcaj: bool;)arc; /*initial false*/
chan chs[1:n](fel op, int id);
chan chm[1:n](int id, tip_date date);
chan stop[1:n](bool);

process Prelucrare[i=1 to n){
        tip_date data, int id, bool term;
        initializari;
        term = false;
        while (not term){
                receive chm[i](id, data);
                if (data == marcaj) /* marcajul este un */
                        /* mesaj de date cu tipar special */
                        send chs[i](receptie-marcaj,id);
                else { /* informeaza nod despre op facuta */
                        send chs[i](receptie, id);
                        realizeaza prelucrare;
                        ....
                        send chm[j](i, data);
                        send chs[i](transmitere, j);
                        ...
                        if (decizie terminare) term = true;
                }
        }
        term = false;
        while (NOT term){
                send chs[i](terminare, i);
                receive stop[i](term);
        }
        stop;
}

process Nod[i=1 to n]{
        bool term = false;
        arc in[1:n], out[1:n];
        int nsemnale =0; /*numar semnale*/
        /*asteptate pe out ptr. marcaje trimise*/
        int prim =0;
        int nmarcaje =0; /*numar marcaje asteptate*/
        /*pe in după date primite*/
        int id, fel k;
        while (true){
                receive chs[i](k, id);
                if (k == receptie){
                        if (prim == 0) prim = id;
                        if (not in[id].activ){
                                in[id].activ = true;
                                nmarcaje += 1;
                        }
                else if (k == receptie-marcaj){
                        in[id].marcaj =true;
                        nmarcaje = nmarcaje-1;
                        if (id==prim) term = true;}
                        /*daca marcaj de la prim, incepe faza terminare*/
                else if (k == transmitere)
                        if (NOT out[id].activ){
                                out[id].activ = true;
                                nsemnale = nsemnale+1;
                        }
                else if (k = semnal){
                        out[id].activ = false;
```

```
                        out[id].marcaj = false;
                        nsemnale = nsemnale-1;}
                    else if (k == terminare)
                        if (NOT term) send stop[i](false);
                            /* nu s-a primit marcaj de la prim */
                        else    /* s-a primit */
                            /* trimite marcaje pe iesirile active*/
                        for [j = 1 to n st j<>prim and
                        out[j].activ and not out[j].marcaj]{

                            send chm[j](i, marcaj);
                            out[j].marcaj = true;
                        }
                    if (nmarcaje <> 0) send stop[i](false);
                        /* nu s-au primit toate marcajele pe intrarile active*/
                    else{   /*nmarcaje = 0 */
                        /* trimite semnale pe intrarile active <> prim*/
                    for [j = 1 to n st in[j].exista and (j<>prim)
                        and in[j].activ]{

                        send chs[j](semnal, i);
                        /* marcheaza legatura inactiva */
                        in[j].activ = false;
                        in[j].marcaj = false;
                    }
                    if (nsemnale<>0)
                    /* mai sunt de primit semnale ptr marcaje trimise*/
                        send stop[i](false);
                    else{ /* nsemnale=0 */
                        if (i<>sursa and prim<>0){
                        /* sursa nu are legaturi de intrare */
                        /* trimite semnale la prim */
                        send chs[prim](semnal, i);
                        in[prim].activ = false;
                        in[prim].marcaj = false;
                    }
                    /* permite terminarea*/
                    send stop[i](true);
                    }
                    }
            }
            }
}
}
```

What is Parallel Computing?
A Not Too Serious Explanation.

Before I explain parallel computing, it's important to understand that

You can run, but you can't hide.

I'll come back to this later.

Suppose you have a lot of work to be done, and want to get it done much faster, so you hire 100 workers. If the work is 100 separate jobs that don't depend on each other, and they all take the same amount of time and can be easily parceled out to the workers, then you'll get it done about 100 times faster. This is so easy that it is called *embarrassingly parallel*. Just because it is embarrassing doesn't mean you shouldn't do it, and in fact it is probably exactly what you should do. A different option would be to parallelize each job, as discussed below, and then run these parallelizations one after another. However, as will be shown, this is probably a less efficient way of doing the work. Occasionally this isn't true because on computers, doing all of the job on one processor may require storing many results on disk, while the parallel job may spread the intermediate results in the RAM of the different processors. RAM is much faster than disks. However, if the program isn't spending a lot of time using the disk then embarrassingly parallel is the smart way to go. Assume this is what you should do unless you analyze the situation and determine that it isn't. Embarrassingly parallel is simple, and if you can get the workers do it for free then it is the cheapest solution as well.

What if the jobs take widely different amounts of time, but still have no interaction? If you have enough of them, just start handing them out until every worker has one, and then when workers are done they come back and request another. This will work fairly well, especially if you can put longer jobs first and shorter ones later. This form of parallelization is still pretty simple (unless it is teenagers that are doing the jobs - see "Do Teenagers Deserve Negative Wages?"). Many of the departmental computers that have more than one processor are run in this fashion, servicing jobs as they arrive. This is also how many airlines run their check-in queue: in queuing theory this is known as a "single queue multiple server" system, which is more efficient than the "multiple queue multiple server" systems common in checkout lines at grocery stores.

Suppose instead that this work you have is only a single job, but it takes a very long time. Now you have to do something to reorganize the job, somehow breaking it into pieces that can be done concurrently. For example, if the job is to build a house, it can be broken up into plumbing, electrical, etc. However, while many jobs can be done at the same time, some have specific orderings, such as putting in the foundation before the walls can go up. If all of the workers are there all of the time, then there will be periods when most of them are just waiting around for some task (such as the foundation) to be finished. Not very cost-effective, and you are not getting the job done 100 times faster. Such is the life of a parallel programmer.

Definition: **Parallel computing** is the use of two or more processors (cores, computers) in combination to solve a single problem.

The programmer has to figure out how to break the problem into pieces, and has to figure out how the pieces relate to each other. For example, a parallel program to play chess might look at all the possible first moves it could make. Each different first move could be explored by a different processor, to see how the game would continue from that point. At the end, these results have to be combined to figure out which is the best first move. Actually, the situation is even more complicated, because if the program is looking ahead several moves, then different starts can end up at the same board position. To be efficient, the program would have to keep track of this, so that if one processor had already evaluated that position, then others would not waste time duplicating the effort. This is how must parallel chess-playing systems work, including the famous IBM Deep Blue machine that beat Kasparov.

Is Parallel Computing Easier or Harder than Serial Computing?

(Standard computing is also known as "serial computing", so you've been doing serial computing for years!)

Easier:

Most of the universe is inherently parallel, with numerous things happening simultaneously. You may put your pants on one leg at a time, but millions of people are putting their pants on simultaneously. Many problems have an abundance of natural parallelism that a programmer can exploit. Sometimes people are forced to rethink the problem when they try to develop a parallel program, and they change their entire approach in order to directly utilize the inherent parallelism.

Harder:

- Blatent poor performance: it is easier to see that you're not doing well. On a serial computer the processor may wait as it moves data from RAM to cache, but it basically appears busy all the time and it is only after you do a more detailed analysis that you determine that it is actually performing poorly. (See "Using Cache Saves Cash") On a parallel computer of 100 processors most people quickly notice that a program is not running 100 times faster. The serial program may not have been very good (see "Why Algorithms Experts Deserve Big Bucks") but it is not generally as obvious as the fact that many of the parallel processors are often idle.

- Little experience: most programmers have little or no experience with parallel computing, and there are few parallel programs to use off-the shelf or even good examples to copy from. Hence people often have to reinvent the parallel wheel (see "Parallelism needs Classes for the Masses"). People developing parallel systems software are similarly behind on their learning curves. Often they reuse material developed for serial systems, even when it causes performance problems (see "Serial Sins in a Parallel World").

- Poor portability: a program may work work on one machine, but when the program is ported to a new machine it may be so different that drastic changes need to be made just to permit the program to be run. Most serial computers have the same basic organization, but this is not so for parallel computers. Some parallel computers, such as the popular cluster systems, are essentially just a collection of computers linked together with Ethernet. They use simple commands, similar to read and write, to communicate among the processors. (The most common system for doing this communication is MPI.) Such parallel computers are known as *message-passing* systems, or *distributed memory* computers. Clusters are usually quite inexpensive because they are built from commodity parts, and they have become quite common in businesses and universities. Distributed memory computers with better performance have faster interconnections among the processors, and software better tuned to support parallelism, but this increases the cost because many of the parts are more non-standard.

  Other parallel computers, such as some of the systems from SGI and Cray, are *shared-memory* systems, where every processor can directly read and write data from every memory location. Think of this as being like everyone using a blackboard to do their calculations, where everyone has chalk and an eraser and can decide to write anywhere on the board. For these systems, there is a standard known as OpenMP, which is a collection of language extensions to Fortran and C/C++. Shared memory computers must use many non-commodity parts to support the sharing, and hence tend to be more expensive than distributed memory systems. However, it is often easier to write a program for shared memory systems than for distributed memory ones.

  Yet other computers exploit graphics processors, GPUs, to achieve parallelism. However, while a GPU may have 100 cores, there are serious restrictions on their programability, and serious performance problems if there is much data movement. Further, the languages to use for GPUs are rapidly evolving, so it is unclear if you should use CUDA, OpenCL, OpenACC, accelerator extensions being incorporated in OpenMP, etc.

- Amdahl's law: if the foundation takes 5% of the time, and only one worker can do it while everything else has to wait until the foundation is done, then you can never get the job done in less than 5% of the original time, no matter how many workers you have. Incidentally, this helps explain why it is much easier to be a successful parallel programmer on a small system. In such a situation, if everything else parallelizes perfectly, then 100 workers would take 5% (serial) + 95%/100 (perfect parallel) = 5.95% of the original time, compared to 100%/100= 1% of the original time if everything was perfectly parallel. 1/(100*0.0595) ≈ 0.17, so the efficiency is only about 17%, where *efficiency* is (time for 1 worker)/[(number of workers)*(the time they take)]. You can think of this as the wages if one worker does it, divided by the total wages when the group does it. 20 workers would work at about 51% efficiency, and 10 workers at about 69%. The fewer the workers, the higher the efficiency.

- Weakest links: if a group of workers all depend on each other, then the group can go no faster than the slowest worker. If one worker is talking on the cellphone trying to arrange a date, everyone slows down, especially if they are trying to listen in. Other weak links can be the compiler, operating system, communication system, etc.

Why can't you hide?

In the past, when someone needed to solve bigger problems they could often wait a bit, and a faster computer would come out. This was mainly due to *Moore's law*, which people interpreted to mean that the speed of computers would double about every two years. However, this isn't true any longer: if you look at the GHz speed of the processors in a desktop computer 2 years ago, versus the speed now, it has barely, if any, increased. Basically this happened because they can't make reliable low-cost processors that run significantly faster.

What has doubled, however, is the number of processors on a single chip. Some smartphones now have 8 cores (processors), you can buy CPU chips with 12 cores, and this number will soon increase. These are called "multi-core" or "many-core" chips. There are also graphics processing units (GPUs) with over 100 highly specialized processors. This is closer to what Moore was talking about, because what he really said was that the number of transistors would keep doubling. The rate of improvement will slow down, but significant increases in the number of transistors should continue for at least a decade.

If there are only a few cores in your computer then it is pretty easy to find tasks that can be done simultaneously, such as waiting for keystrokes and running a browser, i.e., embarrassingly parallel. However, as the number of processors keeps increasing, the parallelization problems mentioned above become increasingly serious. There are now parallel computers that have > 1,000,000 cores, and people are planning for ones that will have > 100,000,000.

Overall, this means that there is a massive need to make use of the parallelism in multi-core chips for almost any problem, and to use many of these combined together for large problems. Job security for me! (and you, if you learn parallel computing)

Some Resources on Parallel Computing

If you want to learn more about parallel computing, there are some books available, though I don't like most of them. Many colleges and universities teach classes in this subject, and there are some tutorials available. For example, the author teaches a parallel computing class and a tutorial on parallel computing. These are aimed at larger problems, not ones that a single multi-core chip is sufficient for. However, my university (and many others) is developing courses addressing multi-core parallelism. Some of my colleagues have even developed a few teaching modules used in high school and elementary school classes, where the students become the parallel processors. If you attend my tutorial or class you too will become a processor in a parallel computer, and one of your tasks will be to flip a coin. You'll have to attend to see why. Don't forget to bring a coin.

There are also resources available via the web - here are some pointers to parallel computing resources such as manuals, software, parallel computers, etc. Here's a video of an overview I presented at Michigan. The quality isn't very good, so I hope it was better in person.

Some addresses for me:

Quentin F. Stout
Computer Science and Engineering
University of Michigan
Ann Arbor, MI 48109-2121

734-763-1518 (office, messages) 734-763-8094 (fax)
qstout @ umich · edu www.eecs.umich.edu/~qstout/

If you are close enough, "Hi" is an effective address.



Copyright © 2000-2016 Quentin F. Stout

Readers–writers problem

From Wikipedia, the free encyclopedia

In computer science, the **readers-writers problems** are examples of a common computing problem in concurrency. There are at least three variations of the problems, which deal with situations in which many threads try to access the same shared memory at one time. Some threads may read and some may write, with the constraint that no process may access the share for either reading or writing, while another process is in the act of writing to it. (In particular, it *is* allowed for two or more readers to access the share at the same time.) A readers-writer lock is a data structure that solves one or more of the readers-writers problems.

Contents

- 1 First readers-writers problem
- 2 Second readers-writers problem
- 3 Third readers-writers problem
- 4 See also
- 5 References
- 6 External links

First readers-writers problem

Suppose we have a shared memory area with the basic constraints detailed above. It is possible to protect the shared data behind a mutual exclusion mutex, in which case no two threads can access the data at the same time. However, this solution is suboptimal, because it is possible that a reader R_1 might have the lock, and then another reader R_2 requests access. It would be foolish for R_2 to wait until R_1 was done before starting its own read operation; instead, R_2 should start right away. This is the motivation for the **first readers-writers problem**, in which the constraint is added that *no reader shall be kept waiting if the share is currently opened for reading*. This is also called **readers-preference**, with its solution:

```
semaphore resource=1;
semaphore mutex=1;
readcount=0;

/* Please note that:
   resource.P() is equivalent to wait(resource)
```

```
   resource.V() is equivalent to signal(resource)
   mutex.P() is equivalent to wait(mutex)
   mutex.V() is equivalent to signal(mutex)
*/

writer() {
    <ENTRY Section>
    resource.P();//Lock the shared file for a writer

    <CRITICAL Section>
    // Writing is done

    <EXIT Section>
    resource.V();//Release the shared file for use by other readers. Writers are allowed if there are no readers requesting it.
}

reader() {
    <ENTRY Section>
    mutex.P();//Ensure that no other reader can execute the <Entry> section while you are in it
        readcount++;//Indicate that you are a reader trying to enter the Critical Section
        if (readcount == 1)//Checks if you are the first reader trying to enter CS
            resource.P();//If you are the first reader, lock the resource from writers. Resource stays reserved for subsequent readers
    mutex.V();//Release <Entry> Section. Let other readers enter the <Entry> section, now that you are done with it.

    <CRITICAL Section>
    // Do the Reading
    // (Critical Section Area)

    <EXIT Section>
    mutex.P();//Ensure that no other reader can execute the <Exit> section while you are in it
        readcount--;//Indicate that you are no longer needing the shared resource. One less readers
        if (readcount == 0)//Checks if you are the last (only) reader who is reading the shared file
            resource.V();//If you are last reader, then you can unlock the resource. This makes it available to writers.
    mutex.V();//Let other readers enter the <Exit> section, now that you are done with it.
}
```

In this solution of the readers/writers problem, the first reader must lock the resource(shared file) if such is available. Once the file is locked from writers, it may be used by many subsequent readers without having them to re-lock it again.

Before entering the CS, every new reader must go through the entry section. However, there may only be a single reader in the entry section at a time. This is done to avoid race conditions on the readers (e.g. two readers increment the readcount at the same time, so noone feels entitled to lock the resource from writers). To accomplish this, every reader which enters the <ENTRY Section> will lock the <ENTRY Section> for themselves until they are done with it. Please note that at this point the readers are not locking the resource. They are only locking the entry section so no other reader can enter it while they are in it. Once the reader is done executing the entry section, it will unlock it by signalling the mutex semaphore. Signalling it is equivallent to: mutex.V() in the above code. Same is valid for the <EXIT Section>. There can be no more than a single reader in the exit section at a time, therefore, every reader must claim and lock the Exit section for themselves before using it.

Once the first reader is in the entry section, it will lock the resource. Doing this will prevent any writers from accessing it. Subsequent readers can just utilize the locked (from writers) resource. The very last reader (indicated by the readcount variable) must unlock the resource, thus making it available to writers.

In this solution, every writer must claim the resource individually. This means that a stream of readers can subsequently lock all potential writers out and starve them. This is so, because after the first reader locks the resource, no writer can lock it, before it gets released. And it will only be released by the very last reader. Hence, this solution does not satisfy fairness.

Second readers-writers problem

The first solution is suboptimal, because it is possible that a reader R_1 might have the lock, a writer W be waiting for the lock, and then a reader R_2 requests access. It would be unfair for R_2 to jump in immediately, ahead of W; if that happened often enough, W would starve. Instead, W should start as soon as possible. This is the motivation for the **second readers-writers problem**, in which the constraint is added that *no writer, once added to the queue, shall be kept waiting longer than absolutely necessary*. This is also called **writers-preference**.

A solution to the writers-preference scenario is:.[1]

```
int readcount, writecount; //(initial value = 0)
semaphore rmutex, wmutex, r_entry, readTry, resource; //(initial value = 1)

//READER
<ENTRY Section>
  r_entry.P();
    readTry.P();//Indicate a reader is trying to enter
      rmutex.P();//lock entry section to avoid race condition with other readers
        readcount++;//report yourself as a reader
        if (readcount == 1)//checks if you are first reader
            resource.P();//if you are first reader, lock the resource
      rmutex.V();//release entry section for other readers
    readTry.V();//indicate you are done trying to access the resource
  r_entry.V();

<CRITICAL Section>
 // reading is performed

<EXIT Section>
  rmutex.P();//reserve exit section - avoids race condition with readers
    readcount--;//indicate you're leaving
    if (readcount == 0)//checks if you are last reader leaving
        resource.V();//if last, you must release the locked resource
  rmutex.V();//release exit section for other readers
```

```
//WRITER
<ENTRY Section>
  wmutex.P();//reserve entry section for writers - avoids race conditions
    writecount++;//report yourself as a writer entering
    if (writecount == 1)//checks if you're first writer
        readTry.P();//if you're first, then you must lock the readers out. Prevent them from trying to enter CS
  wmutex.V();//release entry section

<CRITICAL Section>
  resource.P();//reserve the resource for yourself - prevents other writers from simultaneously editing the shared resource
   // writing is performed
  resource.V();//release file

<EXIT Section>
  wmutex.P();//reserve exit section
    writecount--;//indicate you're leaving
    if (writecount == 0)//checks if you're the last writer
        readTry.V();//if you're last writer, you must unlock the readers. Allows them to try enter CS for reading
  wmutex.V();//release exit section
```

In this solution, preference is given to the writers. This is accomplished by forcing every reader to lock and release the readtry semaphore individually. The writers on the other hand don't need to lock it individually. Only the first writer will lock the readtry and then all subsequent writers can simply use the resource as it gets freed by the previous writer. The very last writer must release the readtry semaphore, thus opening the gate for readers to try reading.

No reader can engage in the entry section if the readtry seamphore has been set by a writer previously. The reader must wait for the last writer to unlock the resource and readtry semaphores. On the other hand, if a particular reader has locked the readtry seamphore, this will indicate to any potential concurrent writer that there is a reader in the entry section. So the writer will wait for the reader to release the readtry and then the writer will immediately lock it for itself and all subsequent writers. However, the writer will not be able to access the resource until the current reader has released the resource, which only occurs after the reader is finished with the resource in the critical section.

The resource semaphore can be locked by both the writer and the reader in their entry section. They are only able to do so after first locking the readtry semaphore, which can only be done by one of them at a time.

If there are no writers wishing to get to the resource, as indicated to the reader by the status of the readtry semaphore, then the readers will not lock the resource. This is done to allow a writer to immediately take control over the resource as soon as the current reader is finished reading. Otherwise, the writer would need to wait for a queue of readers to be done before the last one can unlock the readtry semaphore. As soon as a writer shows up, it will try to set the readtry and hang up there waiting for the current reader to release the readtry. It will then take control over the resource as soon as the current reader is done reading and lock all future readers out. All subsequent readers will hang up at the r_entry semaphore waiting for the writers to be finished with the resource and to open the gate by releasing readtry.

The rmutex and wmutex are used in exactly the same way as in the first solution. Their sole purpose is to avoid race conditions on the readers and writers while they are in their entry or exit sections.

Third readers-writers problem

In fact, the solutions implied by both problem statements can result in starvation — the first readers-writers problem may starve writers in the queue, and the second readers-writers problem may starve readers. Therefore, the **third readers-writers problem** is sometimes proposed, which adds the constraint that *no thread shall be allowed to starve*; that is, the operation of obtaining a lock on the shared data will always terminate in a bounded amount of time. A solution with fairness for both readers and writers might be as follows:

```
int readcount; // (initial value = 0)
semaphore mutex_rdcnt, r, w; // ( initial value = 1 )

//READER
        wait(r);
          wait(mutex_rdcnt);
            readcount++;
            if (readcount == 1)
              wait(w);
          signal(mutex_rdcnt);
        signal(r);

        // reading is performed

        wait(mutex_rdcnt);
          readcount--;
          if (readcount == 0)
            signal(w);
        signal(mutex_rdcnt);

//WRITER
        wait(r);
        wait(w);

         // writing is performed

        signal(w);
        signal(r);
```

Note also that this solution can only satisfy the condition that "no thread shall be allowed to starve" if and only if semaphores preserve first-in first-out ordering when blocking and releasing threads. Otherwise, a blocked writer, for example, may remain blocked indefinitely with a cycle of other writers decrementing the semaphore before it can.

See also

- ABA problem
- Producers-consumers problem
- Dining philosophers problem
- Cigarette smokers problem
- Sleeping barber problem
- Readers–writer lock

References

1. Communications of the ACM :Concurrent Control with "Readers" and "Writers" P.J. Courtois,* F. H, 1971 [1] (http://cs.nyu.edu/~lerner/spring10/MCP-S10-Read04-ReadersWriters.pdf)

- Morris JM (1979) A starvation-free solution to the mutual exclusion problem. Inf Process Lett 8:76–80
- Fair Solution to the Reader-Writer-Problem with Semaphores only. H. Ballhausen, 2003 [2] (http://arxiv.org/pdf/cs/0303005.pdf)
- Faster Fair Solution for the Reader-Writer Problem. V.Popov, O.Mazonka 2013 [3] (http://arxiv.org/ftp/arxiv/papers/1309/1309.4507.pdf)

External links

- Algorithmic description of the third readers-writers problem (http://www.rfc1149.net/blog/2011/01/07/the-third-readers-writers-problem/)

Retrieved from "https://en.wikipedia.org/w/index.php?title=Readers–writers_problem&oldid=670594786"

Categories: Concurrency (computer science)

- This page was last modified on 8 July 2015, at 23:35.
- Text is available under the Creative Commons Attribution-ShareAlike License; additional terms may apply. By using this site, you agree to the Terms of Use and Privacy Policy. Wikipedia® is a registered trademark of the Wikimedia Foundation, Inc., a non-profit organization.

Automatic Refinement of Split Binary Semaphore (extended version)

Damián Barsotti and Javier O. Blanco

Fa.M.A.F., Universidad Nacional de Córdoba,
Córdoba 5000, Argentina

Abstract. Binary semaphores can be used to implement conditional critical regions by using the split binary semaphore (SBS) technique. Given a specification of a conditional critical regions problem, the SBS technique provides not only the resulting programs but also some invariants which ensure the correctness of the solution. The programs obtained in this way are generally not efficient. However, they can be optimized by strengthening these invariants and using them to eliminate unnecessary tests.

We present a mechanical method to perform these optimizations. The idea is to use the backward propagation technique over a guarded transition system that models the behavior of the programs generated by the SBS. This process needs proving heavy implications and simplifying growing invariants. Our method automatically entrusts these tasks to the Isabelle theorem prover and the CVC Lite validity checker. We have tested our method on a number of classical examples from concurrent programming.

1 Introduction

Split Binary Semaphores (SBS) are used to implement conditional critical regions [4, 11]. Since the method is fairly general, the solutions obtained often perform some unnecessary tests when leaving a critical region. This work focuses on the mechanical elimination of these unnecessary tests in the programs.

The SBS technique provides not only the programs implementing conditional critical regions, but also some invariants which ensure its correctness. Starting from them, we find stronger invariants from which it can be automatically deduced that certain tests will always be false and hence can be eliminated.

In order to achieve these optimizations, we model programs as guarded transition systems, for which many invariant generation techniques are well-known. In this work we mainly use the backward propagation technique [3] which provides invariants that are quantifier-free formulae. The proofs involved in their manipulation can be dealt with fully automated provers[1] such as CVC Lite [2].

[1] Some provers (like CVC Lite and Simplify) include some support for first-order quantifiers elimination. Nevertheless, these provers work better and faster on quantifier-free formulae since support for quantifiers is not complete.

This validity checker also supports several interpreted theories (including rational and integer linear arithmetic, arrays, tuples, etc.) which are appropriate for obtaining assertions on programs.

The propagation techniques (backward and forward) are based on the calculus of a fixed point of a formula transformer. One of the advantages of the backward propagation technique is that the sequence of approximations is usually finite. Unfortunately, the formulae produced during this process are large. We use some simplification techniques implemented in CVC Lite and in the Isabelle theorem prover [10] to tackle this problem. Isabelle is an interactive theorem prover that offers efficient tactics for simplification. Besides, we implement other simplification methods using CVC Lite.

The paper is structured as follows. In Sec. 2 we explain the SBS technique. In Sec. 3 we presents an example on the use of this technique. Sec. 4 and 5 present the theoretical framework for constructing the guarded transition system and the method for obtaining the invariants. The guarded transition system that results from the programs generated by SBS technique is developed in Sec. 6 and the refinement procedure in Sec. 7. In Sec. 8 we show some optimizations applied on the refinement procedure. Finally, we show some examples in Sec. 9 and expose our conclusions and suggestions for further work in Sec. 10.

2 Split Binary Semaphores

It is well-known that binary semaphores can easily ensure mutual exclusion and are suitable to implement critical regions. Moreover, they can be used in a systematic way to implement conditional critical regions. We briefly present the solution and refer to the literature for a justification of the method [4, 1, 11, 9].

A set $\{s_0, \ldots, s_n\}$ of binary semaphores is called a *split binary semaphore* if at any time at most one of them equals 1, i.e. the following property is a global invariant for the (multi)program:

$$0 \leq \langle \sum i : 0 \leq i \leq n : s_i \rangle \leq 1 \ .$$

In the execution of the program, every critical region begins with a P on some semaphore, and ends with a V operation on some semaphore (not necessarily the same one). Hence, the invariant ensures mutual exclusion between *any* pair P-V.

Besides ensuring mutual exclusion, SBS satisfies the following *domino rule* [9]. In an execution of the program, if the last V operation is over semaphore s, then the next P operation will be on the same semaphore (notice that there may be more than one P on s). This means that the precondition of a V operation can be taken as the postcondition of any corresponding P operation since shared variables can only be modified inside critical sections. This rule can be formulated as the global invariant $\varphi_{SBS} : \langle \forall s :: s = 0 \vee I_s \rangle$ where I_s is the assertion that holds before every V.s statement and after every corresponding P.s statement.

In order to implement conditional critical regions, the SBS can be used in the following way. To every different condition, a binary semaphore of the SBS

is associated, and another "neutral" semaphore is used for the case in which no condition holds. Then, every critical region will be prefixed by a P operation on the semaphore associated with its precondition. Some care should be taken to introduce V operations to ensure progress. We illustrate the method with two conditional critical regions.

Suppose we want to execute statements $\mathrm{S_o}$, $\mathrm{S_1}$ atomically under conditions B_0, B_1 respectively. Furthermore, the critical regions must preserve a global invariant I. We use binary semaphores s_0, s_1 for each condition and two counters b_0, b_1 which will be used to count the number of processes committed to execute P.s_0, P.s_1 respectively. These counters are necessary to avoid deadlocks. Another semaphore m is used for the case in which no condition holds or there is no process waiting.

The following invariant characterizes the SBS solution for the critical region problem.

$$\begin{aligned}\varphi_{SBS}: \; & (s_0 = 0 \vee (B_0 \wedge 0 < b_0 \wedge I)) \wedge \\ & (s_1 = 0 \vee (B_1 \wedge 0 < b_1 \wedge I)) \wedge \\ & (m = 0 \vee ((\neg B_0 \vee 0 = b_0) \wedge (\neg B_1 \vee 0 = b_1) \wedge I)) \;\; .\end{aligned}$$

Fig. 1 shows the (fully annotated) component obtained by the application of the SBS technique for the atomic execution of $\mathrm{S_o}$.

$\underline{\mathrm{SCC_o}}$
P.m
$\{I \wedge (\neg B_0 \vee b_0 = 0) \wedge (\neg B_1 \vee b_1 = 0)\}$
$b_0 := b_0 + 1$
$\{I \wedge (\neg B_0 \vee b_0 = 1) \wedge (\neg B_1 \vee b_1 = 0 \wedge b_0 > 0\}$
$\underline{\text{if}}$ $B_0 \rightarrow \{I \wedge B_0 \wedge b_0 > 0\}$ V.s_0
$\square \neg B_0 \rightarrow \{I \wedge (\neg B_0 \vee b_o = 0) \wedge (\neg B_1 \vee b_1 = 0)\}$ V.m
$\underline{\text{fi}}$
P.s_0
$\{I \wedge B_0 \wedge b_0 > 0\}$
$b_0 := b_0 - 1$
$\{I \wedge B_0\}$
$\mathrm{S_o}$
$\{I\}$
$\underline{\text{if}}$ $B_0 \wedge b_0 > 0 \rightarrow \{I \wedge B_0 \wedge b_0 > 0\}$ V.s_0
$\square$ $B_1 \wedge b_1 > 0 \rightarrow \{I \wedge B_1 \wedge b_1 > 0\}$ V.s_1
$\square (\neg B_0 \vee b_0 = 0) \wedge (\neg B_1 \vee b_1 = 0) \rightarrow$
$\{I \wedge (\neg B_0 \vee b_o = 0) \wedge (\neg B_1 \vee b_1 = 0)\}$ V.m
$\underline{\text{fi}}$

Fig. 1. SBS generated component

This solution can be simplified by distributing both assignments to b_0 (the increment can be distributed as well since b_0 does not appear in B_0), and then

simplify the first alternative (the P-V pair and afterwards the complementary assignments). The simplified solution is in Fig. 2.

$\underline{\mathrm{SCC_o}}$
P.m ;
$\{I \wedge (\neg B_0 \vee b_0 = 0) \wedge (\neg B_1 \vee b_1 = 0)\}$
$\underline{\text{if}}$ $B_0 \rightarrow \{I \wedge B_0 \wedge b_0 = 0 \wedge (\neg B_1 \vee b_1 = 0)\}$
skip
$\square$ $\neg B_0 \rightarrow \{I \wedge \neg B_0 \wedge (\neg B_1 \vee b_1 = 0)\}$
$b_0 := b_0 + 1$;
$\{I \wedge (\neg B_0 \vee b_0 = 0) \wedge (\neg B_1 \vee b_1 = 0)\}$
V.m ;
P.s_0 ;
$\{I \wedge B_0 \wedge b_0 > 0\}$
$b_0 := b_0 - 1$
$\underline{\text{fi}}$;
$\{I \wedge B_0\}$
$\mathrm{S_o}$;
$\{I\}$
$\underline{\text{if}}$ $B_0 \wedge b_0 > 0 \rightarrow \{I \wedge B_0 \wedge b_0 > 0\}$ V.s_0
$\square$ $B_1 \wedge b_1 > 0 \rightarrow \{I \wedge B_1 \wedge b_1 > 0\}$ V.s_1
$\square$ $(\neg B_0 \vee b_0 = 0) \wedge (\neg B_1 \vee b_1 = 0)$
$\rightarrow \{I \wedge (\neg B_0 \vee b_0 = 0) \wedge (\neg B_1 \vee b_1 = 0)\}$ V.m
$\underline{\text{fi}}$

Fig. 2. SBS generated component

3 Bounded Buffer

The following example shows how the simplifications can be manually done. Nevertheless, we apply the simplifications methodically in order to mechanise them later.

Let's consider the classical *Producer/Consumer* problem through a bounded buffer. The component called *producer* produces some elements and send them to the *consumer* component which will use them. Some synchronisation is needed to avoid writing on a full buffer or reading from an empty one. The formal specification of this problem can be stated as follows.

Pre: $p = 0 \wedge d = 0$	
Prod: $*[$ produce.n ; $\{?\ p - d < N\}$; $buf.(p\ mod\ N) := n$; $p := p + 1$ $]$	Cons: $*[$ $\{?\ 0 < p - d\}$; $m := buf.(d\ mod\ N)$; $d := d + 1$; consume.m $]$

For a solution with an SBS the access to the buffer will be performed in a critical region. Whereas this is unnecessary for one reader and one writer, it is

required for multiple readers or writers. In order to avoid irrelevant details, we deal only with the synchronisation, leaving out the actual update of the buffer. It can be introduced easily in the final solution. Furthermore, since we are only interested in the difference between p and d, we transform coordinates to a single integer variable n which should be invariably equal to $p - d$. The problem then becomes symmetric

Pre: $n = 0$	
Prod: $*[\ \{?\ n < N\}$ $\quad ; n := n + 1$ $\quad]$	Cons: $*[\ \{?\ 0 < n\}$ $\quad ; n := n - 1$ $\quad]$

Since the atomic regions are now a single statement, the following global invariant characterises the problem

$$I : 0 \leq n \leq N \ .$$

We use a split binary semaphore with three components, one for each condition and one neutral semaphore to ensure mutual exclusion when the conditions do not hold or there are not waiting processes.

Following the general scheme proposed earlier, the semaphore invariants then become:

$$\begin{aligned} \varphi_{SBS} : \ & (s = 0 \ \vee (n < N \wedge 0 < b)) \wedge \\ & (t = 0 \ \vee (0 < n \wedge 0 < c)) \wedge \\ & (m = 0 \vee ((n = N \vee 0 = b) \wedge (0 = n \vee 0 = c))) \ . \end{aligned}$$

Note that the condition are simplified as much as possible using the global invariant.

Prod

P.m;
if $n < N \rightarrow$ skip
$\Box\ n = N \rightarrow b := b + 1$; V.$m$;
P.s ; $b := b - 1$
fi;
$n := n + 1$;
if $n < N \wedge 0 < b \rightarrow$ V.s
$\Box\ 0 < n \wedge 0 < c \ \rightarrow$ V.t
$\Box\ (n = N \vee 0 = b) \wedge$
$(0 = n \vee 0 = c) \rightarrow$ V.m
fi

Cons

P.m;
if $0 < n \rightarrow$ skip
$\Box\ 0 = n \rightarrow c := c + 1$; V.$m$;
P.t ; $c := c - 1$
fi;
$n := n - 1$;
if $n < N \wedge 0 < b \rightarrow$ V.s
$\Box\ 0 < n \wedge 0 < c \ \rightarrow$ V.t
$\Box\ (n = N \vee 0 = b) \wedge$
$(0 = n \vee 0 = c) \rightarrow$ V.m
fi

This solution can be improved by removing some of the guards which we can prove will never be chosen. This simplify the evaluation of the guards when leaving a critical region, and hence it can greatly improve the performance, since in a typical case a critical region will be small but it will be entered many times.

In this example we elaborate from [6] tailoring our proof to a partial mechanisation.

The first simple observation is that after an item is produced the buffer cannot be empty. In order to use this idea, we propose to show the correctness of the assertion with the question mark

Prod

P.m;
if $n < N \rightarrow$ skip
$\Box\ n = N \rightarrow b := b + 1$; V.$m$; P.s ; $b := b - 1$
fi;
$n := n + 1$;
$\{0 < n?\}$
if $n < N \wedge 0 < b \rightarrow$ V.s
$\Box\ 0 < n \wedge 0 < c \rightarrow$ V.t
$\Box\ (n = N \vee 0 = b) \wedge (0 = n \vee 0 = c) \rightarrow$ V.m
fi

This is easily proved since $wp.(n := n + 1)(0 < n)$ equals $0 \leq n$ which is implied by the global invariant. The guards of the second if can now be simplified. Symmetrically, condition $n < N$ will be a valid preassertion of the if for the consumer.

Prod

P.m;
if $n < N \rightarrow$ skip
$\Box\ n = N \rightarrow b := b + 1$; V.$m$;
P.s ; $b := b - 1$
fi;
$n := n + 1$;
$\{0 < n\}$
if $n < N \wedge 0 < b \rightarrow$ V.s
$\Box\ 0 < c \rightarrow$ V.t
$\Box\ (n = N \vee 0 = b) \wedge 0 = c \rightarrow$ V.m
fi

Cons

P.m;
if $0 < n \rightarrow$ skip
$\Box\ 0 = n \rightarrow c := c + 1$; V.$m$;
P.t ; $c := c - 1$
fi;
$n := n - 1$;
$\{n < N\}$
if $0 < b \rightarrow$ V.s
$\Box\ 0 < n \wedge 0 < c \rightarrow$ V.t
$\Box\ 0 = b \wedge (0 = n \vee 0 = c) \rightarrow$ V.m
fi

Looking operationally at the program, it seems unlikely that a producer may release another producer. This implies that it is possible that the first guard of the final if in the producer can be eliminated. Actually, a more involved argument is needed to fully justify this, but one advantage of the method advocated here is that if we believe that certain property may hold we can try it out and see what we can achieve. The process that we will describe now will be fully automatized (with one minor caveat).

In order to simplify this guard we need the assertion with the question mark to hold:

Prod

P.m;
if $n < N \rightarrow$ skip
$\Box$ $n = N \rightarrow b := b + 1$; V.$m$; P.s ; $b := b - 1$
fi;
$n := n + 1$;
$\{0 < n\}\{n = N \vee 0 = b?\}$
if $n < N \wedge 0 < b \rightarrow$ V.s
$\Box$ $0 < c \rightarrow$ V.t
$\Box$ $(n = N \vee 0 = b) \wedge 0 = c \rightarrow$ V.m
fi

Taking the wp we can see that $n + 1 = N \vee 0 = b$ should hold after the first if. In the first branch it follows from $b = 0$. In the second branch we need $n + 1 = N \vee 1 = b$ as a valid postcondition for the P.s. This can be achieved by strengthening the invariant for s to $s = 0 \vee (0 < b \wedge n < N \wedge (n+1 = N \vee 1 = b))$. In [6] a slightly stronger and simpler invariant is used. This one, however, is the weakest it can be obtained and one of the obvious candidates for a mechanical calculation. The other possible candidate is the strongest one: we will pursue this idea in another work (see the conclusions)

We have to show now that this new assertion is indeed a valid invariant. The only place where it can be invalidated is before the $V.s$ in the consumer. Taking wp we can see that the invariant for semaphore t has to be strengthened as well into $t = 0 \vee (0 < c \wedge 0 < n \wedge (b \leq 1 \vee n = N))$. Furthermore, the last guard can be simplified provided that the required condition holds.

The annotated program then becomes

Prod

P.m;
if $n < N \rightarrow$ skip
$\Box$ $n = N \rightarrow b := b + 1$; V.$m$;
P.s ; $b := b - 1$
fi;
$n := n + 1$;
$\{0 < n\}$
if $0 < c \rightarrow$ V.t
$\Box$ $0 = c \rightarrow$ V.m
fi

Cons

P.m;
if $0 < n \rightarrow$ skip
$\Box$ $0 = n \rightarrow c := c + 1$; V.$m$;
P.t ; $c := c - 1$
fi;
$n := n - 1$;
$\{n < N\}$
if $0 < b \rightarrow$ V.s
$\Box$ $0 < n \wedge 0 < c \rightarrow$
$\{b \leq 1 \vee n = N?\}$ V.t
$\Box$ $0 = b \wedge (0 = n \vee 0 = c) \rightarrow$ V.m
fi

The remaining proof obligation can be ignored since we plan to eliminate this branch of the if altogether. This is easily done by ensuring the condition $0 = n \vee 0 = c$ prior to the last if of the consumer. The whole process is symmetrical to the one we have just described. At the end we obtain the following invariant

for the condition semaphores, whereas the invariant for the mutual exclusion m remains unchanged:

$$s = 0 \vee 0 < b \wedge n < N \wedge (n + 1 = N \vee 1 = b) \wedge (c \leq 1 \vee 0 = n)$$
$$t = 0 \vee 0 < c \wedge 0 < n \wedge (1 = n \vee 1 = c) \wedge (b \leq 1 \vee n = N)$$

whereas the whole program (without annotations) becomes

Prod	Cons
P.m;	P.m;
if $n < N \rightarrow$ skip	if $0 < n \rightarrow$ skip
$\Box$ $n = N \rightarrow b := b + 1$; V.$m$;	$\Box$ $0 = n \rightarrow c := c + 1$; V.$m$;
P.s ; $b := b - 1$	P.t ; $c := c - 1$
fi;	fi;
$n := n + 1$;	$n := n - 1$;
if $0 < c \rightarrow$ V.t	if $0 < b \rightarrow$ V.s
$\Box$ $0 = c \rightarrow$ V.m	$\Box$ $0 = b \rightarrow$ V.m
fi	fi

The whole process of simplifying these components is carried out (almost) automatically by our method. The user needs only to choose which are the guards that will be eliminated and the system will calculate the new invariants.

4 Guarded Transition System

The following definitions are taken from [12, 3]. Let Σ be a first-order language containing interpreted symbols for concrete domains like booleans, integers and reals. Let $\mathcal{F}$ be the set of first-order formulas with free (typed) variables contained in a finite set $\mathcal{V} = \{x_1, \cdots, x_n\}$ over Σ. We shall denote the sequence of variables $x_1, \cdots, x_n$ by $\bar{x}$.

The usual way to model reactive systems is by using a *guarded transition system* $\mathcal{S} = \langle \mathcal{V}, \Theta, \mathcal{T} \rangle$ where $\Theta \in \mathcal{F}$ is its *initial condition* and $\mathcal{T}$ is a finite set of *guarded transitions*. Each $\tau \in \mathcal{T}$ can be specified as follows:

$$\gamma_\tau \ \longmapsto \ \bar{x} := \bar{e}_\tau(\bar{x})$$

where $\gamma_\tau \in \mathcal{F}$ is the *transition guard* and $\bar{e}_\tau(\bar{x})$ is a sequence of expressions in Σ whose free variables are taken from $\mathcal{V}$; both sequences should have equal length. The formula γ_τ denotes the condition that should hold in order to execute the transition, $\bar{x} := \bar{e}_\tau(\bar{x})$ is a simultaneous assignment which indicate the transformation of the state produced by the transition.

Usually, a transition system has a control variable $vc \in \mathcal{V}$ which ranges over a finite set $\mathcal{L}$ of *locations*. This variable acts as a program counter and the locations act as source and target of each transition. Hence, any $\tau \in \mathcal{T}$ can be written as:

$$vc = l_\tau \wedge \gamma_\tau \ \longmapsto \ \bar{x} := \bar{e}_\tau(\bar{x}) \, ; vc := l'_\tau$$

where $l_\tau, l'_\tau \in \mathcal{L}$, γ_τ is a predicate with variables in $\mathcal{V}/\{vc\}$ and $\bar{x} := \bar{e}_\tau(\bar{x})$ is a simultaneous assignment with variables in $\mathcal{V}/\{vc\}$. For a given transition τ we define the functions $src(\tau) = l_\tau$ and $tgt(\tau) = l'_\tau$.

For each transition, we define a predicate Φ_τ as follows:

$$\Phi_\tau(\bar{x}, \bar{x}') \triangleq \gamma_\tau(\bar{x}) \wedge \bar{x}' = \bar{e}_\tau(\bar{x})$$

where $\bar{x}'$ is the renaming of variables x_i in $\bar{x}$ as x'_i. Note that Φ_τ is a formula with free variables in $\mathcal{V} \cup \mathcal{V}'$ where $\mathcal{V}'$ is the set of variables obtained by renaming the variables x to x' in $\mathcal{V}$. These predicates denote the relation between the values of the variables before execution of the transitions (in a given location $src(\tau)$), with the values of the same variables after its execution (in location $tgt(\tau)$). Hence, we can define the *transition predicate* Φ of a guarded transition system $\mathcal{S}$ as follows:

$$\Phi(\bar{x}, vc, \bar{x}', vc') \triangleq \bigvee_{\tau \in \mathcal{T}} vc = src(\tau) \wedge vc' = tgt(\tau) \wedge \Phi_\tau(\bar{x}, \bar{x}') \ .$$

The formula transformer *weakest precondition* [3, 12] of a transition τ, denoted by $\mathrm{wp}\,(\Phi_\tau)$, is defined as follows:

$$\mathrm{wp}\,(\Phi_\tau)(\varphi(\bar{x})) \triangleq \forall \bar{x}'. (\Phi_\tau(\bar{x}, \bar{x}') \rightarrow \varphi(\bar{x}')) \ .$$

The universal quantifier can be eliminated using substitution:

$$\mathrm{wp}\,(\Phi_\tau)(\varphi(\bar{x})) = \gamma_\tau(\bar{x}) \rightarrow \varphi(\bar{e}_\tau(\bar{x})) \ . \tag{1}$$

In general, the weakest precondition transformer for the whole transition system $\mathcal{S}$, denoted by $\mathrm{WP}\,(\Phi)$, is defined as follows:

$$\mathrm{WP}\,(\Phi)(\phi(\bar{x}, vc)) \triangleq \forall\, \bar{x}', vc'. (\Phi(\bar{x}, vc, \bar{x}', vc') \rightarrow \phi(\bar{x}', vc')) \ .$$

Since the set of location $\mathcal{L}$ is finite, the predicate ϕ can be written as:

$$\phi(\bar{x}, vc) = \bigwedge_{l \in \mathcal{L}} vc = l \rightarrow \phi(\bar{x}, l) \ . \tag{2}$$

If we define the predicates $\phi_l(\bar{x}) = \phi(\bar{x}, l)$ for each location $l \in \mathcal{L}$ then the weakest precondition of the whole system is equivalent to:

$$\mathrm{WP}\,(\Phi)(\phi(\bar{x}, vc)) = \bigwedge_{\tau \in \mathcal{T}} vc = src(\tau) \rightarrow \mathrm{wp}\,(\Phi_\tau)(\phi_{tgt(\tau)}(\bar{x})) \tag{3}$$

or, using (1):

$$\mathrm{WP}\,(\Phi)(\phi(\bar{x}, vc)) = \bigwedge_{\tau \in \mathcal{T}} (vc = src(\tau) \wedge \gamma_\tau(\bar{x})) \rightarrow \phi_{tgt(\tau)}(\bar{e}_\tau(\bar{x})) \ . \tag{4}$$

5 Invariants

Let $\mathcal{S} = \langle \mathcal{V}, \Theta, \mathcal{T} \rangle$ be a guarded transition system and $\mathfrak{R}$ a first-order theory over the language Σ. A formula $\varphi \in \mathcal{F}$ is an *inductive invariant* if $\mathfrak{R} \models \Theta \rightarrow \varphi$ and $\mathfrak{R} \models \varphi \rightarrow \mathrm{WP}\,(\Phi)(\varphi)$, with Φ the transition predicate of the guarded transition system. Since the theory $\mathfrak{R}$ is fixed, we shall not mention it explicitly when we talk about satisfiability and validity in $\mathfrak{R}$. Thus, validity in $\mathfrak{R}$ is denoted by $\models$.

A formula $\phi \in \mathcal{F}$ is an invariant if there exists an inductive invariant φ such that $\models \varphi \rightarrow \phi$. This characterization is sound and relative complete[2].

For a monotone formula transformer $\Gamma : \mathcal{F} \mapsto \mathcal{F}$, such as WP, we write the greatest fixed point as $\nu X.\Gamma(X)$. Its meaning is the usual [3].

Given a formula ϕ, we define the monotone formula transformer $\mathcal{B}$ by

$$\mathcal{B}(Y) \triangleq \phi \wedge \mathrm{WP}\,(\Phi)(Y) \ . \tag{5}$$

The greatest fixed point $\varphi_{\mathcal{B}} : \nu X.\mathcal{B}(X)$ provides the weakest formula $\varphi_{\mathcal{B}}$ satisfying $\models \varphi_{\mathcal{B}} \rightarrow \phi$ and $\models \varphi_{\mathcal{B}} \rightarrow \mathrm{WP}\,(\Phi)(\varphi_{\mathcal{B}})$. Therefore, if $\models \Theta \rightarrow \varphi_{\mathcal{B}}$ then ϕ is an invariant and $\varphi_{\mathcal{B}}$ is an inductive invariant of the system.

Since $\mathcal{B}$ is monotone, if the sequence starting from *True*

$$\underbrace{\mathit{True}}_{\varphi^0} \leftarrow \underbrace{\mathcal{B}(\varphi_0)}_{\varphi^1} \leftarrow \underbrace{\mathcal{B}(\varphi_1)}_{\varphi^2} \leftarrow \cdots \tag{6}$$

converges in finitely many steps, then its limit is $\varphi_{\mathcal{B}}$. From this property we can explore the state space using the *backward propagation technique* [3]: given a candidate invariant ϕ, we can find the fixed point $\varphi_{\mathcal{B}}$ if the sequence converges in finitely many steps. Then, proving $\models \Theta \rightarrow \varphi_{\mathcal{B}}$ we can verify if ϕ is an invariant of the system.

6 SBS as a Transition System

In this section we explain how to obtain a transition system for a set of processes executing the programs generated by the implementation of conditional critical regions using the SBS technique.

There is a common way to construct transition systems which model the behavior of concurrent programs [8]. The main idea is to first obtain a transition system for every individual thread or sequential component (in a standard way) and then construct their parallel composition as a product, relying on the interleaving semantics of concurrency. In this new transition system there is a variable *vc* which ranges over sets of locations instead of a single one. The value of this variable is a subset of $\mathcal{L}$ and it denotes all the locations in which control currently resides.

Note, however, that for SBS programs, the number of threads may grow unboundedly (we only model access to a shared resource, in principle it may be

[2] Completeness is here understood relative to the expressibility of the first-order language.

accessed by many different components). Establishing a bound for these components is unnecessary and artificial. Instead, we construct a different transition system in which the transitions represent coarser atomic actions. This will be possible given certain peculiarities of SBS programs.

Mutual exclusion: as mentioned in Sec. 2 any process executing these programs begins with a P operation and ends with a V operation. Moreover, all the statements between them are executed in mutual exclusion. This is a characteristic of the method: split binary semaphores ensure mutual exclusion between any pair of P and V operations, i.e. at most one semaphore is on at any point of the execution. Mutual exclusion is therefore ensured[3].
From this property it follows that we can consider any sequence of the form $\mathrm{P}.s_i; c_1; \cdots; c_n; \mathrm{V}.s_j$ as atomic. Such sequence will be called section S_p^{ij}. Execution of programs can be seen as the execution of a set $\{S_1^{i_1 j_1}, \cdots, S_n^{i_n j_n}\}$ of such sections.

Domino rule: by the semaphore's semantics we can assert that every V operation is followed by a corresponding P operation applied on the same semaphore, i.e. after a S_p^{ij} ends, only a section S_q^{jk} can begin.

Locality of the variables: the variables used in the SBS programs cannot be modified by other programs, i.e. the system is closed.

The mutual exclusion property implies that the sections S_p^{ij} are executed one-at-a-time for each isolated thread. The possible interleavings among different threads are performed outside these sections, which can be considered atomic. Considering also the other two properties, the behavior of the system can be regarded as the execution of a sequence of sections $S_{p_1}^{i_{p_1} j_{p_1}}; S_{p_2}^{i_{p_2} j_{p_2}}; \cdots; S_{p_n}^{i_{p_n} j_{p_n}}; \cdots$ with $j_{p_k} = i_{p_{k+1}}$. This gives these sort of systems a sequential flavor.

These characteristics allows us to define coarse grained transition systems which are much more suitable for simplification. These systems can be seen as sequential processes with non-deterministic jumps (goto sentences) between V's and P's applied to the same semaphore.

From these facts, each section S_p^{ij} will be modeled as a transition and the locations identify which semaphore is active before and after the section is executed (i.e. s_i and s_j for the section S_p^{ij}). Loosely speaking, each transition will be associated with a sequence of actions executed inside a critical region by a thread.

In the general case, the SBS technique takes as input a set $\{\mathrm{S}_0, \cdots, \mathrm{S}_{m-1}\}$ of programs with its corresponding conditions $\{B_0, \cdots, B_{m-1}\}$, an initial condition Θ' and a global invariant I. Here, m refers to the number of critical regions. For each program S_i the technique generates a component SCC_i implementing its corresponding critical region as shown in Fig. 3.

Only temporarily, we annotate each entry and exit point in a given component SCC_i to identify the sections S_p^{jk} (these labels will not identify nodes of the transition system): the entry points are marked with labels prefixed by "in" and the exit points are prefixed by "out". The former are associated with P operations

[3] For further discussions and an operational consideration, we refer to [4].

$\underline{\text{SCC}_i}$

P.s_m ; (in$_m$)
<u>if</u> B_i $\rightarrow$ skip
$\Box$ $\neg B_i$ $\rightarrow$ $b_i := b_i + 1$;
V.s_m ; (out'$_m$)
P.s_i ; (in$_i$)
$b_i := b_i - 1$
<u>fi</u>;
S$_i$;

<u>if</u> $B_0 \wedge b_0 > 0$	$\rightarrow$	V.s_0	(out$_0$)
$\vdots$		$\vdots$	$\vdots$
$\Box$ $B_{m-1} \wedge b_{m-1} > 0$	$\rightarrow$	V.$s_{(m-1)}$	(out$_{m-1}$)
$\Box$ $\bigwedge_{0 \leq j < m} \neg B_j \vee b_j = 0$	$\rightarrow$	V.s_m	(out$_m$)

<u>fi</u>

Fig. 3. Component with labels

and the later with V operations. The subscripts of the labels indicate which is the semaphore involved. For example, a section (S_p^{mm}) starting with in$_m$ may end in out'$_m$ (if condition $\neg B_i$ holds) or in one of the out$_j$, with $j \in \{0, \cdots, m\}$ (if B_i holds). On the other hand, the sections (S_p^{ij} with $j \in \{0, \cdots, m\}$) starting in in$_i$ should end in one of the exit points out$_j$ (if its corresponding guard is on).

Given the shape of the guarded transition systems, we need to calculate the condition under which a trace can be executed. This can be done by propagating the classical weakest precondition transformer [5] for every guard inside a section S_p^{jk} to the beginning of the section. We enumerate all possible transitions:

- From (in$_m$) to (out'$_m$): Each transition is executed if the condition $\neg B_i$ holds and the state changes only with the increment of b_i. Hence, we have the following m transitions (one for each component SCC$_i$):

$$vc = s_m \wedge \neg B_i \;\longmapsto\; b_i := b_i + 1 \,;\, vc := s_m$$

  with $i \in \{0, \cdots, m-1\}$.
- From (in$_m$) to (out$_j$): These transitions are executed if B_i holds and $B_j \wedge b_j > 0$ holds in the final guarded command. To calculate γ we apply wp on the guard $B_j \wedge b_j > 0$. We have then $m \times m$ transitions as follows:

$$vc = s_m \wedge B_i \wedge \text{wp}\,(S_i)(B_j \wedge b_j > 0) \;\longmapsto\; S_i \,;\, vc := s_j$$

  with $i \in \{0, \cdots, m-1\}$, $j \in \{0, \cdots, m-1\}$.
- From (in$_m$) to (out$_m$): These transitions execute the same program as before but only when the last guard of the final guarded command holds for each SCC$_i$. To calculate the guard γ we apply again wp but this time on guard $\bigwedge_{0 \leq j < m} \neg B_j \vee b_j = 0$:

$$vc = s_m \wedge B_i \wedge \text{wp}\,(S_i)(\textstyle\bigwedge_{0 \leq j < m} \neg B_j \vee b_j = 0) \;\longmapsto\; S_i \,;\, vc := s_m$$

  with $i \in \{0, \cdots, m-1\}$.

– From (in_i) to (out_j): These transitions are executed when $B_j \wedge b_j > 0$ holds in the last guarded command. To calculate the guard for the transition we apply wp , using as program the decrement of b_i followed by program S_i, on the guard $B_j \wedge b_j > 0$. We obtain then $m \times m$ transitions as follows:

$$\begin{array}{c} vc = s_i \wedge \text{wp}\,(b_i := b_i - 1; S_i)(B_j \wedge b_j > 0) \\ \longmapsto\ b_i := b_i - 1\,;S_i\,;vc := s_j \end{array}$$

with $i \in \{0, \cdots, m-1\}$, $j \in \{0, \cdots, m-1\}$.

– From (in_i) to (out_m): These transitions are analogous to the ones considered in the previous item, but are executed when the last guard of the final guarded command holds for each SCC_i. We now apply wp on these guards:

$$\begin{array}{c} vc = s_i \wedge \text{wp}\,(b_i := b_i - 1; S_i)(\bigwedge_{0 \le j < m} \neg B_j \vee b_j = 0) \\ \longmapsto\ b_i := b_i - 1\,;S_i\,;vc := s_m \end{array}$$

with $i \in \{0, \cdots, m-1\}$.

These transitions form the set $\mathcal{T}$ of a transition system $\mathcal{S} = \langle \mathcal{V}, \Theta, \mathcal{T} \rangle$ that model the behavior of the SBS-based process. $\mathcal{V}$ is the set of program variables in $\{\text{S}_0, \cdots, \text{S}_{m-1}\}$ adding the auxiliary variables $\{b_0, \cdots, b_{m-1}\}$ and the control variable vc. The initial condition Θ is:

$$\Theta : vc = s_m \wedge (\textstyle\bigwedge_{0 \le i < m} b_i = 0) \wedge \Theta' \ .$$

The SBS technique also provides an inductive invariant for the system:

$$\varphi_{SBS} : \bigwedge_{0 \le i \le m} vc = s_i \rightarrow \varphi_{s_i} \tag{7}$$

with

$$\begin{array}{l} \varphi_{s_i} : B_i \wedge b_i > 0 \wedge I \qquad , 0 \le i < m \\ \varphi_{s_m} : (\bigwedge_{0 \le j < m} \neg B_j \vee b_j = 0) \wedge I \ . \end{array} \tag{8}$$

As an example we show how to obtain the transition system corresponding to the Producer/Consumer example (Sec. 3). In Fig. 4 we enumerate the lines of the producer program (without any invariant simplification) generated by the SBS technique in order to identify all possible transitions.

From de P operation at line 0 we have four sections that ends at the V operations at lines 2, 5, 6 and 7. We model each one with a transition as in equation 4. The guards γ_τ are obtained finding out the conditions that should hold in order to execute the corresponding section. For example, if the section that ends at line 2 is executed, then, at the begining of this execution, the condition guard $\neg n < N$ in line 2 must hold. Then, this condition is the guard γ_τ for the first transition. Moreover, the simultaneous assignements of the transitions are obtained directly by calculation of the state change produced by the

Prod

0: P.m;
1: if $n < N \rightarrow$ skip
2: $\square\ \neg n < N \rightarrow b := b + 1$; V.$m$; P.s ; $b := b - 1$
3: fi;
4: $n := n + 1$;
5: if $n < N \wedge b > 0 \rightarrow$ V.s
6: $\square\ n > 0 \wedge c > 0 \rightarrow$ V.t
7: $\square\ (\neg n < N \vee b = 0) \wedge (\neg n > 0 \vee c = 0) \rightarrow$ V.m
8: fi

Fig. 4. Producer component

correspondig section, and the locations are represented by the initial semaphore in the P operation and the final one in the ending V operation. In this case the only assignement is $b := b+1$ and the initial and final locations is the semaphore m . Then, the transition is:

$$vc = m \wedge \neg n < N \longmapsto b := b + 1; vc := m \ .$$

We follow with the three next sections begining at line 0. In order to execute them the guard $n < N$ in the line 1 must hold. Moreover, each section ends by executing one test at the final if sentence (line 5). The transitions corresponding to each of the alternatives should be executed only if its final condition holds at the end of the execution. Since the definition of the transition systems require that the guard for each transition is known in advance, we back propagate them with the classical *weakest precondition* transformer ([5]) over the section. The simultaneous assignements of the transitions is the sentence in the line 4 and the locations are the semaphores involved in each trace. For example, for the trace that end at the V operation in the line 5 the transitions is:

$$vc = m \wedge n < N \wedge wp.\, n := n + 1.\, (n < N \wedge b > 0) \longmapsto n := n + 1; vc := s \ .$$

Calculating the result of the wp transformer and simplificating the guard, this transitions can be writed as:

$$vc = m \wedge n + 1 < N \wedge b > 0 \longmapsto n := n + 1; vc := s \ .$$

The rest of transitions starting at line 0 are calculated in the same way.

There are also three sections starting at the P operation in line 2 an ending at the V operations in lines 5, 6 and 7. Each one is represent by transitions begining at location s and ending at locations s, t and m. The guard part of these traces are obtained back propagating the guards of the final if sentence as described above. These traces execute the assignements in the lines 2 and 4 and will be modelled as a single simultaneous assignement. For example, the trace ending at line 6 is modelled as the folowing transition:

$$vc = s \wedge wp.\, b, n := b - 1, n + 1.\, (n > 0 \wedge c > 0) \longmapsto b, n := b - 1, n + 1; vc := t$$

and by calculating the result of the *wp* transformer we get

$$vc = s \wedge n + 1 > 0 \wedge c > 0 \longmapsto b, n := b - 1, n + 1; vc := t \ .$$

The transitions from the consumer program are obtained analogously. Figures 5 and 6 show all the resulting transitions.

Producer transitions:

$$\begin{aligned}
&\tau_1 : vc = m \wedge \neg n < N \longmapsto b := b + 1; vc := m \ , \\
&\tau_2 : vc = m \wedge n + 1 < N \wedge b > 0 \longmapsto n := n + 1; vc := s \ , \\
&\tau_3 : vc = m \wedge n < N \wedge n + 1 > 0 \wedge c > 0 \longmapsto n := n + 1; vc := t \ , \\
&\tau_4 : vc = m \wedge n < N \wedge (\neg n + 1 < N \vee b = 0) \wedge (\neg n + 1 > 0 \vee c = 0) \\
&\qquad \longmapsto n := n + 1; vc := m \ , \\
&\tau_5 : vc = s \wedge n + 1 < N \wedge b - 1 > 0 \longmapsto b, n := b - 1, n + 1; vc := s \ , \\
&\tau_6 : vc = s \wedge n + 1 > 0 \wedge c > 0 \longmapsto b, n := b - 1, n + 1; vc := t \ , \\
&\tau_7 : vc = s \wedge (\neg n + 1 < N \vee b - 1 = 0) \wedge (\neg n + 1 > 0 \vee c = 0) \\
&\qquad \longmapsto b, n := b - 1, n + 1; vc := m
\end{aligned}$$

Consumer transitions:

$$\begin{aligned}
&\tau_8 : \ vc = m \wedge \neg n > 0 \longmapsto c := c + 1; vc := m \ , \\
&\tau_9 : \ vc = m \wedge n > 0 \wedge n - 1 < N \wedge b > 0 \longmapsto n := n - 1; vc := s \ , \\
&\tau_{10} : vc = m \wedge n - 1 > 0 \wedge c > 0 \longmapsto n := n - 1; vc := t \ , \\
&\tau_{11} : vc = m \wedge n > 0 \wedge (\neg n - 1 < N \vee b = 0) \wedge (\neg n - 1 > 0 \vee c = 0) \\
&\qquad \longmapsto n := n - 1; vc := m \ , \\
&\tau_{12} : vc = t \wedge n - 1 < N \wedge b > 0 \longmapsto c, n := c - 1, n - 1; vc := s \ , \\
&\tau_{13} : vc = t \wedge n - 1 > 0 \wedge c - 1 > 0 \longmapsto c, n := c - 1, n - 1; vc := t \ , \\
&\tau_{14} : vc = t \wedge (\neg n - 1 < N \vee b = 0) \wedge (\neg n - 1 > 0 \vee c - 1 = 0) \\
&\qquad \longmapsto c, n := c - 1, n - 1; vc := m
\end{aligned}$$

Fig. 5. Bounded Buffer transitions.

These transitions form the set $\mathcal{T}$ of a transition system $\mathcal{S} = \langle \mathcal{V}, \Theta, \mathcal{T} \rangle$ which models the behavoir of the producer and consumer processes. $\mathcal{V}$ is the set of program variables $\{n, N, b, c, vc\}$. The initial condition Θ is $vc = m \wedge b = 0 \wedge c = 0 \wedge \Theta'$, where Θ' is the initial condition of the problem $n = 0 \wedge N > 0$.

The SBS technique also provides an inductive invariant for the system:

$$\begin{aligned}
\varphi_{SBS} : \ &(vc = s \ \ \rightarrow n < N \wedge b > 0 \wedge n \geq 0) \wedge \\
&(vc = t \ \ \rightarrow n > 0 \wedge c > 0 \wedge n \leq N) \wedge \\
&(vc = m \rightarrow (\neg n < N \vee b = 0) \wedge (\neg n > 0 \vee c = 0) \wedge 0 \leq n \leq N) \ .
\end{aligned}$$

7 Automatic Refinement

We describe the process that automatically performs the required simplifications. In particular, we focus on the elimination of some guards in the final conditional statement of each component.

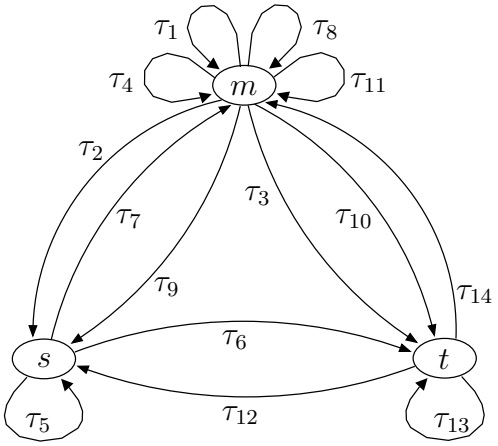


Fig. 6. Bounded Buffer diagram.

To check the possibility of elimination of a given guard, we use the backward propagation technique on a candidate invariant ϕ (5) which models the impossibility of the execution of this guard while the correction of the critical regions is preserved.

For example, if we want to eliminate guard $B_k \wedge b_k > 0$ (for some $k \in \{0, \cdots, m-1\}$) in program SCC_i (Fig. 3) we strengthen the invariant to ensure that any transition with source (in_m) and target (out_k) or with source (in_i) and target (out_k), is never executed. This is achieved by strengthening the inductive invariant φ_{SBS} (7) with the propagation (using wp) of the negation of the guard for each program SCC_i:

$$\phi : \varphi_{SBD} \wedge F_m \wedge F_i \tag{9}$$

with $\quad F_m : vc = s_m \wedge B_i \;\rightarrow\; \text{wp}\,(S_i)(\neg B_k \vee b_k = 0)$
$\quad F_i \;\; : vc = s_i \;\rightarrow\; \text{wp}\,(b_i := b_i - 1; S_i)(\neg B_k \vee b_k = 0)$

the strengthening of the transitions with source (in_m) and target (out_k), and with source (in_i) and target (out_k) respectively.

Since we have a finite number of locations, we represent the set $\mathcal{L}$ as $\{0, \cdots, m\}$. Furthermore, with (2), every assertion ϕ over the transition system can be represented by arrays indexed over $\mathcal{L}$. That is, a formula

$$\phi(\bar{x}, vc) = \bigwedge_{l \in \mathcal{L}} vc = l \rightarrow \phi(\bar{x}, l)$$

will be represented by $[\phi(\bar{x}, 0), \cdots, \phi(\bar{x}, m)]$.

Given the formula ϕ and a formula φ (both represented as arrays) we implement the formula transformer $\mathcal{B}$:

function $\mathcal{B}(\phi, \mathcal{T}, \varphi)$
  for every $i \in \mathcal{L}$ **do**
    $\mathcal{T}_i := \{\tau \in \mathcal{T} : src(\tau) = i\}$;
    $\varphi[i] := \phi[i] \ \wedge \ \bigwedge_{\tau \in \mathcal{T}_i} \mathrm{wp}\,(\Phi_\tau)(\varphi[tgt(\tau)])$;
    $\varphi[i] := \mathfrak{R}$-simplify$(\varphi[i])$;
  end for
  return φ ;

In each iteration the function obtains $\mathcal{B}(\varphi)$ calculating the set $\{\tau \in \mathcal{T} : src(\tau) = i\}$ for each location. The result of the function is calculated using the formula

$$\mathrm{WP}\,(\Phi)(\varphi(\bar{x}, vc)) = \bigwedge_{s \in \mathcal{L}} vc = s \ \rightarrow \bigwedge_{\substack{\tau \in \mathcal{T} \wedge \\ src(\tau) = s}} \mathrm{wp}\,(\Phi_\tau)(\varphi_{tgt(\tau)}(\bar{x}))$$

which is equivalent to (3).

The function $\mathfrak{R}$-simplify performs simplifications in the theory $\mathfrak{R}$ and always returns equivalent formulae. We implement them using CVC Lite and Isabelle. These are described in Sec. 8.

Using the function $\mathcal{B}$ we can implement the algorithm for the fixed point:

function backPropagation$(\phi, \mathcal{T}, k)$
  $\varphi := [True, \cdots, True]$;
  $i := 0$;
  while $i < k$ **do**
    if $\not\models \Theta \rightarrow \mathcal{B}(\phi, \mathcal{T}, \varphi)$ **then**
      return unsat$(\mathcal{B}(\phi, \mathcal{T}, \varphi))$;
    else if $\models \varphi \rightarrow \mathcal{B}(\phi, \mathcal{T}, \varphi)$ **then**
      return converge(φ) ;
    else
      $\varphi := \mathcal{B}(\phi, \mathcal{T}, \varphi)$;
      $i := i + 1$;
    end if
  end while
  return notconverge(φ) ;

This function has a main loop which calculates the values φ_i in (6). The variable φ stores these values. If the value $\mathcal{B}(\phi, \mathcal{T}, \varphi)$ does not satisfy $\models \Theta \rightarrow \mathcal{B}(\phi, \mathcal{T}, \varphi)$, then the function returns the value unsat$(\mathcal{B}(\phi, \mathcal{T}, \varphi))$. In this case the method cannot find an inductive invariant for the formula ϕ. In the other case, the method checks if it has reached the fix point. If it is the case, it returns the value converge(φ) which stores the fix point. We use CVC Lite in order to implement the verifications of these conditions. The main loop do a maximun of k iteration with k a parameter of the algorithm. If the function cannot find a result in k iterations the program finish returning the value notconverge(φ) wich store the last calculated formula.

8 Method Optimizations

During the execution of the procedure described in Sec. 7, the size of the formulae grows in each iteration. This is due to substitutions in the calculation of WP (4) performed by the $\mathcal{B}$ transformer. In order to optimize the procedure, we try to keep the size of the formulae as short as possible. We carry out this task in two ways: by eliminating and simplifying transitions and the candidate invariant before the execution of the method and applying simplification strategies over the formulae obtained in the fix point calculation.

Transition Simplification

The transitions from (in_m) to (out_j) in program SCC_i (Fig. 3) are

$$vc = s_m \wedge B_i \wedge \text{wp}\,(S_i)(B_j \wedge b_j > 0) \;\longmapsto\; S_i\,; vc := s_j$$

with $i \in \{0, \cdots, m-1\}$, $j \in \{0, \cdots, m-1\}$. When $i = j$ the transition is never executed. This is proven by showing that the term corresponding to this transition in (3) is weaker than the candidate invariant ϕ, and hence can be absorbed in the fix point calculation. Then, these transitions will be:

$$vc = s_m \wedge B_i \wedge \text{wp}\,(S_i)(B_j \wedge b_j > 0) \;\longmapsto\; S_i\,; vc := s_j$$

with $i \in \{0, \cdots, m-1\}$, $j \in \{0, \cdots, m-1\}$ and $i \neq j$. The proof is in appendix A.1.

We can eliminate other transitions considering the strength in the candidate invariant ϕ (9). If we try to eliminate the guard $B_k \wedge b_k > 0$ in the program SCC_i, the transitions from (in_m) to (out_k) and from (in_i) to (out_k)

$$vc = s_m \wedge B_i \wedge \text{wp}\,(S_i)(B_k \wedge b_k > 0) \;\longmapsto\; S_i\,; vc := s_k \quad \text{and}$$
$$vc = s_i \wedge \text{wp}\,(b_i := b_i - 1; S_i)(B_k \wedge b_k > 0) \longmapsto\; b_i := b_i - 1\,; S_i\,; vc := s_k$$

can be eliminated. This fact is proven by showing that the terms generated by these transitions are absorbed in the fix point calculation. The proofs are in appendix A.2.

We can also simplify the transition going from (in_m) to (out_m):

$$vc = s_m \wedge B_i \wedge \text{wp}\,(S_i)(\textstyle\bigwedge_{0 \leq j < m} \neg B_j \vee b_j = 0) \;\longmapsto\; S_i\,; vc := s_m$$

with $i \in \{0, \cdots, m-1\}$. These transitions are simplified by eliminating a conjunction term in the subformula $\bigwedge_{0 \leq j < m} \neg B_j \vee b_j = 0$:

$$vc = s_m \wedge B_i \wedge \text{wp}\,(S_i)(\textstyle\bigwedge_{\substack{0 \leq j < m \\ i \neq j}} \neg B_j \vee b_j = 0) \;\longmapsto\; S_i\,; vc := s_m$$

with $i \in \{0, \cdots, m-1\}$. The proof is in the appendix A.4.

Candidate Invariant Simplification

If we try to eliminate the guard $B_i \wedge b_i > 0$ in the program SCC_i (i.e. eliminating the guard that releases the same kind of process) the strengthening of F_m in (9) is not necessary. The proof is in appendix A.3.

Formulae Simplifications

We implement some simplification strategies over the formulae. The function $\mathfrak{R}$-simplify (Sec. 7) implements these strategies over the formulae $\varphi[i]$ in the algorithm. These are quantifier-free formulae due to substitution in (4). Hence we can store them in conjunctive normal form. If some term in the normalized formulae is weaker than the conjunction of the others, this term can be eliminated. The process is applied over all terms in the formula. Also, a similar tactic is applied over the disjunctions inside the conjunctions; if a subterm is stronger than the disjunction of other subterms (in the same conjunctive term), it can be eliminated. The testing of these implications is done with the CVC Lite Validity Checker.

Moreover, other simplifications are implemented using the Isabelle theorem prover. This tool implements default tactics of simplification and the function $\mathfrak{R}$-simplify uses them in order to reduce the size of the formulae.

9 Examples

We test the method over some classical problems. In order to perform this task we develop a program prototype. Fig. 7 shows a schematic diagram of the process performed by this software.

The program consists of two modules: Transition Generator and Invariant Verificator. The first one takes as input a specification of the problem and returns a representation of the transition system with the candidate invariant. The input specification consists of a set $\{\mathrm{S}_0, \cdots, \mathrm{S}_{m-1}\}$ of programs with its corresponding conditions $\{B_0, \cdots, B_{m-1}\}$, an initial condition Θ' a global invariant I (as described in Sec. 6) and the guard that is tested for elimination. The output of this module is a simplified (see Sec. 8) transition system, the initial condition Θ and the candidate invariant. This output is the input of the Invariant Verifier Module which implements the algorithms described in Sec. 7. Both inputs and outputs of the programs are text files. In this section we show the input and the result for each example in a formatted form. The program and the specifications of the examples can be found at `http://www.cs.famaf.unc.edu.ar/~damian/publications/sbdinv/programs/`.

Bounded Buffer

Consider the example outlined in section 3. The specification is:

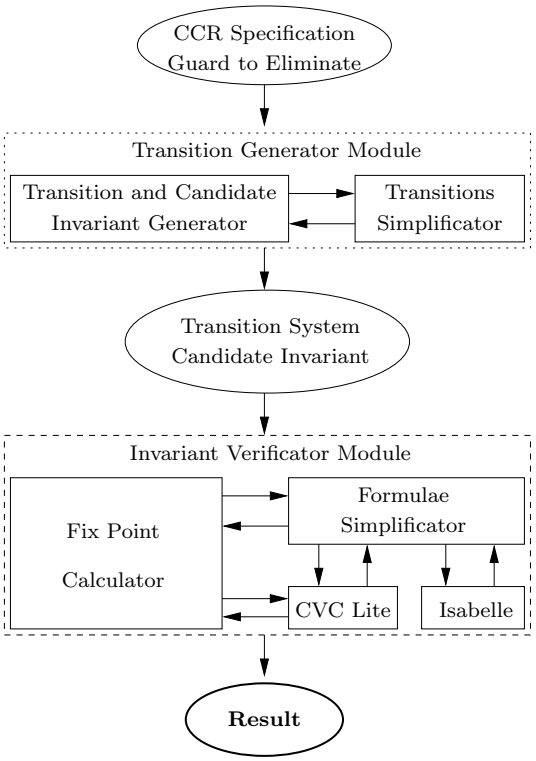


Fig. 7. Process Diagram.

$\Theta' : n = 0 \wedge N > 0$ $I : \ 0 \leq n \wedge n \leq N$	
producer: $B_0 : n < N$ $S_0 : n := n + 1$	consumer: $B_1 : n > 0$ $S_1 : n := n - 1$

First we test the method without any guard elimination. This case is for testing pourpose only (we do not attempt to eliminate any guard). After perform the system simplificactions the set of transitions generated is showed in Fig. 8. In this figure the locations are represented by the semaphore index as the transitions generator do. Note that after to perform the transition simplifications we obtain fewer transitions than the developed in Sec. 6.

The candidate invariant is also automatic generated and is the same SBS invariant of the problem φ_{SBS} because we do not attempt to do any guard elimination:

$$\begin{aligned}\phi : &(vc = 0 \rightarrow 0 \leq n \wedge n < N \wedge b_0 > 0) \wedge \\ &(vc = 1 \rightarrow 0 < n \wedge n \leq N \wedge b_1 > 0) \wedge \\ &(vc = 2 \rightarrow (\neg n < N \vee b_0 = 0) \wedge (\neg n > 0 \vee b_1 = 0) \wedge 0 \leq n \wedge n \leq N) \ .\end{aligned}$$

$$
\begin{aligned}
&vc = 2 \wedge \neg n < N \longmapsto b_0 := b_0 + 1; vc := 2\ ,\\
&vc = 2 \wedge \neg n > 0 \longmapsto b_1 := b_1 + 1; vc := 2\ ,\\
&vc = 2 \wedge n < N \wedge b_1 > 0 \wedge n + 1 > 0 \longmapsto n := n + 1; vc := 1\ ,\\
&vc = 2 \wedge n > 0 \wedge b_0 > 0 \wedge n - 1 < N \longmapsto n := n - 1; vc := 0\ ,\\
&vc = 2 \wedge n < N \wedge (b_1 = 0 \vee \neg n + 1 > 0) \longmapsto n := n + 1; vc := 2\ ,\\
&vc = 2 \wedge n > 0 \wedge (b_0 = 0 \vee \neg n - 1 < N) \longmapsto n := n - 1; vc := 2\ ,\\
&vc = 0 \wedge b_0 - 1 > 0 \wedge n + 1 < N \longmapsto b_0, n := b_0 - 1, n + 1; vc := 0\ ,\\
&vc = 0 \wedge b_1 > 0 \wedge n + 1 > 0 \longmapsto b_0, n := b_0 - 1, n + 1; vc := 1\ ,\\
&vc = 1 \wedge b_0 > 0 \wedge n - 1 < N \longmapsto b_1, n := b_1 - 1, n - 1; vc := 0\ ,\\
&vc = 1 \wedge b_1 - 1 > 0 \wedge n - 1 > 0 \longmapsto b_1, n := b_1 - 1, n - 1; vc := 1\ ,\\
&vc = 0 \wedge (b_0 - 1 = 0 \vee \neg n + 1 < N) \wedge (b_1 = 0 \vee \neg n + 1 > 0)\\
&\qquad\qquad \longmapsto b_0, n := b_0 - 1, n + 1; vc := 2\ ,\\
&vc = 1 \wedge (b_0 = 0 \vee \neg n - 1 < N) \wedge (b_1 - 1 = 0 \vee \neg n - 1 > 0)\\
&\qquad\qquad \longmapsto b_1, n := b_1 - 1, n - 1; vc := 2
\end{aligned}
$$

Fig. 8. Bounded Buffer transitions.

$$
\begin{aligned}
&vc = 2 \wedge \neg n < N \longmapsto b_0 := b_0 + 1; vc := 2\ ,\\
&vc = 2 \wedge \neg n > 0 \longmapsto b_1 := b_1 + 1; vc := 2\ ,\\
&vc = 2 \wedge n < N \wedge b_1 > 0 \wedge n + 1 > 0 \longmapsto n := n + 1; vc := 1\ ,\\
&vc = 2 \wedge n > 0 \wedge b_0 > 0 \wedge n - 1 < N \longmapsto n := n - 1; vc := 0\ ,\\
&vc = 2 \wedge n < N \wedge (b_1 = 0 \vee \neg n + 1 > 0) \longmapsto n := n + 1; vc := 2\ ,\\
&vc = 2 \wedge n > 0 \wedge (b_0 = 0 \vee \neg n - 1 < N) \longmapsto n := n - 1; vc := 2\ ,\\
&vc = 0 \wedge b_1 > 0 \wedge n + 1 > 0 \longmapsto b_0, n := b_0 - 1, n + 1; vc := 1\ ,\\
&vc = 1 \wedge b_0 > 0 \wedge n - 1 < N \longmapsto b_1, n := b_1 - 1, n - 1; vc := 0\ ,\\
&vc = 1 \wedge b_1 - 1 > 0 \wedge n - 1 > 0 \longmapsto b_1, n := b_1 - 1, n - 1; vc := 1\ ,\\
&vc = 0 \wedge (b_0 - 1 = 0 \vee \neg n + 1 < N) \wedge (b_1 = 0 \vee \neg n + 1 > 0)\\
&\qquad\qquad \longmapsto b_0, n := b_0 - 1, n + 1; vc := 2\ ,\\
&vc = 1 \wedge (b_0 = 0 \vee \neg n - 1 < N) \wedge (b_1 - 1 = 0 \vee \neg n - 1 > 0)\\
&\qquad\qquad \longmapsto b_1, n := b_1 - 1, n - 1; vc := 2
\end{aligned}
$$

Fig. 9. Bounded Buffer transitions (eliminating a producer's guard).

Furthermore, the initial condition generated is:

$$\Theta : vc = 2 \wedge n = 0 \wedge b_0 = 0 \wedge b_1 = 0 \wedge N > 0\ .$$

After a few seconds the method converge in one iteration, returning the same invariant ϕ.

The second test we attempt to eliminate the producer's guard $n < N \wedge b_0 > 0$. This guard release producer that wait in the semaphore. The resulting set of transitions is writed in Fig. 9 and the candidate invariant is

$$
\begin{aligned}
\phi : &(vc = 0 \rightarrow 0 \leq n \wedge n < N \wedge b_0 > 0 \wedge (b_0 = 1 \vee \neg n + 1 < N)) \wedge\\
&(vc = 1 \rightarrow 0 < n \wedge n \leq N \wedge b_1 > 0) \wedge\\
&(vc = 2 \rightarrow (\neg n < N \vee b_0 = 0) \wedge (\neg n > 0 \vee b_1 = 0) \wedge 0 \leq n \wedge n \leq N)\ .
\end{aligned}
$$

After a few seconds the method converge in four iterations, returning the following inductive invariant:

$$\begin{aligned}\varphi : (vc = 0 \rightarrow 0 \leq n \wedge n < N \wedge b_0 > 0 \wedge (n < N - 1 \rightarrow b_0 = 1) \\ \wedge (n > 0 \rightarrow b_0 \leq 2 \vee b_1 \leq 1)) \wedge \\ (vc = 1 \rightarrow 0 < n \wedge n \leq N \wedge b_1 > 0) \wedge (n > 1 \rightarrow b_0 \leq 1 \vee b_1 = 1) \\ \wedge (n < N \rightarrow b_0 \leq 1)) \wedge \\ (vc = 2 \rightarrow (\neg n < N \vee b_0 = 0) \wedge (\neg n > 0 \vee b_1 = 0) \\ \wedge 0 \leq n \wedge n \leq N)\end{aligned}$$

wich is implied by the initial condition Θ. Hence we can eliminate this guard.

We also try also to eliminate the consumer's guard $n > 0 \wedge b_1 > 0$ with similar positive result.

For the rests of guards ($n > 0 \wedge b_1 > 0$ in producer and $n < N \wedge b_0 > 0$ in consumer) the method detects the unsatisfability of the formula $\Theta \rightarrow \mathcal{B}(\phi, \mathcal{T}, \varphi)$ in the first iteration.

Table 1 show the details of this results. The first and second columns indicates the program and guard that

Program	Guard	#Trs.	Result	Iterations	Run Time
Producer	Producer	11	eliminated	5	4.16”
Producer	Consumer	10	detects unsat	2	1.23”
Consumer	Producer	10	detects unsat	3	2.60”
Consumer	Consumer	11	eliminated	5	4.17”

Table 1. Bounded Buffer Results.

Greedy Bounded Buffer

This example is a modified version of the bounded buffer problem, where the consumer is greedy and consume two elements instead of one. The specification is:

$\Theta' : n = 0 \wedge N > 1$ $I : \ 0 \leq n \wedge n \leq N$	
producer:	consumer:
$B_0 : n < N$	$B_1 : n > 1$
$S_0 : n := n + 1$	$S_1 : n := n - 2$

For this example the only guard that can be removed is $n > 1 \wedge b_1 > 0$ in the consumer program. This guard release a consumer process waiting in his semaphore.

The transition system generated have 11 transitions and the point fix calculation converge in 6 iterations at the invariant:

$$\begin{aligned}\varphi : (vc = 0 \rightarrow\ & 0 \leq n \wedge 0 < b_0 \wedge n < N \\ & \wedge (3 \leq n \rightarrow b_1 = 1 \vee \neg 0 < b_1) \\ & \wedge (2 + n < N \wedge 1 \leq n \rightarrow (0 < b_1 \rightarrow (0 < -2 + b_0 \rightarrow b_1 = 1))) \\ & \wedge (3 + n < N \rightarrow (0 < b_1 \rightarrow b_1 = 1 \vee \neg 0 < -3 + b_0)) \\ & \wedge (n + 1 < N \wedge 2 \leq n \rightarrow (0 < b_1 \rightarrow b_1 = 1 \vee \neg 0 < b_0 - 1))) \wedge \\ (vc = 1 \rightarrow\ & n \leq N \wedge 0 < b_1 \wedge 1 < n \\ & \wedge (3 < n \rightarrow b_1 = 1) \\ & \wedge (n < N \wedge 3 \leq n \rightarrow (0 < b_1 - 1 \rightarrow (b_1 = 2 \vee \neg 0 < -2 + b_0)))) \\ & \wedge (1 + n < N \rightarrow (0 < b_1 - 1 \rightarrow (0 < -3 + b_0 \rightarrow b_1 = 2))) \wedge \\ (vc = 2 \rightarrow\ & n \leq N \wedge 0 \leq n \wedge (b_1 = 0 \vee \neg 1 < n) \wedge (b_0 = 0 \vee \neg n < N))\end{aligned}$$

For the others guards the method finish detecting the unsatisfability of the formula $\Theta \rightarrow \mathcal{B}(\phi, \mathcal{T}, \varphi)$. Results are shown in table 2.

Program	Guard	#Trs.	Result	Iterations	Run Time
Producer	Producer	11	detects unsat	6	12.78”
Producer	Consumer	10	detects unsat	3	16.78”
Consumer	Producer	10	detects unsat	4	12.47”
Consumer	Consumer	11	eliminated	7	26.62”

Table 2. Greedy Bounded Buffer Results.

Greedy Bounded Buffer M

This example is a modified version of the bounded buffer problem, where the consumer is greedy and consume m elements with m a constant program. The specification is:

$\Theta' : n = 0 \wedge m > 0 \wedge N \geq m$ $I : \ 0 \leq n \wedge n \leq N$	
producer:	consumer:
$B_0 : n < N$	$B_1 : n \geq m$
$S_0 : \ n := n + 1$	$S_1 : \ n := n - m$

For this problem the only guard that can be removed is $n > 1 \wedge b_1 > 0$ in the consumer program. This guard release a consumer process waiting in his semaphore. The method cannot find this possibilty of simplification in $k = 20$ fix point iterations.For the remaining guards the method finish detecting the unsatisfability of the formula $\Theta \rightarrow \mathcal{B}(\phi, \mathcal{T}, \varphi)$. Results are shown in table 3.

Program	Guard	#Trs.	Result	Iterations	Run Time
Producer	Producer	11	detects unsat	6	19.90”
Producer	Consumer	10	detects unsat	2	2.36”
Consumer	Producer	10	detects unsat	3	4.53”
Consumer	Consumer	11	does not converge	20	22850.91”

Table 3. Greedy Bounded Buffer M Results.

General Semaphore

This example is a classic implementation of general semaphores with binary semaphores. The specification of this problem is:

$\Theta' : x = 0$ $I : \ x \geq 0$	
P: $B_0 : x > 0$ $S_0 : x := x - 1$	V: $B_1 : \mathit{True}$ $S_1 : x := x + 1$

In this problem only the guard $x > 0 \wedge b_0 > 0$ cannot be eliminated. This guard is in the V program and releases processes executing P programs. The method finish detecting this impossibility of simplification. For the other guard the methods finish detecting the simplifications. The details of these results are in table 4

Program	Guard	#Trs.	Result	Iterations	Run Time
P	P	11	eliminated	5	2.48”
P	V	10	eliminated	3	0.91”
V	P	10	detects unsat	2	0.72”
V	V	11	eliminated	3	0.83”

Table 4. General Semaphore Results.

Readers and Writers

The last example is the classical readers and writers problem. The specification of this problem is:

$\Theta' : w = 0 \wedge r = 0$ $I : \ r \geq 0 \wedge (w = 0 \vee w = 1 \wedge r = 0)$	
Writer (entrance): $B_0 : w = 0 \wedge r = 0$ $S_0 : \ w := w + 1$	Reader (entrance): $B_2 : w = 0$ $S_2 : \ r := r + 1$
Writer (exit): $Pre : w > 0$ $B_1 : \ \ True$ $S_1 : \ \ w := w - 1$	Reader (exit): $Pre : r > 0$ $B_1 : \ \ True$ $S_1 : \ \ r := r - 1$

In order to achieve the simplifications we need to add two kinds of assertions to the programs. First, given the topology of the programs (i.e. every exit is preceded by an entry), we can add the preconditions of the specification. This means that all the transitions in the writer's exit will have predicate $w > 0$ as extra preconditions and similarly for the reader's exit with $r > 0$. The transition generator implements this feature strengthening the guards γ_τ (see Sec. 4) with the corresponding preconditions in these transitions. Furthermore, since the conditions for the execution of both exit procedures is always true, variables b_2 and b_3 will be invariantly equal to 0. Hence, this two predicates are added to the global invariant.

With these additions, the result of the execution of the guard strenghtening process is summarized in table 5.

Program	Guard	#Trs.	Result	Iterations	Run Time
Writer (entrance)	Writer (entrance)	39	eliminated	2	7.80”
Writer (entrance)	Writer (exit)	38	eliminated	1	7.99”
Writer (entrance)	Reader (entrance)	38	eliminated	2	11.87”
Writer (entrance)	Reader (exit)	38	eliminated	1	15.35”
Writer (exit)	Write (entrance)	38	unsat detected	3	7.82”
Writer (exit)	Write (exit)	39	eliminated	3	7.66”
Writer (exit)	Reader (entrance)	38	unsat detected	3	8.88”
Writer (exit)	Reader (exit)	38	eliminated	3	9.75”
Reader (entrance)	Writer (entrance)	38	eliminated	2	12.05”
Reader (entrance)	Writer (exit)	38	eliminated	4	12.38”
Reader (entrance)	Reader (entrance)	39	unsat detected	5	19.22”
Reader (entrance)	Reader (exit)	38	eliminated	4	4.11”
Reader (exit)	Writer (entrance)	38	unsat detected	3	5.09”
Reader (exit)	Writer (exit)	38	eliminated	3	8.90”
Reader (exit)	Reader (entrance)	38	eliminated	4	4.09”
Reader (exit)	Reader (exit)	39	eliminated	3	3.75”

Table 5. Reader and Writer results.

10 Conclusions and Further Work

Conditional critical regions are a high level design pattern for concurrent programming. Unfortunately, since they are expensive to implement, most programming languages do not provide them as primitives. The SBS technique allows a nice implementation which can be further improved by axiomatic methods. This work provides a tool for performing such simplifications automatically. Since many problems can be solved with conditional critical regions, this method may have a wide range of applications. We performed some experiments with classical concurrent programs and variants of them (e.g. readers and writers, bounded buffer where the consumer is greedy and consume more than one element).

The examples suggest that in many cases the strengthening process terminates. However, until now we were unable to prove a termination theorem which would be of great importance since in this case the simplification could even be implemented in a compiler. Given that SBS was early considered as a valid alternative for implementation of conditional critical regions and monitors [7], these improvements may give a much more efficient implementation of monitors with conditional wait, which are easier to handle than other kinds of monitors.

Whereas the transition systems considered are generated by a SBS program, many of these results can be extended to other kinds of programs in which each transition models an atomic sequence of actions.

Although it works only for a very specific case, this work can be seen as a step towards the use of theorems provers not only for a posteriori verification of systems, but also for their formal construction and optimization.

A Demostrations of Transition System Simplifications

In this section we prove the simplifications of the transition system described in the section 8.

A.1

First, we prove the transitions from (in_m) to (out_i) in the program SCC_i can be eliminated.

From the equations 3 and 5 we obtain the formulas φ^j in each iteration of the fix point (figure 6):

$$\varphi^{j+1}(\bar{x}, vc) = \phi(\bar{x}, vc) \wedge \bigwedge_{\tau \in \mathcal{T}} vc = src(\tau) \rightarrow \text{wp}\,(\tau, \varphi^j_{tgt(\tau)})$$

Using the fact in equation 2 we can rewrite this fix point definition:

$$\varphi^{j+1} = \bigwedge_{\tau \in \mathcal{T}} vc = src(\tau) \rightarrow \phi_{src(\tau)} \wedge \text{wp}\,(\tau, \varphi^j_{tgt(\tau)}) \tag{10}$$

The term in the conjunction of this formulae corresponding to the transition is:

$$vc = m \rightarrow \phi_m \wedge (B_i \wedge wp.\, S_i.\, (B_i \wedge b_i > 0) \rightarrow wp.\, S_i.\, \varphi_i^j)$$

We prove that the candidate invariant ϕ_m absorb the right hand side of this equation:

$$\begin{array}{ll} & \phi_m \rightarrow (B_i \wedge wp.\, S_i.\, (B_i \wedge b_i > 0) \rightarrow wp.\, S_i.\, \varphi_i^j) \\ \Leftarrow & \{\ \phi_m \rightarrow \varphi_m \text{ by construction of } \phi \text{ (equation 9)}\ \} \\ & \varphi_m \rightarrow (B_i \wedge wp.\, S_i.\, (B_i \wedge b_i > 0) \rightarrow wp.\, S_i.\, \varphi_i^j) \\ \Leftarrow & \{\ \varphi_m \rightarrow \neg B_i \vee b_i = 0 \text{ by equation 8}\ \} \\ & \neg B_i \vee b_i = 0 \rightarrow (B_i \wedge wp.\, S_i.\, (B_i \wedge b_i > 0) \rightarrow wp.\, S_i.\, \varphi_i^j) \\ \equiv & \{\ wp \text{ is conjunctive, } S_i \text{ does not depend on } b_i\ \} \\ & \neg B_i \vee b_i = 0 \rightarrow (B_i \wedge b_i > 0 \wedge wp.\, S_i.\, B_i \rightarrow wp.\, S_i.\, \varphi_i^j) \\ \equiv & \{\ \text{Predicate calculus}\ \} \\ & \mathit{True} \end{array}$$

Hence the term corresponding to the transitions does not contributes to the fix point calculation and the transition can be eliminated.

A.2

If we try to eliminate the guard $B_k \wedge b_k > 0$, the resulting strength of the candidate invariant allow to remove the transitions starting from (in_m) to (out_k) and from (in_i) to (out_k).

In the first case, the term in the equation 10 corresponding to the transition is:

$$vc = m \rightarrow \phi_m \wedge (B_i \wedge wp.\, S_i.\, (B_k \wedge b_k > 0) \rightarrow wp.\, S_i.\, \varphi_k^j)$$

We prove that the candidate invariant ϕ_m absorb the right hand side of this implication:

$$\begin{array}{ll} & \phi_m \rightarrow (B_i \wedge wp.\, S_i.\, (B_k \wedge b_k > 0) \rightarrow wp.\, S_i.\, \varphi_k^j) \\ \Leftarrow & \{\ \phi_m \rightarrow (B_i \rightarrow wp.\, S_i.\, (\neg B_k \vee b_k = 0))\ \text{by equation 9}\ \} \\ & (B_i \rightarrow wp.\, S_i.\, (\neg B_k \vee b_k = 0)) \rightarrow (B_i \wedge wp.\, S_i.\, (B_k \wedge b_k > 0) \rightarrow \\ & wp.\, S_i.\, \varphi_k^j) \\ \equiv & \{\ \text{Predicate calculus}\ \} \\ & wp.\, S_i.\, (\neg B_k \vee b_k = 0) \wedge B_i \wedge wp.\, S_i.\, (B_k \wedge b_k > 0) \rightarrow wp.\, S_i.\, \varphi_k^j \\ \equiv & \{\ wp \text{ is conjunctive}\ \} \\ & B_i \wedge wp.\, S_i.\, ((\neg B_k \vee b_k = 0) \wedge B_k \wedge b_k > 0) \rightarrow wp.\, S_i.\, \varphi_k^j \\ \equiv & \{\ \text{Predicate calculus}\ \} \\ & B_i \wedge wp.\, S_i.\, \mathit{False} \rightarrow wp.\, S_i.\, \varphi_k^j \\ \equiv & \{\ \text{Law of Exclude Miracle}\ \} \\ & B_i \wedge \mathit{False} \rightarrow wp.\, S_i.\, \varphi_k^j \\ \equiv & \{\ \text{Predicate calculus}\ \} \\ & \mathit{True} \end{array}$$

The proof for the other case is similar. The term in the equation 10 corresponding to the transition is:

$$vc = i \rightarrow \phi_i \wedge (wp.\,(b_i := b_i - 1; S_i).\,(B_k \wedge b_k > 0) \rightarrow wp.\,(b_i := b_i - 1; S_i).\,\varphi_k^j)$$

We prove that the candidate invariant ϕ_i absorb the right hand side of this implication:

$$\begin{array}{ll}
 & \phi_i \rightarrow (wp.\,(b_i := b_i - 1; S_i).\,(B_k \wedge b_k > 0) \rightarrow wp.\,(b_i := b_i - 1; S_i).\,\varphi_k^j) \\
\Leftarrow & \{\ \phi_i \rightarrow wp.\,(b_i := b_i - 1; S_i).\,(\neg B_k \vee b_k = 0) \text{ by equation 9 }\} \\
 & wp.\,(b_i := b_i - 1; S_i).\,(\neg B_k \vee b_k = 0) \rightarrow (wp.\,(b_i := b_i - 1; S_i).\,(B_k \wedge b_k > \\
 & 0) \rightarrow wp.\,(b_i := b_i - 1; S_i).\,\varphi_k^j) \\
\equiv & \{\ \text{Predicate calculus }\} \\
 & wp.\,(b_i := b_i - 1; S_i).\,(\neg B_k \vee b_k = 0) \wedge wp.\,(b_i := b_i - 1; S_i).\,(B_k \wedge b_k > \\
 & 0) \rightarrow wp.\,(b_i := b_i - 1; S_i).\,\varphi_k^j \\
\equiv & \{\ wp \text{ is conjunctive }\} \\
 & wp.\,(b_i := b_i - 1; S_i).\,((\neg B_k \vee b_k = 0) \wedge B_k \wedge b_k > 0) \rightarrow wp.\,(b_i := \\
 & b_i - 1; S_i).\,\varphi_k^j \\
\equiv & \{\ \text{Predicate calculus }\} \\
 & wp.\,(b_i := b_i - 1; S_i).\,\mathit{False} \rightarrow wp.\,(b_i := b_i - 1; S_i).\,\varphi_k^j \\
\equiv & \{\ \text{Law of Exclude Miracle }\} \\
 & \mathit{False} \rightarrow wp.\,(b_i := b_i - 1; S_i).\,\varphi_k^j \\
\equiv & \{\ \text{Predicate calculus }\} \\
 & \mathit{True}
\end{array}$$

A.3

The next posible simplification is when we tried to eliminate the guard $B_i \wedge b_i > 0$ in a program SCC_i (the guard that release the same kind of process). In this case φ_{SBD} is stronger than F_m. Hence, by the equation 9, we can ignore this strength in the candidate invariant. The proof is at follows:

$$\begin{array}{ll}
 & \varphi_{SBD} \rightarrow F_m \\
\Leftarrow & \{\ \varphi_{SBD} \rightarrow (vc = m \rightarrow \neg B_i \vee b_i = 0) \text{ by equation 8, } F_m \text{ definition} \\
 & \text{(equation 9) }\} \\
 & (vc = m \rightarrow \neg B_i \vee b_i = 0) \rightarrow (vc = m \wedge B_i \ \rightarrow\ wp.\,S_i.\,(\neg B_i \vee b_i = 0)) \\
\equiv & \{\ \text{Predicate calculus }\} \\
 & vc = m \wedge b_i = 0 \wedge B_i \ \rightarrow\ wp.\,S_i.\,(\neg B_i \vee b_i = 0) \\
\equiv & \{\ wp \text{ is disjunctive because } S_i \text{ is deterministic }\} \\
 & vc = m \wedge b_i = 0 \wedge B_i \ \rightarrow\ wp.\,S_i.\,(\neg B_i) \vee wp.\,S_i.\,b_i = 0 \\
\equiv & \{\ S_i \text{ does not depend on } b_i \ \} \\
 & vc = m \wedge b_i = 0 \wedge B_i \ \rightarrow\ wp.\,S_i.\,(\neg B_i) \vee b_i = 0 \\
\equiv & \{\ \text{Predicate calculus }\} \\
 & \mathit{True}
\end{array}$$

A.4

We can also simplify the transitions going from (in_m) to (out_m):

$$vc = m \wedge B_i \wedge wp.\, S_i.\, \Big(\bigwedge_{0 \le j < m} \neg B_j \vee b_j = 0\Big) \;\longmapsto\; S_i\,; vc := m\ ,$$

with $i \in \{0, \cdots, m-1\}$.

The term, in the equation 10, corresponding to the transition is:

$$vc = m \to \phi_m \wedge \Big(B_i \wedge wp.\, S_i.\, \Big(\bigwedge_{0 \le j < m} \neg B_j \vee b_j = 0\Big) \to wp.\, S_i.\, \varphi_m^j\Big)$$

We show a simplified equivalent formula of the right hand side of this implication:

$$\begin{array}{ll}
 & \phi_m \wedge \Big(B_i \wedge wp.\, S_i.\, \Big(\bigwedge_{0 \le j < m} \neg B_j \vee b_j = 0\Big) \to wp.\, S_i.\, \varphi_m^j\Big) \\
\equiv & \{\ wp \text{ is conjunctive }\ \} \\
 & \phi_m \wedge \\
 & \Big(B_i \wedge wp.\, S_i.\, (\neg B_i \vee b_i = 0) \wedge wp.\, S_i.\, \Big(\bigwedge_{\substack{0 \le j < m \\ i \ne j}} \neg B_j \vee b_j = 0\Big) \to wp.\, S_i.\, \varphi_m^j\Big) \\
\equiv & \{\ \text{Predicate calculus }\ \} \\
 & \phi_m \wedge \Big(\neg B_i \vee \neg wp.\, S_i.\, (\neg B_i \vee b_i = 0) \vee \Big(wp.\, S_i.\, \Big(\bigwedge_{\substack{0 \le j < m \\ i \ne j}} \neg B_j \vee b_j = 0\Big) \to wp.\, S_i.\, \varphi_m^j\Big)\Big) \\
\equiv & \{\ wp \text{ is disjunctive because } S_i \text{ is deterministic }\ \} \\
 & \phi_m \wedge \Big(\neg B_i \vee \neg (wp.\, S_i.\, \neg B_i \vee b_i = 0) \vee \Big(wp.\, S_i.\, \Big(\bigwedge_{\substack{0 \le j < m \\ i \ne j}} \neg B_j \vee b_j = 0\Big) \to wp.\, S_i.\, \varphi_m^j\Big)\Big) \\
\equiv & \{\ \phi_m \to (\neg B_i \vee b_i = 0) \text{ by equations 8 and 9 }\ \} \\
 & \phi_m \wedge (\neg B_i \vee b_i = 0) \wedge \\
 & \Big(\neg B_i \vee \neg (wp.\, S_i.\, \neg B_i \vee b_i = 0) \vee \Big(wp.\, S_i.\, \Big(\bigwedge_{\substack{0 \le j < m \\ i \ne j}} \neg B_j \vee b_j = 0\Big) \to wp.\, S_i.\, \varphi_m^j\Big)\Big) \\
\equiv & \{\ \text{Predicate calculus }\ \} \\
 & \phi_m \wedge (\neg B_i \vee b_i = 0) \wedge \\
 & \Big(\neg B_i \vee \Big(wp.\, S_i.\, \Big(\bigwedge_{\substack{0 \le j < m \\ i \ne j}} \neg B_j \vee b_j = 0\Big) \to wp.\, S_i.\, \varphi_m^j\Big)\Big) \\
\equiv & \{\ \phi_m \to (\neg B_i \vee b_i = 0) \text{ by equations 8 and 9 }\ \} \\
 & \phi_m \wedge \Big(\neg B_i \vee \Big(wp.\, S_i.\, \Big(\bigwedge_{\substack{0 \le j < m \\ i \ne j}} \neg B_j \vee b_j = 0\Big) \to wp.\, S_i.\, \varphi_m^j\Big)\Big) \\
\equiv & \{\ \text{Predicate calculus }\ \} \\
 & \phi_m \wedge \Big(B_i \wedge wp.\, S_i.\, \Big(\bigwedge_{\substack{0 \le j < m \\ i \ne j}} \neg B_j \vee b_j = 0\Big) \to wp.\, S_i.\, \varphi_m^j\Big)
\end{array}$$

Hence, the simplified transitions are:

$$vc = m \wedge B_i \wedge wp.\, S_i.\, \Big(\bigwedge_{\substack{0 \le j < m \\ i \ne j}} \neg B_j \vee b_j = 0\Big) \;\longmapsto\; S_i\,; vc := m\ ,$$

with $i \in \{0, \cdots, m-1\}$.

References

[1] G. Andrews. *Foundations of Multithreaded, Parallel, and Distributed Programming*. Addison-Wesley, Reading, Massachusetts, USA, 1999.

[2] C. Barrett and S. Berezin. CVC Lite: A new implementation of the cooperating validity checker. In R. Alur and D. A. Peled, editors, *Proceedings of the* 16th *International Conference on Computer Aided Verification (CAV '04)*, volume 3114 of *Lecture Notes in Computer Science*, pages 515–518. Springer-Verlag, July 2004. Boston, Massachusetts.

[3] N. Bjorner, A. Browne, and Z. Manna. Automatic generation of invariants and intermediate assertions. *Theor. Comput. Sci.*, 173(1):49–87, 1997.

[4] E. W. Dijkstra. A tutorial on the split binary semaphore. `http://www.cs.utexas.edu/users/EWD/ewd07xx/EWD703.PDF`, Mar. 1979.

[5] E. W. Dijkstra and C. S. Scholten. *Predicate calculus and program semantics*. Springer-Verlag New York, Inc., 1990.

[6] R. Hoogerwoord. Two applications of the split binary semaphore. Course notes RH221, 1990.

[7] J. L. W. Kessels. An alternative to event queues for synchronization in monitors. *Commun. ACM*, 20(7):500–503, 1977.

[8] Z. Manna and A. Pnueli. On the faithfulness of formal models. In *Mathematical Foundations of Computer Science*, pages 28–42, 1991.

[9] A. Martin and J. van de Snepscheut. Design of synchronization algorithms. *Constructive Methods in Computing Science*, pages 445–478, 1989.

[10] L. C. Paulson. The Isabelle reference manual. `http://isabelle.in.tum.de/doc/ref.pdf`, 2004.

[11] F. B. Schneider. *On Concurrent Programming*. Graduate texts in computer science. Springer-Verlag New York, Inc., 1997.

[12] A. Tiwari, H. Rueß, H. Saïdi, and N. Shankar. A technique for invariant generation. In T. Margaria and W. Yi, editors, *TACAS 2001 - Tools and Algorithms for the Construction and Analysis of Systems*, volume 2031, pages 113–127, Genova, Italy, Apr. 2001. Springer-Verlag.

Semaphores

INF4140

14.09.11

Lecture 3

Outline

- Barriers
- Semaphores
  - Main ideas
  - Important synchronization problems

Barrier Synchronization

- Parallel computation of disjoint parts (e.g., array elements)
- Loop in each process, each iteration depends on the result from last iteration

```
process Worker[i=1 to n] {
      while (true) {
            Code to implement task i;
            Wait for all n tasks to complete;      # Barrier
      }
}
```

All processes must reach the barrier before any of them can continue

Shared Counter

Processes must be synchronized after computing their (sub)tasks.
The synchronization can be implemented by using a *shared counter*:

```
int count = 0;
process Worker[i=1 to n] {
      while (true) {
            Task i
            < count = count + 1; >
            < await (count == n); >
      }
}
```

Can be implemented by fetch-and-add (FA) instruction
Drawbacks:

- count must be reset between each iteration
- Must be modified by atomic operations
- Inefficient: many processes are reading and writing to count *at the same time*

Flags and coordinators

Goal: Avoid overload of read and write operations on one variable.
Split shared counter → several local variables.

```
Worker[i]:
          arrive[i] = 1;
          < await (continue[i] == 1);>
Coordinator:
          for [i=1 to n] < await (arrive[i]==1);>
          for [i=1 to n] continue[i] = 1;
```

In a loop, the flags must be cleard before the next iteration

Flag synchronization principles:

- The process that waits for a flag is the one that should clear the flag
- A flag should not be set until it is known that it is clear

Flags and coordinators

```
int arrive[1:n] = ([n] 0), continue[1:n] = ([n] 0);

 process Worker[i = 1 to n] {
   while(true) {
     Task i;
     arrive[i] = 1;
     <await (continue[i]==1);>
     continue[i] = 0;
   }
 }
```

```
process Coordinator {
  while(true) {
    for[i = 1 to n] {
      <await (arrive[i]==1);>
      arrive[i] = 0;
    }
    for [i = 1 to n]
      continue[i] = 1;
  }
}
```

Combining Tree Barrier

- The *worker* and *coordinator* roles can be combined
- Combining tree barrier: processes organized in a tree structure. *Arrive* signals are sent up the tree, *continue* signals down

Implementation of Critical Sections

```
        bool lock = false;
Entry:  <await (!lock) lock = true>
        Critical section
Exit:   <lock = false;>
```

Spin lock implementation of entry: `while (TS(lock)) skip`

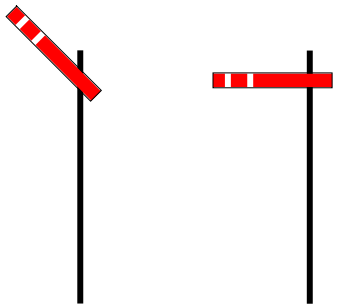
Drawbacks:

- Busy waiting protocols are often complicated
- Inefficient if there are fever processors than processes
  - Should not waste time executing a skip loop
- No clear distinction between variables used for synchronization and computation

Desirable to have a special tools for synchronization protocols

Semaphores

- Introduced by Dijkstra in 1968
- Originates from railroad traffic synchronization
- Railroad semaphores indicates whether the track ahead is clear or occupied by another train



Properties

- Semaphores in concurrent programs work in a similar way
- Used to implement mutex and condition synchronization
- Included in most standard libraries for concurrent programming

Concept

- A semaphore is special kind of shared program variable
- The value of a semaphore is a non-negative integer
- Can *only* be manipulated by the following two atomic operations:
  - P: (Passeren) Wait for signal - want to pass
    - effect: wait until the value is greater than zero, and decrease the value by one
  - V: (Vrijgeven)Signal an event - release
    - effect: increase the value by one

Syntax

- A semaphore is declared by either:
  - sem s; default initial value is zero
  - sem s = 1;
  - sem s[4] = ([4] 1);
- The P operation
  - syntax: P(s)
  - implementation : <await (s>0) s = s - 1;>
- The V operation
  - syntax: V(s)
  - implementation : <s = s + 1;>

Important: No direct access to the value of a semaphore.
For instance a test like `if (s==1) {...}` *is not allowed!*

Kinds of semaphores

- Kinds of semaphores

  General semaphore: possible values - all non-negative integers
  Binary semaphore: possible values - 0 and 1

- Fairness: as for await-statements. In most languages processes delayed while executing P-operations are awaken in the order they where delayed

Example: Mutex

Mutex implemented by a binary semaphore

```
sem mutex = 1;

process CS[i = 1 to n] {
     while (true) {
          P(mutex);
             critical section
          V(mutex);
             noncritical section
     }
}
```

The semaphore is initially 1 and always P before V $\rightarrow$ Binary semaphore

Example: Barrier synchronization

Semaphores may be used for barrier synchronization

```
sem arrive1 = 0, arrive2 = 0;
process Worker1 {
                    ⋮
    V(arrive1);     reach the barrier
    P(arrive2);     wait for other processes
                    ⋮
}
process Worker2 {
                    ⋮
    V(arrive2);     reach the barrier
    P(arrive1);     wait for other processes
                    ⋮
}
```

Split binary semaphores

A set of semaphores may form a split binary semaphore if the sum of all semaphores never exceeds 1. Split binary semaphores can be used as follows to implement mutex.

- Consider a program with more than one binary semaphore
- Let one of these semaphores be initialized to 1 and the others to 0
- Let all processes call P on a semaphore, before calling V on (another) semaphore
- Then the code between the P and the V call will be executed in mutex. All semaphores will have the value 0.

Example: Producer/consumer with split binary semaphores

```
typeT buf; /* Buffer of some type T */
sem empty = 1, full = 0;

process Producer{
    while (true){
        ⋮
        P(empty);
        buf = data;
        V(full);
    }
}
```

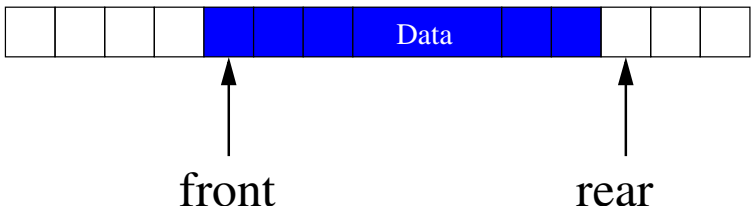
```
process Consumer{
    while (true){
        P(full);
        result = buf;
        V(empty);
        ⋮
    }
}
```

empty and full are both binary semaphores, together they form a split binary semaphore.

This solution works if there are several producers/consumers, but buffer capacity might be a problem

Increasing buffer capacity

- In the example the producer must wait for the consumer to empty the buffer before it can produce a new entry.
- It is a relatively simple task to generalize to a buffer of size n.
- We will use:
  - A ring-buffer represented by an array, and two integers `rear` and `front`.
  - General semaphores to keep track of the number of free slots



Example: Producer/consumer with increased buffer capacity

```
typeT buf[n]; /* Array av en type T */
int front = 0, rear = 0;
sem empty = n, full = 0;

process Producer{
      while (true){
            :
         P(empty);
         buf[rear] = data;
         rear = rear + 1 % n;
         V(full);
      }
}
```

```
process Consumer{
      while (true){
         P(full);
         result = buf[front];
         front = front + 1 % n;
         V(empty);
            :
      }
}
```

What if there are several producers or consumers?

Increasing the number of processes

- Let us consider a situation with several producers and consumers.
- New synchronization problems:
  - Avoid that two producers deposits to `buf[rear]` before `rear` is updated
  - Avoid that two consumers fetches from `buf[front]` before `front` is updated.
- Solutions
  - Introduce a binary semaphore `mutexDeposit` to deny two producers to deposit to the buffer at the same time.
  - Introduce a binary semaphore `mutexFetch` to deny two consumers to fetch from the buffer at the same time.

Example: Producer/consumer with several processes

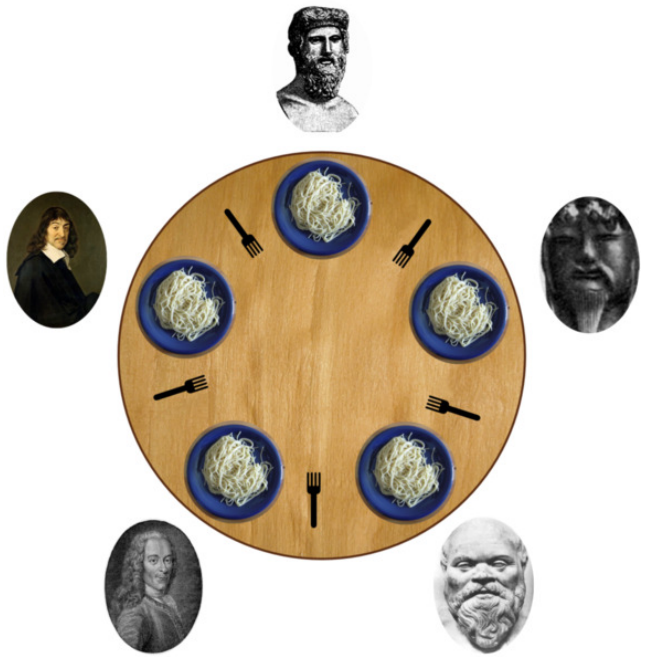
```
typeT buf[T]; int front = 0, rear = 0;
sem empty = n, full = 0, mutexDeposit = 1, mutexFetch = 1;

process Producer[i = 1 to M]{
    while (true){
        .
        .
        .
        P(empty);
        P(mutexDeposit);
        buf[rear] = data;
        rear = rear + 1 % n;
        V(mutexDeposit);
        V(full);
    }
}
```

```
process Consumer[i = 1 to N]{
    while (true){
        .
        .
        .
        P(full);
        P(mutexFetch);
        result = buf[front];
        front = front + 1 % n;
        V(mutexFetch);
        V(empty);
    }
}
```

Problem: Dining philosophers introduction

- Five philosophers sit around a circular table.
- One fork is placed between each pair of philosophers
- The philosophers alternates between thinking and eating
- A philosopher need two forks to eat.



[1] image from wikipedia.org

Example: Dining philosophers

A sketch of the program may look like this:

```
process Philosopher [i = 0 to 4] {
      while (true) {
         think;
         acquire forks;
         eat;
         release forks;
      }
}
```

We have to program the actions `acquire forks` and `release forks`

Example: Dining philosophers 1st attempt

- Let the forks be semaphores
- Let the philosophers pick up the left fork first

```
sem fork[5] = ( [5] 1)
process Philosopher [i = 0 to 4] {
      while (true) {
         think;
         P(fork[i]); P(fork[(i+1)%5]);
         eat;
         V(fork[i]; V(fork[(i+1)%5]);
      }
}
```

deadlock?

Example: Dining philosophers 2nd attempt

To avoid deadlock, let Philosopher4 grab the right fork first.

```
sem fork[5] = ( [5] 1)

process Philosopher [i = 0 to 3] {
      while (true) {
         think;
         P(fork[i]); P(fork[i+1]);
         eat;
         V(fork[i]; V(fork[i+1]);
      }
}

process Philosopher4 {
      while (true) {
         think;
         P(fork[0]); P(fork[4]);
         eat;
         V(fork[0]; V(fork[4]);
      }
}
```

Example: Readers/Writers overview

- Classical synchronization problem
- Reader and writer processes share a database
- Readers reads from the database
- Writers reads and updates the database
- Writers need mutually exclusive access
- When no writers have access, many readers may access the database

Example: Readers/Writers approaches

- Dining philosophers: Pair of processes competes for access to forks
- Readers/writers: Different classes of processes competes for access to the database
  - Readers compete with writers
  - Writers compete both with readers and other writers
- General synchronization problem:
  - Readers must wait until no writers are active in DB
  - Writers must wait until no readers or writers are active in DB
- We will look at two different approaches:
  - Mutex: easy to implement, but unfair
  - Condition Synchronization:
    - Using a split binary semaphore
    - Easy to adapt to different scheduling strategies

Example: Readers/Writers with Mutex (1)

```
sem rw = 1;

process Reader [i = 1 to M] {
    while (true) {
        ⋮
        P(rw);
        Read from DB
        V(rw);
    }
}
```

```
process Writer [i = 1 to N] {
    while (true) {
        ⋮
        P(rw);
        Write to DB
        V(rw);
    }
}
```

But: We want more than one reader simultaneously

Example: Readers/Writers with Mutex (2)

Give access to more than one reader

```
int nr = 0;          # number of active readers
sem rw = 1;          # lock for reader/writer exclusion

process Reader [i = 1 to M] {
    while (true) {
        .
        .
        .
        < nr = nr + 1;
          if (nr == 1) P(rw); >
        Read from DB
        < nr = nr - 1;
          if (nr == 0) V(rw); >
    }
}
```

```
process Writer [i = 1 to N] {
    while (true) {
        .
        .
        .
        P(rw);
        Write to DB
        V(rw);
    }
}
```

We have to make the entry and exit of the CS atomic

Example: Readers/Writers with Mutex (3)

```
int nr = 0;              # number of active readers
sem rw = 1;              # lock for reader/writer exclusion
sem mutexR = 1;          # mutex for readers

 process Reader [i = 1 to M] {
       while (true) {
                 :
          P(mutexR); nr = nr + 1; if (nr == 1) P(rw); V(mutexR)
          Read from DB
          P(mutexR); nr = nr - 1; if (nr == 0) V(rw); V(mutexR)
       }
 }
```

What happens if we have a constant stream of readers?

Example: Readers/Writers with condition synchronization - overview

- The mutex solution solved two separate synchronization problems
  - Reader vs. writer for access to the database
  - Reader vs. reader for access to the counter
- We shall now look at a solution based on condition synchronization

Example: Readers/Writers with condition synchronization - invariant

- Let us try to find a reasonable invariant for the system.
- It must state that:
  - When a writer access the DB, no one else can
  - When no writers access the DB, one or more readers may
- Let us introduce two counters:
  - `nr` is the number of active readers
  - `nw` is the number of active writers
- The invariant may be:

```
RW: (nr == 0 or nw == 0) and nw <= 1
```

Example: Readers/Writers with condition synchronization - updating counters

We need code to update the counters

Reader:

```
< nr = nr + 1; >
Read from DB
< nr = nr - 1; >
```

Writer:

```
< nw = nw + 1; >
Write to DB
< nw = nw - 1; >
```

Put conditions in front of the atomic actions to preserve the invariant.

- Before increasing `nr`: `(nw == 0)`
- Before increasing `nw`: `(nr == 0 and nw == 0)`

Example: Readers/Writers with condition synchronization - without semaphores

```
int nr = 0, nw = 0;
## RW: (nr == 0 or nw == 0 ) and nw <= 1

 process Reader [i=1 to M] {
    while (true) {
                    :
      < await (nw == 0)
        nr = nr + 1; >
      Read from DB
      < nr = nr - 1; >
      }
 }
```

```
 process Writer [i=1 to N] {
    while (true) {
                    :
      < await (nr == 0 and nw == 0)
        nw = nw + 1; >
      Write to DB
      < nw = nw - 1; >
      }
 }
```

Example: Readers/Writers with condition synchronization - converting to split binary semaphores

Implementation of `await`-statements may be done by split binary semaphores.

- May be used to implement different synchronization problems with different guards B_1, $B_2 \ldots$
- Use an entry-semaphore (e) initialized to 1
- For each guard B_i: associate one counter and one delay-semaphore, both initialized to 0
  - Semaphore: delay the processes waiting for B_i
  - Counter: count the number of processes waiting for B_i
- For the readers/writers problem, we need three semaphores and two counters:

```
sem e = 1;
sem r = 0; int dr = 0;     # condition reader:  nw == 0
sem w = 0; int dw = 0;     # condition writer:  nr == 0 and nw == 0
```

Example: Readers/Writers with condition synchronization - converting to split binary semaphores (2)

- `e`, `r` and `w` form a split binary semaphore.
- All execution paths starts with a P-operation and ends with a V-operation $\rightarrow$ Mutex
- we need a signal mechanism `SIGNAL` to pick which semaphore to signal.
- `SIGNAL` must make sure the invariant holds
- B_i holds when a process enters CR because either:
  - the process checks
  - the process is only signaled if B_i holds
- Avoid deadlock by checking the counters before the delay semaphores are signaled.
  - `r` is not signalled (`V(r)`) unless there is a delayed reader
  - `w` is not signalled (`V(w)`) unless there is a delayed writer

Example: Readers/Writers with condition synchronization - with split binary semaphores

```
int nr = 0, nw = 0; int dr = 0; int dw = 0;
sem e = 1; sem r = 0; sem w = 0;
# RW: (nr == 0 ∨ nw == 0 ) ∧ nw <= 1

 process Reader [i=1 to M] {      # Entry condition: nw == 0
    while (true) {
                        :
       # <await (nw == 0) nr = nr + 1;>
       P(e); if (nw>0) { dr=dr+1; V(e); P(r);}
       nr=nr+1; SIGNAL;
       Read from DB
       # < nr = nr - 1; >
       P(e); nr=nr-1; SIGNAL
       }
 }
```

Example: Readers/Writers with condition synchronization - with split binary semaphores (2)

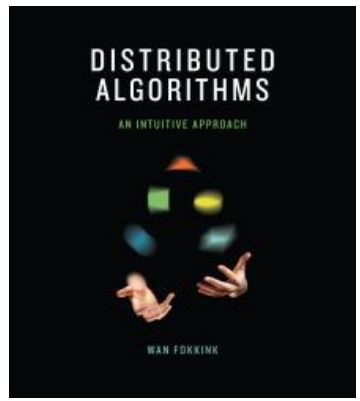
```
process Writer [i=1 to N] {
  # Entry condition: nw == 0 and nr == 0
  while (true) {
                  :
    # <await (nr == 0 and nw == 0) nw = nw + 1; >
    P(e); if (nr > 0 or nw > 0)
            { dw=dw+1; V(e); P(w);}
    nw = nw + 1; SIGNAL;
    Write to DB
    # < nw = nw - 1; >
    P(e); nw = nw - 1; SIGNAL;
    }
}
```

Example: Readers/Writers with condition synchronization - with split binary semaphores (3)

- SIGNAL

```
if (nw == 0 and dr > 0) {
      dr = dr -1; V(r);                          # awaken reader
      }
elseif (nr == 0 and nw == 0 and dw > 0) {
      dw = dw -1; V(w);                          # awaken writer
      }
else
      V(e);                                      # release entry lock
```

Distributed Algorithms



Wan Fokkink

Distributed Algorithms: An Intuitive Approach

MIT Press, 2013 (revised 1st edition, 2015)

Algorithms

A skilled programmer must have good insight into algorithms.

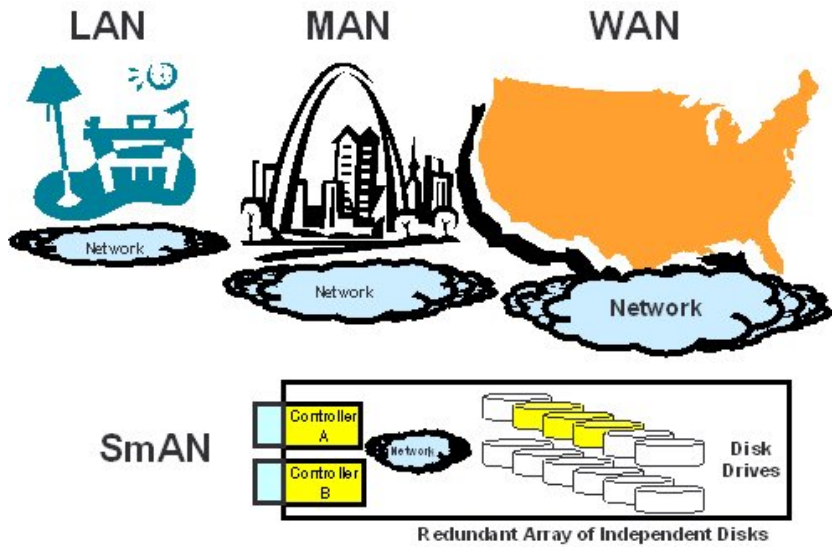
At bachelor level you were offered courses on basic algorithms: searching, sorting, pattern recognition, graph problems, ...

You learned how to detect such subproblems within your programs, and solve them effectively.

You're trained in algorithmic thought for uniprocessor programs (e.g. divide-and-conquer, greedy, memoization).

Distributed systems

A distributed system is an interconnected collection of autonomous processes.



Motivation:

- information exchange
- resource sharing
- parallelization to increase performance
- replication to increase reliability
- multicore programming

Distributed versus uniprocessor

Distributed systems differ from uniprocessor systems in three aspects.

- *Lack of knowledge on the global state*: A process has no up-to-date knowledge on the local states of other processes.

  Example: termination and deadlock detection become an issue.

- *Lack of a global time frame*: No total order on events by their temporal occurrence.

  Example: mutual exclusion becomes an issue.

- *Nondeterminism*: Execution by processes is nondeterministic, so running a system twice can give different results.

  Example: race conditions.

Aim of this course

This course offers a bird's-eye view on a wide range of algorithms for basic challenges in distributed systems.

It aims to provide you with an algorithmic frame of mind for solving fundamental problems in distributed computing.

- Handwaving correctness arguments.
- Back-of-the-envelope complexity calculations.
- Carefully developed exercises to acquaint you with intricacies of distributed algorithms.

Message passing

The two main paradigms to capture communication in a distributed system are message passing and shared memory.

We'll only consider message passing.

(The course *Concurrency & Multithreading* is dedicated to shared memory.)

Asynchronous communication means that sending and receiving of a message are *independent events*.

In case of synchronous communication, sending and receiving of a message are coordinated to form a *single event*; a message is only allowed to be sent if its destination is ready to receive it.

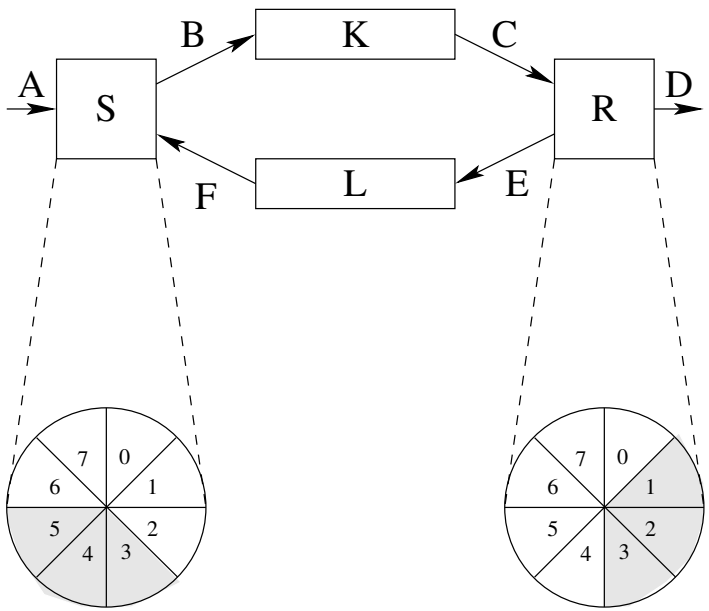
We'll mainly consider asynchronous communication.

Communication protocols

In a computer network, messages are transported through a medium, which may lose, duplicate or garble these messages.

A communication protocol detects and corrects such flaws during message passing.

Example: Sliding window protocols.



Assumptions

Unless stated otherwise, we assume:

- a strongly connected network
- message passing communication
- asynchronous communication
- the delay of messages in channels is arbitrary but finite
- channels are non-FIFO
- processes don't crash
- channels don't lose, duplicate or garble messages
- each process knows only its neighbors
- processes have unique id's

Directed versus undirected channels

Channels can be *directed* or *undirected*.

Question: What is more general, an algorithm for a directed or for an undirected network ?

Remarks:

- Algorithms for undirected channels often include ack's.
- Acyclic networks must always be undirected (else the network wouldn't be strongly connected).

Complexity measures

Resource consumption of an execution of a distributed algorithm can be considered in several ways.

Message complexity: Total number of messages exchanged.

Bit complexity: Total number of bits exchanged.
(Only interesting when messages can be very long.)

Time complexity: Amount of time consumed.
(We assume: (1) event processing takes no time, and (2) a message is received at most one time unit after it is sent.)

Space complexity: Amount of memory needed for the processes.

Different executions require different consumption of resources.

We consider worst- and average-case complexity (the latter with a probability distribution over all executions).

Big O notation

Complexity measures state how resource consumption (messages, time, space) grows in relation to input size.

For example, if an algorithm has a worst-case message complexity of $O(n^2)$, then for an input of size n, the algorithm in the worst case takes *in the order of* n^2 messages.

Let $f, g : \mathbb{N} \to \mathbb{R}_{>0}$.

$f = O(g)$ if, for some $C > 0$, $f(n) \leq C \cdot g(n)$ for all $n \in \mathbb{N}$.

$f = \Theta(g)$ if $f = O(g)$ and $g = O(f)$.

Formal framework

Now follows a formal framework for describing distributed systems, mainly to fix terminology.

In this course, correctness proofs and complexity estimations of distributed algorithms are presented in an *informal* fashion.

(The course *Protocol Validation* treats algorithms and tools to prove correctness of distributed algorithms and network protocols.)

Transition systems

The (global) state of a distributed system is called a configuration.

The configuration evolves in discrete steps, called transitions.

A transition system consists of:

- a set $\mathcal{C}$ of configurations;
- a binary transition relation $\longrightarrow$ on $\mathcal{C}$; and
- a set $\mathcal{I} \subseteq \mathcal{C}$ of initial configurations.

$\gamma \in \mathcal{C}$ is terminal if $\gamma \longrightarrow \delta$ for no $\delta \in \mathcal{C}$.

Executions

An execution is a sequence $\gamma_0\,\gamma_1\,\gamma_2\,\cdots$ of configurations that either is infinite or ends in a terminal configuration, such that:

- $\gamma_0 \in \mathcal{I}$, and
- $\gamma_i \rightarrow \gamma_{i+1}$ for all $i \geq 0$

  (excluding, for finite executions, the terminal γ_i at the end).

A configuration δ is reachable if there is a $\gamma_0 \in \mathcal{I}$ and a sequence $\gamma_0\,\gamma_1\,\gamma_2\,\cdots\,\gamma_k = \delta$ with $\gamma_i \rightarrow \gamma_{i+1}$ for all $0 \leq i < k$.

States and events

A configuration of a distributed system is composed from the states at its processes, and the messages in its channels.

A transition is associated to an event (or, in case of synchronous communication, two events) at one (or two) of its processes.

A process can perform internal, send and receive events.

A process is an initiator if its first event is an internal or send event.

An algorithm is centralized if there is exactly one initiator.

A decentralized algorithm can have multiple initiators.

Assertions

An assertion is a predicate on the configurations of an algorithm.

An assertion is a safety property if it is true in each configuration of each execution of the algorithm.

"*something bad will never happen*"

An assertion is a liveness property if it is true in some configuration of each execution of the algorithm.

"*something good will eventually happen*"

Invariants

Assertion P on configurations is an invariant if:

- $P(\gamma)$ for all $\gamma \in \mathcal{I}$, and
- if $\gamma \rightarrow \delta$ and $P(\gamma)$, then $P(\delta)$.

Each invariant is a safety property.

Question: Give a transition system S and an assertion P such that P is a safety property but not an invariant for S.

Causal order

In each configuration of an asynchronous system, applicable events at different processes are independent.

The causal order $\prec$ on occurrences of events in an execution is the smallest *transitive* relation such that:

- if a and b are events at the same process and a occurs before b, then $a \prec b$; and
- if a is a send and b the corresponding receive event, then $a \prec b$.

This relation is *irreflexive*.

$a \preceq b$ denotes $a \prec b \vee a = b$.

Computations

If neither $a \preceq b$ nor $b \preceq a$, then a and b are called concurrent.

A permutation of concurrent events in an execution doesn't affect the result of the execution.

These permutations together form a computation.

All executions of a computation start in the same initial configuration, and if they are finite, they all end in the same terminal configuration.

Question

Consider the finite execution abc.

Let $a \prec b$ be the only causal relationship.

Which executions are in the same computation ?

Lamport's clock

A logical clock C maps occurrences of events in a *computation* to a partially ordered set such that $a \prec b \Rightarrow C(a) < C(b)$.

Lamport's clock LC assigns to each event a the length k of a longest causality chain $a_1 \prec \cdots \prec a_k = a$.

LC can be computed at run-time:

Let a be an event, and k the clock value of the previous event at the same process ($k = 0$ if there is no such previous event).

* If a is an internal or send event, then $LC(a) = k + 1$.
* If a is a receive event, and b the send event corresponding to a, then $LC(a) = \max\{k, LC(b)\} + 1$.

Question

Consider the following sequences of events at processes p_0, p_1, p_2:

$$\begin{array}{lllll} p_0: & a & s_1 & r_3 & b \\ p_1: & c & r_2 & s_3 & \\ p_2: & r_1 & d & s_2 & e \end{array}$$

s_i and r_i are corresponding send and receive events, for $i = 1, 2, 3$.

Provide all events with Lamport's clock values.

Answer: 1 2 8 9

1 6 7

3 4 5 6

Vector clock

Given processes $p_0, \ldots, p_{N-1}$.

We define a *partial* order on $\mathbb{N}^N$ by:

$$(k_0, \ldots, k_{N-1}) \leq (\ell_0, \ldots, \ell_{N-1}) \Leftrightarrow k_i \leq \ell_i \text{ for all } i = 0, \ldots, N-1.$$

Vector clock VC maps occurrences of events in a computation to $\mathbb{N}^N$ such that $a \prec b \Leftrightarrow VC(a) < VC(b)$.

$VC(a) = (k_0, \ldots, k_{N-1})$ where each k_i is the length of a longest causality chain $a_1^i \prec \cdots \prec a_{k_i}^i$ of events at process p_i with $a_{k_i}^i \preceq a$.

VC can also be computed at run-time.

Question

Consider the following sequences of events at processes p_0, p_1, p_2:

$$\begin{array}{lllll} p_0: & a & s_1 & r_3 & b \\ p_1: & c & r_2 & s_3 & \\ p_2: & r_1 & d & s_2 & e \end{array}$$

s_i and r_i are corresponding send and receive events, for $i = 1, 2, 3$.

Provide all events with vector clock values.

Answer:

(1 0 0)	(2 0 0)	(3 3 3)	(4 3 3)
(0 1 0)	(2 2 3)	(2 3 3)	
(2 0 1)	(2 0 2)	(2 0 3)	(2 0 4)

Vector clock - Correctness

Let $a \prec b$.

Any causality chain for a is also one for b. So $VC(a) \leq VC(b)$.

At the process where b occurs, there is a longer causality chain for b than for a. So $VC(a) < VC(b)$.

Let $VC(a) < VC(b)$.

Consider the longest causality chain $a_1^i \prec \cdots \prec a_k^i = a$ of events at the process p_i where a occurs.

$VC(a) < VC(b)$ implies that the i-th coefficient of $VC(b)$ is $\geq k$.

So $a \preceq b$.

Since a and b are distinct, $a \prec b$.

Snapshots

A snapshot of an execution of a distributed algorithm should return a configuration of an execution *in the same computation*.

Snapshots can be used for:

- Restarting after a failure.
- Off-line determination of stable properties, which remain true as soon as they have become true.

  Examples: deadlock, garbage.
- Debugging.

Challenge: To take a snapshot without freezing the execution.

Snapshots

We distinguish basic messages of the underlying distributed algorithm and control messages of the snapshot algorithm.

A snapshot of a (basic) execution consists of:

- a local snapshot of the (basic) state of each process, and
- the channel state of (basic) messages in transit for each channel.

A snapshot is meaningful if it is a configuration of an execution in the same computation as the actual execution.

Snapshots

We need to avoid the following situations.

1. Process p takes a local snapshot, and then sends a message m to process q, where:

- q takes a local snapshot after the receipt of m, or m is included in the channel state of pq.

2. p sends m to q, and then takes a local snapshot, where:

- q takes a local snapshot before the receipt of m, and m is not included in the channel state of pq.

Chandy-Lamport algorithm

Consider a directed network with *FIFO* channels.

Initiators take a local snapshot of their state, and send a control message $\langle$**marker**$\rangle$ to their neighbors.

When a process that hasn't yet taken a snapshot receives $\langle$**marker**$\rangle$, it

- takes a local snapshot of its state, and
- sends $\langle$**marker**$\rangle$ to all its neighbors.

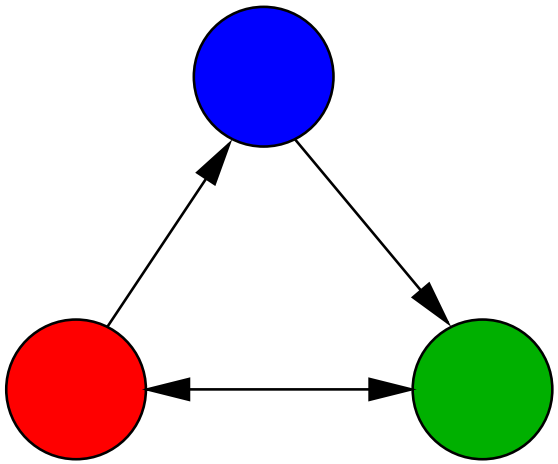
Process q computes as channel state of pq the messages it receives via pq after taking its local snapshot and before receiving $\langle$**marker**$\rangle$ from p.

If channels are FIFO, this produces a meaningful snapshot.

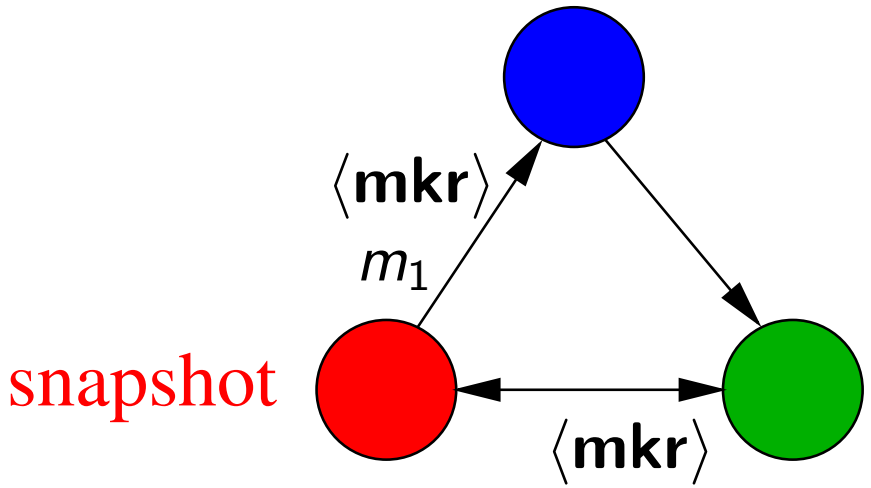
Message complexity: $\Theta(E)$

Worst-case time complexity: $O(D)$

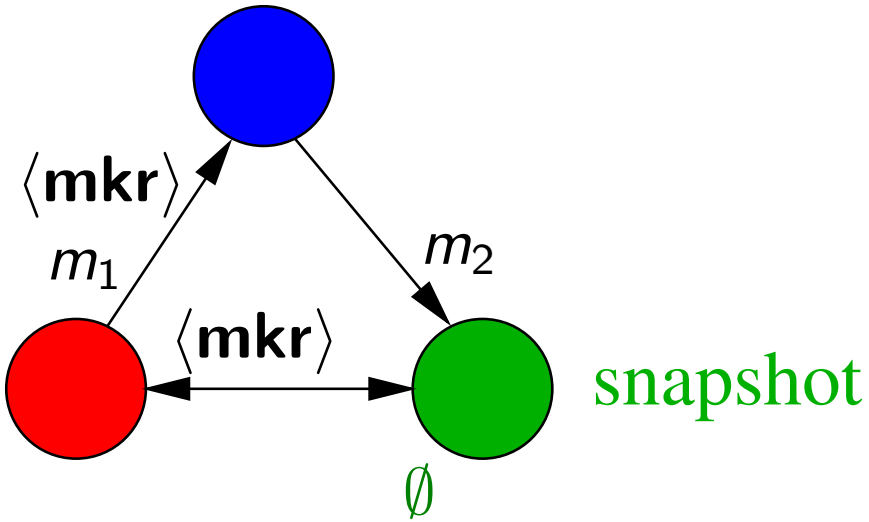
Chandy-Lamport algorithm - Example



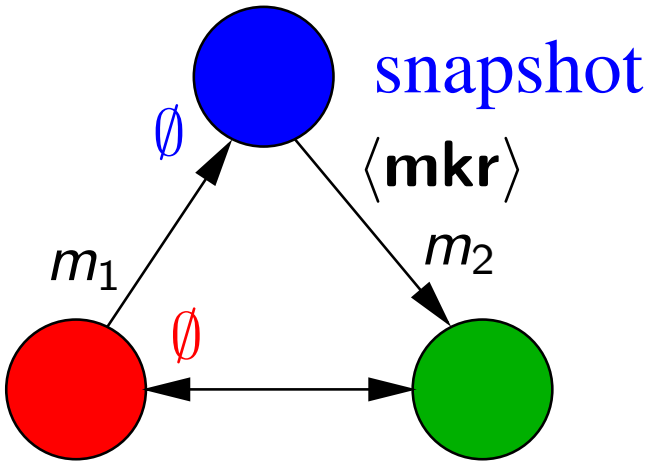
Chandy-Lamport algorithm - Example



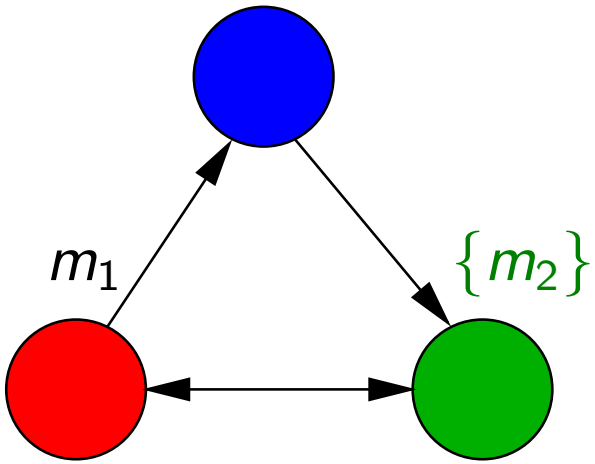
Chandy-Lamport algorithm - Example



Chandy-Lamport algorithm - Example



Chandy-Lamport algorithm - Example



The snapshot (processes red/blue/green, channels $\emptyset, \emptyset, \emptyset, \{m_2\}$) isn't a configuration in the actual execution.

The send of m_1 isn't causally before the send of m_2.

So the snapshot is a configuration of an execution that is in the same computation as the actual execution.

Chandy-Lamport algorithm - Correctness

Claim: If a post-snapshot event e is causally before an event f, then f is also post-snapshot.

This implies that the snapshot is a configuration of an execution that is in the same *computation* as the actual execution.

Proof: The case that e and f occur at the same process is trivial.

Let e be a send and f the corresponding receive event.

Let e occur at p and f at q.

e is post-snapshot at p, so p sent $\langle \textbf{marker} \rangle$ to q before e.

Channels are FIFO, so q receives this $\langle \textbf{marker} \rangle$ before f.

Hence f post-snapshot at q.

Lai-Yang algorithm

Suppose channels are *non-FIFO*. We use piggybacking.

Initiators take a local snapshot of their state.

When a process has taken its local snapshot, it appends *true* to each outgoing basic message.

When a process that hasn't yet taken a snapshot receives a message with *true* or a *control message* (see next slide) for the first time, it takes a local snapshot of its state *before reception of this message*.

Process q computes as channel state of pq the basic messages without the tag *true* that it receives via pq after its local snapshot.

Lai-Yang algorithm - Control messages

Question: How does q know when it can determine the channel state of pq ?

p sends a control message to q, informing q how many basic messages without the tag *true* p sent into pq.

These control messages also ensure that all processes eventually take a local snapshot.

Lai-Yang algorithm - Multiple snapshots

Question: How can multiple subsequent snapshots be supported ?

Answer: Each snapshot is provided with a sequence number.

Basic message carry the sequence number of the last snapshot at the sender (instead of *true* or *false*).

Control messages carry the sequence number of their snapshot.

What we need from last lecture

fully asynchronous message passing framework

channels are non-FIFO, and can be directed or undirected

configurations and transitions at the global level

states and events (*internal/send/receive*) at local level

(de)centralized algorithm; (non-)initiator

causal order $\prec$ on events in an execution

computation of executions, by reordering *concurrent* events

snapshot algorithm to compute a configuration of a *computation*

basic/control algorithm.

Wave algorithms



A decide event is a special internal event.

In a wave algorithm, each computation (also called wave) satisfies the following properties:

- *termination*: it is finite;
- *decision*: it contains one or more decide events; and
- *dependence*: for each decide event e and process p, $f \prec e$ for an event f at p.

Wave algorithms - Example

In the *ring algorithm*, the initiator sends a token, which is passed on by all other processes.

The initiator decides after the token has returned.

Question: For each process, which event is causally before the decide event ?

The ring algorithm is an example of a traversal algorithm.

Traversal algorithms

A traversal algorithm is a *centralized* wave algorithm;
i.e., there is one initiator, which sends around a token.

- In each computation, the token first visits all processes.
- Finally, the token returns to the initiator, who performs a decide event.

Traversal algorithms build a spanning tree:

- the initiator is the root; and
- each non-initiator has as parent the neighbor from which it received the token first.

Tarry's algorithm (from 1895)

Consider an undirected network.

R1 A process never forwards the token through the same channel twice.

R2 A process only forwards the token to its parent when there is no other option.

The token travels through each channel both ways, and finally ends up at the initiator.

Message complexity: $2E$ messages

Time complexity: $\leq 2E$ time units



Gaston Tarry

Tarry's algorithm - Correctness

Claim: The token θ travels through each channel in either direction, and ends up at the initiator.

Proof: A non-initiator holding θ, received θ once more than it sent θ.

So by R1, this process can send θ into a channel.

Hence θ ends at the initiator, after traversing all its channels both ways.

Assume some channel pq isn't traversed by θ both ways.

Let non-initiator p be the earliest visited process with such a channel.

p sends θ to its parent (because θ visits this parent before p).

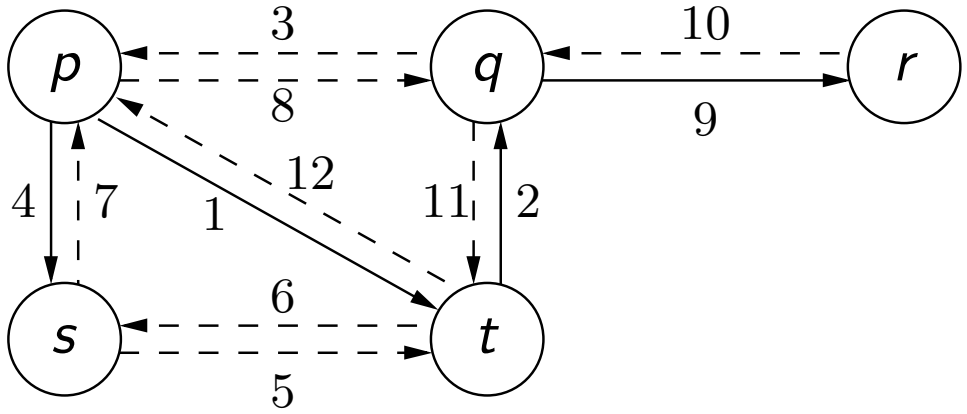
So by R2, p sends θ into all its channels.

Since p sends and receives θ an equal number of times, it also receives θ through all its channels.

So θ travels through pq both ways; contradiction.

Tarry's algorithm - Example

p is the initiator.



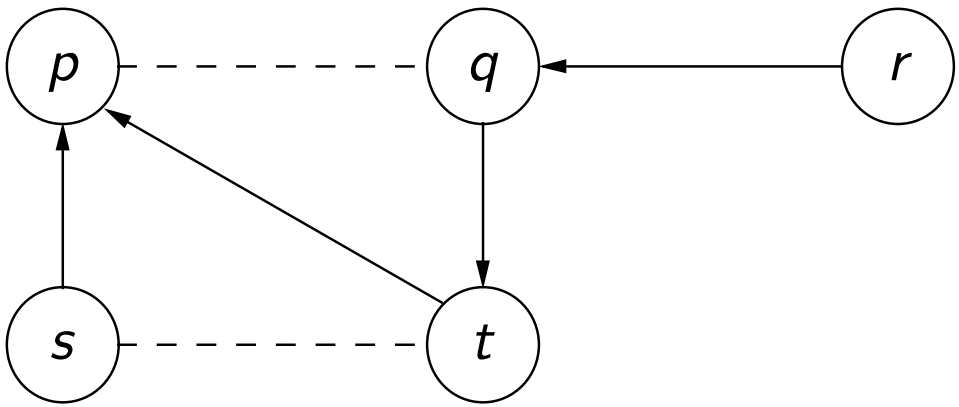
The network is undirected and unweighted.

Arrows and numbers mark the path of the token.

Solid arrows establish a parent-child relation (in the opposite direction).

Tarry's algorithm - Example

The parent-child relation is the reversal of the solid arrows.



Tree edges, which are part of the spanning tree, are solid.

Frond edges, which aren't part of the spanning tree, are dashed.

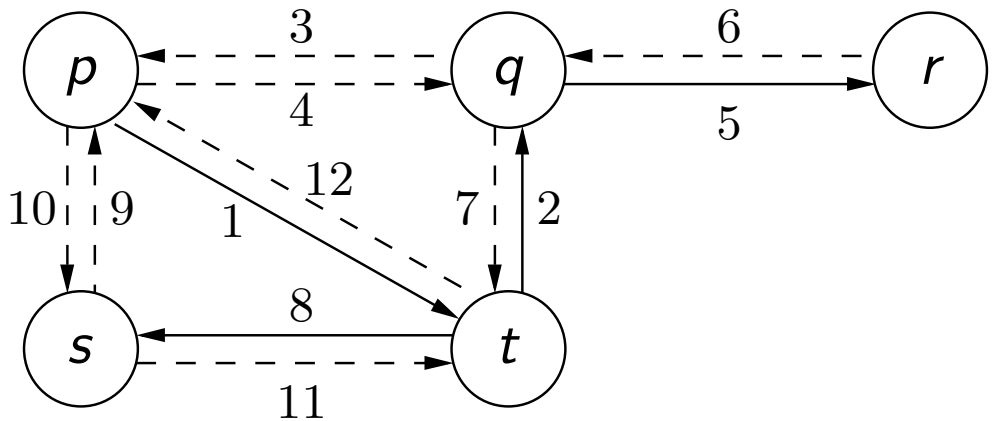
Question: Could this spanning tree have been produced by a depth-first search starting at p?

Depth-first search

Depth-first search is obtained by adding to Tarry's algorithm:

R3 When a process receives the token, it immediately sends it back through the same channel if this is allowed by R1,2.

Example:



In the spanning tree of a depth-first search, all frond edges connect an ancestor with one of its descendants in the spanning tree.

Depth-first search with neighbor knowledge

To prevent transmission of the token through a frond edge, visited processes are included in the token.

The token isn't forwarded to processes in this list (except when a process sends the token back to its parent).

Message complexity: $2N - 2$ messages

Each tree edge carries 2 tokens.

Time complexity: $\leq 2N - 2$ time units

Bit complexity: Up to kN bits per message (where k bits are needed to represent one process).

Awerbuch's algorithm

A process holding the token for the first time informs all neighbors except its parent and the process to which it forwards the token.

The token is only forwarded when these neighbors have all acknowledged reception.

The token is only forwarded to processes that weren't yet visited by the token (except when a process sends the token to its parent).

Awerbuch's algorithm - Complexity

Message complexity: $\leq 4E$ messages

Each frond edge carries 2 info and 2 ack messages.

Each tree edges carries 2 tokens, and possibly 1 info/ack pair.

Time complexity: $\leq 4N - 2$ time units

Each tree edge carries 2 tokens.

Each process waits at most 2 time units for ack's to return.

Cidon's algorithm

Abolish ack's from Awerbuch's algorithm.

The token is forwarded without delay.

Each process p records to which process fw_p it forwarded the token last.

Suppose process p receives the token from a process $q \neq fw_p$.

Then p marks pq as frond edge and *purges* the token.

Suppose process q receives an info message from fw_q.

Then q marks pq as frond edge and continues forwarding the token.

Cidon's algorithm - Complexity

Message complexity: $\leq 4E$ messages

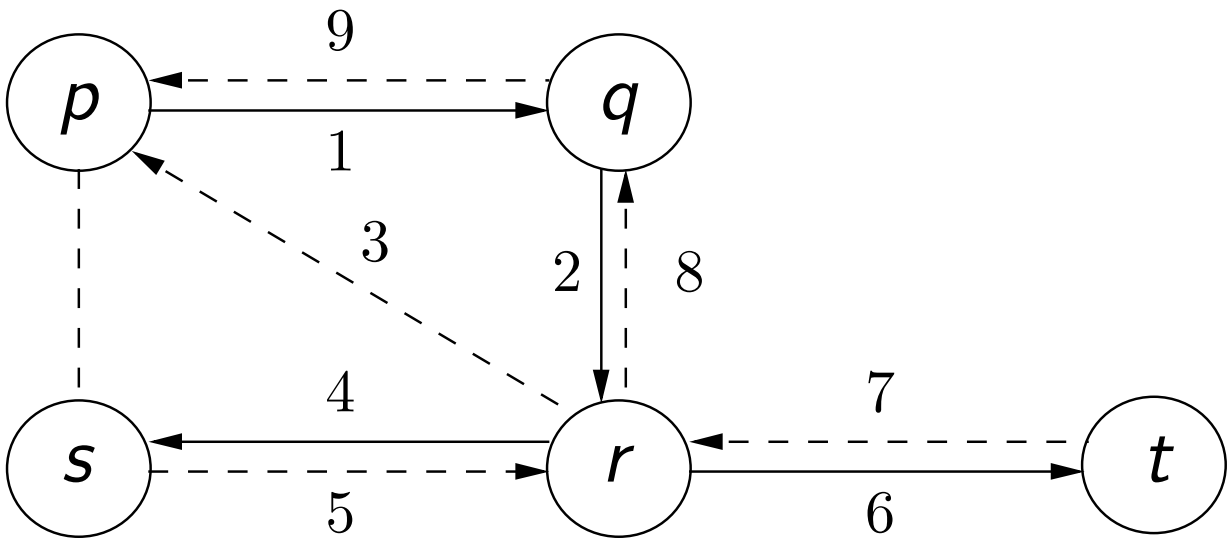
Each channel carries at most 2 info messages and 2 tokens.

Time complexity: $\leq 2N - 2$ time units

Each tree edge carries 2 tokens.

At least once per time unit, a token is forwarded through a tree edge.

Cidon's algorithm - Example



Tree algorithm

The tree algorithm is a decentralized wave algorithm for undirected, acyclic networks.



The local algorithm at a process p:

- p waits until it received messages from all neighbors except one, which becomes its *parent*.

  Then it sends a message to its parent.

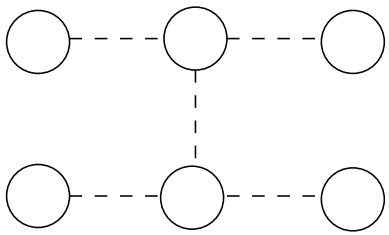
- If p receives a *message* from its parent, it decides.

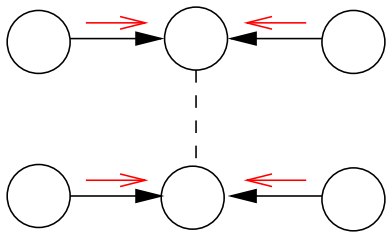
  It sends the decision to all neighbors except its parent.

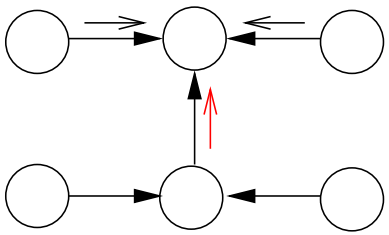
- If p receives a *decision* from its parent, it passes it on to all other neighbors.

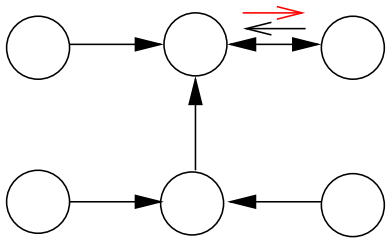
Always *two* (neighboring) processes decide.

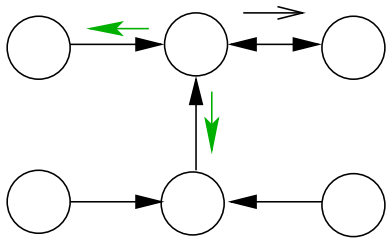
Tree algorithm - Example

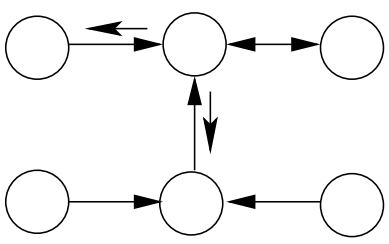


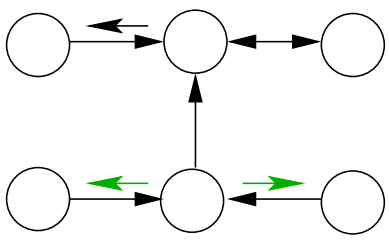


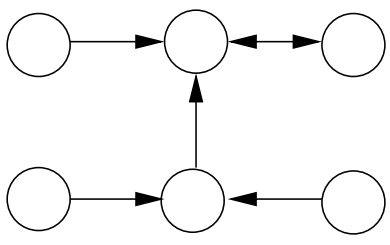












Tree algorithm - Correctness

Claim: In the tree algorithm, exactly two processes decide.

Proof : Suppose some process p never sends a message.

p doesn't receive a message through two of its channels, qp and rp.

q doesn't receive a message through two of its channels, pq and sq.

Continuing this argument, we get a cycle of processes that don't receive a message through two of their channels.

Since the network topology is a tree, there is no cycle; contradiction.

So each process eventually sends a message.

Clearly each channel carries at least one message.

There are $N - 1$ channels, so one channel carries two messages.

Only the two processes connected by this channel decide.

Questions

What happens if the tree algorithm is applied to a network containing a cycle ?

Apply the tree algorithm to compute the size of an undirected, acyclic network.

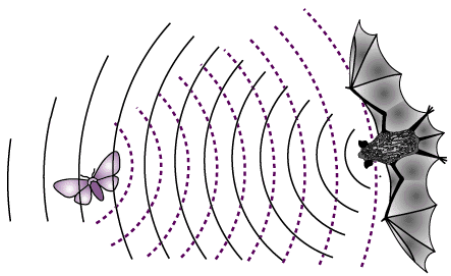
Echo algorithm

The echo algorithm is a centralized wave algorithm for undirected networks.

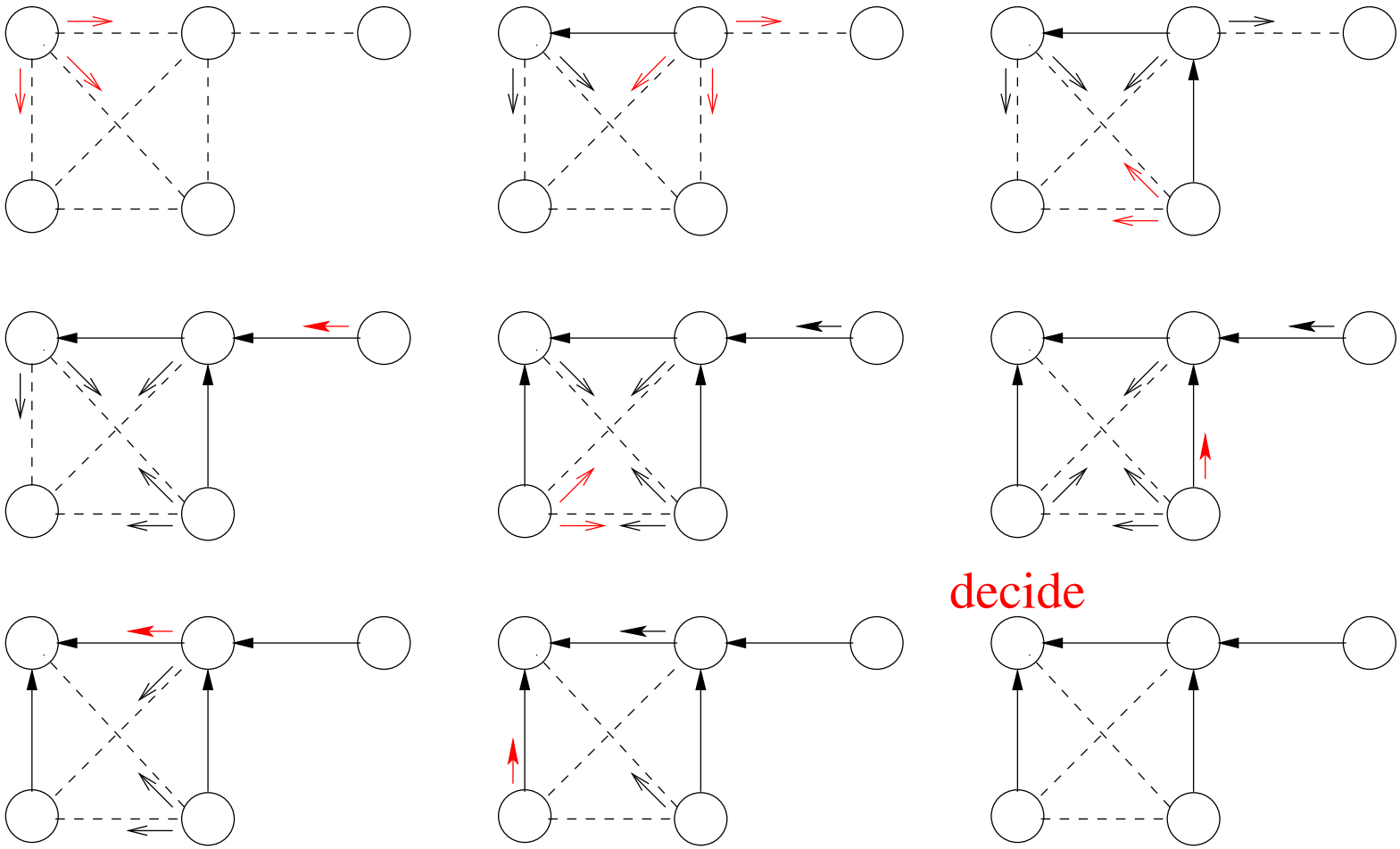
- The initiator sends a message to all neighbors.
- When a non-initiator receives a message for the first time, it makes the sender its *parent*.

  Then it sends a message to all neighbors except its parent.
- When a non-initiator has received a message from all neighbors, it sends a message to its parent.
- When the initiator has received a message from all neighbors, it *decides*.

Message complexity: $2E$ messages



Echo algorithm - Example



Questions

Use the echo algorithm to determine the largest process id.

Let each process initiate a run of the echo algorithm, tagged by its id.

Processes only participate in the "largest" wave they have seen so far.

Which of these concurrent waves complete ?

Communication and resource deadlock

A deadlock occurs if there is a cycle of processes waiting until:

- another process on the cycle sends some input (communication deadlock)
- or resources held by other processes on the cycle are released (resource deadlock)

Both types of deadlock are captured by the N-out-of-M model:
A process can wait for N grants out of M requests.

Examples:

- A process is waiting for one message from a group of processes: $N = 1$
- A database transaction first needs to lock several files: $N = M$.

Wait-for graph

A (non-blocked) process can issue a request to M other processes, and becomes blocked until N of these requests have been granted.

Then it informs the remaining $M - N$ processes that the request can be purged.

Only non-blocked processes can grant a request.

A directed wait-for graph captures dependencies between processes.

There is an edge from node p to node q if p sent a request to q that wasn't yet purged by p or granted by q.

Wait-for graph - Example 1

Suppose process p must wait for a message from process q.

In the wait-for graph, node p sends a request to node q.

Then edge pq is created in the wait-for graph, and p becomes blocked.

When q sends a message to p, the request of p is granted.

Then edge pq is removed from the wait-for graph, and p becomes unblocked.

Wait-for graph - Example 2

Suppose two processes p and q want to claim a resource.

In the wait-for graph, nodes u, v representing p, q send a request to node w representing the resource. Edges uw and vw are created.

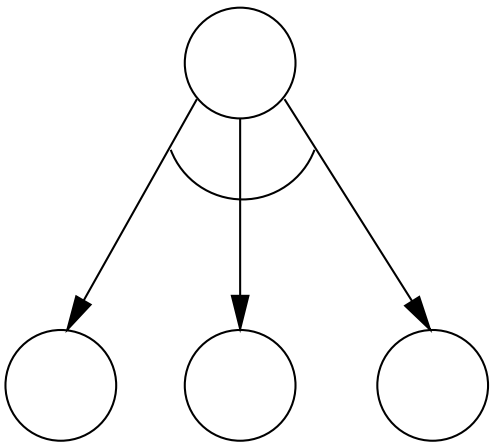
Since the resource is free, the resource is given to say p.
So w sends a grant to u. Edge uw is removed.

The basic (mutual exclusion) algorithm requires that the resource must be released by p before q can claim it.
So w sends a request to u, creating edge wu in the wait-for graph.

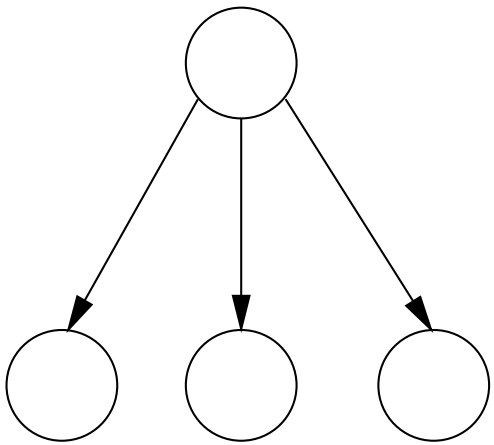
After p releases the resource, u grants the request of w.
Edge wu is removed.

The resource is given to q. Hence w grants the request from v.
Edge vw is removed and edge wv is created.

Drawing wait-for graphs



AND (3–out–of–3) request



OR (1–out–of–3) request

Questions

Draw the wait-for graph for the initial configuration of the tree algorithm, applied to an undirected cycle of three nodes.

Suppose a node in an undirected graph has three neighbors.

Which type of request does it make at the start of the tree algorithm ?

Static analysis on a wait-for graph

A snapshot is taken of the wait-for graph.

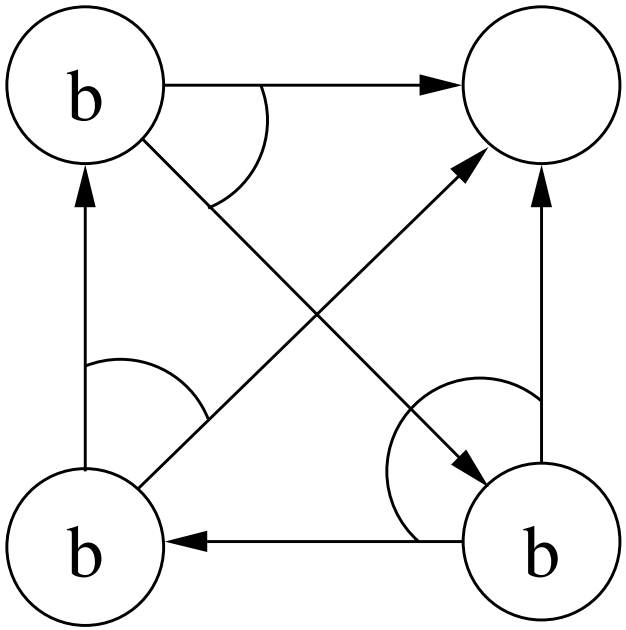
A *static analysis* on the wait-for graph may reveal deadlocks:

- Non-blocked nodes can grant requests.
- When a request is granted, the corresponding edge is removed.
- When an N-out-of-M request has received N grants, the requester becomes unblocked.

  (The remaining $M - N$ outgoing edges are purged.)

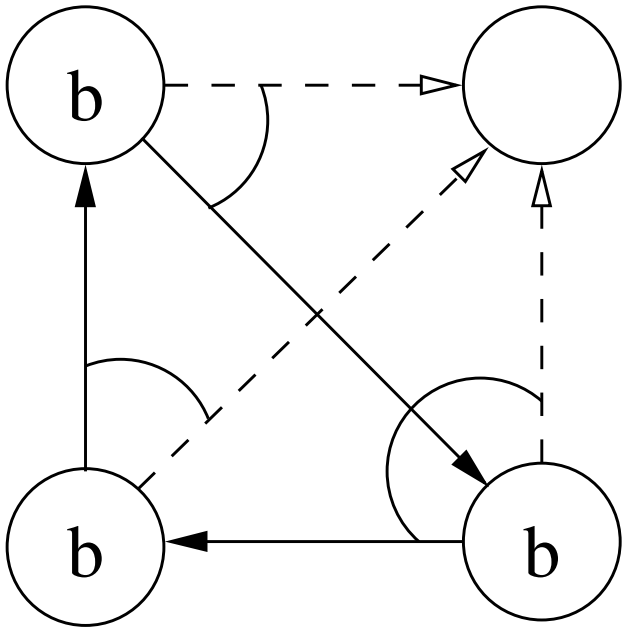
When no more grants are possible, nodes that remain blocked in the wait-for graph are deadlocked in the snapshot of the basic algorithm.

Static analysis - Example 1



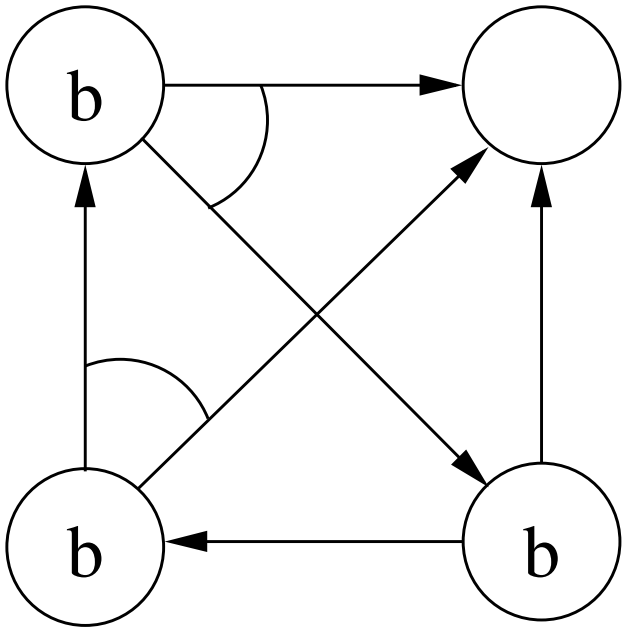
Is there a deadlock ?

Static analysis - Example 1

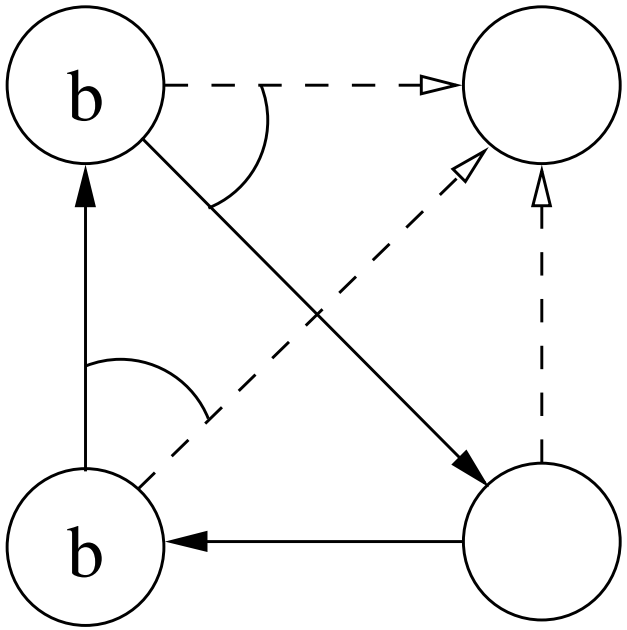


Deadlock

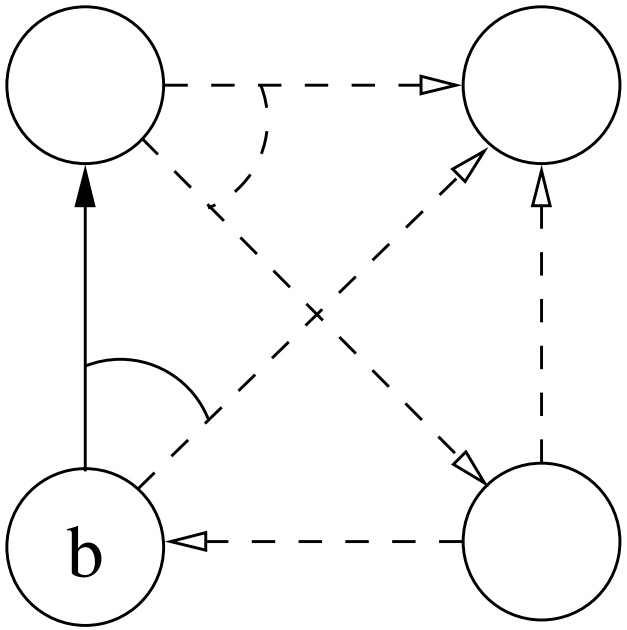
Static analysis - Example 2



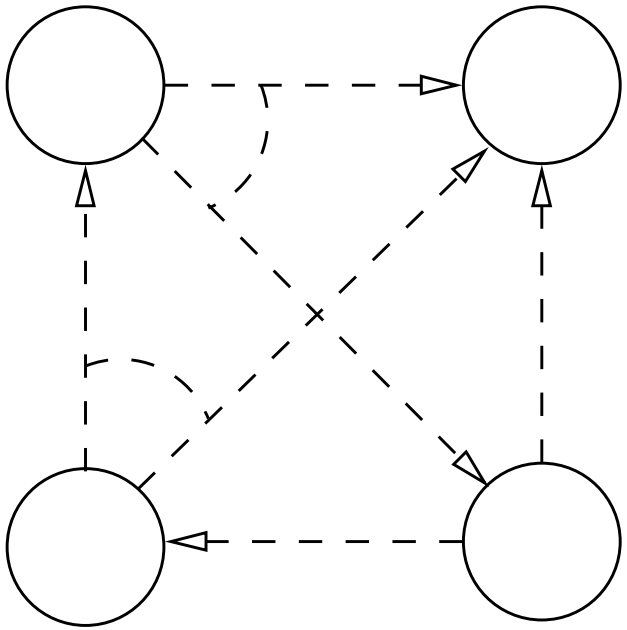
Static analysis - Example 2



Static analysis - Example 2



Static analysis - Example 2



No deadlock

Bracha-Toueg deadlock detection algorithm - Snapshot

Given an undirected network, and a basic algorithm.

A process that suspects it is deadlocked, initiates
a (Lai-Yang) *snapshot* to compute the wait-for graph.

Each node u takes a local snapshot of:

- requests it sent or received that weren't yet granted or purged;
- grant and purge messages in edges.

Then it computes:

Out_u: the nodes it sent a request to (not granted)

In_u: the nodes it received a request from (not purged)

Bracha-Toueg deadlock detection algorithm

$requests_u$ is the number of grants u requires to become unblocked.

When u receives a grant message, $requests_u \leftarrow requests_u - 1$.

If $requests_u$ becomes 0, u sends grant messages to all nodes in In_u.

If after termination of the deadlock detection run, $requests > 0$ at the initiator, then it is deadlocked (in the basic algorithm).

Challenge: The initiator must detect termination of deadlock detection.

Bracha-Toueg deadlock detection algorithm

Initially $notified_u = false$ and $free_u = false$ at all nodes u.

The initiator starts a deadlock detection run by executing *Notify*.

$Notify_u$: $notified_u \leftarrow true$
for all $w \in Out_u$ send NOTIFY to w
if $requests_u = 0$ then $Grant_u$
for all $w \in Out_u$ await DONE from w

$Grant_u$: $free_u \leftarrow true$
for all $w \in In_u$ send GRANT to w
for all $w \in In_u$ await ACK from w

While a node is awaiting DONE or ACK messages,
it can process incoming NOTIFY and GRANT messages.

Bracha-Toueg deadlock detection algorithm

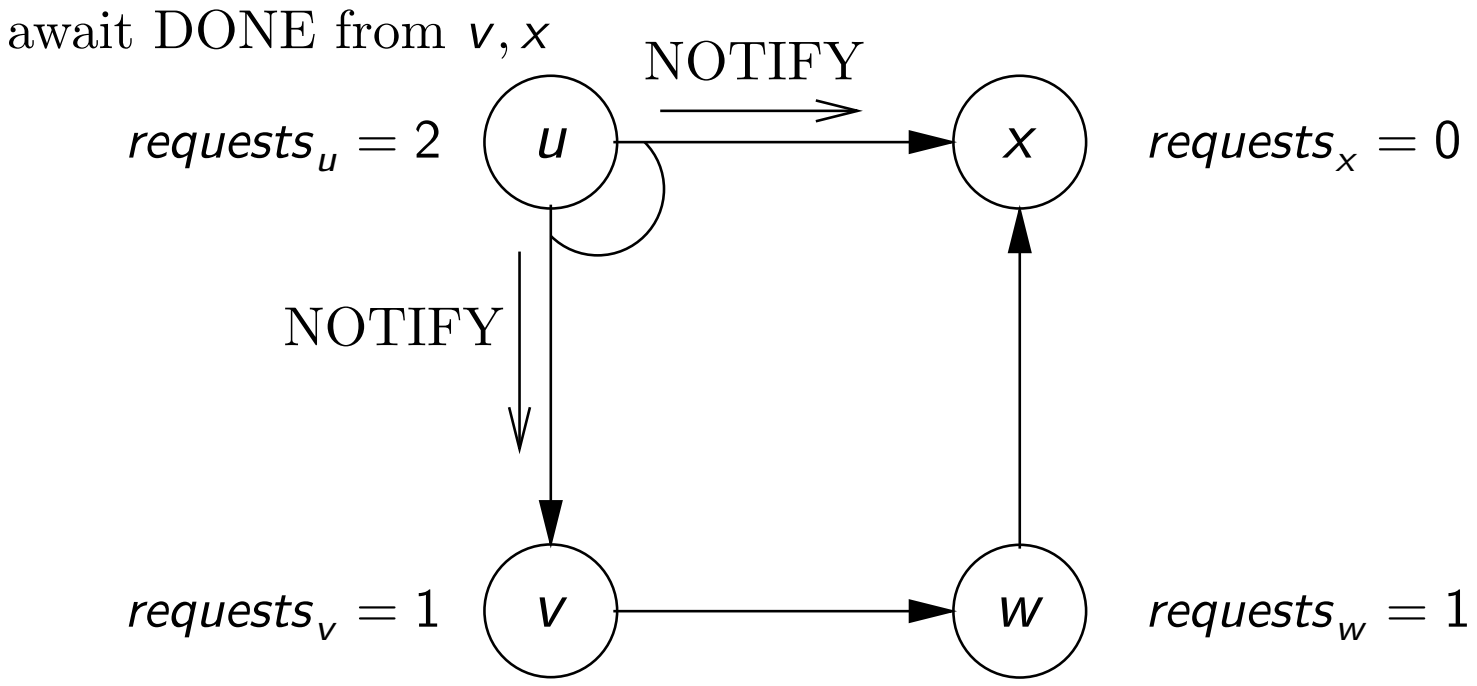
Let u receive NOTIFY.
If $notified_u = false$, then u executes $Notify_u$.
u sends back DONE.

Let u receive GRANT.
If $requests_u > 0$, then $requests_u \leftarrow requests_u - 1$;
if $requests_u$ becomes 0, then u executes $Grant_u$.
u sends back ACK.

When the initiator has received DONE from all nodes in its *Out* set, it checks the value of its *free* field.

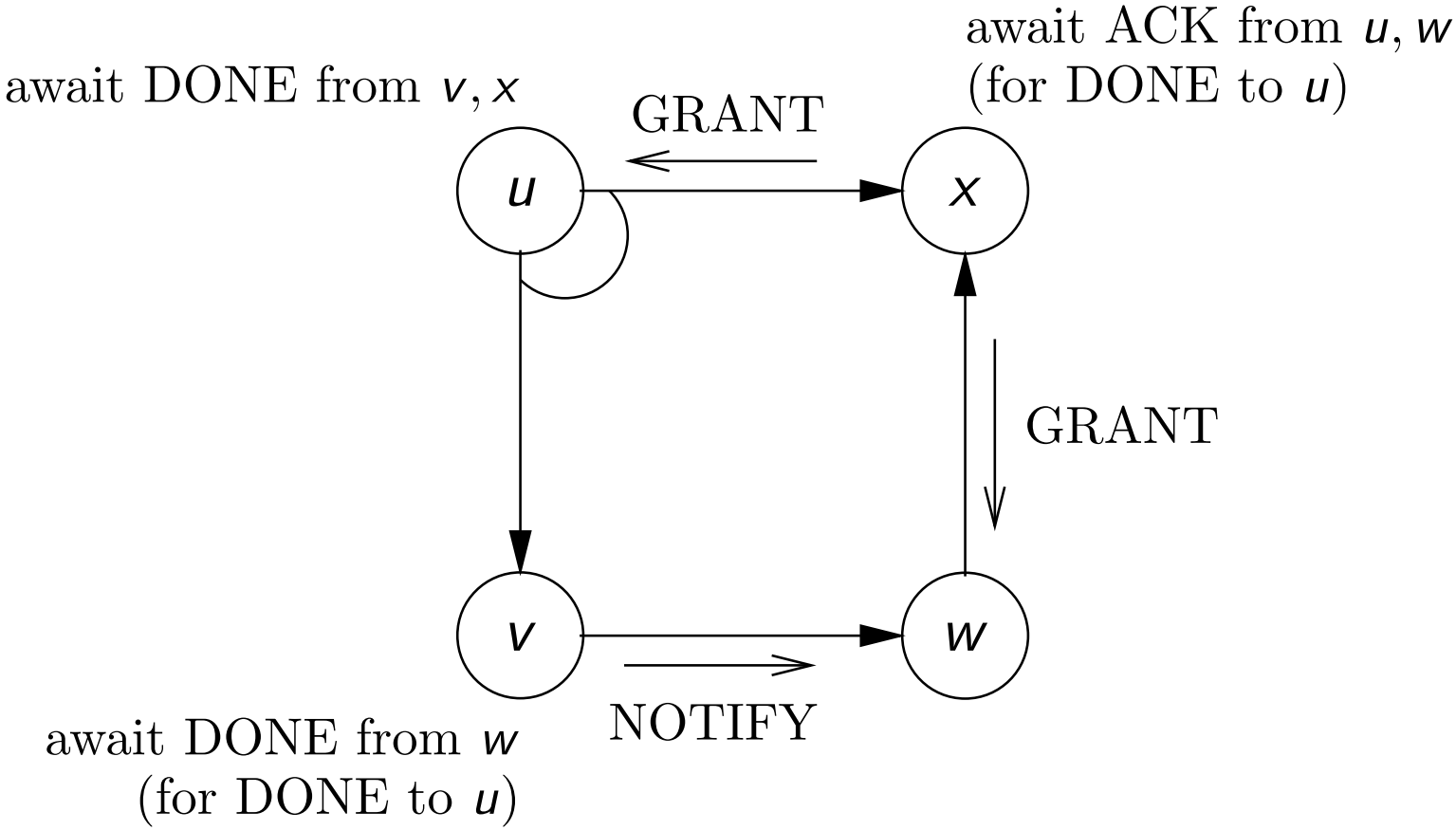
If it is still *false*, the initiator concludes it is deadlocked.

Bracha-Toueg deadlock detection algorithm - Example

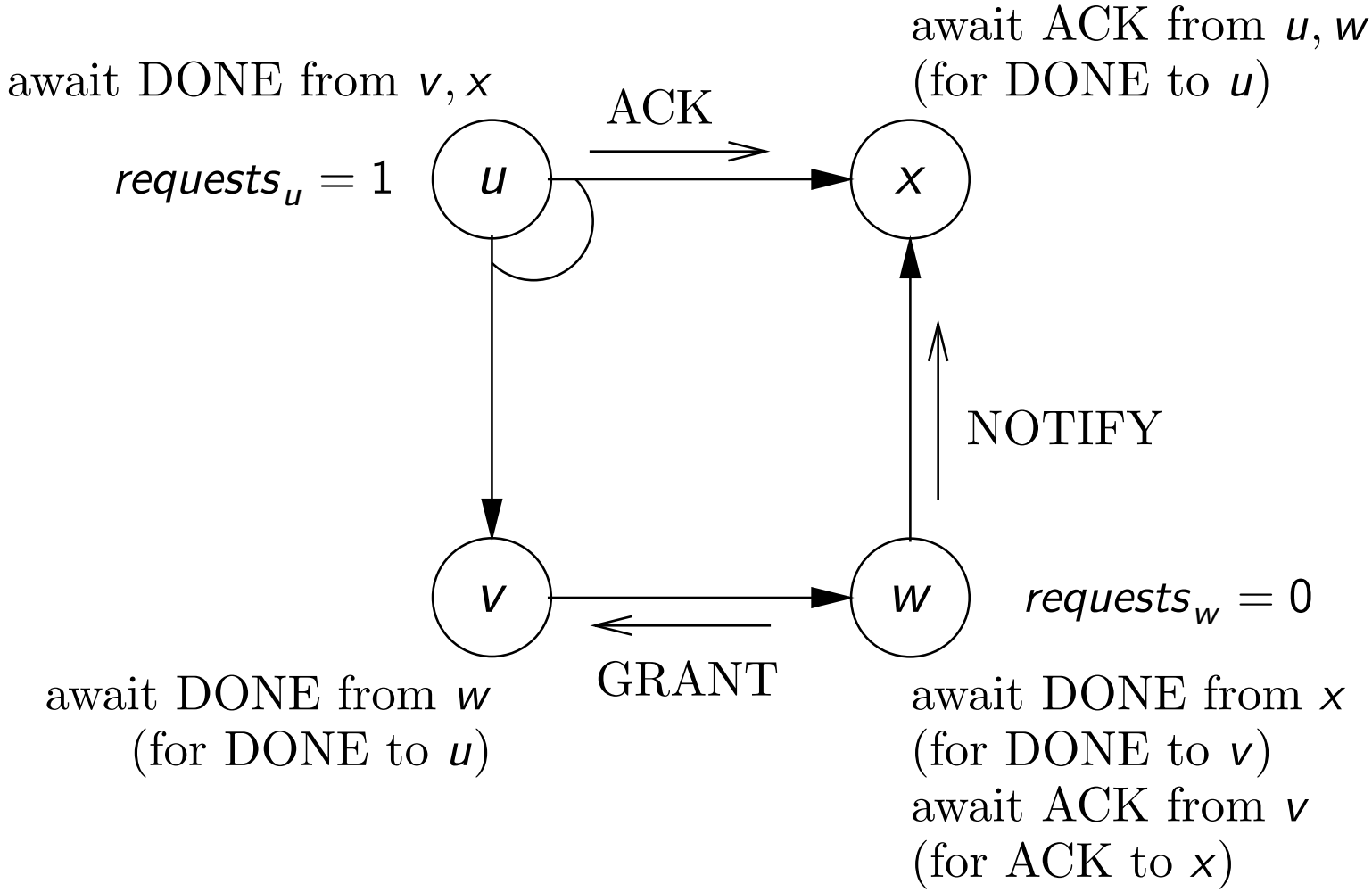


u is the initiator

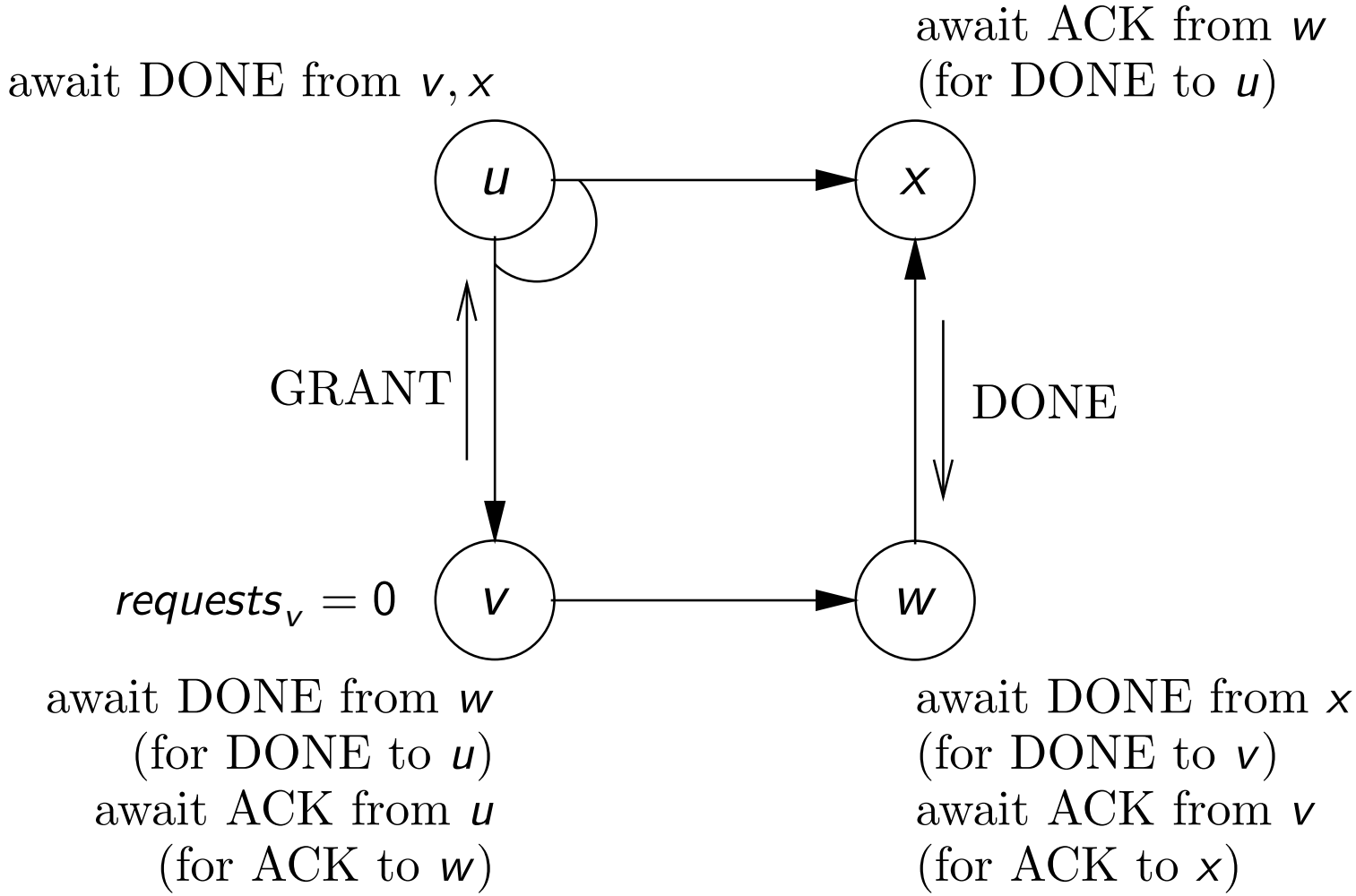
Bracha-Toueg deadlock detection algorithm - Example



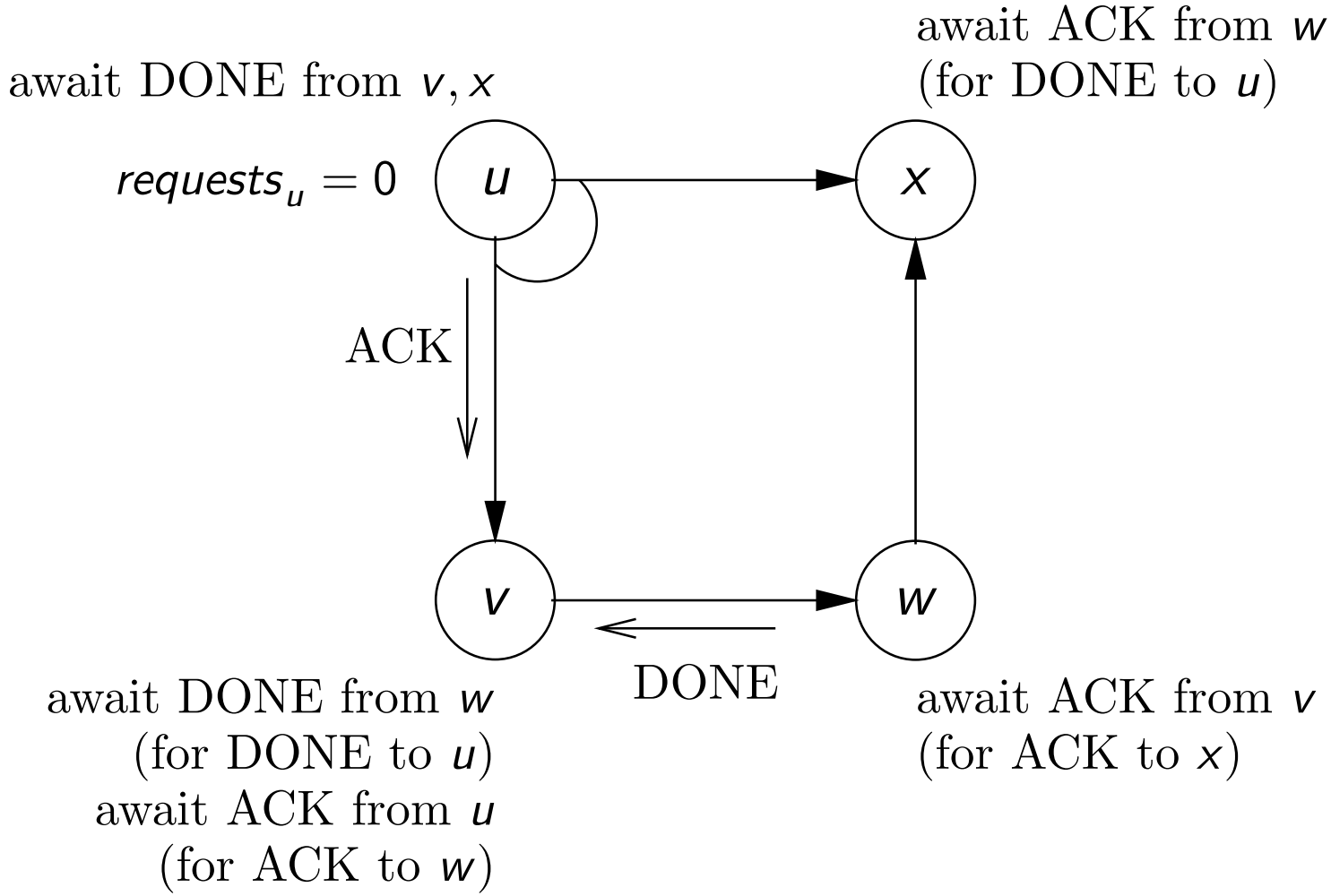
Bracha-Toueg deadlock detection algorithm - Example



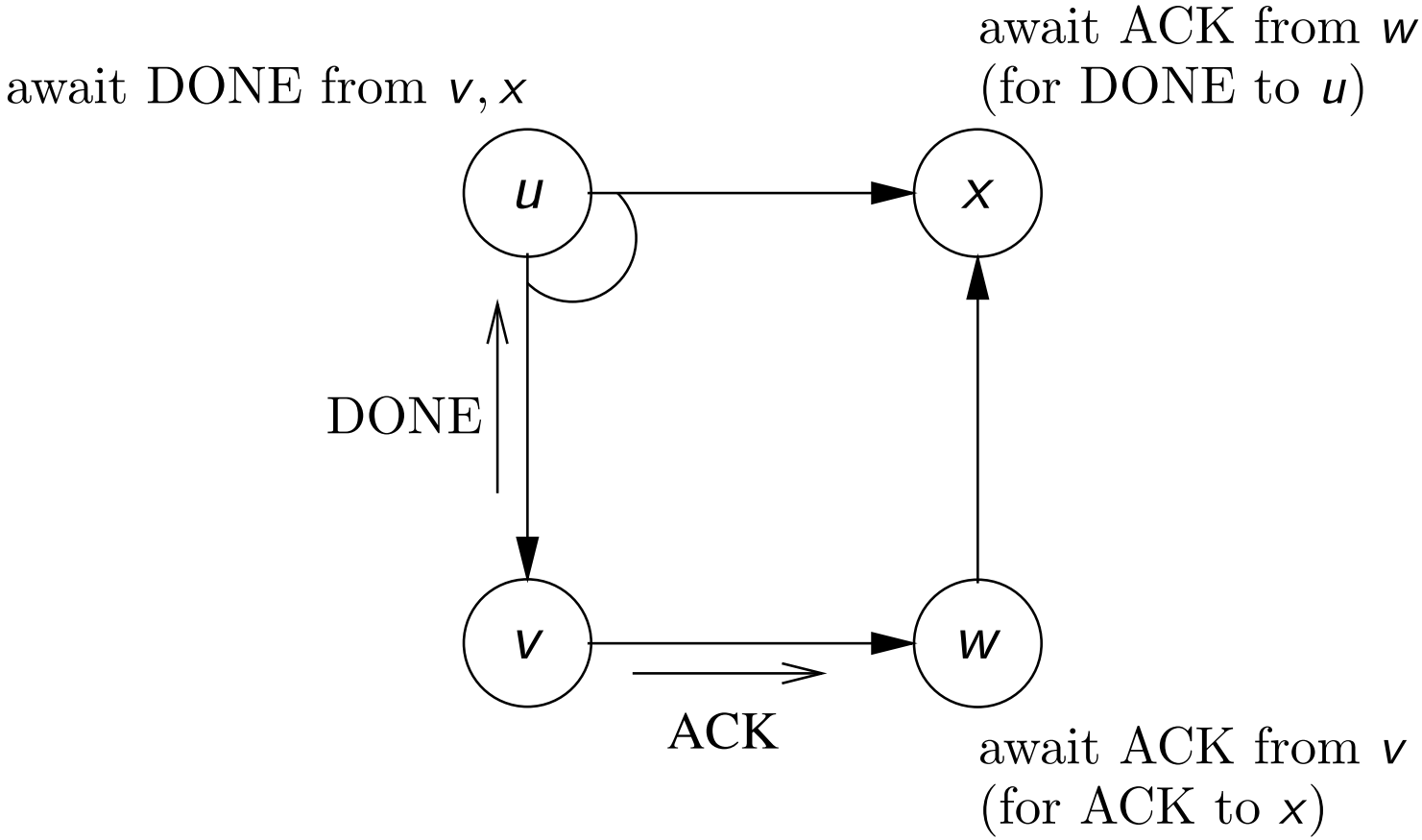
Bracha-Toueg deadlock detection algorithm - Example



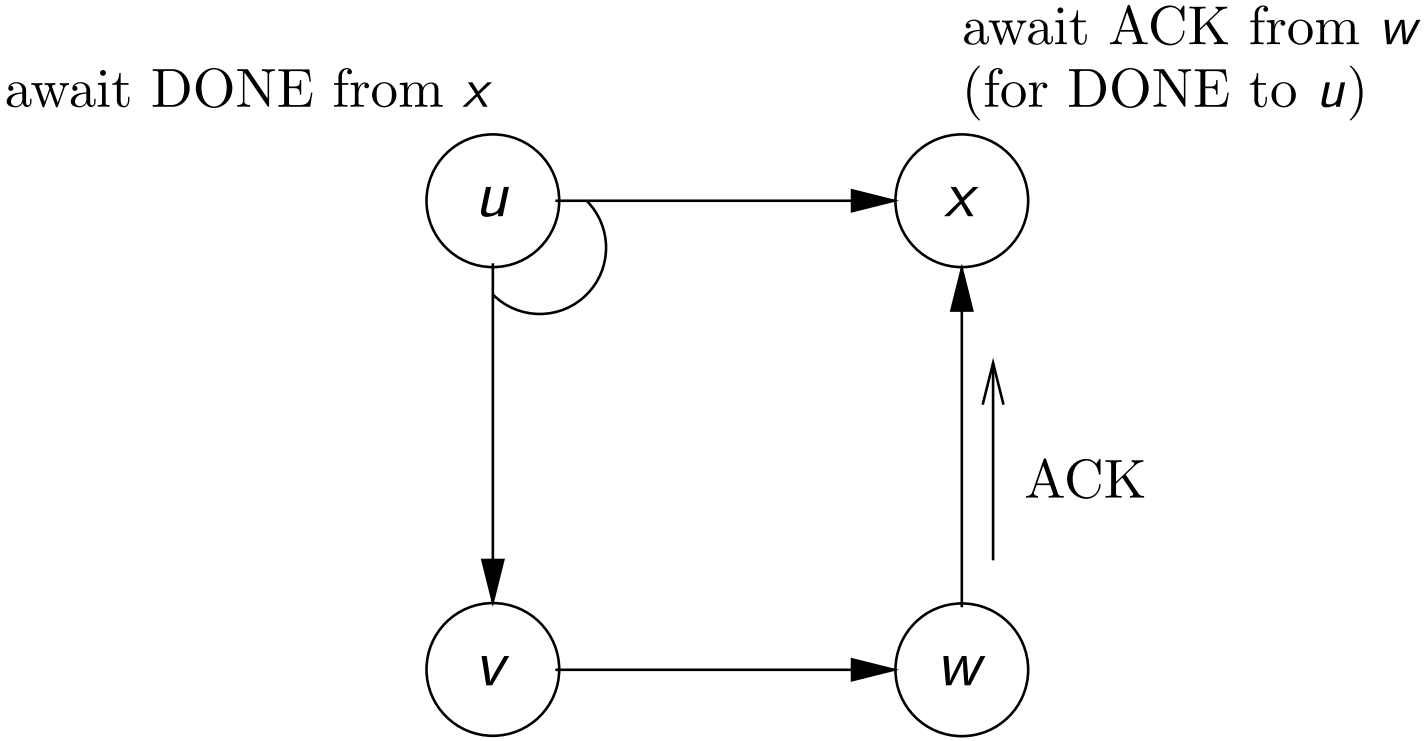
Bracha-Toueg deadlock detection algorithm - Example



Bracha-Toueg deadlock detection algorithm - Example

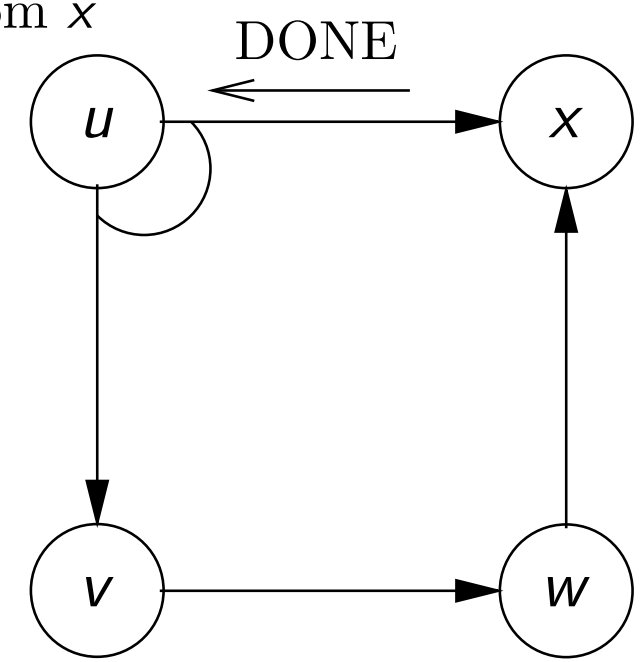


Bracha-Toueg deadlock detection algorithm - Example

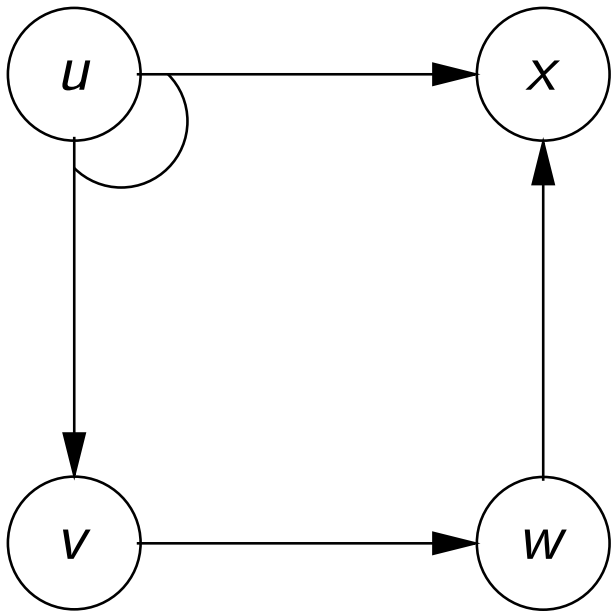


Bracha-Toueg deadlock detection algorithm - Example

await DONE from x



Bracha-Toueg deadlock detection algorithm - Example



$free_u = true$, so u concludes that it isn't deadlocked.

Bracha-Toueg deadlock detection algorithm - Correctness

The Bracha-Toueg algorithm is deadlock-free:

The initiator eventually receives DONE's from all nodes in its *Out* set.

At that moment the Bracha-Toueg algorithm has terminated.

Two types of trees are constructed, similar to the echo algorithm:

1. NOTIFY/DONE's construct a tree T rooted in the initiator.
2. GRANT/ACK's construct disjoint trees T_v,
   rooted in a node v where from the start $requests_v = 0$.

The NOTIFY/DONE's only complete when all GRANT/ACK's have completed.

Bracha-Toueg deadlock detection algorithm - Correctness

In a deadlock detection run, requests are granted as much as possible.

Therefore, if the initiator has received DONE's from all nodes in its *Out* set and its *free* field is still *false*, it is deadlocked.

Vice versa, if its *free* field is *true*, there is no deadlock (yet), *(if resource requests are granted nondeterministically)*.

Question

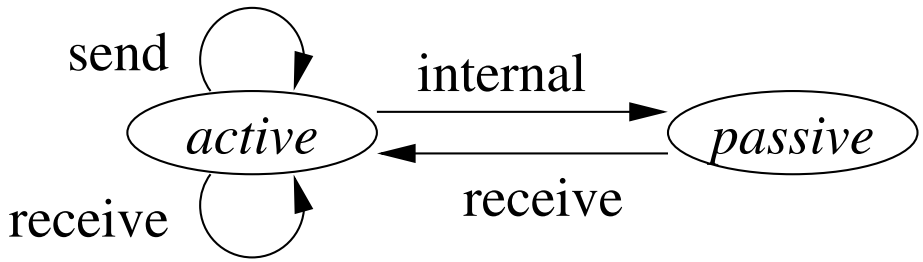
Could we apply the Bracha-Toueg algorithm to itself, to establish that it is deadlock-free ?

Answer: No.

The Bracha-Toueg algorithm can only establish whether a deadlock is present in a snapshot of one computation of the basic algorithm.

Termination detection

The *basic* algorithm is terminated if (1) each process is passive, and (2) no basic messages are in transit.



The *control* algorithm concerns termination detection and announcement.

Announcement is simple; we focus on detection.

Termination detection shouldn't influence basic computations.

Dijkstra-Scholten algorithm

Requires a centralized basic algorithm, and an undirected network.

A *tree* T is maintained, which has the initiator p_0 as the root, and includes all active processes. *Initially*, T consists of p_0.

cc_p estimates (from above) the number of children of process p in T.

- When p sends a basic message, $cc_p \leftarrow cc_p + 1$.
- Let this message be received by q.
  - If q isn't yet in T, it joins T with parent p and $cc_q \leftarrow 0$.
  - If q is already in T, it sends a control message to p that it isn't a new child of p. Upon receipt of this message, $cc_p \leftarrow cc_p - 1$.
- When a non-initiator p is passive and $cc_p = 0$, it quits T and informs its parent that it is no longer a child.
- When the initiator p_0 is passive and $cc_{p_0} = 0$, it calls *Announce*.

Question

Let the initiator send a basic message and then become passive.

Why doesn't it immediately detect termination ?

Shavit-Francez algorithm

Allows a decentralized basic algorithm; requires an undirected network.

A *forest* F of (disjoint) trees is maintained, rooted in initiators.

Initially, each initiator of the basic algorithm constitutes a tree in F.

- When a process p sends a basic message, $cc_p \leftarrow cc_p + 1$.
- Let this message be received by q.
  - If q isn't yet in *a tree in* F, it joins F with parent p and $cc_q \leftarrow 0$.
  - If q is already in *a tree in* F, it sends a control message to p that it isn't a new child of p. Upon receipt, $cc_p \leftarrow cc_p - 1$.
- When a non-initiator p is passive and $cc_p = 0$, it informs its parent that it is no longer a child.

A passive initiator p with $cc_p = 0$ starts a wave, tagged with its id.

Processes in a tree refuse to participate; *decide* calls *Announce*.

Question

What is a drawback of the Shavit-Francez algorithm ?

Answer: Requires one control message for every basic message.

Weight-throwing termination detection

Requires a centralized basic algorithm; allows a directed network.

The initiator has weight 1, all non-initiators have weight 0.

When a process *sends* a basic message, it transfers part of its weight to this message.

When a process *receives* a basic message, it adds the weight of this message to its own weight.

When a non-initiator becomes passive, it returns its weight to the initiator.

When the initiator becomes passive, and has regained weight 1, it calls *Announce*.

Weight-throwing termination detection - Underflow

Underflow: The weight of a process can become too small to be divided further.

Solution 1: The process gives itself extra weight, and informs the initiator that there is additional weight in the system.

An ack from the initiator is needed before the extra weight can be used, to avoid race conditions.

Solution 2: The process initiates a weight-throwing termination detection sub-call, and only returns its weight to the initiator when it has become passive and this sub-call has terminated.

Question

Why is the following termination detection algorithm not correct ?

- Each basic message is acknowledged.
- If a process becomes quiet, i.e. (1) it has become passive, and (2) all basic messages it sent have been acknowledged, then it starts a wave (tagged with its id).
- Only quiet processes take part in the wave.
- If the wave completes, its initiator calls *Announce*.

Answer: Let a process p that wasn't yet visited by the wave make a quiet process q that was already visited active again.

Next p becomes quiet before the wave arrives.

Now the wave can complete while q is active.

Rana's algorithm

Allows a decentralized basic algorithm; requires an undirected network.

Each basic message is acknowledged.

A logical clock provides *(basic and control)* events with a time stamp.

The time stamp of a process is the highest time stamp of its events so far (initially it is 0).

If at time t a process becomes quiet, i.e. (1) it has become passive, and (2) all basic messages it sent have been acknowledged, it starts a wave (of control messages), *tagged with* t (and its id).

Only processes *that have been quiet from a time* $\leq t$ *on* take part in the wave.

If a wave completes, its initiator calls *Announce*.

Rana's algorithm - Correctness

Suppose a wave, tagged with some t, doesn't complete.

Then some process p doesn't take part in this wave.

Due to this wave, p's logical time becomes greater than t.

When p becomes quiet, it starts a new wave, tagged with a $t' > t$.

Rana's algorithm - Correctness

Suppose a quiet process q takes part in a wave,
and is later on made active by a basic message from a process p
that wasn't yet visited by this wave.

Then this wave won't complete.

Namely, let the wave be tagged with t.

When q takes part in the wave, its logical clock becomes $> t$.

By the ack from q to p, in response to the basic message from p,
the logical clock of p becomes $> t$.

So p won't take part in the wave (because it is tagged with t).

Token-based termination detection

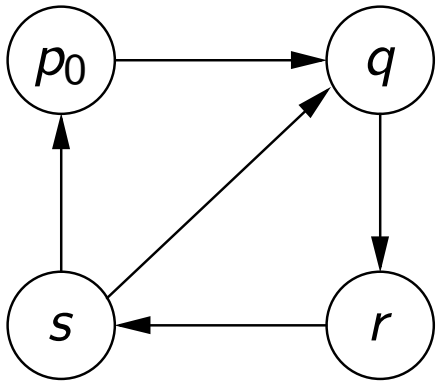
The following centralized termination detection algorithm allows a decentralized basic algorithm and a directed network.

A process p_0 is initiator of a traversal algorithm to check whether all processes are passive.

Complication 1: Due to the directed channels, reception of basic messages can't be acknowledged.

Complication 2: A traversal of only passive processes doesn't guarantee termination (even if there are no basic messages in the channels).

Complication 2 - Example



The token is at p_0; only s is active.

The token travels to r.

s sends a basic message to q, making q active.

s becomes passive.

The token travels on to p_0, which falsely calls *Announce*.

Safra's algorithm

Allows a decentralized basic algorithm and a directed network.

Each process maintains a counter of type $\mathbb{Z}$; initially it is 0.

At each outgoing/incoming basic message, the counter is increased/decreased.

At any time, the sum of all counters in the network is ≥ 0, and it is 0 if and only if no basic messages are in transit.

At each round trip, the token carries the sum of the counters of the processes it has traversed.

Complication: The token may end a round trip with a negative sum, when a visited passive process becomes active by a basic message, and sends basic messages that are received by an unvisited process.

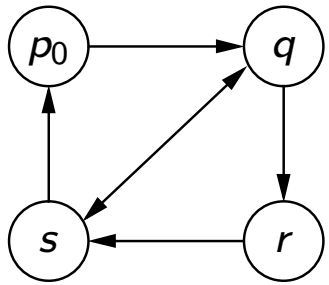
Safra's algorithm

Processes are colored white or black. Initially they are white, and a process that receives a basic message becomes black.

- When p_0 is passive, it becomes white and sends a white token.
- A non-initiator only forwards the token when it is passive.
- When a *black* process receives the (black or white) token, the process becomes white and the token black.
- Eventually the token returns to p_0, who waits until it is passive:
  - If the token is white and the sum of all counters is zero, p_0 calls *Announce*.
  - Else, p_0 sends a white token again.

Safra's algorithm - Example

The token is at p_0; only s is active; no messages are in transit; all processes are white with counter 0.



s sends a basic message **m** to q, setting the counter of s to 1.

s becomes passive.

The token travels around the network, white with sum 1.

The token travels on to r, white with sum 0.

m travels to q and back to s, making them active, black, with counter 0.

s becomes passive.

The token travels from r to p_0, black with sum 0.

q becomes passive.

After two more round trips of the token, p_0 calls *Announce*.

Safra's algorithm - Correctness

When the system has terminated,

- the token will color all processes white, and
- the counters of the processes sum up to zero.

So the token eventually returns to the initiator white with counter 0.

Suppose a token returns to the initiator white with counter 0.

Since the token is white: if reception of a message is included in the counter, then sending this message is included in the counter too.

So, since the counter is 0:

- no process was made active after the token's visit, and
- no messages are in transit.

Question

Any suggestions for an optimization of Safra's algorithm ?

(Hint: Can we do away with black tokens ?)

Answer: When a *black* process gets the token, it purges the token (and becomes white).

When the process becomes passive, it sends a fresh token, tagged with its id.

Garbage collection

Processes are provided with memory.

Objects carry *pointers* to local objects and *references* to remote objects.

A root object can be created in memory; objects are always accessed by navigating from a root object.

Aim of garbage collection: To reclaim inaccessible objects.

Three operations by processes to build or delete a reference:

- *Creation*: The object owner sends a pointer to another process.
- *Duplication*: A process that isn't object owner sends a reference to another process.
- *Deletion*: The reference is deleted at its process.

Reference counting

Reference counting tracks the number of references to an object.

If it drops to zero, and there are no pointers, the object is garbage.

Advantage: Can be performed at run-time.

Drawback: Can't reclaim *cyclic* garbage.

Indirect reference counting

A tree is maintained for each object, with the object at the root, and the references to this object as the other nodes in the tree.

Each object maintains a counter how many references to it have been *created*.

Each reference is supplied with a counter how many times it has been *duplicated*.

References keep track of their parent in the tree, where they were duplicated or created from.

Indirect reference counting

If a process receives a reference, but already holds a reference to or owns this object, it sends back a decrement.

When a duplicated (or created) reference has been deleted, and its counter is zero, a decrement is sent to the process it was duplicated from (or to the object owner).

When the counter of the object becomes zero, and there are no pointers to it, the object can be reclaimed.

Weighted reference counting

Each object carries a total weight (equal to the weights of all references to the object), and a partial weight.

When a reference is created, the partial weight of the object is divided over the object and the reference.

When a reference is duplicated, the weight of the reference is divided over itself and the copy.

When a reference is deleted, the object owner is notified, and the weight of the deleted reference is subtracted from the total weight of the object.

If the total weight of the object becomes equal to its partial weight, and there are no pointers to the object, it can be reclaimed.

Weighted reference counting - Underflow

When the weight of a reference (or object) becomes too small
to be divided further, no more duplication (or creation) is possible.

Solution 1: The reference increases its weight,
and tells the object owner to increase its total weight.

An ack from the object owner to the reference is needed before
the additional weight can be used, to avoid race conditions.

Solution 2: The process at which the underflow occurs
creates an artificial object with a new total weight,
and with a reference to the original object.

Duplicated references are then to the artificial object,
so that references to the original object become *indirect*.

Question

Why is it much more important to address underflow of weight than overflow of a reference counter ?

Answer: At each reference creation and duplication, weight decreases *exponentially* fast, while the reference counter increases *linearly*.

Garbage collection $\Rightarrow$ termination detection

Garbage collection algorithms can be *transformed* into (existing and new) termination detection algorithms.

Given a basic algorithm.

Let each process p host one artificial root object O_p.

There is also a special non-root object Z.

Initially, only initiators p hold a reference from O_p to Z.

Each basic message carries a duplication of the Z-reference.

When a process becomes passive, it deletes its Z-reference.

The basic algorithm is terminated if and only if Z is garbage.

Garbage collection $\Rightarrow$ termination detection - Examples

Indirect *reference counting* $\Rightarrow$ Dijkstra-Scholten *termination detection*.

Weighted *reference counting* $\Rightarrow$ weight-throwing *termination detection*.

Mark-scan

Mark-scan garbage collection consists of two phases:

- A traversal of all accessible objects, which are marked.
- All unmarked objects are reclaimed.

Drawback: In a distributed setting, mark-scan usually requires *freezing the basic computation*.

In mark-copy, the second phase consists of copying all marked objects to contiguous empty memory space.

In mark-compact, the second phase compacts all marked objects without requiring empty space.

Copying is significantly faster than compaction,
but leads to fragmentation of the memory space.

Generational garbage collection

In practice, most objects either can be reclaimed shortly after their creation, or stay accessible for a very long time.

Garbage collection in Java, which is based on mark-scan, therefore divides objects into multiple generations.

- Garbage in the youngest generation is collected *frequently* using mark-*copy*.
- Garbage in the older generations is collected *sporadically* using mark-*compact*.

Routing

Routing means guiding a packet in a network to its destination.

A routing table at node u stores for each $v \neq u$ a neighbor w of u:
Each packet with destination v that arrives at u is passed on to w.

Criteria for good routing algorithms:

- use of optimal paths
- robust with respect to topology changes in the network
- table adaptation to avoid busy edges

Chandy-Misra algorithm

Consider an undirected, weighted network, with weights $\omega_{vw} > 0$.

A *centralized* algorithm to compute all shortest paths to initiator u_0.

Initially, $dist_{u_0}(u_0) = 0$, $dist_v(u_0) = \infty$ if $v \neq u_0$, and $parent_v(u_0) = \perp$.

u_0 sends the message $\langle 0 \rangle$ to its neighbors.

Let node v receives $\langle d \rangle$ from neighbor w. If $d + \omega_{vw} < dist_v(u_0)$, then:

- $dist_v(u_0) \leftarrow d + \omega_{vw}$ and $parent_v(u_0) \leftarrow w$
- v sends $\langle dist_v(u_0) \rangle$ to its neighbors (except w)

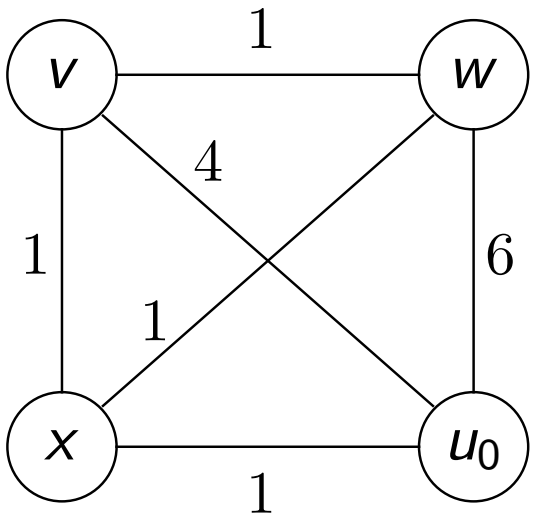
Termination detection by e.g. the Dijkstra-Scholten algorithm.

Question

Why is Rana's algorithm here not a good choice for detecting termination ?

Chandy-Misra algorithm - Example

$dist_{u_0}$	$\leftarrow$	0	$parent_{u_0}$	$\leftarrow$	$\perp$
$dist_w$	$\leftarrow$	6	$parent_w$	$\leftarrow$	u_0
$dist_v$	$\leftarrow$	7	$parent_v$	$\leftarrow$	w
$dist_x$	$\leftarrow$	8	$parent_x$	$\leftarrow$	v
$dist_x$	$\leftarrow$	7	$parent_x$	$\leftarrow$	w
$dist_v$	$\leftarrow$	4	$parent_v$	$\leftarrow$	u_0
$dist_w$	$\leftarrow$	5	$parent_w$	$\leftarrow$	v
$dist_x$	$\leftarrow$	6	$parent_x$	$\leftarrow$	w
$dist_x$	$\leftarrow$	5	$parent_x$	$\leftarrow$	v
$dist_x$	$\leftarrow$	1	$parent_x$	$\leftarrow$	u_0
$dist_w$	$\leftarrow$	2	$parent_w$	$\leftarrow$	x
$dist_v$	$\leftarrow$	3	$parent_v$	$\leftarrow$	w
$dist_v$	$\leftarrow$	2	$parent_v$	$\leftarrow$	x



Chandy-Misra algorithm - Complexity

Worst-case message complexity: Exponential

Worst-case message complexity for minimum-hop: $O(N^2 \cdot E)$

For each root, the algorithm requires at most $O(N \cdot E)$ messages.

Merlin-Segall algorithm

A *centralized* algorithm to compute all shortest paths to initiator u_0.

Initially, $dist_{u_0}(u_0) = 0$, $dist_v(u_0) = \infty$ if $v \neq u_0$,
and the $parent_v(u_0)$ values form a sink tree with root u_0.

Each round, u_0 sends $\langle 0 \rangle$ to its neighbors.

1. Let node v get $\langle d \rangle$ from neighbor w.

   If $d + \omega_{vw} < dist_v(u_0)$, then $dist_v(u_0) \leftarrow d + \omega_{vw}$
   (and v stores w as future value for $parent_v(u_0)$).

   If $w = parent_v(u_0)$, then v sends $\langle dist_v(u_0) \rangle$ to its neighbors
   except $parent_v(u_0)$.

2. When a $v \neq u_0$ has received a message from all neighbors,
   it sends $\langle dist_v(u_0) \rangle$ to $parent_v(u_0)$, and updates $parent_v(u_0)$.

u_0 starts a new round after receiving a message from all neighbors.

Merlin-Segall algorithm - Termination + complexity

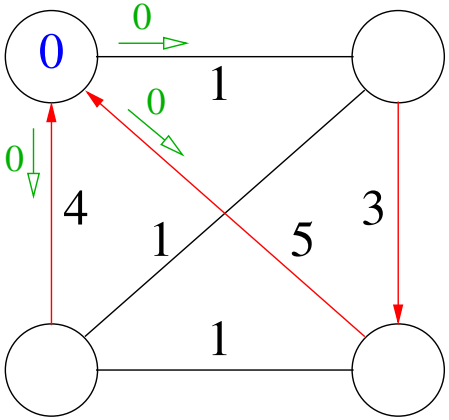
After i rounds, all shortest paths of $\leq i$ hops have been computed.

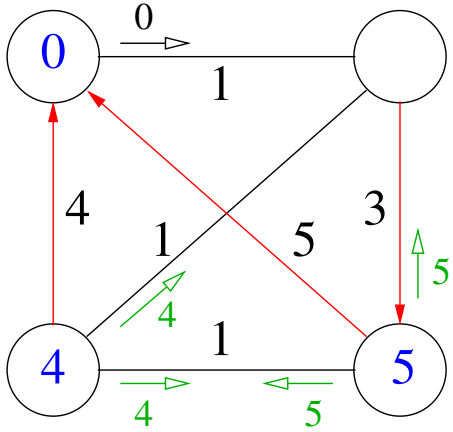
So the algorithm can terminate after $N - 1$ rounds.

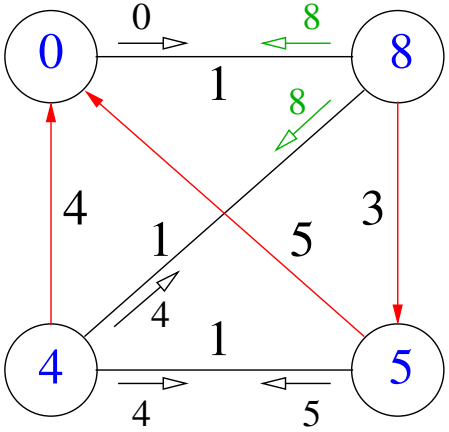
Message complexity: $\Theta(N^2 \cdot E)$

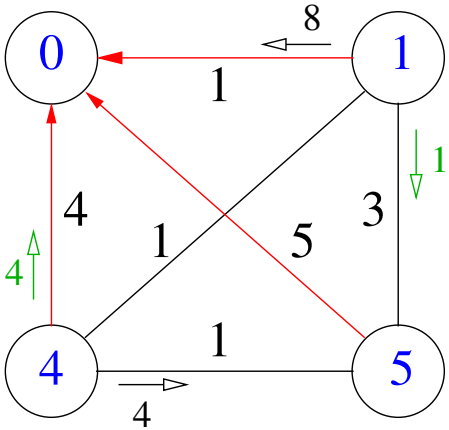
For each root, the algorithm requires $\Theta(N \cdot E)$ messages.

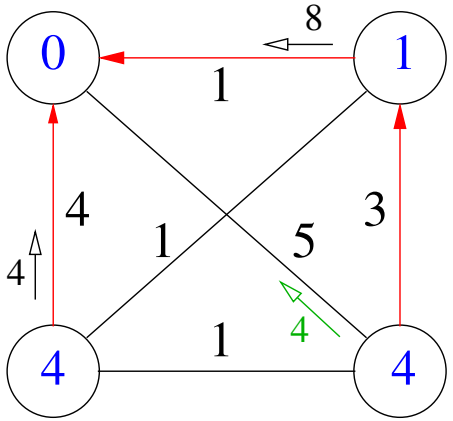
Merlin-Segall algorithm - Example (round 1)



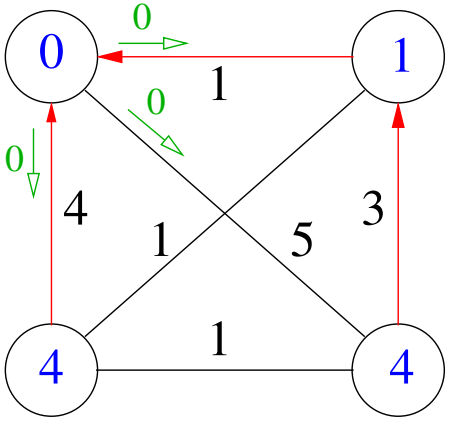


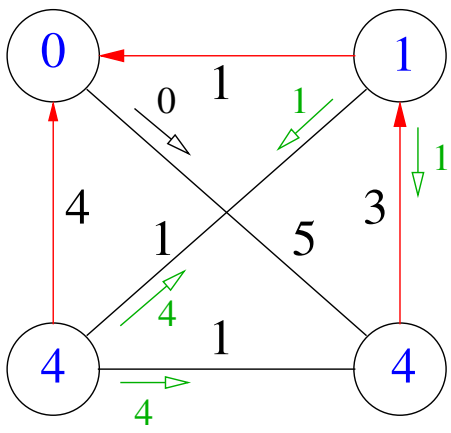


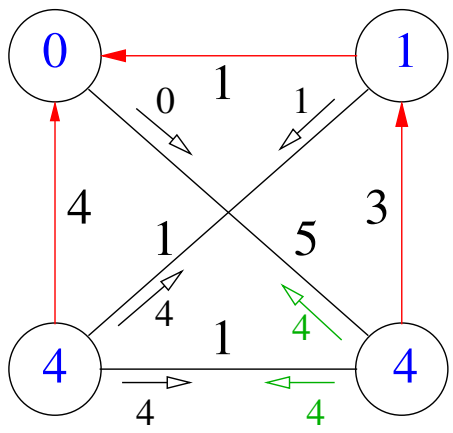


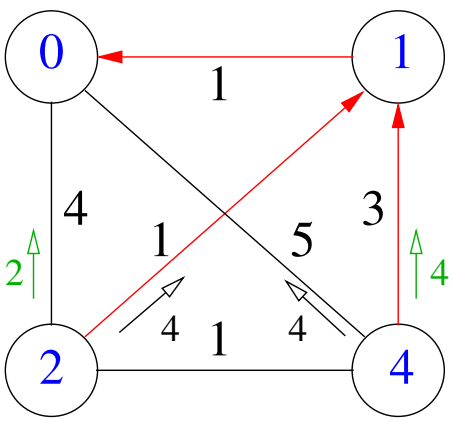


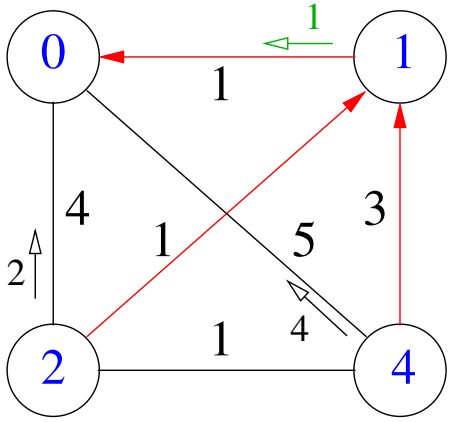
Merlin-Segall algorithm - Example (round 2)



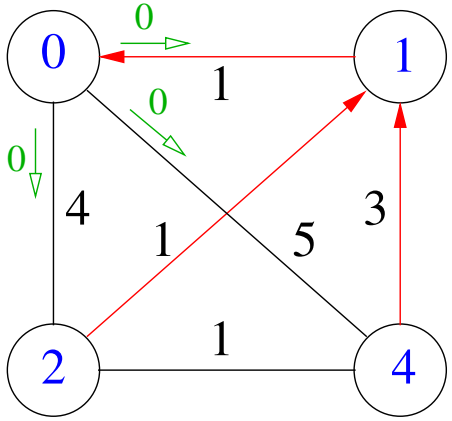


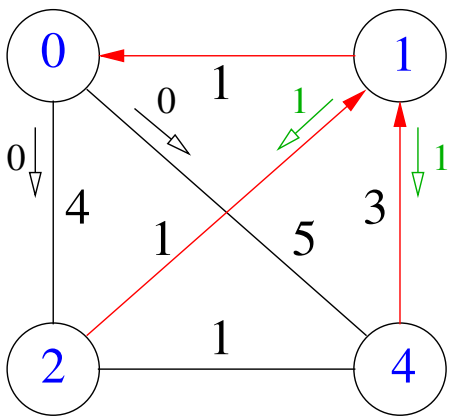


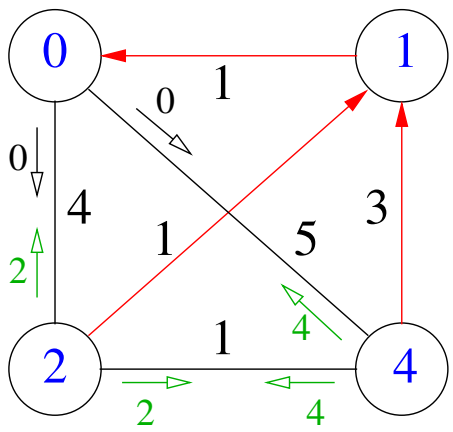


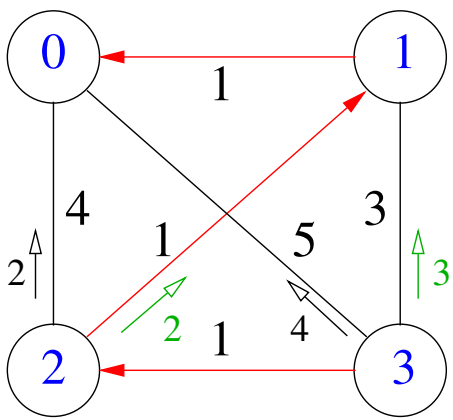


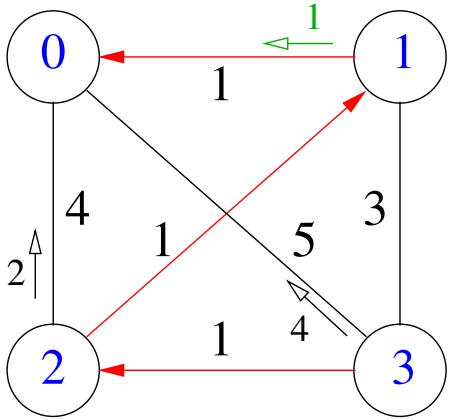
Merlin-Segall algorithm - Example (round 3)

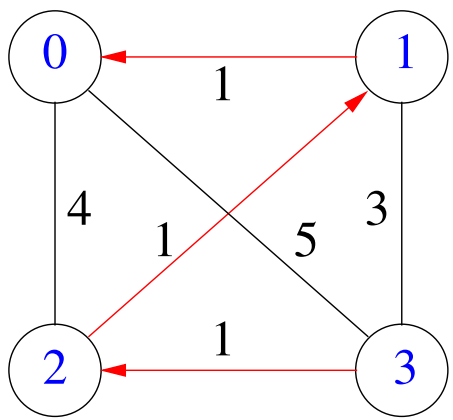












Merlin-Segall algorithm - Topology changes

A number is attached to distance messages.

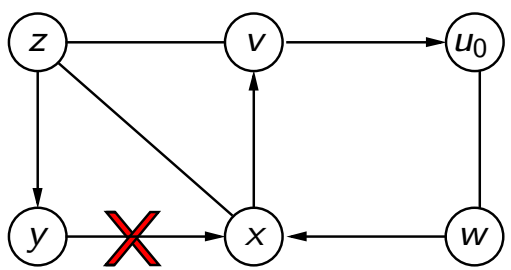
When an edge fails or becomes operational, adjacent nodes send a message to u_0 via the sink tree.

(If the message meets a failed tree link, it is discarded.)

When u_0 receives such a message, it starts a new set of $N - 1$ rounds, *with a higher number*.

If the failed edge is part of the sink tree, the sink tree is rebuilt.

Example:



x informs u_0 (via v) that an edge of the sink tree has failed.

Toueg's algorithm

Computes for each pair u, v a shortest path from u to v.

$d^S(u, v)$, with S a set of nodes, denotes the length of a shortest path from u to v with all *intermediate* nodes in S.

$$
\begin{aligned}
d^S(u,u) &= 0 \\
d^\emptyset(u,v) &= \omega_{uv} \quad \text{if } u \neq v \text{ and } uv \in E \\
d^\emptyset(u,v) &= \infty \quad \text{if } u \neq v \text{ and } uv \notin E \\
d^{S\cup\{w\}}(u,v) &= \min\{\, d^S(u,v),\ d^S(u,w) + d^S(w,v) \,\} \quad \text{if } w \notin S
\end{aligned}
$$

When S contains all nodes, d^S is the standard distance function.

Floyd-Warshall algorithm

We first discuss a *uniprocessor* algorithm.

Initially, $S = \emptyset$; the first three equations define $d^{\emptyset}$.

While S doesn't contain all nodes, a pivot $w \notin S$ is selected:

- $d^{S \cup \{w\}}$ is computed from d^S using the fourth equation.
- w is added to S.

When S contains all nodes, d^S is the standard distance function.

Time complexity: $\Theta(N^3)$

Question

Which complications arise when the Floyd-Warshall algorithm is turned into a distributed algorithm ?

- All nodes must pick the pivots in the same order.
- Each round, nodes need the distance values of the pivot to compute their own routing table.

Toueg's algorithm

Assumption: Each node knows the id's of all nodes.

(Because pivots must be picked uniformly at all nodes.)

Initially, at each node u:

- $S_u = \emptyset$;
- $dist_u(u) = 0$ and $parent_u(u) = \perp$;
- for each $v \neq u$, either
  $dist_u(v) = \omega_{uv}$ and $parent_u(v) = v$ if there is an edge uv, or
  $dist_u(v) = \infty$ and $parent_u(v) = \perp$ otherwise.

Toueg's algorithm

At the w-pivot round, w broadcasts its values $dist_w(v)$, for all nodes v.

If $parent_u(w) = \perp$ for a node $u \neq w$ at the w-pivot round,
then $dist_u(w) = \infty$, so $dist_u(w) + dist_w(v) \geq dist_u(v)$ for all nodes v.

Hence the sink tree toward w can be used to broadcast $dist_w$.

If u is in the sink tree toward w, it sends $\langle \textbf{request}, w \rangle$ to $parent_u(w)$,
to let it pass on $dist_w$.

If u isn't in the sink tree toward w, it proceeds to the next pivot round,
with $S_u \leftarrow S_u \cup \{w\}$.

Toueg's algorithm

Consider any node u in the sink tree toward w.

If $u \neq w$, it waits for the values $dist_w(v)$ from $parent_u(w)$.

u forwards the values $dist_w(v)$ to neighbors that sent $\langle \textbf{request}, w \rangle$ to u.

If $u \neq w$, it checks for each node v whether

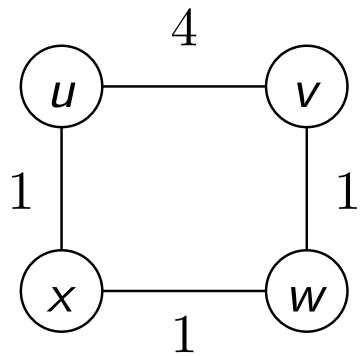
$$dist_u(w) + dist_w(v) < dist_u(v).$$

If so, $dist_u(v) \leftarrow dist_u(w) + dist_w(u)$ and $parent_u(v) \leftarrow parent_u(w)$.

Finally, u proceeds to the next pivot round, with $S_u \leftarrow S_u \cup \{w\}$.

Toueg's algorithm - Example

pivot u	$dist_x(v)$	$\leftarrow$	5	$dist_v(x)$	$\leftarrow$	5
	$parent_x(v)$	$\leftarrow$	u	$parent_v(x)$	$\leftarrow$	u
pivot v	$dist_u(w)$	$\leftarrow$	5	$dist_w(u)$	$\leftarrow$	5
	$parent_u(w)$	$\leftarrow$	v	$parent_w(u)$	$\leftarrow$	v
pivot w	$dist_x(v)$	$\leftarrow$	2	$dist_v(x)$	$\leftarrow$	2
	$parent_x(v)$	$\leftarrow$	w	$parent_v(x)$	$\leftarrow$	w
pivot x	$dist_u(w)$	$\leftarrow$	2	$dist_w(u)$	$\leftarrow$	2
	$parent_u(w)$	$\leftarrow$	x	$parent_w(u)$	$\leftarrow$	x
	$dist_u(v)$	$\leftarrow$	3	$dist_v(u)$	$\leftarrow$	3
	$parent_u(v)$	$\leftarrow$	x	$parent_v(u)$	$\leftarrow$	w



Toueg's algorithm - Complexity + drawbacks

Message complexity: $O(N^2)$

There are N pivot rounds, each taking at most $O(N)$ messages.

Drawbacks:

- Uniform selection of pivots requires that all nodes know the nodes in the network in advance.
- Global broadcast of $dist_w$ at the w-pivot round causes a high bit complexity.
- Not robust with respect to topology changes.

Toueg's algorithm - Optimization

Let $parent_x(w) = u$ with $u \neq w$ at the start of the w-pivot round.

If $dist_u(v)$ doesn't change in this round, then neither does $dist_x(v)$ (for any v).

Upon reception of $dist_w$, u can therefore first update $dist_u$ and $parent_u$, and only forward values $dist_w(v)$ for which $dist_u(v)$ has changed.

Distance-vector routing

The failure of a node or link is eventually detected by its neighbors.

At a change of its links or routing table, nodes send their *routing table* to their neighbors.

Nodes use e.g. the Bellman-Ford algorithm to compute shortest paths.

Link-state routing

Each node periodically sends out a link-state packet, containing

- the node's edges and their weights (based on latency, bandwidth)
- a sequence number (which increases with each broadcast)

Link-state packets are flooded through the network.

Nodes store link-state packets, to obtain a view of the entire network.

Sequence numbers avoid that new info is overwritten by old info.

Each node locally computes shortest paths (e.g. with Dijkstra's alg.).

Question

Flooding entire routing tables (instead of only edges and weights) produces a less efficient algorithm.

Why is that ?

Answer: A routing table may be based on remote edges that have recently crashed.

And, of course, the bit complexity increases dramatically.

Link-state routing - Time-to-live

When a node recovers from a crash, its sequence number starts at 0.

So its link-state packets may be ignored for a long time.

Therefore link-state packets carry a time-to-live (TTL) field.

After this time the information from the packet may be discarded.

To reduce flooding, each time a link-state packet is forwarded, its TTL field decreases.

When it becomes 0, the packet is discarded.

Autonomous systems

The OSPF protocol for routing on the Internet uses *link-state routing*.

The RIP protocol employs *distance-vector routing*.

Link-state / distance-vector routing doesn't scale to the Internet, because it uses flooding / sends entire routing tables.

Therefore the Internet is divided into autonomous systems.

Each autononous system uses the OSPF or RIP protocol.

Border gateway protocol

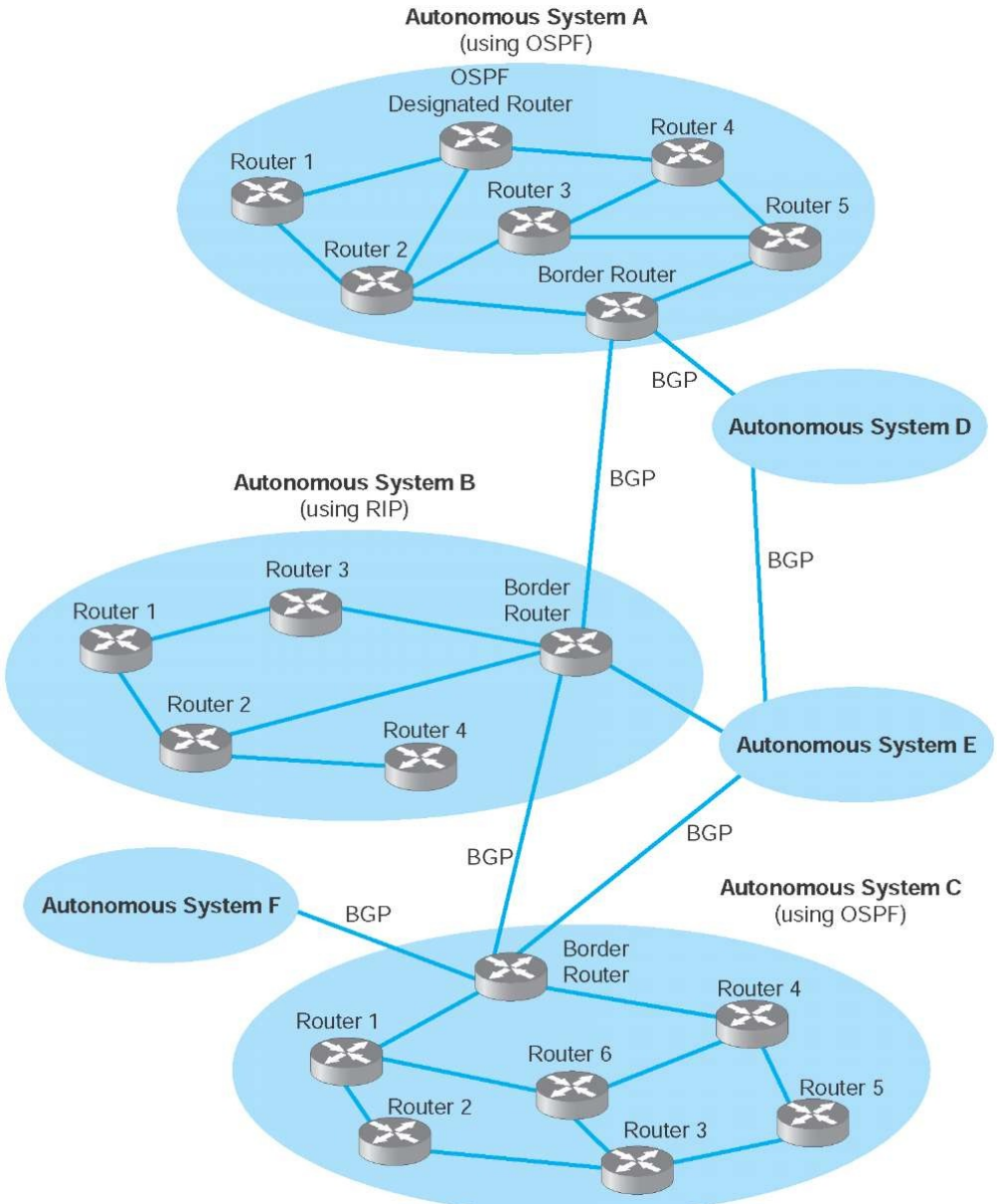
The Border Gateway Protocol routes between autonomous systems.

Routers broadcast updates of their routing tables (a la Chandy-Misra).

A router may update its routing table

- because it detects a topology change, or
- because of an update in the routing table of a neighbor.

Routing on the Internet



Lecture in a nutshell

routing tables to guide a packet to its destination

Chandy-Misra algorithm has *exponential* worst-case message complexity but only $O(N^2 \cdot E)$ for minimum-hop paths

Merlin-Segall algorithm has message complexity $\Theta(N^2 \cdot E)$

Toueg's algorithm has message complexity $O(N^2)$
(but has a high bit complexity, and requires uniform selection of pivots)

link-state / distance-vector routing and the border gateway protocol employ classical routing algorithms on the Internet

Breadth-first search

Consider an undirected, unweighted network.

A breadth-first search tree is a sink tree in which each tree path to the root is minimum-hop.

The Chandy-Misra algorithm for minimum-hop paths computed a breadth-first search tree using $O(N \cdot E)$ messages (for each root).

The following centralized algorithm requires $O(N \cdot \sqrt{E})$ messages (for each root).

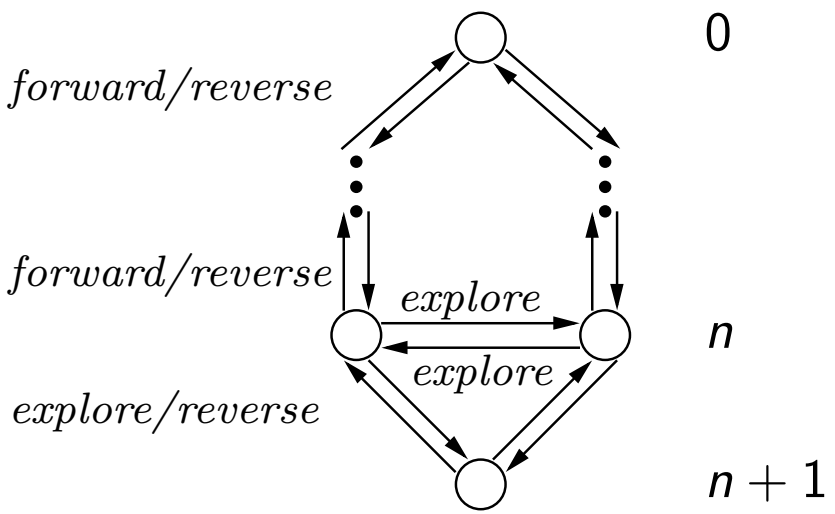
Breadth-first search - A "simple" algorithm

Initially (after round 0), the initiator is at *distance* 0, non-initiators are at *distance* ∞, and *parents* are undefined.

After round $n \geq 0$, the tree has been constructed up to depth n.

Nodes at distance n know which neighbors are at distance $n-1$.

In round $n+1$:



Breadth-first search - A "simple" algorithm

- Messages $\langle$**forward**, $n\rangle$ travel down the tree, from the initiator to nodes at distance n.
- When a node at distance n gets $\langle$**forward**, $n\rangle$, it sends $\langle$**explore**, $n+1\rangle$ to neighbors that aren't at distance $n-1$.

Let a node v receive $\langle$**explore**, $n+1\rangle$.

- If $dist_v = \infty$, then $dist_v \leftarrow n+1$, the sender becomes v's parent, and v sends back $\langle$**reverse**, *true*$\rangle$.
- If $dist_v = n+1$, then v stores that the sender is at distance n, and v sends back $\langle$**reverse**, *false*$\rangle$.
- If $dist_v = n$, then this is a negative ack for the $\langle$**explore**, $n+1\rangle$ that v sends into this edge.

Breadth-first search - A "simple" algorithm

- A *non-initiator* at distance n (or $< n$) waits until all messages $\langle$**explore**, $n+1\rangle$ (resp. $\langle$**forward**, $n\rangle$) have been answered.

  Then it sends $\langle$**reverse**, $b\rangle$ to its parent, where $b = \mathit{true}$ if and only if new nodes were added to its subtree.

- The *initiator* waits until all messages $\langle$**forward**, $n\rangle$ (or, in round 1, $\langle$**explore**, $1\rangle$) have been answered.

  If no new nodes were added in round $n+1$, it terminates.

  Else, it continues with round $n+2$

In round $n+2$, nodes only send a **forward** to children that reported newly discovered nodes in round $n+1$.

Breadth-first search - Complexity

Worst-case message complexity: $O(N^2)$

There are at most N rounds.

Each round, tree edges carry at most 1 **forward** and 1 replying **reverse**.

In total, edges carry 1 **explore** and 1 replying **reverse** or **explore**.

Worst-case time complexity: $O(N^2)$

Round n is completed in at most $2n$ time units, for $n = 1, \ldots, N$.

Frederickson's algorithm

Computes ℓ levels per round, with $1 \leq \ell < N$.

Initially, the initiator is at distance 0, non-initiators are at distance ∞, and parents are undefined.

After round n, the tree has been constructed up to depth ℓn.

In round $n+1$:

- $\langle$**forward**$, \ell n\rangle$ travels down the tree, from the initiator to nodes at distance ℓn.
- When a node at distance ℓn gets $\langle$**forward**$, \ell n\rangle$, it sends $\langle$**explore**$, \ell n + 1\rangle$ to neighbors that aren't at distance $\ell n - 1$.

Frederickson's algorithm - Complications

Complication 1: In round $n+1$, a node at a distance $> \ell n$ may send multiple **explore**'s into an edge.

How can this happen ?

Which (small) complication may arise as a result of this ?

How can this be resolved ?

Solution: **reverse**'s in reply to **explore**'s are supplied with a distance.

Complication 2: A node w may receive a **forward** from a non-parent v.

How can this happen ?

Solution: w can purge this **forward**.

In the previous round, w informed v that it is no longer a child.

Frederickson's algorithm

Let a node v receive $\langle \textbf{explore}, k \rangle$. We distinguish two cases.

- $k < dist_v$:

  $dist_v \leftarrow k$, and the sender becomes v's parent.

  If ℓ doesn't divide k, then v sends $\langle \textbf{explore}, k+1 \rangle$ to its other neighbors.

  If ℓ divides k, then v sends back $\langle \textbf{reverse}, k, \mathit{true} \rangle$.

- $k \geq dist_v$:

  If $k = dist_v$ and ℓ divides k, then v sends back $\langle \textbf{reverse}, k, \mathit{false} \rangle$.

  Else v doesn't send a reply (because it sends $\langle \textbf{explore}, dist_v + 1 \rangle$ into this edge).

Frederickson's algorithm

- A node at distance $\ell n < k < \ell(n+1)$ waits until it has received $\langle \textbf{reverse}, k+1, _ \rangle$ or $\langle \textbf{explore}, j \rangle$ with $j \in \{k, k+1, k+2\}$ from all neighbors.

  Then it sends $\langle \textbf{reverse}, k, \mathit{true} \rangle$ to its parent.

- A *non-initiator* at distance ℓn (or $< \ell n$) waits until all messages $\langle \textbf{explore}, \ell n + 1 \rangle$ (resp. $\langle \textbf{forward}, \ell n \rangle$) have been answered with a **reverse** or **explore** (resp. **reverse**).

  Then it sends $\langle \textbf{reverse}, b \rangle$ to its parent, where $b = \mathit{true}$ if and only if it received $\langle \textbf{reverse}, _, \mathit{true} \rangle$ from a child.

Frederickson's algorithm

- The *initiator* waits until all messages $\langle \textbf{forward}, \ell n \rangle$ (or, in round 1, $\langle \textbf{explore}, 1 \rangle$) have been answered.

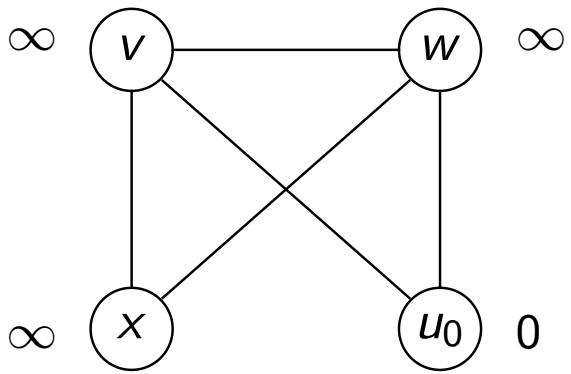
  If it is certain that no unexplored nodes remain, it terminates.

  Else, it continues with round $n + 2$.

In round $n + 2$, nodes only send a **forward** to children that reported newly discovered nodes in round $n + 1$.

Question

Apply Frederickson's algorithm to the network below, with initiator u_0 and $\ell = 2$:



Give a computation in which:

- at some point $dist_w = 2$,
- w becomes the parent of x, and
- in round 2, v sends a spurious **forward** to w.

Frederickson's algorithm - Complexity

Worst-case message complexity: $O(\frac{N^2}{\ell} + \ell \cdot E)$

There are at most $\lceil \frac{N-1}{\ell} \rceil + 1$ rounds.

Each round, tree edges carry at most 1 **forward** and 1 replying **reverse**.

In total, edges carry at most 2ℓ **explore**'s and 2ℓ replying **reverse**'s.

(In total, frond edges carry at most 1 spurious **forward**.)

Worst-case time complexity: $O(\frac{N^2}{\ell})$

Round n is completed in at most $2\ell n$ time units, for $n = 1, \ldots, \lceil \frac{N-1}{\ell} \rceil + 1$.

If $\ell = \lceil \frac{N}{\sqrt{E}} \rceil$, both message and time complexity are $O(N \cdot \sqrt{E})$.

Question

Even with cycle-free routes, we can still get a deadlock. Why ?

Hint: Consider the (bounded) memory at the processes.



Store-and-forward deadlocks

A store-and-forward deadlock occurs when a group of packets are all waiting for the use of a buffer slot occupied by a packet in the group.

A controller avoids such deadlocks.

It prescribes whether a packet can be generated or forwarded, and in which buffer slot it is put next.

Deadlock-free packet switching

Consider an undirected network, supplied with routing tables.

Processes store data packets traveling to their destination in buffers.

Possible events:

- *Generation*: A new packet is placed in an empty buffer slot.
- *Forwarding*: A packet is forwarded to an empty buffer slot of the next node on its route.
- *Consumption*: A packet at its destination node is removed from the buffer.

At a node with an empty buffer, packet generation must be allowed.

Synchronous versus asynchronous networks

For simplicity we assume synchronous communication.

In an asynchronous setting, a node can only eliminate a packet when it is sure that the packet will be accepted at the next node.

Question: How can this be achieved in an undirected network ?

Answer: A packet can only be eliminated by the sender when its reception has been acknowledged.

Destination controller

The network consists of nodes $u_0, \ldots, u_{N-1}$.

T_i denotes the sink tree (with respect to the routing tables) with root u_i for $i = 0, \ldots, N-1$.

In the destination controller, each node carries N buffer slots.

- When a packet with destination u_i is generated at v, it is placed in the ith buffer slot of v.
- If vw is an edge in T_i, then the ith buffer slot of v is linked to the ith buffer slot of w.

Destination controller - Correctness

Theorem: The destination controller is deadlock-free.

Proof: Consider a reachable configuration γ.

Make forwarding and consumption transitions to a configuration δ where no forwarding or consumption is possible.

For each i, since T_i is acyclic, packets in an ith buffer slot can travel to their destination, where they are consumed.

So in δ, all buffers are empty.

Hops-so-far controller

The network consists of nodes $u_0, \ldots, u_{N-1}$.

T_i is the sink tree (with regard to the routing tables) with root u_i for $0 = 1, \ldots, N-1$.

K is the length of a longest path in any T_i.

In the hops-so-far controller, each node carries $K+1$ buffer slots, numbered from 0 to K.

- A generated packet is placed in the 0th buffer slot.
- For each edge vw and any $j < K$, the jth buffer slot of v is linked to the $(j+1)^{\text{th}}$ buffer slot of w, and vice versa.

Hops-so-far controller - Correctness

Theorem: The hops-so-far controller is deadlock-free.

Proof: Consider a reachable configuration γ.

Make forwarding and consumption transitions to a configuration δ where no forwarding or consumption is possible.

Packets in a Kth buffer slot are at their destination.

So in δ, K^{th} buffer slots are all empty.

Suppose all i^{th} buffer slots are empty in δ, for some $1 \leq i \leq K$.

Then all $(i-1)^{\text{th}}$ buffer slots must also be empty in δ.

For else some packet in an $(i-1)^{\text{th}}$ buffer slot could be forwarded or consumed.

Concluding, in δ all buffers are empty.

Acyclic orientation cover

Consider an undirected network.

An acyclic orientation is a directed, acyclic network obtained by directing all edges.

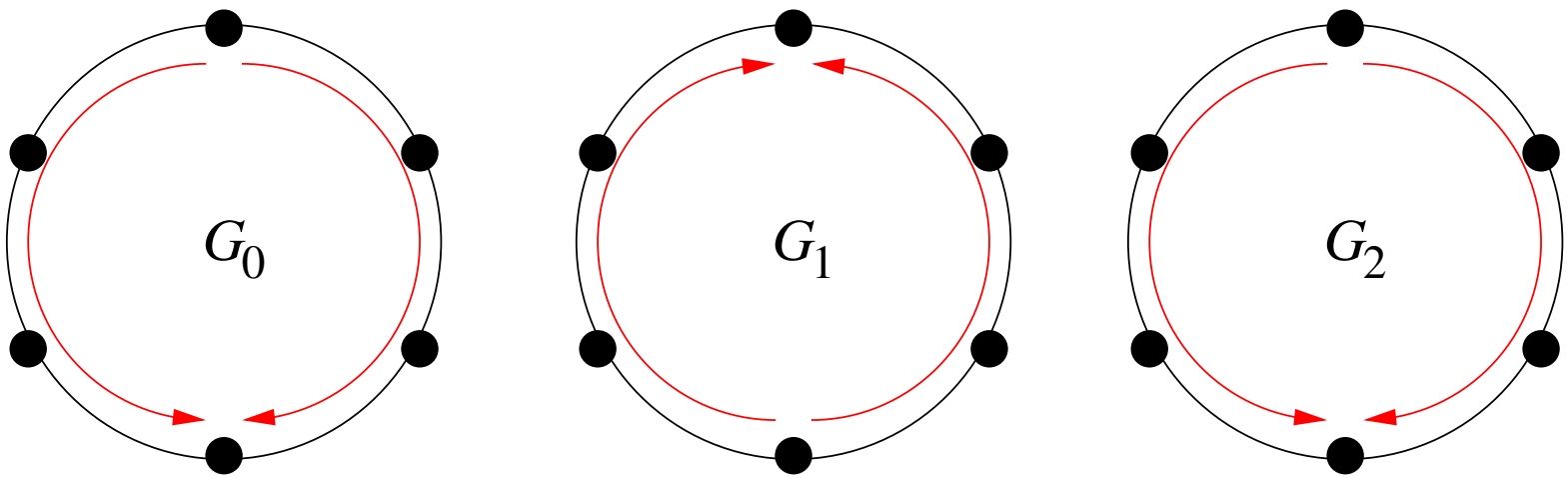
Let $\mathcal{P}$ be a set of paths in the (undirected) network.

An acyclic orientation cover of $\mathcal{P}$ consists of acyclic orientations $G_0, \ldots, G_{n-1}$ such that each path in $\mathcal{P}$ is the concatenation of paths $P_0, \ldots, P_{n-1}$ in $G_0, \ldots, G_{n-1}$.

Acyclic orientation cover - Example

For each undirected ring there exists a cover, consisting of three acyclic orientations, of the collection of minimum-hop paths.

For instance, in case of a ring of size six:



Acyclic orientation cover controller

Let $\mathcal{P}$ be the set of paths in the network induced by the sink trees.

Let $G_0, \ldots, G_{n-1}$ be an acyclic orientation cover of $\mathcal{P}$.

In the acyclic orientation cover controller, nodes have n buffer slots, numbered from 0 to $n-1$.

- A generated packet is placed in the 0th buffer slot.
- Let vw be an edge in G_i.

  The ith buffer slot of v is linked to the ith buffer slot of w.

  Moreover, if $i < n-1$, then the ith buffer slot of w is linked to the $(i+1)^{\text{th}}$ buffer slot of v.

Acyclic orientation cover controller - Intuition

Consider a packet; it is routed via the sink tree of its destination.

Its path is a concatenation of paths $P_0, \ldots, P_{n-1}$ in $G_0, \ldots, G_{n-1}$.

While the packet is in an ith slot with $i < n-1$, it can be forwarded.

If the packet ends up in the $(n-1)^{\text{th}}$ buffer slot at a node, then it is being routed via the last part P_{n-1} of the path.

In that case the packet can be routed to its destination via $(n-1)^{\text{th}}$ buffer slots.

Acyclic orientation cover controller - Example

For each undirected ring there exists a deadlock-free controller that:

- uses three buffer slots per node, and
- allows packets to travel via minimum-hop paths.

Acyclic orientation cover controller - Correctness

Theorem: Let all packets be routed via paths in $\mathcal{P}$.

Then the acyclic orientation cover controller is deadlock-free.

Proof: Consider a reachable configuration γ.

Make forwarding and consumption transitions to a configuration δ where no forwarding or consumption is possible.

Since G_{n-1} is acyclic, packets in an $(n-1)^{\text{th}}$ buffer slot can travel to their destination. So in δ, all $(n-1)^{\text{th}}$ buffer slots are empty.

Suppose all i^{th} buffer slots are empty in δ, for some $i = 1, \ldots, n-1$.

Then all $(i-1)^{\text{th}}$ buffer slots must also be empty in δ.

For else, since G_{i-1} is acyclic, some packet in an $(i-1)^{\text{th}}$ buffer slot could be forwarded or consumed.

Concluding, in δ all buffers are empty.

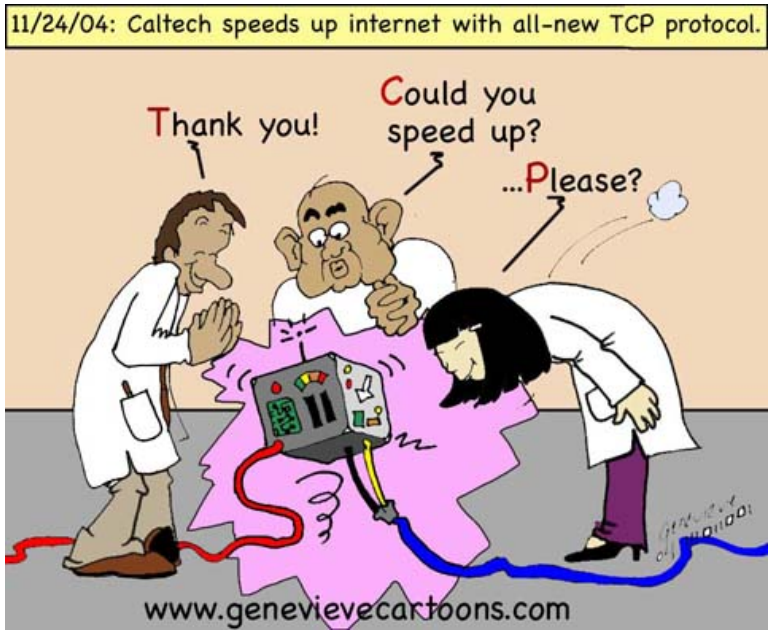
Question

Give an acyclic orientation cover for a ring of size four.

Show how the resulting acyclic orientation cover controller links buffer slots.

Slow-start algorithm in TCP

Back to the asynchronous, pragmatic world of the Internet.



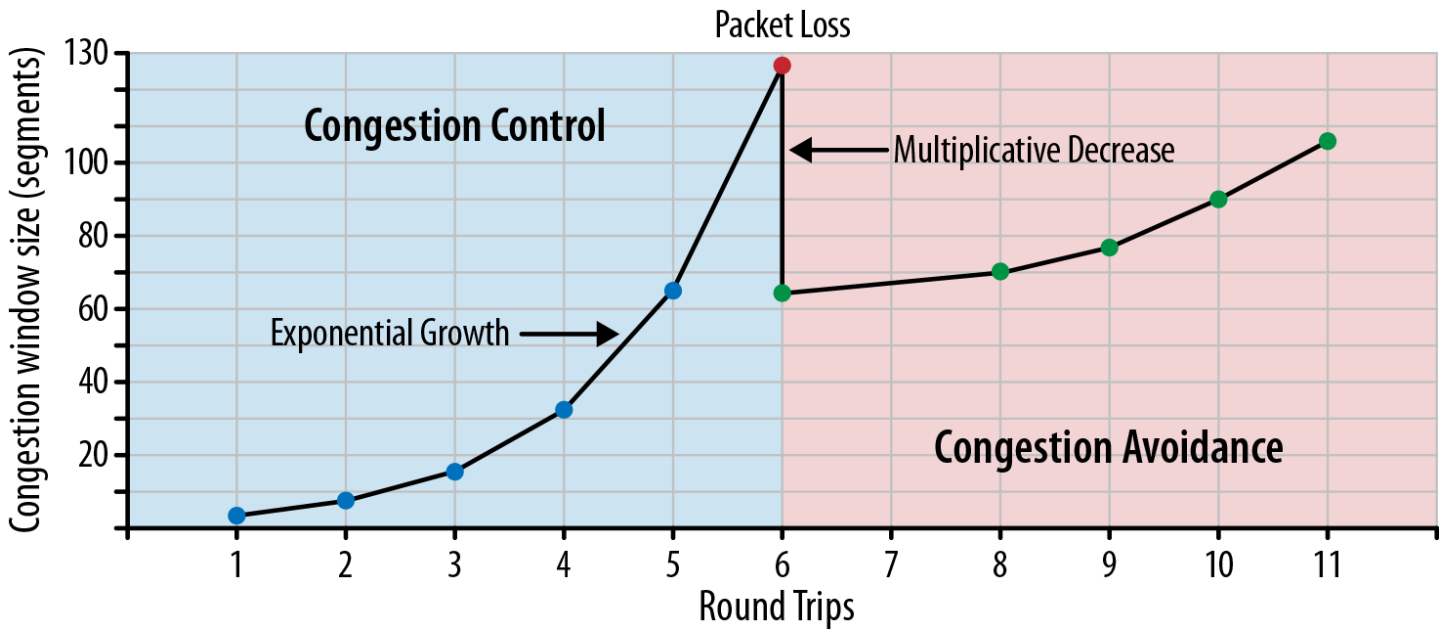
To avoid congestion, in TCP, nodes maintain a congestion window for each of their edges.

It is the maximum number of *unacknowledged* packets on this edge.

Congestion window

The congestion window grows *linearly* with each received ack, up to some threshold.

The congestion window may *double* with every "round trip time".



The congestion window is *reset to the initial size* (in TCP Tahoe) or *halved* (in TCP Reno) with each lost data packet.

Take-home messages of the current lecture

Frederickon's algorithm to compute a breadth-first search tree

iterative deepening (a la Frederickson's alg.)

optimization of a parameter (ℓ) based on a complexity analysis

importance of deadlock-free packet switching

acyclic orientation cover controller

congestion window in TCP

Election algorithms

In an *election algorithm*, each computation should terminate in a configuration where *one* process is the leader.



Assumptions:

- All processes have the *same local algorithm*.
- The algorithm is *decentralized*:

  The initiators can be any non-empty set of processes.
- Process id's are *unique*, and from a *totally ordered set*.

Chang-Roberts algorithm

Consider a directed ring.

Each *initiator* sends a message, tagged with its id.

Let an *active* process p receive a message q:

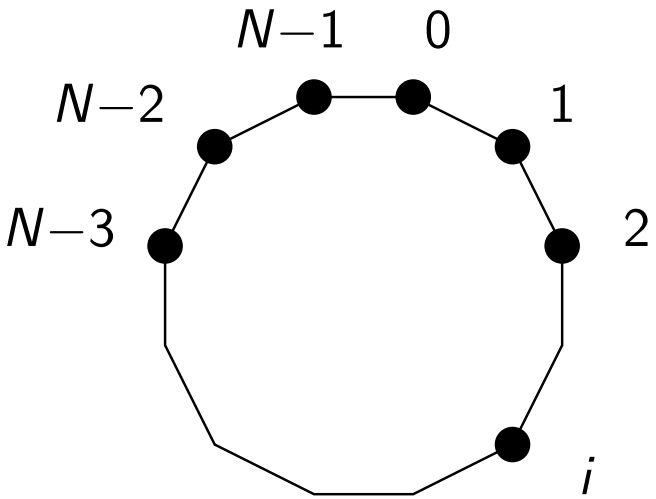
- If $q < p$, then p *purges* the message.
- If $q > p$, then p becomes passive, and *passes on* the message.
- If $q = p$, then p becomes the leader.

Passive processes (including all non-initiators) pass on messages.

Worst-case message complexity: $O(N^2)$

Average-case message complexity: $O(N \log N)$

Chang-Roberts algorithm - Example



anti-clockwise: $\frac{N(N+1)}{2}$ messages

clockwise: $2N-1$ messages

Franklin's algorithm

Consider an undirected ring.

Each *active* process p repeatedly compares its own id with the id's of its nearest *active* neighbors on both sides.

If such a neighbor has a larger id, then p becomes *passive*.

Initially, initiators are active, and non-initiators are passive.

- Each active process sends its id to its neighbors on either side.
- Let active process p receive q and r:
  - if $\max\{q, r\} < p$, then p *sends its id in both directions again*
  - if $\max\{q, r\} > p$, then p becomes passive
  - if $\max\{q, r\} = p$, then p becomes the leader

Passive processes pass on incoming messages.

Franklin's algorithm - Complexity

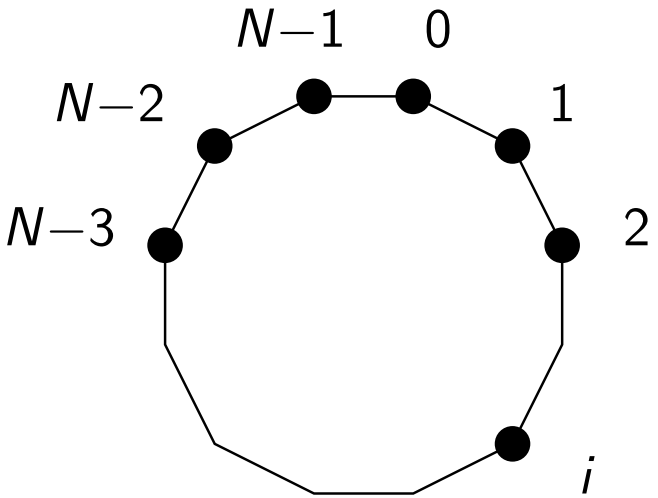
Worst-case message complexity: $O(N \log N)$

In each round, at least half of the active processes become passive.

So there are at most $\lfloor \log_2 N \rfloor + 1$ rounds.

Each round takes $2N$ messages.

Franklin's algorithm - Example



after 1 round only node $N-1$ is active

after 2 rounds node $N-1$ is the leader

Suppose this ring is directed with a clockwise orientation.

If a process would only compare its id with the one of its predecessor, then it would take N rounds to complete.

Dolev-Klawe-Rodeh algorithm

Consider a directed ring.

The comparison of id's of an active process p and
its nearest active neighbors q and r is performed at r.

$$\cdots \blacktriangleright s \cdots \blacktriangleright q \cdots \blacktriangleright p \cdots \blacktriangleright r \cdots \blacktriangleright t \cdots \blacktriangleright$$

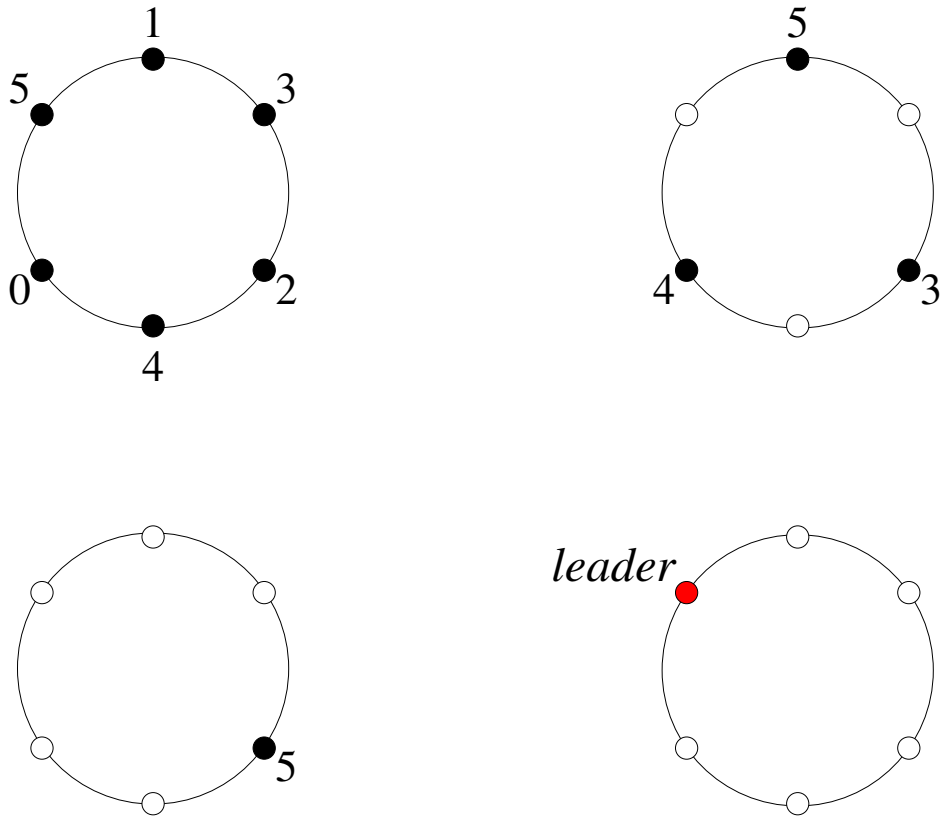
- If $\max\{q, r\} < p$, then r changes its id to p, and sends out p.
- If $\max\{q, r\} > p$, then r becomes passive.
- If $\max\{q, r\} = p$, then r announces this id to all processes.

  The process that originally had the id p becomes the leader.

Worst-case message complexity: $O(N \log N)$

Dolev-Klawe-Rodeh algorithm - Example

Consider the following clockwise oriented ring.



Tree election algorithm for acyclic networks

Question: How can the *tree algorithm* be used to make the process with the largest id in an undirected, acyclic network the leader ?

(Be careful that a leaf may be a non-initiator.)

Start with a wake-up phase, driven by the initiators.

- Initially, initiators send a wake-up message to all neighbors.
- When a non-initiator receives a first wake-up message, it sends a wake-up message to all neighbors.
- A process wakes up when it has received wake-up messages from all neighbors.

Tree election algorithm

The local algorithm at an awake process p:

- p waits until it has received id's from all neighbors except one, which becomes its parent.
- p computes the largest id $\max_p$ among the received id's and its own id.
- p sends a *parent request* to its parent, tagged with $\max_p$.
- If p receives a parent request from its parent, tagged with q, it computes $\max'_p$, being the maximum of $\max_p$ and q.
- Next p sends an *information message* to all neighbors except its parent, tagged with $\max'_p$.
- This information message is forwarded through the network.
- The process with id $\max'_p$ becomes the leader.

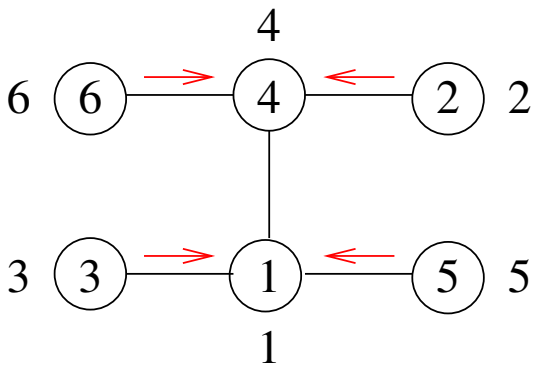
Message complexity: $2N - 2$ messages

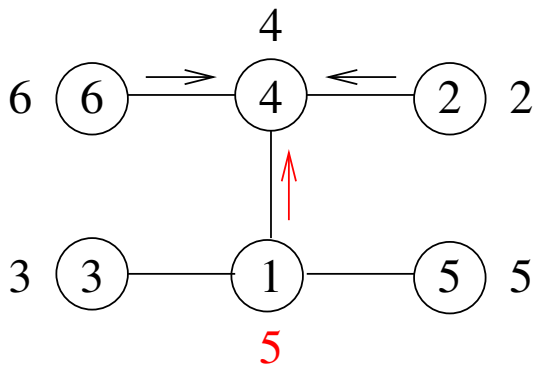
Question

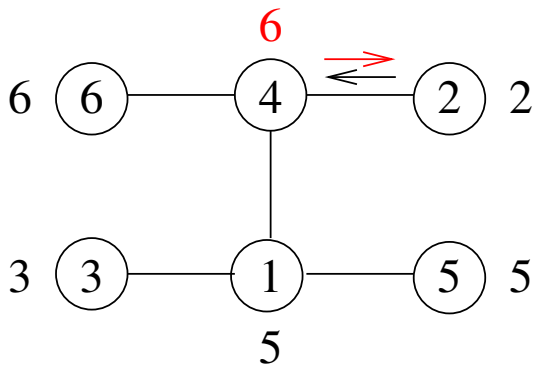
In case a process p receives a parent request from its parent, why does it need to recompute $\max_p$?

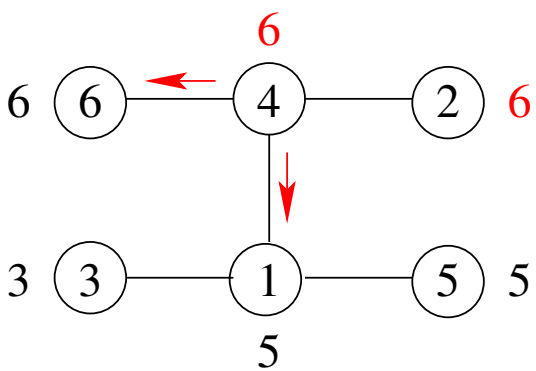
Tree election algorithm - Example

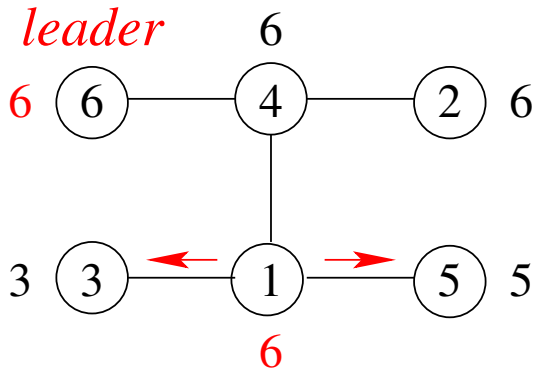
The wake-up phase is omitted.

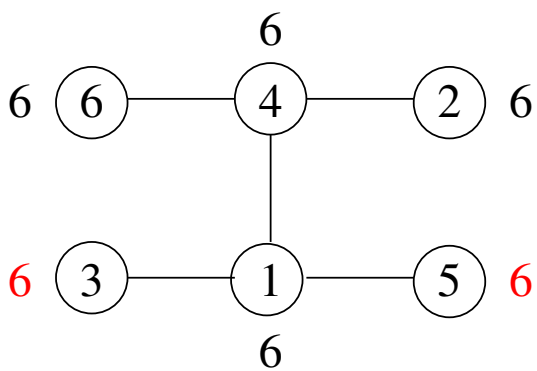












Echo algorithm with extinction

Each *initiator* starts a wave, tagged with its id.

Non-initiators join the first wave that hits them.

At any time, each process takes part in at most one wave.

Suppose a process p in wave q is hit by a wave r:

- if $q < r$, then p changes to wave r
  (it abandons all earlier messages);
- if $q > r$, then p continues with wave q
  (it purges the incoming message);
- if $q = r$, then the incoming message is treated according to the echo algorithm of wave q.

If wave p executes a decide event (at p), p becomes leader.

Worst-case message complexity: $O(N \cdot E)$

Minimum spanning trees

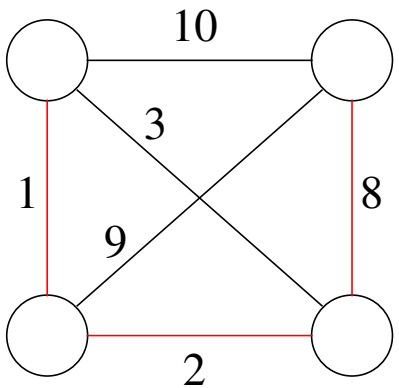
Consider an undirected, weighted network.

We assume that different edges have different weights.

(Or weighted edges can be totally ordered by also taking into account the id's of endpoints of an edge, and using a lexicographical order.)

In a minimum spanning tree, the sum of the weights of the edges in the spanning tree is minimal.

Example:



Fragments

Lemma: Let F be a fragment
(i.e., a connected subgraph of the minimum spanning tree M).

Let e be the *lowest-weight* outgoing edge of F
(i.e., e has exactly one endpoint in F).

Then e is in M.

Proof: Suppose not.

Then $M \cup \{e\}$ has a cycle,
containing e and another outgoing edge f of F.

Replacing f by e in M gives a spanning tree
with a smaller sum of weights of edges.

Kruskal's algorithm

A *uniprocessor* algorithm for computing minimum spanning trees.

- Initially, each process forms a separate fragment.
- In each step, a lowest-weight outgoing edge of a fragment is added to the spanning tree, joining two fragments.

This algorithm also works when edges have the same weight.

Then the minimum spanning tree may not be unique.

Gallager-Humblet-Spira algorithm

Consider an undirected, weighted network,
in which different edges have different weights.

Distributed computation of a minimum spanning tree:

- Initially, each process is a fragment.
- The processes in a fragment F together search for the lowest-weight outgoing edge e_F.
- When e_F is found, the fragment at the other end is asked to collaborate in a merge.

Complications: Is an edge outgoing? Is it lowest-weight?

Level, name and core edge

Each fragment carries a (unique) name $fn : \mathbb{R}$ and a level $\ell : \mathbb{N}$.

Its level is the maximum number of joins any process in the fragment has experienced.

Neighboring fragments $F = (fn, \ell)$ and $F' = (fn', \ell')$ can be joined as follows:

$$\ell < \ell' \ \wedge \ F \xrightarrow{e_F} F' : \quad F \cup F' = (fn', \ell')$$

$$\ell = \ell' \ \wedge \ e_F = e_{F'} : \quad F \cup F' = (\text{weight } e_F, \ell + 1)$$

The core edge of a fragment is the last edge that connected two sub-fragments at the same level. Its end points are the core nodes.

Parameters of a process

Its *state*:

- sleep (for non-initiators)
- find (looking for a lowest-weight outgoing edge)
- found (reported a lowest-weight outgoing edge to the core edge)

The *status* of its *channels*:

- basic edge (undecided)
- branch edge (in the spanning tree)
- rejected (not in the spanning tree)

Name and *level* of its fragment.

Its *parent* (toward the core edge).

Initialization

Non-initiators wake up when they receive a (**connect** or **test**) message.

Each initiator, and non-initiator after it has woken up:

- sets its level to 0
- sets its *lowest-weight* edge to branch
- sends $\langle \textbf{connect}, 0 \rangle$ into this channel
- sets its other channels to basic
- sets its state to found

Joining two fragments

Let fragments $F = (fn, \ell)$ and $F' = (fn', \ell')$ be joined via channel pq.

- If $\ell < \ell'$, then p sent $\langle \textbf{connect}, \ell \rangle$ to q.

  q sends $\langle \textbf{initiate}, fn', \ell', \frac{find}{found} \rangle$ to p.

  $F \cup F'$ inherits the core edge of F'.

- If $\ell = \ell'$, then p and q sent $\langle \textbf{connect}, \ell \rangle$ to each other.

  They send $\langle \textbf{initiate}, \text{weight } pq, \ell + 1, find \rangle$ to each other.

  $F \cup F'$ has core edge pq.

At reception of $\langle \textbf{initiate}, fn, \ell, \frac{find}{found} \rangle$, a process stores fn and ℓ, sets its state to *find* or *found*, and adopts the sender as its parent.

It passes on the message through its other branch edges.

Computing the lowest-weight outgoing edge

In case of $\langle$**initiate**, *fn*, ℓ, *find*$\rangle$, p checks in increasing order of weight if one of its *basic* edges pq is *outgoing*, by sending $\langle$**test**, *fn*, $\ell\rangle$ to q.

While $\ell > \mathit{level}_q$, q *postpones* processing the incoming **test** message.

Let $\ell \leq \mathit{level}_q$.

- If q is in fragment *fn*, then q replies **reject**.

  In this case p and q will set pq to rejected.

- Else, q replies **accept**.

When a basic edge is accepted, or there are no basic edges left, p *stops* the search.

Questions

Why does q only process the incoming $\langle \textbf{test}, _, \ell \rangle$ message from p when $\ell \leq \mathit{level}_q$?

Answer: p and q might be in the same fragment, in which case $\langle \textbf{initiate}, \mathit{fn}, \ell, \mathit{find} \rangle$ is on its way to q.

Why does this postponement not lead to a deadlock ?

Answer: There is always a fragment with a smallest level.

Reporting to the core nodes

- p waits for all branch edges, *except its parent*, to report.
- p sets its state to *found*.
- p computes the minimum λ of (1) these reports, and (2) the weight of its lowest-weight outgoing basic edge (or ∞, if no such channel was found).
- If $\lambda < \infty$, p stores either the branch edge that sent λ, or its basic edge of weight λ.
- p sends $\langle \textbf{report}, \lambda \rangle$ to its parent.

Termination or **changeroot** at the core nodes

A core node receives reports through *all* its branch edges, including the core edge.

- If the minimum reported value $\mu = \infty$, the core nodes terminate.
- If $\mu < \infty$, the core node that received μ first sends **changeroot** toward the lowest-weight outgoing basic edge.

  (The core edge becomes a regular (tree) edge.)

Ultimately **changeroot** reaches the process p that reported the lowest-weight outgoing basic edge.

p sets this channel to branch, and sends $\langle \textbf{connect}, \mathit{level}_p \rangle$ into it.

Starting the join of two fragments

When q receives $\langle \textbf{connect}, level_p \rangle$ from p, $level_q \geq level_p$.

Namely, either $level_p = 0$, or q earlier sent **accept** to p.

- If $level_q > level_p$, then q sets qp to *branch* and sends $\langle \textbf{initiate}, name_q, level_q, \frac{find}{found} \rangle$ to p.
- As long as $level_q = level_p$ and qp isn't a branch edge, q *postpones* processing the **connect** message.
- If $level_q = level_p$ and qp is a branch edge (meaning that q sent $\langle \textbf{connect}, level_q \rangle$ to p), then q sends $\langle \textbf{initiate}, \text{weight } qp, level_q + 1, find \rangle$ to p (and vice versa).

  In this case pq becomes the core edge.

Questions

If $level_q = level_p$ and qp isn't a branch edge, why does q postpone processing the incoming **connect** message from p?

Answer: The fragment of q might be in the process of joining another fragment at level $\geq level_q$, in which case the fragment of p should subsume the name and level of that joint fragment, instead of joining the fragment of q at an equal level.

Why does this postponement not give rise to a deadlock?

(I.e., why can't there be a cycle of fragments waiting for a reply to a postponed **connect** message?)

Answer: Because different channels have different weights.

Question

Suppose a process reported a lowest-weight outgoing basic edge, and in return receives $\langle$**initiate**, *fn*, ℓ, *find*$\rangle$.

Why must it test again whether this basic edge is outgoing ?

Answer: Its fragment may in the meantime have joined the fragment at the other side of this basic edge.

Gallager-Humblet-Spira algorithm - Complexity

Worst-case message complexity: $O(E + N \log N)$

- A rejected channel requires a **test-reject** or **test-test** pair.

Between two subsequent joins, a process:

- receives one **initiate**
- sends at most one **test** that triggers an **accept**
- sends one **report**
- sends at most one **changeroot** or **connect**

A fragment at level ℓ contains $\geq 2^{\ell}$ processes.

So each process experiences at most $\lfloor \log_2 N \rfloor$ joins.

Back to election

By two extra messages at the very end,
the core node with the largest id becomes the leader.

So Gallager-Humblet-Spira induces an election algorithm for general undirected networks.

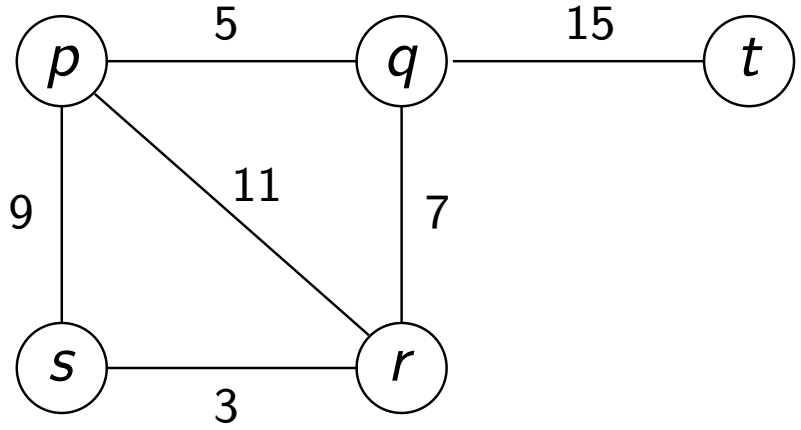
(We must impose an order on channels of equal weight.)

Lower bounds for the average-case message complexity
of election algorithms based on comparison of id's:

Rings: $\Omega(N \log N)$

General networks: $\Omega(E + N \log N)$

Gallager-Humblet-Spira algorithm - Example



pq qp	$\langle$**connect**, 0$\rangle$
pq qp	$\langle$**initiate**, 5, 1, *find*$\rangle$
ps qr	$\langle$**test**, 5, 1$\rangle$
tq	$\langle$**connect**, 0$\rangle$
qt	$\langle$**initiate**, 5, 1, *find*$\rangle$
tq	$\langle$**report**, $\infty\rangle$
rs sr	$\langle$**connect**, 0$\rangle$
rs sr	$\langle$**initiate**, 3, 1, *find*$\rangle$
sp rq	**accept**
pq	$\langle$**report**, 9$\rangle$
qp	$\langle$**report**, 7$\rangle$
qr	$\langle$**connect**, 1$\rangle$
sp rq	$\langle$**test**, 3, 1$\rangle$
ps qr	**accept**
rs	$\langle$**report**, 7$\rangle$
sr	$\langle$**report**, 9$\rangle$
rq	$\langle$**connect**, 1$\rangle$
rq qp qr qt rs	$\langle$**initiate**, 7, 2, *find*$\rangle$
ps sp	$\langle$**test**, 7, 2$\rangle$
pr	$\langle$**test**, 7, 2$\rangle$
rp	**reject**
pq tq qr sr rq	$\langle$**report**, $\infty\rangle$

Lecture in a nutshell

leader election

decentralized, uniform local algorithm, unique process id's

Chang-Roberts and Dolev-Klawe-Rodeh algorithm on directed rings

tree election algorithm

echo algorithm with extinction

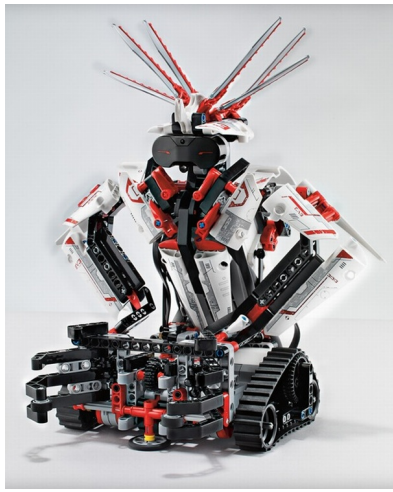
Gallager-Humblet-Spira minimum spanning tree algorithm

Election in anonymous networks

In an anonymous network, processes (and channels) have no unique id.

Processes may be anonymous for several reasons:

- Transmitting/storing id's is too expensive (IEEE 1394 bus).
- Processes don't want to reveal their id (security protocols).
- Absence of unique hardware id's (LEGO Mindstorms).



Question: Suppose there is one *leader*.

How can each process be provided with a unique id ?

Impossibility of election in anonymous rings

Theorem: There is no election algorithm for anonymous rings that always terminates.

Proof: Consider an anonymous ring of size N.

In a symmetric configuration, all processes are in the same state and all channels carry the same messages.

- There is a symmetric initial configuration.
- If γ_0 is symmetric and $\gamma_0 \longrightarrow \gamma_1$, then there are transitions $\gamma_1 \longrightarrow \gamma_2 \longrightarrow \cdots \longrightarrow \gamma_N$ with γ_N symmetric.

In a symmetric configuration there isn't one leader.

So there is an infinite computation in which no leader is elected.

Fairness

An execution is fair if each event that is applicable in infinitely many configurations, occurs infinitely often in the computation.



Each election algorithm for anonymous rings has a *fair* infinite execution.

(Basically because in the proof, $\gamma_0 \longrightarrow \gamma_1$ can be chosen freely.)

Probabilistic algorithms

In a probabilistic algorithm, a process may flip a coin, and perform an event based on the outcome of this coin flip.



Probabilistic algorithms where *all* computations terminate in a *correct* configuration aren't interesting.

Because letting the coin e.g. always flip heads yields a correct non-probabilistic algorithm.

Las Vegas and Monte Carlo algorithms

A probabilistic algorithm is Las Vegas if:

- the probability that it terminates is greater than zero, and
- all terminal configurations are correct.





It is Monte Carlo if:

- it always terminates, and
- the probability that a terminal configuration is correct is greater than zero.

Questions

Even if the probability that a Las Vegas algorithm terminates is 1, this doesn't always imply termination. Why is that ?

Assume a Monte Carlo algorithm, and a (deterministic) algorithm to check whether a run of the Monte Carlo algorithm terminated correctly.

Give a Las Vegas algorithm that terminates with probability 1.

Itai-Rodeh election algorithm

Given an anonymous, directed ring; *all processes know the ring size N*.

We adapt the Chang-Roberts algorithm: each initiator sends out an id, and the largest id is the only one making a round trip.

Each initiator selects a random id from $\{1, \ldots, N\}$.

Complication: Different processes may select the same id.

Solution: Each message is supplied with a hop count.

A message that arrives at its source has hop count N.

If several processes select the same largest id, then they start a new election round, *with a higher round number*.

Itai-Rodeh election algorithm

Initially, *initiators* are active in round 0, and *non-initiators* are passive.

Let p be *active*. At the start of election round n, p randomly selects id_p, sends $(n, id_p, 1, false)$, and waits for a message (n', i, h, b).

The 3rd value is the hop count. The 4th value signals if another process *with the same id* was encountered during the round trip.

- p gets (n', i, h, b) with $n' > n$, or $n' = n$ and $i > id_p$: it becomes *passive* and sends $(n', i, h + 1, b)$.
- p gets (n', i, h, b) with $n' < n$, or $n' = n$ and $i < id_p$: it *purges* the message.
- p gets (n, id_p, h, b) with $h < N$: it sends $(n, id_p, h + 1, true)$.
- p gets $(n, id_p, N, true)$: it proceeds to round $n + 1$.
- p gets $(n, id_p, N, false)$: it becomes the leader.

Passive processes pass on messages, increasing their hop count by one.

Itai-Rodeh election algorithm - Correctness

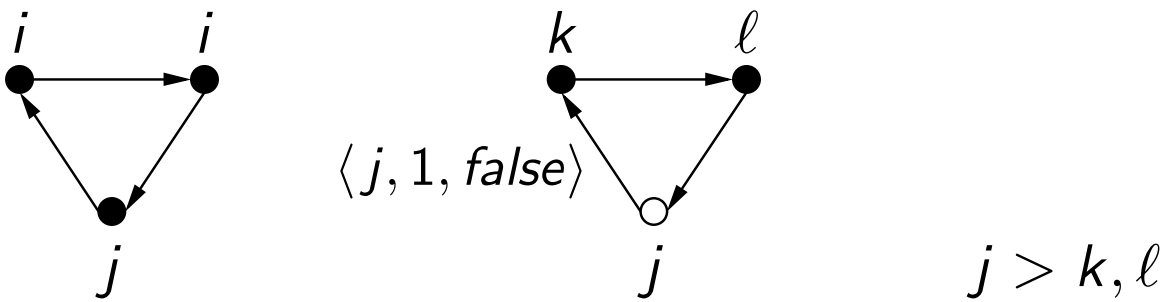
Question: How can an infinite computation occur ?

Correctness: The Itai-Rodeh election algorithm is Las Vegas.

Eventually one leader is elected, with probability 1.

Without rounds, the algorithm would be flawed.

Example:



Average-case message complexity: $O(N \log N)$

Election in arbitrary anonymous networks

The echo algorithm with extinction, with random selection of id's, can be used for election in anonymous undirected networks in which *all processes know the network size*.

Initially, initiators are active in round 0, and non-initiators are passive.

Each active process selects a random id, and starts a wave, tagged with its id and round number 0.

Let process p in wave i of round n be hit by wave j of round n':

- if $n' > n$, or $n' = n$ and $j > i$, then p adopts wave j of round n', and treats the message according to the echo algorithm
- if $n' < n$, or $n' = n$ and $j < i$, then p purges the message
- if $n' = n$ and $j = i$, then p treats the message according to the echo algorithm

Election in arbitrary anonymous networks

Each message sent upwards in the constructed tree reports the size of its subtree.

All other messages report 0.

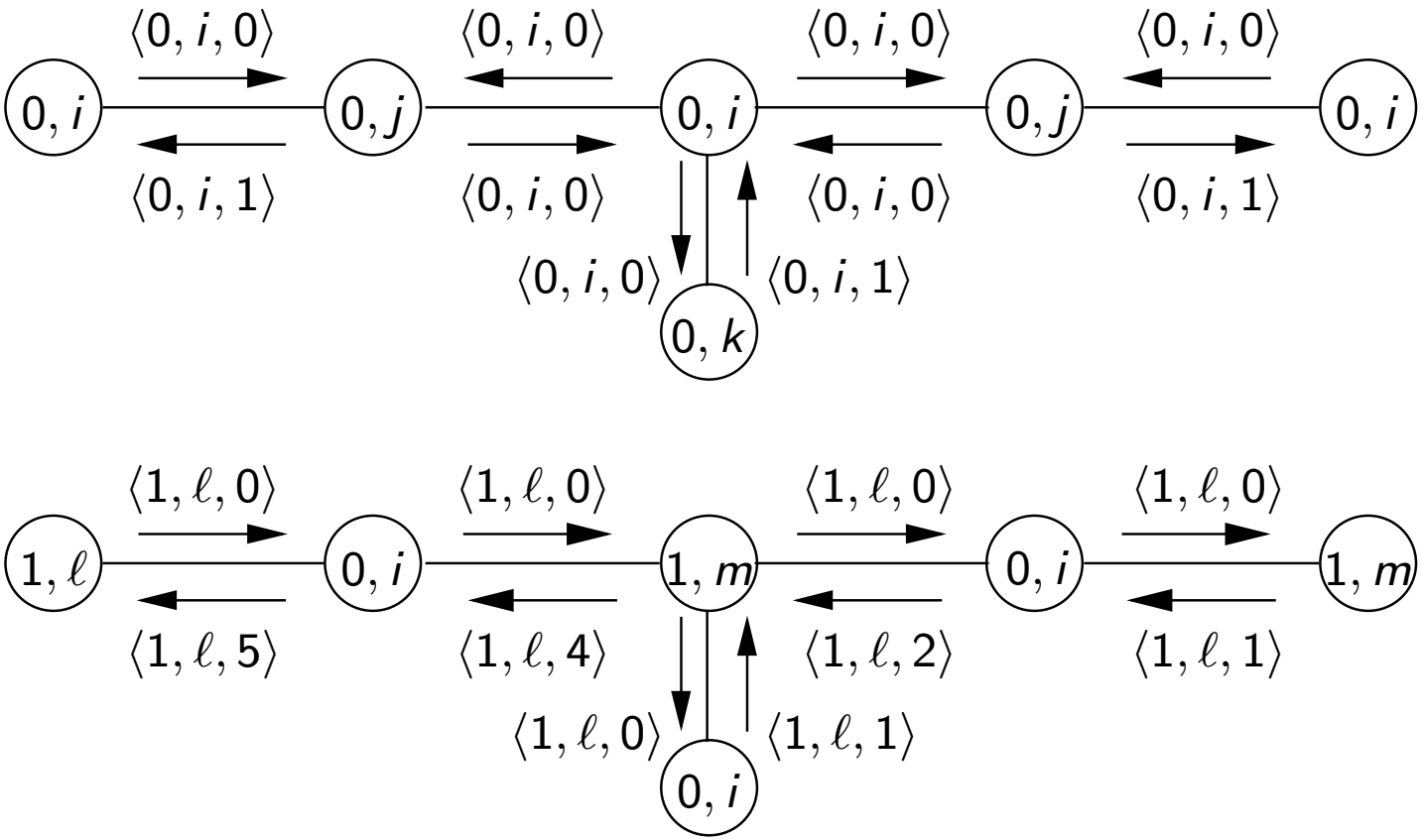
When a process *decides*, it computes the size of the constructed tree.

If the constructed tree covers the network, it becomes the leader.

Else, it selects a new id, and initiates a new wave, in the next round.

Election in arbitrary anonymous networks - Example

$i > j > k > \ell > m$. Only waves that complete are shown.



The process at the left computes size 6, and becomes the leader.

Question

Is there another scenario in which the right-hand side node progresses to round 2 ?

Computing the size of a network

Theorem: There is no Las Vegas algorithm to compute the size of an anonymous ring.

This implies that there is no Las Vegas algorithm for election in an anonymous ring *if processes don't know the ring size*.

Because when a leader is known, the network size can be computed using a wave algorithm.

Impossibility of computing anonymous network size

Theorem: There is no Las Vegas algorithm to compute the size of an anonymous ring.

Proof: Consider an anonymous, directed ring $p_0, \ldots, p_{N-1}$.

Suppose a computation C of a (probabilistic) ring size algorithm terminates with the correct outcome N.

Consider the ring $p_0, \ldots, p_{2N-1}$.

Let each event at a p_i in C be executed concurrently at p_i and p_{i+N}.

This computation terminates with the incorrect outcome N.

Itai-Rodeh ring size algorithm

Each process p maintains an *estimate* est_p of the ring size.

Initially $est_p = 2$. (Always $est_p \leq N$.)

p initiates an estimate round (1) at the start of the algorithm, and (2) at each update of est_p.

Each round, p selects a random id_p in $\{1, \ldots, R\}$, sends $(est_p, id_p, 1)$, and waits for a message (est, id, h). (Always $h \leq est$.)

- $est < est_p$. Then p *purges* the message.
- $est > est_p$.
  - If $h < est$, then p sends $(est, id, h+1)$, and $est_p \leftarrow est$.
  - If $h = est$, then $est_p \leftarrow est + 1$.
- $est = est_p$.
  - If $h < est$, then p sends $(est, id, h+1)$.
  - If $h = est$ and $id \neq id_p$, then $est_p \leftarrow est + 1$.
  - If $h = est$ and $id = id_p$, then p *purges* the message (possibly its own message returned).

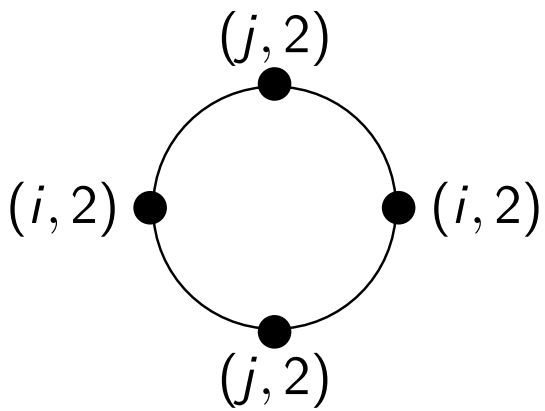
Itai-Rodeh ring size algorithm - Correctness

When the algorithm terminates, $est_p \leq N$ for all p.

The Itai-Rodeh ring size algorithm is a Monte Carlo algorithm.

Possibly, in the end $est_p < N$.

Example:

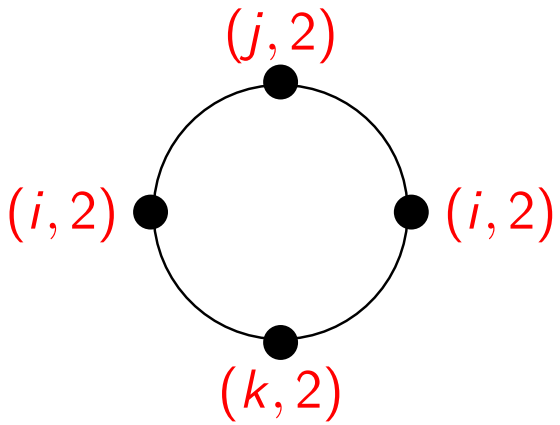


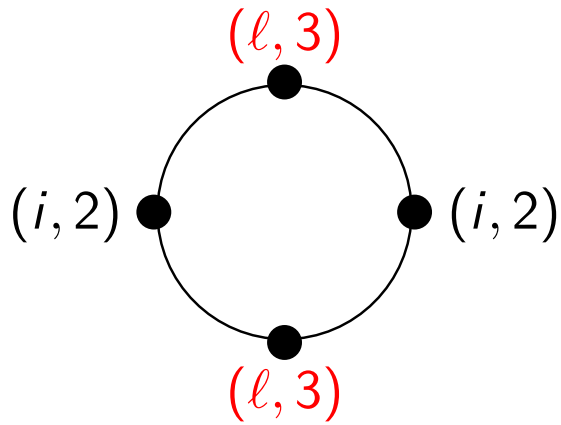
Itai-Rodeh ring size algorithm - Termination

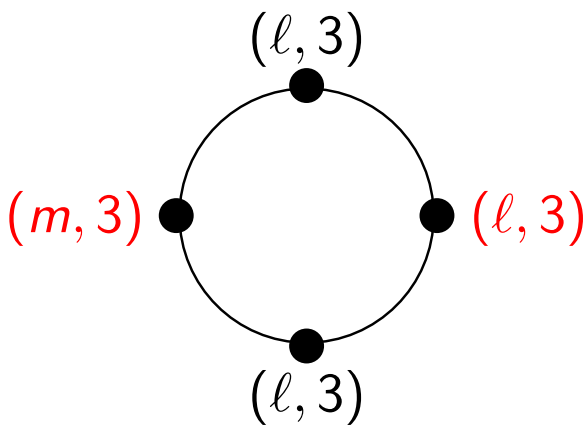
Question: Upon message-termination, is est_p always the same at all p ?

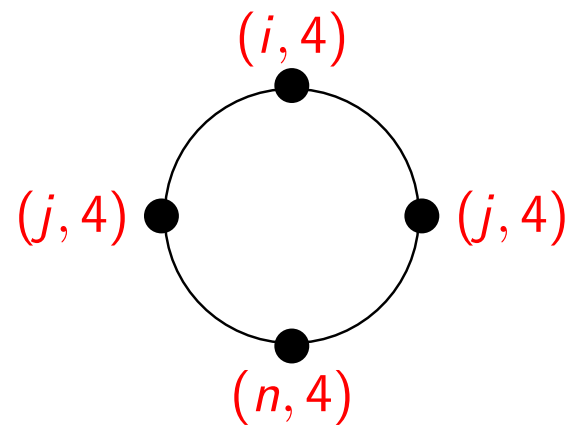
There is no Las Vegas algorithm for general termination detection in anonymous rings.

Itai-Rodeh ring size algorithm - Example









Itai-Rodeh ring size algorithm - Complexity

The probability of computing an incorrect ring size tends to zero when R tends to infinity.

Worst-case message complexity: $O(N^3)$

The N processes start at most $N-1$ estimate rounds.

Each round they send a message, which takes at most N steps.

Question

Give an (always correctly terminating) algorithm for computing the network size of anonymous, acyclic networks.

Answer: Use the tree algorithm, whereby each process reports the size of its subtree to its parent.

IEEE 1394 election algorithm

The IEEE 1394 standard is a serial multimedia bus.

It connects digital devices,
which can be added/removed dynamically.



Transmitting/storing id's is too expensive,
so the network is anonymous.

The network size is unknown to the processes.

The tree algorithm for undirected, acyclic networks is used.

Networks that contain a cycle give a time-out.

IEEE 1394 election algorithm

When a process has one possible parent, it sends a parent request to this neighbor. If the request is accepted, an ack is sent back.

The last two parentless processes can send parent requests to each other simultaneously. This is called root contention.

Each of the two processes in root contention randomly decides to either *immediately send* a parent request again, or to *wait some time* for a parent request from the other process.

Question: Is it optimal for performance to give probability 0.5 to both sending immediately and waiting for some time?

Lecture in a nutshell

anonymous network

impossibility of election in anonymous networks

Las Vegas / Monte Carlo algorithms

Itai-Rodeh election algorithm for directed rings (Las Vegas)

echo election algorithm for anonymous networks (Las Vegas)

no Las Vegas algorithm for computing anonymous network size

Itai-Rodeh ring size algorithm (Monte Carlo)

IEEE 1394 election algorithm

Fault tolerance

A process may (1) *crash*, i.e., execute no further events, or even (2) be *Byzantine*, meaning that it can perform arbitrary events.

Assumption: The network is *complete*, i.e., there is an undirected channel between each pair of different processes.

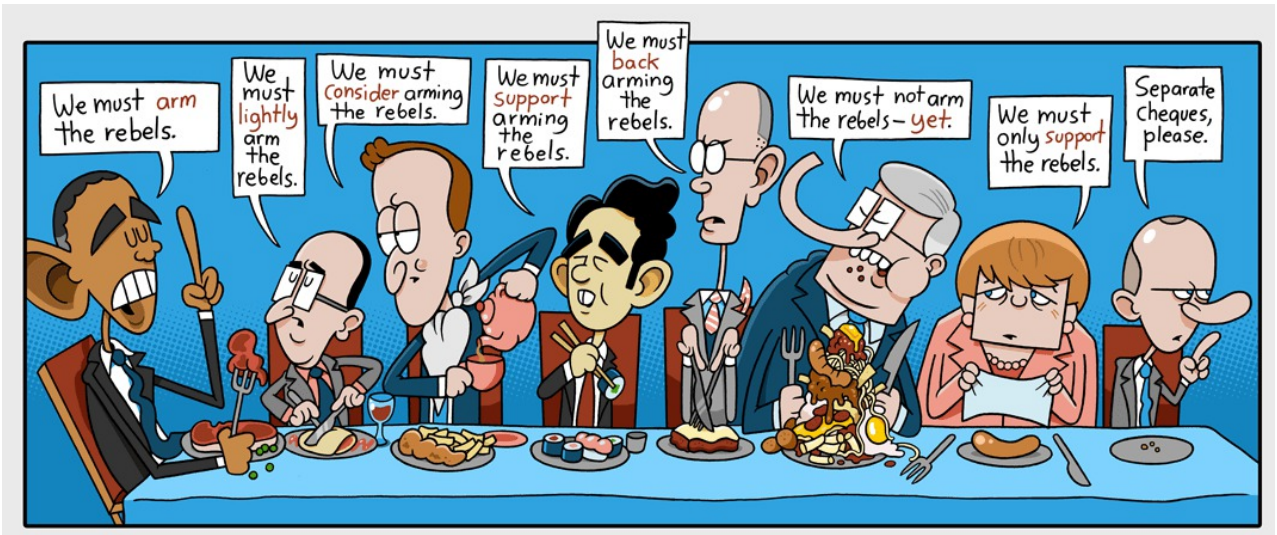
So failing processes never make the remaining network disconnected.

Assumption: Crashing of processes can't be observed.

Consensus

Binary consensus: Initially, all processes randomly select 0 or 1.

Eventually, all correct processes must uniformly decide 0 or 1.



Consensus underlies many important problems in distributed computing: termination detection, mutual exclusion, leader election, ...

Consensus - Assumptions

k-crash consensus: At most k processes may crash.

Validity: If all processes randomly select the same initial value b, then all correct processes decide b.

This excludes trivial solutions where e.g. processes always decide 0.

By validity, each k-crash consensus algorithm with $k \geq 1$ has a bivalent initial configuration that can reach terminal configurations with a decision 0 as well as with a decision 1.

Impossibility of 1-crash consensus

Theorem: No algorithm for 1-crash consensus always terminates.

Idea: A decision is determined by an event e at a process p.

Since p may crash, after e the other processes must be able to decide without input from p.

b-potent set of processes

A set S of processes is called b-potent, in a configuration, if by only executing events at processes in S, some process in S can decide b.

Question: Consider any k-crash consensus algorithm.

Why should each set of $N - k$ processes be b-potent for some b?

Impossibility of 1-crash consensus

Theorem: No algorithm for 1-crash consensus always terminates.

Proof: Consider a 1-crash consensus algorithm.

Let γ be a bivalent configuration: $\gamma \to \gamma_0$ and $\gamma \to \gamma_1$, where γ_0 can lead to decision 0 and γ_1 to decision 1.

- Let the transitions correspond to events at *different* processes.
  Then $\gamma_0 \to \delta \leftarrow \gamma_1$ for some δ. So γ_0 or γ_1 is bivalent.
- Let the transitions correspond to events at *one* process p.
  In γ, p can crash, so the other processes are b-potent for some b.
  Likewise for γ_0 and γ_1. It follows that γ_{1-b} is bivalent.

So each bivalent configuration has a transition to a bivalent configuration.

Hence each bivalent initial configuration yields an infinite computation.

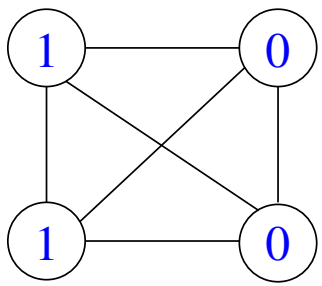
There exist *fair* infinite computations.

Impossibility of 1-crash consensus - Example

Let $N = 4$. At most one process can crash.

There are voting rounds, in which each process broadcasts it value.

Since one process may crash, in a round, processes can only wait for three votes.



The left (resp. right) processes might in every round receive two 1-votes and one 0-vote (resp. two 0-votes and one 1-vote).

(Admittedly, this scheduling of messages is unfair.)

Impossibility of $\lceil \frac{N}{2} \rceil$-crash consensus

Theorem: Let $k \geq \frac{N}{2}$. There is no Las Vegas algorithm for k-crash consensus.

Proof: Suppose, toward a contradiction, there is such an algorithm.

Divide the set of processes in S and T, with $|S| = \lfloor \frac{N}{2} \rfloor$ and $|T| = \lceil \frac{N}{2} \rceil$.

Suppose all processes in S select 0 and all processes in T select 1.

Suppose that messages between processes in S and in T are very slow.

Since $k \geq \frac{N}{2}$, at some point the processes in S must assume the processes in T all crashed, and decide 0.

Likewise, at some point the processes in T must assume the processes in S all crashed, and decide 1.

Question

Give a Monte Carlo algorithm for k-crash consensus for any k.

Answer: Let any process decide for its initial (random) value.

With a (very small) positive probability all correct processes decide for the same value.

Bracha-Toueg crash consensus algorithm

Let $k < \frac{N}{2}$. Initially, each correct process randomly randomly selects 0 or 1, with weight 1. In round n, at each correct, undecided p:

- p sends $\langle n, value_p, weight_p \rangle$ to all processes (including itself).
- p waits until $N - k$ messages $\langle n, b, w \rangle$ have arrived.

  (p purges/stores messages from earlier/future rounds.)

  If $w > \frac{N}{2}$ for a $\langle n, b, w \rangle$, then $value_p \leftarrow b$. *(This b is unique.)*

  Else, $value_p \leftarrow 0$ if most messages voted 0, $value_p \leftarrow 1$ otherwise.

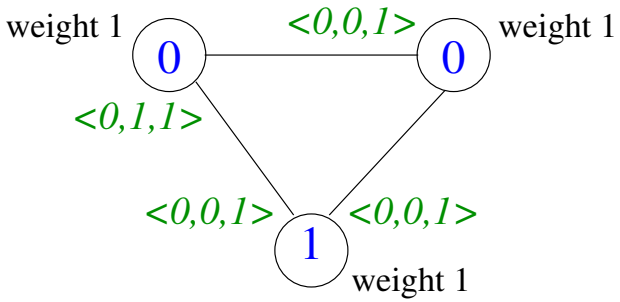
  $weight_p \leftarrow$ the number of incoming votes for $value_p$ in round n.
- If $w > \frac{N}{2}$ for $> k$ incoming messages $\langle n, b, w \rangle$, then p decides b.

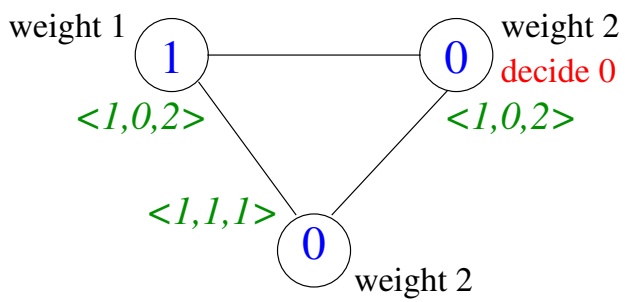
  (Note that $k < N - k$.)

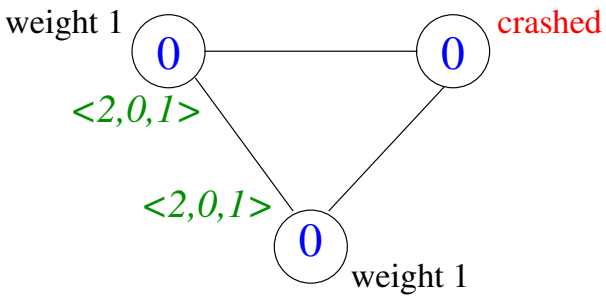
If p decides b, it broadcasts $\langle n + 1, b, N - k \rangle$ and $\langle n + 2, b, N - k \rangle$, and terminates.

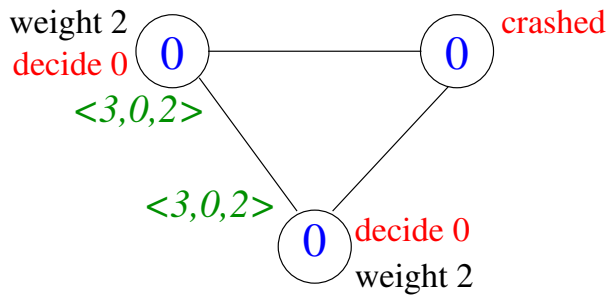
Bracha-Toueg crash consensus algorithm - Example

$N = 3$ and $k = 1$. Each round a correct process requires two incoming messages, and two b-votes with weight 2 to decide b.









(Messages of a process to itself aren't depicted.)

Bracha-Toueg crash consensus algorithm - Correctness

Theorem: Let $k < \frac{N}{2}$. The Bracha-Toueg k-crash consensus algorithm is a Las Vegas algorithm that terminates with probability 1.

Proof (part I): Suppose a process decides b in round n.

Then in round n, $value_q = b$ and $weight_q > \frac{N}{2}$ for $> k$ processes q.

So in round n, each correct process receives a $\langle q, b, w \rangle$ with $w > \frac{N}{2}$.

So in round $n+1$, all correct processes vote b.

So in round $n+2$, all correct processes vote b with weight $N - k$.

Hence, after round $n+2$, all correct processes have decided b.

Concluding, all correct processes decide for the same value.

Bracha-Toueg crash consensus algorithm - Correctness

Proof (part II): *Assumption*: Scheduling of messages is fair.

Due to fair scheduling, there is a chance $\rho > 0$ that in a round n all processes receive the first $N - k$ messages from the same processes.

After round n, all correct processes have the same value b.

After round $n + 1$, all correct processes have value b with weight $N - k$.

After round $n + 2$, all correct processes have decided b.

Concluding, the algorithm terminates with probability 1.

Impossibility of $\lceil \frac{N}{3} \rceil$-Byzantine consensus

Theorem: Let $k \geq \frac{N}{3}$. There is no Las Vegas algorithm for k-Byzantine consensus.

Proof: Suppose, toward a contradiction, there is such an algorithm.

Since $k \geq \frac{N}{3}$, we can choose sets S and T of processes with $|S| = |T| = N - k$ and $|S \cap T| \leq k$.

Suppose all processes in S select 0 and all processes in T select 1.

Suppose that messages between processes in S and in T are very slow.

Suppose all processes that aren't in $S \cup T$ are Byzantine.

The processes in S can then, with the help of the Byzantine processes, decide 0.

Likewise the processes in T can decide 1.

Bracha-Toueg Byzantine consensus algorithm

Let $k < \frac{N}{3}$.

Again, in every round, each correct process:

- broadcasts its value,
- waits for $N - k$ incoming messages, and
- changes its value to the majority of votes in the round.

(No weights are needed.)

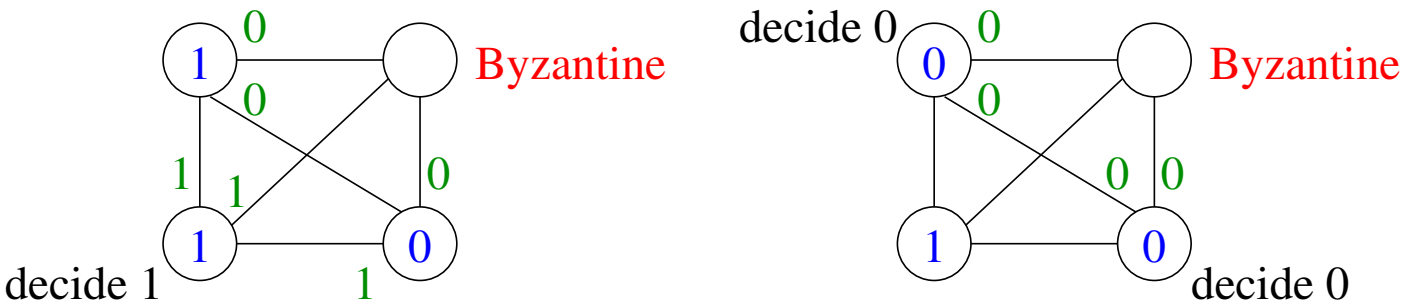
A correct process decides b if it receives $> \frac{N+k}{2}$ b-votes in one round.

(Note that $\frac{N+k}{2} < N - k$.)

Echo mechanism

Complication: A Byzantine process may send different votes to different processes.

Example: Let $N = 4$ and $k = 1$. Each round, a correct process waits for three votes, and needs three b-votes to decide b.



Solution: Each incoming vote is verified using an echo mechanism.

A vote is *accepted* after $> \frac{N+k}{2}$ confirming echos.

Bracha-Toueg Byzantine consensus algorithm

Initially, each correct process randomly selects 0 or 1.

In round n, at each correct, undecided p:

- p sends $\langle \textbf{vote}, n, value_p \rangle$ to all processes (including itself).
- If p receives $\langle \textbf{vote}, m, b \rangle$ from q, it sends $\langle \textbf{echo}, q, m, b \rangle$ to all processes (including itself).
- p counts incoming $\langle \textbf{echo}, q, n, b \rangle$ messages for each q, b.

  When $> \frac{N+k}{2}$ such messages arrived, p *accepts* q's b-vote.
- The round is completed when p has accepted $N - k$ votes.

  If most votes are for 0, then $value_p \leftarrow 0$. Else, $value_p \leftarrow 1$.

Bracha-Toueg Byzantine consensus algorithm

Processes purges/stores messages from earlier/future rounds.

If multiple messages $\langle \textbf{vote}, m, _ \rangle$ or $\langle \textbf{echo}, q, m, _ \rangle$ arrive via the same channel, only the first one is taken into account.

If $> \frac{N+k}{2}$ of the *accepted* votes are for b, then p decides b.

When p decides b, it broadcasts $\langle \textbf{decide}, b \rangle$ and terminates.

The other processes interpret $\langle \textbf{decide}, b \rangle$ as a b-vote by p, and a b-echo by p for each q, for all rounds to come.

Questions

If an undecided process receives $\langle \textbf{decide}, b \rangle$, why can it in general not immediately decide b?

Answer: The message may originate from a Byzantine process.

What happens if all k Byzantine processes keep silent?

Answer: The $N - k$ correct processes reach consensus in two rounds.

Bracha-Toueg Byzantine consensus alg. - Example

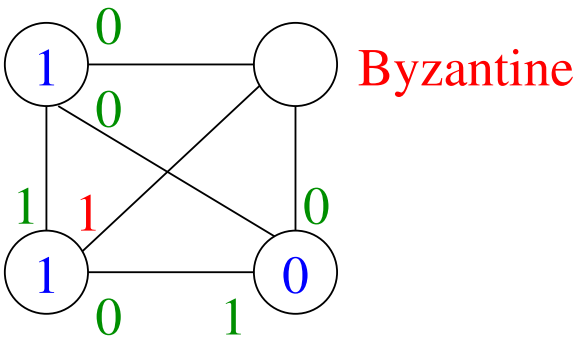
We study the previous example again, now with verification of votes.

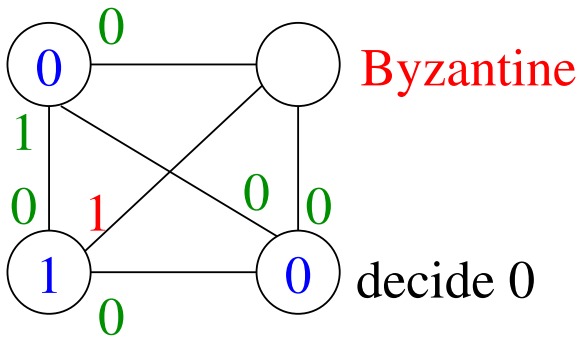
$N = 4$ and $k = 1$, so each round a correct process needs:

- $> \frac{N+k}{2}$, i.e. three, confirmations to accept a vote;
- $N - k$, i.e. three, accepted votes to determine a value; and
- $> \frac{N+k}{2}$, i.e. three, accepted b-votes to decide b.

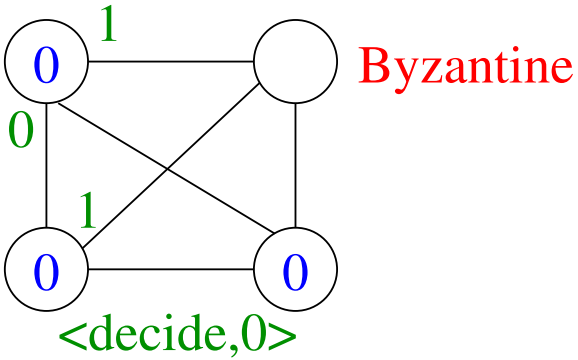
Only relevant **vote** messages are depicted (without their round number).

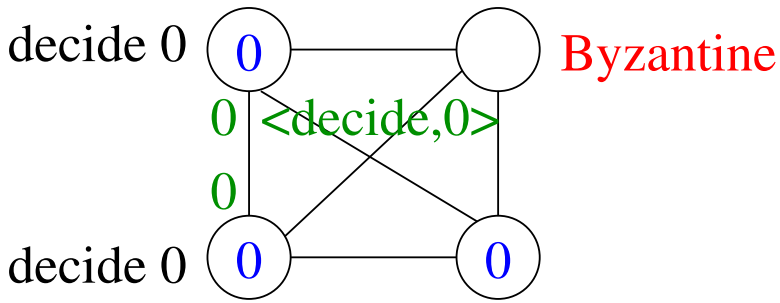
Bracha-Toueg Byzantine consensus alg. - Example





In round zero, the left bottom process doesn't accept vote 1 by the Byzantine process, since none of the other two correct processes confirm this vote. So it waits for (and accepts) vote 0 by the right bottom process, and thus doesn't decide 1 in round zero.





Bracha-Toueg Byzantine consensus alg. - Correctness

Theorem: Let $k < \frac{N}{3}$. The Bracha-Toueg k-Byzantine consensus algorithm is a Las Vegas algorithm that terminates with probability 1.

Proof: Each round, the correct processes eventually accept $N - k$ votes, since there are $\geq N - k$ correct processes. *(Note that* $N - k > \frac{N+k}{2}$*.)*

In round n, let correct processes p and q accept votes for b and b', respectively, from a process r.

Then they received $> \frac{N+k}{2}$ messages $\langle \textbf{echo}, r, n, b \rangle$ resp. $\langle \textbf{echo}, r, n, b' \rangle$.

$> k$ processes, so at least one correct process, sent such messages to both p and q.

So $b = b'$.

Bracha-Toueg Byzantine consensus alg. - Correctness

Suppose a correct process decides b in round n.

In this round it accepts $> \frac{N+k}{2}$ b-votes.

So in round n, correct processes accept $> \frac{N+k}{2} - k = \frac{N-k}{2}$ b-votes.

Hence, after round n, $value_q = b$ for each correct q.

So correct processes will vote b in all rounds $m > n$.

(Because they will accept $\geq N - 2k > \frac{N-k}{2}$ b-votes.)

Bracha-Toueg Byzantine consensus alg. - Correctness

Let S be a set of $N - k$ correct processes.

Assuming fair scheduling, there is a chance $\rho > 0$ that in a round each process in S accepts $N - k$ votes from the processes in S.

With chance ρ^2 this happens in consecutive rounds $n, n + 1$.

After round n, all processes in S have the same value b.

After round $n + 1$, all processes in S have decided b.

Lecture in a nutshell

crashed / Byzantine processes

complete network / crashes can't be observed

(binary) consensus

no algorithm for 1-crash consensus always terminates

if $k \geq \frac{N}{2}$, there is no Las Vegas algorithm for k-crash consensus

Bracha-Toueg k-crash consensus algorithm for $k < \frac{N}{2}$

if $k \geq \frac{N}{3}$, there is no Las Vegas algorithm for k-Byzantine consensus

Bracha-Toueg k-Byzantine consensus algorithm for $k < \frac{N}{3}$

Failure detection

A failure detector at a process keeps track which processes have (or may have) crashed.

Given an upper bound on network latency, and heartbeat messages, one can implement a failure detector.

With a failure detector, the proof of impossibility of 1-crash consensus no longer applies.

For this setting, terminating crash consensus algorithms exist.

Failure detection

Aim: To detect *crashed* processes.

We assume a time domain, with a total order.

$F(\tau)$ is the set of crashed processes at time τ.

$$\tau_1 \leq \tau_2 \Rightarrow F(\tau_1) \subseteq F(\tau_2) \qquad \text{(i.e., no restart)}$$

Assumption: Processes can't observe $F(\tau)$.

$H(p, \tau)$ is the set of processes that p *suspects* to be crashed at time τ.

Each computation is decorated with :

- a failure pattern F
- a failure detector history H

Complete failure detector



We require that failure detectors are complete :

From some time onward, each crashed process is suspected by each correct process.

Strongly accurate failure detector

A failure detector is strongly accurate if only crashed processes are ever suspected.

Assumptions:

- Each correct process broadcasts **alive** every ν time units.
- $d_{\max}$ is a known upper bound on network latency.

A process from which no message is received for $\nu + d_{\max}$ time units has crashed.

This failure detector is *complete* and *strongly accurate*.

Weakly accurate failure detector

A failure detector is weakly accurate if some (correct) process is never suspected by any process.

Assume a complete and *weakly accurate* failure detector.

We give a rotating coordinator algorithm for $(N-1)$-crash consensus.

Consensus with weakly accurate failure detection

Processes are numbered: $p_0, \ldots, p_{N-1}$.

Initially, each process randomly selects 0 or 1. In round n:

- p_n (if not crashed) broadcasts its value.
- Each process waits:
  - either for an incoming message from p_n, in which case it adopts the value of p_n;
  - or until it suspects that p_n has crashed.

After round $N-1$, each correct process decides for its value.

Correctness: Let p_j never be suspected.

After round j, all correct processes have the same value b.

Hence, after round $N-1$, all correct processes decide b.

Eventually strongly accurate failure detector

A failure detector is eventually strongly accurate if
from some time onward, only crashed processes are suspected.

Assumptions:

- Each correct process broadcasts **alive** every ν time units.
- There is an *unknown* upper bound on network latency.

Each process q initially guesses as network latency $d_q = 1$.

If q receives no message from p for $\nu + d_q$ time units, then
q *suspects* that p has crashed.

If q receives a message from a suspected process p, then
p is no longer suspected and $d_q \leftarrow d_q + 1$.

This failure detector is *complete* and *eventually strongly accurate*.

Impossibility of $\lceil \frac{N}{2} \rceil$-crash consensus

Theorem: Let $k \geq \frac{N}{2}$. There is no Las Vegas algorithm for k-crash consensus based on an *eventually strongly accurate* failure detector.

Proof: Suppose, toward a contradiction, there is such an algorithm.

Divide the set of processes in S and T, with $|S| = \lfloor \frac{N}{2} \rfloor$ and $|T| = \lceil \frac{N}{2} \rceil$.

Suppose all processes in S select 0 and all processes in T select 1.

Suppose that for a long time the processes in S suspect the processes in T crashed, and the processes in T suspect the processes in S crashed.

The processes in S can then decide 0, while the process in T can decide 1.

Chandra-Toueg k-crash consensus algorithm

A failure detector is eventually weakly accurate if
from some time onward *some* (correct) process is never suspected.

Let $k < \frac{N}{2}$. A complete and *eventually weakly accurate* failure detector is used for k-crash consensus.

Each process q records the last round lu_q in which it updated $value_q$.

Initially, $value_q \in \{0, 1\}$ and $lu_q = -1$.

Processes are numbered: $p_0, \ldots, p_{N-1}$.

Round n is coordinated by p_c with $c = n \bmod N$.

Chandra-Toueg k-crash consensus algorithm

- In round n, each correct q sends $\langle \textbf{vote}, n, value_q, lu_q \rangle$ to p_c.
- p_c (if not crashed) waits until $N - k$ such messages arrived, and selects one, say $\langle \textbf{vote}, n, b, \ell \rangle$, with ℓ as large as possible.

  $value_{p_c} \leftarrow b$, $lu_{p_c} \leftarrow n$, and p_c broadcasts $\langle \textbf{value}, n, b \rangle$.
- Each correct q waits:
  - either until $\langle \textbf{value}, n, b \rangle$ arrives: then $value_q \leftarrow b$, $lu_q \leftarrow n$, and q sends $\langle \textbf{ack}, n \rangle$ to p_c;
  - or until it suspects p_c crashed: then q sends $\langle \textbf{nack}, n \rangle$ to p_c.
- If p_c receives $> k$ messages $\langle \textbf{ack}, n \rangle$, then p_c decides b, and broadcasts $\langle \textbf{decide}, b \rangle$.

An undecided process that receives $\langle \textbf{decide}, b \rangle$, decides b.

Chandra-Toueg algorithm - Correctness

Theorem: Let $k < \frac{N}{2}$. The Chandra-Toueg algorithm is an (always terminating) k-crash consensus algorithm.

Proof (part I): If the coordinator in some round n receives $> k$ **ack**'s, then (for some $b \in \{0, 1\}$):

(1) there are $> k$ processes q with $lu_q \geq n$, and

(2) $lu_q \geq n$ implies $value_q = b$.

Properties (1) and (2) are preserved in all rounds $m > n$.

This follows by induction on $m - n$.

By (1), in round m the coordinator receives a **vote** with $lu \geq n$.

Hence, by (2), the coordinator of round m sets its value to b, and broadcasts $\langle \textbf{value}, m, b \rangle$.

So from round n onward, processes can only decide b.

Chandra-Toueg algorithm - Correctness

Proof (part II):

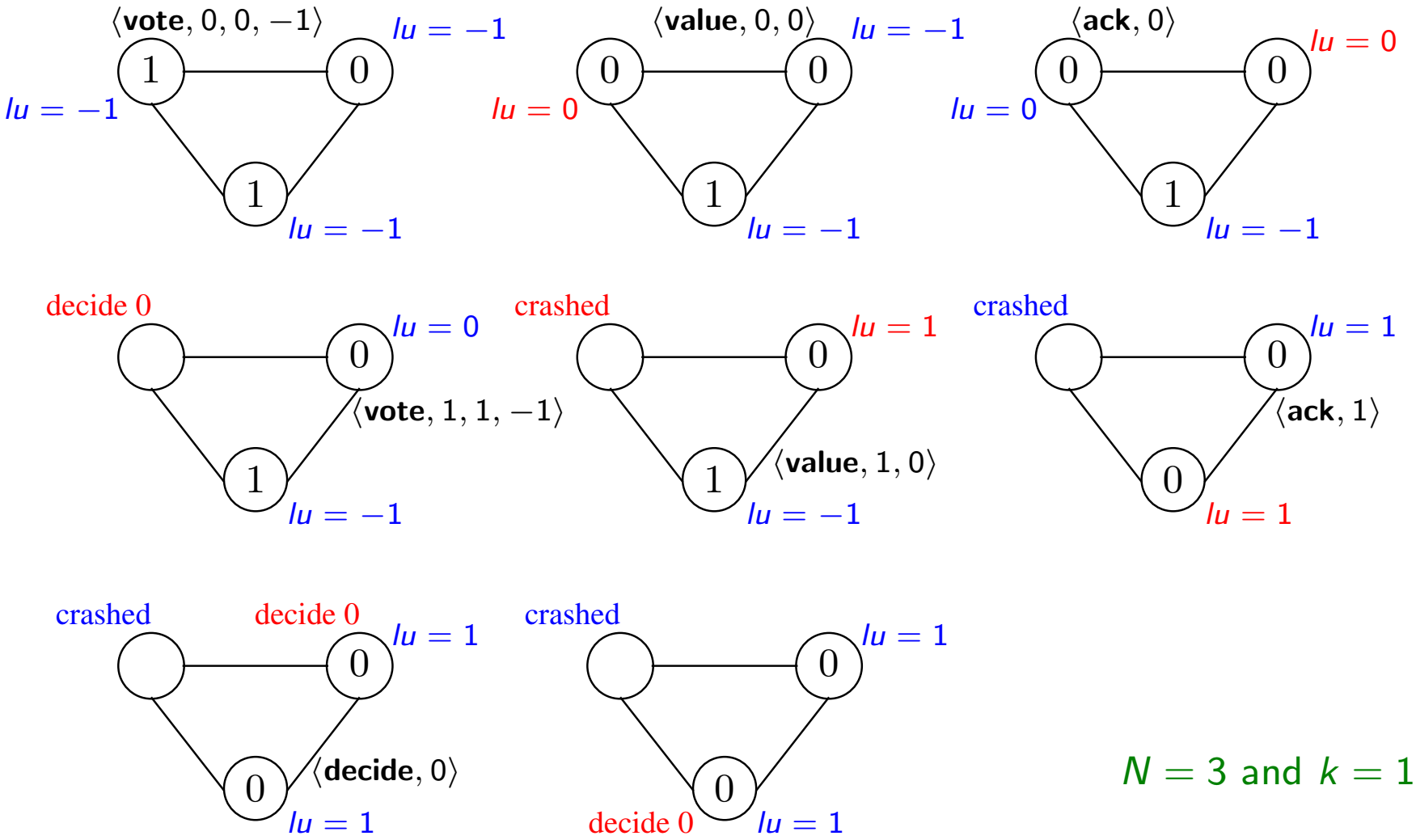
Since the failure detector is eventually weakly accurate,
from some round onward, some process p will never be suspected.

So when p becomes the coordinator, it receives $\geq N - k$ **ack**'s.

Since $N - k > k$, it decides.

All correct processes eventually receive the **decide** message of p,
and also decide.

Chandra-Toueg algorithm - Example



$N = 3$ and $k = 1$

Messages and ack's that a process sends to itself and 'irrelevant' messages are omitted.

Question

Why is it difficult to devise a failure detector for Byzantine processes ?

Answer: Failure detectors are usually based on the *absence* of events.

Local clocks with bounded drift

Let's forget about Byzantine processes for a moment.

The time domain is $\mathbb{R}_{\geq 0}$.

Each process p has a local clock $C_p(\tau)$, which returns a time value at *real* time τ.

Local clocks have bounded drift, compared to real time:

If C_p isn't adjusted between times τ_1 and τ_2, then

$$\frac{1}{\rho}(\tau_2 - \tau_1) \;\leq\; C_p(\tau_2) - C_p(\tau_1) \;\leq\; \rho(\tau_2 - \tau_1)$$

for some *known* $\rho > 1$.

Clock synchronization

At certain time intervals, the processes *synchronize* clocks :

They read each other's clock values, and adjust their local clocks.

The aim is to achieve, for some δ, and all τ,

$$|C_p(\tau) - C_q(\tau)| \leq \delta$$

Due to drift, this precision may degrade over time, necessitating repeated clock synchronizations.

We assume a *known* bound $d_{\max}$ on network latency.

For simplicity, let $d_{\max}$ be much smaller than δ, so that this latency can be ignored in the clock synchronization.

Clock synchronization

Suppose that after each synchronization, at say real time τ, for all processes p, q:

$$|C_p(\tau) - C_q(\tau)| \leq \delta_0$$

for some $\delta_0 < \delta$.

Due to ρ-bounded drift of local clocks, at real time $\tau + R$,

$$|C_p(\tau + R) - C_q(\tau + R)| \leq \delta_0 + (\rho - \frac{1}{\rho})R < \delta_0 + \rho R$$

So synchronizing every $\frac{\delta - \delta_0}{\rho}$ (real) time units suffices.

Impossibility of $\lceil \frac{N}{3} \rceil$-Byzantine synchronizers

Theorem: Let $k \geq \frac{N}{3}$. There is no k-Byzantine clock synchronizer.

Proof: Let $N = 3$, $k = 1$. Processes are p, q, r; r is Byzantine.

(The construction below easily extends to general N and $k \geq \frac{N}{3}$.)

Let the clock of p run faster than the clock of q.

Suppose a synchronization takes place at real time τ.

r sends $C_p(\tau) + \delta$ to p, and $C_q(\tau) - \delta$ to q.

p and q can't recognize that r is Byzantine.

So they have to stay within range δ of the value reported by r.

Hence p can't decrease, and q can't increase its clock value.

By repeating this scenario at each synchronization round,
the clock values of p and q get further and further apart.

Mahaney-Schneider synchronizer

Consider a complete network of N processes.

Suppose at most $k < \frac{N}{3}$ processes are Byzantine.

Each correct process in a synchronization round:

1. Collects the clock values of all processes (waiting for $2d_{\max}$).
2. Discards those reported values τ for which $< N - k$ processes report a value in the interval $[\tau - \delta, \tau + \delta]$.

   (They are from Byzantine processes.)
3. Replaces all discarded/non-received values by an accepted value.
4. Takes the average of these N values as its new clock value.

Mahaney-Schneider synchronizer - Correctness

Lemma: Let $k < \frac{N}{3}$. If in some synchronization round values a_p and a_q pass the filters of correct processes p and q, respectively, then

$$|a_p - a_q| \leq 2\delta$$

Proof: $\geq N - k$ processes reported a value in $[a_p - \delta,\ a_p + \delta]$ to p.

And $\geq N - k$ processes reported a value in $[a_q - \delta,\ a_q + \delta]$ to q.

Since $N - 2k > k$, at least one correct process r reported a value in $[a_p - \delta,\ a_p + \delta]$ to p, and in $[a_q - \delta,\ a_q + \delta]$ to q.

Since r reports the same value to p and q, it follows that

$$|a_p - a_q| \leq 2\delta$$

Mahaney-Schneider synchronizer - Correctness

Theorem: Let $k < \frac{N}{3}$. The Mahaney-Schneider synchronizer is k-Byzantine.

Proof : a_{pr} (resp. a_{qr}) denotes the value that correct process p (resp. q) accepts from or assigns to process r, in some synchronization round.

By the lemma, for all r, $|a_{pr} - a_{qr}| \leq 2\delta$.

Moreover, $a_{pr} = a_{qr}$ for all *correct* r.

Hence, for all correct p and q,

$$|\frac{1}{N}(\sum_{\text{processes } r} a_{pr}) - \frac{1}{N}(\sum_{\text{processes } r} a_{qr})| \leq \frac{1}{N} k 2\delta < \frac{2}{3}\delta$$

So we can take $\delta_0 = \frac{2}{3}\delta$.

There should be a synchronization every $\frac{\delta}{3\rho}$ (real) time units.

Lecture in a nutshell (part I)

complete failure detector

strongly accurate failure detector

rotating coordinator crash consensus algorithm with
a weakly accurate failure detector

eventually strongly accurate failure detector

k-crash consensus for $k \geq \frac{N}{2}$ remains impossible with
an eventually strongly accurate failure detection

Chandra-Toueg crash consensus algorithm with
an eventually weakly accurate failure detector

Lecture in a nutshell (part II)

local clocks with ρ-bounded drift (where ρ is known)

synchronize clocks so that they stay within δ of each other

for $k \geq \frac{N}{3}$, there is no k-Byzantine synchronizer

Mahaney-Schneider k-Byzantine synchronizer

Synchronous networks

Let's again forget about Byzantine processes for a moment.

A synchronous network proceeds in pulses. In one pulse, each process:

1. sends messages
2. receives messages
3. performs internal events

A message is sent and received in the same pulse.

Such synchrony is called lockstep.



Building a synchronous network

Assume ρ-bounded local clocks with precision δ.

For simplicity, we ignore the network latency.

When a process reads clock value $(i-1)\rho^2\delta$, it starts pulse i.

Key question: Does a process p receive all messages for pulse i before it starts pulse $i+1$? That is, for all q,

$$C_q^{-1}((i-1)\rho^2\delta) \leq C_p^{-1}(i\rho^2\delta)$$

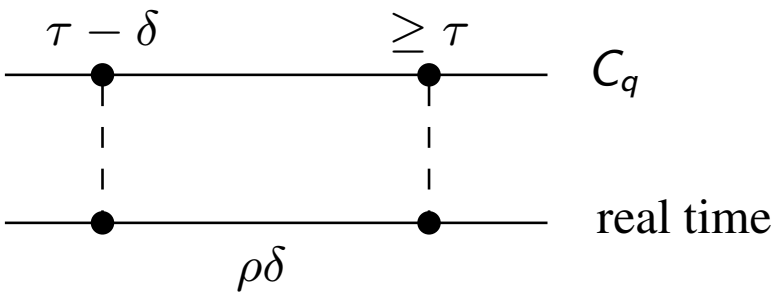
Because then q starts pulse i no later than p starts pulse $i+1$.

($C_r^{-1}(\tau)$ is the moment in time the clock of r returns τ.)

Building a synchronous network

Since the clock of q is ρ-bounded from below,

$$C_q^{-1}(\tau) \leq C_q^{-1}(\tau - \delta) + \rho\delta$$



Since local clocks have precision δ,

$$C_q^{-1}(\tau - \delta) \leq C_p^{-1}(\tau)$$

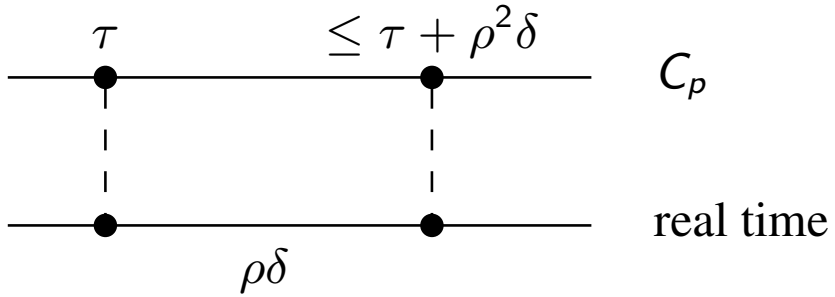
Hence, for all τ,

$$C_q^{-1}(\tau) \leq C_p^{-1}(\tau) + \rho\delta$$

Building a synchronous network

Since the clock of p is ρ-bounded from above,

$$C_p^{-1}(\tau) + \rho\delta \;\leq\; C_p^{-1}(\tau + \rho^2\delta)$$



Hence,

$$\begin{aligned} C_q^{-1}((i-1)\rho^2\delta) &\leq C_p^{-1}((i-1)\rho^2\delta) + \rho\delta \\ &\leq C_p^{-1}(i\rho^2\delta) \end{aligned}$$

Byzantine consensus for synchronous systems

In a synchronous system, the proof of impossibility of 1-crash consensus no longer applies.

For this setting, terminating Byzantine consensus algorithms exist.

Byzantine broadcast



Consider a synchronous network of N processes, in which at most $k < \frac{N}{3}$ processes are Byzantine.

One process g, called the general, is given an input $x_g \in \{0, 1\}$.

The other processes, called lieutenants, know who is the general.

Requirements for k-Byzantine broadcast:

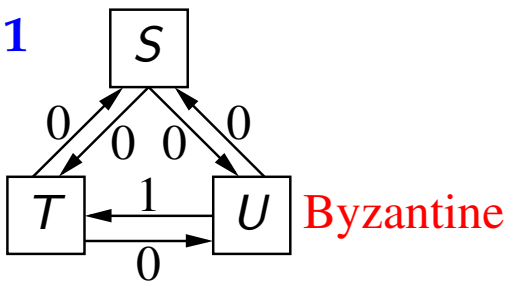
- Termination: Every correct process decides 0 or 1.
- Agreement: All correct processes decide the same value.
- Dependence: If the general is correct, it decides x_g.

Impossibility of $\lceil \frac{N}{3} \rceil$-Byzantine broadcast

Theorem: Let $k \geq \frac{N}{3}$. There is no k-Byzantine broadcast algorithm for synchronous networks.

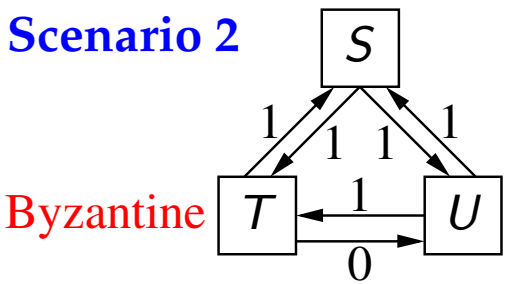
Proof: Divide the processes into three sets S, T and U with each $\leq k$ elements. Let $g \in S$.

Scenario 1



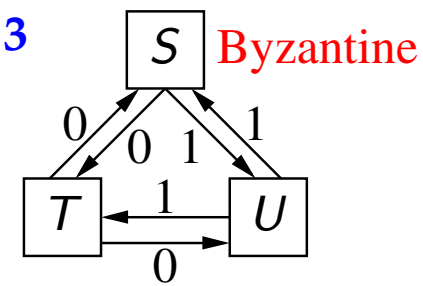
The processes in S and T decide 0

Scenario 2



The processes in S and U decide 1

Scenario 3



The processes in T decide 0 and in U decide 1

Lamport-Shostak-Pease broadcast algorithm

$Broadcast_g(N, k)$ is a k-Byzantine broadcast algorithm for a synchronous network of size N, with $k < \frac{N}{3}$.

Pulse 1: The general g broadcasts and decides x_g.

If a lieutenant q receives b from g then $x_q \leftarrow b$, else $x_q \leftarrow 0$.

If $k = 0$: each lieutenant q decides x_q.

If $k > 0$: for each lieutenant p, each (correct) lieutenant q takes part in $Broadcast_p(N-1, k-1)$ in pulse 2 (g is excluded).

Pulse $k + 1$ ($k > 0$):

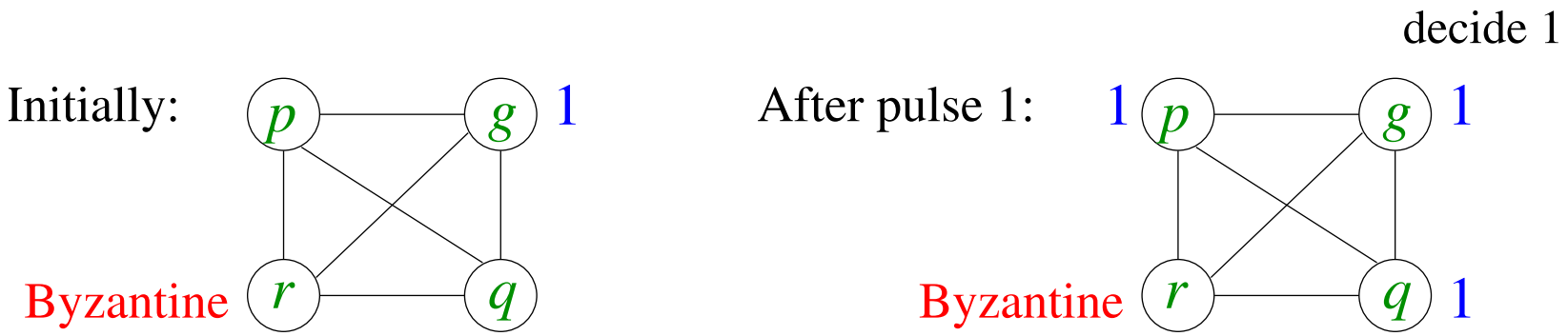
Lieutenant q has, for each lieutenant p, computed a value in $Broadcast_p(N-1, k-1)$; it stores these values in $M_q[p]$.

$x_q \leftarrow major(M_q)$; lieutenant q decides x_q.

(*major* maps each list over $\{0, 1\}$ to 0 or 1, such that if more than half of the elements in M are b, then $major(M) = b$.)

Lamport-Shostak-Pease broadcast alg. - Example 1

$N = 4$ and $k = 1$; general correct.



After pulse 1, g has decided 1, and the correct lieutenants p and q carry the value 1.

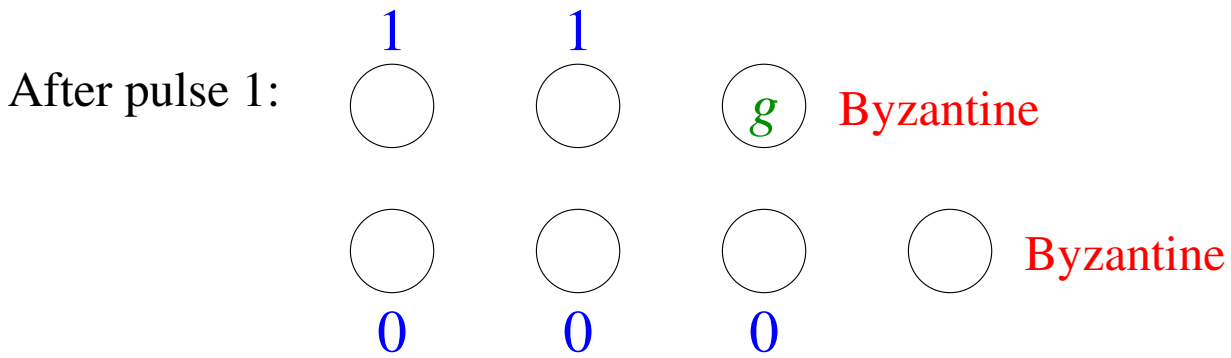
Consider the sub-network without g.

In $Broadcast_p(3, 0)$ and $Broadcast_q(3, 0)$, p and q both compute 1, while in $Broadcast_r(3, 0)$ they may compute an arbitrary value.

So p and q both build a list $\{1, 1, _\}$, and decide 1.

Lamport-Shostak-Pease broadcast alg. - Example 2

$N = 7$ and $k = 2$; general Byzantine. (Channels are omitted.)

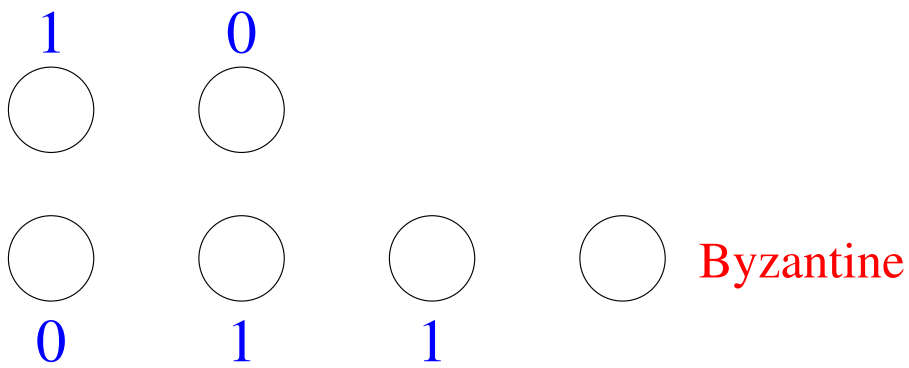


$k - 1 < \frac{N-1}{3}$, so by induction, the recursive calls $Broadcast_p(6, 1)$ lead to the same list $M = \{1, 1, 0, 0, 0, b\}$, for some $b \in \{0, 1\}$, at all correct lieutenants.

So in $Broadcast_g(7, 2)$, they all decide $major(M)$.

Lamport-Shostak-Pease broadcast alg. - Example 2

For instance, in $Broadcast_p(6, 1)$ with p the Byzantine lieutenant, the following values could be distributed by p.



Then the five subcalls $Broadcast_q(5, 0)$, for the correct lieutenants q, would at each correct lieutenant lead to the list $\{1, 0, 0, 1, 1\}$.

So in that case $b = major(\{1, 0, 0, 1, 1\}) = 1$.

Questions

Question: Draw a tree of all recursive subcalls of $Broadcast_g(7, 2)$.

Question: Consider $Broadcast_g(N, 1)$, with a correct general g.

Let fewer than $\frac{N-1}{2}$ lieutenants be Byzantine.

Argue that all correct processes decide x_g.

Lamport-Shostak-Pease broadcast alg. - Correctness

Lemma: If the general g is correct, and $< \frac{N-k}{2}$ lieutenants are Byzantine, then in $Broadcast_g(N, k)$ all correct processes decide x_g.

Proof: By induction on k. Case $k = 0$ is trivial, because g is correct.

Let $k > 0$.

Since g is correct, in pulse 1, at all correct lieutenants p, $x_p \leftarrow x_g$.

Since $\frac{(N-1)-(k-1)}{2} = \frac{N-k}{2}$, by induction, for all correct lieutenants p, in $Broadcast_p(N-1, k-1)$ the value $x_p = x_g$ is computed.

Since a majority of the lieutenants is correct (because $k > 0$), in pulse $k+1$, at each correct lieutenant p, $x_p \leftarrow major(M_p) = x_g$.

Lamport-Shostak-Pease broadcast alg. - Correctness

Theorem: Let $k < \frac{N}{3}$. $Broadcast_g(N, k)$ is an (always terminating) k-Byzantine broadcast algorithm for synchronous networks.

Proof: By induction on k.

If g is correct, the theorem follows from the lemma and $k < \frac{N}{3}$.

Let g be Byzantine (so $k > 0$). Then $\leq k-1$ lieutenants are Byzantine.

Since $k-1 < \frac{N-1}{3}$, by induction, for every lieutenant p, all correct lieutenants compute in $Broadcast_p(N-1, k-1)$ the *same* value.

Hence, all correct lieutenants compute the *same* list M.

So in pulse $k+1$, all correct lieutenants decide $major(M)$.

Partial synchrony

A synchronous system can be obtained if local clocks have known *bounded drift*, and there is a known *upper bound on network latency*.

In a partially synchronous system,

- the bounds on the inaccuracy of local clocks and network latency are unknown, or
- these bounds are known, but only valid from some unknown point in time.

Dwork, Lynch and Stockmeyer showed that, for $k < \frac{N}{3}$, there is a k-Byzantine broadcast algorithm for partially synchronous systems.

These ideas are at the core of the *Paxos* consensus protocol.

Public-key cryptosystems

Given a large message domain $\mathcal{M}$.

A public-key cryptosystem consists of functions $S_q, P_q : \mathcal{M} \to \mathcal{M}$, for each process q, with

$$S_q(P_q(m)) \;=\; P_q(S_q(m)) \;=\; m \qquad \text{for all } m \in \mathcal{M}.$$

S_q is kept *secret*, P_q is made *public*.

Underlying assumption: Computing S_q from P_q is very expensive.

p sends a *secret* message m to q: $P_q(m)$

p sends a *signed* message m to q: $\langle m, S_p(m) \rangle$

Such signatures guarantee that Byzantine processes can't lie about the messages they have received.

Lamport-Shostak-Pease authentication algorithm

Pulse 1: The general broadcasts $\langle x_g, (S_g(x_g), g)\rangle$, and decides x_g.

Pulse i: If a lieutenant q receives a message $\langle v, (\sigma_1, p_1) : \cdots : (\sigma_i, p_i)\rangle$ that is valid, i.e.:

- $p_1 = g$
- $p_1, \ldots, p_i, q$ are distinct
- $P_{p_k}(\sigma_k) = v$ for all $k = 1, \ldots, i$

then q includes v in the set W_q.

If $i \leq k$, then in pulse $i + 1$, q sends to all other lieutenants

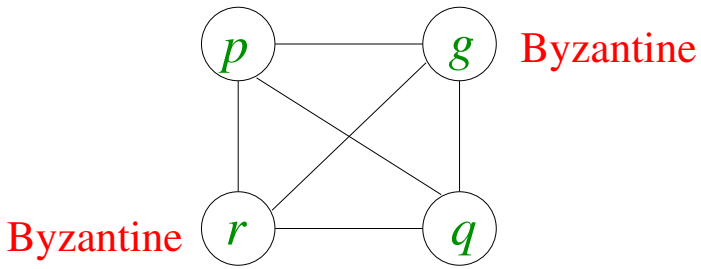
$$\langle v, (\sigma_1, p_1) : \cdots : (\sigma_i, p_i) : (S_q(v), q)\rangle$$

After pulse $k + 1$, each correct lieutenant p decides

v	if W_p is a singleton $\{v\}$, or
0	otherwise (the general is Byzantine)

Lamport-Shostak-Pease authentication alg. - Example

$N = 4$ and $k = 2$.



pulse 1: g sends $\langle 1, (S_g(1), g)\rangle$ to p and q
g sends $\langle 0, (S_g(0), g)\rangle$ to r
$W_p = W_q = \{1\}$

pulse 2: p broadcasts $\langle 1, (S_g(1), g) : (S_p(1), p)\rangle$
q broadcasts $\langle 1, (S_g(1), g) : (S_q(1), q)\rangle$
r sends $\langle 0, (S_g(0), g) : (S_r(0), r)\rangle$ to q
$W_p = \{1\}$ and $W_q = \{0, 1\}$

pulse 3: q broadcasts $\langle 0, (S_g(0), g) : (S_r(0), r) : (S_q(0), q)\rangle$
$W_p = W_q = \{0, 1\}$
p and q decide 0

Lamport-Shostak-Pease authentication alg. - Correctness

Theorem: The Lamport-Shostak-Pease authentication algorithm is an (always terminating) k-Byzantine broadcast algorithm, for any k.

Proof: If the general is correct, then owing to authentication, correct lieutenants q only add x_g to W_q. So they all decide x_g.

Let a correct lieutenant receive a valid message $\langle v, \ell \rangle$ in a pulse $\leq k$.

In the next pulse, it makes all correct lieutenants p add v to W_p.

Let a correct lieutenant receive a valid message $\langle v, \ell \rangle$ in pulse $k+1$.

Since ℓ has length $k+1$, it contains a correct q.

Then q received a valid message $\langle v, _ \rangle$ in a pulse $\leq k$.

In the next pulse, q made all correct lieutenants p add v to W_p.

So after pulse $k+1$, W_p is the same for all correct lieutenants p.

Lamport-Shostak-Pease authentication alg. - Optimization

Dolev-Strong optimization: Each correct lieutenant broadcasts at most two messages, with different values.

Because when it has broadcast two different values, all correct lieutenants are certain to decide 0.

Lecture in a nutshell

ρ-bounded local clocks with precision δ

synchronous network

Byzantine broadcast

no k-Byzantine broadcast if $k \geq \frac{N}{3}$

Lamport-Shostak-Pease broadcast algorithm if $k < \frac{N}{3}$

Lamport-Shostak-Pease authentication algorithm

Mutual exclusion

Processes contend to enter their *critical section*.

A process (allowed to be) in its critical section is called *privileged*.

For each computation we require:

Mutual exclusion: Always at most one process is privileged.

Starvation-freeness: If a process p tries to enter its critical section, and no process stays privileged forever, then p eventually becomes privileged.

Applications: Distributed shared memory, replicated data, atomic commit.

Mutual exclusion with message passing

Mutual exclusion algorithms with *message passing* are generally based on one of the following paradigms.

- Logical clock: Requests to enter a critical section are prioritized by means of logical time stamps.
- Token passing: The process holding the token is privileged.
- Leader election: A process that wants to become privileged sends a request to the leader.
- Quorums: To become privileged, a process needs permission from a quorum of processes.

  Each pair of quorums has a non-empty intersection.

Ricart-Agrawala algorithm

When a process p_i wants to access its critical section, it sends $request(ts_i, i)$ to all other processes, with ts_i its logical time stamp.

When p_j receives this request, it sends permission to p_i as soon as :

- p_j isn't privileged, and
- p_j doesn't have a pending request with time stamp ts_j where $(ts_j, j) < (ts_i, i)$ (lexicographical order).

p_i enters its critical section when it has received permission from all other processes.

When p_i exits its critical section, it sends permission to all pending requests.

Ricart-Agrawala algorithm - Example 1

$N = 2$, and p_0 and p_1 both are at logical time 0.

p_1 sends $request(1, 1)$ to p_0.

When p_0 receives this message, it sends permission to p_1, setting the time at p_0 to 2.

p_0 sends $request(2, 0)$ to p_1.

When p_1 receives this message, it doesn't send permission to p_0, because $(1, 1) < (2, 0)$.

p_1 receives permission from p_0, and enters its critical section.

Ricart-Agrawala algorithm - Example 2

$N = 2$, and p_0 and p_1 both are at logical time 0.

p_1 sends $request(1, 1)$ to p_0, and p_0 sends $request(1, 0)$ to p_1.

When p_0 receives the request from p_1,
it doesn't send permission to p_1, because $(1, 0) < (1, 1)$.

When p_1 receives the request from p_0,
it sends permission to p_0, because $(1, 0) < (1, 1)$.

p_0 and p_1 both set their logical time to 2.

p_0 receives permission from p_1, and enters its critical section.

Ricart-Agrawala algorithm - Correctness

Mutual exclusion: When p sends permission to q:

- p isn't privileged; and
- p won't get permission from q to enter its critical section until q has entered and left its critical section.

  (Because p's pending or future request is larger than q's current request.)

Starvation-freeness: Each request will eventually become the smallest request in the network.

Ricart-Agrawala algorithm - Optimization

Drawback: High message overhead, because requests must be sent to all other processes.

Carvalho-Roucairol optimization: After a process q has exited its critical section, q only needs to send requests to the processes that q has sent permission to since this exit.

Suppose q is waiting for permissions and didn't send a request to p.

If p sends a request to q that is smaller than q's request, then q sends both permission and a request to p.

This optimization is correct since for each pair of distinct processes, at least one must ask permission from the other.

Question

Suppose processes p_0 and p_1 enter and exit their critical section, and concurrently want to become privileged again.

What happens, if the Carvalho-Roucairol optimization is employed ?

Raymond's algorithm

Given an undirected network, with a sink tree.

At any time, the root, holding a token, is privileged.

Each process maintains a FIFO queue, which can contain id's of its children, and its own id. Initially, this queue is empty.

Queue maintenance:

- When a non-root wants to enter its critical section, it adds its id to its own queue.
- When a non-root gets a new head at its (non-empty) queue, it asks its parent for the token.
- When a process receives a request for the token from a child, it adds this child to its queue.

Raymond's algorithm

When the root exits its critical section (and its queue is non-empty),

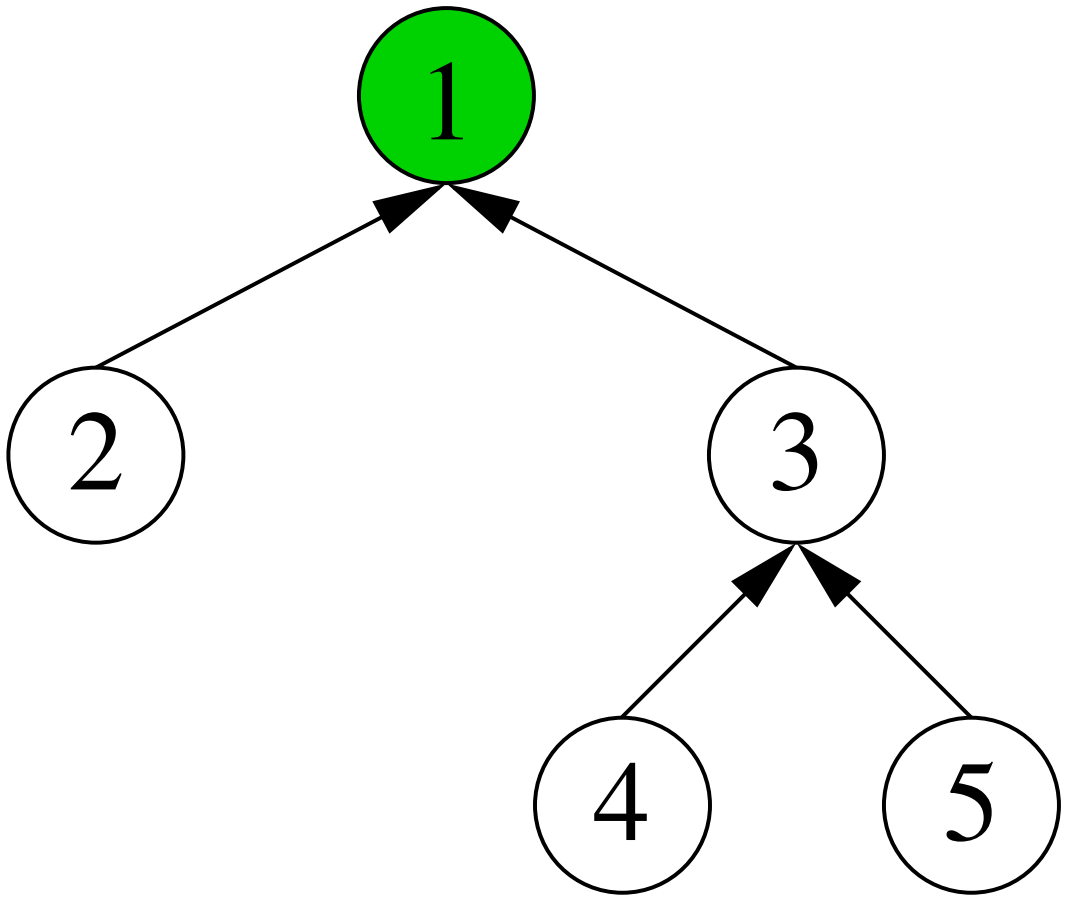
- it sends the token to the process q at the head of its queue,
- makes q its parent,
- and removes q from the head of its queue.

Let p get the token from its parent, with q at the head of its queue:

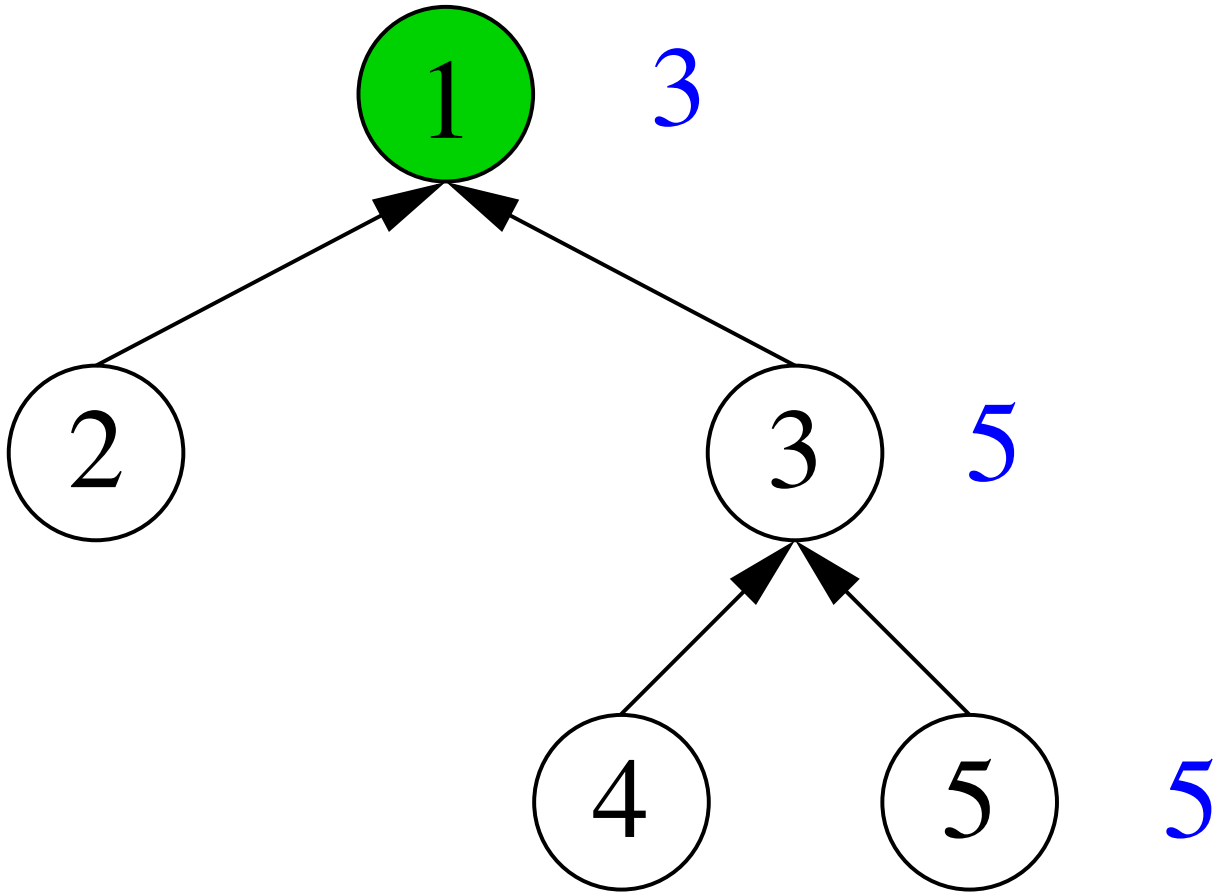
- If $q \neq p$, then p sends the token to q, and makes q its parent.
- If $q = p$, then p becomes the root
  (i.e., it has no parent, and is privileged).

In both cases, p removes q from the head of its queue.

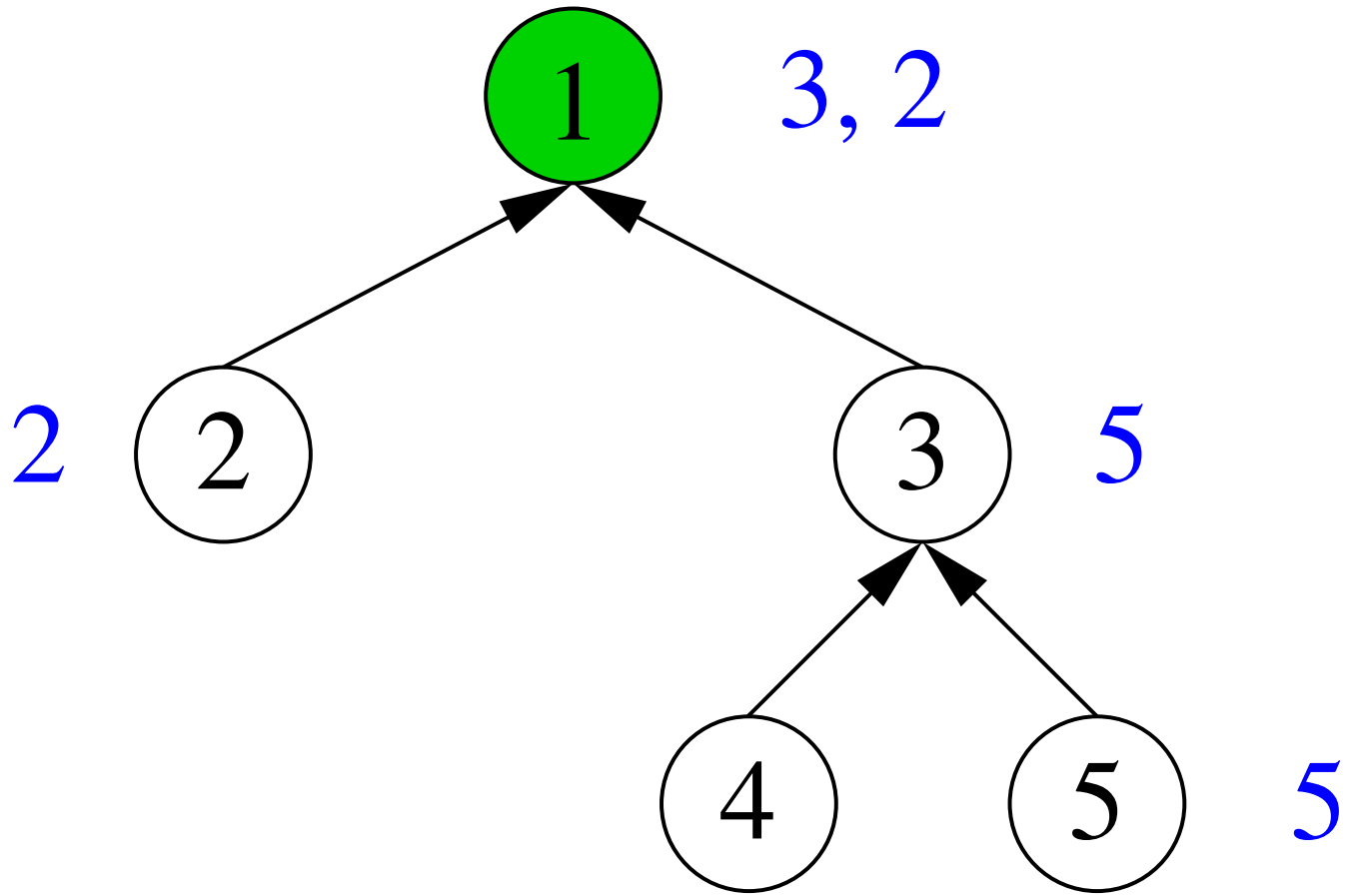
Raymond's algorithm - Example



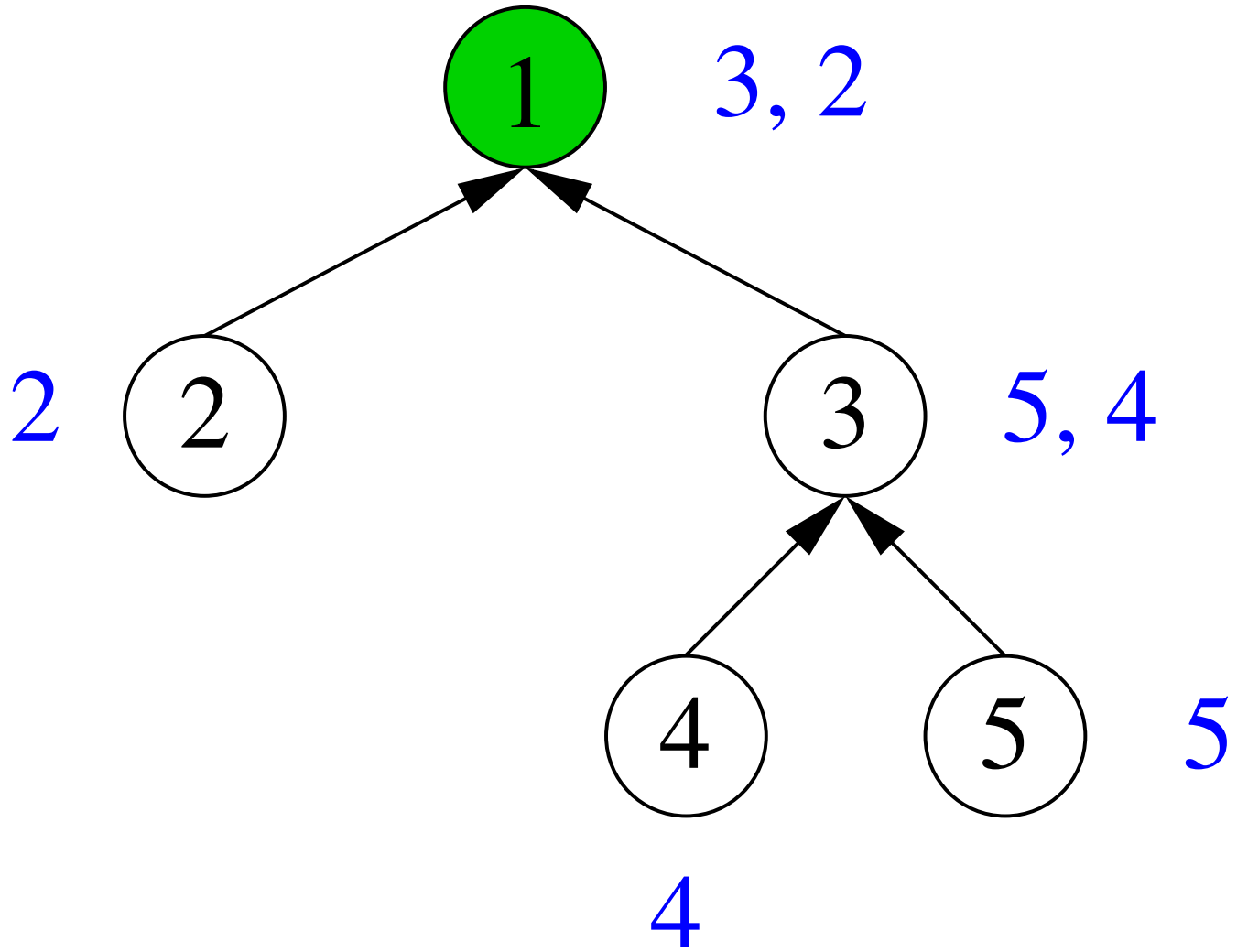
Raymond's algorithm - Example



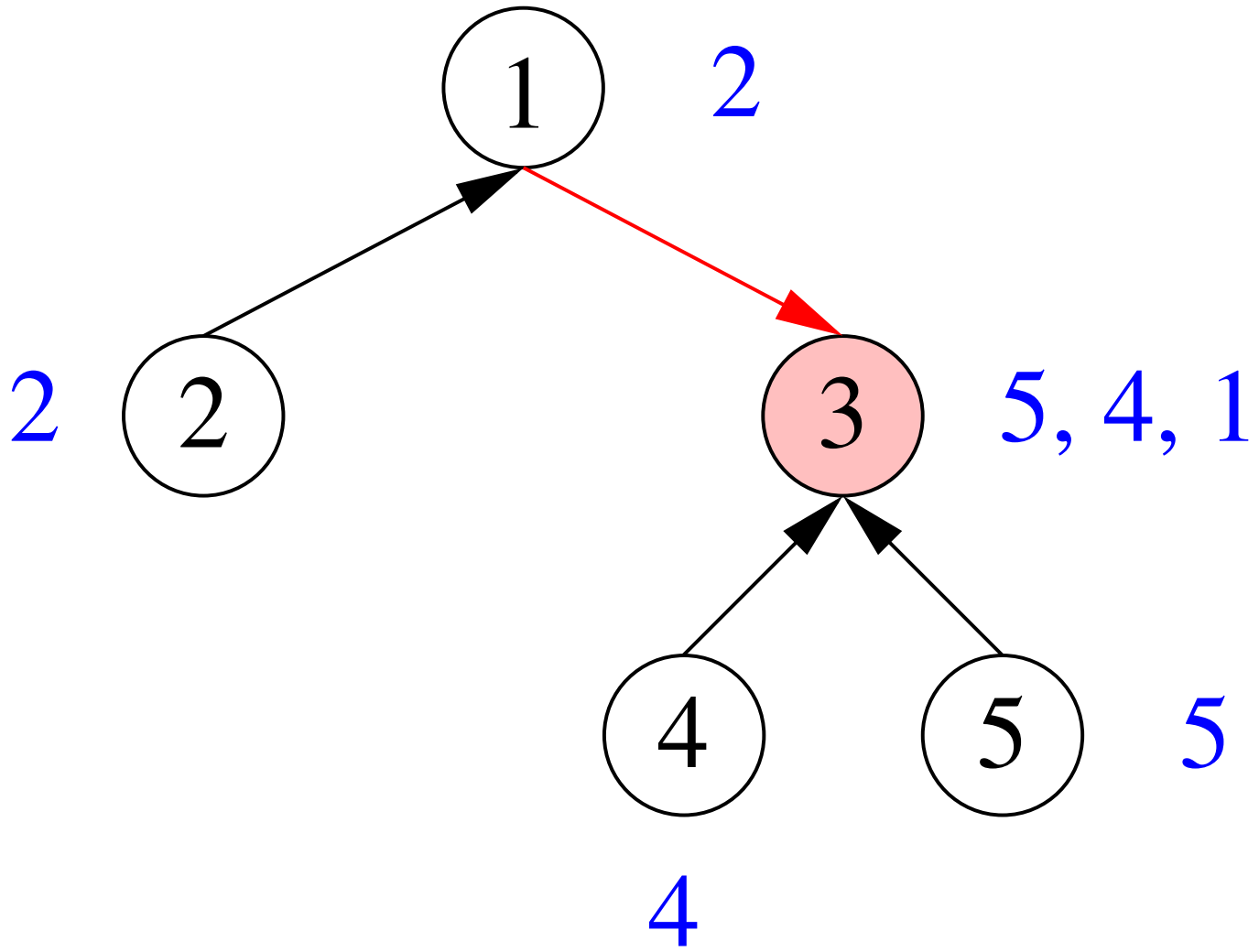
Raymond's algorithm - Example



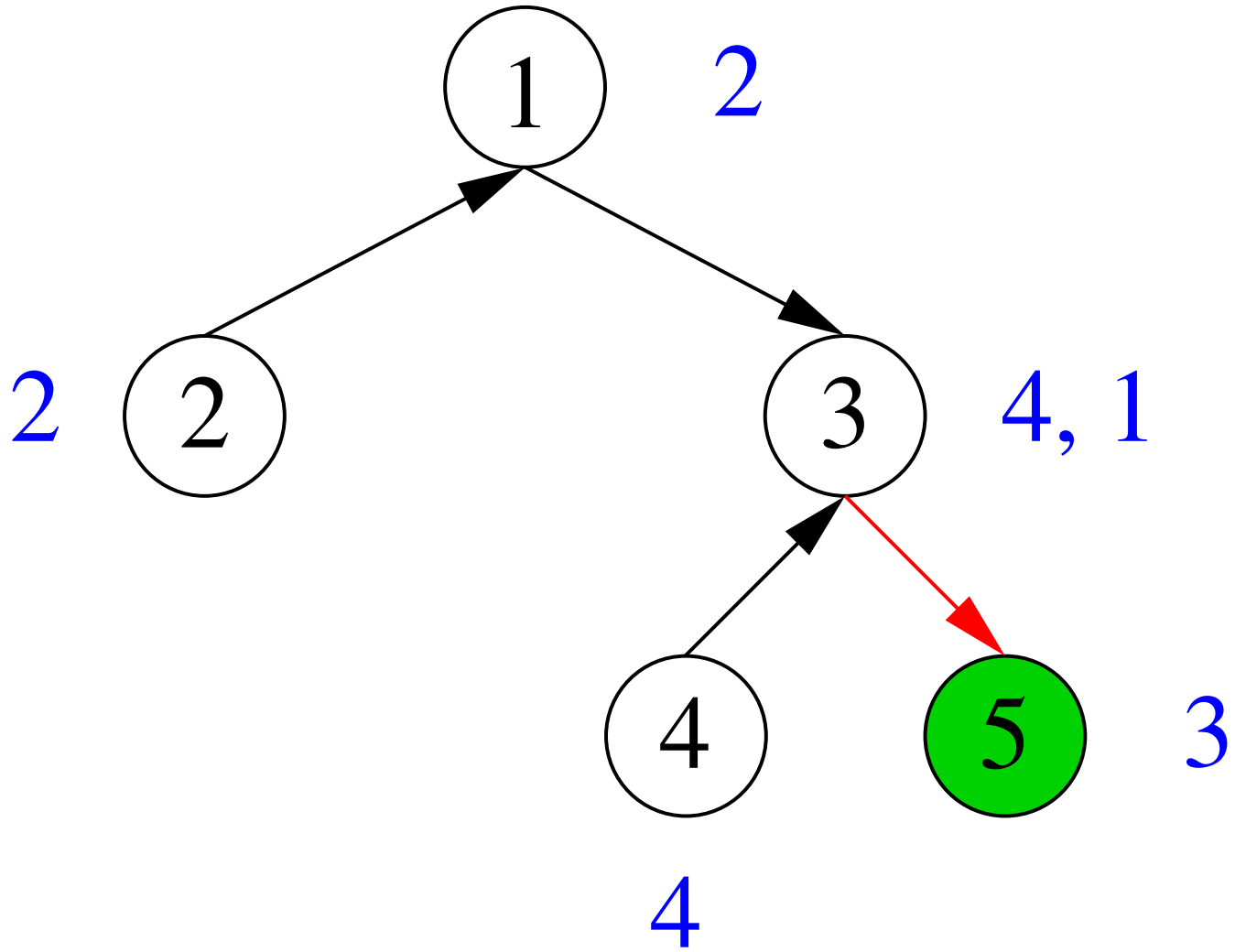
Raymond's algorithm - Example



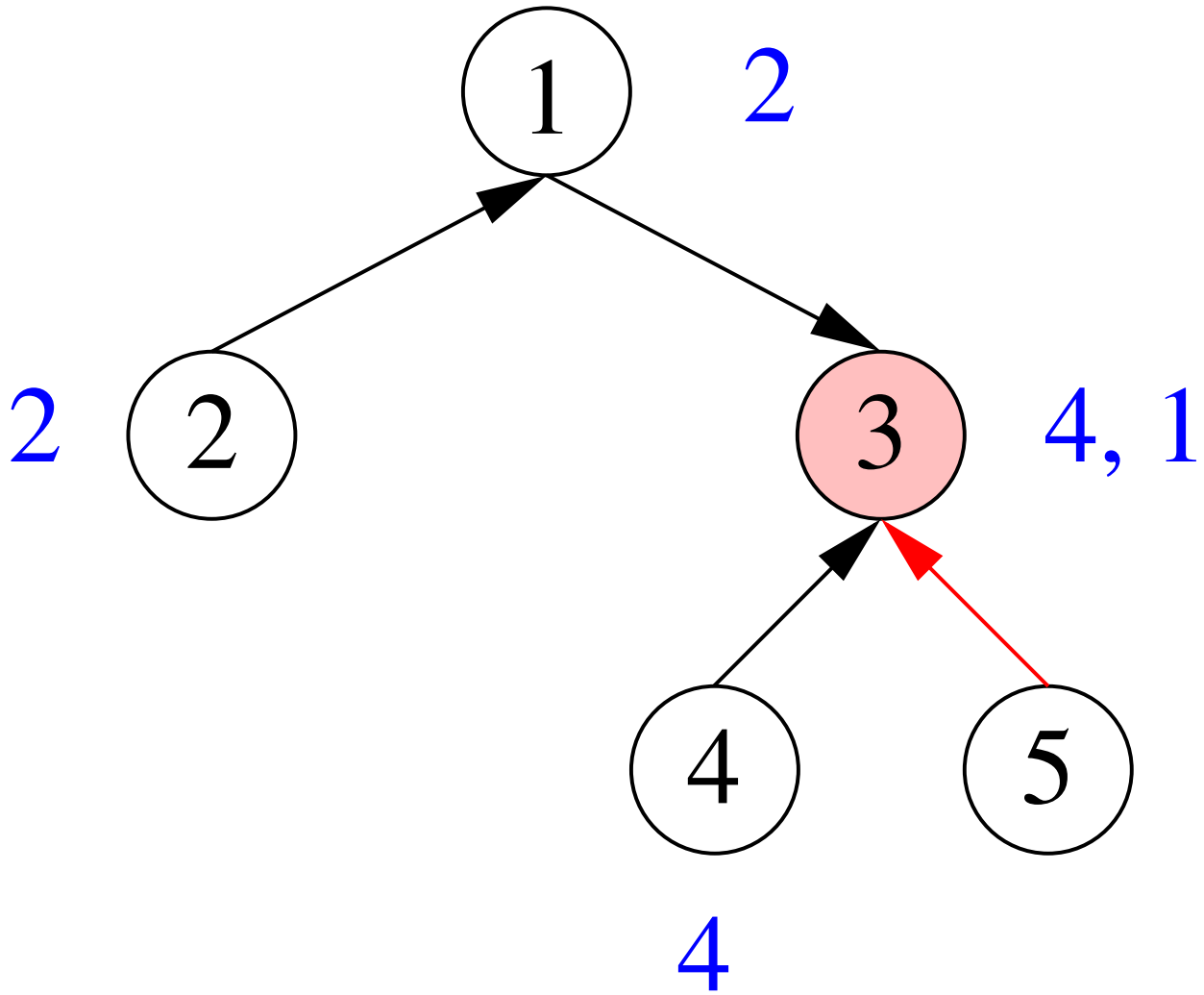
Raymond's algorithm - Example



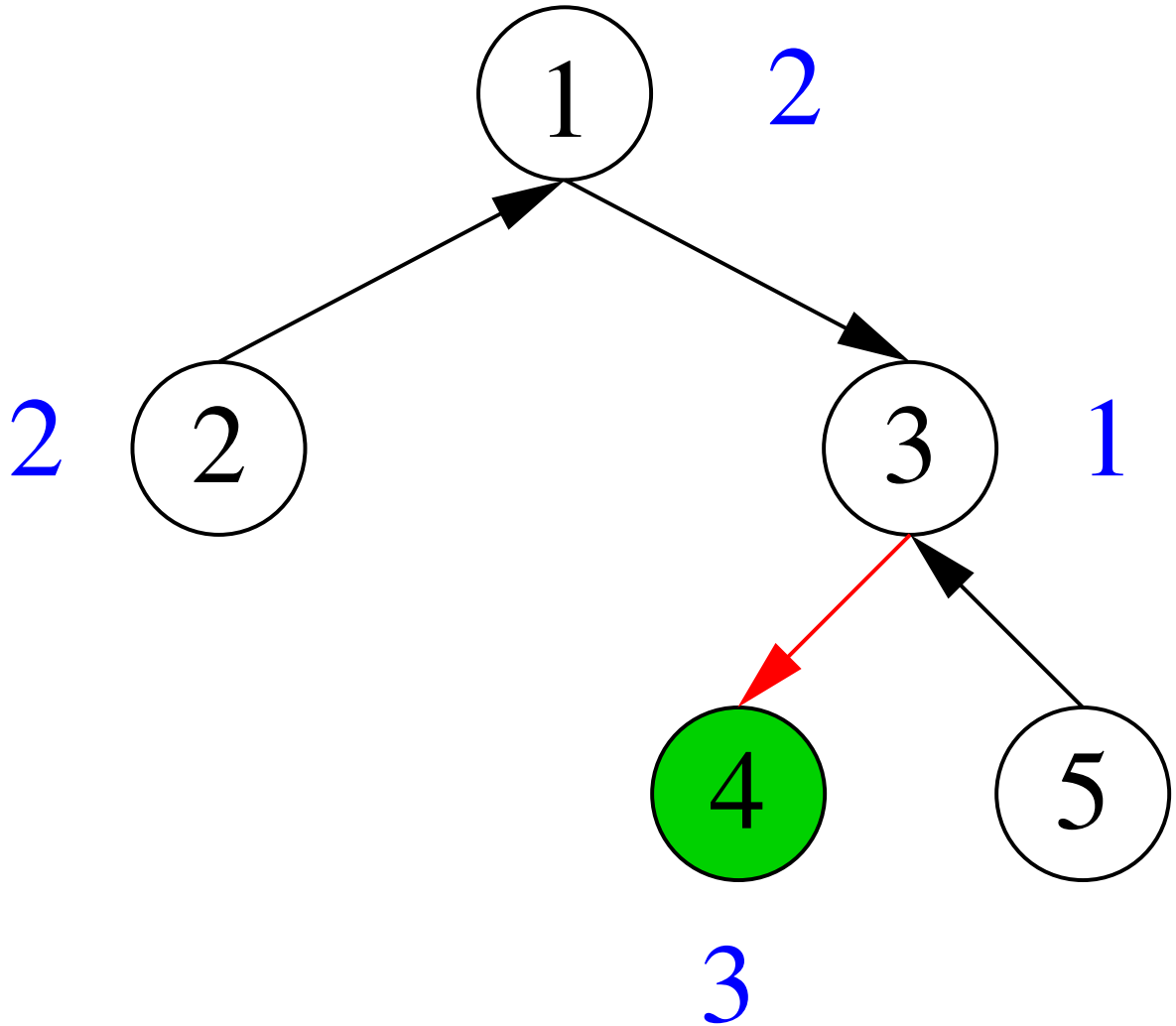
Raymond's algorithm - Example



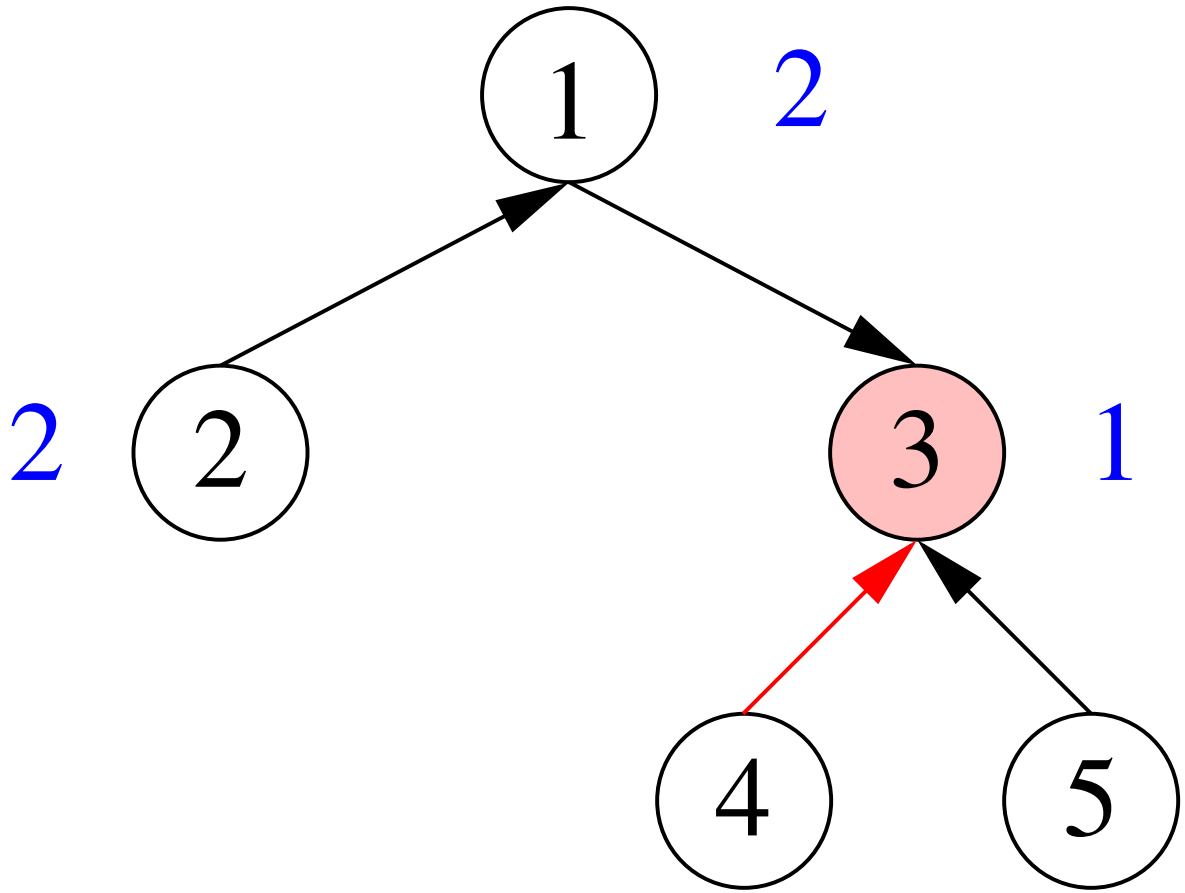
Raymond's algorithm - Example



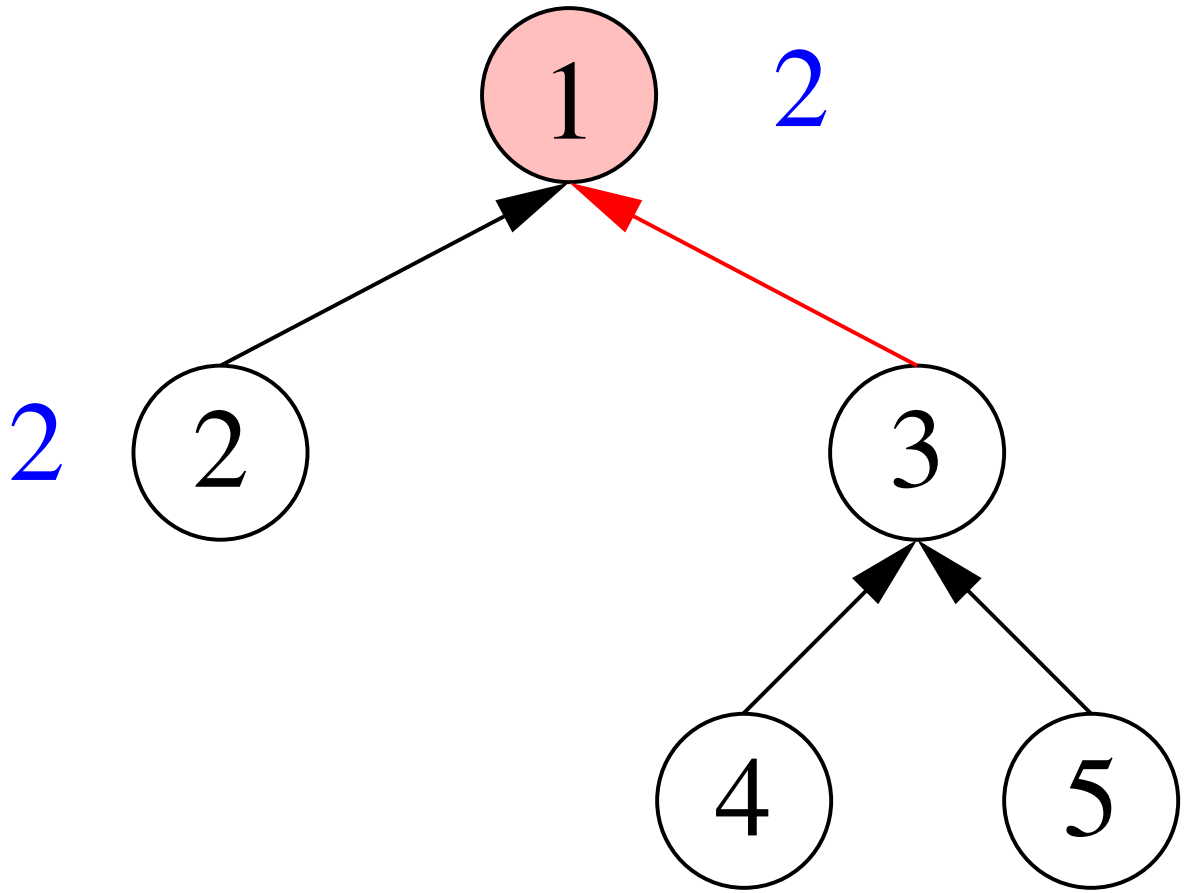
Raymond's algorithm - Example



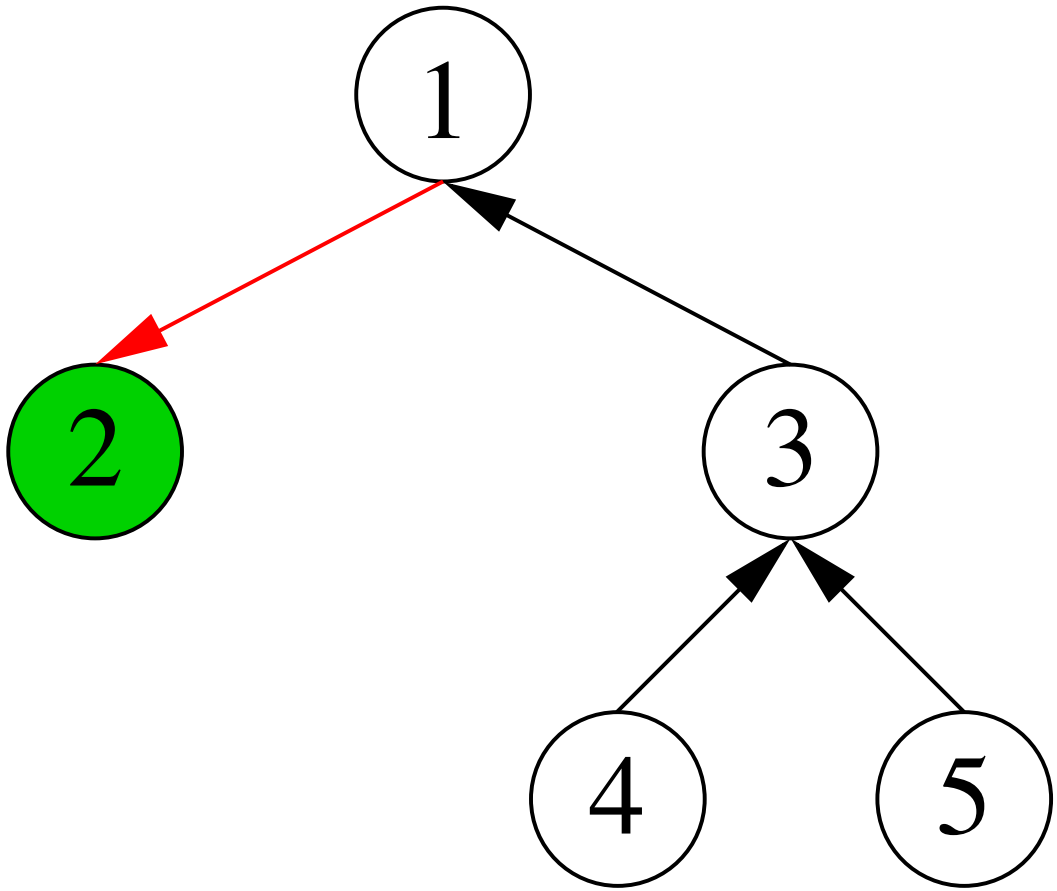
Raymond's algorithm - Example



Raymond's algorithm - Example



Raymond's algorithm - Example



Raymond's algorithm - Correctness

Raymond's algorithm provides mutual exclusion, because at all times there is at most one root.

Raymond's algorithm is starvation-free, because eventually each request in a queue moves to the head of this queue, and a chain of requests never contains a cycle.

Drawback: Sensitive to failures.

Question

What is the Achilles' heel of a mutual exclusion algorithm based on a leader ?

Answer: The leader is a single point of failure.

Agrawal-El Abbadi algorithm

For simplicity we assume that $N = 2^k - 1$, for some $k > 1$.

The processes are structured in a *binary tree* of depth $k - 1$.

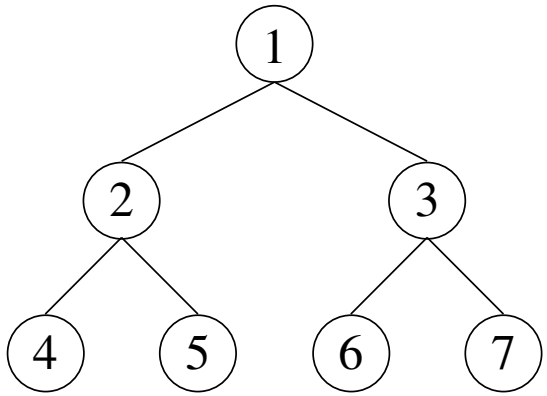
A *quorum* consists of all processes on a path from the root to a leaf.

If a *non-leaf* p has crashed (or is unresponsive),
permission can be asked from all processes on two paths instead:
from each child of p to some leaf.

To enter a critical section, permission from a quorum is required.

Agrawal-El Abbadi algorithm - Example

Example: Let $N = 7$.



Possible quorums are:

- $\{1, 2, 4\}$, $\{1, 2, 5\}$, $\{1, 3, 6\}$, $\{1, 3, 7\}$
- if 1 crashed: $\{2, 4, 3, 6\}$, $\{2, 5, 3, 6\}$, $\{2, 4, 3, 7\}$, $\{2, 5, 3, 7\}$
- if 2 crashed: $\{1, 4, 5\}$ (and $\{1, 3, 6\}$, $\{1, 3, 7\}$)
- if 3 crashed: $\{1, 6, 7\}$ (and $\{1, 2, 4\}$, $\{1, 2, 5\}$)

Question: What are the quorums if 1,2 crashed? And if 1,2,3 crashed? And if 1,2,4 crashed?

Agrawal-El Abbadi algorithm

A process p that wants to enter its critical section, places the root of the tree in a queue.

p repeatedly tries to get permission from the head r of its queue.

If successful, r is removed from p's queue.
If r is a non-leaf, *one* of r's children is appended to p's queue.

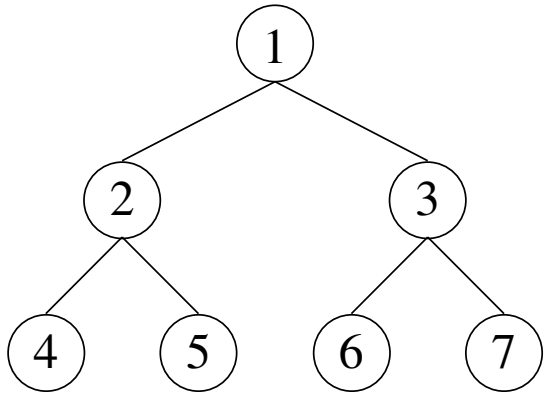
If non-leaf r has crashed, it is removed from p's queue,
and *both* of r's children are appended at the end of the queue
(in a fixed order, to avoid deadlocks).

If leaf r has crashed, p *aborts* its attempt to become privileged.

When p's queue becomes empty, it enters its critical section.

After exiting its critical section, p informs all processes in the quorum that their permission to p can be withdrawn.

Agrawal-El Abbadi algorithm - Example



p and q concurrently want to enter their critical section.

p gets permission from 1, and wants permission from 3.

1 crashes, and q now wants permission from 2 and 3.

q gets permission from 2, and appends 4 to its queue.

q obtains permission from 3, and appends 7 to its queue.

3 crashes, and p now wants permission from 6 and 7.

q gets permission from 4, and now wants permission from 7.

p gets permission from both 6 and 7, and enters its critical section.

Agrawal-El Abbadi algorithm - Mutual exclusion

We prove, by induction on depth k, that each pair of quorums has a non-empty intersection, and so mutual exclusion is guaranteed.

A quorum with 1 contains a quorum in one of the subtrees below 1, while a quorum without 1 contains a quorum in both subtrees below 1.

- If two quorums both contain 1, we are done.
- If two quorums both don't contain 1, then by induction they have elements in common in the two subtrees below process 1.
- Suppose quorum Q contains 1, while quorum Q' doesn't.

  Then Q contains a quorum in one of the subtrees below 1, and Q' also contains a quorum in this subtree.

  By induction, they have an element in common in this subtree.

Agrawal-El Abbadi algorithm - Deadlock-freeness

In case of a crashed process, let its left child be put before its right child in the queue of a process that wants to become privileged.

With such a fixed order, the Agrawal-El Abbadi algorithm is deadlock-free (as long as not all leaves have crashed).

Starvation however could happen.

Agrawal-El Abbadi algorithm - Deadlock-freeness

In case of a crashed process, let its left child be put before its right child in the queue of a process that wants to become privileged.

Let a process p at depth d be greater than any process

- at a depth $> d$ in the binary tree,
- or at depth d and more to the right than p in the binary tree.

A process with permission from r, never needs permission from a $q < r$.

This guarantees that, in case some leave is responsive, eventually some process will become privileged.

Starvation can happen, if a process waits for a permission infinitely long. (This can be easily resolved.)

Self-stabilization

All configurations are initial configurations.

An algorithm is self-stabilizing if every computation eventually reaches a correct configuration.

Advantages:

- fault tolerance
- straightforward initialization



Self-stabilizing *operating* and *database systems* have been developed.

Self-stabilization - Shared memory

In a *message-passing* setting, processes might all be initialized in a state where they are waiting for a message.

Then the self-stabilizing algorithm wouldn't exhibit any behavior.

Therefore, in self-stabilizing algorithms, processes communicate via variables in *shared memory*.

We assume that a process can read the variables of its neighbors.

Dijkstra's self-stabilizing token ring

Processes $p_0, \ldots, p_{N-1}$ form a directed ring.

Each p_i holds a value $x_i \in \{0, \ldots, K-1\}$ with $K \geq N$.

- p_i for each $i = 1, \ldots, N-1$ is privileged if $x_i \neq x_{i-1}$.
- p_0 is privileged if $x_0 = x_{N-1}$.

Each privileged process is allowed to change its value, causing the loss of its privilege:

- $x_i \leftarrow x_{i-1}$ when $x_i \neq x_{i-1}$, for each $i = 1, \ldots, N-1$
- $x_0 \leftarrow (x_0 + 1) \bmod K$ when $x_0 = x_{N-1}$

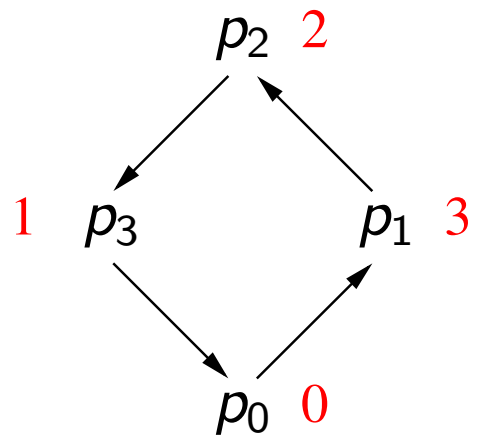
If $K \geq N$, then Dijkstra's token ring *self-stabilizes*.

That is, each computation eventually satisfies *mutual exclusion*.

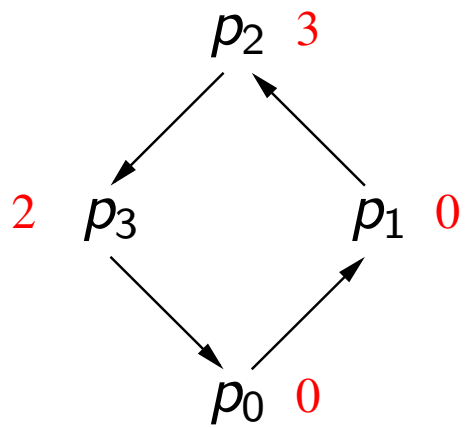
Moreover, Dijkstra's token ring is *starvation-free*.

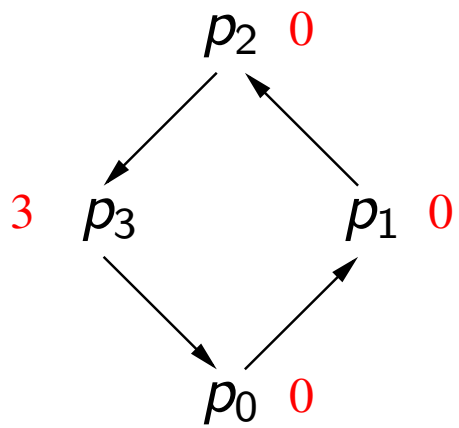
Dijkstra's token ring - Example

Let $N = K = 4$. Consider the initial configuration



It isn't hard to see that the ring self-stabilizes. For instance,





Dijkstra's token ring - Correctness

Theorem: If $K \geq N$, then Dijkstra's token ring self-stabilizes.

Proof: In each configuration at least one process is privileged.

An event never increases the number of privileged processes.

Consider an (infinite) computation. After at most $\frac{1}{2}(N-1)N$ events at $p_1, \ldots, p_{N-1}$, an event must happen at p_0.

So during the computation, x_0 ranges over all values in $\{0, \ldots, K-1\}$.

Since $p_1, \ldots, p_{N-1}$ only copy values, they restrict to their $\leq N-1$ values as long as x_0 equals x_i for some $i = 1, \ldots, N-1$.

Since $K \geq N$, at some point, $x_0 \neq x_i$ for all $i = 1, \ldots, N-1$.

The next time p_0 becomes privileged, clearly $x_i = x_0$ for all $0 < i < N$.

So then mutual exclusion has been achieved.

Question

Let $N \geq 3$. Argue that Dijkstra's token ring self-stabilizes if $K = N - 1$.

This lower bound for K is sharp! (See the next slide.)

Answer: Consider any computation.

At some moment, p_{N-1} copies the value from p_{N-2}.

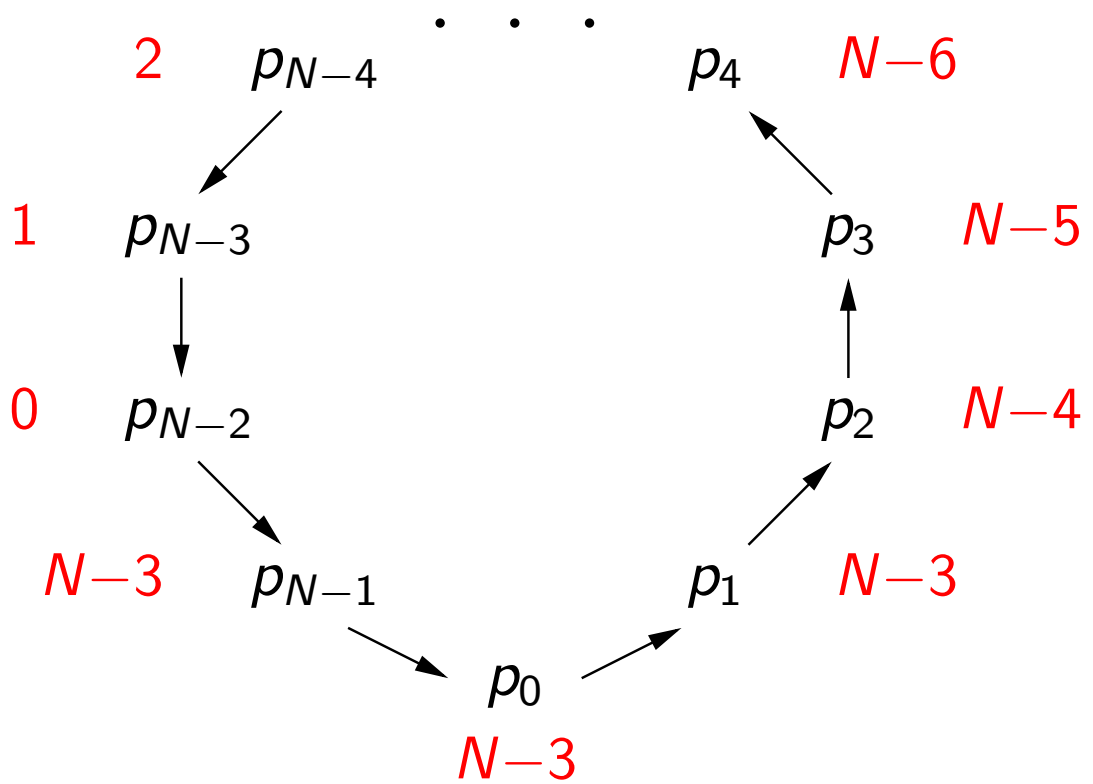
Then $p_1, \ldots, p_{N-1}$ hold $\leq N - 2$ different values (because $N \geq 3$).

Since $p_1, \ldots, p_{N-1}$ only copy values, they hold these $\leq N - 2$ values as long as x_0 equals x_i for some $i = 1, \ldots, N - 1$.

Since $K \geq N - 1$, at some point, $x_0 \neq x_i$ for all $i = 1, \ldots, N - 1$.

Dijkstra's token ring - Non-stabilization if $K = N - 2$

Example: Let $N \geq 4$ and $K = N - 2$, and consider the following initial configuration.



It doesn't always self-stabilize.

Afek-Kutten-Yung self-stabilizing spanning tree algorithm

We compute a spanning tree in an undirected network.

As always, each process is supposed to have a unique id.

The process with the largest id becomes the *root*.

Each process p maintains the following variables :

$parent_p$: its parent in the spanning tree

$root_p$: the root of the spanning tree

$dist_p$: its distance from the root via the spanning tree

Afek-Kutten-Yung spanning tree algorithm - Complications

Due to arbitrary initialization, there are three complications.

Complication 1: Multiple processes may consider themselves root.

Complication 2: There may be a cycle in the spanning tree.

Complication 3: $root_p$ may not be the id of any process in the network.

Afek-Kutten-Yung spanning tree algorithm

A *non-root* p declares itself *root*, i.e.

$$parent_p \leftarrow \perp \qquad root_p \leftarrow p \qquad dist_p \leftarrow 0$$

if it detects an inconsistency in its *root* or *parent* value, or with the *root* or *dist* value of its parent :

- $root_p \leq p$, or
- $parent_p = \perp$, or
- $parent_p \neq \perp$, and $parent_p$ isn't a neighbor of p or $root_p \neq root_{parent_p}$ or $dist_p \neq dist_{parent_p} + 1$.

Question

Suppose that during an application of the Afek-Kutten-Yung algorithm, the created directed network contains a cycle with a "false" root.

Why is such a cycle always broken ?

Answer: At some p on this cycle, $dist_p \neq dist_{parent_p} + 1$.

So p declares itself root.

Afek-Kutten-Yung spanning tree algorithm

A *root* p makes a neighbor q its parent if $p < root_q$:

$$parent_p \leftarrow q \qquad root_p \leftarrow root_q \qquad dist_p \leftarrow dist_q + 1$$

Complication: Processes can infinitely often rejoin a component with a false root.

Afek-Kutten-Yung spanning tree alg. - Example

Given two processes 0 and 1.

$parent_0 = 1 \quad parent_1 = 0 \quad root_0 = root_1 = 2 \quad dist_0 = 0 \quad dist_1 = 1$

Since $dist_0 \neq dist_1 + 1$, 0 declares itself *root*:

$parent_0 \leftarrow \perp \qquad root_0 \leftarrow 0 \qquad dist_0 \leftarrow 0$

Since $root_0 < root_1$, 0 makes 1 its *parent*:

$parent_0 \leftarrow 1 \qquad root_0 \leftarrow 2 \qquad dist_0 \leftarrow 2$

Since $dist_1 \neq dist_0 + 1$, 1 declares itself *root*:

$parent_1 \leftarrow \perp \qquad root_1 \leftarrow 1 \qquad dist_1 \leftarrow 0$

Since $root_1 < root_0$, 1 makes 0 its *parent*:

$parent_1 \leftarrow 0 \qquad root_1 \leftarrow 2 \qquad dist_1 \leftarrow 3$ et cetera

Afek-Kutten-Yung spanning tree alg. - Join Requests

Before p makes q its parent, it must wait until q's component has a proper root. Therefore p first sends a *join request* to q.

This request is forwarded through q's component, toward the root of this component.

Join requests are only forwarded between "consistent" processes.

The root sends back an *ack* toward p, which retraces the path of the request.

When p receives this ack, it makes q its *parent* :

$$parent_p \leftarrow q \qquad root_p \leftarrow root_q \qquad dist_p \leftarrow dist_q + 1$$

Afek-Kutten-Yung spanning tree alg. - Example

Given two processes 0 and 1.

$parent_0 = 1 \quad parent_1 = 0 \quad root_0 = root_1 = 2 \quad dist_0 = dist_1 = 0$

Since $dist_0 \neq dist_1 + 1$, 0 declares itself *root*:

$parent_0 \leftarrow \perp \qquad root_0 \leftarrow 0 \qquad dist_0 \leftarrow 0$

Since $root_0 < root_1$, 0 sends a *join request* to 1.

This join request doesn't immediately trigger an ack.

Since $dist_1 \neq dist_0 + 1$, 1 declares itself *root*:

$parent_1 \leftarrow \perp \qquad root_1 \leftarrow 1 \qquad dist_1 \leftarrow 0$

Since 1 is now a proper root, it replies to the join request of 0 with an ack, and 0 makes 1 its *parent*:

$parent_0 \leftarrow 1 \qquad root_0 \leftarrow 1 \qquad dist_0 \leftarrow 1$

Afek-Kutten-Yung spanning tree alg. - Shared memory

A process can only be forwarding and awaiting an ack for at most one join request at a time.

(That's why in the previous example 1 can't send 0's join request on to 0.)

Communication is performed using *shared memory*, so join requests and ack's are encoded in shared variables.

The path of a join request is remembered in local variables.

For simplicity, join requests are here presented in a message passing framework with synchronous communication.

Afek-Kutten-Yung spanning tree alg. - Consistency check

Given a ring with processes p, q, r, and $s > p, q, r$.

Initially, p and q consider themselves root; r has p as parent and considers s the root.

Since $root_r > q$, q sends a join request to r.

Without the consistency check, r would forward this join request to p.

Since p considers itself root, it would send back an ack to q (via r), and q would make r its parent and consider s the root.

Since $root_r \neq root_p$, r makes itself root.

Now we would have a symmetrical configuration to the initial one.

Afek-Kutten-Yung spanning tree alg. - Correctness

Each component in the network with a false root has an inconsistency, so a process in this component will declare itself root.

Since processes can only be involved in one join request at a time, each join request is eventually acknowledged.

Since join requests are only passed on between consistent processes, processes can only finitely often join a component with a false root (each time due to improper initial values of local variables).

These observations imply that eventually false roots will disappear, the process with the largest id in the network will declare itself root, and the network converges to a spanning tree with this process as root.

Lecture in a nutshell

mutual exclusion

Ricart-Agrawala algorithm with a logical clock

Raymond's algorithm with token passing

Agrawal-El Abbadi algorithm with quorums

self-stabilization

Dijkstra's self-stabilizing mutual exclusion algorithm

Afek-Kutten-Yung self-stabilizing spanning tree algorithm

Edsger W. Dijkstra prize in distributed computing

2000: Lamport, *Time, clocks, and the ordering of events in a distributed system*, 1978

2001: Fischer, Lynch, Paterson, *Impossibility of distributed consensus with one faulty process*, 1985

2002: Dijkstra, *Self-stabilizing systems in spite of distributed control*, 1974

2004: Gallager, Humblet, Spira, *A distributed algorithm for minimum-weight spanning trees*, 1983

2005: Pease, Shostak, Lamport, *Reaching agreement in the presence of faults*, 1980

2007: Dwork, Lynch, Stockmeyer, *Consensus in the presence of partial synchrony*, 1988

2010: Chandra, Toueg, *Unreliable failure detectors for reliable distributed systems*, 1996

2014: Chandy, Lamport, *Distributed snapshots: determining global states of distributed systems*, 1985

2015: Ben-Or, *Another advantage of free choice: completely asynchronous agreement protocols*, 1983

The 2003, 2006, 2012 award winners are treated in *Concurrency & Multithreading*.

A tutorial on the split binary semaphore.

The purpose of this note is threefold. It has been written to introduce the student to

1) the technique of the so-called "split binary semaphore" --originally discovered, but at the same time not recommended, by C.A.R.Hoare--

2) the use of formal techniques in the development of multiprograms

3) the now canonical example of "The readers and the writers", which the student must know anyhow; it will be used as a carrier for the other two purposes. The problem of the readers and the writers was designed by D.L. Parnas.

We consider two classes of cyclic processes, called "readers" and "writers" respectively. With "ncs" standing for "non-critical section", they can be described by the programs

reader: do true → ncs; READ od (1)

writer: do true → ncs; WRITE od

respectively. Here "READ" and "WRITE" denote their respective critical sections, critical in the sense that when a writer is engaged in its critical section, it must be the only process engaged in its critical section. This problem can be solved in many different ways --the problem is canonical in the sense that everybody proposing new synchronization primitives has solved it in his way-- , we shall now solve it using a split binary semaphore.

Our first step is the introduction of variables, in terms of which we express our synchronization requirement formally; we call them "ar" and "aw" --short for "number of active readers" and "number of active writers" respectively-- and consider the following multiprogram:

initial state: ar = 0, aw = 0, mutex = 1 (2)

```
reader:  do true → ncs;                                        [cont.(2)]
                   P(mutex); ar:= ar + 1; V(mutex);
                   READ;
                   P(mutex); ar:= ar - 1; V(mutex)
         od
writer:  do true → ncs;
                   P(mutex); aw:= aw + 1; V(mutex);
                   WRITE;
                   P(mutex); aw:= aw - 1; V(mutex)
         od
```

For multiprogram (2) the invariance of

P0: $ar \geq 0$ and $aw \geq 0$

follows immediately in the usual way "from the topology of the program". (Associate with $reader_i$ an additional variable c_i, initially = 0, and satisfying $0 \leq c_i \leq 1$; replace $ar := ar + 1$ by $c_i, ar := 1, ar + 1$ and $ar := ar - 1$ by $c_i, ar := 0, ar - 1$ and observe the invariance of $ar = \sum c_i$, etc.)

Note. Looking at program (2) and interpreting it operationally, one might be tempted to say: clearly $ar \geq$ the number of readers engaged in READ, and as "a number of readers" is never negative, $ar \geq 0$ follows immediately, and similarly for $aw \geq 0$. I prefer not to do so, to prove the invariance of P0 from the uninterpreted text (2), and to conclude from P0 that it does not prohibit the interpretation of ar and aw as natural numbers. (End of Note.)

In terms of ar and aw I propose the required invariance of

P1: $aw = 0$ or ($aw = 1$ and $ar = 0$)

as a suitable formal expression of our operational requirement "when a writer is engaged in its critical section, it must be the only process engaged in its critical section". The term $aw = 0$ is intended to cover the situation when no writer is engaged in its critical section, the term $aw = 1$ and $ar = 0$

should cover the case when a writer is engaged in its critical section.

Note. The attentive reader will already have decided that he should not expect more eloquence from me on the "suitability" of the proposal captured by P1 . (End of Note.)

In order to prevent ar:= ar + 1 from violating P1 we can make it into a guarded command of the form

wp("ar:= ar + 1", P1) → ar:= ar + 1 .

The axiom of assignment gives for this guard

aw = 0 or (aw = 1 and ar + 1 = 0)

but on account of P0 the second term is false, and we simplify

aw = 0 → ar:= ar + 1 .

In order to prevent aw:= aw + 1 from violating P1 we consider

wp("aw:= aw + 1", P1) → aw:= aw + 1 .

The axiom of assignment gives for this guard

aw + 1 = 0 or (aw + 1 = 1 and ar = 0)

but, again, P0 admits simplification of this guard --its first term is false-- and we find

aw = 0 and ar = 0 → aw:= aw + 1 .

The decrease ar:= ar - 1 is similarly guarded

wp("ar:= ar - 1", P1) → ar:= ar - 1 ,

and we get for the guard with the axiom of assignment

aw = 0 or (aw = 1 and ar - 1 = 0) ,

which, thanks to P1 ' can be simplified as

aw = 0 → ar:= ar - 1 .

The known invariance of P0 tells us that the precondition of ar:= ar - 1

implies wp("ar:= ar - 1", P0) , i.e. implies ar - 1 $\geq$ 0 ; on account of P1 this implies aw = 0 , and, therefore, we can simplify to

true → ar:= ar - 1

i.e. the decrease ar:= ar - 1 need not be guarded at all.

The verification that also aw:= aw - 1 need not be guarded is left to the reader.

Inserting the guards as derived we get

initial state: ar = 0, aw = 0, mutex = 1 (3)

```
reader:  do true → ncs;
§                  P(mutex); if aw = 0 → ar:= ar + 1 fi; V(mutex);
                   READ;
                   P(mutex); ar:= ar - 1; V(mutex)
         od

writer:  do true → ncs;
§§                 P(mutex); if aw = 0 and ar = 0 → aw:= aw + 1 fi; V(mutex);
                   WRITE;
                   P(mutex); aw:= aw - 1; V(mutex)
         od
```

Multiprogram (3) has been designed so as to leave P0 and P1 invariant, and that is fine. It has, however, a major drawback: both alternative constructs, in the lines marked § and §§ respectively, may lead to abortion! The so-called split binary semaphore provides us with a technique for preventing the selection of the critical sections marked § and §§ respectively under those circumstances in which they would lead to abortion.

We replace the single binary semaphore mutex by three, also binary, semaphores -- m , r , and w say-- , related to the original mutex by

mutex = m + r + w ,

and replace in multiprogram (3) each P(mutex) by P(m), P(r), or P(w)

--the three ways of decreasing mutex by 1-- and each V(mutex) by V(m), V(r), or V(w) --the three ways of increasing mutex by 1-- . (Because m, r, and w are semaphores, we thus guarantee $0 \leq \text{mutex} \leq 1$.) More precisely: we replace the P(mutex) marked by § by P(r) , replace the P(mutex) marked by §§ by P(w) , and the P(mutex) that opens a critical section that cannot lead to abortion by P(m) .

We can now avoid selection of an aborting critical section by guarding V(r) by aw = 0 and by guarding V(w) by aw = 0 and ar = 0 , because the precondition of a V-operation on a component of a split binary semaphore can be taken as the postcondition of the corresponding P-operation. Our analysis so far would suggest that it suffices to replace V(mutex) everytime by

Q: **if** true → V(m) [] aw = 0 → V(r) [] aw = 0 **and** ar = 0 → V(w) **fi** (4)

This, however, is too naive. To start with: how do we translate the initialization mutex = 1 ? The initialization m = 0, r = 1, w = 0 is too restrictive: if all readers remain in their ncs , no writer could perform WRITE. The initialization m = 0 , r = 0, w = 1 has to be rejected on similar grounds. In order to make the only remaining possible initialization m = 1, r = 0, w = 0 acceptable, readers and writers should encounter as first P-operation one that cannot lead to abortion. We can satisfy this requirement by inserting in both readers and writers of (3) after ncs

P(mutex); V(mutex);

before performing the substitution described above. This would lead to the following multiprogram

initial state: ar = 0, aw = 0, m = 1, r = 0, w = 0 (5)

```
reader: do true → ncs; P(m); Q;
                  P(r) {aw = 0}; ar:= ar + 1; Q;
                  READ;
                  P(m); ar:= ar - 1; Q
        od
```

```
writer:  do true → ncs; P(m); Q;                                   [cont.(5)]
                   P(w) {aw = 0 and ar = 0}; aw:= aw + 1; Q;
                   WRITE;
                   P(m); aw:= aw - 1; Q
         od
```

Multiprogram (5) is, however, still too naive: the non-determinacy, that has been introduced by Q as given in (4), has lead to a system with the danger of deadlock. The recipe for its prevention is, however, universal.

1) At initialization and at each V-operation, the "type of the next P-operation" --i.e. the component of the split binary semaphore on which this program will perform its next P-operation-- must be uniquely defined.
Our program (5) satisfies this condition, as do all programs without initial non-determinacy nor non-determinacy between a V-operation and the dynamically next P-operation. If a program does not satisfy this condition we can always make it satisfying it by introducing one or more extra components of the split binary semaphore, and by replacing essentially

```
if true → P(component 1);...
 [] true → P(component 2);...
fi
```

by

```
if true → P(extra component); Q; P(component 1);...
 [] true → P(extra component); Q; P(component 2);...
fi
```

Task. Prove that for this substitution process only a finite number of different extra components is needed. (End of Task.)

2) With each component of the binary semaphore we associate a counter, initialized to the number of processes for which the first P-operation is of the corresponding type.

3) After each P-operation we insert a decrease by 1 of the counter associated with its type.

4) Before each Q we insert an increase by 1 of the counter associated with the type of the dynamically next P-operation. (Thanks to step 1, this is a well-defined counter.)

Note. For the operationally minded: each counter can be interpreted as the number of processes "ready" or "headed" for a P-operation on the corresponding component. (End of Note.)

5) Strengthen in Q the guarding of each V(component) by the requirement that the corresponding counter is positive.

Associating with the semaphore components m, r, and w the counters bm, br, and bw respectively, carrying out the above transformations on program (5) leads to

initial state: ar = 0, aw = 0, bm = number of processes, br = 0, bw = 0 (6)
m = 1, r = 0, w = 0

```
reader: do true → ncs;
                  P(m); bm:= bm - 1; br:= br + 1; Q;
                  P(r); br:= br - 1; ar:= ar + 1; bm:= bm + 1; Q;
                  READ;
                  P(m); bm:= bm - 1; ar:= ar - 1; bm:= bm + 1; Q
        od

writer: do true → ncs;
                  P(m); bm:= bm - 1; bw:= bw + 1; Q;
                  P(w); bw:= bw - 1; aw:= aw + 1; bm:= bm + 1; Q;
                  WRITE;
                  P(m); bm:= bm - 1; aw:= aw - 1; bm:= bm + 1; Q
        od
```

with Q short for

```
Q:    if bm > 0 → V(m)                                          (6')
      [] aw = 0 and br > 0 → V(r)
      [] aw = 0 and ar = 0 and bw > 0 → V(w)
      fi
```

Note. The transformation of the introduction of the counters and of the strengthening of the guards in Q by the requirement that the corresponding counters be positive, excludes the danger of deadlock. If the original requirement --in our case: the invariance of P1-- entailed intrinsically the danger of deadlock, this danger is now made manifest by the danger of abortion in Q . The systematic procedure for dealing with that situation falls outside the scope of this tutorial. (End of Note.)

Our original system was free from deadlock, hence we must --see above note-- be able to prove the absence of the danger of abortion in Q . The precondition of Q implies everywhere in (6)

$$bm + br + bw = \text{number of readers} + \text{number of writers}$$
$$ar + br \leq \text{number of readers}$$
$$aw + bw \leq \text{number of writers}$$

Task. Verify the above three assertions. (End of Task.)

From the above we conclude

$$ar + aw \leq bm \quad , \text{ and hence}$$
$$bm > 0 \ \underline{\text{or}} \ (ar = 0 \ \underline{\text{and}} \ aw = 0) \quad .$$

Assuming the number of processes to be larger than zero --otherwise the danger of abortion is absent anyhow!-- , we also have

$$bm > 0 \ \underline{\text{or}} \ br > 0 \ \underline{\text{or}} \ bw > 0$$

and from the last two relations it follows that at least one of the guards of Q as given in (6') is true.

* * *

The above form of Q as given in (6') is still very non-deterministic: we have the strategic freedom of strengthening the guards as long as we avoid the danger of abortion. As it stands, our solution does not exclude that readers or a writer are denied access to their critical section READ or WRITE without reason. Within Q we can give "priority" to V(r) or V(w) by strengthening the guard of V(m) to the conjunction of the complements of the other two guards:

Am: $(aw > 0 \text{ or } br = 0) \text{ and } (aw > 0 \text{ or } ar > 0 \text{ or } bw = 0)$

Denoting by Ar and Aw respectively:

Ar: $aw = 0 \text{ and } br > 0$

Aw: $aw = 0 \text{ and } ar = 0 \text{ and } bw > 0$

we can now substitute for Q in (6)

Q: $\text{if } Am \rightarrow V(m) \;[]\; Ar \rightarrow V(r) \;[]\; Aw \rightarrow V(w) \text{ fi}$. (7')

All (now superfluous) operations on bm can be omitted; using the post-conditions of the P-operations, the substitution instances of Q as given by (7') can be simplified. Thus we derive from (6) the multiprogram

initial state: ar = 0, aw = 0, br = 0, bw = 0, m = 1, r = 0, w = 0 (7)

reader:

```
do true → ncs;
    P(m); br:= br + 1; if aw > 0 → V(m) [] aw = 0 → V(r) fi;
    P(r); br, ar := br - 1, ar + 1; if br = 0 → V(m) [] br > 0 → V(r) fi;
    READ;
    P(m); ar:= ar - 1; if ar > 0 or bw = 0 → V(m)
                       [] ar = 0 and bw > 0 → V(w)
                       fi
od
```

writer:

```
do true → ncs;
    P(m); bw:= bw + 1; if aw > 0 or ar > 0 → V(m)
                       [] aw = 0 and ar = 0 → V(w)
                       fi;
    P(w); bw, aw := bw - 1, aw + 1; V(m);
    WRITE;
    P(m); aw:= aw - 1; if br = 0 and bw = 0 → V(m)
                       [] br > 0 → V(r)
                       [] bw > 0 → V(w)
                       fi
od
```

Remark. An inspection of the alternative constructs in (7) shows that only the very last one is non-deterministic: here we have, therefore, still a strategic freedom. Investigate the consequences of replacing the last guarded command $bw > 0 \rightarrow V(w)$ by $bw > 0$ and $br = 0 \rightarrow V(w)$. (End of Remark.)

* * *

In the above we have derived our final program as the end of a sequence of successive versions. We have done so for educational reasons, with the intent of introducing the different aspects of programs synchronized with a split binary semaphore one after the other. This is not meant as a suggestion that in the case of actual program design one should write down all those successive versions! The experienced programmer knows that "outside the critical sections" as delineated by the P - V pairs, we have an invariant of the form

$$(m = 0 \text{ or } Am) \text{ and } (r = 0 \text{ or } Ar) \text{ and } (w = 0 \text{ or } Aw)$$

and focusses his attention on the A's; having chosen them he derives the final code. I make this remark because so-called "program transformations" are sometimes suggested as a practical way of program derivation --not by me, for as a rule it leads to an undue amount of writing-- .

5th March 1979
Plataanstraat 5
5671 AL NUENEN
The Netherlands

prof.dr.Edsger W.Dijkstra
Burroughs Research Fellow

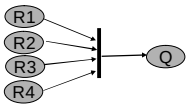
Lesson 7

Semaphore Use In Synchronization

Ch 6 [BenA 06]

Consumer Producer Revisited
Readers and Writers
Baton Passing
Private Semaphores
Resource Management

Synchronization with Semaphores




```
sem gate = -3;   # must know number of R's (!)

Process R[i = 1 to 4]
....
V(gate);  # signal Q
...

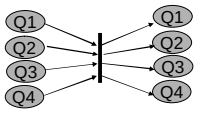
Process Q
....
P (gate)
...
# how to prepare for next time?
# sem_set (gate, -3) ??
```

```
sem g[i = 1 to 4] = 0;

Process R[i = 1 to 4]
....
V(g[i]);  # signal Q
...

Process Q
....
P(g[1]); P(g[2]); P(g[3]); P(g[4]);
...
# Q must know number of R's
```


Barrier Synchronization with Semaphores



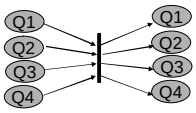

```
sem g[i = 1 to 4] = 0;
    cont = 0;

Process Q[i = 1 to 4]
....
V(g[i]);  # signal others
P(cont); # wait for others
...

Process Barrier
....
P(g[1]); P(g[2]); P(g[3]); P(g[4]);  #wait for all
V(cont); V(cont); V(cont); V(cont); #signal all
...
# Barrier must know number of Q's
```

- Barrier is implemented as separate *process*
  - This is just one possibility to implement the barrier
  - Cost of process switches?
  - How many process switches?

Barrier Synchronization with Barrier OS-Primitive



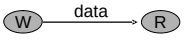
- Specific synchronization primitive in OS
  - Implemented with semafores…
  - No need for extra process – less process switches

```
barrier br;

barrier_init (br, 4);      # must be done before use

process Q[i = 1 to 4]
  ....
  barrier_wait (br)        # wait until all have reached this point
  if (pid==1)              # is this ok? is this done in time?
        barrier_init (br, 4)
  ...
```


Communication with Semaphores




```
Sem mutex=1, data_ready = 0;
Int buffer;  # one data item buffer

Process W
....
P(mutex)
   write_buffer(data)
V(mutex)
V(data_ready);  # signal Q
...

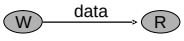
Process R
....
P(data_ready);  # wait for data
P(mutex)
   read_buffer(data)
V(mutex)
....
```

- What is wrong?

W might rewrite data buffer before R reads it

  - Might have extra knowledge to avoid the problem

Communication with Semaphores Correctly




```
Sem mutex=1, data_ready = 0, buffer_empty=1;
Int buffer

Process W
....
P(buffer_empty);
P(mutex)
   write_buffer(data)
V(mutex)
V(data_ready);  # signal Q
...

Process R
....
P(data_ready);  # wait for data
P(mutex)
   read_buffer(data)
V(mutex)
V(buffer_empty)
```

- Fast W can not overtake R now
- One reader R, one writer W, binary semaphores
- Communication with buffer in shared memory
  - Use: 1 producer – 1 consumer – size 1 buffer

Producer-Consumer with Binary Semaphores

(Liisa Marttinen)

- Binary semaphore has values 0 and 1
  - OS or programming language library
- Semaphore does not keep count
  - Must have own variable *count* (nr of elements in buffer)
    - Protect it with critical section — mutex
- Two important state changes
  - Empty buffer becomes not empty
    - Consumer may need to be awakened — items
  - Full buffer becomes not full
    - Producer may need to be awakened — space

Simple Solution #1

(Producer-Consumer with Binary Semaphores)

```
typeT buf[n];          /* n element buffer */
int front=0,           /* read from here */
    rear=0,            /* write to this one */
    count=0;           /* nr of items in buf */
sem space=1,           /* need this to write */
    items=0,           /* need this to read */
    mutex=1;           /* need this to update count */
```


Sol. #1

```
process Producer [i=1 to M] {
while(true) {
   ... produce data …
   P(space);      /* wait until space to write*/
   P(mutex);
     buf[rear] = data; rear = (rear+1) %n; count++;
     if (count == 1) V(items);  /* first item to empty buffer */
     if (count < n) V(space);   /* still room for next producer */
   V(mutex);
   }
}
```

```
process Consumer [i=1 to N] {
while(true) {
    P(items);       /* wait until items to consume */
    P(mutex);
      data=buf[front]; front = (front+1) %n; count--;
      if (count == n-1) V(space);  /* buffer was full */
      if (count > 0) V(items);    /* still items for next consumer */
    V(mutex);
     ... consume data …
    }
}
```


Evaluate Solution #1

- Simple solution
  - Mutex and synchronization ok
  - Mutex inside space or items
    - Get space first and then mutex
- Buffer reserved for one producer/consumer at a time
  - Does not allow for simultaneous buffer use — Not good
    - Producer inserts item to "rear"
    - Consumer removes item from "front" — Simulta-neously?
- First waiting producer/consumer advances when signalled
  - Queued in semaphores

Better Solution #2

(Producer-Consumer with Binary Semaphores)

```
typeT buf[n];          /* n element buffer */
int front=0,           /* read from here */
    rear=0,            /* write to this one */
    count=0;           /* nr of items in buf */
sem space=1,           /* need this to write */
    items=0,           /* need this to read */
    mutex=1;           /* need this to update count */
```


Sol. #2

```
process Producer [i=1 to M] {
while(true) {
   ... produce data …
   P(space);      /* wait until space to write*/
   buf[rear] = data; rear = (rear+1) %n; /* outside mutex, ok? */
   P(mutex);
     count++;                    /* this must be in mutex */
     if (count == 1) V(items);  /* first item to empty buffer */
     if (count < n) V(space);   /* still room for next producer */
   V(mutex);
} }
```

```
process Consumer [i=1 to N] {
while(true) {
    P(items);       /* wait until items to consume */
    data=buf[front]; front = (front+1) %n; /* outside mutex, ok? */
    P(mutex);
      count--;
      if (count == n-1) V(space);  /* buffer was full */
      if (count > 0) V(items);  /* still items for next consumer */
    V(mutex);
     ... consume data …
} }
```

Evaluate Solution #2

- Relatively simple solution
  - Data copying (insert, remove) outside critical section
    - Protected by a semaphore (*items* and *space*)
- Simultaneous insert and remove ops
  - Producer inserts item to "rear"
  - Consumer removes item from "front"
- First waiting producer/consumer advances when signalled
  - Queued in semaphores

Another Solution #3

(Producer-Consumer with Binary Semaphores)

Ehto-synkro-nointi

- Use condition synchronization
  - Do P(space) or P(items) only when needed
    - Expensive op?
    - Requires execution state change (kernel/user)?

```
typeT buf[n];      /* n element buffer */
int front=0,       /* read from here */
    rear=0,        /* write to this one */
    count=0,       /* nr of items in buf */
    cwp=0,         /* nr of waiting producers */
    cwc=0;         /* nr of waiting consumers */
sem space=1,       /* need this to write */
    items=0,       /* need this to read */
    mutex=1;       /* need this to update count */
```


Sol. #3

```
process Producer [i=1 to M] {
while(true) {
   ... produce data …
    P(mutex);
      while (count == n)   /* usually not true? while, not if !*/
           { cwp++; V(mutex); P(space); P(mutex); cwp-- }
      buf[rear] = data; rear = (rear+1) %n;  count++;
      if (count == 1 && cwc>0) V(items);
      if (count < n && cwp>0) V(space);
   V(mutex);
} }
```

```
process Consumer [i=1 to N] {
while(true) {
    P(mutex);
      while (count == n)    /* while, not if !*/
           { cwc++; V(mutex); P(items); P(mutex); cwc-- }
      data=buf[front]; front = (front+1) %n; count--;
      if (count == n-1 && cwp>0) V(space);
      if (count > 0 && cwc > 0) V(items);
    V(mutex);
     ... consume data …
} }
```


Evaluate Solution #3

- No simultaneous insert and remove ops
  - Data copying inside critical section
- In general case, only mutex semaphore operations needed
  - Most of the time?
  - Can they be busy-wait semaphores?
- First waiting producer/consumer does not necessarily advance when signalled
  - Someone else may get mutex first
    - E.g., consumer signals (Vspace), another producer gets mutex and places its data in buffer.
  - Need "while" loop in waiting code
  - Unfair solution even with strong semaphores?
    - How to fix?
    - Baton passing (pass critical section to next process)?

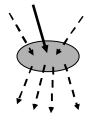
Solutions #1, #2, and #3

- Which one is best? Why? When?
- How to maximise concurrency?
  - Separate data transfer (insert, remove) from permission to do it
    - Allow obtaining permission
      (e.g., code with *P(space)* and updating *count*)
      for one process run concurrently with data transfer for another process
      (e.g., code with *buf[rear] = data; …*)
    - Need new mutexes to protect data transfers and index (*rear, front*) manipulation
  - Problem: signalling to other producers/consumers should happen in same critical section with updating count, but should happen only after data transfer is completed

Readers and Writers Problem

- Shared data structure or data base
- Two types of users: readers and writers
- Readers
  - Many can read at the same time
  - Can not write when someone reads
  - Can not read when someone writes
- Writers
  - Read and modify data
  - Only one can be active at the same time
  - Can be active only when there are no readers





Jeff Magee example (Imperial College, London)

http://www.doc.ic.ac.uk/~jnm/book/book_applets/ReadersWriters.html


```
sem rw = 1;
process Reader[i = 1 to M] {
  while (true) {
    ...
    P(rw);     # grab exclusive access lock
    read the database;
    V(rw);     # release the lock
  }
}
process Writer[j = 1 to N] {
  while (true) {
    ...
    P(rw);     # grab exclusive access lock
    write the database;
    V(rw);     # release the lock
  }
}
```

reader entry protocol

reader exit protocol

writer entry protocol

writer exit protocol

(Fig 4.8 [Andr00])

- Simple solution
  - Only one reader or writer at a time (not good)


```
int nr = 0;       # number of active readers
sem rw = 1;       # lock for reader/writer synchronization
process Reader[i = 1 to M] {
  while (true) {
    ...
    ⟨ nr = nr+1;
      if (nr == 1) P(rw);  # if first, get lock ⟩
    read the database;
    ⟨ nr = nr-1;
      if (nr == 0) V(rw);  # if last, release lock ⟩
  }
}
process Writer[j = 1 to N] {
  while (true) {
    ...
    P(rw);
    write the database;
    V(rw);
  }
}
```

std mutex

Only the first reader waits

Release mutex before P(rw)? (no need)

Writers may starve – not good.
Writers have no chance to cut in between readers.

Jeff Magee example

How should you adjust the readers to starve writers?

(Fig 4.9 [Andr00])

Readers and Writers with Baton Passing Split Binary Semaphore

- Component semaphores e, r, w — 0 ≤ e+r+w ≤1
  - Mutex wait in P(e), initially 1
  - Readers wait in P(r) if needed, initially 0 (Fig 4.13 [Andr00])
  - Writers wait in P(w) if needed, initially 0 (Alg. 6.21 [BenA06])
- In critical control areas only one process advances at a time
  - Wait in e, r, or w — P(e) … V(e)
- One advances, others wait in e, r or w
  - New reader/writer: wait in P(e) — P(e) … V(r)
  - Waiting for read turn: V(e); P(r) — P(e) … V(w)
    - Wait while not holding mutex — P(r) … V(r)
  - Waiting for write turn: V(e); P(w) — P(r) … V(w)
    - Wait while not holding mutex …
  - When done, pass the baton (turn) to next one — P(w) … V(e) …


```
int nr = 0,   ## RW: (nr == 0 or nw == 0) and nw <= 1
    nw = 0;
sem e = 1,    # controls entry to critical sections
    r = 0,    # used to delay readers
    w = 0;    # used to delay writers
              # at all times 0 <= (e+r+w) <= 1
int dr = 0,   # number of delayed readers
    dw = 0;   # number of delayed writers

process Reader[i = 1 to M] {
  while (true) {
    # ⟨await (nw == 0) nr = nr+1;⟩
      P(e);
      if (nw > 0)
        { dr = dr+1; V(e); P(r); }
      nr = nr+1;
      SIGNAL;
    read the database;
    # ⟨nr = nr-1;⟩
      P(e);
      nr = nr-1;
      SIGNAL;
  }
}

process Writer[j = 1 to N] {
  while (true) {
    # ⟨await (nr==0 and nw==0) nw = nw+1;⟩
      P(e);
      if (nr > 0 or nw > 0)
        { dw = dw+1; V(e); P(w); }
      nw = nw+1;
      SIGNAL;
    write the database;
    # ⟨nw = nw-1;⟩
      P(e);
      nw = nw-1;
      SIGNAL;
  }
}
```

Andrews Fig. 4.12: Outline of readers and writers with passing the baton.

(rStart) (rExit) (wStart) (wExit)

P(e) V(w) ? P(r) V(w) ? P(r) … V(e) ? P(e) … V(e) ? P(e) … V(w) ?

Baton passing = "do not just release CS, give it to someone special…"

Baton passing

- When done your own mutex zone, wake up next … (one or more semaphores control the same mutex)

SIGNAL()

- –If reader waiting and no writers: V(r)
  - Do not release mutex (currently reserved e, r, or w)
  - New reader will continue with mutex already locked "pass the mutex baton to next reader"
    - – No one else can come to mutex zone in between
  - Last waiting reader will close the mutex with V(e)
  - Can happen concurrently when reading database
- –Else if writer waiting and no readers: V(w)
  - Do not release mutex, pass baton to writer
- –Else (let new process to compete with old ones): V(e)
  - Release mutex to let new process in the game (to execute entry or exit protocols)
  - New process gets in mutex only when no old one can be advance
  - Can happen concurrently when reading database

Baton Passing with SIGNAL

SIGNAL – CS baton passing, priority on readers

```
if (nw == 0  and  dr > 0) {
   dr = dr -1;
   V(r);                  # wake up waiting reader
   }
else if  (nr == 0  and nw == 0  and  dw > 0) {
   dw = dw -1;
   V(w);                  # wake up waiting writer
   }
else
   V(e);                  # let new process to mix
```

"pass the baton within CS"

"just complete CS"

not possible in wStart, rExit

not possible in rStart

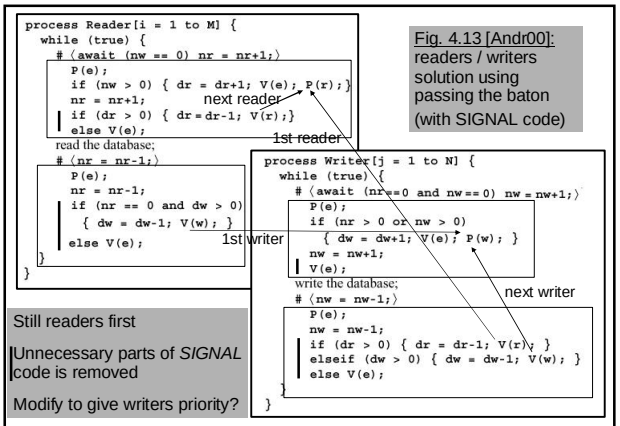


Fig. 4.13 [Andr00]: readers / writers solution using passing the baton (with SIGNAL code)

Still readers first

Unnecessary parts of *SIGNAL* code is removed

Modify to give writers priority?

Resource Management

- Problem
  - Many types of resources
  - N units of given resource
  - Request allocation: K units
    - Wait suspended until resource available
- Solution
  - Semaphore mutex (init 1)
  - Semaphore Xavail
    - init N – wait for available resource
    - init 0 - wait for permission to continue

use printer
use webcam
access database
access CS
allocate memory
allocate buffer
use comm port
get user focus
etc. etc.

Simple Bad Solution

```
sem Xmutex = 1, Xavail = N

Xres_request ()  # one unit at a time
   P(Xmutex)
   P(Xavail)  # ok if always
              # allocate just 1 unit
   take 1 unit  # not simple,
                # may take long time?
   V(Xmutex);

Xres_release ()
   P(Xmutex)
   return 1 unit
   V(Xavail);
   V(Xmutex);
```

- What is wrong?
  - everything
- Mutex?
- Deadlock?
- Unnecessary delays?
  - Each P() may result in (long) delay?
  - Hold mutex while waiting for resource
    - Very, very bad
    - Others can not get mutex to release resources…

Another Not So Good Solution

```
sem Xmutex = 1, Xavail = N

Xres_request ()   # one unit at a time
   P(Xavail)      # ok if always
                  # allocate just 1 unit
   P(Xmutex)
    take 1 unit   # not simple,
                  # may take long time?
   V(Xmutex);

Xres_release ()
   P(Xmutex)
         return 1 unit
   V(Xmutex);
   V(Xavail);
```

- What is wrong?
  - Works only for resources allocated and freed one unit at a time
- Mutex?
  - Mutex of control data?
  - Mutex of resource allocation data structures?

Resource Management with Baton Passing Split Semaphore

```
sem Xmutex = 1, Xavail = 0 (not N)  ; split semaphore
    ; (short wait)  (long wait)
Xres_request (K) – request K units of given resource
   P(Xmutex)
   if "not enough free units" {V(Xmutex); P(Xavail);}
   take K units    ; assume short time
   if "requests pending and enough free units"
   else V(Xmutex);

Xres_release (K)
   P(Xmutex)
   return K units
   if "requests pending and enough free units" {V(Xavail);}
   else V(Xmutex);
```

CS

CS

baton passing {V(Xavail); }

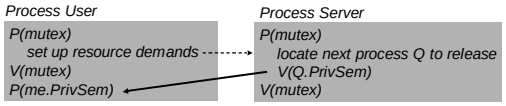
baton passing

Problems with Resource Management

- Need strong semaphores
- Strong semaphores are FIFO
  - What if 1st in line want 6 units, 2nd wants 3 units, and there are 4 units left?
  - What about priorities?
    - Each priority class has its own semaphore
    - Baton passing within each priority class?
  - How to release just some specific process?
    - Strong semaphore releases 1st in line
    - Answer: private semaphores

Private Semaphore

- Semaphore, to which only one process can ever make a P-operation
  - Initialized to 0, belongs to that process
- Usually part of PCB (process control block) for the process
  - Can create own semaphore arrays for this purpose
- Process makes demands, and then waits in private semaphore for turn
- Most often just one process at a time
  - Usually P(mutex) does not lead to process switches
- Usually still need to wait in private semaphore



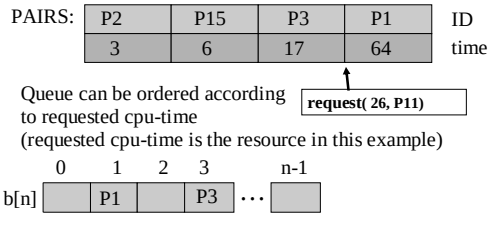
Shortest Job Next (Private Semaphore Use Example)

- Common resource allocation method
  - Here: *time = amount of resource requested*
  - Here: just select next job (with shortest time)
  - Here: just one job (at most) holding the resource at a time
- Use private semaphores

```
request(time,id): # requested time, user id
   P(e);
   if (!free) DELAY();  # wait for your turn
   free = false;        # got it!
   V(e);                # not SIGNAL(), only 1 at a time
```

```
release():
   P(e);
   free = true;
   SIGNAL();    # who gets the next one?
                # pass baton, or release mutex
```


- DELAY:
  - Place delayed process in queue PAIRS (ordered in ascending requested resource amount order) in correct place
  - V(e) – release mutex
  - Wait for your turn in private semaphore P(b[ID])
    - Each process has private semaphore, where only that process waits (initial value 0)
    - PAIRS queue determines order, one always wakes up the process at the head of the queue
      - Priority: smallest resource request first
- SIGNAL (in Release)
  - If someone waiting, take first one (time, ID), and wake up that process: V(b[ID]);
  - o/w V(e)



Private semaphore b[ID] for each process ID: 0 ..n-1

Process release is dependent on its location in PAIRS. When resource becomes free, the 1st process in line may advance.


```
bool free = true;
sem e = 1, b[n] = ([n] 0);  # for entry and delay
typedef Pairs = set of (int, int);
Pairs pairs = ∅;
## SIN:  pairs is an ordered set ∧ free ⇒ (pairs == ∅)
request(time,id):
  P(e);
  if (!free) {
    insert (time,id) in pairs;
    V(e);            # release entry lock
    P(b[id]);        # wait to be awakened
  }
  free = false;
  V(e);    # optimized since free is false here
release():
  P(e);
  free = true;
  if (P != ∅) {
    remove first pair (time,id) from pairs;
    V(b[id]);  # pass baton to process id
  }
  else V(e);
```

CS

CS

Andr00 Fig. 4.14 Shortest job next (cpu scheduling policy) allocation using semaphores.

Semaphore Feature Summary

- Many implementations and semantics
  - Be careful to use
  - E.g., is the (process) scheduler called after each V()?
    - Which one continues with processor, the process executing V() or the process just woken up?
  - Busy wait vs. suspend state?
- Hand coded synchronization solutions
  - Can solve almost any synchronization problem
  - Baton passing is useful and tricky
    - Explicit handover of some resource
  - Be careful to use
    - Do not leave mutex'es open
    - Do not suspend inside mutex
    - Avoid deadlocks
    - Do (multiple) P's and V's in correct order

16.11.2008 Copyright Teemu Kerola 2007 37

Capitolul 1 Introducere

Domeniul algoritmilor paraleli şi distribuiți preocupă de mai mult timp un număr mare de specialişti. Despre el, ca de altfel despre algoritmi în general, s-a scris mult şi din diferite puncte de vedere. Un efort considerabil a fost depus pentru găsirea celor mai bune soluţii paralele sau distribuite ale diverselor probleme, prin adaptarea unor soluţii secvenţiale cunoscute sau pornind de la premise conceptuale total distincte de acestea. A doua abordare ține cont de diferențele dintre calculul secvențial și calculul paralel, respectiv distribuit. Dacă în primul caz instrucţiunile programului sunt executate una după alta într-un acelasi sistem, în calculul paralel mai multe părți ale unui program sunt executate simultan în procesoare diferite care comunica prin memorie comună, iar în calculul distribuit diferitele părți ale programului sunt executate în sisteme diferite care comunică doar prin transmitere de mesaje prin rețeaua care leagă între ele aceste sisteme. Algoritmii paraleli și distribuiți au la bază modele diferite de calculul secvențial (cel mai folosit este modelul (*Communicating Sequential Processes* dezvoltat de *Hoare*) care includ reprezentări ale concurenței, atomicizării şi sincronizării diferitelor părți ale unui program. În plus, ei folosesc modele de programare care reprezintă comunicarea între procese prin date partajate sau prin mesaje. Având în vedere aceste aspecte, este uşor de înţeles că algoritmii paraleli si distributi **NU sunt** simple extensii sau versiuni ale celor secventiali, pentru dezvoltarea lor fiind folosite alte **abordari**.

Lucrarea de faţă vizează studierea sistematică a algoritmilor paraleli şi distribuiți pentru câteva din cele mai reprezentative clase de aplicaţii. Aceasta presupune adoptarea unui limbaj de descriere adecvat şi utilizarea lui pentru prezentarea şi analiza celor mai cunoscute soluţii ale claselor de probleme selectate. Limbajul pseudocod pe care il folosim, inspirat de propunerea lui G.R.Andrews [???]. este **simplu**, ascunde detaliile de implementare pe care le gasim inevitabil in limbajele de programare sau bibliotecile specializate, dar incorporeaza conceptele si modelele de programare de care avem nevoie.

Descrierile algoritmilor sunt acompaniate de discutarea unor aspecte relative la metodele de dezvoltare a algoritmilor paraleli şi distribuiţi, precum şi la analiza complexităţii lor. Demersul nostru ajută la deprinderea metodelor de proiectare (ingineria algoritmilor paraleli și distribuiți) și stimuleaza inițiativa în găsirea unor soluții noi.

Dacă în urmă cu câțiva ani studierea algoritmilor paraleli și distribuiți era apanajul

unui grup relativ restrâns de specialişti, situaţia actuală este cu totul diferită. Modelele şi tiparele de rezolvare paralelă şi distribuită sunt folosite curent, unele pe scara largă. Un exemplu este modelul Map-Reduce de la Google. În plus, calculatoarele moderne au mai multe procesoare sau procesoare "multi-core". În fine, compilatoarele pentru limbajele concurente (Java) şi bibliotecile de comunicare a mesajelor (MPI) sunt furnizate de toate firmele mari de software.

Pentru a încadra mai bine subiectul, vom încerca să răspundem succint la următoarele întrebări:

- care sunt principalele categorii de aplicaţii paralele şi distribuite?
- care sunt principalele clase conceptuale?
- care sunt principalele metode de programare?
- care sunt principalele clase de sisteme folosite?

1.1. Categorii de aplicaţii

Există mai multe motive pentru a utiliza calculul paralel în realizarea aplicaţiilor: scurtarea timpului de execuţie, realizarea unei fiabilităţi crescute, specializarea funcţională şi natura inerent paralelă a aplicaţiilor.

Una din căile de **creştere a performanţelor** unei aplicaţii este execuţia simultană a diferitelor parţi ale programului (parţi pe care le numim procese) pe procesoare distincte. De cele mai multe ori, procesele nu sunt complet independente, ci comunică între ele. În funcţie de frecvenţa comunicărilor, paralelismul are o **granularitate** mai mare (comunicări rare) sau mai fină (comunicări frecvente).

O mare parte dintre algoritmii paraleli prezentaţi în literatură se referă la această categorie de aplicaţii. Problemele care trebuie rezolvate aici sunt legate atât de găsirea unei împărţiri adecvate în procese a prelucrărilor ce trebuie realizate, cât şi de găsirea celor mai potrivite mecanisme de comunicare sau sincronizare între procese.

Pentru aplicaţii la care **siguranţa în funcţionare** este un factor critic (de exemplu, comanda unui avion), replicarea unor părţi ale lor (procese sau date) poate constitui o soluţie de toleranţă la defecte. Daca anumite procesoare se defectează, replicile lor le pot suplini. Ceea ce interesează aici sunt tehnicile de realizare corectă a comutării, cu alte cuvinte protocoalele ce asigură comportarea corectă în sisteme în care funcţionalitatea componentelor este parţială.

Unele aplicaţii pot fi împărţite în faze cu **funcţionalităţi distincte**: de calcul în virgulă mobilă, de prelucrări grafice, de prelucrare de fişiere, de imprimare etc. Aceasta permite exploatarea mai bună a caracteristicilor diferitelor componente ale sistemelor de calcul, pentru realizarea unor performanţe deosebite (folosirea unor procesoare în virgulă mobilă, a unor procesoare grafice, a unor servere de fişiere sau de imprimare). Problemele importante sunt aici cele legate de partajarea eficientă a resurselor sistemelor de calcul.

Unele aplicaţii sunt **inerent paralele**. Un exemplu îl constituie accesul simultan la o bază de date. Problemele importante aici sunt excluderea mutuală şi protecţia utilizatorilor.

Sunt patru categorii de *specialişti* interesaţi de calculul paralel şi distribuit:

- specialişti în calculatoare (computer scientists) din domeniile arhitectura, limbaje, algoritmi (căutare şi sortare, recunoaşterea formelor, toleranţa la defecte, algoritmi cu înalt grad de paralelism, algoritmi cu ascunderea latentelor), metodologii;
- specialişti în calcul numeric (computational scientists) interesaţi în pachete de programe pentru calcul matriceal, sisteme de ecuaţii lineare şi nelineare, cu derivate parţiale etc.
- alte domenii ştiinţifice şi inginereşti (tehnologii şi materiale noi, biomedicina, fizica nucleară şi altele)
- comerciali (data mining, analize de risc etc.)

Cerinţele acestora şi caracteristicile aplicaţiilor lor sunt prezente succint în tabelul 1.1.

Tabel 1.1

Domeniu	**Ştiinţific şi ingineresc**	**Comercial**
Caracteristici	virgulă mobilă cod de dimensiune redusă rata I/E redusă puţine date puţini utilizatori	întregi cod de dimensiune mare rata I/E mare multe date mulţi utilizatori
Cerinţe	performanţa scalabilitate	performanţa siguranţa în funcţionare disponibilitate

1.2. Clase conceptuale

Desprinderea unor clase conceptuale în paralelism este importantă ca sprijin în prima fază de abordare a unei probleme. Încadrarea într-una din aceste clase poate dirija proiectantul pe parcursul conceperii soluţiei paralele a unei probleme. Clasificarea adoptată aici este una din mai multele prezentate în literatura de specialitate. Ea aparţine lui Carriero şi Gelernter şi priveşte paralelismul în termenii:

- rezultatului unui program,
- agendei de activităţi a programului,
- unui ansamblu de specialişti care împreună constituie programul.

Pentru a înţelege mai bine fiecare concept, vom lua ca exemplu construcţia unei case. Folosirea mai multor persoane la acest lucru conduce inerent spre paralelism. Dar, există mai multe căi de folosire a lucrătorilor.

Putem porni prin divizarea produsului final în mai multe componente: fundaţie, pereţi, ferestre, uşi, contor electric şi conductori, contor de apă, ţevi, chiuvete etc. Putem apoi trece la construirea simultană a componentelor şi la asamblarea lor, pe măsură ce sunt terminate. Fiecărui lucrător i se repartizează o parte a produsului final, de a cărui construire se îngrijeşte. Toţi pornesc lucrul simultan şi lucrează până în momentul când continuarea depinde de o anumită restricţie (de exemplu, de o altă componentă). Acesta este paralelismul **rezultat**, numit astfel deoarece rezolvarea unei probleme pleacă de la împarţirea rezultatului în mai multe fragmente, care sunt calculate în paralel, luând în consideraţie (bineânţeles) restricţiile impuse de problemă.

Un exemplu tipic este calculul sumei a doi vectori de **n** elemente, S = A+B. Elementele lui S sunt independente între ele şi pot fi calculate în paralel, într-un singur pas, folosind **n** procesoare, după următoarea schemă:

```
co [j = 1 to n] {S[i] = A[i] + B[i]}
```

O altă posibilitate este de a alcătui o **agendă de activităţi** necesare pentru a construi casa. Unele dintre ele se vor executa în paralel, altele sunt condiţionate de o anumită ordine. Oricum, lucrătorii vor fi repartizaţi activităţilor curente din agendă, în numărul necesar şi fără a se face vreo diferenţiere între ei. Fiecare lucrează cât timp restricţiile naturale ale problemei le permite acest lucru.

Cea mai cunoscută **paradigmă** de implementare a acestei clase de paralelsim este "replicated workers" sau "master-worker". Un proces **master** iniţializează calculele şi crează un număr de procese **worker** identice. În mod repetat, fiecare worker preia o sarcină şi o execută. Programul se execută la fel, indiferent de numărul proceselor worker şi se termină odată cu epuizarea sarcinilor.

Pentru exemplificare, să considerăm o problemă de minim. Dat fiind un ansamblu E de înregistrări, E = {r} şi o funcţie de cost f:E->R care asociază fiecărei înregistrări **r** un cost **f(r)**, se cere **min f(r)** cu **r** din E. Procesul master umple un sac cu înregistrările din E. Fiecare proces worker extrage o înregistrare **r**, calculează **f(r)** şi, dacă găseşte o valoare inferioară minimului parţial curent, o transmite master-ului. Acesta actualizează minimul curent. Programul se termină la "golirea" sacului.

În fine, putem avea în vedere faptul că lucrătorii sunt specializaţi şi că realizarea unei case reclamă diferite specialităţi. Toţi lucrătorii pot începe simultan. Iniţial unii vor aştepta, până când condiţiile necesare efectuării operaţiei pentru care sunt specializaţi sunt îndeplinite. Oricum, la un moment dat, vor lucra mai multe persoane. Aceasta este abordarea paralelismului ca **ansamblu de specialişti**.

Această abordare este specifică lucrului în "banda de asamblare" (pipeline) şi se pretează în cazul prelucrării mai multor obiecte identice (de exemplu un grup de case). În cazul general, ea conduce la programe organizate ca o reţea logică, în care nodurile execută calcule relativ autonome, iar comunicarea între noduri se face dupa tipare predictibile. Un exemplu tipic este cel al simulării unor circuite, în care funcţionarea fiecărei componente este reprezentată de descrierea unui proces.

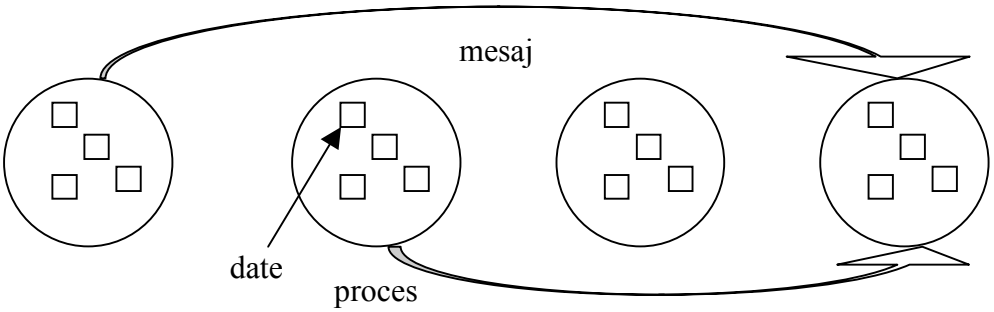
Autorii acestei clasificări insistă asupra faptului că diversele clase nu sunt net diferenţiate între ele. De asemenea, evidenţiază că **paralelismul de date**, considerat de alţi autori ca o clasă conceptuală importantă, nu este decât o formă particulară a **agendei de activităţi**. În paralelismul de date, fiecare operaţie este aplicată sincron tuturor componentelor unei structuri de date. Asupra acestui subiect vom reveni, arătând în ce mod alegerea unei clase conceptuale influenţează rezultatul proiectării.

1.3. Metode de programare

Odată stabilită clasa conceptuală cea mai potrivită pentru aplicaţie, urmează proiectarea algoritmului folosind o metodă de programare adecvată.

În programarea folosind **comunicare de mesaje** (figura 1.1), fiecare proces încapsulează structurile de date folosite (locale procesului), comunicarea între procese făcându-se prin mesaje. Fiecare proces are acces doar la datele sale locale. Pentru comunicare, procesele includ acţiuni explicite de transmitere şi recepţie de mesaje.

Comunicarea de mesaje exprimă într-un mod natural paralelismul "specialist". Fiecare proces este un nod al reţelei de specialişti, între care comunicarea se face prin mesaje.



*F*igura 1.1

O abordare complet diferită (figura 1.2) încapsulează fiecare proces într-un obiect. Procesul este responsabil doar de calculul valorii corespunzătoare obiectului respectiv. Mediul exterior este interesat doar în cunoaşterea valorii unui obiect, nu şi în modul în care ea este calculată. Comunicarea nu se face prin acţiuni de transmitere şi recepţie. Pur şi simplu, când un proces are nevoie de valoarea produsă de un alt proces, el o citeşte prin acces la obiectul respectiv. Aceste obiecte sunt aşa numitele **structuri active (vii) de date**, sau obiecte active.

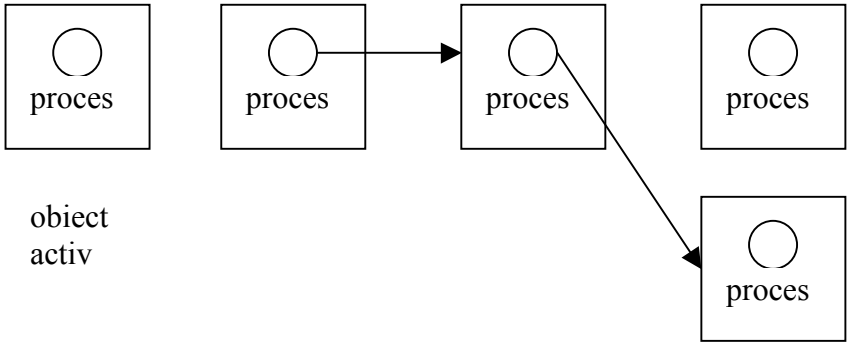


Figura 1.2

Această metodă exprimă bine paralelismul rezultat. Reluând exemplul sumei a doi vectori, fiecare obiect poate reprezenta un element S[i] al vectorului *suma* şi poate încapsula procesul ce calculează valoarea sa.

Între cele două abordări extreme se află cea la care procesele şi datele sunt entităţi distincte. Procesele pot comunica prin citirea şi scrierea **datelor partajate** (figura 1.3).

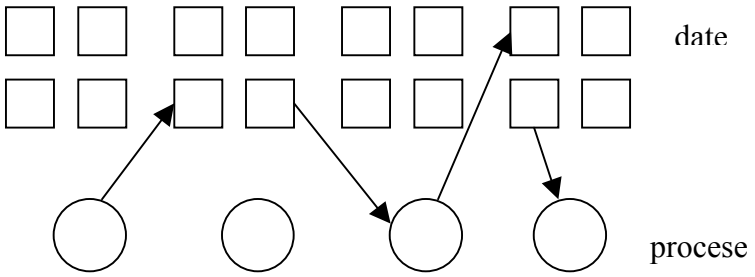


Figura 1.3

Această metodă se potriveşte bine cu paralelsimul "agenda", datele partajate reprezentând un mijloc eficient de comunicare între procesele *master* şi *worker*.

Tabelul 1.2 arată o corespondenţă posibilă între clasele conceptuale şi metodele de programare paralelă. Ea nu trebuie interpretată rigid. Aşa cum pentru o problemă de

rezolvat, programatorul are la dispoziţie modele aparţinând, eventual, unor clase conceptuale diferite, în acelaşi fel, pentru un model el are la dispoziţie mai multe metode de programare.

Tabel 1.2

Clase conceptuale	**Metode de programare**
rezultat	obiecte active
agenda de activităţi	date partajate
ansamblu de specialişti	comunicare de mesaje

1.4. Clase de sisteme paralele şi distribuite

În principiu, ar trebui ca efortul de găsire a unor algoritmi paraleli eficienţi să nu ţină seama de particularităţile unora sau altora din maşinile pe care se face implementarea acestora, ci să caute să exploateze cât mai bine paralelismul inerent problemei studiate. În practică, însă, trebuie avute în vedere detalii de implementare legate de maşina, care pot afecta (uneori într-o măsură importantă) timpul de execuţie.

Una din clasificările clasice adoptate pentru calculatoarele paralele este cea a lui **Flynn** (1966). Ea consideră fluxurile de instrucţiuni şi de date, pentru a distinge între următorele categorii:

- SISD -Single Instruction stream Single Data stream (flux unic de instrucţiuni, flux unic de date);
- SIMD -Single Instruction stream Multiple Data stream;
- MISD -Multiple Instruction stream Single Data stream;
- MIMD -Multiple Instruction stream Multiple Data stream.

O altă clasificare, asemănătoare precedenteia, consideră ca elemente de bază procesoarele şi memoria, făcând distincţie între următoarele categorii de sisteme:

- tablouri de procesoare (processor array)
- procesoare în linie de asamblare (pipeline processors)
- sisteme cu memorie partajată (multi-processors)
- sisteme cu transmitere de mesaje (distributed systems).

Deşi există legături evidente între cele două clasificări, nu se poate face între ele o corespondenţă unu la unu. Astfel, deşi modelul SIMD descrie tabloul de procesoare, el este folosit uneori şi pentru procesoare în **linie de asamblare**. Pentru acestea din urmă, modelele SISD şi MISD sunt, de asemenea, de multe ori aplicabile. A doua

clasificare prezentată este uneori preferată pentru corespondenţele directe pe care le are în programare. Reprezentările grafice asociate modelelor lui Flynn sunt cele din figura 1.4.

SISD este modelul "von Newmann" din anii '40. El nu include, în esenţă, forme de paralelism, operaţiile unui algoritm fiind executate secvenţial, de un singur procesor.

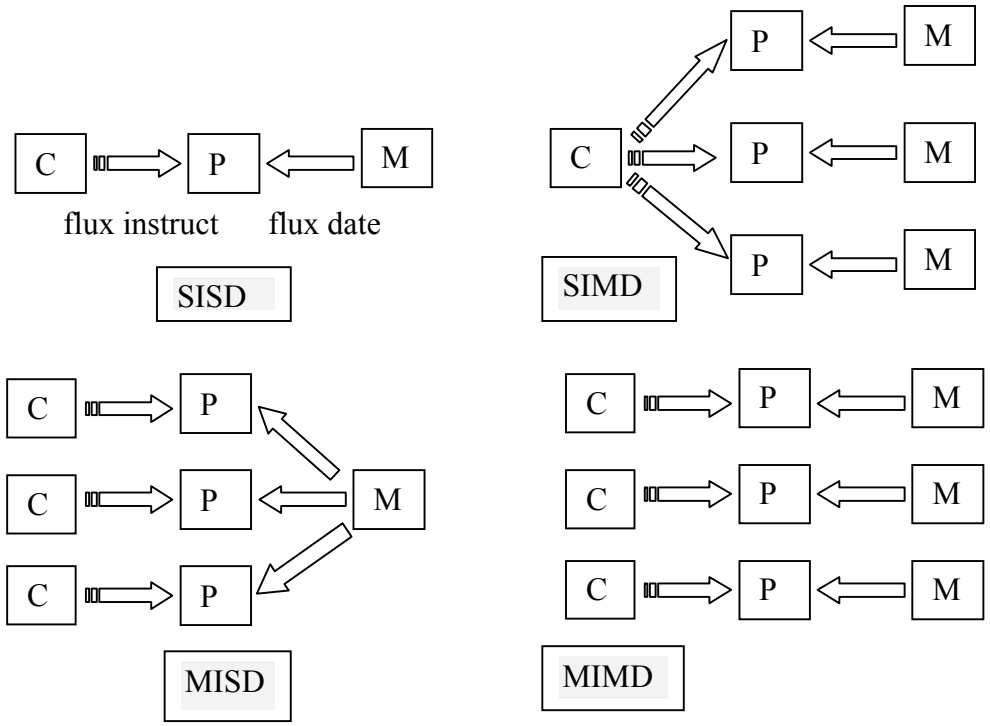


Figura 1.4
C - Control, P - Procesor, M - Memorie

În modelul MISD există un singur flux de date şi mai multe fluxuri de control. În ficare pas, procesoarele au acces la o aceeaşi dată din memorie, căreia îi pot aplica tratamente diferite, corespunzătoare unor fluxuri de instrucţiuni diferite. De exemplu, putem stabili dacă un număr natural este sau nu prim, într-un singur pas, dacă există un număr de procesoare egal cu numărul de divizori potenţiali. Mai general, procesoarele pot face operaţii diferite asupra unei aceleiaşi valori: primul poate afla

rădăcina pătrată, al doilea calculează puterea a treia a aceluiaşi număr, iar un alt procesor află dacă numărul este sau nu prim.

Spre deosebire de modelele la care ne vom referi în continuare, MISD este mai rigid şi are o utilitate practică foarte restransă.

Un **tablou de procesoare** (figura 1.5) constă dintr-un număr de procesoare identice, aflate sub controlul unei singure unităţi de comandă. Fiecare procesor are acces separat la datele sale locale, astfel încât, la un moment dat, o aceeaşi operaţie este executată simultan asupra mai multor unităţi de date. Acest mod de lucru sincron este tipic *tablourilor de procesoare*. În mod natural, operaţiile caracteristice sunt cele matriceale. Aşa cum s-a menţionat, fiecare procesor are propria sa memorie locală. Pentru actualizarea ei este necesară comunicarea cu celelalte procesoare sau cu un calculator gazdă. Comunicarea poate fi realizată în două moduri, de unde rezultă două clase de sisteme SIMD: prin memorie partajată (SM - shared memory - SIMD) şi prin reţea de interconectare.

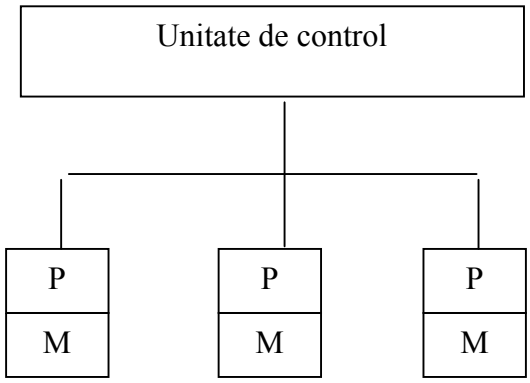


Figura 1.5

Primele sisteme, cunoscute şi sub numele de **PRAM** (Parallel Random - Access Machine), au diferite variante, după cum accesele la aceeaşi locaţie de memorie pentru *citire* sau *scriere* sunt concurente sau exclusive. Evident că performanţele algoritmilor sunt influenţate de acest aspect, aşa cum rezultă şi din exemplul următor, care se referă la o problemă de căutare.

Fie un fişier cu **n** intrări, unde **n** este foarte mare. Dându-se o valoare oarecare **a**, se cere să se precizeze dacă ea este în fişier. Considerăm că fişierul este împărţit în N

sub-fişiere, fiecare sub-fişier S_i, de dimensiune n/N, fiind accesibil unui procesor P_i distinct (sunt deci N procesoare P_1, ..., P_N). Fiecare procesor P_i citeşte valoarea lui **a**, caută această valoare în sub-fişierul S_i şi, în caz de succes, setează valoarea unei variabile rezultat R.

Problema poate fi rezolvată în **n/N** paşi. Condiţia este însă ca accesul la valoarea lui **a** pentru citire să fie concurent. În plus, dacă fişierul *nu are înregistrări distincte*, este necesar ca şi accesul la scriere pentru R să fie concurent.
Dacă prima condiţie nu este îndeplinită, valoarea lui **a** trebuie **difuzată** explicit tuturor proceselor. Deoarece comunicarea între procese se poate face doar prin memoria comună, difuzarea presupune folosirea unor variabile în această memorie.

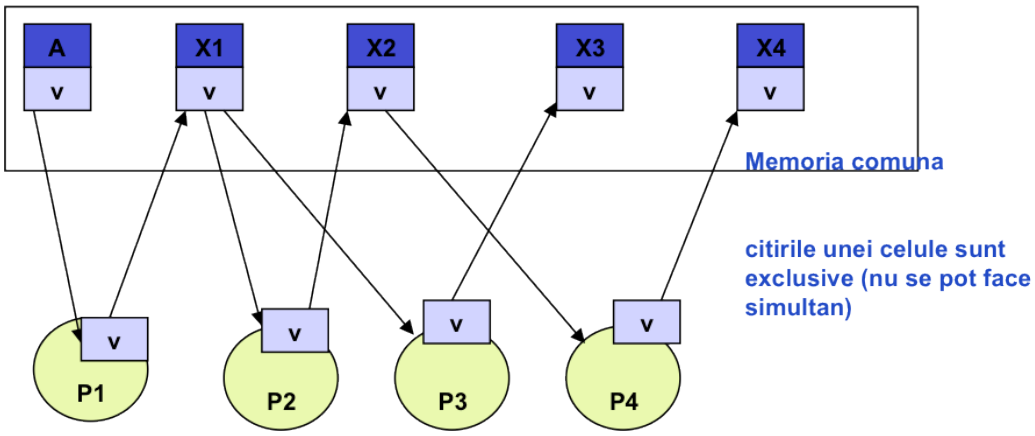


Figura 1.6

Difuzarea se poate face astfel (Figura 1.6):
- P_1 citeşte valoarea lui **a** într-o variabilă proprie şi o memorează apoi în x1 din memoria comună;
- P2 citeşte valoarea din x1 în variabila sa proprie şi o memorează în x2 din memoria comună;
- P_3 şi P_4 citesc valoarea din variabilele x1 respectiv x2, o memorează în variabilele lor proprii şi apoi o memorează în x3 şi x4 din memoria comună;
- P_5, P_6, P_7 şi P_8 citesc valoarea din variabilele x1, x2, x3, si x4, o memorează în variabilele lor proprii şi apoi o memorează în x5, x6, x7 şi x8 din memoria comună ş.a.m.d.

Deoarece numărul de procese care cunosc valoarea lui **a** se dublează la fiecare pas, rezultă că difuzarea durează $\log_2 N$ paşi. Întregul algoritm reclamă deci **log N + n/N** paşi în cazul cel mai defavorabil.

Există şi variante de terminare a algoritmului imediat ce un procesor a regăsit valoarea lui **a** în sub-fişierul corespunzător. În acest caz valoarea răspunsului R trebuie difuzată celorlalte procese. De data aceasta, valoarea rezultatului este inspectată de fiecare proces, înainte de a continua căutarea în propriul fişier, ceea ce conduce la un total de **log N + (n/N).log N** paşi în cazul cel mai defavorabil.

În privinţa sistemelor **SIMD interconectate** în reţea, topologiile uzuale sunt mai simple decât graful complet. Prezentăm câteva variante:
a) tabloul uni-dimensional
b) tabloul bi-dimensional
c) arbore binar complet
d) amestecare perfectă (perfect shuffle)

- date fiind N procesoare P_0, ..., P_{N-1}, cu N o putere a lui 2, P_i şi P_j sunt legate printr-o linie uni-direcţională, unde

  j = 2.i pentru 0 <= i <= N/2-1
  j = 2.i-N+1 pentru N/2 <= i <= N-1.

Adesea, fiecare procesor cu număr par este legat prin conexiune bi-direcţională cu succesorul său. Pentru N=8, se obţine configuraţia din figura 1.7.

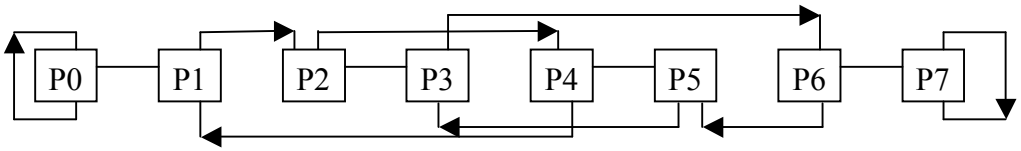


Figura 1.7

e) cub
f) hipercub.

Alegerea unei configuraţii depinde de aplicaţie, de performanţele dorite şi de numărul procesoarelor disponibile. Astfel, pentru suma a **n** numere, o topologie avantajoasă este arborele cu **n/2** frunze şi **log n** nivele. Operaţia se realizează în **log n** paşi (vezi figura 1.8).

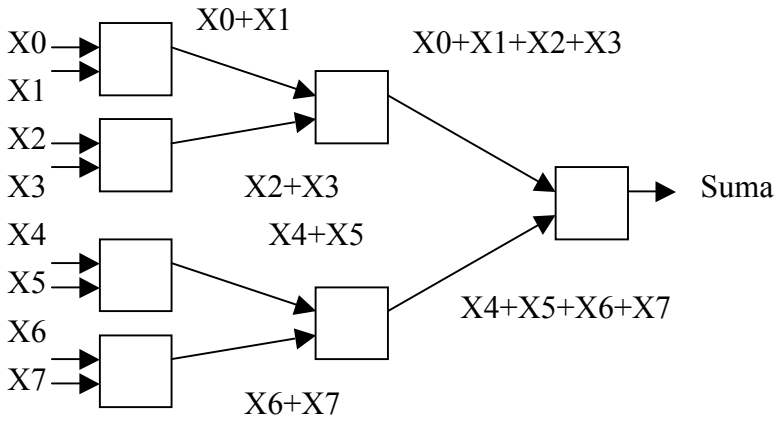


Figura 1.8

În cazul a **m** grupuri de câte **n** numere, sumele pot fi aflate în **log n + m - 1** paşi, folosind tehnica liniei de asamblare (pipeline), la care ne referim ceva mai departe în această secţiune.

Exemplele de sisteme SIMD includ mai multe modele comerciale, dintre care cele mai cunoscute sunt **CM-2** produs de Thinking Machines şi **MP-2** produs de Maspar. Din păcate, nici unul din aceste modele (care se bazează pe matrice de procesoare) nu a cunoscut succesul de piaţă. În schimb, mai larg acceptate au fost sistemele vectoriale, dintre care modelele **Cray-1**, **C90** şi **T90** au dominat domeniul calculului ştiinţific. Mai recent, unitățile de procesare grafică **GPU**s (Graphics Processing Units) sunt implementate ca mașini SIMS, fiind adesea utilizate și în alte categorii de procesări decât cele grafice.

Exemple de limbaje de programare: DAP-FORTRAN, Actus (bazat pe Pascal), CMLISP (Connection Machine Lisp). Caracteristicile sistemelor se reflectă la nivelul limbajelor. Astfel, pentru FORTRAN, tablourile sunt tratate ca tipuri fundamentale, cu operatori specifici operaţiilor matriceale.

Prelucrarea în **linie de asamblare** este un principiu utilizat în multe fabrici şi este împrumutat în sistemele de calcul. Ideea este de a împărţi o prelucrare în mai multe procese care se derulează unul după altul şi se execută fiecare pe un procesor aparte. Fiecare proces depinde de rezultatul procesului precedent din secvenţă şi furnizează rezultate procesului următor.

Dacă este necesară efectuarea mai multor prelucrări, atunci, derularea lor poate avea loc în paralel: în timp ce procesorul i execută un proces, procesorul i-1 execută procesul i-1 din prelucrarea ce urmează, iar procesorul i+1 execută procesul i+1 din prelucrarea precedentă.

Să considerăm calculul următor:

```
for i:=1 to 64 do
  begin
    E[i]:=A[i]+B[i];
    F[i]:=E[i]*C[i];
    G[i]:=F[i]-D[i]
  end
```

executat pe trei procesoare specializate pentru operaţiile de adunare, înmulţire şi respectiv scădere. Derularea în timp a operaţiilor, în condiţiile unei prelucrări în linie de asamblare la nivelul instrucţiunilor poate fi cea din figura 1.9.

E[I]:=	**1**	**2**	**3**	**4**			**n**		
F[I]:=		**1**	**2**	**3**	**4**		**n-1**	**n**	
G[I]:=			**1**	**2**	**3**		**n-2**	**n-1**	**n**
t	**1**	**2**	**3**	**4**	**5**		**n**	**n+1**	**N+2**

Figura 1.9

O clasificare a sistemelor cu prelucrare în linie de asamblare ţine cont de nivelul de complexitate al proceselor, care poate fi:

- al operaţiilor aritmetice (faze de instrucţiune)
- al instrucţiunilor
- al programelor.

Clasa cea mai generală este cea a sistemelor MIMD. Şi aici întâlnim cele două variante de comunicare: prin memorie partajată (sisteme multiprocesoare) şi prin transmitere de mesaje (sisteme multicalculatoare).

Un sistem cu **memorie partajată** (figura 1.10) constă din mai multe procesoare care au acces la module de memorie comună (sisteme multiprocesor). Procesoarele pot executa procese diferite, dar care trebuie corelate între ele (sincronizate) pentru a evita interferenţele (şi deci erorile de prelucrare). Numeroase modele au fost propuse pentru

programarea acestor sisteme. Dintre ele, amintim: primitivele P si V, regiunile critice, monitoarele.

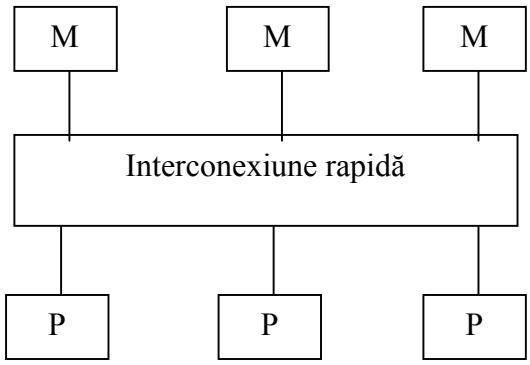


Figura 1.10

Există destul de multe sisteme experimentale care se înscriu în această categorie. Dintre cele comerciale mai recente, amintim **Sun Enterprise 10000**, care este un sistem din categoria UMA (Uniform Memory Access) pentru care procesoarele au acelaşi timp de acces la oricare modul de memorie. Configuraţia maximă a unui sistem include 64 de procesoare UltraSPARC distribuite în 16 grupuri a câte 4 procesoare. Fiecare grup este dispus pe o placă împreună cu 4 module de memorie RAM de câte 1 GB fiecare. Conexiunea între procesoare şi blocurile de memorie este asigurată de un *switch crossbar* 16*16, numit *Gigaplane-XB*. Acesta are funcţia de a transfera datele între memorie şi cache-urile procesoarelor. Transferul se face pe 16 octeţi, astfel că sunt necesare 4 cicluri pentru a transfera o line de cache care are 64 octeţi. În afara *switch*-ului, mai există patru magistrale de adrese care sunt folosite pentru *supraveghere* (*snooping*). Fiecare bus este folosit pentru o pătrime din spaţiul de adrese fizice, astfel că pentru selecţia unei magistrale din patru sunt folosiţi doi biţi de adresă. Când un procesor nu găseşte datele în propria memorie cache el plasează adresa pe magistrala de adrese corespunzătoare pentru a vedea dacă datele sunt în memoria cache a altui procesor. Toate grupurile de procesoare inspectrează toate cele patru magistrale. Cea care are datele cerute le furnizează imediat. Dacă nu există răspuns, datele nu există în cache şi trebuie aduse din memorie.

Uneori, memoria partajată este **distribuită**, astfel încât există um modul de memorie mai apropiat de fiecare procesor. Accesul la acest modul este mai rapid pentru un procesor şi mai lent pentru celelalte. Din acest motiv, modelul se mai numeşte **NUMA** (Non Uniform Memory Access). Pentru a scurta timpul de acces, unele

sisteme păstrează o parte a datelor în memoriile cache ale procesoarelor, care, evident trebuie păstrate coerente (**ccNUMA** - cache coherent NUMA). Aceste sisteme se mai numesc sisteme cu memorie distribuită, alteori sisteme cu memorie locală (figura 1.11). Exemple de sisteme ccNUMA includ *SGI Origin 2000* şi *Sequent NUMA-Q 2000*.

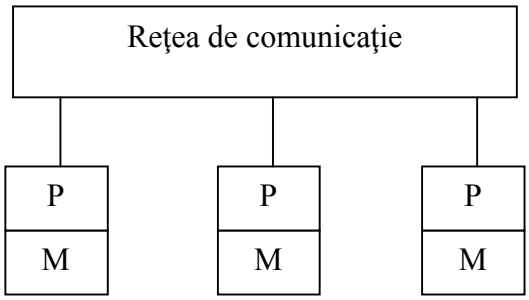


Figura 1.11

Sistemele cu **transmitere de mesaje** (se mai numesc **multicalculatoare**) constau din calculatoare interconectate. Procesoarele execută procese diferite, iar sincronizarea este realizată prin schimburi de mesaje. Diferenţele între sisteme se manifestă prin:

- comunicare sincronă sau asincronă de mesaje
- schema de comunicare -directă de la proces la proces
  -prin cutii poştale globale
  -prin canale de legătură între procese

Există două categorii de multicalculatoare. Prima categorie, cunoscută sub denumirea de **MPP**, *Massively Parallel Processors*, include sisteme în care procesoarele (în număr ridicat) sunt interconectate prin reţele de mare viteză, construite special ca soluţii "de firmă", pentru a realiza performanţe foarte ridicate. Exemple de sisteme din această categorie sunt: *Cray T3E* şi *IBM SP-2*.

A doua categorie include sisteme construite din PC-uri sau staţii de lucru interconectate prin reţele comerciale. Astfel de sisteme se numesc **NOW** (*Networks Of Workstations*) sau **COW** (*Clusters Of Workstations*).

Un exemplu de aplicaţie pentru multi-calculatoare este jocul de şah, în care se generează mutările posibile, a căror mulţime poate fi organizată arborescent: rădăcina este configuraţia curentă; fiii săi corespund configuraţiilor după prima mutare ş.a.m.d.

Găsirea mai rapidă a celei mai bune mutări se poate face distribuind unor procesoare distincte căutarea în sub-arbori diferiţi.

Comunicarea între procesoare este necesară pentru transmiterea celei mai bune soluţii găsite până la un anumit moment şi pentru transmiterea unor sub-arbori în vederea continuării inspectării. La un moment dat, două procesoare distincte pot executa secvenţe diferite de instrucţiuni, ceea ce este în concordanţă cu caracteristicile sistemelor MIMD.

Capitolul 2. Limbajul de descriere a algoritmilor

În accepţiunea noastră un **program paralel / distribuit** este o colecţie de procese secvențiale comunicante. Exisă mai multe categorii de programe concurente, delimitate în funcţie de raportul dintre procese şi procesoarele pe care acestea se execută. În **programele paralele**, procesele sunt executate de procesoare distincte care comunică prin **variabile partajate** aflate în memoria comună. Uneori, organizarea unui program ca o colecţie de procese este utilă chiar în cazul în care acestea se execută pe un singur procesor, intercalat. Fiecare proces poate fi planificat şi executat independent de celelalte, existând practic mai multe fire de execuţie (threads) în acelaşi **program multithread**. Noţiunea de program multithread se extinde şi la situaţia în care numărul de procese dintr-un program este mai mare ca numărul de procesoare pe care acestea se pot executa. În **programele distribuite**, procesele se execută pe maşini interconectate printr-o reţea de calculatoare şi, ca urmare, comunică prin schimb de mesaje.

Deşi există multe posibilităţi de a structura colecţia de procese dintr-un program concurent, s-au impus câteva **tipare**: paralelismul iterativ, paralelismul recursiv, producători şi consumatori, clienţi şi servere, procese egale (interacting peers). Ne vom referi la acestea pe parcursul prezentării limbajului de descriere a algoritmilor şi a unor exemple semnificative.

În mod natural, descrierea adoptată pentru programele paralele se încadrează în modelul CSP (Communicating Sequential Processes) propus de Hoare. Ea se bazează pe construcţiile clasice ale unui limbaj de programare secvențială, extinse cu notaţiile ce permit concurenţa, comunicarea şi sincronizarea.

Prezentăm la început notaţiile (pseudocod) corespunzătoare descrierii algoritmilor secvenţiali. În secţiunile următoare vom introduce construcţii specifice programării paralele (bazate pe variabile partajate şi comunicare de mesaje).

2.1. Declaraţii

Tipuri de bază sunt considerate următoarele:
boolean - bool
întreg - int
real - real / double
caracter - char
şir - string [lung-max]

Declaraţia variabilelor are forma generală

tip id = expresie;

care specifica tipul, numele și valoarea inițială a variabilei (iniţializările sunt opţionale). Exemplu: int i, j=1;

Definiţiile de constante au forma generală

const tip id = expresie;

Exemplu: const int c=1;

Tablourile se declară prin specificarea tipului de bază, a numelui, a tipului indicilor (domeniul) și a valorii inițiale

tip id [domeniu] = expresie;

Exemple:
int a[5]; sau int a[1:5];
int b[1:10];
int c[1:5] = ([5] 0);
double c[15,15] = ([15] ([15] 1.0));

Forma "int a[1:5];" este utilă pentru a specifica valorile limită ale domeniului, putându-se face ușor difierența între a[1:5] și a[0:4], acolo unde este cazul.

Constructorul de tablou "([5] 0)" permite specificarea valorii inițiale a tabloului "c[1:5]" ale cărui elemente au toate valoarea inițială 0.

Tipuri definite de utilizator

Definirea tipul enumerat are forma generala

enum id {enumerare_simboluri};

Exemplu: **enum** zile {l, ma, mi, j, v, s, d};

O **înregistrare** se defineşte ca o colecţie de cîmpuri cu tipuri potenţial diferite

typedef struct {definitie_campuri} id;

Exemplu:

```
typedef struct {
int front=1;
int rear=1;
int size=0;
int contents[5];} queue;
queue q1, q2[5];
```

În exemplu, este definit tipul "queue", folosit apoi pentru declararea unei variabile simple si a unui tablou cu elemente de tip "queue".

2.2. Instrucţiuni

Atribuirea x=e evaluează expresia e şi asociază rezultatul variabilei **x**. Aceasta poate fi un scalar, un tablou, o înregistrare, un element de tablou, un cîmp de înregistrare.

Interschimbarea x1:=:x2 interschimbă valorile celor două variabile.

skip este instrucţiune inoperantă. Execuţia ei se termină imediat şi nu are nici un efect asupra variabilelor programului.

Ieșirea dintr-o instrucţiune compusă se face prin **exit;**.

Decizia are una din formele generale următoare:

if (conditie) instructiune;

if (conditie) {bloc instructiuni};

if (conditie) {bloc instructiuni}
else if (conditie) {bloc instructiuni}
else (conditie) {bloc instructiuni};

Exemplu:

```
if (x>y) m = x;
```

else if (y>x) m = y;
else m = x+y;

Iteraţia (while) are forma generală

while (conditie) {bloc instructiuni};

Ciclul cu contor (fa) are forma generală

for [cuantificatori] {bloc instructiuni}

Fiecare cuantificator specifică un domeniu de valori pentru o variabilă contor (de tip implicit int):

variabila = val_init to val_finala st B

Corpul ciclului este executat pentru acele valori ale variabilei contor pentru care valoarea expresiei booleene B este *true* (**s**uch **t**hat B).

În cazul mai multor cuantificatori, aceştia se separă prin virgule. Corpul ciclului se execută pentru fiecare combinaţie de valori ale variabilelor contor, cu variabila cea mai din dreapta variind cel mai rapid.

Exemple:
1) transpunerea matricei m:

for [i=1 to n, j=i+1 to n] m[i,j]:=:m[j,i];

2) ordonarea crescătoare a unui vector:

for [i=1 to n, j=i+1 **to** n **st** a[i]>a[j]] a[i]:=:a[j];

2.3. Proceduri

O procedură defineşte un tipar parametrizat al unei operaţii. Forma sa generală este următoarea:

```
tip_rez p_id(tip_1 f1, … ,tip_n fn){
declaraţii
instrucţiuni – pot contine return
}
```

Definiția include:

tip_rez tipul rezultatului întors de procedură
p_id numele procedurii
fi sunt parametrii formali cu tipurile tip_i

O procedură fără **returns** este apelată explicit prin instrucţiunea **call**:

call p_id(e_1,...,e_n)

în timp ce o funcţie (cu returns) apare ca operând într-o expresie:

x = p(e_1,...,e_n).

În ambele cazuri, e_i sunt expresii, avînd acelaşi tip cu parametrii formali corespunzători.

Exemplu: calculul factorialului.

```
int fact(int i){
if (i<0) return -1;
else if (i==0 OR i==1) return 1;
else {return i*fact(i-1);}
```

2.4. Concurenţă şi sincronizare

În accepţiunea noastră, un program paralel descrie două sau mai multe procese cooperante. Ca urmare, dacă la execuţia unui program secvenţial există un singur fir al controlului (thread of control), la cea a unui program paralel există mai multe fire, cîte unul pentru fiecare proces.

Fie S_i un program secvenţial (declaraţii + instrucţiuni). Construcţia:

co S1 || S2 || ... || Sn oc

specifică execuţia paralelă a programelor S1 ... Sn, efectul fiind o anumită întreţesere a acţiunilor lor atomice. Execuţia lui **co** se termină atunci cînd se termină execuţia fiecărui Si.

Exemplu:

```
x=0; y=0;
co x = x+1; || y =y+1; oc
z=x+y;
```

Variabilele declarate înainte de **co** sunt comune proceselor din **co**. Procesele pot declara şi variabile locale, domeniul unei astfel de variabile fiind procesul în care

apare declaraţia.

Adesea, un program paralel conţine procese care execută acelaşi calcul asupra unor elemente de tablou diferite. Descrierea se poate face folosind cuantificatori la fel ca pentru instrucţiunea **for**.

co [cuantificator]{Sj}

Exemplu:

```
co [j=1 to n] {a[j]=0; b[j]=0;}
```

Fiecare proces are o copie proprie a lui j, care se comportă în cadrul procesului ca o constantă. Construcţia precedentă descrie **n** procese distincte.

Exemplu: înmulţirea a două matrice:

```
real a[1:n,1:n], b[1:n,1:n], c[1:n,1:n];
co [i = 1 to n, j= 1 to n] {
      real sum = 0;
      for [k = 1 to n]
          sum=sum+a[i,k]*b[k,j];
      c[i,j] = sum;
}
```

Aici sunt n^2 procese, fiecare având constantele locale *i* şi *j*, precum şi variabilele locale *sum* şi *k*. În cazul unor procese cu descrieri lungi, se foloseşte şi următoarea formă, în care se asociază nume proceselor paralele conţinute de un program.

Exemplu

```
real a[1:n,1:n], b[1:n,1:n], c[1:n,1:n];

process Prod (i = 1 to n, j= 1 to n) {
      real sum = 0;
      for [k = 1 to n]
          sum=sum+a[i,k]*b[k,j];
      c[i,j] = sum;
}
```

2.5. Atomicitatea şi sincronizarea

Aşa cum s-a precizat, execuţia unui program paralel poate fi privită ca o întreţesere a acţiunilor atomice ale proceselor. Cînd procesele interacţionează, nu toate

combinaţiile sunt acceptabile. De exemplu, să considerăm următorul program şi să luăm ca acţiuni atomice citirea şi scrierea unei variabile:

```
int y= 0 ; z= 0 ;
co x=y+z || y=1 ; z=2 oc
```

Dacă adunarea este realizată prin încărcarea valorii lui y într-un registru şi adăugarea ulterioară a valorii lui z, valoarea lui x poate fi 0, 1, 2, sau 3, în funcţie de vitezele celor două procese. La prima vedere, evaluarea unei expresii nu poate fi considerată acţiune atomică în context paralel, decît dacă variabilele din expresie nu sunt alterate de alte procese.

In realitate, atomicitatea este asigurată de o condiţie mai slabă, numită proprietatea **cel-mult-o-dată** (at-most-once), care se formulează astfel:

O **expresie** *e* satisface proprietatea de *cel-mult-o-dată* dacă ea se referă cel mult o dată la cel mult o variabilă simplă (element de tablou sau cîmp de înregistare) care ar putea fi schimbată de un alt proces în timpul evaluării lui *e*. O **instrucţiune** de atribuire ***x= e*** satisface proprietatea de cel-mult-o-dată fie dacă *e* satisface această proprietate şi *x* nu este citită de un alt proces (adică *x* este o variabilă locală), ori dacă *x* este o variabilă simplă şi *e* nu se referă la nici o variabilă care ar putea fi schimbată de un alt proces.

De exemplu, nici o atribuire din construcţia:

```
co x = y+1 || y = x+1 oc
```

nu satisface proprietatea menţionată. De asemenea, atribuirea x = y+z din exemplul dat la începutul secţiunii nu satisface această proprietate, ea referind două variabile modificate de un alt proces.

În cazurile cînd proprietatea anterioară nu este satisfacută, sau dacă dorim gruparea mai multor acţiuni într-o singură acţiune atomică de granularitate mai mare, este necesar un mecanism de sincronizare.

Pentru a specifica acţiuni atomice folosim paranteze unghiulare. De exemplu, <e> înseamnă că expresia e trebuie evaluată atomic. Pentru sincronizare, folosim construcţia:

```
<await (B) S;>
```

unde B este o expresie booleană, iar S este o succesiune de instrucţiuni (secvenţiale) care se termină cu siguranţă.

Semnificaţia este următoarea: se aşteaptă îndeplinirea condiţiei B, după care se execută S. Execuţia este atomică, deci B este cu siguranţă *true* la începerea execuţiei lui S şi nici o stare internă a lui S nu este vizibilă din afară.

Exemplu:

```
<await (s>0) s=s-1>
```

întîrzie procesul executat pînă cînd s devine pozitiv, apoi decrementeză pe s.

Forma generală a instrucţiunii **await** specifică atît excluderea mutuală, cît şi sincronizarea condiţionată. Ea admite formele :

`<s;>`	pentru a exprima doar excluderea mutuală şi
`<await (B);>`	pentru a exprima doar sincronizarea condiţionată.

Exemplu: excludere mutuală la folosirea unei resurse comune.

Procesele P[i] al unei colecții de procese P[i=1 to n] folosesc o aceeași resursă comună (de exemplu, un același fișier sau un același dispozitiv de imprimare). Deoarece accesul simultan la resursa comună ar putea produce erori de funcționare, în orice moment resursa trebuie folosită de cel mult un proces. Operațiile de folosire a resursei comune constituie o **secțiune critică** a proceselor. Pentru simplitate considerăm că fiecare proces include o **secțiune critică** în care are acces exclusiv la resursa partajată, urmată de o **secțiune necritică** în care folosește doar resurse care îi sunt locale.

Problema este cum controlăm "intrarea" proceselor în **secțiunea critică** astfel încât să se asigure **excluderea mutuală** între procese, la utilizarea resursei comune. Pentru asta se folosește o variabilă **s** care are valoarea 1 dacă resursa comună este liberă și 0 dacă resursa este folosită de un proces. Descrierea soluției este următoarea:

```
int s = 1;                  /*s=1 – resursa liberă*/
                            /*s=0 – resursa folosita*/
process P[i=1 to n]{
   while (true){
      <await (s>0) s = s-1;>;       /*acaparare resursa*/
      Sectiune critica;
      <s = s+1>;                    /*eliberare resursa*/
      Sectiune necritica
   }
}
```

Pentru intrarea în secțiunea critică, procesul P[i] execută operația **atomică**

```
<await (s>0) s = s-1;>;
```

Atomicitatea asigură că operația este executată în întregime, de un singur proces:
- dacă s==0 procesul este pus în așteptare;
- dacă s>0, evaluarea condiției și modificarea lui **s** sent executate ca o singură operație (execuția nu poate fi întreruptă de nici un alt proces);
- dacă mai multe procese încearcă să execute operația, doar unul primește permisiunea de execuție, celelalte ramânând în așteptare.

La ieșirea din secțiunea critică procesul P[i] execută operația atomică

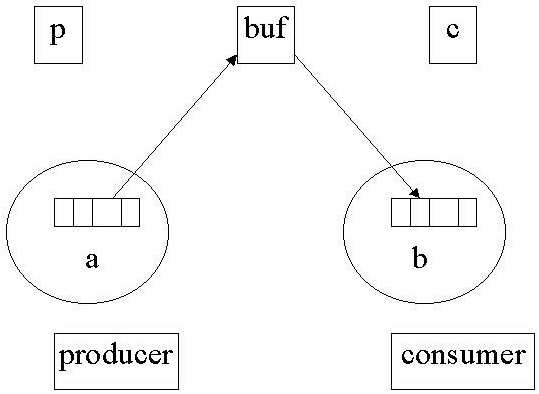
```
<s = s+1>;
```

prin care valoarea lui **s** devine 1; eventual, un alt proces va putea executa operația de acaparare a resursei comune.

Exemplu. Problema producător-consumator.

Se consideră un producător și un consumator care comunică printr-un singur tampon partajat. Producătorul generează și pune câte o valoare în tampon, iar consumatorul preia câte o valoare din tampon. Presupunem că tamponul poate păstra o singură valoare.

În soluția prezentată aici, considerăm că producătorul păstrează într-un tablou **a[1:n]** valorile furnizate; consumatorul pune în tabloul **b[1:n]** valorile preluate din tampon. Soluția trebuie să asigure că valorile ajung în tabloul **b** în același număr și în aceeași ordine ca în **a**.



Soluția are următoarea descriere:

```
int buf, p=0, c=0;
     /*p = numarul de valori transmise in buf*/
     /*c = numarul de valori preluate din buf*/
process producer{
   int a [1:n];
   while (p<n){
      <await (p==c);>
      buf = a[p+1];
      p = p+1
   }
}
process consumer{
   int b [1:n];
   while (c<n){
      <await (p>c);>
      b[c+1] = buf;
      c = c+1
   }
}
```

Observaţii: acţiunile atomice se rezumă la testarea condiţiilor, fără a cuprinde şi celelalte acţiuni asupra variabilelor comune. Semnificațiile celor două condiții sunt:

- p=c atunci când tamponul **buf** este "gol"; condiția este îndeplinită la început (p=c=0) sau după ce consumatorul preia o valoare din tampon;
- p>c după ce producătorul pune o valoare în **buf**.

Sugerăm cititorului să încerce găsirea unei justificări neformale a corectitudinii soluţiei.

Problemă. Elaboraţi o soluţie pentru comunicare printr-un tampon limitat, buf[1:k], cu $1 < k < n$.

Sincronizarea prin **await**, deşi generală, este criticată din motive legate atât de claritatea descrierilor rezultate, cât şi de eficienţa posibilelor implementări:
- conduce uneori la soluţii complicate, greu de înţeles şi de demonstrat;
- nu face o separare clară între variabilele utilizate pentru sincronizare şi cele folosite pentru rezultatele calculelor;

- tehnica de implementare a acestei sincronizări (numită *busy waiting* - aşteptare activă) este ineficientă, fiecare proces testînd continuu condiţia din **await** pînă cînd aceasta devine adevărată.

Nu este, deci, de mirare că s-au căutat mecanisme de sincronizare care să înlăture neajunsurile menţionate (semafoare, regiuni critice, monitoare, bariere, comunicare de mesaje). Dintre acestea, ne referim în continuare la *bariere de sincronizare* şi la *comunicarea prin mesaje*, des utilizate în exprimarea algoritmilor paraleli.

2.6. Bariere de sincronizare

Barierele reprezintă un mecanism de sincronizare asociat unei clase de paralelism numită **paralelism de date**. Corespondentul arhitectural al acesteia îl constituie modelul SIMD.

Un algoritm pentru paralelism de date este un algoritm **iterativ** care prelucrează în paralel şi în mod repetat elementele unui tablou. Acest gen de algoritmi este în mod natural asociat modelului SIMD. El este însă util şi în cazul multiprocesoarelor asincrone.

Un atribut important al algoritmilor iterativi paraleli este că fiecare iteraţie depinde (în general) de rezultatul iteraţiei precedente. O variantă este să se organizeze fiecare iteraţie ca o colecţie de procese paralele:

```
while (true){
co [k = 1 to n] {cod_proces-k;}
}
```

Soluţia este ineficientă deoarece creează şi distruge procese la fiecare iteraţie. O a doua soluţie, mai eficientă, creează procesele o singură dată, făcînd sincronizarea lor la trecerea de la o iteraţie la alta:

```
co[k= 1 to n]{
   while (true){
   cod_proces_k;
   barrier;
   }
}
```

Acest tip de sincronizare se numeşte **barieră**.

Pentru a da algoritmilor o formă generală, utilizăm o notaţie explicită pentru marcarea barierelor, şi anume **barrier**. Menţionăm însă că sistemele SIMD asigură prin construcţie sincronizarea şi elimină necesitatea programării explicite a barierelor. În cele ce urmează, prezentăm soluţiile unor probleme tipice din categoria paralelismului de date.

Exemplu: calcul grila

În multe aplicaţii, ca prelucrarea imaginilor sau rezolvarea ecuaţiilor diferenţiale, se utilizează operaţii asupra unor structuri matriceale sau de graf, numite calcule grilă sau calcule plasă. În grafică, ideea este de a utiliza o matrice care corespunde valorilor pixelilor. În rezolvarea unor ecuaţii diferenţiale, elementele de la exteriorul matricei sunt iniţializate cu valorile de graniţă, scopul fiind găsirea valorilor elementelor din mijloc, ceea ce corespunde găsirii unei soluţii stabile a ecuaţiei.

În ambele cazuri, tiparul prelucrărior este următorul:

iniţializează matricea
while (neterminat){
  calculează o nouă valoare pentru fiecare punct
  evaluează terminarea
}

La fiecare iteraţie, noile valori ale punctelor sunt calculate în paralel.

Pentru exemplificare, considerăm o soluţie a ecuaţiei lui Laplace ($\Delta^2(\Phi) = 0$) în două dimensiuni. Fie grila[0:n+1, 0:n+1] o matrice de puncte. Putem utiliza în calcul o metodă cu diferenţe finite (de exemplu, Jacobi). La fiecare iteraţie, calculăm o valoare a fiecărui punct interior ca media aritmetică a celor patru vecini cei mai apropiaţi. Metoda este staţionară, astfel că putem termina iteraţiile atunci cînd noile valori pentru punctele interioare diferă cu mai puţin de EPS de valorile anterioare.

```
real grila[0:n+1, 0:n+1], noua[0:n+1, 0:n+1];
bool converge = false;
process CalculGrila[i=1 to n, j=1 to n]{
   while (not converge){
      noua[i,j] = (grila[i-1,j]+grila[i+1,j]+
```

```
            grila[i,j-1]+ grila[i,j+1])/4;
        barrier;
        test_convergenta;
        barrier;
        grila[i,j] = noua[i,j];
        barrier;
    }
}
```

Matricea **noua** este folosită pentru ca valorile din **grila** să depindă doar de vechile valori. Calculele se termină atunci cînd toate valorile din **noua** diferă cu mai puţin de EPS de valorile din **grila**. Aceste diferenţe pot fi calculate în paralel, dar rezultatele trebuie combinate, ceea ce poate fi făcut utilizînd un algoritm prefix paralel.

Problemă (Etichetarea regiunilor)
Dată fiind o imagine bidimensională (o grilă de valori de pixeli) să se identifice şi să se eticheteze în mod unic toate regiunile. O regiune este un set maximal de pixeli vecini, care au aceeaşi valoare.

2.7. Paralelismul de date

2.7.1. Operaţii cu vectori

Considerăm în continuare calculul paralel al sumelor tuturor prefixelor unui tablou linear. Mai precis, dat fiind tabloul a[1:n], calculăm s[1:n], unde s[i] este suma primelor i elemente ale tabloului **a**.

Soluţia secvenţială uzuală este următoarea:

```
s[1] = a[1];
for [i = 2 to n] s[i] = a[i] + s[i-1];
```

Pentru a obţine o paralelizare a acestei soluţii, să considerăm mai întîi problema mai simplă a calculului sumei tuturor elementelor unui tablou. În acest scop, am putea însuma la început elementele tabloului două cîte două. În etapa următoare, însumăm rezultatle obţinute tot două cîte două şi aşa mai departe. Un exemplu, pentru un tablou cu 8 elemente este dat în figura 2.1.

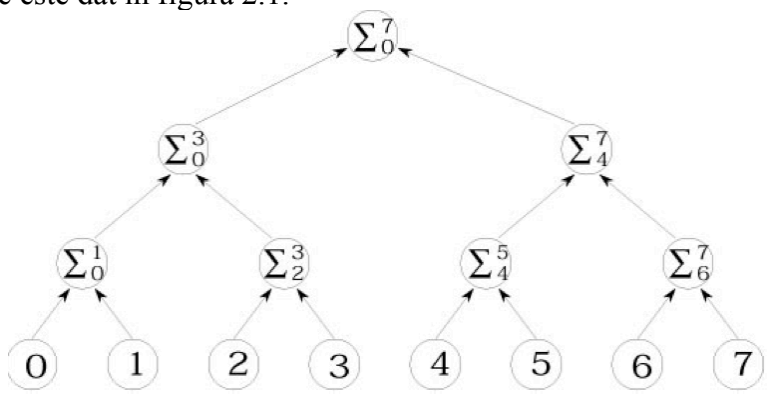


Figura 2.1

Nu este greu de văzut că putem calcula suma elementelor în $\log_2 n$ paşi, folosind n-1 procesoare.

Pentru sumele prefixelor, adaptăm această soluţie, bazată pe ideea dublării, la fiecare pas, a numărului elementelor ce au fost însumate. În acest scop, prezentăm mai întîi o schemă modificată a însumării tuturor elementelor, care permite deducerea mai simplă a algoritmului paralel.

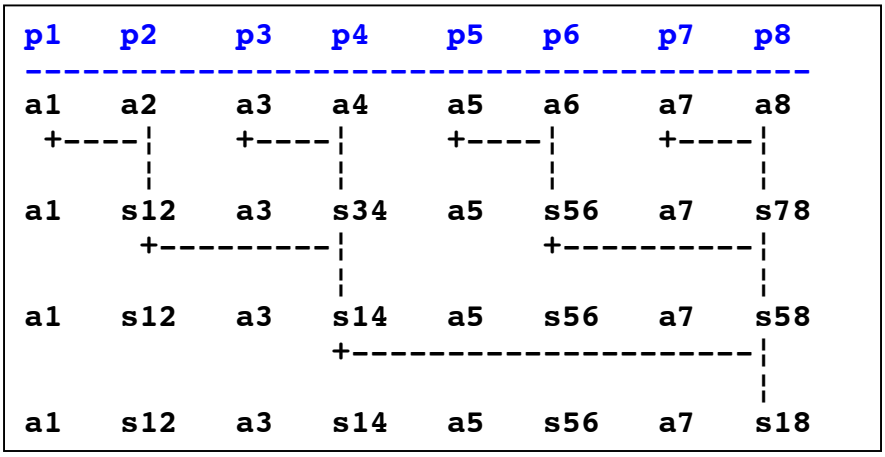


Figura 2.2

```
int a [n];
process suma [k=1 to n]{     // suma ~ procese pi din figura 2.2
      for [j = 1 to sup(log2 n)]{
            if (k mod 2 j == 0) a[k] = a[k-2j -1] + a[k];
      barrier;
      }
}
```

Din păcate, această soluţie este aplicabilă doar dacă numărul elementelor tabloului este o putere a lui 2. În caz contrar, putem adăuga un număr corespunzător de elemente nule. O altă soluţie pleacă de la observaţia că în algoritmul de însumare, numărul de procesoare folosite efectiv se înjumătăţeşte la fiecare pas, tot mai multe procesoare rămînînd libere, Dacă folosim aceste procesoare putem ajunge la o soluţie generală, valabilă pentru un număr oarecare de elemente. În acest scop, încercăm să utilizăm fiecare procesor la fel ca celelalte: la fiecare pas j, valorii parţiale a[k] i se adaugă valoarea elementului din poziţia $k-2^{j-1}$, dacă aceasta există. Pentru un tablou cu şase elemente, obţinem schema din figura 2.3.

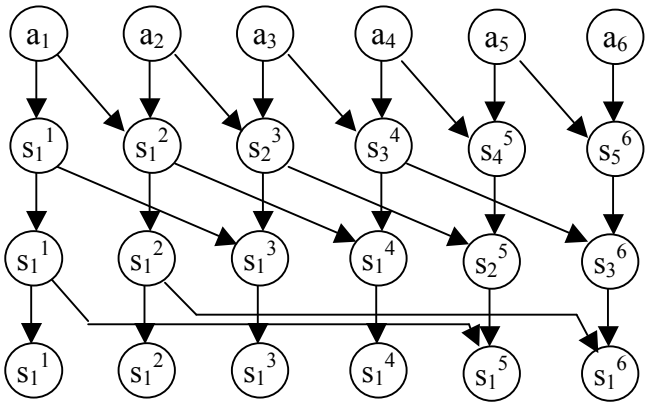


Figura 2.3

Soluţia obţinută ar avea următoarea descriere (varianta 1):

```
int a [n];
process suma [k=1 to n]{
     for [j = 1 to sup(log2 n]{
           if (k-2^(j-1)>=1) a[k] = a[k-2^(j-1)] + a[k];
     barrier;
}
```

Deşi algoritmul pare satisfăcător, el ascunde o potenţială eroare de sincronizare. Să observăm că, la fiecare ciclu, procesele actualizează valorile parţiale a[k]. Pentru un calcul corect, ar trebui ca toate procesele să execute simultan operaţiile de adunare şi apoi toate să facă atribuirile. Deoarece acest lucru impune introducerea unei noi bariere între adunare şi atribuire, cele două acţiuni trebuie despărţite. Acest lucru este posibil prin introducerea unui tablou **temp**, folosit ca depozitar temporar al vechilor valori ale tabloului **a**. Soluţia devine următoarea (varianta 2):

```
int a[1:n], temp[1:n];
process suma [k=1 to n]{
     for [j = 1 to sup(log2 n)]{
          temp[k] = a[k];
          barrier;
          if (k-2^(j-1)>=1) a[k] = temp[k-2^(j-1)] + a[k];
          barrier;
     }
}
```

În fine, putem evita operaţiile de logaritmare şi de ridicare la putere, prin utilizarea unei variabile **d** care păstrează distanţa faţă de poziţia elementului **k**, la care se află elementul cu care acesta se combină (varianta 3):

```
int a[1:n], temp[1:n];
process suma[k:1..n]{
   int d = 1;
   while (d<n){
     temp[k] = a[k];
     barrier;
     if (k-d>=1) a[k] = temp[k-d] + a[k];
     barrier;
     d = 2*d;
   }
}
```

Problemă. Ce modificări trebuie aduse algoritmului pentru a face **temp** locală fiecărui proces?

Putem face două **observaţii** importante:
- în forma finală, algoritmul calculează toate sumele prefix;
- algoritmul poate fi folosit pentru orice operaţie binară asociativă: înmulţirea, maxim, minim, sau logic, şi logic, sau-exclusiv etc.

Descriere algoritmi SIMD

Având în vedere că in algoritmii SIMD toate acţiunile sunt sincronizate (deci orice acţiune ar trebui urmată de o barieră), în descrierea lor eliminăm sincronizările prin bariere. Calculul sumelor prefix poate fi reprezentat astfel (varianta SIMD):

```
int a[1:n];
process suma [k=1 to n]{
   int temp;            /*temp este locala procesului k*/
   for [j = 1 to sup(log2 n)]{
      if (k-2^(j-1)>=1){ temp = a[k-2^(j-1)];
                         a[k] = temp + a[k];
      }
}
```

2.7.2. Operaţii cu liste

Organizarea arborescentă a datelor este mult utilizată pentru motivul că ea conduce la algoritmi de inserare şi căutare avînd complexitate sup(log n). Folosirea paralelismului de date permite atingerea unei performanţe similare chiar şi pentru structuri lineare cum sunt listele. Prezentăm în continuare algoritmul de găsire a sfîrşitului unei liste simplu înlănţuite. Metoda poate fi aplicată şi altor operaţii, cum ar fi: sumele parţiale ale valorilor elementelor din listă, inserarea unui element într-o listă de priorităţi, punerea în corespondenţă a două liste.

Considerăm o listă de pînă la n elemente, păstrate într-un tablou data[1:n], legăturile între elemente fiind păstrate într-un tablou distinct, leg[1:n]. Capul listei este păstrat într-un element separat, cap. Dacă un element i face parte din listă, atunci fie cap = i, fie există un alt element j, cuprins între 1 şi n, astfel încît leg[j] = i. Legatura ultimului element al listei este 0. De asemenea, leg[k] = 0 dacă elementul corespunzător nu face parte din listă.

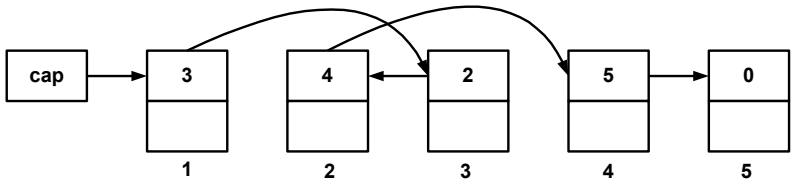


Figura 2.4

Algoritmul secvenţial standard porneşte din capul listei şi parcurge legăturile între elemente, pînă atinge sfîrşitul listei. De aceea, timpul de execuţie este proporţional cu numărul de elemente. În varianta paralelă, folosind tehnica "dublării" din algoritmul sumelor prefixelor, putem aduce timpul la sup(log n). În acest scop, folosim un tablou end[1:n], şi dedicăm un proces separat, Afla(i), pentru calculul fiecărui element end[i], al acestui tablou. Presupunem că lista conţine cel puţin două elemente.

Iniţial, fiecare proces poziţionează end[i] la valoarea lui leg[i]. Valoarea end[i] este actualizată în mod repetat. La fiecare ciclu, dacă end[i] şi end[end[i]] sunt ambele diferite de 0, atunci Afla(i) setează end[i] la valoarea end[end[i]]. Astfel, după un ciclu, legăturile din end indică elemente aflate la două legături distanţă faţă de elementele curente, după două cicluri pe cele aflate la patru legături distanţă etc. După sup(log n) paşi, toate elementele vor indica sfîrşitul listei. Algoritmul este următorul:

```
int leg[1:n], end[1:n];
process Afla [i=1 to n){
   int nou, d=1;
   end[i] = leg[i];
   barrier;
   while (d<n){
     nou = 0;        //tampon pentru legatura urmatoare
     if (end[i]<>0 and end[end[i]]<>0) nou:=end[end[i]];
     barrier;
     if (nou<>0) end[i]:=nou;
     barrier;
     d = 2*d
   }
}
```

În varianta SIMD dispar barierele, iar ciclul şi decizia se contopesc.

```
int leg[1:n], end[1:n];
process Afla [i=1 to n){
   end[i] = leg[i];
   while (end[i]<>0 and end[end[i]]<>0)
       end[i] =end[end[i]];
}
```

Observaţie. Algoritmul se poate aplica şi pentru găsirea rădăcinilor unei păduri de arbori dirijaţi.

Variantă. Considerăm că pentru fiecare nod i, leg[i] indică părintele nodului, într-un arbore al pădurii. De asemenea, dacă r este rădăcina atunci leg[r] = r. În aceste condiţii, algoritmul devine:

```
int leg[1:n], end[1:n];
process Afla [i=1 to n){
   end[i] = leg[i];
   while (end[i]<> end[end[i]])
       end[i] = end[end[i]];
}
```

Combinaţie sume prefix - pointer jumping.
Considerăm că fiecărui nod i, i se asociază o pondere w[i]. Putem calcula, pentru fiecare i, suma ponderilor nodurilor aflate pe calea de la nodul i la rădăcina arborelui din care face parte. Se dă condiţia w[r] = 0 pentru orice rădăcină r.

```
int leg[1:n], end[1:n], w[1:n];
process Afla [i=1 to n){
   end[i] = leg[i];
   while (end[i]<> end[end[i]]){
       w[i] = w[i] + w[end[i]];
       end[i] = end[end[i]];
   }
}
```

Probleme
- Calculul sumelor parţiale ale elementelor unei liste înlănţuite.
- Punerea în corespondenţă a elementelor a două liste. Algoritmul asociază fiecărui element al unei liste un pointer către "prietenul" său din cealaltă listă. Complexitatea cerută este logaritmică.
- Algoritmul de calcul al sfîrşitului unei liste simplu înlănţuite, în varianta în care lista poate avea mai puţin de două elemente.

Capitolul 3 - Complexitatea algoritmilor paraleli şi distribuiţi

Orice proces de proiectare a unui algoritm urmăreşte obţinerea unui rezultat corect şi performant. În cazul algoritmilor paraleli şi distribuiţi, performanţa nu poate fi caracterizată printr-o singură cifră. Ea depinde de categoria de aplicaţii în care este folosit algoritmul, de multe ori performanţa fiind o funcţie de caracteristicile produsului program realizat (timp de execuţie, memorie ocupată, număr de procesoare utilizate, scalabilitate, eficienţă, fiabilitate, toleranţă la defectări, portabilitate), dar şi de costurile implicate de diferitele faze ale ciclului de viaţă (proiectare, implementare şi întreţinere). Ca în orice situaţie în care există mai multe alternative, şi în cazul algoritmilor paraleli alegerea trebuie făcută pe baza înţelegerii costului diferitelor variante. Estimarea costurilor reclamă folosirea unor metrici şi a unor modele de performanţă. Ele permit evaluarea comparativă a soluţiilor, identificarea zonelor de gâtuire, eliminarea ineficienţelor, încă înainte de a investi efort în implementarea lor. Deoarece şi etapa de evaluare trebuie să fie eficientă, modelele alese pentru a fundamenta alegerea soluţiei trebuie să fie un compromis între efortul de modelare şi importanţa îmbunătăţirilor realizate prin consumarea lui.

Recunoscând importanţa tuturor factorilor de performanţă, descrierea care urmează se limitează la o parte a lor, concentrându-se în special pe analiza timpului de execuţie şi a scalabilităţii. Alegerea este determinată de faptul că, de cele mai multe ori, acestea reprezintă aspectele cele mai problematice ale proiectării aplicaţiilor paralele şi distribuite. În analizele concrete pe care un proiectant le face, evaluarea acestor performanţe trebuie realizată în codiţiile înţelegerii contextului dat de ceilalţi factori menţionaţi mai sus.

3.1. Sortarea pe un vector de procesoare

Pentru a discuta măsurile de performanţă asociate algoritmilor paraleli, pornim de la un exemplu simplu: sortarea unui şir de numere folosind un sistem vectorial (un tablou unidimensional de procesoare), ca şi cel prezentat în figura 3.1.

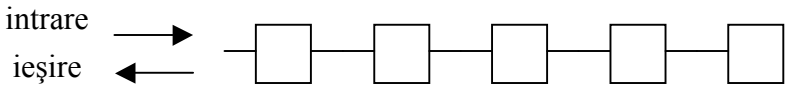


Figura 3.1

Fiecare procesor interior este conectat prin legături bidirecţionale cu vecinii săi, din dreapta şi din stânga. Procesoarele extreme pot avea doar câte o legătură şi pot servi ca puncte de intrare/iesire pentru celelalte procesoare.

Fiecare procesor are o memorie locală şi propriul său program. Sistemul are o funcţionare sincronă: un ceas global comandă execuţia simultană a operaţiilor de către procesoare. La fiecare pas, fiecare procesor realizează operaţiile următoare:

- recepţia unor date de la vecini;
- inspecţia memoriei locale;
- execuţia calculelor specificate de program;
- modificarea memoriei locale;
- transmiterea unor date către vecini.

O astfel de funcţionare se numeşte **sistolică**.

Pentru a sorta o listă de N numere, folosim un vector de N procesoare. Fiecare procesor execută un algoritm compus din doua faze. In fiecare pas al **primei faze**, fiecare procesor realizează următoarele operaţii:
1) acceptă o valoare de la vecinul din stânga;
2) compară valoarea cu cea memorata local;
3) transmite valoarea mai mare vecinului din dreapta;
4) memorează local valoarea mai mică.
Un exemplu este ilustrat în figura 3.2.

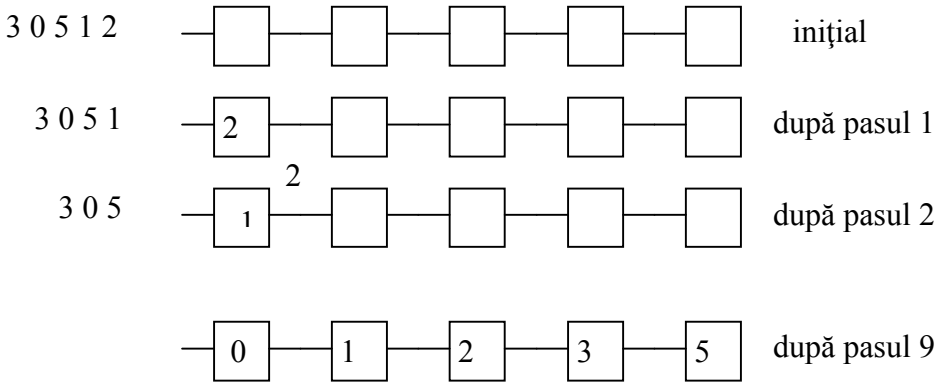


Figura 3.2

Nu este greu de arătat că după 2N-1 paşi, celulele conţin elementele şirului în ordine crescătoare. Astfel, prima celulă examinează N elemente din şir, reţine valoarea cea

mai mică şi transmite restul spre dreapta. A doua celulă, examinează restul de N-1 valori, o reţine pe următoarea mai mică şi transmite restul celorlalte celule. Prin inducţie, rezultă că celula N primeşte valoarea maximă a şirului. Similar, deoarece o valoare ajunge în prima celulă după un pas, în celula a doua după trei paşi ş.a.m.d., rezultă prin inducţie că valoarea maximă ajunge în ultima celula dupa 2N-1 paşi.

In **a doua fază** a algoritmului, valorile ordonate sunt scoase, prin celula din stânga. Există mai multe variante:

1. fiecare procesor începe să transmită spre stânga, imediat ce numerele sunt sortate;
2. cunoscând poziţia sa din vector şi numărând valorile inspectate, fiecare procesor calculează momentul când începe să transmită spre stânga;
3. procesorul din dreapta este special; el începe să transmită spre stânga imediat ce primeşte o valoare; toate celelalte încep să transmită spre stânga imediat ce primesc o valoare dinspre dreapta;
4. fiecare procesor începe să transmită spre stânga imediat ce nu mai primeşte o valoare dinspre stânga.

Din cele patru metode doar ultimele doua nu impun cunoaşterea poziţiei procesoarelor în vector şi contorizarea valorilor. Metoda 3 reclama 4N-3 paşi, în timp ce 4 cere 3N-1 paşi pentru ambele faze de sortare şi de scoatere a rezultatelor.

Exerciţiu. Alcătuiţi descrieri pentru metodele 3 si 4, considerând pe rând modelele de calcul paralel cu memorie comuna şi cu transmitere de mesaje. În primul caz, comunicarea între procese se realizează prin variabile comune special dedicate.

3.2. Măsuri de performanţa

Măsurile obişnuite de apreciere a algoritmilor paraleli sunt:

- **timpul total** T necesar execuţiei şi
- **numarul de procesoare utilizate** P.

In algoritmul prezentat, P = N şi T = O(N).

Observaţie. In secţiunea curentă timpii se referă la cazul cel mai defavorabil.

De multe ori, se compară performanţele atinse de un algoritm cu cele mai bune performanţe ale unui algoritm secvenţial care rezolva aceeaşi problemă. Astfel, dacă G este timpul de execuţie al celui mai rapid algoritm secvenţial, se defineşte **accelerarea** S (speedup) corespunzătoare unui algoritm paralel ca raportul dintre G si T, adică S = G/T. In cazul algoritmului precedent, S= O(N log N)/ O(N) = O(log N).

Ideal ar fi ca algoritmul paralel găsit să fie de P ori mai rapid decât cel mai bun algoritm secvenţial, sau măcar S = O(P). Un exemplu este căutarea unei valori într-un fişier distribuit, folosind maşini SIMD cu acces concurent la citire şi exclusiv la scriere. Fiecare procesor inspectează un sub-fişier de lungime **n/P** în timp **O(n/P)**. Timpul necesar unui algoritm secvenţial pentru căutarea în întreg fişierul de lungime **n**, este **O(n)**. Spunem, în acest caz, că acceleraţia este liniară. Teoretic, nu se poate obtine o acceleraţie mai bună. Astfel, dat fiind un algoritm paralel care se execută în T paşi pe P procesoare, putem oricând folosi un singur procesor pe care procesele să se execute secvenţial în TP pasi. Ca urmare, G <= TP. Semnul **<** folosit aici subliniază faptul că, dacă serializarea unui algoritm paralel este totdeauna posibilă (simularea lui pe o maşină multiprogramată), în schimb, nu există o procedură efectivă de paralelizare a unui algoritm secvenţial (există algoritmi secvenţiali care, prin natura lor, nu pot fi paralelizaţi).

O acceleraţie liniară este realizabilă destul de rar, deoarece:

• împartirea problemei în P subprobleme, fiecare rezolvabilă în 1/P din timpul algoritmului secvenţial corespunzător nu este mereu posibilă;

Să considerăm că într-un algoritm procentajul **f** din totalul calculelor trebuie să se desfăşoare secvenţial. Obtinem:

T = f * G + (1-f) * G / P

sau

S = 1 / (f + (1-f) / P)

Relaţia arată că oricâte procesoare s-ar folosi, acceleraţia nu poate fi mai mare de 1/f (legea lui **Amdahl**). Pe de altă parte, dependenţa lui S de **f** este neliniara, o creştere mică a lui **f** ducând la o scădere importantă a acceleărării S.

• topologia sistemului paralel folosit în rezolvarea problemei impune uneori restricţii de timp.

Astfel, în adunarea a **n** numere folosind **n-1** procesoare dispuse în topologie arborescentă, nu se poate obţine un număr de paşi sub **log n**, câţi sunt necesari pentru propagarea unei valori de la o frunză (intrarea) la rădăcina arborelui (ieşirea). Oricum, G/P este o margine inferioară pentru timpul de execuţie al oricărui algoritm paralel care foloseşte P procesoare.

O alta măsură de performanţă importantă este **costul** C algoritmului. Acesta este definit ca produsul dintre timpul de execuţie şi numărul de procesoare, adică C = TP.

In cazul nostru avem C = O(N^2). Această măsură poate fi folosită în caracterizarea ineficienţei datorate nefolosirii complete a procesoarelor, de către un algoritm paralel.

Mai precis, **eficienţa** E a unui algoritm paralel este raportul E = G/C dintre timpul de lucru al algoritmului secvenţial optim şi costul algoritmului paralel. Avem

E = G/C = G/(TP) = S/P

deci eficienţa poate fi exprimată şi ca raportul dintre accelerare şi numărul de procesoare. In cazul exemplului dat avem E = O((log N)/N).

3.3. Cazul unui număr redus de procesoare

Majoritatea măsurilor de performanţa sunt calculate pentru situaţia ideală în care numărul de procesoare utilizate egaleaza numărul de procese. In exemplul dat anterior, numărul de procesoare utilizate este N, programul de sortare cuprinzând N procese. Ce se întamplă în cazul unui numar mai mic de procesoare?

Există o procedură generală foarte simplă de execuţie a unui algoritm proiectat pentru o reţea de P1 procesoare, pe o reţea de P2 procesoare, unde P2<P1. Metoda constă în simularea a sup(P1/P2) procesoare pe fiecare procesor real. Cea mai simpla variantă este de a plasa pe procesorul **i** procesele

(i-1)*sup(P1/P2)+1, (i-1)*sup(P1/P2)+2,..., i*sup(P1/P2).

Orice algoritm care poate fi executat în T paşi pe P1 procesoare, poate fi executat în T*sup(P1/P2) paşi în cazul a numai P2 procesoare.

3.4. Calculul detaliat al complexităţii

De multe ori, paralelizarea calculului se face pentru atingerea unui timp de execuţie cât mai redus. In legătură cu acest aspect, apare firească întrebarea dacă un anumit algoritm este optim sau nu, deci dacă există sau nu un algoritm cu un timp mai mic de execuţie.

Răspunsul la această întrebare depinde, printre altele, de **modelul de evaluare** adoptat. In analiza facută anterior pentru algoritmul de sortare, paşii algoritmului au fost număraţi în termenii unor operaţii asupra cuvintelor. O alta posibilitate este de a considera **modelul bit**, mai detaliat. O astfel de analiză este justificată de existenţa unor maşini la care paralelismul se manifestă chiar la nivelul operaţiilor asupra biţilor

de date, precum şi de interesul teoretic pe care astfel de măsuri de complexitate îl prezintă.

Punctul cheie în detalierea operaţiilor algoritmului de sortare la nivel de bit îl constituie comparaţiile. O metodă simplă este compararea numerelor bit cu bit, secvenţial, începând cu bitul cel mai semnificativ. A doua metodă foloseşte un arbore binar complet. Numerele sunt comparate bit cu bit, informaţiile rezultate fiind apoi condensate prin împerecheri succesive, până la obţinerea răspunsului final. Procedeul este ilustrat în figura 3.3.

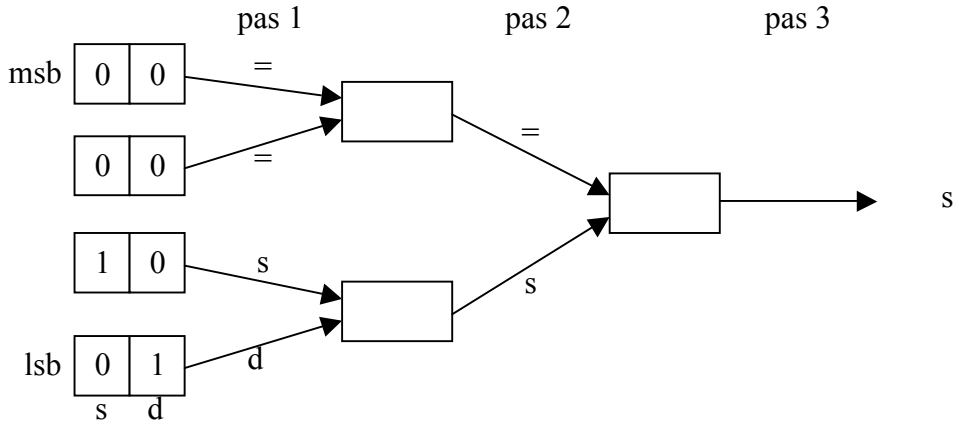


Figura 3.3
(msb – cel mai semnificativ bit; lsb – cel mai puţin semnificativ bit; s- valoarea din stânga; d - valoarea din dreapta)

Arborele binar este superior algoritmului secvenţial, din două motive. Mai întâi, el utilizează log k + 1 paşi pentru a afla rezultatul, în timp ce primul cere k paşi. Apoi, acelaşi arbore poate fi folosit pentru a specifica procesoarelor care dintre cele doua numere trebuie trecut mai departe şi care trebuie păstrat (aşa cum reiese din figura 3.4).

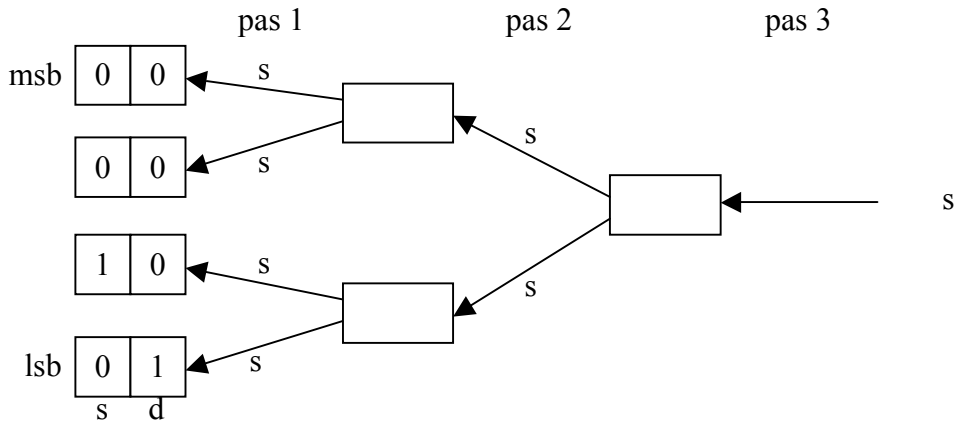


Figura 3.4

Cu aceasta, dupa 2*log k paşi, comparaţia celor două numere este terminată şi cei k biţi ai numarului mai mare pot fi mutaţi simultan spre următoarea celulă. Inlocuind fiecare procesor din vectorul folosit pentru sortare, cu un arbore complet de procesoare de un bit, putem construi o reţea cu (2k-1)N procesoare de un bit, care face (2N-1)*2*log k paşi pentru a executa prima fază a algoritmului.

Există însă un algoritm mai bun de comparare, care, în mod surprinzător, se bazează pe un tablou linear de procesoare şi nu pe un arbore complet. Secretul constă în utilizarea acestuia în linie de asamblare (pipeline).

Ideea de bază este ilustrată în figura 3.5, în care se consideră un vector de trei procesoare pe bit, folosit pentru a găsi cea mai mică dintre valorile 2, 0, 5, 1, 2 (evident, în reprezentarea lor binară).

Vectorul de procesoare face, în mod repetat, compararea valorii memorate cu valoarea primită din stânga, o reţine pe cea mai mică şi o transmite în dreapta pe cealaltă.

Soluţia are următoarele particularităţi:

- execuţia în linie de asamblare face ca în timp ce un procesor compară o pereche de biţi a două numere succesive, procesorul de sub el să lucreze cu biţii mai puţin semnificativi ai perechii de numere anterioare, iar procesorul de deasupra să lucreze cu biţii mai semnificativi ai perechii de numere succesoare;
- la fiecare pas, un procesor primeşte la intrare un bit, iar de la procesorul de deasupra o informaţie asupra comparaţiei făcute de el la pasul anterior (aceasta

comparaţie se referă la biţii mai semnificativi ai aceloraşi numere) - **s** dacă numărul de la intrare este mai mare, **d** dacă numărul memorat este mai mare, = dacă cele două numere sunt egale (în rangurile mai mari decât rangul curent);

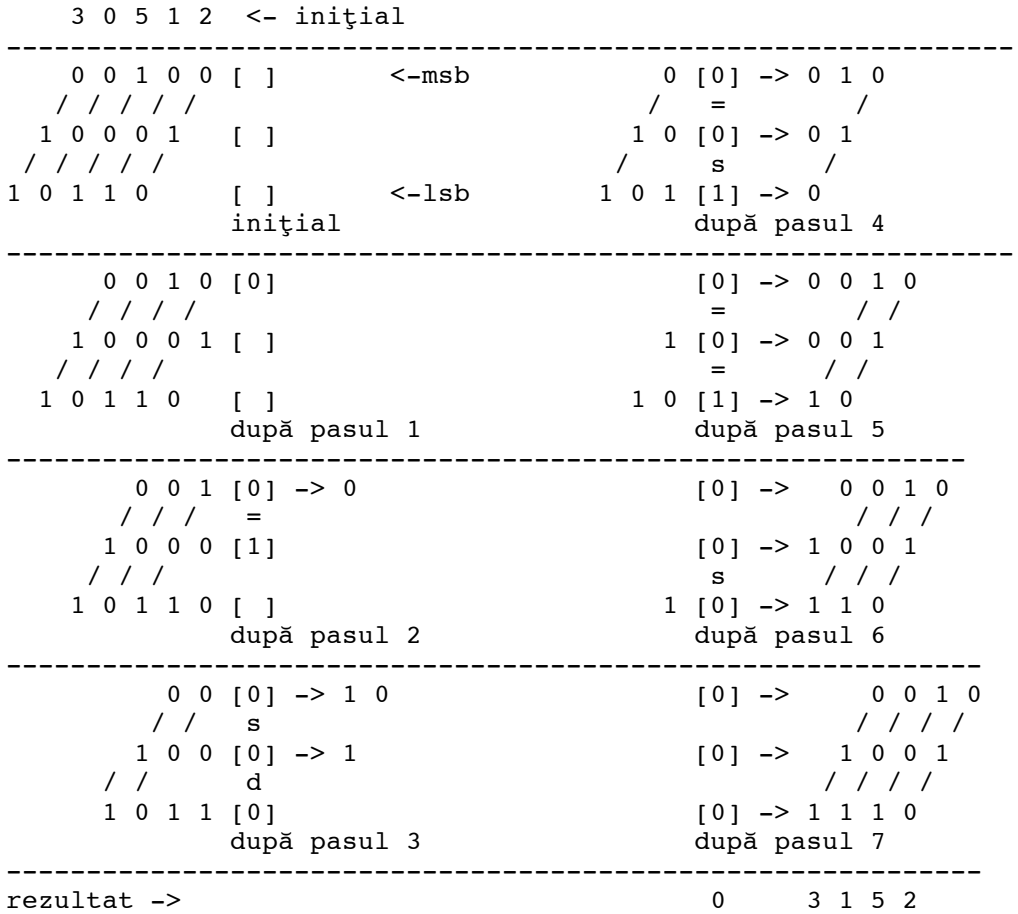


Figura 3.5

- dacă primeşte **s**, procesorul transmite bitul de la intrare la ieşire şi transmite **s** în jos;
- dacă primeşte **d**, procesorul transmite bitul memorat la ieşire, memorează bitul de la intrare şi transmite **d** în jos;
- dacă primeşte =, transmite bitul mai mare la ieşire, memorează bitul mai mic şi transmit în jos **s**, **d** sau = în mod corespunzator;
- când nu mai primeşte nimic la intrare, procesorul scoate în stanga valoarea memorată şi apoi toate valorile primite din dreapta (faza a doua).

Se poate observa că acţiunea vectorului de procesoare pe un bit este similara celei a primului procesor de cuvânt, din algoritmul de ordonare. In ambele cazuri, cea mai mică valoare este pastrată, restul valorilor fiind transmis spre dreapta. Putem, deci, combina **N** vectori de câte **k** biţi, formând astfel o matrice de procesoare, pentru a realiza sortarea celor **N** numere. Prima fază a sortării se poate executa în **(k-1)+(2N-1) = 2N+k-2** paşi, la nivel de bit. A doua fază a sortării se poate derula la fel ca în modelul "cuvânt", ceea ce conduce la sortarea completă a **N** cuvinte de câte **k** biţi în **3N+k-2** paşi cu **N*k** procesoare pe un bit.

3.5. Limite inferioare

Exemplul anterior este util sub două aspecte. Mai întâi, el arată rolul **topologiei** reţelelor de procesoare în atingerea unor performanţe bune în calculul paralel. Ne putem imagina cu uşurinţă că diferitele combinaţii de acţiuni exemplificate aici pentru operaţii foarte simple, pot apare în general, în calculul paralel, deci şi pentru acţiuni de o complexitate mai mare.

In al doilea rând, el ne permite discutarea unor aspecte legate de **optimalitatea** algoritmilor paraleli. Dacă ne referim la algoritmul prezentat în termenii vitezei sau eficienţei, putem spune ca el nu este optimal, deoarece există algoritmi mai rapizi care utilizează chiar mai puţine procesoare. Reţeaua necesară pentru aceşti algoritmi este însă mai complexă decât o matrice **k*N**. Dacă ne **limităm la matrice k*N**, atunci putem afirma că algoritmul este optim, deoarece orice algoritm de sortare executat pe o astfel de configuraţie reclamă un timp **O(k+N)**. Motivele sunt următoarele:

1) Dacă sunt folosite numai **k** procesoare pentru introducerea datelor, sunt necesari **N** paşi numai pentru citirea tuturor celor **N** numere.

2) **Diametrul** unei reţele este distanţa maximă între oricare doua noduri ale sale. Distanţa între două noduri este numărul minim de legături traversate pentru a ajunge de la un nod la celalalt. Pentru o reţea de k*N noduri, diametrul este N+k-2. Diametrul este o indicaţie bună asupra complexităţii, deoarece comunicarea informaţiei între nodurile aflate la o distanţă **d** consumă **d** paşi ai algoritmului.

De exemplu, presupunând că la sortare numerele $x_1 = x_{11}0...01$, $x_2=...=x_N= 010...0$ sunt dispuse iniţial în matricea de procesoare astfel că bitul **i** cel mai semnificativ al numărului **j** este dispus în matrice în poziţia i,j, atunci timpul trebuie să fie cel puţin

k+N-2. Aceasta deoarece bitul cel mai puţin semnificativ al celui mai mare număr este 1 dacă şi numai dacă $x_{11} = 1$, ceea ce înseamnă că informaţia din pozitia 1,1 trebuie să ajungă în poziţia k,N pentru ca sortarea să fie terminată.

3) **Lărgimea bisecţiei** unei reţele este numărul minim de legături care trebuie indepartate pentru a împărţi reţeaua în două subreţele separate, având acelaşi număr de noduri (sau diferenţa de un nod). Această caracteristică este importantă deoarece uneori valorile calculate de o jumatate a reţelei sunt necesare celeilalte jumătăţi.

De exemplu, în sortare, este posibil ca numerele mai mari să fie plasate în jumătatea stângă a matricei de procesoare, iar numerele mai mici în dreapta. Ca urmare, **(k*N)/2** biţi trebuie să treacă din stânga în dreapta reţelei şi invers. Cu o lărgime a bisecţiei de k legături, sunt necesari cel putin **N/2** paşi.

Caracteristicile prezentate trebuie folosite cu grijă în calculul complexităţii. Există exemple care arată că luarea lor în consideraţie fără discernământ poate conduce la aprecieri eronate. Astfel, în [Leighton92] se menţionează cazul sortării a **N** numere de **K** biţi folosind un arbore binar complet. O configuraţie posibilă este prezentată în figura 3.6.

Figura arată conţinutul procesoarelor frunză, după sortare. Procesorul rădăcina are **log N** celule (unde **N** este numărul valorilor şirului de sortat). Procesoarele din nodurile intermediare ale arborelui sunt de o celula. In fine, procesoarele frunză au câte **K** celule (unde **K** este numărul de biţi din reprezentarea unei valori din şirul de sortat).

Diametrul reţelei este **2.log N + 2.K - 2**, iar lărgimea benzii de intrare a datelor este **K.N** (presupunând că toate numerele sunt introduse deodată). In fine, lărgimea bisecţiei este 1, dacă log N = 1 şi 2 în celelalte cazuri.

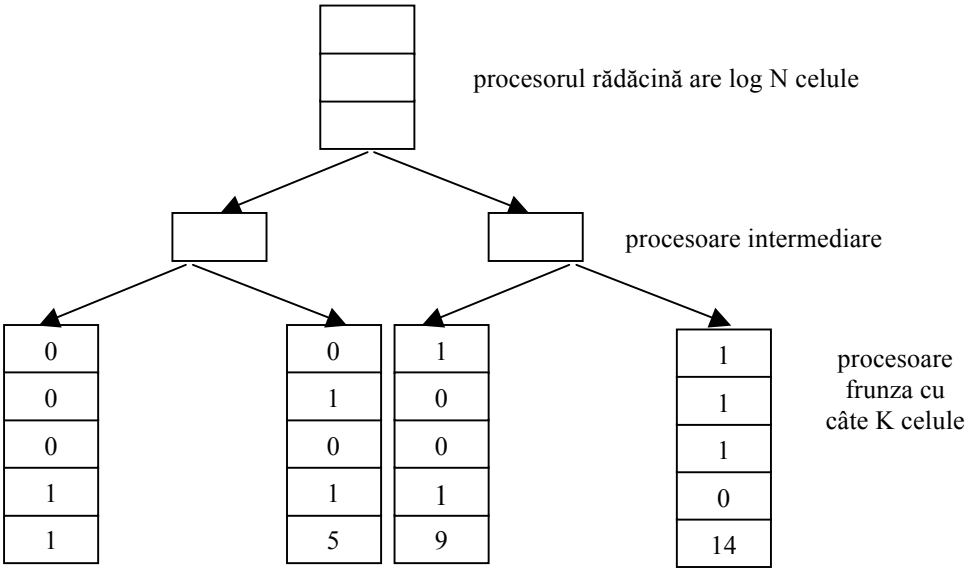


Figura 3.6

Deoarece, în cel mai rău caz, algoritmul trebuie să mute **KN** biţi prin bisecţie, ar trebui ca algoritmul de sortare să solicite **O(KN)** paşi. Cu toate acestea, pentru situaţii în care **k** este mic, rezultatul este cu totul altul. In particular, pentru **k=1**, există un algoritm **O(log N)** care realizează sortarea.

3.6. Proprieţăti ale modelului de evaluare

Exemplele de evaluare prezentate au evidenţiat modul de considerare a timpului de calcul şi de comunicare în analiza algoritmilor paraleli. Sistematizăm în continuare proprietăţile pe care se bazează aceste analize.

Proprietăţile procesoarelor. Fiecare procesor are un control local. Acţiunile lui depind de conţinutul memoriei locale şi de intrările proprii. Complexitatea depinde de nivelul operaţiilor considerate: pe bit sau pe cuvant. Uneori **pachetul** este considerat ca unitate de date, operaţiile uzuale fiind recepţia, memorarea şi transmisia unui pachet.

Proprietăţile interconexiunilor. Presupunerile care se fac uzual despre reţeaua de procesoare sunt următoarele:

- topologia este fixă;

- gradul fiecărui nod este limitat;
- dimensiunea reţelei este o funcţie polinomială (uzual lineară) de dimensiunea problemei.

Proprietăţile protocolului de intrare/iesire. Constrângerea uzuală este ca o intrare este prevazută o singura dată. Complexitatea include astfel şi costul difuzarii intrării, dacă este cazul. In plus, locul şi momentul apariţiei fiecărei intrări şi, corespunzător a fiecărei ieşiri, trebuie specificate în avans (adică înainte de începerea execuţiei algoritmului).

3.7. Modele generale

Cele prezentate anterior evidenţiază faptul că analiza complexităţii algoritmilor paraleli presupune folosirea unui **model formal**, care să permită evaluarea parametrilor de performanţă, cum sunt timpul de execuţie, lucrul, accelerarea etc. Modelul folosit în exemplul dat este particular unei anumite categorii de maşini paralele, SIMD cu memorie locală. Din păcate, acesta nu este singurul model folosit în calculul paralel, deoarece organizarea maşinilor paralele diferă mult mai mult între ele decât cea a maşinilor secvenţiale. Pe de altă parte, folosirea unui model particular, care să surprindă cât mai fidel caracteristicile fiecărei categorii de maşini este nepractică, datorită numărului mare de modele care ar fi necesare. O alternativă este folosirea unui număr mic de modele abstracte, care nu încearcă să reproducă fidel comportarea vreunei maşini paralele particulare, dar au avantajul că algoritmii dezvoltaţi pe baza lor pot fi traduşi cu uşurinţă în programe care se dovedesc eficiente pe maşinile paralele reale.

Modelele paralele generalizează modelul secvenţial RAM, prin introducerea mai multor procesoare. Ele se clasifică în trei categorii, în care este uşor să plasăm arhitecturile de maşini paralele definite în capitolul anterior:

- maşini cu memorie locală (din care face parte şi modelul folosit anterior)
- maşini cu memorie modulară şi
- maşini paralele cu acces aleator la memorie PRAM.

O maşină cu memorie locală cuprinde o mulţime de procesoare, fiecare cu propria sa memorie, ataşate unei subreţele de interconectare. Fiecare procesor poate accesa direct propria memorie şi prin intermediul reţelei de interconectare celelalte memorii. Accesul local se consideră că durează o unitate de timp, accesul distant, la memoria

unui alt procesor depinde de performanţele reţelei de interconectare şi de nivelul traficului celorlalte procesoare.

O maşină cu memorie modulară constă din N procesoare şi M module de memorie, ataşate unei reţele comune. Un procesor accesează un modul de memorie prin transmiterea unei cereri prin reţea. Uzual, timpul de aces al oricărui procesor la oricare modul este uniform, dar depinde de performanţele reţelei şi de "tiparul" operaţiilor de acces la memorie.

O maşină PRAM constă din N procesoare conectate la o memorie comună, partajată. Se consideră că, într-un singur pas, procesele pot avea acces simultan, direct la memoria partajată.

Desigur, modelele sunt foarte generale şi destul de îndepărtate de structura şi funcţionarea maşinilor reale. De exemplu, nici o maşină reală actuală nu se conformează în mod ideal modelului PRAM, realizând într-o unitate de timp, orice acces la memoria partajată. Cu toate acestea, modelele menţionate au fost folosite în dezvoltarea algoritmilor paraleli şi distribuiţi, datorită simplităţii lor şi a uşurinţei de folosire în proiectarea algoritmilor.

3.7.1. Modelul grafurilor orientate aciclice (work-depth)

O alternativă naturală la utilizarea unor modele orientate pe categorii de maşini este folosirea unor modele orientate spre algoritmi. Fără a ţine seama de particularităţile sistemelor de calcul paralele şi distribuite pe care urmează a fi executaţi algoritmii dezvoltaţi, modelele facilitează exprimarea fără constrângeri artificiale a rezolvării problemelor (cum ar fi numărul de procesoare sau topologia sistemului distribuit), exploatând la maximum paralelismul intrinsec al problemelor. O clasă importantă a unor astfel de modele este cea denumită "lucru-adâncmie", la care ne referim în continuare.

Multe calcule pot fi reprezentate în forma unor grafuri (circuite) orientate aciclice, după următoarele reguli:

- fiecare intrare este reprezentata ca un nod fără arce de intrare
- fiecare operatie este reprezentată ca un nod ale cărui arce de intrare provin de la nodurile care reprezintă operanzii
- o ieşire este reprezentată ca un nod fără arce de ieşire.

Un exemplu este prezentat în figura 3.7, care descrie un algoritm de calcul al sumei elementelor unui tablou unidimensional.

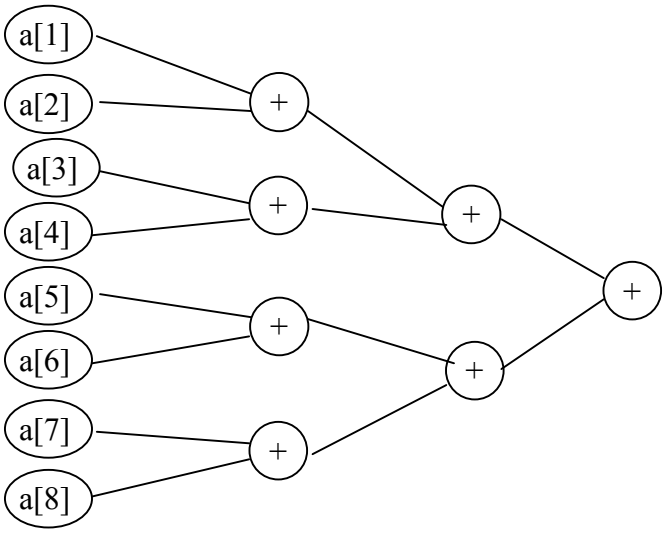


Figura 3.7

Graful descrie operaţiile executate de algoritm şi constrângerile de precedenţă asupra ordinii în care operaţiile sunt executate. Modelul este independent de orice arhitectură şi orice număr concret de procesoare. El pune în evidenţă:

- **lucrul** efectuat de algoritm, adică numărul total de operaţii
- **adâncimea**, adică lungimea celui mai lung lanţ de dependenţe secvenţiale din algoritm.

Raportul lucru / adâncime se numeşte **paralelismul algoritmului**. In exemplul din figura 3.7, lucrul cerut de calculul sumei este de 7 operaţii. Adâncimea este de trei operaţii şi poate fi calculată uşor pornind de la orice frunză şi numărând operaţiile pe calea până la rădăcina arborelui.

In acest caz, generalizarea este foarte simplă: pentru însumarea a n numere sunt necesare **n-1** operaţii, adâncimea fiind de **log n**.

3.7.2. Folosirea modelului pentru calculul complexităţii

Modelul de graf poate fi folosit pentru aprecierea complexităţii algoritmilor paraleli pornind de la forma acestora într-un limbaj de descriere. Să reluăm problema anterioară, a cărei descriere este următoarea:

```
real a[1:n], b[1:n];              /* presupunem n = 2^k */
real s;
process suma [i = 1 to n]{
    for [i = 1 to n] b[i] = a[i];
    for [h = 1 to log n]
        if (i <= n/2^h)
                b[i] = b[2*i-1] + b[2*i];
    if (i==1) s = b[1];
}
```

Descrierea nu conţine nici o mentiune despre numărul de procesoare, sau despre felul în care operaţiile vor fi alocate procesoarelor. In schimb, el poate fi analizat în număr de unităti de timp necesare, în fiecare unitate de timp executându-se în paralel orice număr de operaţii.

Astfel, numărul total de unităţi de timp este log n + 2, deci T(n) = O(log n). In prima unitate de timp sunt executate în paralel n operaţii. In iteraţia h se fac $n/2^h$ operaţii, unde 1 <= h <= log n. In ultima unitate de timp are loc o singura operaţie. Rezultă că lucrul făcut de algoritm este $W(n) = n + \sum_{h = 1, \log n} n/2^h + 1 = O(n)$.

3.7.3. Principiul de planificare pentru PRAM

Deşi numărul modelelor ce ar putea fi utilizate în dezvoltarea algoritmilor paraleli este relativ mare, există, din fericire, deseori tehnici automate de traducere a algoritmilor de la un model la altul. Aceste traduceri conservă lucrul realizat de algoritm, în limitele unui factor constant. Teorema următoare, datorată lui **Brent**, descrie modul de traducere de la modelul **work-depth** la un algoritm PRAM.

Daca se ia în consideraţie numărul de procesoare **p** al unei anumite configuraţii de sistem de calcul, atunci cel mult **p** operaţii pot fi executate în paralel în fiecare unitate de timp. Principiul enunţat în continuare arată că, pentru un algoritm care ruleaza în T(n) unităţi de timp, executând W(n) operaţii, se poate obţine o adaptare a algoritmului care să ruleze pe **p** procesoare PRAM în cel mult **inf(W(n)/p) + T(n)** unităţi de timp (**teorema lui Brent**).

Justificare

Fie $\mathbf{W_i(n)}$ numărul de operaţii executate în unitatea **i** de timp, unde 1 <= i <= T(n). Pentru fiecare **i**, 1 <= i <= T(n), fiecare set de $W_i(n)$ operaţii se execută în $W_i(n)/p$ paşi paraleli, pe cele **p** procesoare. In cazul în care execuţia este reuşită, algoritmul PRAM corespunzător pentru **p** procesoare ia mai puţin de

$\sum_i \sup(W_i(n)/p) <= \sum_i (\inf(W_i(n)/p) + 1) <= \inf(W(n)/p) + T(n)$

paşi paraleli.

Exemplificare

Pentru exemplificare, să considerăm că executăm algoritmul de însumare a elementelor unui tablou pe un sistem PRAM cu **p** procesoare, unde $p = 2^q <= n = 2^k$. Impărţim tabloul A în **p** subtablouri, astfel încât procesorul **s** să fie responsabil de subtabloul **s**, adică de elementele A[l*(s-1)+1] până la A[l*s]. Iniţial, procesorul **s** copiază aceste elemente în B[l*(s-1)+1] până la B[l*s]. La fiecare nivel al arborelui binar, elementele B se înjumătăţesc faţă de nivelul anterior, dar se reîmpart în mod egal între cele **p** procesoare. Numărul de operaţii posibil concurente la nivelul **h** este $n/2^h$. Dacă acest număr este mai mare decât numărul de procesoare $p = 2^q$, adică

$n/2^h = 2^{k-h} >= p = 2^q$

ceeace echivalează cu

$k-h >= q$

atunci operaţiile se împart egal procesoarelor. Altfel, operaţiile sunt executate de cele 2^{k-h} procesoare de indici inferiori.

```
real A[1:n], B[1:n];          /* n = 2^k si p = 2^q */
real S;
int l = n/p;
process suma [s = 1 to p]{
    for [j = l*(s-1)+1 to l*(s-1)+l] B[j] = A[j];
    for [h = 1 to log n]{
       if (k-h >= q)              /*nr op >= nr proc */
       for [j = 2 k-h-q*(s-1) + 1 to 2 k-h-q*s]
           B[j] = B[2*j-1] + B[2*j];
       else
         if (s <= 2 k-h) B[s] = B[2*s-1] + B[2*s];
    }
    if (s = 1) S = B[1];
}
```

Notă. Algoritmul merge pentru p >= n/2.

Pentru a estima timpul de execuţie al algoritmului, observăm că primul pas ia O(n/p) unităţi de timp, deoarece fiecare procesor execută n/p atribuiri. Iteraţia **h** din ciclu ia $O(n/(2^h p))$, deoarece un procesor are de executat cel mult $sup(n/(2^h p))$ operaţii. Ultimul pas ia o unitate de timp. Ca urmare, timpul de execuţie pentru **p** procesoare, $T_p(n)$ este inferior lui

$$n/p + \sum_{h=1, \log n} sup(n/(2^h p)) <= n/p + \sum_{h=1, \log n} (inf(n/(2^h p)) + 1) <=$$
$$<= n/p + \sum_{h=1, \log n} inf(n/(2^h p)) + \log n <= 2n/p + \log n$$

şi deci $T_p(n) = O(n/p + \log n)$, ceeace corespunde principiului planificării.

3.7.4. Lucru şi cost

Cele doua noţiuni sunt corelate între ele. Dat fiind un algoritm de lucru W(n) şi de timp T(n), algoritmul poate fi simulat pe **p** procesoare PRAM într-un timp dat de formula:

$$T_p(n) = O(W(n)/p + T(n))$$

Costul corespunzător (produsul timp*numar_procesoare) este atunci egal cu

$$C_p(n) = T_p(n)\, p = O(W(n) + p\, T(n))$$

In general, costul este mai mare decât lucrul unui algoritm. Aceasta se datorează utilizării parţiale a procesoarelor pe parcursul execuţiei unui algoritm paralel. Astfel, pentru algoritmul precedent, considerând că se utilizează **n** procesoare, cel mult jumătate pot fi active la prima execuţie a ciclului **fa** exterior, cel mult un sfert rămân active la a doua execuţie a ciclului ş.a.m.d. Abordarea prezentată presupune utilizarea procesoarelor libere pentru alte sarcini.

Există, bineînţeles, anumite **cerinţe** pentru realizarea unor algoritmi PRAM eficienţi. Astfel, numărul de procesoare **p** trebuie să fie mai mic decât **n** (dimensiunea problemei) şi adaptiv (dependent de dimensiunea problemei). Uzual, se consideră că p(n) este satisfăcător dacă este o funcţie subliniara de **n**, de exemplu rădăcina pătrată din **n**. In orice caz, algoritmul să nu depindă de numărul de procesoare.

Timpul de execuţie $T_p(n)$, dependent de dimensiunea problemei, să fie mic, semnificativ mai mic decât cel al variantei secvenţiale. De asemenea, el să fie adaptiv (faţă de dimensiunea problemei) şi invers proporţional cu numărul de procesoare.

In fine, costul să fie minim. De multe ori se compară costul algoritmului paralel cu costul celui mai bun algoritm secvenţial care rezolvă aceeaşi problemă. Dezvoltarea unor algoritmi cost-optimali va fi prezentată într-o altă secţiune.

3.7.5. Implementare pentru reţele de procesoare

In cazul implementării algoritmilor paraleli pe reţele de procesoare, există mai multe posibilităţi de alegere a topologiei: liniară, inel, arbore, matrice, cub, hipercub. De regulă, folosirea unor topologii diferite conduce la rezolvări diferite ale aceleiaşi probleme. Transpunerea soluţiei pentru un hipercub este deosebit de simplă, datorită capacităţii acestuia de adaptare la diferite topologii, printre care şi aceea de arbore.

Un hipercub constă din $p = 2^d$ procesoare, indexate de la 0 la p-1, interconectate intr-un cub boolean d-dimensional, definit după cum urmează. Fie reprezentarea binară a lui **i**, 0 <= i <= p-1, de forma $i_{d-1}i_{d-2}\ldots i_0$. Două procesoare sunt conectate dacă indicii lor diferă doar într-o singură pozitie a reprezentărilor lor binare. De exemplu, într-un cub 4-dimensional, nodul 0 (0000) este conectat cu nodurile 1 (0001), 2 (0100), 4 (0100) si 8 (1000).

Hipercubul are o structură recursivă. Se poate extinde un cub d-dimensional la unul (d+1)-dimensional, prin dublarea primului şi prin realizarea legăturilor între nodurile corespunzătoare din cele doua cuburi d-dimensionale. Indexul nodurilor din primul cub se extinde prin adăugarea cifrei semnificative 0, iar indexul nodurilor din cel de al doilea prin adăugarea bitului semnificativ 1.

Hipercubul este popular datorită regularităţii structurii, diametrului redus (egal cu log p) şi datorită abilităţii de a manevra simplu şi rapid multe calcule.

Pentru exemplul pe care îl rezolvăm aici, fiecare element A[i] al tabloului cu **n** elemente este memorat într-un nod P[i] al hipercubului cu **n** noduri ($n=2^d$). Algoritmul de calcul al sumei consta din d iteraţii şi memorează rezultatul în P[0]. Primul pas constă în însumarea elementelor care se află în noduri ai căror indecşi diferă prin cifra cea mai semnificativă. Valorile sunt memorate în cubul (d-1)-dimensional cu nodurile având indecşii cu cifra cea mai semnificativă egală cu zero. In cazul în care n=8, se realizează următoarele calcule:

A[0] = A[0] + A[4]
A[1] = A[1] + A[5]
A[2] = A[2] + A[6]
A[3] = A[3] + A[7]

Celelalte iteraţii continuă după reguli similare. Pentru exemplul ales avem:
A[0] = A[0] + A[4] + A[2] + A[6]
A[1] = A[1] + A[5] + A[3] + A[7]
iar în ultimul pas, în A[0] obţinem întreaga sumă.

In algoritmul următor, i(l) denotă indexul obţinut din **i** prin complementarea bitului **l**. Operaţia A[i] := A[i] + A[i(l)] se face în doi paşi: în primul pas, procesorul P[i] copiază valoarea A[i(l)] preluată de la procesorul P[i(l)] pe legătura dintre cele două; în al doilea pas, P[i] face adunarea şi memorează suma in A[i].

```
real A [0:n-1];                  /* n = 2^d */
process suma [i = 0 to n-1]{
    for [l = d-1 to 0]          /*ciclul se repeta de d=log n ori*/
     if (i <= 2^l – 1)          /*i <=3 cand l = 2*/
          A[i] = A[i] + A[i(l)];
}
```

Algoritmul se termină în O(log n) unităţi de timp, la fel ca şi implementarea PRAM.

Problemă. Descrieţi algoritmul de însumare pentru cazul în care numărul de procesoare **p** este mai mic decât numărul de elemente **n** şi fiecare nod memorează **n**/**p** elemente ale tabloului A.

Capitolul 4. Dezvoltarea algoritmilor folosind variabile partajate

Una din problemele centrale ale dezvoltării algoritmilor folosind variabile partajate este **sincronizarea**, în cele două forme cunoscute: excluderea mutuală şi sincronizarea condiţionată. Prima formă înseamnă gruparea mai multor acţiuni ale unui proces într-o singură acţiune atomică, ascunzând astfel stările intermediare. Cea de a doua formă presupune întârzierea execuţiei unui proces până la îndeplinirea unei anumite condiţii.

Nu este, deci, de mirare că în istoria algoritmilor cu variabile partajate s-a consumat un efort considerabil pentru găsirea unor mecanisme de rezolvare sistematică a problemei sincronizării, câteva rezultate notabile fiind reprezentate de:

- semafoare,
- regiuni critice,
- monitoare.

Câteva dintre problemele "clasice" rezolvate prin utilizarea acestora sunt:

- producători-consumatori,
- cititori-scriitori,
- problema filozofilor,
- problema bărbierului.

Problemele mentionate isi au izvorul in domeniul sistemelor de operare dar sunt utilizate astazi in multe alte domenii care pun accent pe paralelizarea codului. Fiecare din soluţiile găsite îşi are importanţa sa deosebită, toate la un loc alcătuind o "bibliotecă" de modele de rezolvare aflate la dispoziţia proiectanţilor. Înglobarea lor în soluţiile diferitelor probleme nu este întotdeauna o sarcină uşoară, dacă se urmăreşte obţinerea unor variante eficiente.

În acest capitol prezentăm soluțiile câtorva probleme clasice utilizand mecanismul **semafoarelor**. Acest mecanism foloseste o variabila cu un set limitat de valori intregi. Valoarea unui semafor poate fi modificata doar prin intermediul a doua operatii **primitive**, notate P si V, motiv pentru care variabilei i se asociaza un tip special, notat aici **sem**. Primitivele P si V pot fi definite folosind notatiile de sincronizare si atomicitate prezentate intr-un capitol anterior:

sem bin = 1 ;
P(s) :: <**await** (bin>0) bin = bin-1;>
V(s):: <bin = bin+1;>

Semaforul **bin** poate avea doar valorile 0 sau 1, motiv pentru care este numit **semafor binar**. Operatia P(bin) se poate executa doar daca bin=1; prin executia sa, valoarea lui **bin** devine 0. Operatia V nu include conditionari, rolul ei fiind cresterea valorii lui **bin** cu o unitate. Asa cum se va vedea in exemplele urmatoare, programele sunt structurate astfel incat oricare proces sa poata executa o primitiva V doar **dupa** executia unei primitive pereche P (cu alte cuvinte, doar cand valoarea semaforului este 0).

Valoarea initiala a semaforului poate fi mai mare ca 1, caz in care avem de a face cu un **semafor general**.

sem gen = 10 ; /* valoarea initiala a lui **gen** este 10 */
P(s) : <**await** (gen >0) gen = gen -1;>
V(s): < gen = gen +1;>

Teoretic, un semafor general poate avea orice valoare intreaga ne-negativa. Practic, valoarea semaforului **gen** nu coboara sub 0 si (prin programarea corespunzatoare a primitivelor P si V) nu depaseste valoarea initiala (care este, deci, valoarea maxima a semaforului).

Alte detalii despre semafoare si utilizarea lor sunt prezentate in exemplele ce urmeaza.

4.1 Sectiuni critice

Fiecare proces Proc[i] al unei colectii de procese Proc[i=1 to n] executa ciclic operatii intr-o sectiune critica in care are acces exclusiv la anumite resurse partajate de toate procesele, urmata de o sectiune necritica in care foloseste doar resurse locale. Problema ce trebuie rezolvata este asigurarea ca, in orice moment, cel mult un proces sa se afle in sectiunea critica. Cu alte cuvinte, trebuie controlata „intrarea" in si „iesirea" din sectiunea citica astfel incat sa se asigure excluderea mutuala a proceselor la executia sectiunilor critice.

Primitivele P si V asigura, intr-o forma simpla, accesul exclusiv la resursele partajate. Pentru asta, putem folosi un semafor **mutex** a carui valoare sa specifice daca resursele sunt libere (mutex=1, nici un proces nu este in sectiunea critica) sau ocupate (mutex=0, un proces este in sectiunea critica). Astfel, intrarea unui proces in sectiunea critica trebuie conditionata de executia cu succes a unei primitive P(mutex). La iesirea din sectiunea critica, procesul trebuie sa execute o primitiva V(mutex).

```
sem mutex = 1;
        /*denumirea mutex provine de la mutual exclusion */
process P[i=1 to n]{
   while (true){
      P(mutex);                          /*acaparare sectiune critica*/
      Sectiune critica;
      V(mutex);                          /*eliberare sectiune critica*/
      Sectiune necritica;
   }
}
```

Pentru intelegerea functionarii, este important sa ne amintim ca, prin definitie, operatiile P si V sunt atomice (doar una din ele se poate executa la un moment dat). Astfel, daca mutex=1 si mai multe procese vor sa intre in sectiunea critica, primul care executa primitiva P gaseste **mutex** la valoarea 1, o modifica la 0 si continua cu executia sectiunii critice. Executia celorlalte procese este suspenadata pana cand procesul aflat in sectiunea critica termina si pune **mutex** din nou pe 1.

4.2 Producator si consumator

Problema: se considera un proces Producator si un proces Consumator care comunica printr-un singur tampon partajat. Un producator pune o valoare in tampon, iar un consumator ia o valoare din tampon. Trebuie asigurat ca valorile nu sunt suprascrise si nu sunt citite din tampon de mai multe ori.

Si in acest caz avem de a face cu procese care folosesc o resursa comuna, tamponul prin care se comunica valorile intre ele. Evident ca accesul trebuie sa fie exclusiv (la un moment dat, un singur proces are acces la tampon). Doar ca procesul Producator trebuie sa aiba acces cand tamponul este gol, in timp ce procesul consumator trebuie sa aiba acces cand tamponul este plin. Un singur semafor, care ar permite sau nu accesul, nu este suficient. In schimb, folosirea a doua semafoare, **gol** pentru accesul la tampon gol si **plin** pentru accesul la tampon plin, ar rezolva problema. In aceasta varianta, procesul Producator ar intra in sectiunea critica daca primitiva P(gol) este executata cu succes, iar procesul Consumator intra in seciunea critica daca P(plin) este executata cu succes. La iesirea din sectiunea critica, procesul Producator trebuie sa anunte, prin V(plin), ca tampolul a fost umplut, iar procesul Consumator trebuie sa anunte, prin V(gol), ca tamponul a fost golit.

```
typeT buf;
```

```
sem gol = 1, plin = 0;        /* sem. binare */

process Producator (i= 1 to M){
          typeT v;
          while (true){
                v = produce ();
                P(gol);
                buf = v;
                V(plin)
          }
}

process Consumator [i= 1 to N){
          typeT w;
          while (true){
                    P(plin);
                    w := buf;
                    V(gol);
                    consuma (w);
          }
}
```

Semafoarele **gol** si **plin** sunt **binare** si asigura excluderea mutuala a proceselor, in sensul ca, la un moment dat, un singur proces (fie Producator, fie Consumator) depune sau extrage o valoare din buf. Valorile initiale ale semafoarelor marcheaza absenta unei valori din **buf** (gol=1, plin=0). Ca urmare, un proces Producator va putea pune o valoare in **buf**, modificand semafoarele pentru a marca prezenta unei valori in **buf** (gol=0, plin=1). Ca urmare, un proces Consumator va putea prelua valoarea, golind astfel tamponul **buf** si modificand semafoarele pentru a marca golirea tamponului (gol=1, plin=0).

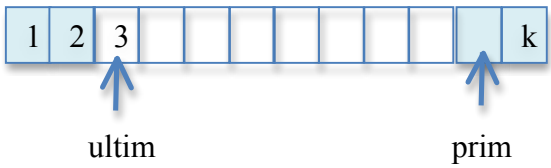
Din aceasta descriere, trebuie sa remarcam ca cele doua semafoare **gol** si **plin** nu sunt independente unul de altul, in orice moment cel mult unul din ele putand avea valoarea 1. In plus, atunci cand unul din procese este in sectiunea critica ambele semafoare au valoarea 0. Un astfel de grup in care suma tuturor semafoarelor nu depaseste niciodata valoare 1 se numeste **semafor binar splitat** (split binary semaphore). Regulile de baza ale unui semafor binar splitat sunt urmatoarele:
- initial, doar unul din semafoare are valoarea 1, restul zero;
- oricare proces care utilizeaza grupul de semafoare executa mai intai primitiva P pe un semafor din grup, doar dupa aceea executa primitiva V pe oricare semafor din grup;

- codul dintre primitivele P si V se va executa in excludere mutuala, toate semafoarele avand valoarea 0.

4.3 Comunicarea producator si consumator prin tampon limitat

Aceasta varianta foloseste drept tampon un tablou, care permite pastrarea mai multor valori la un moment dat. Cerintele sunt pastrate, adica trebuie asigurat ca valorile nu sunt suprascrise si nu sunt citite din tampon de mai multe ori. Se adauga conditia, care la varianta precedenta era indeplinita implicit, ca ordinea in care valorile sunt consumate sa fie cea in care valorile sunt produse.

De data aceasta, semafoarele **gol** si **plin** trebuie sa fie generale. Semaforul **gol** numara locurile goale din **buf**, iar **plin** numara valorile existente in **buf**. In plus, pentru a nu limita numarul valorilor transmise, alegem un tampon circular de dimensiune k.



Valorile sunt plasate in tampon in ordinea elementelor tabloului, de la 1 la k circular (dupa elementul k urmeaza 1). Primul element din tampon va fi indicat de o variabila **prim**, iar primul loc gol prin variabila **ultim**. Initial, ambele variabile au valoarea 1.

In functionare, procesul **Producator** acapareaza un loc gol (daca nu exista atunci se blocheaza pana se elibereaza unul) si depune o valoare. Pozitia curenta pentru **Producator** este desemnata de variabila **ultim**.

Procesul **Consumator** acapareaza o valoare depusa (daca nu exista atunci se blocheaza pana apare una) si preia valoarea din **buf**. Pozitia curenta pentru **Consumator** este desemnata de variabila **prim**. Operatiile de depunere si de extragere nu se exclud mutual. Procesul **Producator** poate depune o valoare in pozitia **ultim** din **buf** in timp ce **Consumator** preia una din pozitia **prim** din **buf**.

```
typeT buf [1:k];
sem gol= k; plin= 0;    /*sem. generalizate*/

process Producator {
```

```
      typeT v;
      int ultim = 1;    /*pozitia goala in tampon*/
      while (true){
            v = produce ();
            P(gol);                /*exista locuri goale?*/
            buf[ultim] = v; ultim = ultim mod k + 1;
            V(plin);
      }
}

process Consumator {
      typeT w;
      int prim = 1;  /*primul element din tampon*/
      while (true){
            P(plin);              /*exista valori in buf?*/
            w = buf[prim]; prim = prim mod k + 1;
            V(gol);
            consuma (w);
      }
}
```

Solutia tamponului circular este utila in situatii in care producerea, respectiv consumul valorilor se face in rafale: producatorul poate furniza mai multe valori fara a astepta preluarea imediata a fiecarei valori de catre consumator; la randul sau, consumatorul poate prelua valorile depuse in tampon in timp ce producatorul executa alte operatii de calcul.

4.4 Mai multi producatori si mai multi consumatori

Aceasta varianta este o extindere a solutiei anterioare, ea incluzand mai multi producatori si mai multi consumatori care partajeaza tamponul circular. Trebuie asigurat ca valorile nu sunt suprascrise si nu sunt citite din tampon de mai multe ori. De asemenea, ordinea in care valorile sunt consumate trebuie sa fie cea in care valorile sunt produse.

Sa nu uitam ca fiecare producator vrea sa plaseze o valoare produsa, in primul loc liber al tamponului. Asta cere ca variabila care indica acest loc (**ultim**) sa fie partajata intre producatori, iar accesul producatorilor sa fie exclusiv. Pentru a gestiona excluderea intre producatori vom folosi un semafor **mutexP**.

Aceeasi situatie o gasim la consumatori, care vor avea acces la variabila comuna care indica prima valoare din tampon (**prim**), excluderea mutuala fiind gestionata prin semaforul **mutexC**.

```
typeT buf[1:k];
sem gol = k, plin = 0;          //sem. generalizate
int prim = 1, ultim = 1;
sem mutexP = 1, mutexC = 1;

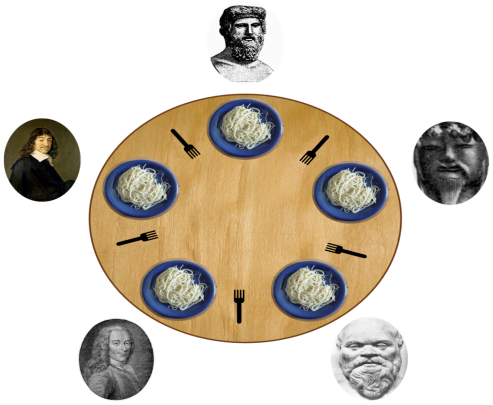
process Producator[i = 1 to M){
     typeT v;
     while (true){
          v = produce ();
          P(gol);
               P(mutexP);
               buf[ultim]=v; ultim = ultim mod k + 1;
               V(mutexP);
          V(plin);
     }
}

process Consumator[i = 1 to N){
    typeT w;
    while (true){
         P(plin);
              P(mutexC);
              w := buf[prim]; prim := prim mod k + 1;
              V(mutexC);
         V(gol);
         consuma (w);
     }
}
```

4.5 Problema filozofilor

Cinci filozofi stau in jurul unei mese rotunde si isi petrec timpul gandind si mancand. In mijlocul mesei este o farfurie cu spaghete. Pentru a putea manca, un filozof are nevoie de doua furculite. Pe masa sunt cinci furculite, fiecare situate intre doi filozofi vecini. Regula este ca fiecare filozof poate folosi furculitele din imediata sa apropiere. Problema este sa se scrie un program care simuleaza comportarea

filozofilor. In particular, trebuie evitata situatia in care nici un filozof nu poate acapara ambele furculite (de exemplu, ia furculita din stanga sa si nu ii mai da drumul).



Daca toti filozofii ar lua mai intai furculita din stanga si apoi pe cea din dreapta se poate ajunge la blocare definitiva prin asteptare circulara: fiecare asteapta o resursa pe care procesul care a acaparat-o nu o elibereaza. Solutia ca sa se evite asteptarea circulara este ca unul din filozofi sa procedeze altfel si anume sa ia mai intai furculita din dreapta si apoi pe cea din stanga.

```
sem f[1:5] = ([5] 1);
process Filozof [i=1 to 4){
while (true){
      P(f[i]); P(f[i+1]);
      mananca;
      V(f[i]); V(f[i+1]);
      gandeste;}
}
process Filozof[5]{
while (true){
      P(f[1]); P(f[5]);
      mananca;
      V(f[1]); V(f[5]);
      gandeste;}
}
```

4.6 Problema cititorilor si scriitorilor

Problema: exista doua tipuri de procese, cititori si sciitori care impart o resursa. Mai multi cititori pot avea acces simultan la resursa comuna daca nici un scriitor nu o modifica in acelasi timp. Un scriitor poate modifica resursa daca ea nu este accesata de nici un alt proces, fie el scriitor sau cititor.

Algoritmul foloseste un semafor **rw** pentru excluderea mutuala intre procese. El asigura ca in orice moment, avem una din urmatoarele situatii:
- resursa nu este utilizata;
- resursa este utilizata de un singur scriitor;
- resursa este utilizata de unul sau mai multi cititori.

Semaforul este folosit in modul clasic de scriitori. Un scriitor poate intra in sectiunea critica daca operatia P(rw) se executa cu succes. La iesirea din sectiunea critica procesul scriitor executa o operatie V(rw) si elibereaza resursa.

Semaforul **rw** este gestionat putin mai complicat de cititori, deoarece resursa poate fi folosita simultan de mai multi cititori. Daca resursa este libera, un cititor o poate acapara daca operatia P(rw) reuseste. Alte procese cititor se pot alatura si pot intra in sectiunea critica fara a mai modifica semaforul **rw**. De asemenea, procesele cititor pot parasi sectiunea critica fara a modifica semaforul. Doar ultimul cititor care paraseste sectiunea critica trebuie sa elibereze resursa executand un V(rw). Pentru a determina care procese cititor executa operatiile P si V, algoritmul tine evidenta numarului de cititori, **nr** care folosesc resursa. Deoarece **nr** este partajata de cititori si operatiile asupra lui trebuie executate in regim de exclusivitate (un cititor la un moment dat), acesul la aceasta variabila este controlat de un semafor de excludere mutuala **mutexR**.

```
sem rw = 1, mutexR = 1;
int nr = 0;

process Cititor [i = 1 to m){
while (true){
     P(mutexR);
          nr = nr+1;
          if (nr = 1) P(rw);  /*daca primul cititor*/
     V(mutexR);
     citeste din resursa comuna;
     P(mutexR)
          nr = nr-1;
```

```
          if (nr = 0) V(rw);   /*daca ultim cititor*/
     V(mutexR);}
}

process Scriitor (j = 1 to n){
while (true){
     P(rw);
     scrie in resursa comuna;
     V(rw);}
}
```

Solutia acorda preferinta cititorilor: daca un cititor acapareaza resursa, alti cititori pot sa i se alature si sa citeasca. Doar cand ultimul cititor a terminat, resursa este eliberata (V(rw)). Teoretic, procesele scriitor pot fi intarziate oricat de mult (termenul folosit este **starvation** – infometarea scriitorilor), in detrimentul actualizarii prompte a valorii resursei partajate.

Solutia pe care o prezentam in continuare asigura ca:
(1) noile cereri de la cititori sunt intarziate (cititorul este pus in asteptare) daca un scriitor este in sectiunea critica sau asteapta sa intre in sectiunea critica;
(2) un cititor este trezit din asteptare doar daca nu exista un scriitor in asteptare sau in sectiunea critica.

Pentru a aplica aceasta politica, avem nevoie de variabile de contorizare a cititorilor si scriitorilor care folosesc resursa sau sunt in asteptarea accesului la resursa:
nw - numarul de scriitori care folosesc resursa,
nr - numarul de cititori care folosesc resursa,
dw - numarul de scriitori care asteapta sa foloseasca resursa,
dr - numarul de cititori care asteapta sa foloseasca resursa.

Cu aceste notatii, conditiile anterioare pot fi specificate astfel:
(1) nw>0 or dw>0
(2) nw=0 and dw=0.
Similar, un scriitor este intarziat daca un alt scriitor sau un cititor se afla deja in sectiunea critica, adica:
nr>0 or nw>0.

Pentru controlul accesului la sectiunile critice conform acestor conditionari, definim urmatorul grup de semafoare:

e – semafor de intrare – controleaza intrarea intr-o actiune atomica si faciliteaza astfel executia mai multor operatii ca o singura actiune atomica;
r – semafor pentru intarzierea cititorilor (asteapta ca nw=0);
w – semafor pentru intarzierea scriitorilor (asteapta ca nw=0 si nr=0).
In plus este nevoie de executia mai multor operatii ca o singura actiune atomica folosind in acest scop un semafor care cotroleaza intrarea intr-o astfel de actiune:

Cele trei semafoare alcatuiesc impreuna un **semafor binar splitat**. Tehnica de executie a primitivelor, numita "pasarea stafetei", se bazeaza pe urmatoarele reguli:
- initial un semafor este 1 si un proces poate prelua stafeta printr-o primitiva P asupra acestuia;
- cand un proces detine stafeta (se afla intr-o sectiune critica si toate semafoarele sunt 0), el poate executa o operatie V asupra unuia din cele trei semafoare, pasand stafeta unui alt proces, care “asteapta” la acel semafor (intr-o operatie P).

```
int nr = 0;     /*nr cititori care folosesc resursa */
int nw = 0;     /*nr scriitori care folosesc resursa */
int dr = 0;     /*nr cititori intarziati */
int dw = 0;     /*nr scriitori intarziati */
sem e = 1;      /*intrare sectiune atomica */
sem r = 0;      /*intarzie cititori daca nw>0 sau dw>0*/
sem w = 0;      /*asteapta ca nw=0 si nr=0*/

process Cititor [i = 1 to m]{
     while (true){
          P(e);
          if (nw>0 or dw>0){
             dr = dr+1; V(e); P(r);
          }
          nr = nr+1;
          if (dr>0) {dr = dr-1; V(r);}
          else if (dr=0) V(e);
          }
          citeste din resursa comuna;
          P(e);
          nr = nr-1;
          if (nr = 0 and dw>0){dw = dw-1; V(w);}
          else if (nr>0 or dw=0) V(e);
          }
     }
}
```

```
process Scriitor [j = 1 ton){
     while (true){
          P(e);
          if (nr>0 or nw>0){dw = dw+1; V(e); P(w);}
          nw = nw+1;
          V(e);
          scrie in resursa comuna;
          P(e);
          nw = nw-1;
          if (dr>0 and dw=0){dr = dr-1; V(r);}
          else if (dw>0){dw = dw-1; V(w);}
          else if (dr=0 and dw=0) V(e);
          }
     }
}
```

Capitolul 5. Dezvoltarea aplicaţiilor pentru calculatoare SM SIMD

O clasă importantă de sisteme paralele este cea care corespunde modelului SIMD cu memorie partajată (Shared Memory SIMD). Ele sunt cunoscute şi sub denumirea de PRAM (Parallel Random Access Machines). În funcţie de modul de acces la memorie pentru operaţiile de citire şi scriere, sistemele SM SIMD se subdivid în:
- EREW (Exclusive Read Exclusive Write);
- CREW (Concurrent Read Exclusive Write);
- ERCW (Exclusive Read Concurrent Write);
- CRCW (Concurrent Read Concurrent Write).

Descrierea urmăreşte prezentarea unor metode generale de dezvoltare a algoritmilor paraleli (partiţionarea, divide et impera) şi a unei abordări posibile pentru obţinerea algoritmilor cost-optimali.

Câteva **caracteristici** importante ale algoritmilor SM SIMD sunt următoarele:
- algoritmii indică acţiunile executate de procesoare (în general, fiecare procesor este dedicat execuţiei acţiunilor unui proces)
- algoritmii explicitează separat acţiunile de citire şi cele de scriere (pentru a putea aprecia costul operaţiilor în raport cu categoriile de sisteme PRAM menţionate)
- lansarea şi terminarea proceselor au "overhead" redus (instrucţiuni **co ... oc** pot fi incluse în cicluri)
- algoritmii se încadrează, de obicei, în clasa "fine grain" (granularitate fină).

Aceste caracteristici sunt subliniate prin notaţia specifică PRAM, care a fost prezentată în capitolul 2 al lucrării.

5.1. Proprietăţile (dorite ale) algoritmilor paraleli

Număr de procesoare

În cele ce urmează, **n** reprezintă dimensiunea problemei, iar **p(n)** este numărul de procesoare utilizate la execuţia algoritmului. Numărul de procesoare trebuie să îndeplinească următoarele cerinţe:
- **p(n)** trebuie sa fie mai mic dacât **n**

  Este nerealist să presupunem că avem **n** procesoare. Uzual se ia **p(n)** ca funcţie sublineară de **n** (de exemplu, **log n** sau **sqrt(n)**)
- **p(n)** trebuie să fie adaptiv

  Algoritmul nu trebuie să depindă de un număr fix de procesoare.

Timp de execuţie
În cele ce urmează, **t(n)** este timpul de execuţie în funcţie de dimensiunea problemei. Condiţiile sunt următoarele:
- **t(n)** să fie mic,
  semnificativ mai mic decât timpul de execuţie al variantei secvenţiale
- **t(n)** sa fie adaptiv,
  şi să fie invers proportional cu **p(n)**

Cost
În cele ce urmează, **c(n)** este costul algoritmului c(n) = t(n) * p(n). Condiţiile sunt următoarele:
- **c(n)** să fie minim
  deci **algoritmul să fie cost-optimal**.

Exemplu

Algoritmul de **selecţie** a unei valori din secvenţa S = {s1, ..., sn}, prezentat mai departe în acest capitol, se execută pe o maşină EREW SM SIMD cu N procesoare, unde N<n. Algoritmul are proprietăţile formulate mai înainte.

(i) Algoritmul foloseşte $p(n) = n^{1-x}$ procesoare, cu $0<x<1$, unde valoarea lui x se obţine din formula $N = n^{1-x}$. Deci, p(n) este subliniar şi adaptiv.

(ii) Algoritmul se execută în $t(n) = O(n^x)$, cu x obţinut ca mai sus. Deci, timpul de execuţie este mai scurt decât cel al unui algoritm secvenţial. Totodată, algoritmul este adaptiv: cu cât este mai mare p(n), cu atât este mai mic t(n).

(iii) Costul este $c(n) = n^{1-x} * O(n^x) = O(n)$, ceea ce reprezintă un optim.

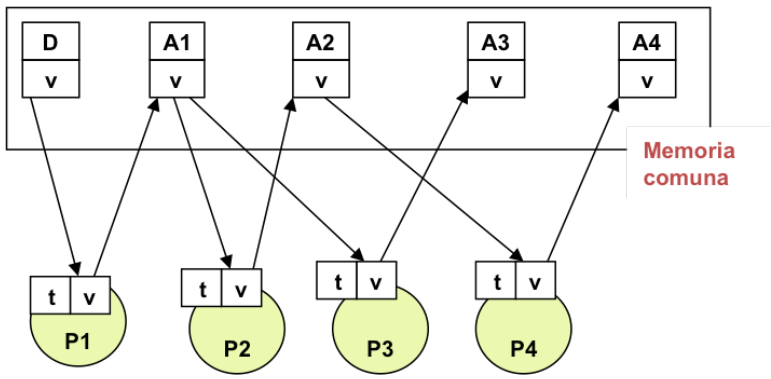
Observaţii
i) Satisfacerea cerinţelor menţionate este dificilă, uneori imposibilă.
ii) In practică, pentru valorile c(n), p(n), t(n) se consideră valorile rotunjite obţinute printr-o estimare pesimistă.

5.2. Două proceduri utile

Două operaţii tipice apar în execuţia algoritmilor paraleli în sisteme EREW SIMD:
- fiecare procesor trebuie să citească conţinutul unei celule din memoria comună (difuzarea unei valori);
- fiecare procesor trebuie să calculeze valoarea unei funcţii dependente de datele deţinute de celelalte procesoare (calcule prefix).

Difuzarea unei valori

Fie D celula din memoria comună ce trebuie difuzată celor N procesoare ale unui sistem EREW SIMD. Algoritmul BROADCAST presupune folosirea unui tablou A[1:N]. Acţiunile de citire şi de scriere sunt explicitate distinct şi explicit în descrierea algoritmului. Acest tratament este specific sistemelor PRAM.




```
int D, A[1:N];
Pas 1: Process P[1]{
          int t;
          1.1. citeşte valoarea din D;
          1.2. o memoreaza în propria memorie t;
          1.3. o scrie în A[1];
}

Pas 2: Process P[j=2 to N]{
       int t;
       for [i = 0 to (log N -1)]{
           if (j >= 2^i+1 and j <= 2^i+1){
               2.1. citeşte valoarea din A[j-2^i];
               2.2. o memorează în propria memorie t;
```

```
                2.3. o scrie în A[j];
            }
        }
}
```

Acţiunile de citire şi de scriere sunt specificate distinct şi explicit în descrierea algoritmului, iar paşii executaţi sincron de procesoare sunt foarte bine precizaţi.

Observaţii
- Deoarece numărul de procesoare care deţin valoarea din D se dublează la fiecare iteraţie, timpul de execuţie este O(log N).
- Dimensiunea tabloului A poate fi redusă la N/2, valorile memorate în ultimul pas fiind inutile.
- Utilizare tipică: difuzarea lui n, în cazul algoritmilor adaptivi.

Calculul sumelor prefix
Fiind dat vectorul A[1:N], fiecare procesor Pi calculează suma
A[1] + ...+ A[i].
Algoritmul este următorul:

```
int a[1:n];
process suma [k=1 to n]{
   int temp;          /*temp este locala procesului k*/
   for [j = 1 to sup(log2 n)]
      if (k-2^(j-1)>=1){
         temp = a[k-2^(j-1)];
         a[k] = temp + a[k];
      }
}
```

Observaţie
- Deoarece numărul procesoarelor care termină de calculat suma parţială se dublează la fiecare pas, calculul se face în O(log N) paşi folosind O(N) procesoare.

5.3. Căutarea paralelă

În cele ce urmează, prezentăm soluţia unei probleme de căutare a unei valori **x** într-o secvenţă ordonată **S** de **n** valori, pentru o maşină SM SIMD. Pentru simplitate, considerăm că elementele secvenţei sunt toate distincte între ele, adică:

s[1] < s[2] < ... < s[n].

Metoda secvenţială de căutare binară constă în compararea lui **x** cu elementul din mijlocul secvenţei **S**. În funcţie de rezultatul acestei comparaţii, căutarea se termină sau continuă cu una din cele două jumătăţi separate de elementul din mijloc. Algoritmul are complexitatea O(log n).

Să considerăm mai întâi cazul unui sistem EREW format din **N** procesoare, 1<=N<=n. Pentru a face cunoscută valoarea **x** tuturor procesoarelor, se poate folosi un algoritm de difuzare (broadcast) care are complexitatea O(log N). Secvenţa S este divizată în N subsecvenţe de lungime n/N, fiecare subsecvenţă fiind atribuită unui procesor. Folosind o procedură de căutare binară pe secvenţa atribuită, un procesor găseşte rezultatul în O(log(n/N)) în cazul cel mai defavorabil. Timpul total cerut este deci de ordinul O(log N) + O(log(n/N)) adică O(log n), acelaşi cu timpul necesar căutării binare pe un procesor secvenţial.

În cazul unui sistem CREW, difuzarea valorii lui **x** se poate face într-un singur pas. Mai mult, se poate adopta o politică de căutare diferită, obţinându-se o performanţă mai bună pentru timpul de execuţie.

Algoritmul se bazează pe împărţirea, în fiecare etapă, a secvenţei în care se face căutarea în N+1 subsecvenţe de aceeaşi lungime. Fiecare din cele N procesoare inspectează o valoare **s** aflată în poziţia dintre două secvenţe adiacente. Dacă x<s atunci toate elementele aflate la dreapta lui **s** sunt ignorate în pasul următor. Similar, dacă x>s sunt ignorate elementele mai mici ca **s**. Următorul subinterval de căutare se obţine făcand intersecţia subintervalelor păstrate de diferitele procesoare. Deoarece fiecare etapă se aplică unei secvenţe de dimensiune 1/(N+1) din secvenţa precedentă, sunt necesare $O(\log_{N+1}(n+1))$ etape.

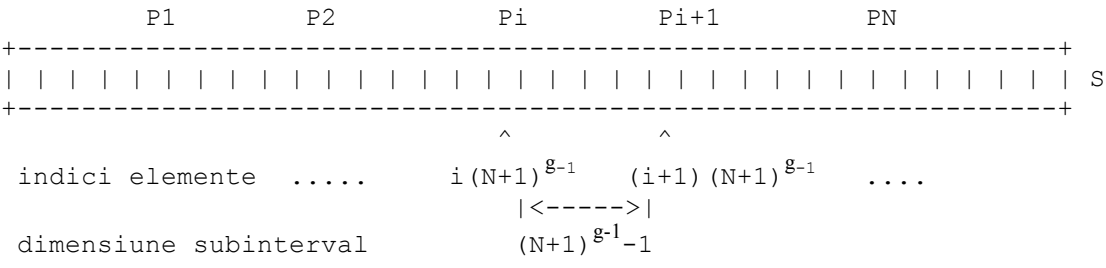


Figura 5.1

Fie **g** cel mai mic întreg astfel că n <= $(N+1)^g$-1. Cu alte cuvinte, g = sup(log(n+1)/log(N+1)). Se poate arăta prin inducţie că algoritmul are cel mult **g** etape. Astfel, afirmaţia este banală pentru g=0.
Fie relaţia adevărată pentru $(N+1)^{g-1}$-1. Pentru a inspecta o secvenţă de lungime $(N+1)^g$-1, procesorul Pi compară **x** cu s[j], unde j = $i(N+1)^{g-1}$, aşa cum se arată în figura .1. În urma comparaţiei, se reţine o subsecvenţă de dimensiune $(N+1)^{g-1}$-1, care (conform ipotezei de inducţie) se poate inspecta în g-1 etape.

O demonstraţie mai sugestivă consideră mai întai ultimul pas al algoritmului, în care fiecare element al secvenţei rămase este verificat de un procesor, deci $n_1 = N$. Secvenţa provine din pasul anterior de la una de lungime (N+1)(N+1) -1 = n_2. Deci, un sistem cu N procesoare poate acoperi în doi paşi o secvenţă de lungime cel mult $n_2 = (N+1)^2$ -1 (vezi figura 5.2).

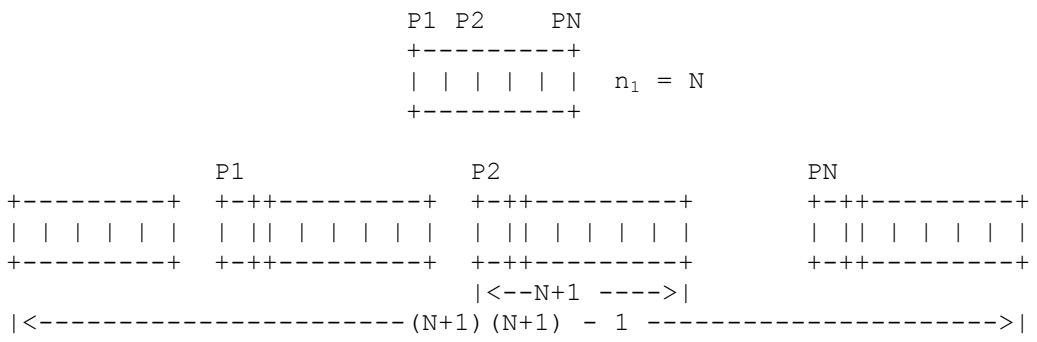


Figura 5.2

În general, un sistem cu N procesoare poate "acoperi" în **g** paşi o secvenţă de lungime cel mult $n_g = (N+1)^g$-1. În primul pas, această secvenţă este împărţită în subsecvenţe de $n_{g-1} = (N+1)^{g-1}$-1 elemente, procesorul P_i inspectând elementul din poziţia $i*(N+1)^{g-1}$ (considerand că primul element are poziţia 1). Dacă primul element are indicele q, atunci poziţia care corespunde lui Pi este $(q-1)+i*(N+1)^{g-1}$.

Ca urmare, dată fiind lungimea **n** a unei secvenţe de numere, numărul de paşi **g** ai procedurii de căutare rezultă din relaţia n<=$(N+1)^g$-1, adică este

g = sup(log(n+1)/log(N+1)).

Exemplul 1
Pentru exemplificarea execuţiei algoritmului, fie secvenţa de numere dată în figura .3a şi x=45, N=3. Deoarece n=15, g este cel mai mic număr care satisface relaţia

$(N+1)^g >= n+1$, adică $4^g >= 16$. Rezultă g=2. Lungimea unei subsecvenţe în primul pas este egală cu 3. Rezolvarea este schiţată în figura 5.3.

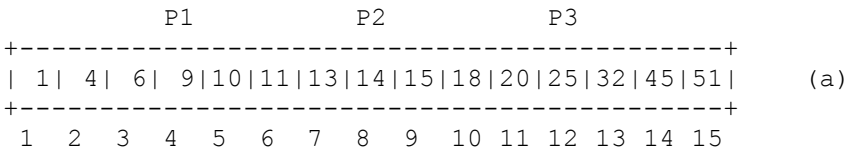


Figura 5.3a

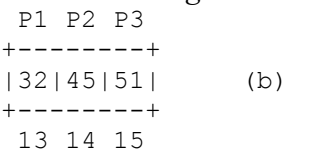


Figura 5.3b

Exemplul 2

Să considerăm o a doua execuţie, cu N=2, x=21. Deci, din relaţia $(N+1)^g>=n+1$ rezultă $3^g>=16$, deci g=3. Lungimea unei subsecvenţe în primul pas este egală cu 8. Rezolvarea este schiţată în figura 5.4.

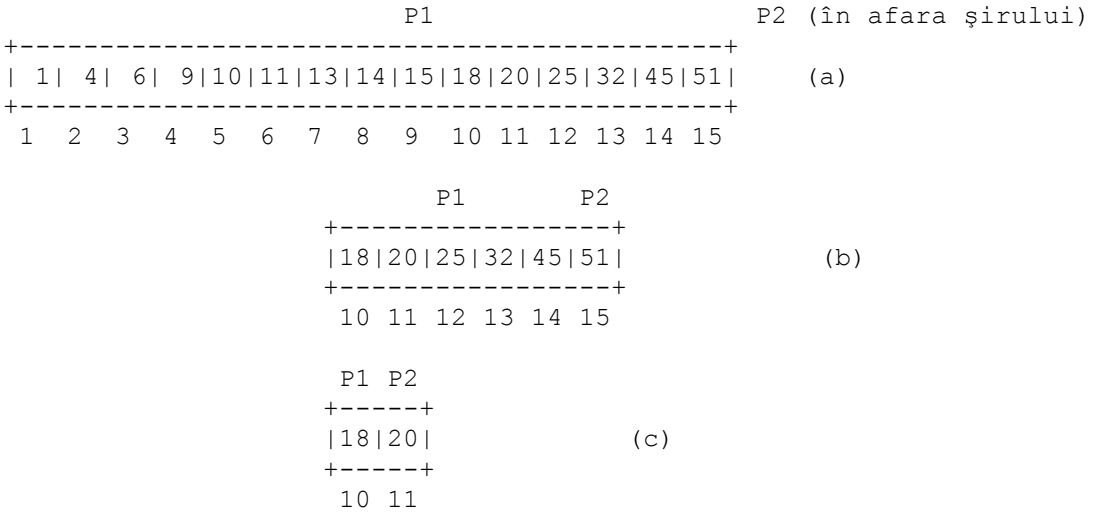


Figura 5.4

Pentru a determina subintervalul care rămane în fiecare etapă, fiecare proces Pi, 1<=i<=N utilizează o variabilă c[i] care ia una din valorile **stg** sau **dr**, după cum trebuie reţinută partea din stanga, respectiv cea din dreapta. În plus, se folosesc c[0]=dr şi c[N+1]=stg. Iniţial limitele subsecvenţei de căutare sunt q=1 şi r=n. De asemenea, **g** are iniţial valoarea numărului maxim de etape

$g = \sup(\log(n+1)/\log(N+1))$

şi scade cu o unitate la fiecare iteraţie.

Algoritmul calculează valorile c[i] şi găseşte **i** pentru care c[i-1]<>c[i]. Pe baza lui se determină următorul subinterval [q,r] de căutare. Mai precis,

$q := q+(i-1)(N+1)^{g-1}+1$, iar

$r := q+i(N+1)^{g-1} -1$.

Un singur procesor calculează noile valori pentru **q** şi **r**, celelalte având acces la aceste valori în timp constant.

În descrierea ce urmează sunt marcate cele trei etape ale algoritmului, precum şi paşii care-i compun.

```
/* Initializarea indicilor secventei de cautat */
int q = 1;                          /* limita stg interval crt */
int r = n;                          /* limita dr interval crt */
/* Initializarea rezultatului k si a numarului de pasi g */
int x, k = 0;             /* x = numar cautat  k = pozitia lui in sir */
int g = sup(log(n+1)/log(N+1))
enum sens {stg,dr};
sens c[0:N+1];            /*c[i] – x este la stg / dr pozitiei procesului i*/
c[0] = dr; c[N+1] = stg;
int j[1:N];              /* j[i] = pozitia din sir inspectata de proces i*/
process P[i = 1 to N]{
/* gaseste pozitia k unde se afla valoarea x */
    while (q<=r and k=0){          /* subsir exista si valoare negasita */
      j[i] = (q-1) + i*(N+1)^(g-1);
/* P[i] compara x cu s[j[i]] si determina partea de secv acceptata */
      if (j[i] <= r)              /* pozitie in interior sir */
         if (s[j[i]] == x) k = j[i];       /* pozitie gasita */
        else if (s[j[i]] > x) c[i] = stg;
         else c[i] = dr;
      else {j[i] = r+1;           /*j[i] indica o pozitie afara din sir*/
            c[i] = stg;}
/* calculeaza indicii subsecventei urmatoare */
/* doar un proces poate face modificarea intervalului de cautare*/
```

```
        if (c[i] <> c[i-1]){
             r = j[i] – 1;
             if (i>1) q = j[i-1] + 1;}
/* doar procesul N executa operatia urmatoare */
        if (i==N and c[i]<>c[i+1]) q = j[i] +1;
        g := g-1;
     }        /*end while*/
}             /* end process */
```

Observaţie
Operaţiile paralele din algoritm se execută în acelaşi timp în toate procesoarele. Operaţiile secvenţiale sunt executate de un singur procesor.

Pentru **exemplificarea** execuţiei algoritmului, fie secvenţa de numere dată în figura .5a şi x=45, N=3. Iniţial, q=1, r=15, k=0 şi g=2. Rezultă j_1=4, j_2=8 şi j_3=12. Ca urmare, în prima etapă:
P1 compară x cu s[4] şi deoarece 45>9 alege c[1]=dr;
P2 compară x cu s[8] şi deoarece 45>14 alege c[2]=dr;
P3 compară x cu s[12] şi deoarece 45>25 alege c[3]=dr.
Rezultă c[3]<c[4]. Ca urmare, se pune q=13, iar r rămane 15. Secvenţa se restrânge la cele trei elemente din figura .5b, iar g devine 1.

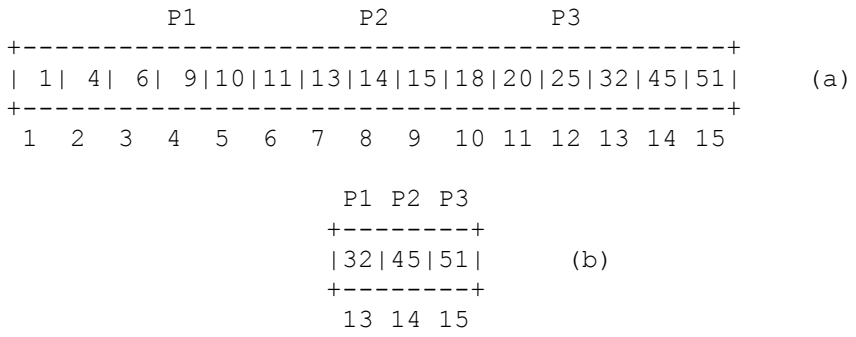


Figura 5.5

În a doua etapă, se obţin valorile j_1=13, j_2=14 şi j_3=15. Ca urmare:
P1 compară x cu s[13] şi deoarece 45>32 alege c[1]=dr;
P2 compară x cu s[14] şi deoarece 45=45 pune k=14 şi lasă c[2] neschimbat;
P3 compară x cu s[15] şi deoarece 45<51 alege c[3]=stg.

Noile valori pentru q şi r sunt 15, respectiv 14. Oricum, iteraţiile se opresc datorită condiţiei k<>0.

Să considerăm o a doua execuţie, cu N=2, x=21. Iniţial g=3. În prima iteraţie avem j_1=9 şi j_2=18>15, deci în afara secvenţei. Ca urmare:
P1 compară x cu s[9] şi deoarece 21>15 alege c[1]=dr;
P2 pune j_2 la 16 (deoarece valoarea calculată este în afara secvenţei) şi actualizează c[2]=stg.
Deoarece c[1]<>c[2], se stabilesc noile valori q=10 şi r=15, obţinându-se subsecvenţa din figura 5.6.b şi g=2.

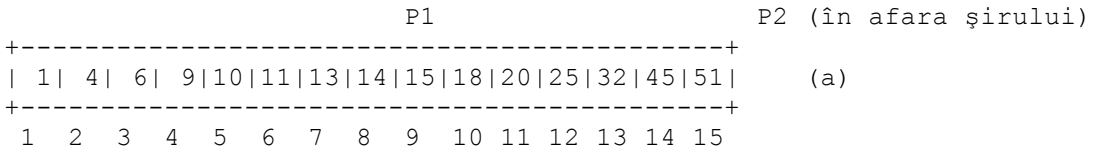


Figura 5.6a

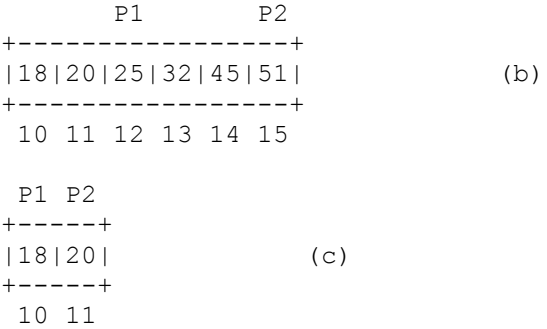


Figura 5.6b,c

În a doua iteraţie, P1 calculează j_1=9+3 şi compară x cu s[12]. Deoarece 21<25 se obţine c[1]=stg. Similar, j_2=15 şi, deoarece 21<51, avem c[2]=stg. Ca urmare, c[0]<>c[1] şi intervalul se restrânge pentru a treia iteraţie la r=11 şi **q** nemodificat (figura 5.6c).

În a treia iteraţie, j_1 este 10, P1 calculand c[1] = dr, deoarece 21>18. Similar, P2 calculează c[2] = dr, deoarece 21>20. Ca urmare **q** capătă valoarea 12, mai mare decât valoarea lui r provocând astfel ieşirea din ciclu cu k=0.

Observaţii

Deoarece pasul 3 al algoritmului precedent se execută de cel mult **g** ori, rezultă că algoritmul are complexitatea $t(n) = O(\log_{N+1}(n+1))$. Lucrul efectuat de algoritm este $O(N \log_{N+1}(n+1))$, deci algoritmul nu este cost-optimal. Cu toate acestea, se poate arăta că el este optim din punct de vedere al timpului de calcul. Pentru a arăta acest lucru, să observăm că, folosind N procesoare, algoritmul poate compara **x** cu cel mult N valori s[i], la un moment dat. După aceste comparaţii şi după eliminarea din secvenţă a elementelor despre care se ştie că nu sunt egale cu **x**, lungimea subsecvenţei rămase este de cel puţin:

$\sup((n-N)/(N+1)) >= (n-N)/(N+1) = [(n+1)/(N+1)]-1.$

După **g** repetiţii, secvenţa rămasă este de lungime

$[(n+1)/(N+1)^g]-1.$

Deci, numărul de iteraţii cerute de orice algoritm paralel trebuie să fie mai mare de cel mai mic **g** pentru care avem:

$[(n+1)/(N+1)^g]-1 <=0$, adică

$g = \sup(\log(n+1)/\log(N+1))$.

În cazul în care N=n problema este rezolvată în timp constant.

În privinţa cerinţei ca toate elementele secvenţei S să fie **distincte**, ea poate fi înlăturată dacă actualizarea lui **k** la găsirea unei coincidenţe este realizată printr-o procedură specială care evită conflictele de acces simultan (modelele folosite până acum sunt cu scriere exclusivă). Această procedură ia un timp $O(\log N)$, mărind astfel timpul de execuţie al algoritmului la:

$t(n) = O(\log(n+1)/\log(N+1)) + O(\log N)$.

Putem aprecia importanţa acestui timp suplimentar dacă luăm cazul N=n, în care nu se realizează nici o îmbunătăţire faţă de algoritmul secvenţial.

Menţinerea eficienţei algoritmului în cazul în care în şir pot exista **valori egale** se poate realiza prin folosirea unui model CRCW. În acest caz, este posibil ca la un conflict (mai multe elemente egale cu **x** se află în secvenţa S), doar cea mai mică valoare a indicilor elementelor respective să fie dată lui **k**.

6. Comunicarea prin mesaje

În cele prezentate pînă aici, comunicarea între procese este realizată prin date comune (globale), iar sincronizarea prin mecanismul furnizat de instrucţiunea **await**. Transmiterea de mesaje, la rîndul său, îndeplineşte cele două funcţii: **comnicarea** este realizată deoarece datele ajung de la transmiţător la receptor, iar **sincronizarea** este realizată deoarece un mesaj nu poate fi recepţionat înainte de a fi transmis.

La utilizarea comunicării prin mesaje, canalele sunt obiectele puse în comun de procese. Absenţa unor variabile comune reclamă tehnici speciale, diferite de cele folosite în cazul memoriei partajate. Ele ţin cont de faptul că procesele comunicante prin mesaje pot fi distribuite unor procesoare care nu au memorie comună. Din acest motiv, **programele paralele** care utilizează comunicare de mesaje se mai numesc **programe distribuite**. Asta nu înseamnă că astfel de programe nu pot fi executate de sisteme cu memorie comună. În acest caz, canalele sunt implementate folosind memoria partajată şi nu o reţea de comunicaţie.

6.1. Comunicarea asincronă prin mesaje

În cazul comunicării prin mesaje asincrone, **canalele** sunt cozi nelimitate de mesaje. Un proces adaugă un mesaj unei cozi prin execuţia unei operaţii **send**. Deoarece cozile sunt teoretic infinite, operaţia nu este blocantă. Un proces primeşte un mesaj dintr-o coadă prin execuţia unei operaţii **receive**. Cînd coada este vidă, operaţia întîrzie procesul, pînă la primirea unui mesaj. Canalele păstrează ordinea mesajelor, deci sunt cozi FIFO.

Declaraţia unui canal are forma următoare:

chan nume_canal (tip_1 id_1,..., tip_n id_n)

în care se specifică numele canalului, precum şi numele şi tipurile cîmpurilor care compun un mesaj transmis prin acel canal.

Se pot declara şi **tablouri (grupuri indexate) de canale** ca în exemplul următor:

chan rezultat [1:n](int)

în care se declară n canale avînd mesaje cu un singur cîmp de tip întreg. Numele cîmpurilor sunt opţionale, fapt rezultat şi din exemplul specificat.

Un proces tansmite un mesaj unui canal executînd operaţia

send nume_canal ($expresie_1$,..., $expresie_n$)

unde expresiile trebuie să aibă aceleaşi tipuri cu cele ale cîmpurilor mesajelor, specificate în declaraţia canalului. Efectul execuţiei unui send este calculul valorilor expresiilor, alcătuirea unui mesaj cu aceste valori şi adăugarea lui la sfîrşitul cozii asociate canalului.

Un proces recepţionează un mesaj prin execuţîa unei operaţii

receive nume_canal (variabila$_1$,..., variabila$_n$)

unde variabilele trebuie să aibă aceleaşi tipuri cu cîmpurile din declaraţia canalului. Efectul este întîrzierea receptorului pînă cînd în coadă există cel puţin un mesaj, urmată de preluarea primului mesaj din coadă şi atribuirea valorilor cîmpurilor variabilelor din listă.

Deoarece canalele sunt (declarate) globale, orice proces poate transmite şi recepţiona pe un canal. Cînd canalele sunt folosite astfel, ele mai sunt numite **cutii poştale**. Cînd receptorul este unic, avem de-a face cu un **port**. În fine, atunci cînd şi transmiţătorul este unic, avem de-a face cu o **legătură**.

O altă operaţie utilă este testul existenţei mesajelor pe un canal, avînd forma
empty (nume_canal)
şi întrocînd valoarea adevărat sau fals, după cum canalul nu are nici un mesaj, sau are cel puţin unul. Utilizarea acestui test trebuie făcută cu grijă, avînd în vedere că:
- dacă testul este adevărat, nu înseamnă că la reluarea execuţiei procesului canalul este în continuare gol;
- dacă testul este fals, nu înseamnă că următorul **receive** va produce recepţia imediată a unui mesaj (decît dacă există un singur receptor).

Exemplu. (Filtru) Prezentăm codul unui proces care transformă un flux de caractere într-o succesiune de linii de lungime maximă cunoscută.

```
chan input (char), output (char [1:Maxl]);
process car_linii {
     char line [1:Maxl]; int i =1;
     while true {
          receive input (line[i]);
          while (line[i] <> CR and i < Maxl) {
              /*line[1:i] = ultimele i caractere introduse*/
```

```
                i = i+1; receive input (line[i]);
            }
            send output (line); i = 1}
        }
}
```

Alt exemplu. (Replicated workers)
În problema cuadraturii, trebuie calculată aria mărginită de graficul unei funcţii continue ne-negative f(x), axa x şi dreptele x=l şi x=r. Soluţia clasică impune împărţirea intervalului [l,r] într-un număr de subintervale şi folosirea unui trapez pentru a aproxima aria corespunzătoare fiecărui subinterval. Dublăm apoi numărul de subintervale din [l,r] şi comparăm rezultatul obţinut cu cel anterior. Dacă diferenţa este acceptabilă calculul se termină, altfel se recurge la o nouă dublare a numărului de subintervale.

O soluţie mai bună este cea adaptivă, în care se porneşte procesul cu aria obţinută prin considerarea unui singur subinterval [l,r] şi apoi a două subintervale [l,m] şi [m,r]. Dacă rezultatele sînt suficient de apropiate, aria trapezului mare este considerată o aproximaţie bună a ariei de sub curba f(x). Dacă nu, problema se împarte în două subprobleme care se tratează separat: calculul ariei din subintervalul [l,m] şi calculul ariei din subintervalul [m,r]. Se repetă acest proces pînă cînd se obţine o soluţie acceptabilă pentru fiecare subinterval. Spre deosebire de prima soluţie, în a doua se generează noi subprobleme doar pentru o parte din subintervale. În ambele cazuri, subproblemele sînt independente, ceea ce permite paralelizarea uşoară a algoritmului.

În acest scop, se utilizează un proces **administrator**, care generează prima problemă şi aşteaptă rezultatul, precum şi mai multe procese de **lucru**, care rezolvă subproblemele, generând eventual alte subprobleme. Procesele de lucru împart un acelaşi canal sac, ce conţine problemele de rezolvat. O problemă este caracterizată prin cinci valori: limitele subintervalului a şi b, valorile f(a) şi f(b), precum şi aria corespunzătoare (pentru a evita recalcularea sa). Când un proces de lucru găseşte răspunsul la o subproblemă, el trimite administratorului, printr-un canal separat, rezultatul. Programul se termină cînd administratorul detectează calculul întregii arii.

```
chan sac (real a, b, fa, fb, aria);
chan rezultat (real a, b, aria);
process Administrator {
  real l, r, fl, fr, a, b, aria, total;
```

```
  alte variabile marcare intervale terminate;
  fl = f(l); fr = f(r);
  aria = (fl+fr)*(r-l)/2;
  send sac(l, r, fl, fr, aria);
  while (nu s-a calculat toata aria) {
     receive rezultat (a, b, aria);
     total = total + aria;
     marcheaza intervalul [a,b] terminat;
  }
}

process Lucru [i = 1 to n] {
  real a, b, m, fa, fb, fm;
  real arias, ariad, ariat, dif;
  while (true) {
     receive sac (a, b, fa, fb, ariat);
     m := (a+b)/2; fm := f(m);
     calculeaza arias si ariad;
     dif := ariat - arias - ariad;
     if (dif mic) send rezultat (a, b, ariat);
     else {send sac (a, m, fa, fm, arias);
              send sac (m, b, fm, fb, ariad);}
  }
}
```

Exerciţii

1) In programul precedent, toate procesele de lucru recepţionează din acelaşi canal (sac). Modificaţi programul astfel încat să se folosească doar canale cu un singur receptor şi mai mulţi transmiţători.

2) Scrieţi un program de sortare rapidă folosind procese replicate. Iniţial, tabloul a[1:n] este local administratorului. La terminarea programului, şirul sortat trebuie înapoiat administratorului care-l va memora în a[1:n].

3) Folosiţi metoda proceselor replicate pentru a găsi toate soluţiile în problema celor 8 regine.

6.2. Comunicarea sincronă prin mesaje

Comunicarea asincronă de mesaje are unele neajunsuri privind confirmarea recepţiei şi eficienţa:

- pentru ca transmiţătorul să fie informat de recepţia mesajului comunicat, este nevoie

de un mesaj de răspuns;
- dacă răspunsul nu soseşte, transmiţătorul nu poate şti dacă mesajul a fost sau nu primit (receptorul a căzut înainte de , sau după primirea mesajului);
- spaţiul ocupat de mesajele transmise dar nerecepţionate poate fi mare şi deci ineficient utilizat.

Comunicarea **sincronă** elimină aceste neajunsuri. Aici, ambele primitive, **send** şi **receive** sunt blocante. Astfel, la execuţia unei transmisii, procesul aşteaptă pînă cînd un altul efectuează o recepţie din acelaşi canal. Transmiţătorul poate continua dacă mesajul a fost efectiv preluat. Mesajele nu trebuie să fie memorate temporar în cozi.

Pe de altă parte, comunicarea sincronă este uneori neconvenabilă. După cum se vede, ea este o legătură unu la unu între procese, astfel încît anumite operaţii, cum ar fi difuzarea de mesaje, sunt greu de realizat.

Un proces tansmite **sincron** un mesaj unui canal executînd operaţia

synch_send nume_canal (expresie$_1$,..., expresie$_n$)

Exemplu – comunicare producator / consumator

```
chan prodcons (real);
process prod {
    real date[n];
    for [i = 1 to n] {
        calculeaza date[i];
        synch_send prodcons (date[i]);
            /* intarzie transmitatorul */
    }       /* daca receptorul este mai lent */
}
process cons {
    real buf[n];
    for [i = 1 to n] {
        receive prodcons (buf[i]);
        prelucreaza datele primite;
    }
}
```

6.3. Modelul CSP pentru comunicarea sincronă

Pentru reprezentarea primitivelor de comunicare sincronă, putem folosi şi notaţia din CSP (Hoare) relativă la comunicarea cu gardă. Aceasta poate fi folosită în egală măsură în comunicaţia asincronă sau în tehnica **rendezvous**. (Spre deosebire de comunicarea sincronă, numită şi rendezvous simplu, în tehnica rendezvous extins un proces acceptă un apel executat de un alt proces, prin intermediul unei instrucţiuni specifice **input** sau **accept**, realizează diverse prelucrări şi apoi întoarce rezultatele obţinute.)

Modelul CSP foloseşte construcţii diferite de notaţiile prezentate de noi anterior, nu doar pentru comunicarea sincronă ci şi pentru decizie (selecţie), iteraţie şi altele. Prezentăm pe scurt aceste notaţii.

Declaraţia variabilelor are forma generală
var id_1 : tip_1 := val_1; ... ; id_n : tip_n := val_n;
unde iniţializările sunt opţionale.

Comanda cu gardă are forma generală
B -> S
unde B este o expresie booleană
S este o instrucţiune (simplă sau compusă). S nu poate fi executată dacă B nu este *true*.

Selecţia (if) are forma generală
if B_1 -> S_1
[] B_2 -> S_2
...
[] B_n -> S_n
fi

Gărzile sunt evaluate într-o ordine arbitrară. Dacă B_i este true atunci se execută S_i. Cu alte cuvinte, dacă mai multe gărzi sunt *true*, alegerea instrucţiunii executate este nedeterministă.

Iteraţia (do) are forma generală

do B_1 -> S_1
[] B_2 -> S_2
...
[] B_n -> S_n
od

Comunicarea sincronă între procese are loc prin primitive de intrare/ieşire, de forma:

Destinaţie ! port (e_1,...,e_n)
Sursă ? port (v_1,...,v_n)

unde **Destinaţie** şi **Sursă** sunt procese, iar **port** identifică unul din canalele de comunicaţie ale acestora. Pentru prima dintre primitive, de ieşire, valorile expresiilor e_i sunt transmise portului specificat al procesului **Destinaţie**. Portul poate lipsi, dacă nu există posibilitatea de confuzie, iar dacă mesajul are un singur cîmp, se poate renunţa şi la paranteze.

Pentru a doua primitivă, mesajul primit la portul specificat, de la procesul **Sursă**, este asociat variabilelor v_i.

De menţionat că primitivele se execută simultan în tandem, dacă:
- instrucţiunea de ieşire apare în procesul specificat de instrucţiunea de intrare;
- instrucţiunea de intrare apare în procesul specificat de instrucţiunea de ieşire;
- identificatorii porturilor coincid;
- toate atribuirile de forma $v_i := e_i$ ar fi valide.

Fie instrucţiunile:

```
(1) X ? (a,b) şi
(2) DIV ! (3*x+y, 12).
```

Conform primeia, se preiau de la procesul X două valori şi se atribuie variabilelor **a** şi **b**. Prin cea de a doua, se evaluează cele două expresii şi se transmit procesului DIV rezultatele. Dacă procesul DIV execută comanda (1), iar procesul X comanda (2), atunci cele două se derulează simultan şi au acelaşi efect ca atribuirea (a,b) := (3*x+y, 12).

Exemple.
1) **Filtru**. Procesul **Copy** face copierea unor valori de la procesul **vest** la procesul **est**:

```
process Copy { char c;
      do true -> vest ? c; est ! c od
}
```

2) **Client - Server**. Calculul cmmdc printr-un proces server, CMMDC:

```
   process CMMDC { int id, x, y;
       do true -> CLIENT[*] ? arg (id,x,y);
            do x > y -> x := x-y;
            [] x < y -> y := y-x
            od
            CLIENT[id] ! result (x)
       od
   }
```

CLIENT-ul trebuie să conţină instrucţiunile

```
   ...CMMDC ! arg (i,v1, v2); CMMDC ? result (r);...
```

În acest exemplu, procesul CMMDC acceptă argumente de intrare de la orice proces client (CLIENT[*]) şi trimite răspunsul către clientul al cărui nume este primul argument (**id**) inclus în mesajul primit pe portul **est**.

Primitivele de intrare/ieşire pot fi extinse la instrucţiuni de **comunicare cu gardă**, care au forma:

B; C -> S;

unde B este o expresie boleană opţională
C este o primitivă de comunicare opţională, iar
S este o listă de instucţiuni.

Dacă B este omisă, ea are valoarea implicită true. Dacă C este omisă, expresia se reduce la o comandă cu gardă, uzuală.

Împreună B şi C alcătuiesc **garda**. Garda este rezolvată cu succes dacă B este *true* şi execuţia lui C se poate face imediat (este în aşteptare o instrucţiune de comunicare pereche). În acest caz, se execută mai întîi comanda de comunicare C şi apoi secvenţa S. Pe de altă parte, dacă B este *false* garda nu este reuşită. În fine, garda se blochează dacă B este *true*, dar C nu poate fi executată imediat.

Comunicarea cu gardă este utilizată în instrucţiunile **if** şi **do**. Astfel, la un **if**, dacă cel puţin o gardă se rezolvă cu succes, una din secvenţele asociate este luată la întîmplare şi executată. Dacă toate gărzile sunt nereuşite atunci **if** se termină. Dacă nici o gardă

nu este reuşită, dar anumite gărzi sunt blocate, atunci execuţia se întîrzie pînă cînd una reuşeşte. Similar se execută şi **do**.

În gărzi pot apare atît instrucţiuni de intrare, cît şi instrucţiuni de ieşire.

Exemple.

1) **Filtru**. Transmitere cu înlocuire. Transmite către procesul est caracterele produse de procesul vest. Înlocuieşte fiecare pereche de asteriscuri ** cu caracterul săgeată ^. Prin convenţie, ultimul caracter transmis de vest nu este un asterisc.

```
process INLOCUIRE {
    do char c; vest?c ->
        if c<>asterisc -> est!c
        [] c=asterisc -> vest?c;
          if c<>asterisc -> est!asterisc; est!c
          [] c=asterisc -> est!sageata
          fi
        fi
    od
```

2) **Comunicare printr-un tampon limitat**.

```
process Copy { char buffer [1:10];
    int front = 1, rear = 1, count = 0;
    do count<10; vest ? buffer[rear] ->
          count = count+1; rear = (rear mod 10) +1
    [] count > 0; est ! buffer [front] ->
          count = count -1; front = (front mod 10) +1
    od
```

Procesul are două gărzi. Prima este reuşită dacă există loc în tampon şi **vest** este gata să trimită un caracter. A doua reuşeşte cînd există caractere în tampon şi **est** este pregătit să accepte un caracter. De remarcat că instrucţiunea **do** nu se termină niciodată, deoarece cel puţin una din condiţiile booleene din gărzi este îndeplinită.

7. Execuţia programelor distribuite

In cazul programelor distribuite, **timpul total de execuţie** se defineşte ca timpul scurs

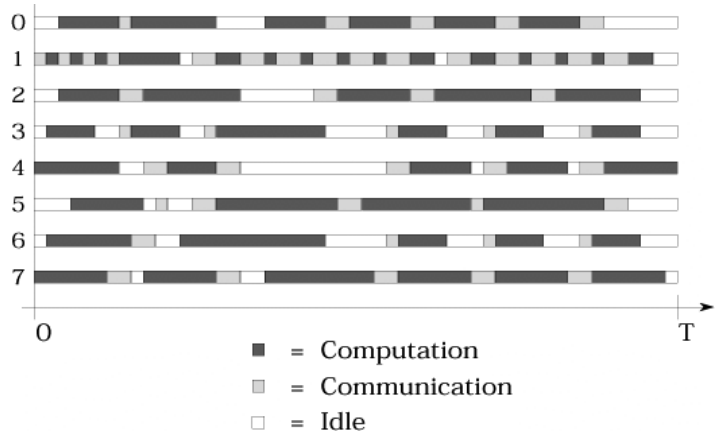


Figura 3.8 (Foster 1995)

din momentul când primul procesor începe execuţia sa (mai precis a unuia din procesele programului distribuit) până în momentul în care ultimul procesor termină execuţia. În evoluţia unui proces se disting mai multe perioade: de calcule, de comunicare şi de inactivitate, astfel că timpul total de execuţie se exprimă ca suma timpilor de calcul (T_{comp}), de comunicare (T_{commun}) si de inactivitate (T_{idle}) raportată la numarul de procesoare (P):

$$T = (1/P)\ (T_{comp} + T_{commun} + T_{idle})$$

sau

$$T = (1/P)\ (\ \Sigma_{i=0,P-1}\ T^i_{comp} + \Sigma_{i=0,P-1}\ T^i_{commun} + \Sigma_{i=0,P-1}\ T^i_{idle})$$

unde:

T^i_{comp} = timpul total de calcul pe procesorul i
T^i_{commun} = timpul total de comunicare al procesorului i
T^i_{idle} = timpul total de inactivitate al procesorului i.

În calculul analitic al timpului total de execuţie, se neglijează, de regulă, **timpul de inactivitate**.

Timpul de calcul depinde de dimensiunea problemei, N, numarul de taskuri sau de procesoare si de caracteristicile procesoarelor si memoriei.

Timpul de comunicare este timpul consumat pentru transmiterea şi pentru recepţia mesajelor. Aceştia diferă de la un tip de comunicaţie la alta; astfel, dacă două procese sunt executate de procesoare diferite, comunicarea între ele are o durată mai mare (comunicarea între procesoare consumă mai mult timp); comunicarea între două procese executate de acelaşi procesor este mai eficientă (consumă mai puţin timp). Pentru simplificare, se consideră că cele două tipuri de comunicări au durate comparabile şi se utilizează o formulă comună de calcul. Timpul de comunicare a unui mesaj (transmitere sau recepţie) are două componente:
t_s - corespunde timpului consumat de operaţiile de pregătire a comunicării şi este independent de lungimea mesajului;
t_w – corespunde timpului de comunicare a unui cuvânt din mesaj.

Cu acestea, timpul de comunicare a unui mesaj de lungime L devine:

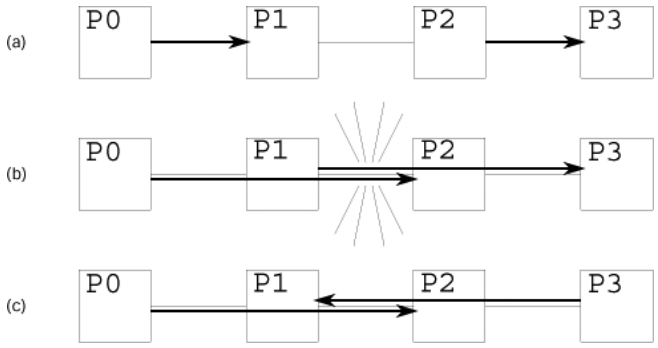
$$\mathbf{Tmsg = t_s + t_wL}$$

Formula este confirmată experimental (Foster 1995) prin măsurarea duratei transmisiei dus-întors a unui mesaj (round-trip time).

Într-un model de cost mai detaliat, se poate lua în considerare competiţia între procese pentru comunicarea pe un acelaşi canal. Evident această competiţie lungeşte timpul de comunicare. Daca S procese vor sa utilizeze în acelaşi timp un canal, durata comunicarii efective se măreşte de S ori (evident, timpul de pregătire nu este afectat):

$$\mathbf{T_{msg\text{-}b} = t_s + t_wSL}$$

Valoarea lui S diferă de 1 doar dacă cele S procese transmit în acelaşi sens pe un canal, aşa cum se arată în Figura 3.9: în cazul (b), P0 şi P1 partajează canalul (P1,P2), în timp ce în cazul (c), P0 şi P3 transmit în sensuri diferite pe acest canal şi se consideră că transmisiile nu interferă.



(a) S = 1 (b) S = 2 (c) S = 1

Figura 3.9 (Foster 1995)

Ca aplicaţie, considerăm **algoritmul lui Floyd** pentru calculul drumurilor celor mai scurte într-un graf reprezentat prin matricea de adiacenţe I, a cărui variantă secvenţială este următoarea:

```
for [k = 0 to N-1]
    for [i = 0 to N-1]
      for [j = 0 to N-1]
            I[i,j] k+1 = min(I[i,j]k, I[i,k]k + I[k,j]k);
```

Dacă t_c este durata unei iteraţii în algoritmul precedent, **timpul total de execuţie** al algoritmului secvenţial este:

$$T_{secv} = t_c N^3$$

Vom analiza două variante distribuite, ambele folosind P procesoare. În **prima variantă**, se face o descompunere uni-dimensională a matricei I în P grupuri de linii (fiecărui procesor i se asociază N/P linii). Algoritmul poate folosi cel mult N procesoare, deci întotdeauna numărul de procesoare este mai mic dacât dimensiunea matricei, P<N. Fiecare procesor păstrează local una sau mai multe linii din I şi este responsabil de calculul valorilor elementelor din liniile respective. Dacă procesorul **p** păstrează liniile de la `p_i_start` la `p_i_end` atunci codul său se poate descrie în felul următor:

```
for [k = 0 to N-1]
```

```
for [i = local_i_start to local_i_end]
  for [j = 0 to N-1]
        I[i,j] k+1 = min(I[i,j]k, I[i,k]k + I[k,j]k);
```

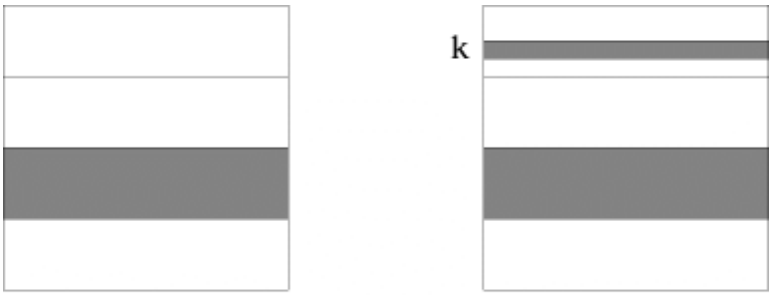


Figura 3.10 (Foster 1995)

În pasul k, procesoarele au nevoie de linia k a matricei I_k (Figura 3.10). Procesorul care păstrează această linie o comunică în **log P** paşi folosind algoritmul de difuzare, înainte ca fiecare din celelalte procesoare să înceapă calculul liniilor matricei I_{k+1} de care răspund. Deoarece fiecare mesaj are N cuvinte, timpul de difuzare a unei linii este:

$$\log P \, (t_s + t_w N).$$

Timpul total de execuţie al acestei variante distribuite este:

$$T_{dist\text{-}1} = t_c \, N^3/P + N \log P \, (t_s + t_w N)$$

El este compus din timpul de calcul $t_c \, N^3/P$, care este timpul algoritmului secvenţial împărţit la numarul de procesoare plus timpul de comunicare $N \log P \, (t_s + t_w N)$, care este de N ori timpul de difuzare a unei linii.

În **a doua variantă**, facem o descompunere bi-dimensională, pe linii şi coloane: fiecare procesor păstrează local elementele aflate pe una sau mai multe linii şi simultan pe una sau mai multe coloane din I şi este responsabil de calculul valorilor elementelor din liniile şi coloanele respective. Evident, numarul de procesoare P nu poate fi mai mare de N^2. Cele P procesoar se dispun într-o matrice de √P linii pe √P coloane. Dacă procesorul **p** păstrează elementele aflate pe liniile de la `p_i_start` la `p_i_end` şi simultan pe coloanele de la `p_j_start` la `p_j_end` atunci codul său se poate descrie în felul următor:

```
for [k = 0 to N-1]
      for [i = local_i_start to local_i_end]
        for [j = local_j_start to local_j_end]
                  I[i,j] k+1 = min(I[i,j]k, I[i,k]k + I[k,j]k);
```

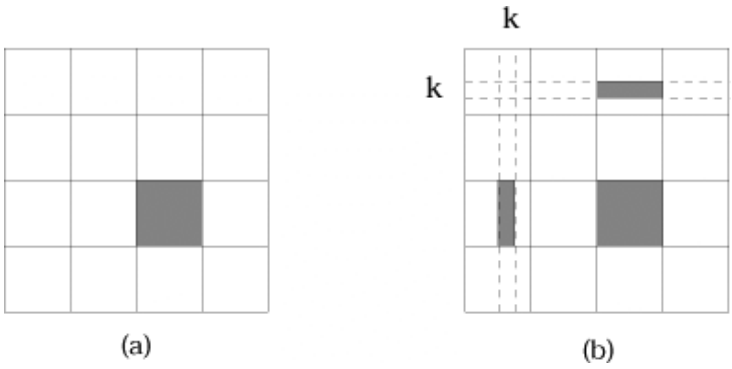


Figura 3.11 (Foster 1995)

În fiecare pas k, fiecare proces are nevoie de datele sale locale şi de câte N/√P valori de la alte două procese care păstrează:
- elementele din linia k şi coloanele de la `p_j_start` la `p_j_end`
- elementele din coloana k şi liniile de la `p_i_start` la `p_i_end`.

Fiecare din cele două mesaje are N/√P elemente, astfel încât difuzarea unuia durează

$$\log \sqrt{P} \, (t_s + t_w N/\sqrt{P})$$

deoarece difuzarea se face unui număr de √P procese.

În cei N paşi, operaţia se repetă de 2N ori. Ca urmare, timpul total de execuţie:

$$T_{dist\text{-}2} = t_c N^3/P + 2N \log \sqrt{P} \, (t_s + t_w N/\sqrt{P})$$
$$= t_c N^3/P + N \log P \, (t_s + t_w N/\sqrt{P}).$$

3.9. Modelul LogP

Un model mai adecvat studiului performanţelor pe maşini reale este **LogP** (Figura 3.12).

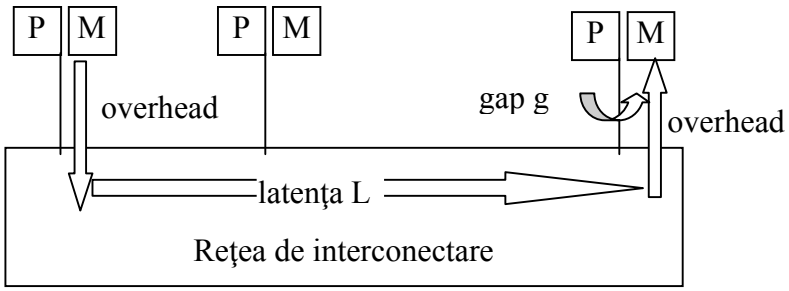


Figura 3.12 Modelul LogP

Modelul consideră P procesoare, fiecare cu memoria sa locală, legate printr-o reţea de interconectare. Parametrii modelului sunt:

- L – latenţa, sau întârzierea de transmitere a unui mesaj conţinând un număr mic de cuvinte, de la procesorul / memoria sursă la destinatar
- o - overhead, durata pentru care procesorul este angajat în transmiterea sau recepţia fiecărui mesaj; în timpul acesta procesoul nu poate face alte operaţii
- g - gap, intervalul minim de timp între două transmiteri succesive sau două recepţii succesive la acelaşi modul
- P - numărul de module procesor / memorie.

Reţeaua are o capacitate limitată, astfel că cel mult sup(L/g) mesaje pot fi în tranzit la un moment dat, de la orice procesor sau la oricare procesor. Cea mai simplă operaţie de comunicare, transmiterea unui singur pachet de la o maşină la alta, cere un timp de L+2o. O operaţie cerere/răspuns, precum o citire sau o scriere blocantă, ia un timp de 2L+4o. Fiecare procesor este implicat un timp de 2o, restul timpului putând fi utilizat pentru calcul, sau pentru alte operaţii de intrare/ieşire.

Ca aplicaţie considerăm **difuzarea unei valori**. Conform modelului PRAM, transmiterea are loc conform tiparului următor:

P0 -> P1	de la P0 la P1
P0, P1 -> P2, P3	de la P0 şi P1 la P2 şi P3
P0, P1, P2, P3 -> P4, P5, P6, P7	...

Presupunând g>=0 şi luând P=8, o=2, g=4, L=6, obţinem din calcul un timp total de difuzare egal cu 30 de unitaţi (Figura 3.13).

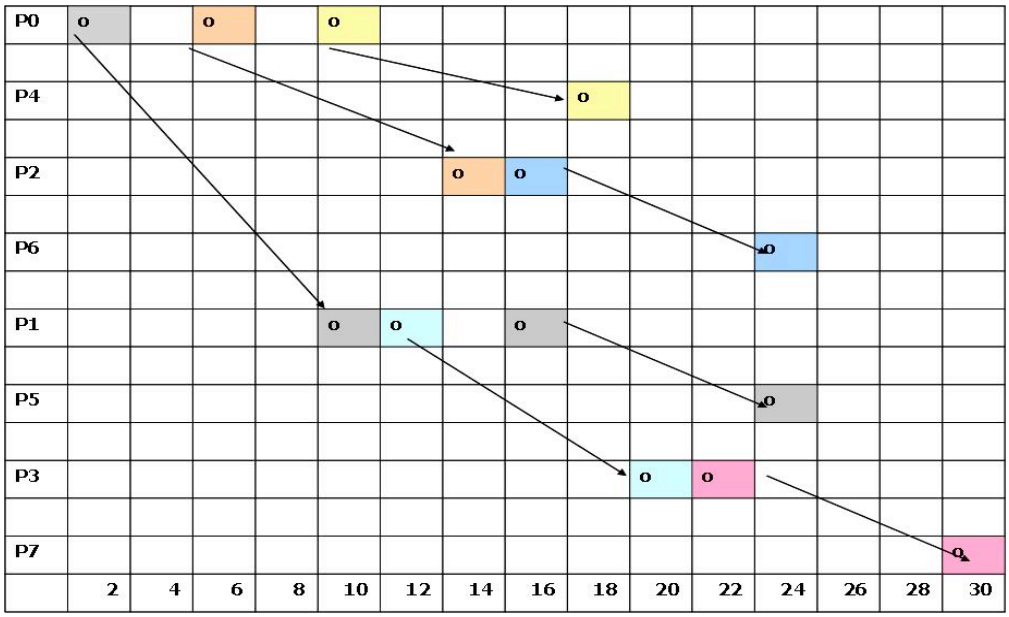


Figura 3.13. Schema de difuzare pentru tiparul PRAM

Pentru modelul LogP, adoptăm convenţia că **orice procesor care primeşte o valoare, o transmite imediat altor procesoare**, cu condiţia ca nici un procesor să nu primească mai mult de o valoare. Graful corespunzător variantei optime este cel din figura 3.14. Timpul corespunzător este de 24 de unităţi (Figura 3.15).

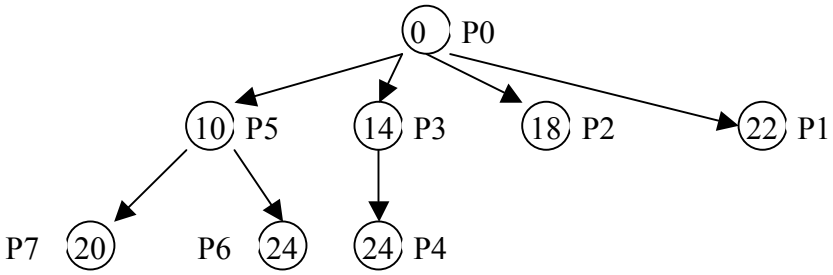


Figura 3.14. Graful variantei optime

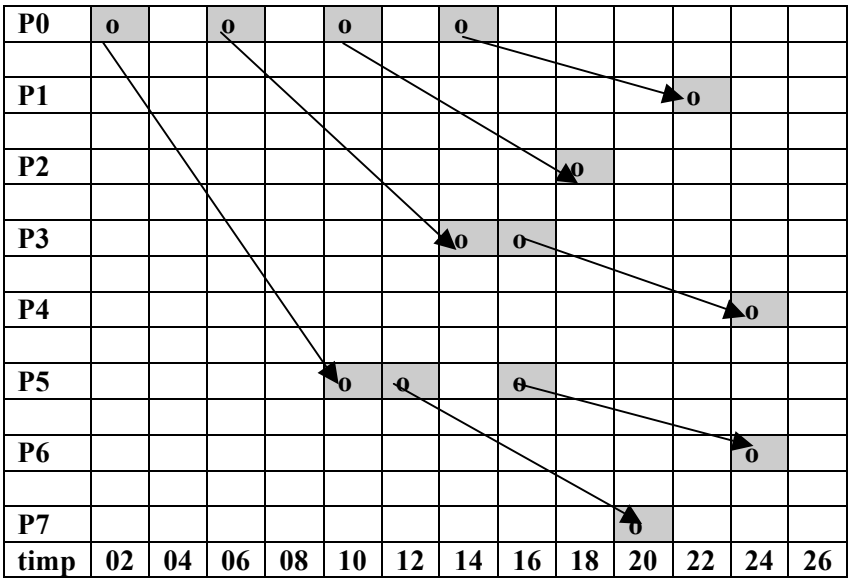


Figura 3.15. Schema de difuzare pentru varianta optimă

MPI - Message Passing Interface

1. Generalităţi

MPI este un **standard** pentru comunicarea prin mesaje, elaborat de MPIForum. În definirea lui au fost utilizate caracteristicile cele mai importante ale unor sisteme anterioare, bazate pe comunicaţia de mesaje. A fost valorificată experienţa de la IBM, Intel (NX/2) Express, nCUBE (Vertex), PARMACS, Zipcode, Chimp, PVM, Chameleon, PICL.

MPI **are la bază** modelul proceselor comunicante prin mesaje: un calcul este o colecţie de procese secvenţiale care cooperează prin comunicare de mesaje.

MPI **este o bibliotecă** nu un limbaj. El specifică convenţii de apel pentru mai multe limbaje de programare: C, C++, FORTRAN.77, FORTRAN90.

MPI a ajuns la versiunea 2, trecând succesiv prin versiunile:

- MPI 1 (1993), un prim document, orientat pe comunicarea punct la punct.
- MPI 1.0 (iunie 1994) este versiunea finală, adoptată ca standard; include comunicarea punct la punct şi comunicarea colectivă.
- MPI 1.1 (1995) conţine corecţii şi extensii ale documentului iniţial din 1994, modificările fiind univoce.
- MPI 1.2 (1997) conţine extensii şi clarificări pentru MPI 1.1
- MPI 2 (1997) include funcţionalităţi noi:
    - Procese dinamice
    - Comunicarea “one-sided”
    - Operaţii de I/E paralele

Obiectivele MPI:

- Proiectarea unei interfeţe de programare a aplicaţiilor
- Comunicare eficientă
- Să fie utilizabil în medii eterogene
- Portabilitate
- Interfaţa de comunicare să fie sigură (erorile tratate dedesubt)
- Apropiere de practici curente (**PVM**, NX, Express, p4)
- Semantica interfeței să fie independentă de limbaj

2. Comunicarea punct la punct

2.1. Operaţii de bază

Operaţiile de bază pentru comunicarea punct la punct sunt **send** şi **receive**. În MPI ele apar ca extensii necesare ale operaţiilor similare din alte biblioteci de comunicare prin mesaje. Justificăm în cele ce urmează utilitatea acestor extensii.

Considerăm mai întâi operaţia de transmitere de mesaje, în forma “clasică”, sumilară cu cea întâlnită, de exemplu, la socketi:

send (adresă, lungime, destinaţie, tag)

unde

- **adresa** identifică începutul unei zone de memorie unde se află mesajul de transmis
- **lungime** este lungimea în octeţi a mesajului
- **destinaţie** este identificatorul procesului căruia i se trimite mesajul (uzual un număr întreg)
- **tag** (**flag**) este un întreg ne-negativ care restricţionează recepţia mesajului. Acest argument permite programatorului să rearanjeze mesajele în ordine, chiar dacă ele nu sosesc în secvenţa dorită.

Acest set de parametri este un bun compromis între ceea ce utilizatorul doreşte şi ceea ce sistemul poate să ofere: transferul *eficient* al unei zone contigue de memorie de la un proces la altul. În particular, sistemul oferă mecanismele de păstrare în coadă a mesajelor, astfel că o operaţie de recepţie

recv (adresă, maxlung, sursă, tag, actlung)

se execută cu succes doar dacă mesajul are tag-ul specificat. Mesajele care nu corespund aşteaptă în coadă.

În cele mai multe sisteme, **sursa** este un argument de ieşire, care indică originea mesajului. În alte cazuri este folosită pentru a restricţiona recepţia. Ceilalţi parametri reprezintă, respectiv:

- adresă, maxlung – descrierea zonei receptoare de mesaj
- actlung – numărul de octeţi efectiv recepţionaţi

Formele prezentate impun restricţii considerate inacceptabile pentru aplicații distribuite. Analizăm pe rând aspectele.

2.1.1. Descrierea zonelor tampon

Perechea (adresă, lungime) este inadecvată pentru că:

1. În multe cazuri mesajul nu ocupă o zonă contiguă. Este dezirabilă o formă care descrie distribuţia originală a datelor.
2. Mesajele conţin valori cu tip; într-un mediu eterogen, transmiterea lor ca simple şiruri de octeţi este inadecvată.

Transmiterea ca date cu tip ar putea fi uşurată de rutine adecvate de conversie, făcând parte din biblioteca de comunicare. În MPI, tamponul de comunicaţie este definit de tripleta

(adresă, contor, tip_de_date)

care descrie un număr **contor** de valori de tipul **tip_de_date** situate începând cu adresa **adresă**. Astfel, (A, 300, MPI_DOUBLE) reprezintă un vector A de 300 valori reale, în C. O implementare MPI va asigura că cele 300 de valori sunt recepţionate corect, chiar dacă reprezentarea lor în maşina destinatarului diferă de reprezentarea pe maşina sursă.

În plus, un utilizator poate construi (folosind rutinele de bibliotecă) propriile sale tipuri de date, care se pot referi la date din zone ne-contigue de memorie.

2.1.2. Contextul

Fiecare comunicare de mesaj se derulează într-un anumit context.. Mesajele sunt intotdeauna primite în contextul în care au fost transmise. Mesajele transmise în contexte diferite nu interferă.

Contextele sunt alocate de sistem (în timpul execuţiei, ca răspuns la cererea utilizatorilor) şi sunt folosite, ca şi câmpurile **tag** pentru restricţionarea recepţiei mesajelor.

Să considerăm o aplicaţie care apelează o rutină de bibliotecă (ce face, de exemplu, transpunerea unei matrici). Este important să se asigure că tag-urile mesajelor folosite în bibliotecă sunt distincte de cele

folosite în restul aplucaţiilor. Altfel pot apare erori. În figură sunt reprezentate patru procese, care apelează o rutină de bibliotecă (reprezentată prin zona dreptunghiulară). Unul din procese termină mai devreme execuția, după care generează un mesaj (linia punctată) interceptat de procesul P2 în cadrul rutinei de bibliotecă. Utilizarea unor contexe diferite în afara și în interiorul bibliotecii evită această situaţie (figura din dreapta), procesul P2 nepreluând mesajul decât după revenirea la contexul în care el a fost transmis (deci după ce termină execuția rutinei de bibliotecă).

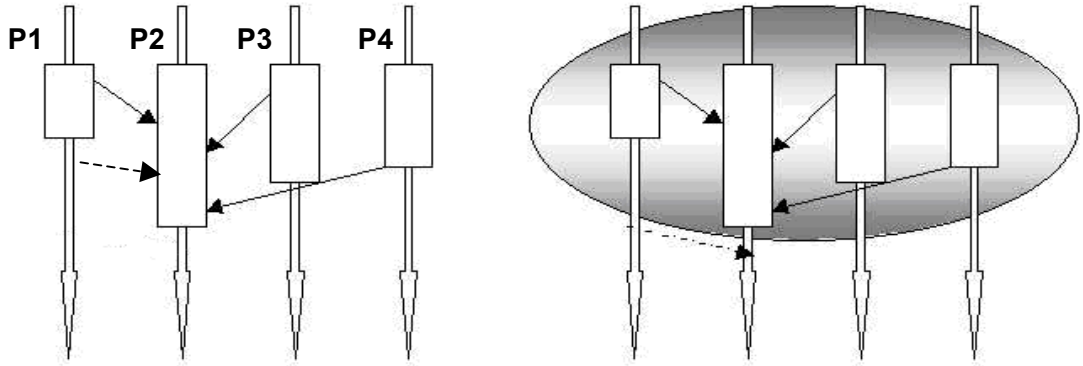


Figura 1

2.1.3. Grupurile de procese

Contextul este **partajat** de un grup de procese. Procesele unui grup sunt ordonate şi fiecare proces este identificat prin numărul său de ordine din grup. Pentru un grup de ***n*** procese, numerele valide au valori de la ***0*** la ***n-1***.

2.1.4. Comunicator

Informaţiile de **contex**t şi de **grup** sunt asamblate într-un parametru suplimentar al operaţiilor de comunicare, denumit **comunicator**. Mesajele poartă, în afara datelor, informaţii care permit diferenţierea lor şi recepţia lor selectivă. Aceste informaţii sunt:

- Sursă
- Destinatar
- Tag
- Comunicator

şi constituie **plicul** (anvelopa) mesajului. MPI prevede un comunicator predefinit, MPI_COMM_WORLD., care permite comunicarea cu orice proces accesibil după iniţializarea MPI şi asocierea unui număr de ordine (rank) fiecărui proces. Rolul şi modalitatea de a defini noi comunicatori vor fi discutate mai târziu.

Cu aceste elemente, sintaxa operaţiilor **send** şi **receive** *blocante* este următoarea:

MPI_SEND (buf, count, datatype, dest, tag, comm)

unde parametrii sunt:

- IN buf, adresa iniţială a tamponului sursă
- IN count, numărul de elemente (întreg ne-negativ)
- IN datatype, tipul fiecărui element
- IN dest, numărul de ordine al destinatarului (întreg)
- IN tag, tipul mesajului (întreg)
- IN comm, comunicatorul implicat

MPI_RECV (buf, count, datatype, source, tag, comm, status)

unde parametrii sunt:

- OUT buf, adresa iniţială a tamponului destinatar
- IN count, numărul de elemente din tampon (întreg ne-negativ)
- IN datatype, tipul fiecărui element
- IN source, numărul de ordine al sursei (întreg)
- IN tag, tipul mesajului (întreg)
- IN comm, comunicatorul implicat
- OUT status, starea, element (structură) ce indică sursa, tipul şi contorul mesajului efectiv primit

În limbajul C aceste funcţii au următoarele prototipuri, valorile returnate reprezentând coduri de eroare:

```
int MPI_Send(void *buf, int count, MPI_Datatype datatype, int dest, int tag, MPI_Comm comm);
int MPI_Recv(void *buf, int count, MPI_Datatype datatype, int source, int tag, MPI_Comm comm, MPI_Status *status);
```

Operaţia **send** este blocantă. Ea nu redă controlul apelantului până când mesajul nu a fost preluat din tamponul sursă, acesta din urmă putând fi reutilizat de transmiţător. Mesajul poate fi copiat direct în tamponul destinatar (dacă se execută o operaţie **recv**) sau poate fi salvat într-un tampon temporar al sistemului. Memorarea temporară a mesajului *decuplează* operaţiile **send** şi **recv**, dar este consumatoare de resurse. Alegerea unei variante aparţine implementatorului MPI.

[a]i operaţia **recv** este blocantă: controlul este redat programului apelant doar când mesajul a fost recepţionat. Există şi alte operaţii MPI, care permit programatorului să controleze modul de comunicare. Vom reveni asupra acestui subiect.

2.2. Un exemplu complet

Utilizarea funcţiilor **send** şi **recv** este ilustrată în următorul exemplu:

```
# include “mpi.h”
main (int argc, char **argv)
{ char message[40];
  int myrank;
  MPI_Status status;
  MPI_Init (&argc, &argv);
  MPI_Comm_rank (MPI_COMM_WORLD, &myrank);
  if (myrank==0)
    { strcpy (message, “Hello, there”);
      MPI_Send (message, strlen(message), MPI_CHAR, 1, 99,
          MPI_COMM_WORLD);
    }
  else { MPI_Recv (message, 20, MPI_CHAR, 0, 99, MPI_COMM_WORLD, &status);
        printf (“\tReceived: %s \n”, message);
      }
  MPI_Finalize();
}
```

În acest exemplu, procesul 0 (myrank==0) trimite un mesaj procesului 1 folosind operaţia **send**. Procesul 1 (myrank!=0) primeşte acest mesaj prin operaţia **recv** şi îl afişează. Descrierile celor două procese coincid, iar diferenţierea între acţiunile lor se face prin selecţia după numărul de ordine al procesului.

2.3. Mai multe despre recepţie

Recepţia unui mesaj este guvernată de valorile din anvelopa sa. Un mesaj este primit de destinatarul său dacă anvelopa conţine valorile specificate de operaţia MPI_Recv pentru argumentele **source**, **tag** şi **comm**. Receptorul poate specifica MPI_ANY_SOURCE, respectiv MPI_ANY_TAG pentru a accepta mesaje de la orice sursă şi/sau cu orice tag. El nu poate specifica valori oarecare pentru comunicatorul **comm** !! Deci mesajul nu poate fi primit decât de receptorul adresat şi pentru un acelaşi comunicator

În cazul folosirii unor valori ANY pentru sursă şi tag, valorile corespunzătoare unui mesaj recepţionat pot fi aflate din parametrul **status**. În limbajul C, status este o structură cu trei câmpuri, MPI_SOURCE, MPI_TAG, MPI_ERROR, care poate conţine însă şi câmpuri adiţionale

Lungimea mesajului recepţionat poate fi aflată prin apelul operaţiei **MPI_Get_count**.

2.4. Corespondenţa tipurilor şi conversiile datelor

Corespondenţa tipurilor datelor se referă două aspecte:

- Corespondenţa tipurilor limbajului gazdă (C) cu tipurile specificate în operaţiile de comunicare;
- Corespondenţa tipurilor la transmiţător şi receptor.

În plus, folosirea standardului în medii eterogene necesită **conversii** de tip. Aceste aspecte nu sunt tratate în lucrarea de faţă.

2.5. Alte operaţii MPI

Prezentarea exemplului anterior ne oferă prilejul să comentăm şi alte operaţii MPI. Fiecare program MPI trebuie să conţină un apel al operaţiei **MPI_Init**. Rolul acesteia este de a iniţializa "mediul" în care programul va fi executat. Ea trebuie executată *o singură dată*, *înaintea* altor operaţii MPI. Pentru a evita execuţia sa de mai multe ori (şi deci provocarea unei erori), MPI oferă posibilitatea verificării dacă MPI_Init a fost sau nu executată. Acest lucru este făcut prin

MPI_INIATIALIZED (flag)

- OUT flag, este **true** dacă MPI_Init a fost deja executată

În limbajul C funcţia are următorul prototip:

```
int MPI_Initialized(int *flag);
```

Aceasta este, de altfel, singura operaţie ce poate fi executată înainte de MPI_Init.

Revenind la MPI_Init, forma sa generală este

MPI_INIT()

iar în limbajul C prototipul este

```
int MPI_Init (int *argc, char ***argv);
```

Funcţia C de iniţializare acceptă ca argumente **argc** şi **argv**, argumente ale funcţiei **main**, a căror utilizare nu este fixată de standard şi depinde de implementare.

Perechea funcţiei de iniţializare este **MPI_Finalize**, care trebuie executată de fiecare proces, pentru a "închide" mediul MPI. Forma acestei operaţii este următoarea:

MPI_FINALIZE()

iar în limbajul C prototipul este

```
int MPI_Finalize (void);
```

Utilizatorul trebuie să se asigure că toate comunicaţiile în curs de desfăşurare s-au terminat înainte de apelul operaţiei de finalizare. După MPI_Finalize, nici o altă operaţie MPI nu mai poate fi executată (nici măcar una de iniţializare).

Un proces poate afla poziţia sa în grupul asociat unui comunicator prin apelul operaţiei

MPI_COMM_RANK (comm, rank)

- IN comm, este comunicatorul implicat
- OUT rank, este rangul procesului apelant

În limbajul C funcţia are următorul prototip:

```
int MPI_Comm_rank(MPI_Comm comm, int *rank);
```

Procesele din exemplul dat folosesc răspunsul acestei operaţii în decizia care urmează în program.

2.6. Alte moduri de comunicare

Modul standard de comunicare în MPI este blocant: controlul nu este redat apelantului până când mesajul nu a fost preluat din tamponul de emisie (într-un tampon intermediar **sau** în tamponul receptorului), astfel încât procesul transmiţător poate să modifice din nou tamponul de emisie.

Memorarea temporară a mesajelor, în cazul unor operaţii blocante, nu este întotdeauna operaţională. Dacă mesajele transmise sunt foarte lungi şi sistemul nu dispune de suficientă memorie temporară, ele trebuie transferate direct din tamponul sursă în tamponul destinatar. Aceasta presupune, însă, că transmiţătorul este blocat până la execuţia unei recepţii corespondente.

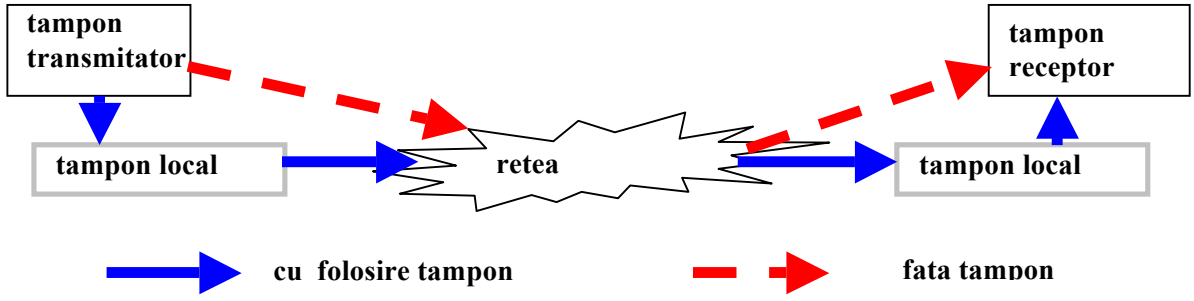


Figura 2

În cazul unei combinaţii de procese care-şi transmit mesaje în lanţ, mecanismul conduce la degradarea performanţelor, aşa cum rezultă şi din exemplul prezentat în figura 3:

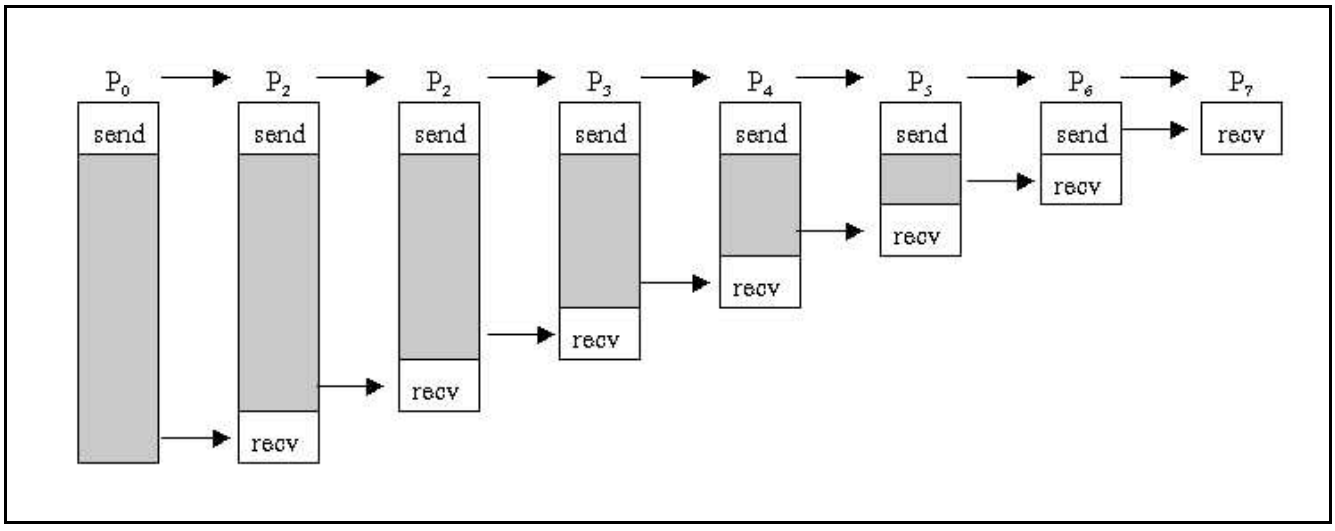


Figura 3

Fiecare proces, cu excepţia primului şi a ultimului, conţine o pereche de operaţii send-recv: prima transmite un mesaj către următorul proces din secvenţă, a doua recepţionează un mesaj de la precedentul proces. Transmiterea directă a datelor între tampoanele de emisie şi recepţie secvenţializează execuţiile operaţiilor de comunicare, procesele rămânând în aşteptare (zonele colorate din figura 1) perioade lungi de timp. Există mai multe soluţii pentru a controla execuţia operaţiilor de comunicare astfel încât programul să nu depindă de cantitatea de memorie tampon oferită de sistem.

2.6.1. Ordonarea operaţiilor send-recv

O soluţie este împerecherea operaţiilor, de exemplu, procesele de rang par execută operaţiile în ordinea send-recv, în timp ce procesele de rang impar în ordine inversă, recv-send.

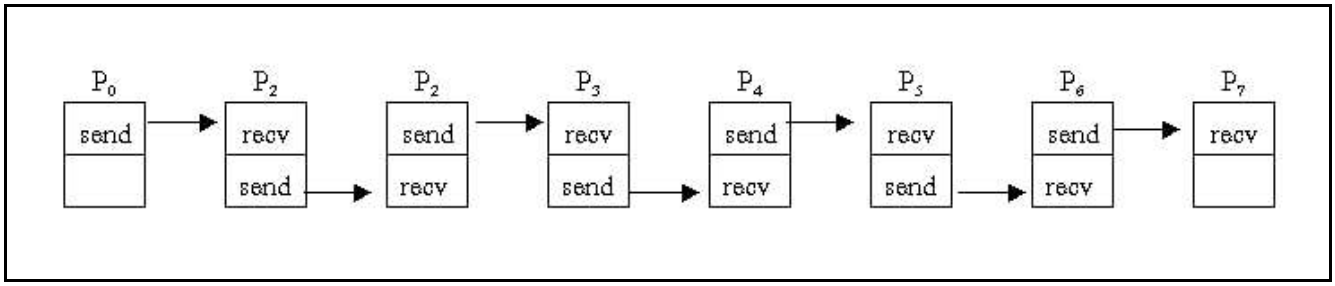


Figura 4

2.6.2. Operaţii send-recv combinate

Schema precedentă este greu de implementat în cazul unei combinaţii mai complicate de procese. O alternativă este utilizarea operaţiei **MPI_Sendrecv**, care combină într-un singur apel transmiterea unui mesaj către o destinaţie cu recepţia unui mesaj de la un alt proces. Subsistemul de comunicare se ocupă de combinarea operaţiilor astfel încât să se evite blocarea definitivă a proceselor.

Ambele operaţii, send şi recv, folosesc acelaşi comunicator, dar eventual diferite tag-uri. Tampoanele de emisie şi de recepţie trebuie să fie distincte.

Operaţia are şi o variantă, **MPI_Sendrecv_replace**, în care acelaşi tampon este folosit atât pentru emisie cât şi pentru recepţie, astfel că mesajul transmis este înlocuit cu mesajul recepţionat.

2.6.3. Transmitere prin tampon

MPI permite programatorului să prevadă un tampon în care datele sunt plasate până la livrarea lor. Operaţi **MPI_Bsend** se termină, eventual înainte de demararea unei recepţii corespondente, odată cu plasarea datelor în zona tampon. Programatorul poate rezerva această zonă prin operaţia **MPI_Buffer_attach**, în care specifică dimensiunea dorită, suficient de mare pentru a păstra mesajele în tranzit. Când tamponul nu mai este necesar, se foloseşte **MPI_Buffer_detach**.

Exemplul următor ilustrează utilizarea acestora:

```
#define BuffSize 100000
int size;
char *buff;
MPI_Buffer_attach(malloc(BuffSize), BuffSize);
/* Tamponul de BuffSize octeti poate fi folosit acum */
MPI_Bsend( ... );
MPI_Buffer_detach(&buff, &size);
/* Tamponul redus la zero */
MPI_Buffer_attach(buff, size);
/* Tamponul disponibil din nou */
```

Observaţii:

- După detaşare, utilizatorul poate reutiliza sau dealoca spaţiul ocupat de tampon
- Unele operaţii, de ataşare şi deataşare, au un argument de tip **void***; ele sunt folosite diferit – un **pointer** la tampon este transmis la *attach*; **adresa pointerului** este utilizată la *detach*, astfel că acest apel întoarce valoarea pointerului. Argumentele sunt definite ambele ca void * (şi nu void * respectiv void **) pentru a evita conversii forţate de tip (cast). În exemplul precedent, &buff, care este de tip char **, poate fi transmisă ca argument fără a folosi **cast**.

2.6.4. Transmitere sincronă

Operaţia de transmitere sincronă **MPI_Ssend** poate demara înaintea recepţiei mesajului, dar se termină doar când operaţia de recepţie corespondentă a început să preia mesajul transmis.

2.6.5. Transmitere în modul pregătit

Operaţia de transmitere în modul pregătit, **MPI_Rsend** poate fi demarată **numai dacă** recepţia corespondentă a fost demarată anterior.

2.7. Comunicaţia non-blocantă

Pentru a permite suprapunerea calculelor cu transferul de date, MPI oferă şi operaţii send, recv **neblocante**, pentru toate formele de comunicare: standard, cu tampon, sincron, în mod pregătit. Controlul este redat procesului apelant imediat după demararea operaţiei, împreună cu un identificator al cererii (de tip **MPI_Request**). Programatorul trebuie să specifice, prin operaţii explicite, terminarea transferului. Oricum, el poate programa alte operaţii (de prelucrare) în paralel cu transferul datelor.

MPI_Isend demarează o operaţie standard neblocantă de transmisie. Terminarea operaţiei poate fi verificată prin **MPI_Test**. În plus, **MPI_Wait** pune în aşteptare procesul executant până la terminarea operaţiei. Pe parcursul desfăşurării operaţiei de transmitere, zona care conţine mesajul nu trebuie modificată.

Operaţia **MPI_Irecv** începe o recepţie neblocantă. Terminarea ei poate fi testată cu **MPI_Test** şi aşteptată cu **MPI_Wait**.

Pentru testarea sau aşteptarea terminării uneia sau a tuturor operaţiilor neblocante ale unei colecţii de operaţii, MPI furnizează operaţiile

MPI_Testall	MPI_Testany	MPI_Testsome
MPI_Waitall	MPI_Waitany	MPI_Waitsome

De exemplu, putem începe două operaţii neblocante, aşteptând apoi terminarea ambelor operaţii. Aceasta ne permite realizarea unei benzi de asamblare în care fiecare proces primeşte date din stânga şi le transmite în dreapta. În timp ce sosesc noile date, precedentele sunt folosite în calcule.

```
MPI_Request request[2];
MPI_Status status[2];
. . .
while (! done)
  {  MPI_Irecv (buf1, . . . , &request[0]);
     MPI_Isend (buf2, . . . , &request[1]);
/* calcule cu buf2 */
     MPI_Waitall (2, request, status);
/* schimba buf1 cu buf2 */
  }
```

Observaţie:

- Un server cu mai mulţi clienţi ar trebui să folosească **MPI_Waitsome** şi nu **MPI_Waitany**, pentru a nu "îngheţa" nici un client.

MPI_Waitsome aşteaptă până când cel puţin una din cererile active este terminată, dar întoarce numărul cererilor terminate şi lista indicilor acestora (în tabloul de cereri). Se pot trata apoi toate cererile terminate.

MPI_Waitany întoarce indexul unei singure cereri, alese aleator, ceea ce poate duce la "îngheţarea" unui anumit client, defavorizat de această alegere, asa cum se arata in exemplul urmator.

Considerăm o colectie de **size** procese. Procesele de la 0 la size-2 sunt **clienti** iar procesul **size-1** are rolul de **server**. Clientii produc in mod repetat blocuri de date pe care le trimit serverului. Serverul asteapta primirea blocurilor de date de la oricare client. Asteptarea poate fi programata prin **MPI_Waitany** sau prin **MPI_Waitsome**. In primul caz, serverul este instiintat ca a primit un bloc de date de la un client ales aleator de sistem, va prelua datele si va initia o noua asteptare pentru acel client. In aceasta varianta, este posibil ca alegerea aleatoare sa "ocoleasca" sistematic unul din cleinti. In al doilea caz, serverul este instiintat despre toti clientii de la care s-au primit blocuri de date, le va prelua si va programa asteptarea unui nou bloc de date de la fiecare client.

```
typedef struct {
       char data [MSIZE];
       int dsize;
} Buffer;
Buffer buf[];
MPI_Request req[];
MPI_Status status[];
int index[];
...
MPI_Comm_rank(comm, &rank);
MPI_Comm_size(comm, &size);
if(rank != size-1) {                                  /* client code */
  buf = (Buffer *) malloc(sizeof(Buffer));
  while(1) {                                        /* clientii pregatesc si trimit repetat blocuri de date */
      produce_request( buf->data, &buf->dsize);
      MPI_Send(buf->data, buf->dsize, MPI_CHAR, size-1, tag, comm);
  }
}
else
{         /* rank == size-1; server code */
  buf = (Buffer *) malloc(sizeof(Buffer)*(size-1));
  req = (MPI_Request *) malloc(sizeof(MPI_Request)*(size-1));
  status = (MPI_Status *) malloc(sizeof(MPI_Status)*(size-1));
  index = (int *) malloc(sizeof(int)*(size-1));
  for(i=0; i < size-1; i++)                          /* programeaza asteptarea receptiei datelor */
    MPI_Irecv(buf[i].data, MSIZE, MPI_CHAR, i, tag, comm, &req[i]);
  while (1){                                        /* asteapta sosiri blocuri de date */
       MPI_Waitsome(size-1, req, &count, index, status);
       for (i=0; i < count; i++){
          j = index[i];
         MPI_Get_count(&status[i], MPI_CHAR, &buf[j].dsize);
         process_request(buf[j].data, buf[j].dsize);
         MPI_Irecv(buf[j].data, MSIZE, MPI_CHAR, j, tag, comm, &req[j]);
       }
  }
}
```

2.8. Cereri de comunicare persistente

Când ciclul din exemplul anterior (pipeline!) se repetă de multe ori este avantajoasă crearea unei cereri de transmitere / recepţie **persistente**, folosite ulterior de oricâte ori. Aceasta simplifică dialogul dintre program şi "infrastructura" de comunicare, evitând retransmiterea parametrilor operaţiei, la fiecare apel. Programatorul poate crea o cerere persistentă (**MPI_Send_init**, **MPI_Recv_init**, şi toate celelalte variante, cu tampon sincrone şi în mod pregătit), poate declanşa operaţia (**MPI_Start**, **MPI_Startall**) de comunicare, poate aştepta terminarea (**MPI_Wait**) pentru a declanşa o nouă operaţie de comunicare, sau poate dealoca cererea persistentă cu **MPI_Request_free**.

```
MPI_Recv_init ( . . ., &request);
MPI_Start (request); // are semantica operatiei immediate MPI_Irecv
. . .
MPI_Wait (&request);
MPI_Request_free (&request);
```

2.9. Procese nule

În cazul transferurilor între procesele benzii de asamblare, este convenabil ca pentru procesele de la margine să se considere ca sursă (procesul din stânga) respectiv destinaţie (procesul din dreapta) nişte procese **fictive**.

Valoarea specială **MPI_PROC_NULL** poate fi folosită în locul unui rang de proces sursă sau destinatar. Comunicarea cu un proces fictiv nu are nici un efect. O transmitere de mesaj se termină întotdeauna cu succes. O recepţie de la un proces fictiv se încheie imediat fără a modifica tamponul de recepţie.

3. Tipuri de date MPI

O caracterisitică importantă a MPI este includerea unui argument referitor la tipul datelor transmise / recepţionate.

MPI are o mulţime bogată de tipuri predefinite: toate tipurile tipurile de bază din C (şi din FORTRAN) plus MPI_BYTE şi MPI_PACKED. De asemenea, MPI furnizează constructori pentru tipuri derivate şi mecanisme pentru descrierea unui tip general de date.

3.1. Aspecte generale

În cazul general, mesajele pot conţine valori de tipuri diferite, care ocupă zone de memorie de lungimi diferite şi ne-contigue. În MPI un **tip de date** este un obiect care specifică o secvenţă de tipuri de bază şi deplasările asociate acestora, relative la tamponul de comunicare pe care tipul îl descrie. O astfel de secvenţă de perechi (tip, deplasare) se numeşte **harta tipului**. Secvenţa tipurilor (ignorând deplasările) formează **semnătura tipului** general.

Typemap = $\{(type_0, disp_0), \ldots, (type_{n-1}, disp_{n-1})\}$

Ex. {(int, 0), (int, 4), (short, 8), (byte, 10), (short, 12)}

Typesig = $\{type_0, \ldots, type_{n-1}\}$

Ex, {int, int, short, byte, short}

Harta tipului, împreună cu o adresă de bază **buf** descriu complet un tampon de comunicare:

- Acesta are *n* intrări
- Fiecare intrare *i* are tipul $type_i$ şi începe la adresa buf+ $disp_i$

Observaţie:

- Ordinea perechilor în Typemap nu trebuie să coincidă cu ordinea valorilor din tamponul de comunicare.

Putem asocia un **titlu** (handle) unui tip general şi putem folosi acest titlu în operaţiile de transmitere / recepţie, pentru a specifica tipul datelor comunicate.

Tipurile de bază sunt cazuri particulare, predefinite. De exemplu, MPI_INT are harta {(int, 0)}, cu o intrare de tip int şi cu deplasament zero.

Pentru a înţelege modul în care MPI asamblează datele, utilizăm noţiunea de **extindere** (extent). Extinderea unui tip este spaţiul dintre primul şi ultimul octet ocupat de intrările tipului, rotunjit superior din motive de aliniere.

De exemplu, Typemap = {(double, 0), (char, 8)} are extinderea **16**, dacă valorile double trebuie aliniate la adrese multiplu de 8. Deci, chiar dacă dimensiunea tipului este **9**, din motive de aliniere, o nouă valoare începe la o distanţă de 16 octeţi de începutul valorii precedente.

Extinderea şi dimensiunea unui tip de date pot fi aflate prin apelurile funcţiilor

```
int MPI_Type_extent (MPI_Datatype datatype, MPI_Aint *extent);
int MPI_Type_size(MPI_Datatype datatype, int *size);
```

Deplasările sunt relative la o anumită adresă iniţială de tampon. Ele pot fi substituite prin **adrese absolute**, care reprezintă deplasări relative la "adresa zero" simbolizată de constanta MPI_BOTTOM. Dacă se folosesc adrese absolute, argumentul **buf** din operaţiile de transfer trebuie să capete valoarea MPI_BOTTOM. Adresa absolută a unei locaţii de memorie se află prin

```
int MPI_Address (void *location, MPI_Aint *address);
```

unde **MPI_Aint** este un tip întreg care poate reprezenta o adresă oarecare (de obicei este int) MPI_Address reprezintă un mijloc comod aflare a deplasărilor, chiar dacă nu se foloseşte adresarea absolută în operaţiile de comunicare.

3.2. Tipuri derivate

MPI prevede **constructori** de tipuri derivate.

3.2.1. Tipul contiguu

Cel mai simplu constructor produce un nou tip de date făcând mai multe copii ale unui tip existent, cu deplasări care sunt multipli ai extensiei tipului vechi.

De exemplu, dacă **oldtype** are harta {(int, 0) , (double, 8)} atunci noul tip creat prin

```
MPI_Type_contiguous (2, oldtype, &newtype);
```

are harta {(int, 0) , (double, 8), (int, 16) , (double, 24)}

Noul tip trebuie încredinţat sistemului înainte de utilizare:

```
MPI_Type_commit (&newtype);
```

pentru ca acesta să poată face optimizările de performanţă posibile. Când un tip de date nu mai este folosit, el trebuie eliberat cu:

```
MPI_Type_free (&newtype);
```

Observaţie:

- Utilizarea tipului contiguu este echivalentă cu folosirea unui contor mai mare ca 1 în operaţiile de transfer. Astfel apelul:

```
MPI_Send (buffer, count, datatype, dest, tag, comm);
```

este similar cu:

```
MPI_Type_contiguous (count, datatype, &newtype);
MPI_Type_commit (&newtype);
MPI_Send (buffer, 1, newtype, dest, tag, comm);
MPI_Type_free (&newtype);
```

3.2.2. Tipul vector

Acesta permite specificarea unor date situate în zone necontigue de memorie. Elementele tipului vechi pot fi separate între ele de spaţii având lungimea egală cu un multiplu al extinderii tipului (deci cu un **pas constant**). Pentru exemplificare, să considerăm o matrice de 5 linii şi 7 coloane (cu elementele de tip float memorate pe linii). Pentru a construi tipul de date corespunzător unei coloane folosim operaţia:

```
MPI_Type_vector (5, 1, 7, MPI_FLOAT, &newtype);
```

unde

- 5 este numărul de blocuri;
- 1 este numărul de elemente din fiecare bloc (în cazul în care acest număr este mai mare decât 1, un bloc se obţine prin concatenarea numărului respectiv de copii ale tipului vechi);
- 7 este **pasul**, deci numărul de elemente între începuturile a două blocuri vecine;
- MPI_FLOAT este vechiul tip.

Pentru a transmite coloana a treia a matricii folosim:

```
MPI_Send (&a[0][2], 1, newtype, dest, tag, comm);
```

3.2.3. Tipul hvector

Este similar tipului **vector**, exceptând faptul că pasul se dă în număr de octeţi şi nu în număr de elemente.

3.2.4. Tipul indexed

În cazul acestui tip, fiecare bloc are un număr particular de copii ale tipului vechi şi un deplasament diferit de ale celorlalte. Deplasamentele sunt date în multipli ai extinderii vechiului tip.

Utilizatorul trebuie să specifice ca argumente ale constructorului de tip un tablou de numere de elemente per bloc şi un tablou de deplasări ale blocurilor.

3.2.5. Tipul hindexed

Este similar tipului indexat, cu diferenţa că deplasările sunt măsurate în octeţi.

3.2.6. Tipul struct

Este o generalizare a tipului precedent, prin aceea că permite ca fiecare bloc să constea din replici ale unor tipuri de date diferite.

3.2.7. Exemplu

În multe probleme de simulare din fizică (N-Body) trebuie considerate interacţiuni între particule sau obiecte. O reprezentare posibilă a unei colecţii de particule este următoarea:

```
# define MAX_PART 1000
Struct Part_struct
 { int class;     /* clasa particulei */
   double d[6];   /* coordonatele particulei */
```

```
   char b[7];     /* alte informatii */
  };
struct Part_struct particle[MAX_PART];
int i, dest, rank;
MPI_Comm comm;

/* constructia tipului care descrie structura MPI a unei particule, ParticleType */
MPI_Datatype ParticleType;
MPI_Datatype type[3] = {MPI_INT, MPI_DOUBLE, MPI_CHAR}; /* semnat */
int blocklen [3] = { 1, 6, 7};   /* lungimi blocuri */
MPI_Aint disp[3];              /* deplasari */
int base;

/* calculul deplasarilor */
MPI_Address (particle, disp);
MPI_Address (particle[0].d, disp+1);
MPI_Address (particle[0].b, disp+2);
base=disp[0];
for (i=0; i<3; i++) disp [i] -= base;
MPI_Type_struct ( 3, blocklen, disp, type, &ParticleType);

/* transmiterea intregului tablou */
MPI_Type_commit (&ParticleType);
MPI_Send (particle, MAX_PART, ParticleType, dest, tag, comm);
```

Construim **tipul** ce corespunde particulelor de clasa zero, precedate de numărul acestor intrări. Construcţia se face în două etape:

- Se construieşte tipul indexat al particulelor cu clasa zero, **Zparticle**; particulele succesive având clasa zero sunt tratate ca un bloc;
- Se construieşte tipul struct **Ztype** cu contorul de particule "zero" şi particulele propriu zise.

```
MPI_Datatype Zparticle;    /* toate particulele cu clasa zero */
MPI_Aint zdisp [MAX_PART]; /* tabloul deplasarilor blocurilor */
int zblock [MAX_PART],j,k; /* tabloul numerelor de elem per bloc */
MPI_Aint zdisp [MAX_PART]; /* tabloul deplasarilor blocurilor */

MPI_Datatype Ztype;       /* include contorul de particule zero */
int zzblock [2]={1,1};     /* lungimi blocuri, */
MPI_Aint zzdisp [2];       /* deplasari si */
MPI_Datatype Zztype[2];    /* tipuri pentru Ztype */

/* particule consecutive cu clasa 0 sunt tratate ca un bloc */
j=0;                    /* numar de blocuri */
for(i=0; i<MAX_PART; i++)  /* parcurge toate particulele */
 if (particle[i].class == 0)
```

```
   {  for (k=i+1; (k<MAX_PART)&&(particle[k].class == 0); k++) ;
      zdisp[j] = i;
      zblock[j] = k-i;    /* lungimea blocului */
      j ++;
      i = k;
   }
MPI_Type_indexed(j, zblock, zdisp, ParticleType, &Zparticle);

/* pregatesc contorul de particule */
MPI_Address(&j, zzdisp);
MPI_Address(particle, zzdisp+1);
zztype[0] = MPI_INT;
zztype[1] = Zparticle;
MPI_Type_struct(2, zzblock, zzdisp, zztype, &Ztype);

MPI_Type_commit(&Ztype);
MPI_Send (MPI_BOTTOM, 1, Ztype, dest, tag, comm);
/* adrese absolute pentru valori ale unor variabile diferite */
```

Temă:

- Descrierea transmiterii primelor două coordonate ale tuturor intrărilor
- Descrierea folosind adresa absolută în tipurile de date

3.3. Eficienta transmisiei

Folosirea tipurilor derivate faciliteaza nu doar programarea mai simpla a aplicatiilor distribuite ci si cresterea eficientei executiei acestora. Pentru a arata acest lucru, consideram transmitere a două coloane ale une matrice “a” de m linii şi n coloane de la un proces la altul. In Figura 5, m=6 si n=5, cele doua coloane de transmis fiind a doua si a treia.

5	1	2	3	4
6	7	4	6	9
9	3	2	7	0
4	1	8	3	8
8	9	2	1	4
7	1	2	3	4

5	1	2	3	4
6	7	4	6	9
9	3	2	7	0
4	1	8	3	8
8	9	2	1	4
7	1	2	3	4

Figura 5

O variantă posibilă este transmiterea celor doua coloane ca operaţii separate pe fiecare coloană. In acest scop, definim un nou tip vector (newtype) care are **m** blocuri de cate un element, situate la distanta **n** unul de altul. Transmiterea celor doua coloane se face prin doua mesaje separate, fiecare mesaj incluzand o valoare de tip **newtype**.

```
MPI_Type_vector (m, 1, n, MPI_FLOAT, &newtype);
for (i=1; i<3; i++)
        MPI_Send (&a[0][i], 1, newtype, ...);
```

Transmiterea se poate face mai eficient daca definim un tip (newtype2) care reprezinta doua coloane ale matricei: de data asta, tipul are **m** blocuri de cate **doua** elemente ale matricei, situate la distanta **n** unul de altul. Transmiterea celor doua coloane se face printr-o singura operatie MPI_Send.

```
MPI_Type_vector (m, 2, n, MPI_FLOAT, &newtype2);
MPI_Send (&a[0][1], 1, newtype2,...);
```

4. Topologii de procese

4.1. Generalităţi

Procesele unui grup de au ranguri de la 0 la *n*-1 (*n* fiind numărul de procese din grup). În multe aplicaţii, ordonarea liniară a proceselor, după rang, nu reflectă tiparul de comunicare între procese. Adesea, procesele trebuie aranjate în topologii **grilă** cu două sau mai multe dimensiuni. În cazul general, tiparul de comunicare între procese corespunde unui **graf**. Un astfel de aranjament al proceselor, care reflectă comunicările punct la punct dintre ele reprezintă **topologia virtuală** a grupului de procese.

Trebuie făcută distincţie între topologia virtuală a proceselor şi **topologia reală** a sistemului pe care acestea sunt executate. Topologia virtuală poate fi exploatată de sistem în plasarea proceselor pe procesoare, pentru a ameliora performanţele. Cum se face acest lucru este o chestiune ce iese din sfera problemelor MPI. Oricum, să reţinem că topologia virtuală reflectă caracteristicile aplicaţiei şi poate fi folosită în îmbunătăţirea performanţelor.

Sunt cunoscute tehnici standard de **mapare** a unor topologii grilă / tor pe hipercuburi sau grile de procesoare. Pentru topologiile graf metodele sunt mai complexe şi se bazează adesea pe euristici.

În afara informaţiei furnizate algoritmului de plasare a proceselor pe procesoare, topologiile virtuale au ca rol **facilitarea scrierii** programelor, într-o formă mai uşor de înţeles.

4.2. Topologii virtuale

Topologia virtuală poate fi modelată ca un **graf**, în care nodurile reprezintă procese, iar arcele perechile de procese care comunică.

Nu trebuie ca o comunicare să fie precedată de o **deschidere** explicită de canal. Ca urmare, absenţa unui arc între două noduri din graf nu înseamnă că procesele respective nu vor putea comunica între ele. Pur şi simplu, înseamnă că o astfel de comunicare nu contează la maparea topologiei virtuale de procese pe o topologie reală de precesoare.

Arcele grafului de comunicare nu sunt **ponderate**. Experienţe cu tehnici similare din PARMACS arată că această informaţie (două noduri sunt sau nu conectate) este suficientă pentru o bună mapare. O altă variantă ar complica interfaţa MPI şi ar face mai dificilă activitatea programatorului, care ar trebui să specifice mai multe informaţii la conceperea aplicaţiilor.

Deşi specificarea topologiei în termeni de graf este foarte generală, în multe aplicaţii specificarea unei **topologii mai simple, regulate**, este mai convenabilă. De exemplu inel, grile bi- sau tri-dimensionale,

tor. Aceste topologii sunt complet definite prin numărul de dimensiuni şi prin numărul de procese pe fiecare axă de coordonate.

Maparea unor topologii grilă sau tor este mai simplă decât maparea unui graf. De aici şi interesul din MPI pentru tratarea separată, explicită a acestora.

Într-o structură **carteziană**, numărătoarea pe fiecare coordonată începe de la zero, iar variaţia corespunde unei numărări pe linii. De exemplu, pentru o grilă de 2x2, corespondenţa între rangul proceselor şi coordonatele lor este următoarea:

Coord (x,y)	Rang k
Coord (0,0)	0
Coord (0,1)	1
Coord (1,0)	2
Coord (1,1)	3

O descompunere carteziană bi-dimensională este următoarea:

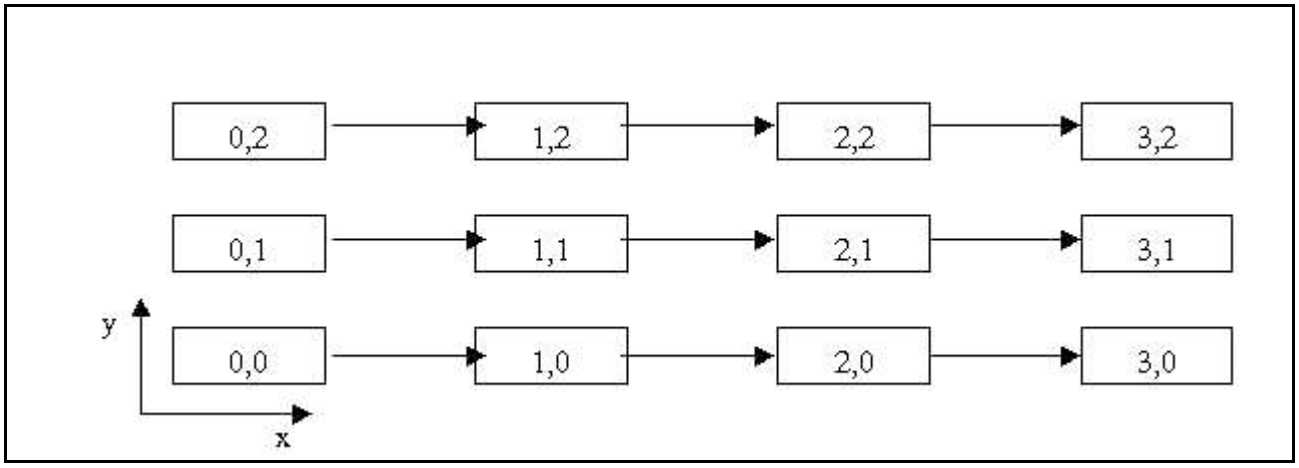


Figura 6.

În figură este schiţată şi o deplasare cu 1 după prima dimensiune.

4.3. Rutine pentru topologii carteziene

MPI include o colecţie de rutine pentru definirea, examinarea, manipularea topologiilor carteziene. Funcţia

MPI_Cart_create (comm_old,ndims,dims,periods,reorder,comm_cart);

crează o topologie carteziană a proceselor din grupul specificat de **comm_old** şi întoarce un nou comunicator, **comm_cart**, la care este ataşată informaţia de topologie carteziană (ca informaţie ascunsă).

Utilizatorul poate preciza numărul de dimensiuni **ndims**, şi numărul de procese pe fiecare dimensiune în tabloul **dims**. Argumentul **periods** (**tablou** boolean de dimensiune **ndims**) precizează dacă grila este periodică sau nu, pe fiecare dimensiune (pentru o grilă periodică, ultimul ultimul element pe dimensiunea respectivă este conectat cu primul)

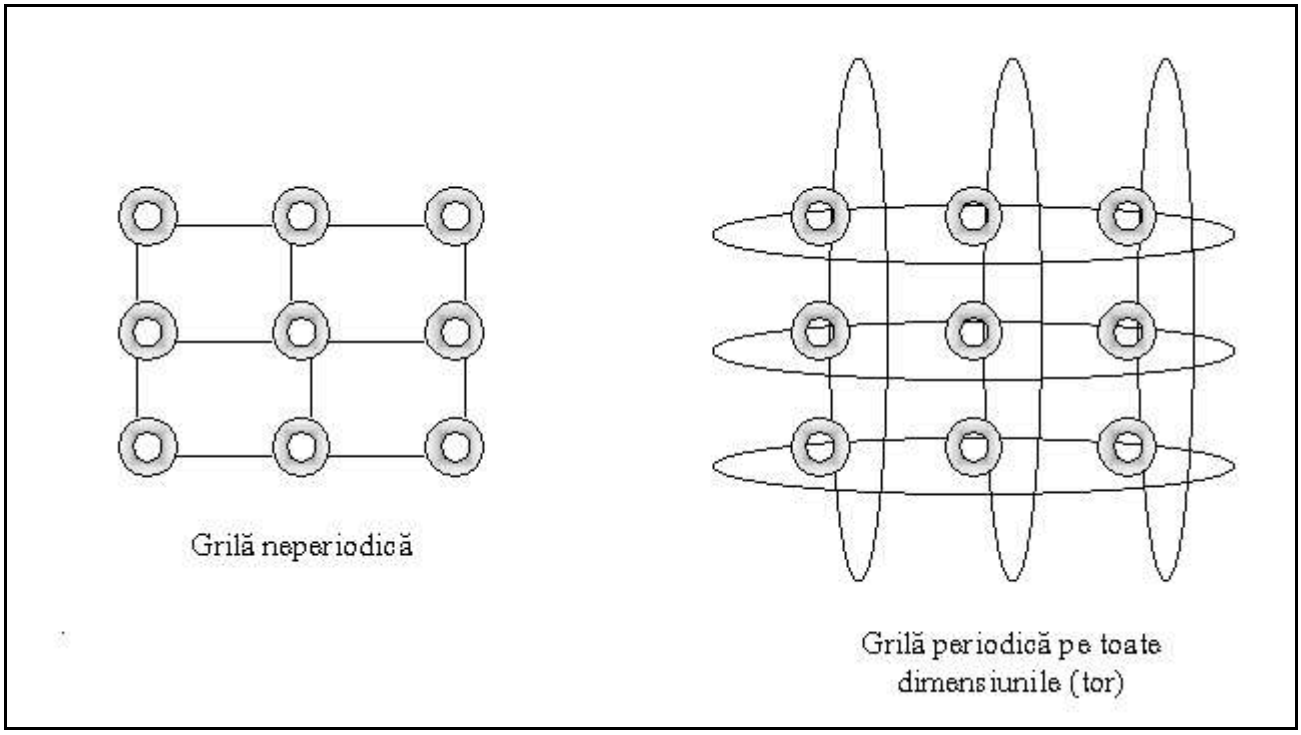


Figura 7

În fine, parametrul **reorder** precizează dacă rangurile proceselor pot fi reordonate sau nu, lăsând sau nu MPI să găsească o asignare convenabilă a elementelor topologiei. Descompunerea din figura 6 poate fi produsă de

```
dims[0] = 4; periods [0] = 0;
dims[1] = 3; periods [1] = 0;
MPI_Cart_create (MPI_COMM_WORLD,2,dims,periods,1,comm_cart);
```

Parametrul **reorder** are valoarea 1, astfel încât se permite reordonarea proceselor.

Vectorul **dims** poate fi calculat convenabil prin funcţia

```
MPI_Dims_create (nrnodes, ndims, dims);
```

care realizează o distribuţie echilibrată a celor **nrnodes** procese pe cele **ndims** dimensiuni.

Pentru a putea afla **poziţia** şi, implicit, **vecinii**, unui proces inclus într-o structură carteziană sunt două soluţii. Una este

```
MPI_Cart_get (comm, maxdims, dims, periods, coords);
```

care furnizează dimensiunile **dims**, perioadele **periods** şi coordonatele **coords** ale procesului apelant. Cea de-a doua

```
MPI_Cart_coords (comm, rank, maxdims, coords);
```

întoarce coordonatele procesului cu rangul **rank** din grupul **comm**. Argumentul **maxdims** dă dimensiunea vectorilor **dims**, **periods** şi **coords** în programul apelant.

O altă posibilitate este legată de operaţia executată de proces, în general o **deplasare** de date de-a lungul unei axe de coordonate. Astfel, să considerăm o grilă bidimensională pentru calculul produsului a două matrice. Fiecare element [i,j] păstrează elementele a_{ij}, b_{ij} ale celor două matrice şi calculează elementul c_{ij} al produsului. În acest scop sunt necesare deplasări pe linii ale elementelor **a**, în aşa fel încât [i,j] să intre în posesia tuturor elementelor a_{i1}, a_{i2}, . . . , a_{ij}, . . . , a_{in} şi deplasări pe coloane ale elementelor **b** astfel încât [i,j] să intre în posesia tuturor elementelor b_{1j}, ba_{2j}, . . . , b_{nj}. Produsul poate fi

calculat cu formula cunoscută $c_{ij} = \sum_k a_{ik} * b_{kj}$. O operaţie **MPI_Sendrecv** are ca parametru rangul unui proces sursă, pentru recepţie, şi rangul unui proces destinaţie pentru transmisie. Funcţia

```
MPI_Cart_shift (comm, direction, disp, ranksource, rankdest);
```

furnizează ambele informaţii. Utilizatorul specifică direcţia de coordonate (coordonata a cărei valoare este modificată de deplasare) şi dimensiunea pasului, **disp**, pozitiv sau negativ.

În funcţie de periodicitatea specificată pentru coordonata respectivă, deplasarea este sau nu circulară. În a doua situaţie, funcţia întoarce valoarea **MPI_PROC_NULL** dacă este cazul (pentru procesele de la margine).

4.3.1. Exemplu

Comunicatorul **comm** are asociată o topologie carteziană bidimensională, periodică. Un tablou bidimensional de valori float este memorat, un element per proces, în variabila **A**. Se doreşte modificarea tabloului prin deplasarea după direcţia 0 printr-un număr de paşi dat de a doua coordonată (coords[1]). Mesajele vor avea tagul 0.

```
int coords[2];
MPI_Comm_rank(comm, &rank);
MPI_Cart_coords (comm, rank, 2, coords);
MPI_Cart_shift (comm, 0, coords[1], &source, &dest);
MPI_Sendrecv_replace(&A,1,MPI_FLOAT,dest,0,source,0,comm,&status);
```

5. Comunicarea colectivă

Operaţiile colective implică un grup de procese. Pentru execuţie, operaţia colectivă trebuie apelată de **toate** procesele, cu argumente corespondente. Procesele implicate sunt definite de argumentul **comunicator**, care precizează totodată contextul operaţiei.

Multe operaţii colective, cum este **difuzarea**, presupun existenţa unui proces deosebit de celelalte, aflat la originea transmiterii sau recepţiei. Un astfel de proces se numeşte **rădăcină**. Anumite argumente sunt specifice rădăcinii şi sunt ignorate de celelalte procese, altele sunt comune tuturor.

Operaţiile colective se împart în mai multe categorii:

- Sincronizare de tip **barieră** a tuturor proceselor grupului
- **Comunicări** colective, în care se include:
  - Difuzarea
  - Colectarea datelor de la membrii grupului la rădăcină
  - Distribuirea datelor de la rădăcină spre membrii grupului
  - Combinaţii de colectare / distribuire (allgather şi alltoall)
- Calcule colective:
  - Operaţii de reducere
  - Combinaţii reducere / distribuire

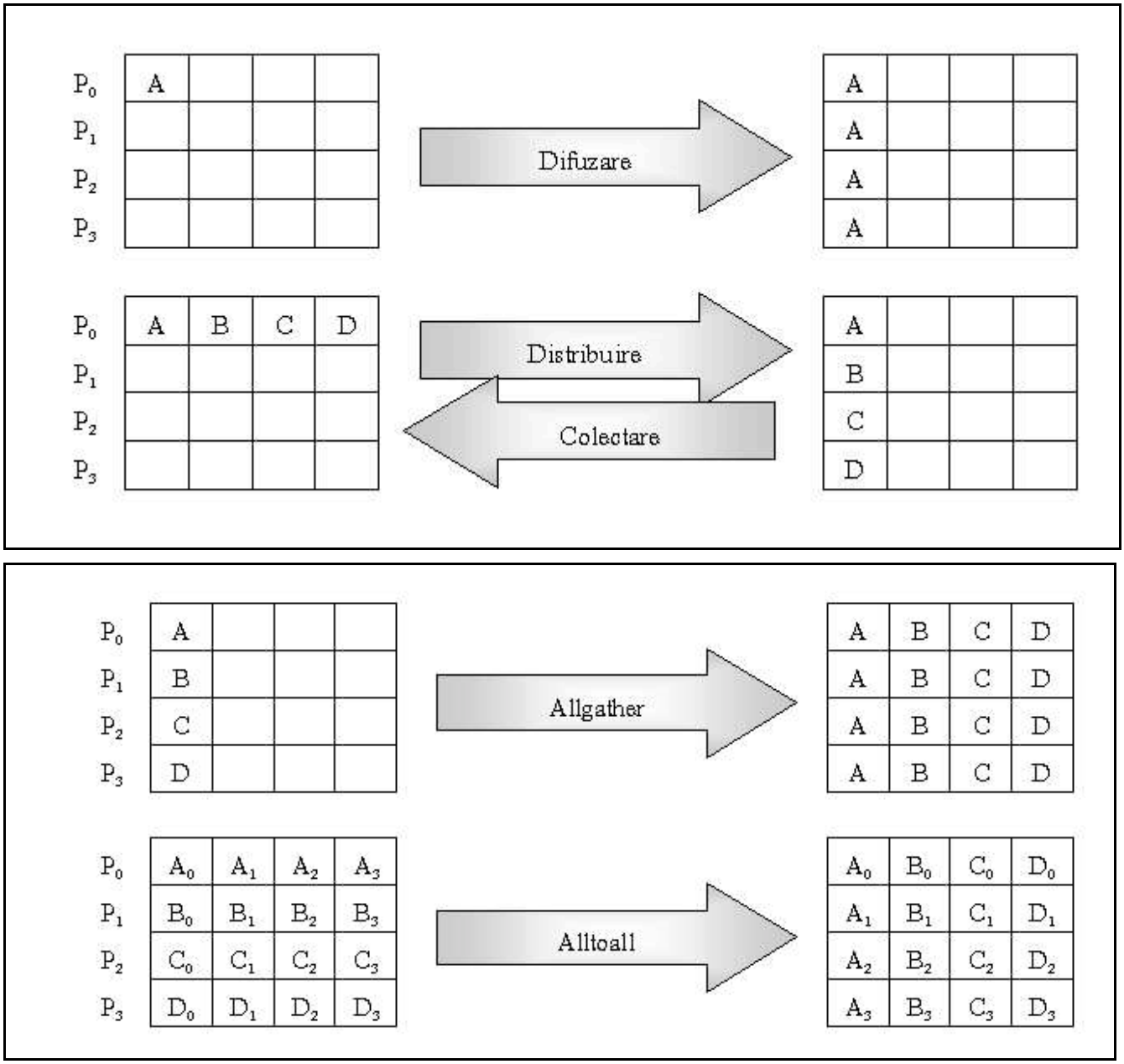


Figura 8

5.1. Caracteristici comune operaţiilor colective

5.1.1. Tipul datelor

Cantitatea de date transmisă trebuie să corespundă cu cantitatea de date recepţionată. Deci, chiar dacă hărţile tipurilor diferă, semnăturile lor trebuie să coincidă.

5.1.2. Sincronizarea

Terminarea unui apel are ca semnificaţie faptul că apelantul poate utiliza tamponul de comunicare, participarea sa în operaţia colectivă încheindu-se. Asta nu arată că celelalte procese din grup au terminat operaţia. Ca urmare, exceptând sincronizarea barieră, o operaţie colectivă nu poate fi folosită pentru sincronizarea proceselor.

5.1.3. Comunicator

Comunicările colective pot folosi aceiaşi comunicatori ca cele punct la punct. MPI garantează diferenţierea mesajelor generate prin operaţii colective de cele punct la punct.

5.1.4. Implementare

Rutinele de comunicare colectivă pot fi bazate pe cele punct la punct. Aceasta sporeşte portabilitatea, faţă de implementările bazate pe rutine ce exploatează arhitecturi paralele.

5.2. Difuzarea şi calculele colective

Exemplul următor se referă la calculul lui π prin integrare numerică cu formula: $\int_0^1 \frac{1}{1+x^2} dx = \arctan(x)\,|_0^1 = \frac{\pi}{4}$.

Pentru calculul lui π se consideră integrarea funcţiei $f(x) = \frac{4}{1+x^2}$, folosind metoda dreptunghiurilor. Intervalul (0,1) se împarte într-un număr de *n* subintervale şi se însumează ariile dreptunghiurilor având ca bază fiecare subinterval. Pentru execuţia algoritmului în paralel, se atribuie, fiecăruia dintre procesele din grup, un anumit număr de subintervale. Cele **două operaţii colective** care apar în rezolvare sunt:

- Difuzarea valorilor lui *n*, tuturor proceselor
- Însumarea valorilor calculate de procese

Operaţia

MPI_Bcast(buffer, count, datatype, root, comm)

difuzează mesajul (buffer, count, datatype) de la procesul **root** la toate procesele grupului din **comm**.

Operaţia

MPI_Reduce(sendbuf, recvbuf, count, datatype, op, root, comm)

combină valorile din **sendbuf** ale fiecărui proces, folosind operaţia **op** şi pune rezultatul în **recvbuff** în procesul **root**.

Tamponul de intrare este definit de (sendbuf, count, datatype), iar cel de ieşire de (recvbuf, count, datatype). Ambele tampoane au acelaşi număr de elemente, cu acelaşi tip. În cazul în care fiecare proces specifică o secvenţă de elemente, operaţia se realizează separat, pentru fiecare intrare a secvenţei.

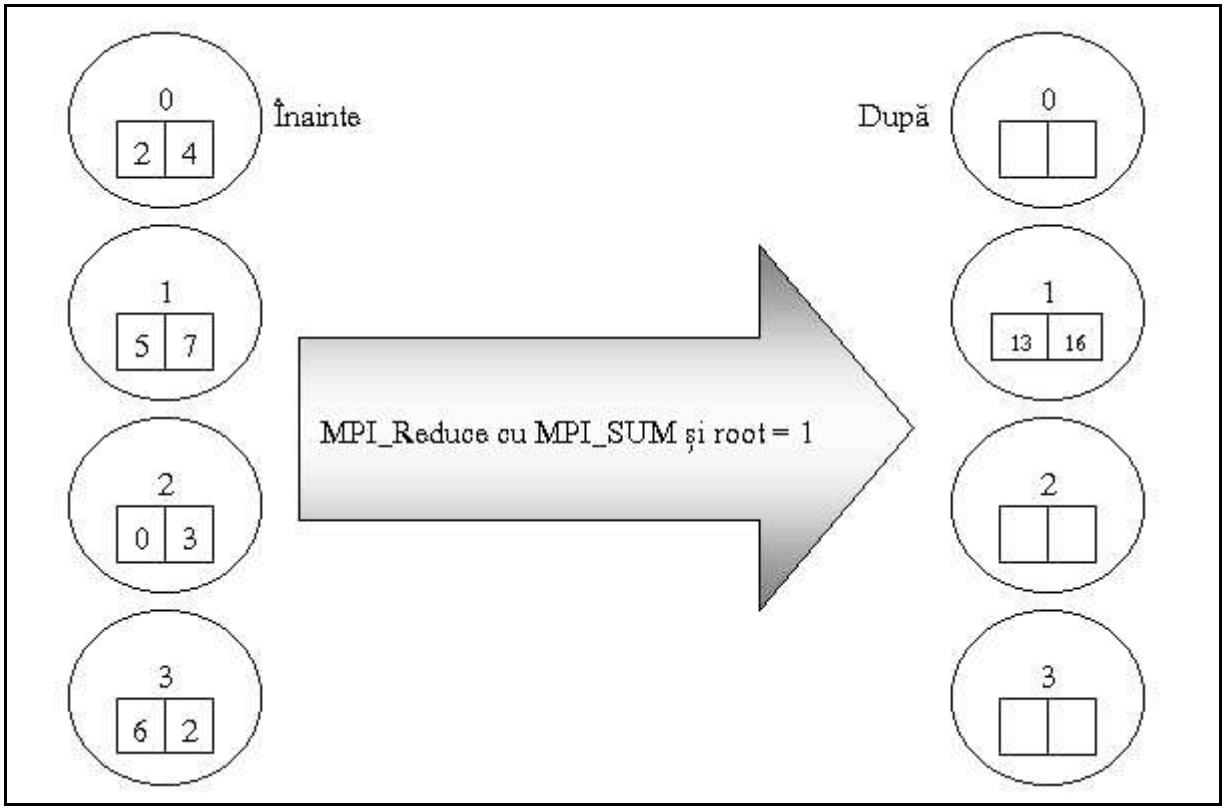


Figura 9

```
# include “mpi.h”
# include <math.h>

int main (int argc, char **argv)
{ int n, myid, numprocs, i, rc;
  double mypi, pi, h, sum, x, a;
  MPI_Status status;

  MPI_Init (&argc, &argv);
  MPI_Comm_size (MPI_COMM_WORLD, &numprocs);
  MPI_Comm_rank (MPI_COMM_WORLD, &myid);
  while (1)
   {  if (myid==0)
        {  printf(“Numar subintervale (0=gata): ”);
           scanf(“%d”,&n);
        }
      MPI_Bcast (&n, 1, MPI_INT, 0, 99, MPI_COMM_WORLD);
      if(!n) break;
      h=1.0/(double)n;
      sum=0.0;
      for (i = myid + 1; i <= n; i += numprocs)
       {  x = h*((double)i-0.5);
          sum += (4.0 / (1.0 + x*x));
       }
```

```
      mypi = h * sum;
      MPI_Reduce (&mypi, &pi, 1, MPI_DOUBLE, MPI_SUM, 0,
        MPI_COMM_WORLD);
      if (!myid) printf("pi este %f\n", pi);
   }
  MPI_Finalize();
}
```

MPI furnizează o mulţime de operatori de reducere asociativi şi comutativi, pentru aflarea maximului, muinimului, sumei, produsului, conjuncţiei, disjuncţiei, etc.

5.3. Comunicări colective

Pentru exemplificare, considerăm transferul, de la fiecare proces la rădăcină, a 100 de valori întregi. La destinaţie, grupurile de 100 de valori sunt plasate la distanţa **stride** între ele.

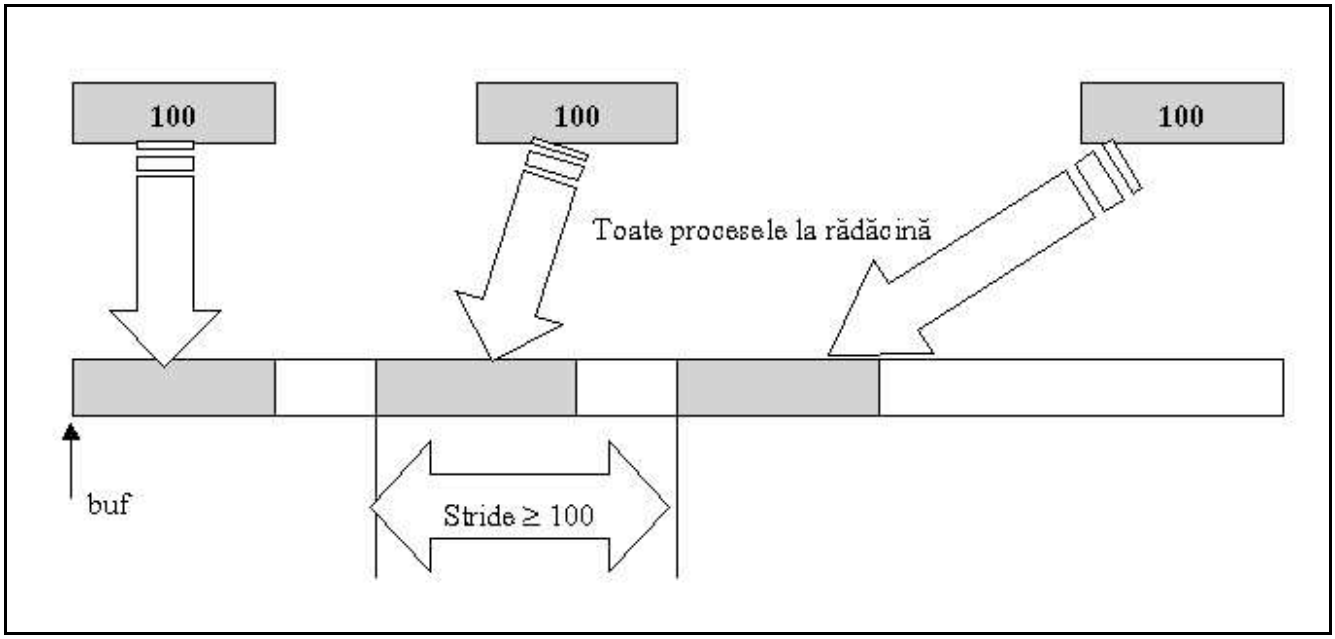


Figura 10

Se utilizează operaţia

```
MPI_Gatherv (sendbuf, sendcount, sendtype, recvbuf, recvcounts, displs, recvtype, root, comm);
```

care permite ca fiecare proces (inclusiv root) să transmită conţinutul tamponului său (sendbuf, sendcount, sendtype) procesului **root**. Contoarele pot fi diferite, astfel că la **root** valorile care le corespund sunt reprezentate de un tablou, **recvcounts**. De asemenea, datele sunt plasate în tamponul **recvbuf** al procesului **root**, în poziţii care sunt descrise de vectorul **displs**. Datele transmise de procesul *j* sunt plasate începând din poziţia displs[*j*] (măsurată în elemente de **recvtype**).

Tamponul de recepţie este ignorat pentru toate procesele diferite de root. Specificaţia contoarelor şi deplasărilor nu trebuie să conducă la scrierea de mai multe ori a unei locaţii din root.

```
MPI_Comm comm;
int gsize, sendarray[100];
int root, *rbuf, stride;
int *displs, i, *rcounts;
. . .
MPI_Comm_size (MPI_COMM_WORLD, &gsize);
```

```
rbuf = (int *) malloc (gsize * stride * sizeof(int));
displs = (int *) malloc (gsize * sizeof(int));
rcounts = (int *) malloc (gsize * sizeof(int));

for (i = 0; i < gsize; i ++)
  {  displs [i] = i * stride;
     rcounts [i] = 100;
  }
MPI_Gatherv(sendarray, 100, MPI_INT, vbuf, rcounts, displs, MPI_INT,      root, comm);
```

Observaţie.

- Programul este **eronat** pentru stride <100!

5.4. Corectitudinea operaţiilor colective

Aşa cum s-a precizat mai devreme, operaţiile colective pot avea sau nu efectul de **sincronizare** a tuturor proceselor din grup. Ca urmare, programele trebuie să invoce operaţiile colective astfel încât să se evite blocarea definitivă, **indiferent dacă operaţiile sunt sau nu sincronizate**. De aici rezultă câteva reguli:

- Operaţiile colective trebuie executate în aceeaşi ordine de toate procesele grupului
- Nu trebuie să existe dependenţe circulare între operaţii
- Operaţiile punct la punct blocante nu trebuie să "încalece" operaţiile colective

Următorul exemplu este eronat:

```
switch(rank)
  {  case 0: MPI_Bcast (..., 0, comm);
             MPI_Send (..., 1, tag, comm);
             break;
     case 1: MPI_Recv (..., 0, tag, comm, status);
             MPI_Bcast (..., 0, comm);
             break;
  }
```

A treia regulă evidenţiază un aspect foarte important: deşi contextele operaţiilor punct la punct şi colective, asociate unui comunicator sunt distincte şi, deci, traficul operaţiilor punct la punct este independent de traficul generat de operaţiile colective o programare necorespunzătoare a operaţiilor poate conduce la interblocarea proceselor, datorită posibilelor restricţii impuse de sincronizarea operaţiilor.

5.5. Determinismul operaţiilor

Modelele de programare bazate pe comunicarea de mesaje sunt nedeterministe: ordinea sosirii a două mesaje transmise de două procese A şi B unui al treilea proces C este nedefinită. (din contră, MPI asigură că două mesaje transmise de un acelaşi proces A ajung în aceeaşi ordine la B). Este **rolul programatorului** de a asigura că un calcul este determinist, acolo unde el trebuie să fie aşa.

Determinismul poate fi garantat prin definirea unui canal pentru fiecare pereche de procese care comunică. În acest caz, procesul C poate face diferenţa între mesajele primite de la A şi cele primite de la B, deoarece ele sosesc pe canale diferite. **MPI nu acceptă conceptul de canal**, dar are mecanisme **similare**, care permit unui receptor să specifice **sursa** mesajului, **tag**-ul şi **contextul**. Oricum, deoarece

MPI permite specificarea oricărei surse, MPI_ANY_SOURCE şi a oricărui tag, MPI_ANY_TAG, folosirea acestor parametri trebuie făcută cu grijă.

Să considerăm următorul fragment de program, în care un proces transmite şi primeşte de **max** ori mesaje de **date** şi apoi un mesaj rezultat.

```
for(i=0;i<max;i++)
 {  MPI_Send (buff, 600, MPI_FLOAT, r1, 1, MPI_COMM_WORLD);
    MPI_Recv (buff, 600, MPI_FLOAT, MPI_ANY_SOURCE, MPI_ANY_TAG,
         MPI_COMM_WORLD, &status);
                                                  .
                                                  .
                                                  .
 }
MPI_Send (buff, 300, MPI_FLOAT, r2, 2, MPI_COMM_WORLD);
MPI_Recv (buff, 300, MPI_FLOAT, MPI_ANY_SOURCE, MPI_ANY_TAG,
         MPI_COMM_WORLD, &status);
```

Deoarece nu se specifică nici o sursă şi nici un tag în operaţiile de recepţie, ordinea între ultimul mesaj de date şi cel de rezultate se poate inversa, conducând la un rezultat eronat. Eroarea poate fi îndreptată prin specificarea unor valori precise pentru tag-uri şi, eventual, pentru procesele sursă.

8. Ceasuri logice şi ordonarea evenimentelor

8.1 Soluţia lui Lamport

În vorbirea curentă se afirmă că un eveniment **a** s-a produs înaintea altui eveniment **b** dacă momentul de timp la care s-a produs **a** precede momentul producerii lui **b**. Pentru algoritmi, un eveniment poate fi execuția unei acțiuni (operații). În cazul execuţiei unui algoritm secvențial, ordinea evenimentelor poate fi stabilită uşor, operațiile algoritmului fiind executate secvențial. La execuția unui algoritm distribuit, ordinea evenimentelor este mai greu de stabilit: deși fiecare proces execută acțiunile secvențial, execuția paralelă a proceselor face ca evenimentele corespunzătoare diverselor procese să fie intrețesute în timp. Cu toate acestea, ordinea evenimentelor este uneori importantă. De exemplu, o politică echitabilă pentru intrarea proceselor în secțiunea critică ar fi ordinea în care s-au făcut cererile. Ordonarea evenimentelor la sistemele distribuite întâmpină dificultăţi, deoarece reclamă ca sistemele de calcul să conţină ceasuri de timp real, iar ceasurile să măsoare cu precizie timpul. O soluţie mai realistă este aceea a introducerii unor **ceasuri logice**, care să marcheze ordinea evenimentelor fără a da cu exactitate momentele producerii lor. În acest scop, se defineşte relaţia **petrecut înainte** pe mulţimea evenimentelor dintr-un sistem, notată cu **->**, ca fiind cea mai slabă relaţie care satisface următoarele trei condiţii:

- în situaţia că **a** şi **b** sunt evenimente din acelaşi proces şi **a** îl precede pe **b**, atunci a->b;
- când **a** reprezintă transmiterea unui mesaj de către un proces, iar **b** recepţia aceluiaşi mesaj de către un altul, atunci a->b;
- dacă a->b şi b->c atunci a->c.

Să considerăm în continuare o modalitate simplă de a introduce ceasuri logice într-un program paralel. Pentru aceasta, în loc să se ataşeze unui eveniment timpul real de producere, i se asociază un număr de ordine, care să precizeze poziţia sa în raport cu celelalte evenimente. Mai precis, pentru fiecare proces P_i, se defineşte ceasul logic ca o funcţie C_i care asociază fiecărui eveniment **a** din P_i un număr unic $C_i(a)$. Întregul sistem de ceasuri poate fi privit ca o funcţie C definită pe mulţimea tuturor evenimentelor cu valori în mulţimea numerelor naturale, care asociază oricărui eveniment **a** un număr $C(a)=C_i(a)$, dacă **a** este un eveniment al procesului P_i. Un astfel de sistem de ceasuri este corect dacă, pentru orice evenimente **a** şi **b**, a->b implică C(a)<C(b).

Realizarea practică a unui sistem de ceasuri logice presupune folosirea pentru fiecare proces A a unei variabile **cl**, a cărei valoare se măreşte la producerea evenimentelor şi, pentru fiecare mesaj, a unui câmp care să indice momentul transmiterii sale. Ceasurile logice sunt incrementate conform următoarelor reguli:
(1) când A transmite un mesaj, el incrementează **cl** cu 1 şi apoi actualizează timpul **tt** asociat mesajului la valoarea lui **cl**;
(2) când A primeşte un mesaj având asociat timpul **tt**, el actualizează **cl** la valoarea maximă dintre **cl** şi **tt** + 1 şi apoi incrementează **cl** cu 1.

Folosind ceasurile logice, putem asocia un moment de timp fiecărui eveniment. Pentru **send** acesta este timpul înregistrat în mesaj. Pentru **receive** acesta este valoarea lui **cl** din procesul receptor după actualizarea la maximul dintre **cl** şi **tt**+1 şi incrementarea ei. Aceasta asigură că, dacă un eveniment **a** se produce înaintea unui eveniment **b**, timpul asociat lui **a** să fie mai mic decât timpul asociat lui **b**. Se induce astfel o ordonare parţială pe mulţimea evenimentelor unui program.

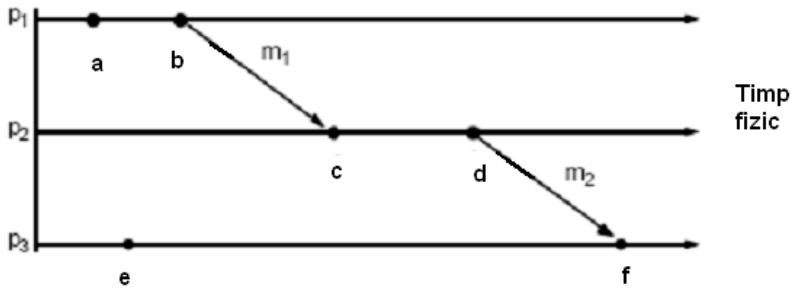


Figura 8.1Trei procese şi evenimentele corespunzătoare

În figura 8.1 sunt reprezentate evenimentele interne şi de comunicare între trei procese cu respectarea momentelor de timp fizic la care s-au produs. Considerând că ceasurile logice asociate proceselor au iniţial valoarea 0, prin actualizarea lor după regulile prezentate mai înainte se obţin valorile de timp logic prezentate în Figura 8.2.

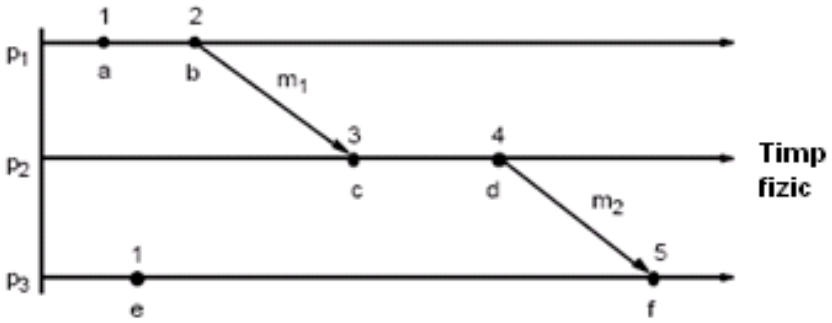


Figura 8.2 Valorile de timp logic asociate evenimentelor din Figura 8.1

Deoarece două evenimente din procese diferite pot avea acelaşi timp logic de producere (de exemplu, evenimentele *a* şi *e* din Figura 8.2), pentru a obţine o **ordonare totală** se poate recurge la ordonarea proceselor şi la dispunerea evenimentelor cu acelaşi timp logic în ordinea proceselor corespunzătoare.

Ca aplicaţie, prezentăm utilizarea ordonării totale a evenimentelor în **realizarea excluderii mutuale** într-un sistem cu transmitere asincronă de mesaje, prin semafoare distribuite. Soluţia are la baza următorul mecanism: când un proces execută o operaţie P (sau V) el difuzează mesaje celorlalte procese, analizând răspunsurile acestora pentru a determina continuarea execuţiei. Folosim forma:

broadcast ch[m]

pentru a desemna transmiterea mesajului **m** pe fiecare din canalele unui tablou ch[1:n]. Toate cele **n** replici ale lui **m** transmise prin **broadcast** poartă aceeaşi amprentă de timp, calculată la fel ca pentru o singură operaţie **send**.

Implementarea semafoarelor se face prin două colecţii de procese. Procesele **Utiliz[i]** iniţiază operaţiile P sau V, prin difuzare de mesaje pe canalele **opsem**. Procesele **Ajutor[i]** implementează operaţiile P si V.

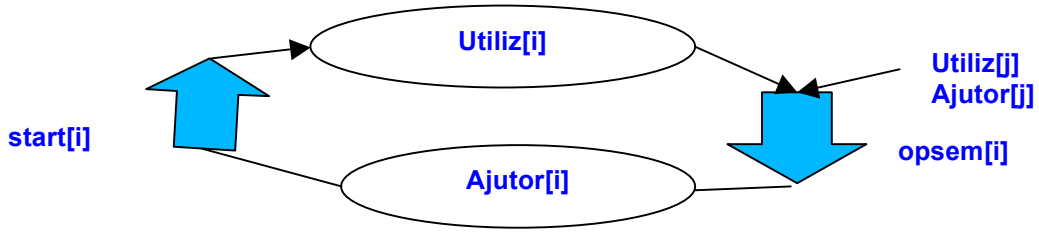


Figura 8.3

La o operaţie P sau V, un proces **Utiliz** difuzează un mesaj tuturor proceselor (inclusiv propriului) **Ajutor**. Mesajul conţine identitatea propriului ajutor, un indicator P sau V şi un moment de timp, acelaşi în toate copiile mesajului.

Fiecare proces **Ajutor** are o coadă locală de mesaje **qm** şi un ceas logic **cl**. Când un proces **Ajutor** primeşte un mesaj P sau V, el îl memorează în **qm**, care este ordonată după momentele de timp conţinute de mesaje (cu amendamentul suplimentar asupra ordonării proceselor). Mesajele unor procese diferite nu sunt recepţionate în aceeaşi ordine de toate procesele **Ajutor**. Mai mult, un mesaj cu un timp mai redus poate fi recepţionat după un altul cu un timp mai mare. Dar, mesajele trimise de un acelaşi proces vor fi recepţionate de celelalte procese în ordinea generării şi vor avea momente de timp crescătoare. Ca urmare, dacă **qm** conţine un mesaj **m** cu timpul **ts** şi procesul recepţionează mesaje cu un timp mai mare de la toate celelalte procese, este sigur că nu va mai apare ulterior un mesaj cu un timp mai mic. În acest moment **m** este **complet confirmat** şi, odată cu el, toate celelalte mesaje aflate în faţa lui în **qm**. Acestea constituie un **prefix stabil**, nici un alt mesaj neputând fi inserat între cele din prefix.

Totodată, când un proces **Ajutor** primeşte un mesaj P sau V, acesta **difuzează** o confirmare **ack**. Mesajele **ack** au asociaţi timpi de producere, dar ele nu sunt memorate la recepţie în **qm** şi nu sunt confirmate la rândul lor. Ele folosesc doar la a determina când un mesaj P sau V devine complet confirmat.

Fiecare proces **Ajutor** simulează operaţiile cu semafoare. El foloseşte o variabilă locală **sem**, pentru a păstra valoarea semaforului. Când procesul primeşte un **ack**, el actualizează **prefixul stabil** din **qm**. Pentru fiecare mesaj V din prefix, procesul incrementează **sem** şi şterge mesajul. Procesul examinează apoi mesajele P. Dacă sem>0, procesul decrementează **sem** şi şterge mesajul P. Mesajele P sunt tratate de toate procesele în aceeaşi ordine.

Algoritmul este următorul:

```
enum fel {V, P, ack};
chan opsem[1:n](int transm, fel op, int timp);
chan start[1:n](int timp);

process Utiliz [i = 1 to n]{
int cl = 0;                 /*ceas logic*/
int ts;                     /*timestamp mesaj*/
```

```
...
cl = cl+1;
broadcast opsem(i, P, cl);    /*operatia P*/
receive start[i](ts);         /*asteapta permisiune*/
cl = max(cl, ts) + 1;
    sectiune critica;
cl = cl+1;
broadcast opsem(i, V, cl);    /*operatia V*/
    sectiune ne-critica;
…
}

process Ajutor [i = 1 to n){
typedef struct {int transm, fel k, int ts} qelem;
queue qm [lmax];    /*coada ordonata dupa timestamp tm*/
int cl = 0;
int sem = valoare_initiala;
int transm;
fel k;
int ts;

while (true) {
  receive opsem[i](transm, k, ts);
  cl = max(cl, ts) + 1;
  if (k==P or k==V) {
    insereaza (transm,k,ts) in loc corespunzator in qm;
    cl = cl+1; broadcast opsem(i, ack, cl);}
  else if (k==ack) {
    inregistreaza primirea unui ack;
    for [mesajele V complet confirmate ] {
          scoate mesaj din qm;
          sem = sem+1;}
    for [mesajele P complet confirmate st sem>0] {
          scoate mesaj (transm, k, ts) din qm;
          sem = sem-1;
          if (transm=i) {cl := cl+1; send start[i](cl); }
  }
}    /*end process*/
```

Tema. Alcătuiţi algoritmul distribuit de excludere mutuală care funcţionează în modul descris în continuare.

Un proces care doreşte să intre în secţiunea critică trimite mesaje de cerere **request** tuturor celorlalte procese. Pentru a putea intra efectiv în secţiunea critică, este necesar să primească de la fiecare câte un mesaj de răspuns **reply**. La recepţia unui mesaj **request**, un proces poate determina dacă el sau procesul care a facut cererea ar trebui să intre în secţiunea critică. Când el are prioritate, mesajul **reply** este întârziat; altfel, el este transmis imediat procesului ce a generat cererea.

8.2 Ceasuri logice vectoriale

Cu soluţia ceasurilor logice dată de Lamport, avem următoarea relaţie:

e precede f => amprenta_logică (e) < amprenta_logică (f)

deci, dacă **e** precede **f** atunci timpul logic asociat lui **e** este mai mic decât cel al lui **f**. Pe de altă parte,

amprenta_logică (e) < amprenta_logică (f) **!=>** e precede f

adică, faptul că timpul logic al lui **e** este mai mic decât cel al lui **f** nu înseamnă neapărat că **e** precede **f**. De exemplu, în Figura 8.2, evenimentul **e** are timpul logic 1 iar **c** are timpul logic 3, dar nu se poate spune ca **e** precede **c**.

Prin utilizarea **ceasurilor logice vectoriale**, o astfel de implicaţie este garantată. Ceasurile logice vectoriale sunt definite în modul descris în continuare. Fiecare proces *Pi* are asociat un vector *Vi[1..n]*, reprezentând ceasul său logic vectorial, în care:

Vi[i] este numărul de evenimente produse în procesul P[i];

Vi[j] este numărul de evenimente despre care *P[i]* ştie (a aflat) că au avut loc la *P[j]*.

Obs. *V* nu este o matrice, ci o colecţie de vectori, fiecare *Vi* din această colecţie fiind păstrat de procesul *P[i]* în memoria sa locală.

Procesul *P[i]* actualizează *Vi* la fiecare eveniment din P[i] (trimitere / recepţie de mesaj sau alt eveniment). Când *P[i]* **transmite** mesajul *m* (eveniment "send m"):

P[i] incrementează *Vi[i]*

P[i] adaugă *Vi* la *m* ca vector de amprente de timp curent *vt(m)*

Când *P[j]* **primeşte** mesajul *m* împreună cu *vt(m)* (eveniment "receive m"):

P[j] ajustează vectorul: *Vj [k]* = max{*Vj[k]*,*vt(m)[k]*} pentru fiecare k

P[j] incrementează *Vj[j]* cu 1

Figura 8.4 arată rezultatul aplicării acestor reguli pe exemplul din Figura 8.2.

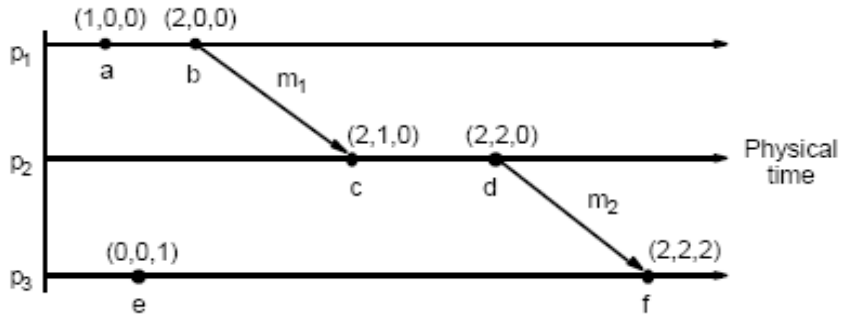


Figura 8.4 Aplicarea regulilor ceasurilor logice vectoriale

Pe multimea vectorilor de amprente de timp, VT pot fi stabilite câteva relaţii. Fie VT1 şi VT2 doi vectori VT cu câte N elemente. Atunci,

- VT1 = VT2 ⇔ VT1[i] = VT2[i], pentru i = 1, ..., N
- VT1 <= VT2 ⇔ VT1[i] <= VT2[i], pentru i = 1, ..., N
- VT1 < VT2 ⇔ VT1 <= VT2 şi VT1 <> VT2

Relaţiile anterioare ne permit definirea ordonării cauzale a evenimentelor. Fie vt(a) şi vt(b) vectorii de amprente de timp asociaţi evenimentelor **a** şi **b**. Atunci:

- vt(a) < vt(b) => evenimentul **a** precede cauzal evenimentul **b**
- vt(a) !< vt(b) and vt(b) !< vt(a) and vt(a) != vt(b) ⇔ evenimentele **a** şi **b** sunt concurente

Aplicaţie: Ordonare Cauzală Multicast

Presupunem că procesele unei colecţii P comunică între ele doar prin mesaje cu difuzare (orice mesaj este trimis tuturor proceselor din colecţie). Se cere ca oricare două mesaje **m** şi **m'** transmise în cadrul colecţiei să respecte **dependenţa cauzală**, ceeace înseamnă că dacă mesajul **m** a fost trimis înaintea mesajului **m'** atunci, la orice proces **p** din colecţie, livrarea lui **m** să aibă loc înaintea livrării lui **m'**:

$$vt(m) < vt(m') \Rightarrow livrare_p(m) \rightarrow livrare_p(m').$$

Pentru asta, fiecare proces foloseşte un protocol care întârzie eventual livrarea mesajelor astfel încât aceasta să aibă loc în ordinea cauzală. Protocolul se bazează pe o variantă uşor modificată a ceasurilor logice vectoriale, pe care o numim **vectori de timp** pentru a face diferenţierea faţă de varianta de bază. În noua variantă, ceasurile sunt avansate doar la operaţiile de transmitere de mesaje.

Ca şi în varianta iniţială, fiecare proces *P*[i = 1..n] are asociat un vector *Vi [1..n]*, cu toate elementele având iniţial valoarea 0, reprezentând vectorul său de timp, în care:

Vi [i] este numărul de evenimente de transmitere de mesaje produse în procesul P[i];

Vi [j] este numărul de evenimente de transmitere de mesaje despre care *P[i]* ştie (a aflat) că au avut loc la *P[j]*.

Procesul *P[i]* actualizează *Vi* la fiecare eveniment de trimitere sau recepţie de mesaj din P[i].

Când un proces sursă *P[s]* **transmite** mesajul *m* (eveniment "send m"):

P[s] incrementează *Vs[s]*

P[s] adaugă *Vs* la *m* ca vector de amprente de timp curent *vt(m)*.

Amprenta *vt(m)* spune receptorului câte evenimente (din alte procese) au precedat **m** şi ar putea influenţa cauzal pe **m**.

Când procesul de destinaţie *P[d]* primeşte mesajul *m* împreună cu *vt(m)* (eveniment "receive m"), mesajul este **livrat** (făcut disponibil procesului pentru prelucrare) doar dacă nu se violeaza constrangerile de cauzalitate. Mai precis, mesajul este păstrat într-o coadă de întârziere până când sunt îndeplinite următoarele două condiţii:

$vt(m)[s] = Vd[s] + 1$

(m este următorul mesaj pe care **d** îl aştepta de la **s**);

$vt(m)[k] =< Vd[k]$ pentru $k <> s$

(toate mesajele primite deja de P[s] când a trimis **m** au fost primite şi de P[d] când acesta a primit **m**).

Când mesajul este livrat, Vd este actualizat conform regulilor vectorilor de timp:

$Vd[k] = \max\{Vd[k], vt(m)[k]\}$ pentru fiecare k = 1,n.

Un exemplu de întârziere a mesajelor este prezentat în figura 8.5. Mesajul cu vectorul de timp (1,1,0) este livrat după mesajul (1,0,0) deşi este primit înaintea acestuia de procesul P3.

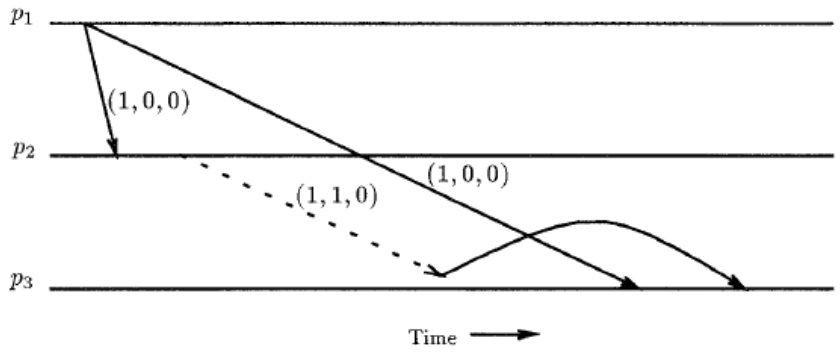


Figura 8.5 Întârzierea unui mesaj (Birman 1991)

Livrarea mesajului **m** poate determina livrarea altor mesaje, primite anterior, care acum îndeplinesc condiţia de cauzalitate.

Acţiunile protocolului la livrarea mesajelor pot fi rezumate astfel:

– dacă vt(m)[k] > Vd [k] pentru un oarecare k atunci se întârzie **m**

  Motivul este că P[s] a primit mesaje de care mesajul curent poate fi cauzal dependent, dar pe care P[d] încă nu le-a primit;

– dacă vt(m)[s] > V(d)[s]+1 atunci se întârzie **m**

  Motivul este că mai sunt mesaje de la P[s] pe care P[d] nu le-a primit; aceasta asigura ordinea FIFO pentru canale care sunt nonFIFO

– dacă vt(m)[s] < Vd [s] atunci rejectează *m*

  Motivul: **m** este un duplicat al unui mesaj primit anterior.

9. Algoritmi undă

9.1 Noţiuni preliminare

9.1.1. Calcul

În capitolul 2 menţionam că un **program concurent** constă dintr-o colecţie de procese paralele comunicante, iar **execuţia** sa poate fi privită ca o întreţesere a **acţiunilor atomice** ale proceselor. Efectul unei acţiuni este modificarea stării colecției de procese, numită convențional **configurație**. Ca urmare, o **execuție** a programului poate fi modelată printr-o succesiune de configurații.

Exemplul 1. Considerăm programul distribuit constând din (colecția a) două procese, cu următoarea descriere:

```
process P1 { int a =0;          process P2 { int b =0;
             a = a+1;}                       b = b+1;}
```

Considerând configurația ca ansamblul valorilor celor două variabile pe care se aplică acțiunile proceselor, configurația inițială este <0,0>. Dacă operația procesului P1 se execută mai întâi, configurația devine:

<1,0>

Notația folosită evidențiază valorile variabilelor **a** și **b** în această ordine. O execuție posibilă a programului poate fi modelată prin secvența de configurații:

(<0,0> , <1,0> , <1,1>)

Programul concurent poate fi modelat ca un **sistem de tranziții**, care pune în evidență **configuraţiile** colecţiei de procese, **tranziţiile** între configurații şi configurația inițială:

$$S = (C, \rightarrow, I)$$

unde:

- C este o mulţime de configuraţii,
- → este relaţia de tranziţie binară pe C,
- I este setul configuraţiilor iniţiale (o submulţime a lui C).

Un **sistem de tranziţii** evidențiază mulţimea tuturor **configuraţiilor** posibile ale sistemului (software), tranziţiile pe care sistemul le poate face între configuraţii şi configuraţiile din care sistemul poate porni.

Pentru exemplul dat, avem următoarele:

- mulțimea de configurații C = (<0,0>,<0,1>,<1,0>,<1,1>)
- relația de tranziție → este definită prin tranzițiile
  - <0,0> → <0,1>
  - <0,0> → <1,0>
  - <0,1> → <1,1>
  - <1,0> → <1,1>
- mulțimea configurațiilor inițiale este I = {<0,0>}.

O **execuţie** a lui S este o **secvenţă maximală** de configurații E = (γ_0, γ_1, γ_2, …), unde γ_0 aparţine lui I şi, pentru i>=0, există tranzițiile $\gamma_i \rightarrow \gamma_{i+1}$.

O secvenţă E este **maximală** dacă este infinită sau dacă sfârşeşte într-o configuraţie **terminală**. O configurație **terminală** γ nu are succesor (nu există δ astfel încât $\gamma \rightarrow \delta$).

O execuție, în exemplul dat, poate fi

E = (<0,0>,<0,1>,<1,1>)

în care configurația <1,1> este terminală.

O configuraţie δ este **tangibilă** din γ dacă există o secvenţă γ_0, γ_1, γ_2, …, γ_k de configurații astfel încât

$\gamma = \gamma_0$

$\delta = \gamma_k$

$\gamma_i \rightarrow \gamma_{i+1}$, pentru i = 1, k-1.

Prin convenţie, se utilizează termeni diferiţi pentru elementele sistemului de tranziţii si pentru procese. Astfel, o **configuraţie** corespunde reuniunii **stărilor** proceselor, iar o **tranziţie** corespunde unui **eveniment** al unui **proces**. Evenimentele pot fi **interne**, de calcul, sau de comunicare, **send** sau **receive**.

Cu aceasta, unei **execuţii** E îi corespunde o **secvenţă de evenimente** din diferite procese. Dacă două evenimente **depind** unul de altul (spunem că sunt într-o relație de **ordine cauzală** - unul din evenimente depinde de celălalt) ele vor apărea în aceeași ordine în toate execuțiile. Altfel, ordinea celor două evenimente poate diferi de la o execuție la alta.

Pentru o execuţie E, **relaţia de ordine cauzală** < între două evenimente **a** şi **b** este cea mai slabă relaţie care satisface următoarele proprietăți:

- dacă **a** şi **b** sunt două evenimente diferite ale aceluiaşi proces şi **a** se produce înaintea lui **b** atunci **a<b**
- dacă **a** este un eveniment **send** iar **b** evenimentul **receive** corespunzător atunci **a<b**
- dacă **a<c** şi **b<c** atunci **a<c** (< este tranzitivă).

Dacă nici **a<b** şi nici **b<a** atunci **a** şi **b** sunt **concurente**.

Exemplul 2. Considerăm două procese **P1** și **P2**, care au variabilele locale **a**, respectiv **b** și care comunică printr-un canal **Can**. Inițial a=0 și b=0. Evenimentele constând din execuția acțiunilor proceselor sunt notate de la ev1 la ev4.

chan Can (int);

process P1{ int a =0;		**process** P2{ int b =0;	
a = a+2;	(ev1)	b = b+1;	(ev3)
send Can (a);}	(ev2)	receive Can (b);}	(ev4)

Evenimentul ev4 depinde de ev2 deoarece procesul P2 nu poate executa acțiunea de recepție a unui mesaj decât după ce P1 a trimis acel mesaj. Ca urmare, în orice execuție, ordinea evenimentelor este ev2<ev4. În schimb, evenimentele ev1 și ev3 pot fi executate în orice ordine de cele două procese, fiind astfel concurente. Ordinea execuției acestor evenimente poate fi ev1<ev3 sau ev3<ev1.

In exemplul 1, execuțiile următoare, notate E și F au același rezultat:

E: (<0,0> , <0,1> , <1,1>) operatia b = b+1 prima

F: (<0,0> , <1,0> , <1,1>) operatia a = a+1 prima

Evenimentele ce corespund celor două operații de adunare sunt concurente și se pot executa în orice ordine.

Evenimentele unei execuţii E se pot produce în orice altă ordine consistentă cu relaţia de cauzalitate, fără a afecta rezultatul execuţiei. Noua ordine a evenimentelor produce o altă secvenţă de configuraţii, F despre care spunem că este **echivalentă** cu prima. Mai precis, o execuţie E este **echivalentă** cu F (se notează E~F) dacă:

- au aceeaşi colecţie de evenimente (ordinea diferă)
- evenimentele respectă aceeaşi relaţie de ordine cauzală
- ultima configuraţie a lui E coincide cu ultima configuraţie a lui F.

Obs. Două execuţii echivalente pot să nu aibă aceleaşi configuraţii.

De exemplu, execuţiile E şi F sunt echivalente: ultimele configuraţii ale celor două execuţii coincid, dar execuţiile nu au aceleaşi configuraţii:
E: (<0,0> , <0,1> , <1,1>)
F: (<0,0> , <1,0> , <1,1>)

Folosind relaţia "**~**" putem împărţi execuţiile unui sistem de tranziţii în clase de echivalenţă. Vom numi **calcul** (*computation*) al unui algoritm distribuit, o **clasă de echivalenţă** (sub relaţia ~) a execuţiilor algoritmului.

Cu alte cuvinte, un calcul este o colecţie de evenimente parţial ordonată prin relaţia de ordine cauzală. Numărul de evenimente dintr-un calcul C este notat |**C**|, iar submulţimea evenimentelor produse în procesul **p** este notată **C**p.

9.2. Algoritmi undă (Wave algorithms)

Mulţi algoritmi distribuiţi se bazează pe transmiterea de mesaje în conformitate cu o anumită **schemă**, dependentă de topologie, care asigură participarea tuturor proceselor. Ca **exemple** menţionăm algoritmii pentru: difuzarea informaţiei, realizarea unei sincronizări globale între procese, declanşarea unui eveniment în fiecare proces, calculul unei funcţii în care fiecare proces participă cu o parte a datelor de intrare. **Algoritmii undă** implementează astfel de scheme de transmitere de mesaje.

Un algoritm undă este un algoritm distribuit care are următoarele **proprietăţi**:
- **terminare** – fiecare calcul este finit,
- **decizie** – fiecare calcul conţine cel puţin un eveniment de decizie (*decide*),
- **dependenţă** – în fiecare calcul, fiecare *decide* este precedat cauzal de un eveniment din fiecare proces.

Pentru un astfel de algoritm, un **calcul** (*computation*) este denumit **undă** (*wave*). Se face distincţie între două categorii de procese: cele care iniţiază execuţia, numite **iniţiatoare** (*starters*) şi celelalte procese, **ne-iniţiatoare** (*followers*). Este uşor de înţeles că primul eveniment dintr-un proces iniţiator este unul intern sau un *send*, în timp ce primul eveniment dintr-un proces ne-iniţiator este un *receive*.

Algoritmii undă pot fi **clasificaţi** după mai multe criterii, prezentate succint în continuare.

Centralizarea. Într-un algoritm centralizat există un singur iniţiator. Într-un algoritm descentralizat există un set arbitrar de iniţiatori.

Topologia. Poate fi inel, arbore, clică, graf etc. Topologia este **fixă** atunci când nu se produc modificări topologice; ea este **conectată** dacă există o cale între oricare două procese. În general, topologia este nedirecţionată (legăturile între procese sunt bi-direcţionale).

Cunoştinţele iniţiale ale unui proces pot include identitatea proprie (nume), identităţile vecinilor, **sensul direcţiei** (sau ordinea vecinilor, prezentată într-un paragraf următor).

Numărul de decizii. Regula generală este "cel mult o decizie în fiecare proces." Putem avea una din următoarele situaţii: un singur proces decide; toate procesele decid; doar unele decid.

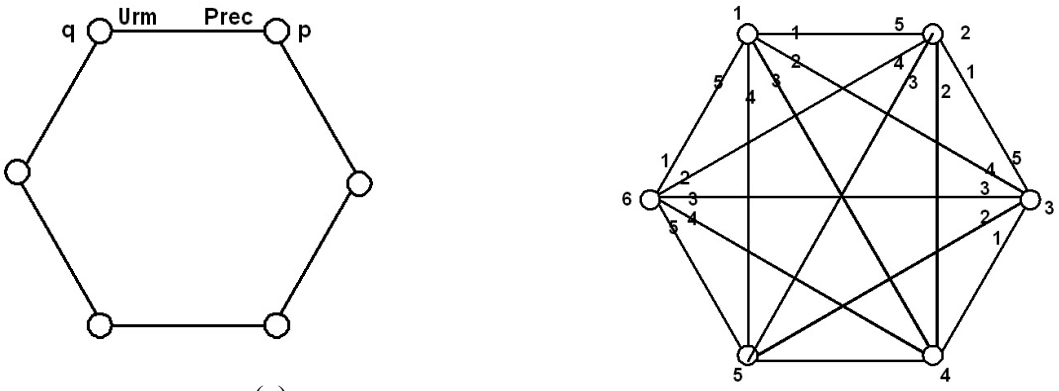
Complexitatetea unui algoritm distribuit se măsoară în: număr de mesaje schimbate, număr de biţi interschimbaţi; timpul necesar pentru un calcul. Pentru complexitatea în timp se aplică următoarele reguli:
- un proces poate executa orice număr finit de evenimente în timp nul;
- durata de la transmiterea la recepţia unui mesaj este de cel mult o unitate de timp.

Sensul legăturilor

Complexitatea comunicării într-un algoritm distribuit depinde de topologie, dar şi de cunoaşterea ei la nivelul fiecărui nod. În notaţia folosită până acum, instrucţiunile de comunicare de mesaje făceau apel la identificatori globali ai canalelor utilizate, care nu includeau nici un fel de informaţie asupra topologiei globale a reţelei de procese. O alternativă utilă în cazul topologiilor regulate este utilizarea unui singur set de etichete pentru oricare proces, care ar corespunde **sensului legăturilor** cu procesele vecine sau **direcţiilor** către nodurile vecine. Dimensiunea setului depinde de topologie (de ex. 2 direcţii pentru inel, 4 pentru tor, etc). Se adaugă o **condiţie suplimentară de consistenţă** diferită de la o topologie la alta.

Topologia **Inel**. Pentru fiecare nod din inel (Figura 9.1a) sunt 2 direcţii, denumite: **Prec** (precedent) şi **Urm** (următor). Condiţia de consistenţă este:

precedentul lui p este q ⇔ următorul lui q este p



(a) (b)

Figura 9.1 Un inel (a) şi o clică (b)

Clica. Clica (Figura 9.1b) conţine N noduri de grad N-1. Setul direcţiilor este {1, 2, …, N-1}, iar condiţia de consistenţă este:

direcţia de la nodul i la j are eticheta (j-i) mod N

Hipercub. Pentru un hipercub n dimensional (Figura 9.2), setul direcţiilor este {0,…n-1}, iar condiţia de consistenţă este:

doua noduri (b_0,…,b_{n-1}), (c_0,…,c_{n-1}) legate prin muchie cu eticheta i difera doar in bitul i

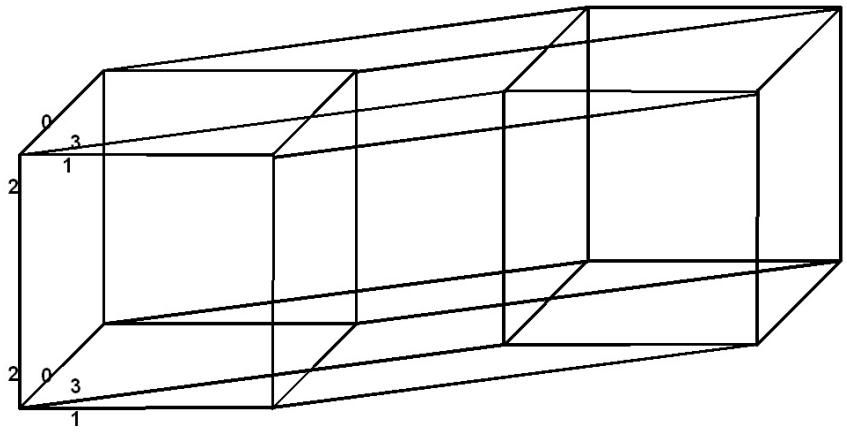


Figura 9.2 Hipercub

Cu acestea, notaţia pentru transmiterea unui **mesaj** poate avea două forme:
(1) transmitere prin adresare directă, folosită atunci când transmiţătorul cunoaşte identitatea unică, globală a canalului **q** către receptor

send ch[q] (mesaj)

(2) transmitere prin adresare indirectă, folosind etichete locale ale direcţiilor

send ch[direcţie] (mesaj)

unde **direcţie** identifică unul din canalele pe care procesul poate transmite.

9.3. Cativa algoritmi undă

Prezentăm în continuare câţiva algoritmi undă elementari.

9.3.1. Algoritmul inel

Algoritmul are următoarele caracteristici:

- fiecare proces are un vecin dedicat, Urm
- transmiterea foloseşte adresarea prin direcţie (Urm, Prec)
- toate canalele selectate prin Urm formeaza un ciclu Hamiltonian

- algoritmul este centralizat: inițiatorul trimite un token (jeton) care este pasat de fiecare proces de-a lungul ciclului până ajunge înapoi la inițiator; inițiatorul ia apoi decizia.

Descrierea este următoarea:

```
chan token[1:n] (tok_type tok);

/*initiator*/
process P[I]{
tok_type tok;
send token[Urm](tok);
receive token[I](tok);
decide

/*non-initiatori*/
process P[k=1 to n, k<>I]{
tok_type tok;
receive token[k](tok);
send token[Urm](tok);
```

Număr de mesaje = n

Cele **n** procese pasează un token de la unul la altul; în total se transmit n mesaje.

Timp = n

Trimiterea (send) și recepția (receive) fiecărui mesaj durează o unitate de timp.

Obs. **Regula folosită pentru calculul complexităţii este următoarea**:

complexitatea în timp a unui algoritm distribuit este timpul maxim cerut de un calcul al algoritmului, in următoarele condiţii:

- un proces poate executa în timp zero orice număr finit de evenimente;
- timpul între transmiterea unui mesaj şi recepţia lui este de (cel mult) o unitate de timp.

9.3.2. Algoritmul arbore

Algoritmul se aplică unei topologii arbore sau unei topologii arbitrare în care se cunoaşte un arbore de acoperire.

Fiecare nod cunoaşte identitatea proprie şi identităţile vecinilor. **Iniţiatorii** sunt toate nodurile frunză, fiecare având un singur vecin.

În **algoritm**:
- fiecare proces trimite exact un mesaj;
- când un proces a primit un mesaj pe fiecare canal incident mai puţin unul (condiţie îndeplinită iniţial de frunze) el trimite un mesaj pe canalul rămas;
- când un proces a primit câte un mesaj pe toate canalele sale atunci **decide**.

```
chan ch[1:n] (int id, tok_type tok);
    /* fiecare proces are un canal propriu */

process Proc[p = 1 to n]{
  int r = numar_vecini_p;
  bool Vecini[1:n] = vecinii_lui_p;
      /* Vecini[i]=true daca i este vecin lui p */
  bool rec[1:n] = ([n] false);
      /* rec[i]=true daca p a primit mesaj de la i */
  tok_type tok; int id, q0;

  while (r>1){receive ch[p](id,tok);
         rec[id] = true; r = r-1};}
/* de la un singur vecin, q0, nu s-a primit mesaj */
  afla q0 pentru care: Vecini[q0] and NOT rec[q0];
  send ch[q0](p, tok);
  receive ch[p](q0, tok); rec[q0] := true;
  decide;
  /* informeaza celelalte procese despre decizie */
  /* for [q = 1 to n st Vecini[q] and q<>q0]
                              send ch[q](p,tok); */
}
```

Algoritmul de bază se termină când unul din procese decide. În descriere sunt adăugate (in formă de comentariu) și instrucțiunile de transmitere a deciziei către vecini.

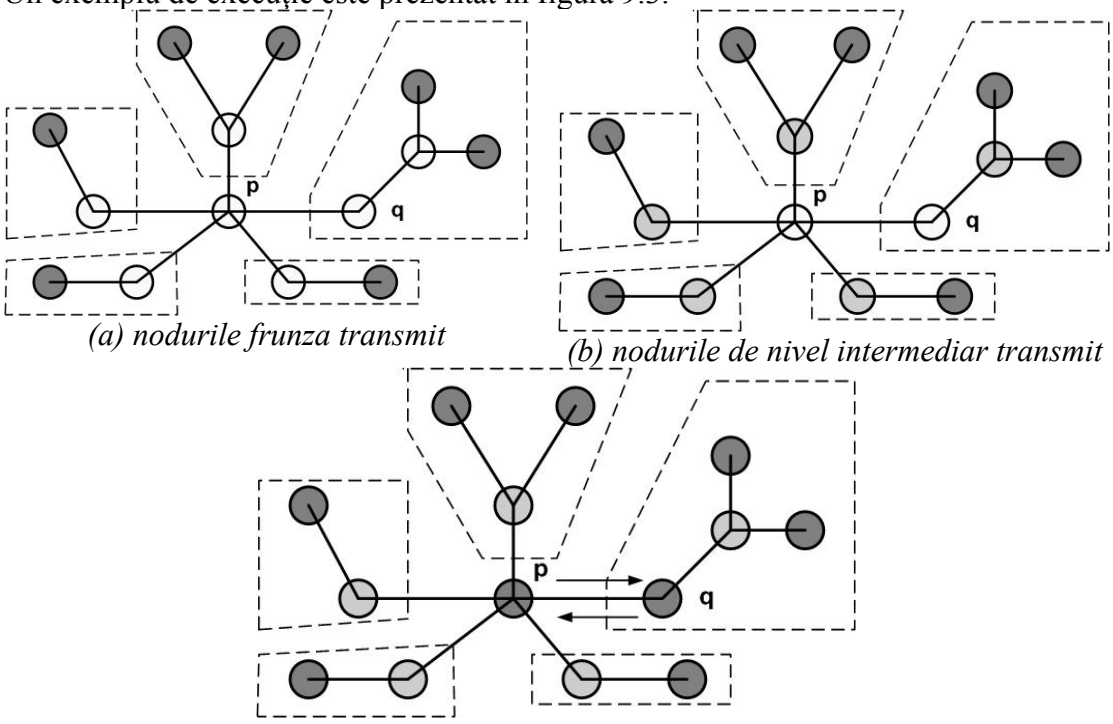
Număr de mesaje = **n**

Fiecare proces trimite un mesaj și sunt **n** procese.

Timp = O(D)

Mesajul trimis de o frunză atinge orice alt nod într-un număr de pași mai mic decât diametrul D al arborelui, fiecare pas fiind făcut într-o unitate de timp.

Un exemplu de execuţie este prezentat în figura 9.3.



(a) nodurile frunza transmit

(b) nodurile de nivel intermediar transmit

(c) p si q transmit reciproc şi, după recepţie, decid

Figura 9.3 Exemplu de execuţie al algoritmului arbore (Tel 1994)

În continuare, justificăm următoarea

Teoremă. Algoritmul "arbore" este un algoritm undă. (justificarea ignoră instrucţiunea inclusă în ultimul comentariu al descrierii algoritmului).

Demonstrația constă în analiza celor trei proprietăți fundamentale (terminare, decizie, dependență) ale algoritmmilor undă.

(1) **calcul finit**

Deoarece fiecare proces trimite cel mult un mesaj, algoritmul foloseşte cel mult N mesaje. Ca urmare, algoritmul atinge o configuraţie terminală γ după un număr finit de paşi.

(2) **în γ, cel puţin un proces a executat un eveniment "*decide*"**

Demonstraţia se face prin reducere la absurd. Deoarece γ este terminală, în γ nu exită nici un mesaj în tranzit.

Pentru **n** noduri avem **n-1** legături bidirecţionale între vecini, deci sunt **2n-2** posibile recepţii de mesaje (pentru care procesele aşteaptă în bucla while după "r" din algoritm). Atunci când K procese au trimis mesaje (când execuţia s-a terminat, deci s-a atins configutaţia γ), un număr de F = 2n–2–K recepţii rămân neconsumate.

Presupunem prin absurd că nici un proces nu a executat "*decide*" în γ. Asta înseamnă că fiecare proces este "blocat" la una din cele două operaţii **receive** din algoritm. Deoarece n-K procese nu au trimis mesaj, ele aşteaptă la primul receive, fiecare mai având de primit cel puţin 2 mesaje. K procese au trimis un mesaj, dar nici unul nu a decis (ele aşteaptă la al doilea **receive**). Ca urmare, fiecare mai are de primit un mesaj. Rezultă de aici că F >= 2(n-K)+K. Înlocuind F cu expresia sa din formula anterioară obţinem:

$$2n–2–K >= 2(n-K)+K$$

$$\text{sau} \quad -2 >= 0,$$

ceeace este absurd. Deci, cel puţin un proces a executat "decide" în γ.

(3) **"decide" este precedat de un eveniment in fiecare proces**

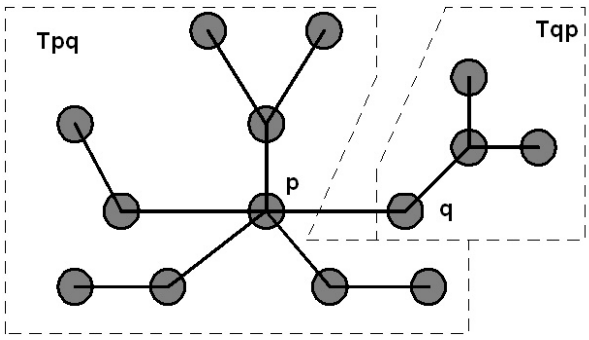


Figura 9.4 Subseturile Tpq şi Tqp (Tel 1994)

Fie:

Tpq – subsetul proceselor tangibile din **p** fără a parcurge legătura **pq**

Tqp – subsetul proceselor tangibile din **q** fără a parcurge legătura **qp**

şi evenimentele:

f_{pq} p trimite un mesaj lui q,

g_{pq} q primeşte un mesaj de la p.

Se demonstrează prin inducţie peste evenimentele de recepţie că:

pentru orice s din Tpq există e € Cs : e ≤ g_{pq}

unde Cs este submulţimea evenimentelor produse în procesul **s** iar ≤ denotă relaţia de precedenţă. Presupunem că aceasta este adevărată pentru toate evenimentele *receive* care preced g_{pq} şi demonstrăm că este adevărată şi pentru g_{pq}.

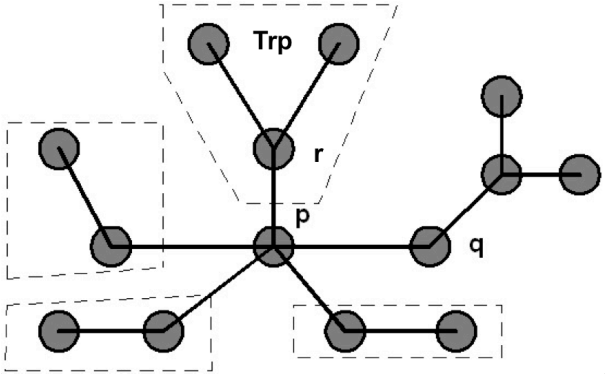


Figura 9.5 Descompunerea lui Tpq (Tel 1994)

Conform algoritmului, p trimite mesaj lui q după ce primește mesaje de la toți ceilalți vecini r. În figura 9.5 sunt arătați toți sub-arborii care au ca rădăcină acești vecini. Trp este unul dintre ei și raționamemtele prezentate aici pentru Trp se pot aplica și pentru restul sub-arborilor din Tpq.

Conform inducției, pentru orice s din Trp, există un eveniment e ≤ g_{rp}; apoi:

$g_{rp} \leq f_{pq}$ – p trimite mesaj lui q după ce primește mesaj de la r;

$f_{pq} \leq g_{pq}$ – mesajul este primit de p dupa ce este transmis de p;

rezultă e ≤ g_{pq}

și prin extindere la ceilalți sub-arbori din Tpq rezulta că relația e ≤ g_{pq} este valabilă pentru orice proces din Tpq.

Demonstrația este similară pentru procesele din Tqp, unde:

e ≤ g_{qp} - pentru orice proces există un eveniment care precede recepția de către p a mesajului de la q;

$g_{qp} \leq d_p$ primirea mesajului de la q precede luarea deciziei de către p.

Rezultă că pentru orice proces **s** există **e** în C_s astfel că e ≤d_p.

9.3.3. Algoritm ecou

Algoritmul prezentat în continuare se aplică unor **topologii arbitrare de graf** și este **centralizat** (exista un singur initiator notat I în algoritm). Versiunea discutată aici a fost propusă de A. Segall in lucrarea "Distributed network protocols, IEEE Transactions on Information Theory, 29:23-35, 1983 (o variantă anterioară, mai puţin eficientă aparţine lui Chang).

Ideea algoritmului este folosirea inundării rețelei cu mesaje **tok**. Inițiatorul trimite câte un mesaj tuturor vecinilor. La rândul lor, vecinii trimit câte un mesaj vecinilor lor și așa mai departe. Inițiatorul așteaptă să primească un răspuns de la fiecare vecin după care ia decizia. Pentru ca decizia inițiatorului sa fie precedată de un eveniment în fiecare proces, se stabileşte un **arbore de acoperire** peste topologia arbitrară de graf. Astfel, vecinul de la care un proces primește primul mesaj este marcat drept **părintele** său în arborele de acoperire. După primirea mesajului, procesul îl difuzează tuturor vecinilor mai puțin părintelui. La rândul lui, primește mesaje de la toți vecinii (mai puțin de la părinte, de la care a primit deja un mesaj), Când primeste mesaje de la toți vecinii, trimite mesaj către părinte. Este ușor de văzut că trimiterile mesajelor către părinți sunt ordonate, începând cu cele ale nodurilor frunză în arborele de acoperire și terminând cu mesajele trimise procesului rădăcină. Se asigură astfel că luarea deciziei de către inițiator este precedată de un eveniment în fiecare din celelalte procese.

```
chan ch[1:n] (int id, tok_type tok);
const int I = id_initiator;
process Proc(I){
  int r = numar_vecini_I;
  bool Vecini[1:n] = vecinii_lui_I;
  tok_type tok;
  int id;
  for [i=1 to n st Vecin[i]]
    send ch[i](I, tok);
  while (r>0){receive ch[I](id, tok);
              r = r-1; }
  decide;
}

process Proc[p=1 to n, p<>I){
  int r = numar_vecini_p;
  bool Vecini[1:n] = vecinii_lui_p;
  tok_type tok;
```

```
  int id, parinte;
  receive ch[p](parinte, tok); r = r-1;
  for [q=1 to n st Vecini[q] and q<>parinte]
    send ch[q](p, tok);
  while (r>0){receive ch[p](id, tok);
              r = r-1};
  send ch[parinte](p, tok);
}
```

Număr mesaje = 2|E|

se trimit două mesaje pe fiecare legatură bi-direcţională.

Timp = O(N)

în fiecare etapă a algoritmului, cel puţin încă un nod (care până atunci nu a primit nimic) primeşte un prim mesaj.

9.3.4. Algoritmul fazelor

Algoritmul fazelor este **descentralizat**. El se aplică unor **topologii arbitrare** în care sunt evidenţiate **canale unidirectionale** (digraf). Ca urmare, vecinii unui proces sunt grupaţi în două categorii: **in**-vecini, de la care procesul primeşte mesaje şi **out**-vecini, cărora procesul le trimite mesaje. În algoritm se presupune că procesele cunosc diametrul grafului D (sau o valoare D'>D).

Ideea algoritmului este următoarea: fiecare proces trimite exact D mesaje fiecărui out-vecin. Funcţionarea proceselor este împărţită în faze; în fiecare fază un proces p trimite un mesaj fiecărui out-vecin şi primeşte un mesaj de la fiecare in-vecin. Astfel, după două faze, un proces primeşte informaţii propagate (prin intermediul in-vecinilor săi) de la procesele situate la distanţa 2 faţă de el. După D faze, un proces primeşte mesaje de la toate procesele situate la distanţă mai mică sau egală cu D (altfel spus, de la procesele de la care există o cale de distanţă cel mult D până la procesul curent). După aceea procesul decide.

În descrierea următoare, există două categorii de procese: Iniţiatori – care pornesc cu trimiterea unui mesaj fiecărui out-vecin şi Ne-iniţiatori – care aşteaptă primirea unui prim mesaj şi apoi trimit fiecărui out-vecin mesajele din prima fază a algoritmului.

```
chan ch[1:n] (int id, tok_type tok);
const int D = diametrul_retelei;
```

```
process Proc [p = 1 to n]{
  bool in[1:n] = in-vecinii_lui_p;
    /* in[i]==true daca i este invecin */
  bool out[1:n] = out-vecinii_lui_p;
    /* out[i]==true daca i este outvecin */
  int rec[1:n] = ([n] 0);
    /* rec[q] = numar mesaje primite de la q */
  int sent = 0;
    /* nr. mesaje transmise out-vecinilor */
  tok_type tok;
  int id;

  if (p este initiator){
        for [q = 1 to n st out[q]]{
           send ch[q](p, tok); sent = sent+1;}
/*min(rec) = min(rec[q]),q=1..n and in[q])*/
  while (min(rec) < D){
      receive ch[p](id, tok);
      rec[id] = rec[id]+1;
      if (min (rec) >= sent and sent < D){
        for [q=1 to n st out[q]]
          send ch[q](p,tok);
        sent = sent+1;
      }
  }
  decide;
}
```

Număr mesaje = 2D|E|, unde E este multimea canalelor bidirecționale;
sunt D faze; în fiecare fază se trimite un mesaj pe fiecare direcție a canalelor.

Timp = 2D
în fiecare fază, se trimit mesaje către out-vecini după ce au fost recepționate mesaje de la toți in-vecinii.

Comentariu. Dacă p este inițiator, el transmite mesaje tuturor out-vecinilor și face sent=1. În bucla **while**, el așteaptă apoi primirea unui set de mesaje de la in-vecini, până când primește cel putin un mesaj de la fiecare in-vecin [min(rec)>=1]. Când condiția este îndeplinită, el transmite mesajele din următoarea fază. De fiecare dată, trecerea la transmiterea mesajelor din faza k+1 se face după primirea de la toți in-vecinii a mesajelor din faza k.

Dacă **p** nu este inițiator, el intră direct in bucla **while**, unde așteaptă primirea unui prim mesaj. Deoarece, pentru el, min(rec)=0 și sent=0, procesul trece la transmiterea mesajelor din prima fază, după care așteaptă în buclă primirea mesajelor de la in-vecini. Comportamentul este apoi similar cu cel al proceselor inițiatoare.

În continuare, justificăm următoarea
Teoremă. Algoritmul "fazelor" este un algoritm undă.

(1) **calcul finit**
Fiecare proces trimite cel mult D mesaje prin fiecare canal. Ca urmare, algoritmul atinge o configurație terminala γ după un număr finit de pași.

(2) **în γ, fiecare proces a executat un eveniment "decide"**
Considerăm că γ este o configurație terminala a unui calcul C și presupunem că există cel puțin un inițiator în C (pot fi mai mulți).

(2.1) fiecare proces a trimis cel puțin un mesaj
Există cel puțin un inițiator care transmite mesaje out-vecinilor săi. La rândul său, fiecare out-vecin trimite mesaje cel târziu când primește un mesaj.

(2.2) în **γ** fiecare proces a decis
Observație. Un proces decide atunci când iese din bucla **while**, deci atunci când conditia *min(rec)<D* nu mai este îndeplinită. Asta se întâmplă când *min(rec)* devine egal cu D, adică procesul a primit mesaje de la toți in-vecinii în faza D. Cu alte cuvinte, nu există vreun proces care să fi trimis mai puțin de D mesaje. Pentru demonstratie, luăm în considerație procesul cu cel mai mic număr de mesaje trimise.

Fie p procesul cu cel mai mic $sent_p$ in γ. Pentru orice proces q este îndeplinită relația $sent_q >= sent_p$ in γ. În particular, ține dacă q este in-vecin al lui p adică $min_q (sent_q) >= sent_p$ pentru q € in. Dar $rec_p[q] = sent_q$ (în γ, toate mesajele transmise au fost recepționate), deci $min_q (rec_p[q]) >= sent_p$. Rezultă $sent_p$ =D; altfel p ar fi trimis mesaje suplimentare la ultima recepție (vezi decizia din algoritm). Rezultă $sent_p$ =D pentru toti p, deci $rec_p[q]$ =D pentru orice canal qp și fiecare proces a decis.

(3) **"decide" este precedat de un eveniment in fiecare proces**
Folosiim notațiile:
$f_{pq}^{(i)}$ al i-lea eveniment "p transmite lui q"
$g_{pq}^{(i)}$ al i-lea eveniment "q primeste de la p"
≤ relatia "precede pe"

Ne referim la decizia din procesul **p**. Deoarece D este diametrul, pentru orice **q** există o cale la **p** cu lungime <= D, pe care o notăm **q** = p(0), p(1), …, p(k) = **p**, cu lungimea k<=D. Într-o fază, transmisia precede recepția aceluiași mesaj, deci

$$\mathbf{f}_{p(i)p(i+1)}^{(i+1)} \leq \mathbf{g}_{p(i)p(i+1)}^{(i+1)} \text{ pentru } 0 <= i < k$$

Din algoritm, recepția unui mesaj precede transmiterea lui în faza urmatoare, adică

$$\mathbf{g}_{p(i)p(i+1)}^{(i+1)} \leq \mathbf{f}_{p(i+1)p(i+2)}^{(i+2)} \text{ pentru } 0 <= i < k-1$$

Aplicând succesiv relațiile, rezultă $\mathbf{f}_{p(0)p(1)}^{(1)} \leq \mathbf{g}_{p(k-1)p(k)}^{(k)}$
În plus, din algoritm, $\mathbf{g}_{p(k-1)p(k)}^{(k)} \leq \mathbf{d}_p$ deci decizia lui p este precedată de un eveniment în orice q: $\mathbf{f}_{qp(1)}^{(1)} \leq \mathbf{d}_p$.

9.3.5. Algoritmul fazelor pentru clici

Într-o topologie **clică** (vezi Fig. 9.1b) cu n noduri, fiecare proces are n-1 vecini (la fiecare proces sunt conectatre direct toate celelalte procese). Ca urmare, diametrul este 1, fiecare proces primind (direct) mesaje de la fiecare din celelalte prcese. În total se transmit n(n-1) mesaje, comunicarea lor făcndu-se în paralel pe canale unidirecționale. Pentru evidența mesajelor primite, fiecare proces folosește o singură variabilă simplă.

Algoritmul este relativ simplu: fiecare proces trimite un mesaj tuturor vecinilor săi; inițiatorul începe execuția cu această operație, un alt proces făcând transmiterea imediat ce primește primul mesaj, de la un alt proces. În continuare, fiecare proces așteaptă primirea unui mesaj de la fiecare vecin și apoi decide.

```
chan ch[1:n] (int id, tok_type tok);
 //canale unidirectionale

process Proc [p = 1 to n){
  bool Vecini[1:n] = in-vecinii_lui_p;
  int n_vecini = numar_vecini_p;
  int rec = 0;
    //un mesaj primit per vecin
  int sent = 0;
  tok_type tok;
  int id;

  if (p este initiator){
      for [q =1 to n st Vecini[q]]
      send ch[q](p, tok);
```

```
        sent = sent+1;
    }
    while (rec < n_vecini){
        receive ch[p](id, tok); rec = rec+1;
        if (sent == 0){
        for [q =1 to n st Vecini[q]]
         send ch[q](p, tok);
         sent = sent+1;}
    }
    decide;
}
```

Număr mesaje = n(n-1)
Timp = 2
transmiterea mesajelor se face in paralel.

9.3.6. Algortimul lui Finn

Algoritmul nu cere cunoașterea diametrului. El se bazează pe cunoașterea identificatorilor proceselor, folosiți în mesajele transmise între procese.

Ideea: fiecare proces **p** păstrează două mulțimi de identificatori cu următoarele proprietăți:
• **Inc** – mulțimea proceselor **q** pentru care un eveniment în **q** precede cel mai recent eveniment din **p**;
• **NInc** – mulțimea proceselor **q** pentru care fiecare vecin **r** are un eveniment care precede cel mai recent eveniment din **p**.

Proprietățile mulțimilor **Inc** și **NInc** se mențin astfel:
- Initial
Inc = {p}
NInc = Ø
- In algoritm, procesul **p** trimite mesaje care conțin **Inc** și **NInc** de fiecare dată când unul din **Inc** sau **NInc** se modifică (crește) prin adăugarea unui nou identificator. Când **p** primește mesaje cu **Inc** și **NInc**, el adaugă noile procese la propriile **Inc** și **Ninc** (actualizează **Inc** și **NInc** prin operații de reuniune). Când **p** a primit un mesaj de la toți in-vecinii, **p** este adăugat la **Ninc**.

Inc și **NInc** includ treptat identificatorii proceselor. Un proces **p** decide atunci când mulțimea **Ninc** pe care el o gestionează conține identificatorii tuturor proceselor.

Algoritmul este descentralizat. În algoritm se folosesc canale bidirecționale, similar algoritmului fazelor, iar graful care modelează topologia colecției de procese este tare conex. Fiecare proces își cunoaște in-vecinii și out-vecinii. În descriere introducem un nou tip, **SOP** care desemnează o mulțime de identificatori de procese.

```
typedef SOP {multime identificatori};
chan ch[1:n] (int id, SOP Inc, SOP NInc);
process Proc [p = 1 to n){
  SOP Inc = {p};
  SOP NInc = Ø;
  bool rec[1:n] = ([n] false);
     /* de la cine a receptionat */
  bool out[1:n] = out-vecinii_lui_p;
  bool in[1:n] = in-vecinii_lui_p;
  SOP rInc, rNInc;
  int id;
  if (p este initiator)
        for [q = 1 to n st out[q]]
           send ch[q](p, Inc, NInc);


while (Inc <> NInc){
    receive ch[p](id, rInc, rNInc);
    Inc = Inc U rInc; NInc = NInc U rNInc;
    rec[id] = true;
    if (rec[id], ¥id=1..n and in[id])
      NInc = NInc U {p};
    if (Inc sau NInc modificat)
     for[q=1 to n st out[q]]
          send ch[q](p, Inc, NInc);
}
decide;
}
```

Număr mesaje <= 4N|E|

sunt 2|E| canale uni-direcționale; în cel mai rău caz, se modifică o singură mulțime dintre Inc și NInc prin adaugarea unei singure identitați; rezultă că pe fiecare canal se transmit cel mult 2N mesaje

Timp = O(D)

timpul de propagare a unei schimbări petrecute în mulțimile Inc sau NInc este proporțional cu diametrul grafului care modelează topologia.

Teorema. Algoritmul lui Finn este un algoritm undă

(1) calcul finit
În algoritm, se trimite un mesaj la fiecare modificare a Inc sau Ninc. Numărul de elemente din cele două mulțimi luate împreună este inițial 1 și ajunge la 2N. Ca urmare, numărul de mesaje transmise este limitat superior la 4N|E|.

(2) în γ, fiecare proces a executat un eveniment "decide"
2.1. în γ fiecare proces a trimis cel puțin un mesaj pe fiecare canal
Când p trimite primul mesaj vecinilor săi, fiecare vecin q adaugă p la Inc_q și trimite un mesaj tuturor vecinilor săi. Fiecare vecin actualizează Inc și trimite un mesaj mai departe. Din faptul că graful este tare conex rezultă proprietatea menționată.

2.2. în γ fiecare proces a decis
2.2.1. pentru oricare proces p, Inc_p conține toate procesele în configurația γ.
Dacă există canalul unidirecțional **pq**, atunci orice modificare a lui Inc_p este trimisă de p lui q, care o include în Inc_q; rezultă că Inc_q include Inc_p. Deoarece graful este tare conex, avem și Inc_p include Inc_q; rezultă $Inc_p = Inc_q$ pentru pentru orice p și q și că, pentru orice p, Inc_p conține identitățile tuturor proceselor.

2.2.2. pentru orice p și q, $NInc_p = NInc_q$
Fiecare proces a trimis un mesaj pe fiecare canal în γ. Ca urmare, fiecare proces a recepționat de la toți vecinii (pentru orice id € in: rec[id]). Conform algoritmului, p este adăugat la $NInc_p$ care este apoi difuzată celorlalte procese. Deoarece graful este tare conex și orice $NInc_p$ conține p, $NInc_p$ va conține toate procesele în γ. Deci, pentru orice p, $Inc_p = NInc_p$ și fiecare proces a decis în γ.

(3) "decide" este precedat de un eveniment in fiecare proces
Fie q un in-vecin al lui p și fie evenimentul intern lui q, $\mathbf{a_q}$**: NInc = NInc U {q}** care modifică $NInc_q$. Conform algoritmului, evenimentul $\mathbf{a_p}$ (intern procesului p) se produce dacă pentru fiecare in-vecin r al lui q un eveniment din r precede a_q.
Procesul p decide doar cand $Inc_p = NInc_p$ = setul tuturor proceselor. Când procesul p ia decizia sunt îndeplinite următoarele:
- p € Inc
- in-vecinul q € Inc și q € NInc
Deci, decizia lui p este precedată de un eveniment din q și de un eveniment din fiecare din in-vecinii lui q.
Relația este valabilă pentru oricare două procese din care unul este in-vecinul celuilalt. Rețeaua fiind conexă, decizia lui p este precedată de un eveniment din fiecare din celelalte procese.

Capitolul 10. Alegerea unui lider

Alegerea unui lider este necesară ca fază premergătoare execuției unui algoritm centralizat, în care doar unul din procesele unei colecții urmează să execute operații de inițializare sau să joace rolul de coordonator în derularea algoritmului. Evident, se presupune că desemnarea statică a liderului nu este aplicabilă, de regulă pentru că nu se cunoaște compoziția exactă a grupului de procese. (Este cazul algoritmilor folosiți la refacerea unor sisteme după defectare - crash). În schimb, fiecare proces își cunoaște propria identitate și vecinii. Identitățile proceselor aparțin unei mulțimi total ordonate, astfel că alegerea liderului înseamnă determinarea procesului care are identitatea cea mai mică (sau cea mai mare).

Algoritmii de alegere a unui lider sunt descentralizați (mai multe procese pot declanșa alegerea) și presupun luarea unei decizii cu participarea tuturor proceselor. De aceea, algoritmii undă reprezintă soluții naturale ale problemei.

10.1. Alegerea unui lider cu algoritmi undă

Soluția are următoarele caracteristici:

- toate procesele au același algoritm local
- algoritmul este **descentralizat** (poate fi inițiat de oricare subset de procese)
- în fiecare calcul algoritmul atinge o **configurație finală** în care un proces este lider, toate celelalte au pierdut.

O variantă mai “slabă” este cea în care un proces este lider și este "conștient" de acest lucru, în timp ce celelalte procese nu "știu" ca au pierdut. Evident, liderul poate înștiința celelalte procese că el a câștigat, aducând astfel soluția la cazul precedent.

Fiecare proces are una din următoarele stări:

sleep – procesul nu a executat nici o acțiune
leader – procesul a câștigat alegerile
lost – procesul a pierdut.

În soluțiile următoare, sunt acceptate următoarele presupuneri:

- sistemul este asincron (comunicație asincronă, fără un ceas global)
- fiecare proces este identificat printr-un nume unic
- identitățile sunt extrase dintr-un **set total ordonat**
- aflarea liderului se concretizează în **aflarea procesului cu cel mai mic identificator**.

Soluția prezentată aici se aplică unei topologii arbore sau pe care s-a definit ad hoc un arbore de acoperire. Topologia arbore **impune ca** cel puțin toate frunzele să fie inițiatori. În realitate, doar unul sau unele procese, care nu sunt neapărat frunze în topologia ansamblului de procese, observă căderea liderului curent și ar putea declanșa alegerea unui nou lider. Pentru a putea folosi algoritmul arbore, se adaugă o fază de *wakeup*, în care procesul (procesele) care au observat căderea liderului curent trimite (trimit) mesaje *wakeup* tuturor celorlalte procese, inclusiv frunzelor. Odată trezite, procesele frunză inițiază alegrea noului lider.

Fiecare proces folosește variabilele:
ws – boolean, asigură că fiecare proces transmite mesaje *wakeup* cel mult o dată;
wr – int, contorizează mesajele de *wakeup* recepționate.

Când un proces a primit *wakeup* prin fiecare canal, el începe algoritmul de alegere pentru a determina procesul cu **cea mai mica** identitate. Când un proces decide, el află identitatea liderului și, în funcție de ea, procesul trece în starea **leader** sau **lost**.

```
chan ch[1:n](int id_transm, int id_mic);
chan wakeup[1:n]();
process Proc(p = 1 to n){
 bool ws = false;
 int wr = 0;
 int r = numar_vecini_p;
 bool Vecini[1:n] = vecinii_lui_p;
 bool rec [1:n] = ([n]*false);
 int V = p;
 enum state {sleep, leader, lost}
 state stare = sleep;
 int id;


/* aici incepe faza wake-up */
 if (p este initiator){
     ws = true;
     for [q = 1 to r st Vecini[q]]
        send wakeup[q]();
 }
 while (wr < numar_vecini_p){
     receive wakeup[p](); wr = wr+1;
```

```
     if (NOT ws) ->
     ws = true;
     for [q = 1 to n st Vecini[q]]
      send wakeup[q]();
 }

/* aici incepe algoritmul tree */
 do (r>1){
  receive ch[p](q,id);
    rec[q] = true; r = r-1;
    V = min(V, id);
 }

/* de la un singur vecin, q0 nu s-a primit mesaj */
  afla q0: Vecini[q0] and NOT rec[q0];
  send ch[q0](p, V);
  receive ch[p](q0, id);
  V = min (V, id);
  if (V=p) stare = leader;
  else stare = lost;
/*transmite vecinilor decizia*/
  for [q = 1 to n st Vecini[q] and q<>q0]
     send ch[q](p,V);
}
```

Numar mesaje = O(N)

Pe fiecare canal se trimit două mesaje *wakeup* şi două *token*-uri ceeace înseamnă un total de 4N-4 mesaje.

Timp = O(D)

În D paşi după ce primul proces porneşte algoritmul, fiecare proces a trimis mesajele *wakeup*. Ca urmare, în D+1 paşi fiecare proces a pornit unda. Prima decizie se ia la D paşi după pornire, iar ultima după alţi D paşi. Rezultă totalul de 3D+1 paşi.

10.2 Algoritmi pentru topologia inel

Una din situaţiile cele mai comune este cea în care procesele, identificate prin numere distincte între ele, sunt organizate într-o topologie inel.

Problema cere, ca și mai inainte, desemnarea prin conses a unui singur proces drept lider al grupului de procese. De data aceasta convenim ca el să fie procesul cu identificatorul **maxim**. Nu se cunoaște apriori numărul de procese din grup. De asemenea, dispunerea proceselor în inel este oarecare (nu neapărat în ordinea numerelor de identificare). În fine, nu există un control centralizat al alegerii liderului (soluția este distribuită).

Exemplu de utilizare: un inel de controlere în care jetonul de control (care asigură protocolul de coordonare a operațiilor din inel) s-a pierdut. Situația este detectată printr-un timeout, la care fiecare controler declanșează procedura de selecție a liderului, a cărui sarcină este de a genera un nou jeton.

10.2.1 Algoritmul LeLann

Algoritmul realizează următoarele acțiuni:

1. Fiecare proces transmite în inel un mesaj care conține identificatorul său
2. Fiecare proces colectează identificatorii celorlalte procese
3. Fiecare proces calculează maximul.
4. Procesul al cărui număr este egal cu maximul devine lider.

Complexitate $O(n^2)$ mesaje comunicate - fiecare proces transmite un mesaj care trece pe la toate celelalte.

Descrierea detaliată a algoritmului este următoarea.

```
chan ch[Ids](token tok, int id_mare);
process Proc(p = 1 to n){
typedef SOP {multime identificatori};
SOP List = {p};
enum state (candidate,leader,lost);
state stare = candidate;
send ch[Urm](tok, p);
receive ch[p](tok, q);
while (q<>p) List = List U {q};
          send ch[Urm](tok, q);
          receive ch[p](tok, q);
}
if (p == max(List)) stare = leader;
else stare = lost;
```

Algoritmul se bazează pe proprietatea FIFO a canalelor, conform căreia ordinea mesajelor transmise pe inel se păstrează. Astfel, oricare proces **p** va trimite mesajul propriu **(tok, p)** urmat în ordine de mesajele **(tok, q)** primite de la alte procese. Având în vedere topologia inel folosită, mesajul propriu va ajunge înapoi la procesul **p** după ce acesta a primit şi retrimis mesaje de la toate celelalte procese. Deci, cand **(tok, p)** ajunge înapoi la procesul **p**, variabila sa **List** va conține identificatorii tuturor proceselor. Acest lucru este valabil pentru toate procesele, urmarea fiind că toate vor identifica un același maxim, deci un acelasi nou lider al colecției de procese.

10.2.2 Algoritmul Lelann-Chang-Robert

Algoritmul LeLann nu este prea eficient, procesele având de găsit, în paralel, maximul dintr-o mulțime de identificatori. Varianta **Lelann-Chang-Robert** ocolește acest neajuns prin eliminarea pe parcurs a valorilor mai mici, în paralel cu pasarea mesajelor mai mari de la un proces la altul. Mesajele sunt transmise în inel **în sensul acelor de ceasornic**. Astfel:

1. Fiecare proces p transmite procesului aflat în **dreapta** un mesaj cu identificatorul propriu, p.
2. Un proces p care primește un mesaj **q** il compară cu identificatorul propriu **p**
    a. **if** q > p -> transmite q în dreapta
    b. **if** q < p -> elimină m (nu transmite nimic în dreapta)
    c. **if** q = p -> procesul curent devine lider.

Propozitie: Algoritmul detectează un singur cel mai mare număr.
Argument: Un mesaj întâlnește toate celelalte procese înainte de a reveni, eventual, la transmițător. Numai mesajul cu valoarea cea mai mare nu este eliminat de procesele parcurse.

Descrierea detaliată a algoritmului este următoarea.

```
chan ch[1:n](token tok, intid_mare);
process Proc(p = 1 to n){
enum state (candidate, leader, lost);
state stare = candidate;
send ch[Urm](tok, p);
while (state <> leader){
    receive ch[p](tok, q);
    if (q==p) stare = leader;
```

```
    else if (q>p){
      if (stare == candidate){stare = lost;
                  send ch[Urm](tok, q);
    }
}
```

Observații
Condiția de ieșire din buclă este "state = leader". Aceasta are două consecințe:

- un proces care devine **lost** rămâne în buclă pentru a pasa alte mesaje pe care le primeste
- doar procesul cu identificatorul maxim termină; celelalte vor rămâne blocate în așteptarea unui token; pentru deblocare, liderul ales poate trimite un mesaj special prin care anunță identificatorul lui și faptul că alegerea s-a terminat.

Algoritm modificat
În cazul în care nu toate procesele sunt inițiatori, se pot introduce următoarele modificări:

- Cel puțin un proces inițiază alegerea și se auto-marchează.
- Când un mesaj **m** ajunge la un proces nemarcat, acesta va genera un mesaj cu **id** propriu, se va marca și va continua cu prelucrarea mesajului **m**.
- Liderul ales asigură că fiecare proces se va demarca (pentru execuția corectă a unei viitoare alegeri de lider).

Performanța
Măsurată în timp și număr de mesaje

Timp
Dacă toate procesele pornesc simultan, este necesar un singur ciclu prin inel. Deci, timpul este O(n), unde **n** este numărul de procese.

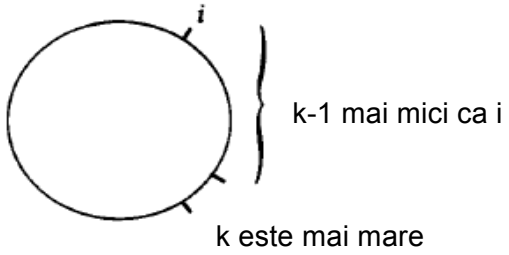
Similar, dacă procesul cu numărul maxim pornește primul timpul este O(n). Altfel, în cel mult n-1 pași mesajul de marcare ajunge la procesul cu **id** maxim și în n pași se face alegerea acestuia ca lider. Deci timpul este O(n).

Număr mesaje
a) *Cazul cel mai bun.* Procesele sunt ordonate crescător în sensul acelor de ceasornic. Fiecare mesaj (exceptând mesajul n) este transmis o dată. Mesajul n este transmis de n ori. Rezulta 2n-1 mesaje.

b) *Cazul cel mai rău*. Procesele sunt ordonate descrescător în sensul acelor de ceasornic. Toate mesajele sunt eliminate de procesul n.
Mesajul i este transmis de i ori. Rezultă

$$\Sigma_{i=1,n} i = n(n + 1)/2.$$

c) *Cazul mediu*.



Sunt **i-1** procese mai mici ca **i** şi **n-i** mai mari ca **i**. Fie C(a,b) numărul de moduri în care putem alege b obiecte din a (combinări de a luate câte b).

Probabilitatea *P(i, k)* ca mesajul **i** sa fie pasat de **k** ori este egală cu probabilitatea ca cei k-1 vecini ai lui i în sensul acelor de ceasornic să fie mai mici ca i **şi** vecinul k să fie mai mare, adică:

$$P(i,k) = [C(i-1,k-1)/C(n-1,k-1)]*[(n-i)/(n-k)]$$

Cunoscând că mesajul n se transmite întotdeauna de n ori şi că există doar un singur asemenea mesaj, considerăm doar n-1 mesaje, fiecare fiind transmis de cel mult n-1 ori. Numărul probabil de transmiteri pentru mesaje altele decat n este:

$$Ei(k) = \Sigma_{k=1,n-1} k.P(i,k)$$

Numărul total de transmiteri pentru toate mesajele este:

$$E(k) = n + \Sigma_{i=1,n-1} \Sigma_{k=1,n-1} k.P(i,k)$$

sau

$$E(k) = n + \Sigma_{k=1,n-1} n/(k+1) = n(1 + 1/2 + 1/3 + ... + 1/n) \sim= n(C + \ln n)$$

Rezultă că numărul mediu de mesaje este **O(n ln n)**.

10.2.3 Algoritmul Hirschberg-Sinclair

Spre deosebire de algoritmul Chang si Roberts care are o complexitate medie O(n ln n), algoritmul Hirschberg-Sinclair are complexitatea O(n ln n) în cel mai defavorabil caz.

El lucrează pe un inel bidirecțional și presupune că procesele pot detecta din ce direcție vine un mesaj astfel că pot trimite răspunsuri către transmițător. Ca urmare, operațiile de comunicare folosite de procese au o semantică particulară, diferită de cea definită de noi anterior și anume:

- un proces poate iniția mesaje în ambele direcții prin *sendboth*.
- un proces poate pasa un mesaj (eventual modificat) prin *sendpass*.
- un proces poate trimite un răspuns în direcția din care a primit un mesaj prin *sendecho*.

Algoritmul operează în faze. În **faza k**, un proces inițiază mesaje care sunt transmise în ambele direcții pe căi de lungime 2^k (1, 2, 4, 8 …). Procesele de pe o cale analizează mesajul. Dacă un proces decide că nu poate câștiga alegerea el va pasa mesajul și nu va mai iniția mesaje proprii. Dacă procesul decide că sursa nu poate câștiga alegerea, transmite un ecou negativ sursei. Ultimul proces din calea 2^k transmite un ecou pozitiv sursei (dacă nu poate câștiga alegerea).

Un proces care primește propriul mesaj câștigă; în acest caz, calea "acoperă" toate procesele. El va trimite apoi un mesaj pentru a anunța celelalte procese că a câștigat.

Algoritmul are descrierea dată în continuare.

Execuția pentru alegere:

```
stare = candidate
maxnum = 1          /* lungimea caii curente */
while (stare == candidate){
  sendboth ("from", myvalue, 0, maxnum);
                    /* 0 = pasii facuti pe calea curenta */
  await both replies (but react to other  messages);
  if (either reply is "no")
       stare = lost;
  maxnum = 2*maxnum
  }
```

La recepție mesaj ("from", value, num, maxnum):

```
if (value < myvalue) sendecho ("no", value);
else if (value > myvalue){
  status = lost;      /* nu va mai trimite mesajele sale */
  num = num + 1;            /* a mai facut un pas */
  if (num < maxnum)
      sendpass ("from",value, num, maxnum);
  else sendecho ("ok", value);
                  /* ultimul proces din cale trimite OK */
 else status = won;
```

La recepție mesaj ("no", value) sau ("ok", value):

```
if (value <> myvalue) sendpass the message;
else this is a reply the processor was awaiting;
```

Netratat în algoritm - acțiunile unui proces neinițiator (care nu "știe" de alegerea în curs): la primirea unui mesaj devine conștient de alegerea în curs și poate deveni *candidate* dacă identitatea are o valoare mai mare decât sursa mesajului.

Analiza complexității

Un proces x inițiaza mesaje pe căi de lungime 2^i numai dacă el nu este "învins" de un proces aflat la distanta 2^{i-1} (în oricare direcție) de x. În orice grup de $2^{i-1} + 1$ procese consecutive, cel mult unul poate inița mesaje pe căi de lungime 2^i. Deși este posibil ca toate cele n procese să inițieze căi de lungime 1, cel mult [n/2] (adică *superior* de n/2) procese vor inița căi de lungime 2, cel mult [n/3] de lungime 4, cel mult [n/5] de lungime 8 etc.

Un proces care inițiază mesaje pe căi de lungime 2^i provoacă trimiterea de mesaje în ambele sensuri, dus-întors. Ca urmare, vor fi transmise cel mult $4*2^i$ mesaje. Numărul total de mesaje este cel mult

$$4*(1*n + 2*[n / 2] + 4*[n / 3] + 8*[n / 5] + \ldots + 2^i*[n/(2^{i-1} + 1)] + \ldots).$$

Fiecare din termenii din paranteze este mai mic decat 2n și există nu mai mulți de 1 + [log n] termeni. Motivul este ca nici un proces nu va pasa mesaje pe căi de lungime 2n sau mai mari deoarece, odată ce un proces a inițiat căi de lungime cel puțin n și mesajul este acceptabil pe toată lungime inelului, procesul devine lider și nu mai

inițiază mesaje. Ca urmare, numărul total de mesaje transmise este mai mic decât 8n + 8[n log n] = **O(n log n)**.

Pentru **complexitatea în timp**, considerăm că transmiterea mesajelor se suprapune în timp (atât cat permite algoritmul) și că fiecare mesaj este transmis în cel mult o unitate de timp. Ca urmare, complexitatea este lineară cu numărul de procese. Mai precis:

$$Timp = 2*(1 + 2 + 4 + \ldots + 2^i + \ldots + n).$$

Când n este o putere alui 2 (cazul cel mai favorabil):

$$\mathbf{Timp = 4n - 2}.$$

Cand n este cu 1 mai mare decât o putere a lui 2 (cazul cel mai defavorabil):

$$\mathbf{Timp = 6n - 6}.$$

Bibliografie

Ernest Chang and Rosemary Roberts *An Improved Algorithm for Decentralized Extrema-Finding in Circular Configurations of Processes*, Communications of **the** ACM Number 5 Volume 22 May 1979

D.S. Hirschberg and J.B. Sinclair *Decentralized Extrema-Finding in Circular Configurations of Processors*, Communications of **the** ACM Number 11 Volume 23 November 1980

Gary L. Peterson *An O(n log n) Unidirectional Algorithm for the Circular Extrema Problem*, ACM Transactions on Programming Languages and Systems, Vol. 4, No. 4, October 1982, Pages 758-762.